

SPCA103N

POSTGRADUATE COURSE

Master of Computer Applications

FIRST YEAR

FIRST SEMESTER

CORE PAPER - III

DATABASE MANAGEMENT SYSTEMS



**INSTITUTE OF DISTANCE EDUCATION
UNIVERSITY OF MADRAS**

WELCOME

Warm Greetings.

It is with a great pleasure to welcome you as a student of Institute of Distance Education, University of Madras. It is a proud moment for the Institute of Distance education as you are entering into a cafeteria system of learning process as envisaged by the University Grants Commission. Yes, we have framed and introduced as per AICTE Choice Based Credit System(CBCS) in Semester pattern from the academic year 2020-21. You are free to choose courses, as per the Regulations, to attain the target of total number of credits set for each course and also each degree programme. What is a credit? To earn one credit in a semester you have to spend 30 hours of learning process. Each course has a weightage in terms of credits. Credits are assigned by taking into account of its level of subject content. For instance, if one particular course or paper has 4 credits then you have to spend 120 hours of self-learning in a semester. You are advised to plan the strategy to devote hours of self-study in the learning process. You will be assessed periodically by means of tests, assignments and quizzes either in class room or laboratory or field work. In the case of PG (UG), Continuous Internal Assessment for 20(25) percentage and End Semester University Examination for 80 (75) percentage of the maximum score for a course / paper. The theory paper in the end semester examination will bring out your various skills: namely basic knowledge about subject, memory recall, application, analysis, comprehension and descriptive writing. We will always have in mind while training you in conducting experiments, analyzing the performance during laboratory work, and observing the outcomes to bring out the truth from the experiment, and we measure these skills in the end semester examination. You will be guided by well experienced faculty.

I invite you to join the CBCS in Semester System to gain rich knowledge leisurely at your will and wish. Choose the right courses at right times so as to erect your flag of success. We always encourage and enlighten to excel and empower. We are the cross bearers to make you a torch bearer to have a bright future.

With best wishes from mind and heart,

DIRECTOR

MCA

FIRST YEAR - FIRST SEMESTER

CORE PAPER - III

DATABASE MANAGEMENT SYSTEMS

COURSE WRITER

J. ADAMKANI, M.Sc., M.Phil., M.Tech.,

Assistant Professor in Computer Science

The New College

Chennai - 600 014.

(Lessons 1 to 7)

S. JAWAHAR, M.C.A., M.Phil.,

Assistant Professor in Computer Science

Asan Memorial College of Arts and Science,

Jaladampet, Chennai - 600 100.

(Lessons 8-14)

COORDINATION & EDITING

Dr. S. Sasikala

Assistant Professor in Computer Science

Institute of Distance Education

University of Madras

Chepauk Chennai - 600 005.

MCA
FIRST YEAR
FIRST SEMESTER
Core Paper - III
DATABASE MANAGEMENT SYSTEMS
SYLLABUS

Objective of the course: This course introduces the concepts of database systems design and to understand the context, phases and techniques for designing and building database information systems in business. To understand the components of a computerized database information system.

Course Outcomes: After successful completion of this course, the students should be able to Design a correct, new database information system for a business functional area and implement the design in either SQL or NoSQL To understand the concepts of open source databases.

Unit-I: Introduction to Database Systems – Relational Model – Structure – Relational Algebra – Null Values – SQL – Set Operation – Views – Advanced SQL – Embedded SQL – Recursive Queries – The Tuple Relational Calculus – Domain Relational Calculus.

Unit-II: E-R Model – Constraints – E-R- Diagrams Weak Entity Sets – Reduction to Relational Schemes – Relational Database Design – Features of Relational Design – Automatic Domains and First Normal Form – Decomposition using Functional Dependencies – Multivalued Dependencies – More Normal Forms – Web Interface – Object – Based Databases – Structured Types and inheritance in SQL – Table inheritance – Persistent.

Unit-III: Storage and File Structure – RAID – File Organisation – Indexing and Hashing – B Tree – B Tree Index files - Static and Dynamic Hashing – Query Processing – Sorting & Join Operators – Query Optimization – Choice of Evaluation Plans.

Unit-IV: Transaction Management – Implementation of Atomicity and Durability – Serializability – Recoverability – Concurrency Control – Dead Lock Handling – Recovery System – Buffer Management.

Unit-V: Database – System Architecture – Client Server – Architectures – Parallel System – Network Types – Distributed Database – Homogeneous and Heterogeneous Database – Directory System – Case Study – Oracle – MSSQL Server.

Recommended Texts

- 1) A. Silberschatz, H.F. Korth and S. Sudharshan, 2006, Database System Concepts, 5th Edition, Tata McGraw Hill, New Delhi.

Reference Books

- 1) J. D. Ullman, 1988, Principles of Database Systems, Galgotia Publishers, New Delhi
- 2) C.J. Date, 1985, An Introduction to Database Systems, Third Edition, Narosa, New Delhi.
- 3) Elmasri and Navathe, 1999, Fundamentals of Database Systems, Third Edition, Pearson Education, Delhi.
- 4) C. Ritchie, 2004, Relational Database Principals, 2nd Edition, Thomson, Singapore.

Website, E-learning resources

- 1) <http://www.cse.iitb.ac.in/dbms/Data/Papers-Local/DBConceptsBook/slide-dir/>

MCA
FIRST YEAR
FIRST SEMESTER
Core Paper - III
DATABASE MANAGEMENT SYSTEMS
SCHEME OF LESSONS

Sl.No.	Title	Page
1.	Introduction to Database Systems	1
2.	Relational Databases	33
3.	Structure Query Language	66
4.	Advanced SQL and Relational Calculas	88
5.	The Entity Relationship Model	121
6.	Relational Database Design	150
7.	Object Based Databases	175
8.	Storage and File Struture	195
9.	Indexing and Hashing	233
10.	Query Processing	273
11.	Transction Management	317
12.	Concurrency Control	338
13.	Database System Architecture	379
14.	Distributed Databases	405

Lesson - 1

INTRODUCTION TO DATABASE SYSTEMS

CONTENTS

- 1.0 Aims and Objectives**
- 1.1 Introduction**
- 1.2 Purpose of Database Systems**
- 1.3 View of Data**
- 1.4 Database Languages**
- 1.5 Relational Databases**
- 1.6 Database Design**
- 1.7 Data Storage and Querying**
- 1.8 Transaction Management**
- 1.9 Database Architecture**
- 1.10 Database Users and Administrators**
- 1.11 Summary**
- 1.12 Questions for Discussion**
- 1.13 Suggested Reading**

1.0 Aims and Objectives

After studying this lesson you should be able to:

- Define Database system
- Explain the purpose of database system
- Define View of Data
- Describe Database design and Query

1.1 Introduction

A **database-management system (DBMS)** is a collection of interrelated data and a set of programs to access those data. The collection of data, usually referred to as the **database**, contains information relevant to an enterprise. The primary goal of a DBMS is to provide a way to store and retrieve database information that is both *convenient* and *efficient*.

Database systems are designed to manage large bodies of information. Management of data involves both defining structures for storage of information and providing mechanisms for the manipulation of information. In addition, the database system must ensure the safety of the information stored, despite system crashes or attempts at unauthorized access. If data are to be shared among several users, the system must avoid possible anomalous results.

Database-System Applications

Databases are widely used. Here are some representative applications:

- *Enterprise Information*
 - * *Sales*: For customer, product, and purchase information.
 - * *Accounting*: For payments, receipts, account balances, assets and other accounting information.
 - * *Human resources*: For information about employees, salaries, payroll taxes, and benefits, and for generation of paychecks.
 - * *Manufacturing*: For management of the supply chain and for tracking production of items in factories, inventories of items in warehouses and stores, and orders for items.
 - * *Online retailers*: For sales data noted above plus online order tracking, generation of recommendation lists, and maintenance of online product evaluations.
- *Banking and Finance*
 - * *Banking*: For customer information, accounts, loans, and banking transactions.
 - * *Credit card transactions*: For purchases on credit cards and generation of monthly statements.

- * *Finance*: For storing information about holdings, sales, and purchases of financial instruments such as stocks and bonds; also for storing real-time market data to enable online trading by customers and automated trading by the firm.
- *Universities*: For student information, course registrations, and grades (in addition to standard enterprise information such as human resources and accounting).
- *Airlines*: For reservations and schedule information. Airlines were among the first to use databases in a geographically distributed manner.
- *Telecommunication*: For keeping records of calls made, generating monthly bills, maintaining balances on prepaid calling cards, and storing information about the communication networks.

1.2 Purpose of Database Systems

Database systems arose in response to early methods of computerized management of commercial data. As an example of such methods, typical of the 1960s, consider part of a university organization that, among other data, keeps information about all instructors, students, departments, and course offerings. One way to keep the information on a computer is to store it in operating system files. To allow users to manipulate the information, the system has a number of application programs that manipulate the files, including programs to:

- Add new students, instructors, and courses
- Register students for courses and generate class rosters
- Assign grades to students, compute grade point averages (GPA), and generate transcripts

System programmers wrote these application programs to meet the needs of the university.

New application programs are added to the system as the need arises. For example, suppose that a university decides to create a new major (say, computer science). As a result, the university creates a new department and creates new permanent files (or adds information to existing files) to record information about all the instructors in the department, students in that major, course offerings, degree requirements, etc. The university may have to write new application programs to deal with rules specific to the new major. New application

programs may also have to be written to handle new rules in the university. Thus, as time goes by, the system acquires more files and more application programs.

This typical **file-processing system** is supported by a conventional operating system. The system stores permanent records in various files, and it needs different application programs to extract records from, and add records to, the appropriate files. Before database management systems (DBMSs) were introduced, organizations usually stored information in such systems.

Keeping organizational information in a file-processing system has a number of major disadvantages:

- **Data redundancy and inconsistency.** Since different programmers create the files and application programs over a long period, the various files are likely to have different structures and the programs may be written in several programming languages. Moreover, the same information may be duplicated in several places (files). For example, if a student has a double major (say, music and mathematics) the address and telephone number of that student may appear in a file that consists of student records of students in the Music department and in a file that consists of student records of students in the Mathematics department. This redundancy leads to higher storage and access cost. In addition, it may lead to **data inconsistency**; that is, the various copies of the same data may no longer agree. For example, a changed student address may be reflected in the Music department records but not elsewhere in the system.
- **Difficulty in accessing data.** Suppose that one of the university clerks needs to find out the names of all students who live within a particular postal-code area. The clerk asks the data-processing department to generate such a list. Because the designers of the original system did not anticipate this request, there is no application program on hand to meet it. There is, however, an application program to generate the list of *all* students. The university clerk has now two choices: either obtain the list of all students and extract the needed information manually or ask a programmer to write the necessary application program. Both alternatives are obviously unsatisfactory. Suppose that such a program is written, and that, several days later, the same clerk needs to trim that list to include only those students who have taken at least 60 credit hours. As expected, a program to generate such a list does not exist. Again, the clerk has the preceding two options, neither of which is satisfactory.

The point here is that conventional file-processing environments do not allow needed data to be retrieved in a convenient and efficient manner. More responsive data-retrieval systems are required for general use.

- **Data isolation.** Because data are scattered in various files, and files may be in different formats, writing new application programs to retrieve the appropriate data is difficult.
- **Integrity problems.** The data values stored in the database must satisfy certain types of **consistency constraints**. Suppose the university maintains an account for each department, and records the balance amount in each account. Suppose also that the university requires that the account balance of a department may never fall below zero. Developers enforce these constraints in the system by adding appropriate code in the various application programs. However, when new constraints are added, it is difficult to change the programs to enforce them. The problem is compounded when constraints involve several data items from different files.
- **Atomicity problems.** A computer system, like any other device, is subject to failure. In many applications, it is crucial that, if a failure occurs, the data be restored to the consistent state that existed prior to the failure. Consider a program to transfer \$500 from the account balance of department *A* to the account balance of department *B*. If a system failure occurs during the execution of the program, it is possible that the \$500 was removed from the balance of department *A* but was not credited to the balance of department *B*, resulting in an inconsistent database state. Clearly, it is essential to database consistency that either both the credit and debit occur, or that neither occur. That is, the funds transfer must be *atomic*—it must happen in its entirety or not at all. It is difficult to ensure atomicity in a conventional file-processing system.
- **Concurrent-access anomalies.** For the sake of overall performance of the system and faster response, many systems allow multiple users to update the data simultaneously. Indeed, today, the largest Internet retailers may have millions of accesses per day to their data by shoppers. In such an environment, interaction of concurrent updates is possible and may result in inconsistent data. Consider department *A*, with an account balance of \$10,000. If two department clerks debit the account balance (by say \$500 and \$100, respectively) of department *A* at almost exactly the same time, the result of the

concurrent executions may leave the budget in an incorrect (or inconsistent) state. Suppose that the programs executing on behalf of each withdrawal read the old balance, reduce that value by the amount being withdrawn, and write the result back. If the two programs run concurrently, they may both read the value \$10,000, and write back \$9500 and \$9900, respectively. Depending on which one writes the value last, the account balance of department A may contain either \$9500 or \$9900, rather than the correct value of \$9400. To guard against this possibility, the system must maintain some form of supervision. But supervision is difficult to provide because data may be accessed by many different application programs that have not been coordinated previously.

As another example, suppose a registration program maintains a count of students registered for a course, in order to enforce limits on the number of students registered. When a student registers, the program reads the current count for the courses, verifies that the count is not already at the limit, adds one to the count, and stores the count back in the database. Suppose two students register concurrently, with the count at (say) 39. The two program executions may both read the value 39, and both would then write back 40, leading to an incorrect increase of only 1, even though two students successfully registered for the course and the count should be 41. Furthermore, suppose the course registration limit was 40; in the above case both students would be able to register, leading to a violation of the limit of 40 students.

- **Security problems.** Not every user of the database system should be able to access all the data. For example, in a university, payroll personnel need to see only that part of the database that has financial information. They do not need access to information about academic records. But, since application programs are added to the file-processing system in an ad hoc manner, enforcing such security constraints is difficult.

1.3 View of Data

A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data. A major purpose of a database system is to provide users with an *abstract* view of the data. That is, the system hides certain details of how the data are stored and maintained.

1.3.1 Data Abstraction

For the system to be usable, it must retrieve data efficiently. The need for efficiency has led designers to use complex data structures to represent data in the database. Since

many database-system users are not computer trained, developers hide the complexity from users through several levels of abstraction, to simplify users' interactions with the system:

- **Physical level.** The lowest level of abstraction describes *how* the data are actually stored. The physical level describes complex low-level data structures in detail.
- **Logical level.** The next-higher level of abstraction describes *what* data are stored in the database, and what relationships exist among those data. The logical level thus describes the entire database in terms of a small number of relatively simple structures. Although implementation of the simple structures at the logical level may involve complex physical-level structures, the user of the logical level does not need to be aware of this complexity. This is referred to as **physical data independence**. Database administrators, who must decide what information to keep in the database, use the logical level of abstraction.
- **View level.** The highest level of abstraction describes only part of the entire database. Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database. Many users of the database system do not need all this information; instead, they need to access only a part of the database. The view level of abstraction exists to simplify their interaction with the system. The system may provide many views for the same database.

Figure 1.1 shows the relationship among the three levels of abstraction.

An analogy to the concept of data types in programming languages may clarify the distinction among levels of abstraction. Many high-level programming

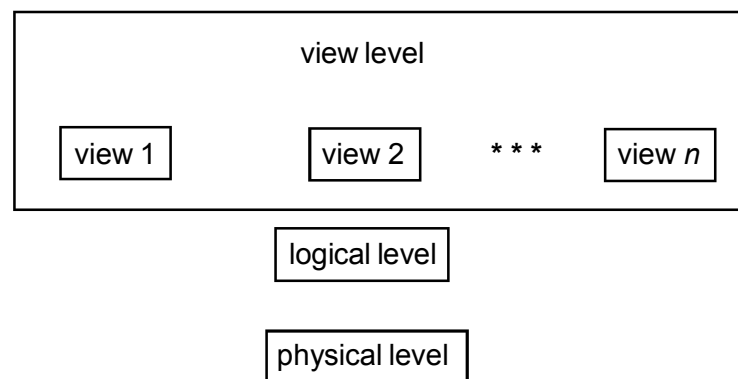


Figure 1.1 The three levels of data abstraction.

languages support the notion of a structured type. For example, we may describe a record as follows:¹

```
type instructor = record
    ID : char (5);
    name: char (20);
    deptname: char (20);
    salary: numeric (8,2);
end;
```

This code defines a new record type called *instructor* with four fields. Each field has a name and a type associated with it. A university organization may have several such record types, including *department*, with fields *deptname*, *building*, and *budget*

- *course*, with fields *courseid*, *title*, *deptname*, and *credits*
- *student*, with fields *ID*, *name*, *deptname*, and *totcred*

At the physical level, an *instructor*, *department*, or *student* record can be de-scribed as a block of consecutive storage locations. The compiler hides this level of detail from programmers. Similarly, the database system hides many of the lowest-level storage details from database programmers. Database administra-tors, on the other hand, may be aware of certain details of the physical organiza-tion of the data.

At the logical level, each such record is described by a type definition, as in the previous code segment, and the interrelationship of these record types is defined as well. Programmers using a programming language work at this level of abstraction. Similarly, database administrators usually work at this level of abstraction.

Finally, at the view level, computer users see a set of application programs that hide details of the data types. At the view level, several views of the database are defined, and a database user sees some or all of these views. In addition to hiding details of the logical level of the database, the views also provide a security mechanism to prevent users from accessing certain parts of the database. For example, clerks in the university registrar office can see only that part of the database that has information about students; they cannot access information about salaries of instructors.

1.3.2 Instances and Schemas

Databases change over time as information is inserted and deleted. The collection of information stored in the database at a particular moment is called an **instance** of the

database. The overall design of the database is called the database **schema**. Schemas are changed infrequently, if at all.

The concept of database schemas and instances can be understood by analogy to a program written in a programming language. A database schema corresponds to the variable declarations (along with associated type definitions) in a program. Each variable has a particular value at a given instant. The values of the variables in a program at a point in time correspond to an *instance* of a database schema.

Database systems have several schemas, partitioned according to the levels of abstraction. The **physical schema** describes the database design at the physical level, while the **logical schema** describes the database design at the logical level. A database may also have several schemas at the view level, sometimes called **subschemas**, that describe different views of the database.

Of these, the logical schema is by far the most important, in terms of its effect on application programs, since programmers construct applications by using the logical schema. The physical schema is hidden beneath the logical schema, and can usually be changed easily without affecting application programs. Application programs are said to exhibit **physical data independence** if they do not depend on the physical schema, and thus need not be rewritten if the physical schema changes.

We study languages for describing schemas after introducing the notion of data models in the next section.

1.3.3 Data Models

Underlying the structure of a database is the **data model**: a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints. A data model provides a way to describe the design of a database at the physical, logical, and view levels.

There are a number of different data models that we shall cover in the text. The data models can be classified into four different categories:

- **Relational Model.** The relational model uses a collection of tables to represent both data and the relationships among those data. Each table has multiple columns, and each column has a unique name. Tables are also known as **relations**. The relational model is an example of a record-based model. Record-based models are so named because the database is structured in fixed-format records of several types. Each table contains records of a particular type. Each

record type defines a fixed number of fields, or attributes. The columns of the table correspond to the attributes of the record type. The relational data model is the most widely used data model, and a vast majority of current database systems are based on the relational model. Chapters 2 through 8 cover the relational model in detail.

- **Entity-Relationship Model.** The entity-relationship (E-R) data model uses a collection of basic objects, called *entities*, and *relationships* among these objects. An entity is a “thing” or “object” in the real world that is distinguishable from other objects. The entity-relationship model is widely used in database design, and Chapter 7 explores it in detail.
- **Object-Based Data Model.** Object-oriented programming (especially in Java, C++, or C#) has become the dominant software-development methodology. This led to the development of an object-oriented data model that can be seen as extending the E-R model with notions of encapsulation, methods (functions), and object identity. The object-relational data model combines features of the object-oriented data model and relational data model. Chapter 22 examines the object-relational data model.
- **Semistructured Data Model.** The semistructured data model permits the specification of data where individual data items of the same type may have different sets of attributes. This is in contrast to the data models mentioned earlier, where every data item of a particular type must have the same set of attributes. The **Extensible Markup Language (XML)** is widely used to represent semistructured data. Chapter 23 covers it.

Historically, the **network data model** and the **hierarchical data model** preceded the relational data model. These models were tied closely to the underlying implementation, and complicated the task of modeling data. As a result they are used little now, except in old database code that is still in service in some places. They are outlined online in Appendices D and E for interested readers.

1.4 Database Languages

A database system provides a **data-definition language** to specify the database schema and a **data-manipulation language** to express database queries and updates. In practice, the data-definition and data-manipulation languages are not two separate languages; instead they simply form parts of a single database language, such as the widely used SQL language.

1.4.1 Data-Manipulation Language

A **data-manipulation language (DML)** is a language that enables users to access or manipulate data as organized by the appropriate data model. The types of access are:

- Retrieval of information stored in the database
- Insertion of new information into the database
- Deletion of information from the database
- Modification of information stored in the database

There are basically two types:

- **Procedural DMLs** require a user to specify *what* data are needed and *how* to get those data.
- **Declarative DMLs** (also referred to as **nonprocedural DMLs**) require a user to specify *what* data are needed *without* specifying how to get those data.

Declarative DMLs are usually easier to learn and use than are procedural DMLs. However, since a user does not have to specify how to get the data, the database system has to figure out an efficient means of accessing data.

A **query** is a statement requesting the retrieval of information. The portion of a DML that involves information retrieval is called a **query language**. Although technically incorrect, it is common practice to use the terms *query language* and *data-manipulation language* synonymously.

There are a number of database query languages in use, either commercially or experimentally. We study the most widely used query language, SQL, in Chapters 3, 4, and 5. We also study some other query languages in Chapter 6.

The levels of abstraction that we discussed in Section 1.3 apply not only to defining or structuring data, but also to manipulating data. At the physical level, we must define algorithms that allow efficient access to data. At higher levels of abstraction, we emphasize ease of use. The goal is to allow humans to interact efficiently with the system. The query processor component of the database system (which we study in Chapters 12 and 13) translates DML queries into sequences of actions at the physical level of the database system.

1.4.2 Data-Definition Language

We specify a database schema by a set of definitions expressed by a special language called a **data-definition language (DDL)**. The DDL is also used to specify additional properties of the data.

We specify the storage structure and access methods used by the database system by a set of statements in a special type of DDL called a **data storage and definition** language. These statements define the implementation details of the database schemas, which are usually hidden from the users.

The data values stored in the database must satisfy certain **consistency con-straints**. For example, suppose the university requires that the account balance of a department must never be negative. The DDL provides facilities to specify such constraints. The database system checks these constraints every time the database is updated. In general, a constraint can be an arbitrary predicate per-taining to the database. However, arbitrary predicates may be costly to test. Thus, database systems implement integrity constraints that can be tested with minimal overhead:

- **Domain Constraints.** A domain of possible values must be associated with every attribute (for example, integer types, character types, date/time types). Declaring an attribute to be of a particular domain acts as a constraint on the values that it can take. Domain constraints are the most elementary form of integrity constraint. They are tested easily by the system whenever a new data item is entered into the database.
- **Referential Integrity.** There are cases where we wish to ensure that a value that appears in one relation for a given set of attributes also appears in a cer-tain set of attributes in another relation (referential integrity). For example, the department listed for each course must be one that actually exists. More precisely, the *deptname* value in a *course* record must appear in the *deptname* attribute of some record of the *department* relation. Database modifications can cause violations of referential integrity. When a referential-integrity con-straint is violated, the normal procedure is to reject the action that caused the violation.
- **Assertions.** An assertion is any condition that the database must always satisfy. Domain constraints and referential-integrity constraints are special forms of assertions. However, there are many constraints that we cannot express by using only these special forms. For example, “Every department must have at

least five courses offered every semester” must be expressed as an assertion. When an assertion is created, the system tests it for validity. If the assertion is valid, then any future modification to the database is allowed only if it does not cause that assertion to be violated.

- **Authorization.** We may want to differentiate among the users as far as the type of access they are permitted on various data values in the database. These differentiations are expressed in terms of **authorization**, the most common being: **read authorization**, which allows reading, but not modification, of data; **insert authorization**, which allows insertion of new data, but not modification of existing data; **update authorization**, which allows modification, but not deletion, of data; and **delete authorization**, which allows deletion of data. We may assign the user all, none, or a combination of these types of authorization.

The DDL, just like any other programming language, gets as input some instructions (statements) and generates some output. The output of the DDL is placed in the **data dictionary**, which contains **metadata**—that is, data about data. The data dictionary is considered to be a special type of table that can only be accessed and updated by the database system itself (not a regular user). The database system consults the data dictionary before reading or modifying actual data.

1.5 Relational Databases

A relational database is based on the relational model and uses a collection of tables to represent both data and the relationships among those data. It also includes a DML and DDL. In Chapter 2 we present a gentle introduction to the fundamentals of the relational model. Most commercial relational database systems employ the SQL language, which we cover in great detail in Chapters 3, 4, and 5. In Chapter 6 we discuss other influential languages.

1.5.1 Tables

Each table has multiple columns and each column has a unique name. Figure 1.2 presents a sample relational database comprising two tables: one shows details of university instructors and the other shows details of the various university departments.

The first table, the *instructor* table, shows, for example, that an instructor named Einstein with *ID* 22222 is a member of the Physics department and has an annual salary of \$95,000. The second table, *department*, shows, for example, that the Biology department is located in the Watson building and has a budget of \$90,000. Of course, a real-world

university would have many more departments and instructors. We use small tables in the text to illustrate concepts. A larger example for the same schema is available online.

The relational model is an example of a record-based model. Record-based models are so named because the database is structured in fixed-format records of several types. Each table contains records of a particular type. Each record type defines a fixed number of fields, or attributes. The columns of the table correspond to the attributes of the record type.

It is not hard to see how tables may be stored in files. For instance, a special character (such as a comma) may be used to delimit the different attributes of a record, and another special character (such as a new-line character) may be used to delimit records. The relational model hides such low-level implementation details from database developers and users.

We also note that it is possible to create schemas in the relational model that have problems such as unnecessarily duplicated information. For example, suppose we store the department *budget* as an attribute of the *instructor* record. Then, whenever the value of a particular budget (say that one for the Physics department) changes, that change must to be reflected in the records of all instructors

<i>ID</i>	<i>name</i>	<i>deptname</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

<i>deptname</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table**Figure 1.2 A sample relational database.**

associated with the Physics department. In Chapter 5, we shall study how to distinguish good schema designs from bad schema designs.

1.5.2 Data-Manipulation Language

The SQL query language is nonprocedural. A query takes as input several tables (possibly only one) and always returns a single table. Here is an example of an SQL query that finds the names of all instructors in the History department:

```
select instructor.name
from instructor
where instructor.deptname = 'History';
```

The query specifies that those rows from the table *instructor* where the *deptname* is History must be retrieved, and the *name* attribute of these rows must be displayed. More specifically the result of executing this query is a table with a single column labeled *name*, and a set of rows, each of which contains the name of an instructor whose *deptname*, is History. If the query is run on the table in Figure 1.2, the result will consist of two rows, one with the name El Said and the other with the name Califieri.

Queries may involve information from more than one table. For instance, the following query finds the instructor ID and department name of all instructors associated with a department with budget of greater than \$95,000.

```

select instructor.ID, department.deptname

from instructor, department

where instructor.deptname= department.deptname and

       department.budget > 95000;

```

If the above query were run on the tables in Figure 1.2, the system would find that there are two departments with budget of greater than \$95,000—Computer Science and Finance; there are five instructors in these departments. Thus, the result will consist of a table with two columns (*ID, deptname*) and five rows: (12121, Finance), (45565, Computer Science), (10101, Computer Science), (83821, Computer Science), and (76543, Finance).

1.5.3 Data-Definition Language

SQL provides a rich DDL that allows one to define tables, integrity constraints, assertions, etc.

For instance, the following SQL DDL statement defines the *department* table:

```

create table department

(deptname    char (20),

building     char (15),

budget numeric (12,2));

```

Execution of the above DDL statement creates the *department* table with three columns: *deptname*, *building*, and *budget*, each of which has a specific data type associated with it. We discuss data types in more detail in Chapter 3. In addition, the DDL statement updates the data dictionary, which contains metadata (see Section 1.4.2). The schema of a table is an example of metadata.

1.5.4 Database Access from Application Programs

SQL is not as powerful as a universal Turing machine; that is, there are some computations that are possible using a general-purpose programming language but are not possible using SQL. SQL also does not support actions such as input from users, output to displays, or communication over the network. Such computations and actions must be written in a *host* language, such as C, C++, or Java, with embedded SQL queries that access the data in the database. **Application programs** are programs that are used to interact with the database in this fashion.

Examples in a university system are programs that allow students to register for courses, generate class rosters, calculate student GPA, generate payroll checks, etc. To access the database, DML statements need to be executed from the host language. There are two ways to do this:

- By providing an application program interface (set of procedures) that can be used to send DML and DDL statements to the database and retrieve the results.

The Open Database Connectivity (ODBC) standard for use with the C language is a commonly used application program interface standard. The Java Database Connectivity (JDBC) standard provides corresponding features to the Java language.

- By extending the host language syntax to embed DML calls within the host language program. Usually, a special character prefaces DML calls, and a preprocessor, called the **DML precompiler**, converts the DML statements to normal procedure calls in the host language.

1.6 Database Design

Database systems are designed to manage large bodies of information. These large bodies of information do not exist in isolation. They are part of the operation of some enterprise whose end product may be information from the database or may be some device or service for which the database plays only a supporting role.

Database design mainly involves the design of the database schema. The design of a complete database application environment that meets the needs of the enterprise being modeled requires attention to a broader set of issues. In this text, we focus initially on the writing of database queries and the design of database schemas. Chapter 9 discusses the overall process of application design.

1.6.1 Design Process

A high-level data model provides the database designer with a conceptual framework in which to specify the data requirements of the database users, and how the database will be structured to fulfill these requirements. The initial phase of database design, then, is to characterize fully the data needs of the prospective database users. The database designer needs to interact extensively with domain experts and users to carry out this task. The outcome of this phase is a specification of user requirements.

Next, the designer chooses a data model, and by applying the concepts of the chosen data model, translates these requirements into a conceptual schema of the database. The

schema developed at this **conceptual-design** phase provides a detailed overview of the enterprise. The designer reviews the schema to confirm that all data requirements are indeed satisfied and are not in conflict with one another. The designer can also examine the design to remove any redundant features. The focus at this point is on describing the data and their relationships, rather than on specifying physical storage details.

In terms of the relational model, the conceptual-design process involves decisions on *what* attributes we want to capture in the database and *how to group* these attributes to form the various tables. The “what” part is basically a business decision, and we shall not discuss it further in this text. The “how” part is mainly a computer-science problem. There are principally two ways to tackle the problem. The first one is to use the entity-relationship model (Section 1.6.3); the other is to employ a set of algorithms (collectively known as *normalization*) that takes as input the set of all attributes and generates a set of tables (Section 1.6.4).

A fully developed conceptual schema indicates the functional requirements of the enterprise. In a **specification of functional requirements**, users describe the kinds of operations (or transactions) that will be performed on the data. Example operations include modifying or updating data, searching for and retrieving specific data, and deleting data. At this stage of conceptual design, the designer can review the schema to ensure it meets functional requirements.

The process of moving from an abstract data model to the implementation of the database proceeds in two final design phases. In the **logical-design phase**, the designer maps the high-level conceptual schema onto the implementation data model of the database system that will be used. The designer uses the resulting system-specific database schema in the subsequent **physical-design phase**, in which the physical features of the database are specified. These features include the form of file organization and the internal storage structures; they are discussed in Chapter 10.

1.6.2 Database Design for a University Organization

To illustrate the design process, let us examine how a database for a university could be designed. The initial specification of user requirements may be based on interviews with the database users, and on the designer’s own analysis of the organization. The description that arises from this design phase serves as the basis for specifying the conceptual structure of the database. Here are the major characteristics of the university.

- The university is organized into departments. Each department is identified by a unique name (*deptname*), is located in a particular *building*, and has a *budget*.
- Each department has a list of courses it offers. Each course has associated with it a *courseid*, *title*, *deptname*, and *credits*, and may also have associated *prerequisites*.
- Instructors are identified by their unique *ID*. Each instructor has *name*, associated department (*deptname*), and *salary*.
- Students are identified by their unique *ID*. Each student has a *name*, an associated major department (*deptname*), and *totcred* (total credit hours the student earned thus far).
- The university maintains a list of classrooms, specifying the name of the *building*, *room number*, and room *capacity*.
- The university maintains a list of all classes (sections) taught. Each section is identified by a *courseid*, *secid*, *year*, and *semester*, and has associated with it a *semester*, *year*, *building*, *roomnumber*, and *timeslotid* (the time slot when the class meets).
- The department has a list of teaching assignments specifying, for each instructor, the sections the instructor is teaching.
- The university has a list of all student course registrations, specifying, for each student, the courses and the associated sections that the student has taken (registered for).

A real university database would be much more complex than the preceding design. However we use this simplified model to help you understand conceptual ideas without getting lost in details of a complex design.

1.6.3 The Entity-Relationship Model

The entity-relationship (E-R) data model uses a collection of basic objects, called *entities*, and *relationships* among these objects. An entity is a “thing” or “object” in the real world that is distinguishable from other objects. For example, each person is an entity, and bank accounts can be considered as entities.

Entities are described in a database by a set of **attributes**. For example, the attributes *deptname*, *building*, and *budget* may describe one particular department in a university, and

they form attributes of the *department* entity set. Similarly, attributes *ID*, *name*, and *salary* may describe an *instructor* entity²

The extra attribute *ID* is used to identify an instructor uniquely (since it may be possible to have two instructors with the same name and the same salary). A unique instructor identifier must be assigned to each instructor. In the United States, many organizations use the social-security number of a person (a unique number the U.S. government assigns to every person in the United States) as a unique identifier.

A **relationship** is an association among several entities. For example, a *member* relationship associates an instructor with her department. The set of all entities of the same type and the set of all relationships of the same type are termed an **entity set** and **relationship set**, respectively.

The overall logical structure (schema) of a database can be expressed graph-ically by an *entity-relationship (E-R) diagram*. There are several ways in which to draw these diagrams. One of the most popular is to use the **Unified Modeling Language (UML)**. In the notation we use, which is based on UML, an E-R diagram is represented as follows:

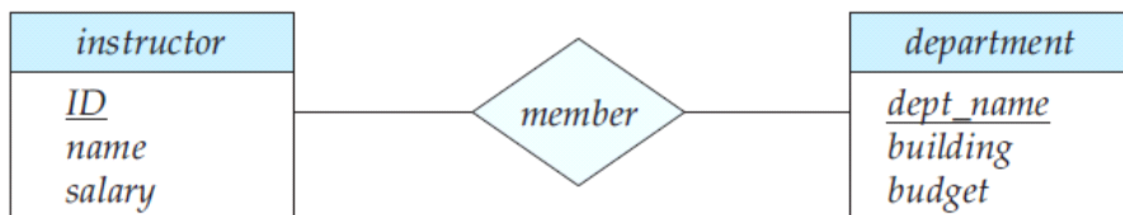


Figure 1.3 A Sample E-R diagram

- Entity sets are represented by a rectangular box with the entity set name in the header and the attributes listed below it.
- Relationship sets are represented by a diamond connecting a pair of related entity sets. The name of the relationship is placed inside the diamond.

As an illustration, consider part of a university database consisting of instructors and the departments with which they are associated. Figure 1.3 shows the corresponding E-R diagram. The E-R diagram indicates that there are two entity sets, *instructor* and *department*, with attributes as outlined earlier. The diagram also shows a relationship *member* between *instructor* and *department*.

In addition to entities and relationships, the E-R model represents certain constraints to which the contents of a database must conform. One important constraint is **mapping**

cardinalities, which express the number of entities to which another entity can be associated via a relationship set. For example, if each instructor must be associated with only a single department, the E-R model can express that constraint.

The entity-relationship model is widely used in database design, and Chapter 7 explores it in detail.

1.6.4 Normalization

Another method for designing a relational database is to use a process commonly known as normalization. The goal is to generate a set of relation schemas that allows us to store information without unnecessary redundancy, yet also allows us to retrieve information easily. The approach is to design schemas that are in an appropriate *normal form*. To determine whether a relation schema is in one of the desirable normal forms, we need additional information about the real-world enterprise that we are modeling with the database. The most common approach is to use **functional dependencies**, which we cover in Section 8.4.

To understand the need for normalization, let us look at what can go wrong in a bad database design. Among the undesirable properties that a bad design may have are:

- Repetition of information
- Inability to represent certain information

<i>ID</i>	<i>name</i>	<i>Salary</i>	<i>deptname</i>	<i>building</i>	<i>budget</i>
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

Figure 1.4 *The faculty table.*

We shall discuss these problems with the help of a modified database design for our university example.

Suppose that instead of having the two separate tables *instructor* and *department*, we have a single table, *faculty*, that combines the information from the two tables (as shown in Figure 1.4). Notice that there are two rows in *faculty* that contain repeated information about the History department, specifically, that department's building and budget. The repetition of information in our alternative design is undesirable. Repeating information wastes space. Furthermore, it complicates updating the database. Suppose that we wish to change the budget amount of the History department from \$50,000 to \$46,800. This change must be reflected in the two rows; contrast this with the original design, where this requires an update to only a single row. Thus, updates are more costly under the alternative design than under the original design. When we perform the update in the alternative database, we must ensure that every tuple pertaining to the History department is updated, or else our database will show two different budget values for the History department.

Now, let us shift our attention to the issue of "inability to represent certain information." Suppose we are creating a new department in the university. In the alternative design above, we cannot represent directly the information concerning a department (*deptname*, *building*, *budget*) unless that department has at least one instructor at the university. This is because rows in the *faculty* table require values for *ID*, *name*, and *salary*. This means that we cannot record information about the newly created department until the first instructor is hired for the new department.

1.7 Data Storage and Querying

A database system is partitioned into modules that deal with each of the responsibilities of the overall system. The functional components of a database system can be broadly divided into the storage manager and the query processor components.

The storage manager is important because databases typically require a large amount of storage space. Corporate databases range in size from hundreds of gigabytes to, for the largest databases, terabytes of data. A gigabyte is approximately 1000 megabytes (actually 1024) (1 billion bytes), and a terabyte is 1 million megabytes (1 trillion bytes). Since the main memory of computers cannot store this much information, the information is stored

on disks. Data are moved between disk storage and main memory as needed. Since the movement of data to and from disk is slow relative to the speed of the central processing unit, it is imperative that the database system structure the data so as to minimize the need to move data between disk and main memory.

The query processor is important because it helps the database system to simplify and facilitate access to data. The query processor allows database users to obtain good performance while being able to work at the view level and not be burdened with understanding the physical-level details of the implementation of the system. It is the job of the database system to translate updates and queries written in a nonprocedural language, at the logical level, into an efficient sequence of operations at the physical level.

1.7.1 Storage Manager

The *storage manager* is the component of a database system that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system. The storage manager is responsible for the interaction with the file manager. The raw data are stored on the disk using the file system provided by the operating system. The storage manager translates the various DML statements into low-level file-system commands.

Thus, the storage manager is responsible for storing, retrieving, and updating data in the database.

The storage manager components include:

- **Authorization and integrity manager**, which tests for the satisfaction of integrity constraints and checks the authority of users to access data.
- **Transaction manager**, which ensures that the database remains in a consistent (correct) state despite system failures, and that concurrent transaction executions proceed without conflicting.
- **File manager**, which manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
- **Buffer manager**, which is responsible for fetching data from disk storage into main memory, and deciding what data to cache in main memory. The buffer

manager is a critical part of the database system, since it enables the database to handle data sizes that are much larger than the size of main memory.

The storage manager implements several data structures as part of the physical system implementation:

- **Data files**, which store the database itself.
- **Data dictionary**, which stores metadata about the structure of the database, in particular the schema of the database.
- **Indices**, which can provide fast access to data items. Like the index in this textbook, a database index provides pointers to those data items that hold a particular value. For example, we could use an index to find the *instructor* record with a particular *ID*, or all *instructor* records with a particular *name*. Hashing is an alternative to indexing that is faster in some but not all cases.

1.7.2 The Query Processor

The query processor components include:

- **DDL interpreter**, which interprets DDL statements and records the definitions in the data dictionary.
- **DML compiler**, which translates DML statements in a query language into an evaluation plan consisting of low-level instructions that the query evaluation engine understands. A query can usually be translated into any of a number of alternative evaluation plans that all give the same result. The DML compiler also performs **query optimization**; that is, it picks the lowest cost evaluation plan from among the alternatives.
- **Query evaluation engine**, which executes low-level instructions generated by the DML compiler.

1.8 Transaction Management

Often, several operations on the database form a single logical unit of work. An example is a funds transfer, as in Section 1.2, in which one department account (say *A*) is debited

and another department account (say *B*) is credited. Clearly, it is essential that either both the credit and debit occur, or that neither occur. That is, the funds transfer must happen in its entirety or not at all. This all-or-none requirement is called **atomicity**. In addition, it is essential that the execution of the funds transfer preserve the consistency of the database. That is, the value of the sum of the balances of *A* and *B* must be preserved. This correctness requirement is called **consistency**. Finally, after the successful execution of a funds transfer, the new values of the balances of accounts *A* and *B* must persist, despite the possibility of system failure. This persistence requirement is called **durability**.

A **transaction** is a collection of operations that performs a single logical function in a database application. Each transaction is a unit of both atomicity and consistency. Thus, we require that transactions do not violate any database-consistency constraints. That is, if the database was consistent when a transaction started, the database must be consistent when the transaction successfully terminates. However, during the execution of a transaction, it may be necessary temporarily to allow inconsistency, since either the debit of *A* or the credit of *B* must be done before the other. This temporary inconsistency, although necessary, may lead to difficulty if a failure occurs.

It is the programmer's responsibility to define properly the various transactions, so that each preserves the consistency of the database. For example, the transaction to transfer funds from the account of department *A* to the account of department *B* could be defined to be composed of two separate programs: one that debits account *A*, and another that credits account *B*. The execution of these two programs one after the other will indeed preserve consistency. However, each program by itself does not transform the database from a consistent state to a new consistent state. Thus, those programs are not transactions.

Ensuring the atomicity and durability properties is the responsibility of the database system itself—specifically, of the **recovery manager**. In the absence of failures, all transactions complete successfully, and atomicity is achieved easily.

However, because of various types of failure, a transaction may not always complete its execution successfully. If we are to ensure the atomicity property, a failed transaction must have no effect on the state of the database. Thus, the database must be restored to the state in which it was before the transaction in question started executing. The database

system must therefore perform **failure recovery**, that is, detect system failures and restore the database to the state that existed prior to the occurrence of the failure.

Finally, when several transactions update the database concurrently, the consistency of data may no longer be preserved, even though each individual transaction is correct. It is the responsibility of the **concurrency-control manager** to control the interaction among the concurrent transactions, to ensure the consistency of the database. The **transaction manager** consists of the concurrency-control manager and the recovery manager.

The basic concepts of transaction processing are covered in Chapter 14. The management of concurrent transactions is covered in Chapter 15. Chapter 16 covers failure recovery in detail.

The concept of a transaction has been applied broadly in database systems and applications. While the initial use of transactions was in financial applications, the concept is now used in real-time applications in telecommunication, as well as in the management of long-duration activities such as product design or administrative workflows. These broader applications of the transaction concept are discussed in Chapter 26.

1.9 Database Architecture

We are now in a position to provide a single picture (Figure 1.5) of the various components of a database system and the connections among them.

The architecture of a database system is greatly influenced by the underlying computer system on which the database system runs. Database systems can be centralized, or client-server, where one server machine executes work on behalf of multiple client machines. Database systems can also be designed to exploit parallel computer architectures. Distributed databases span multiple geographically separated machines.

Most users of a database system today are not present at the site of the database system, but connect to it through a network. We can therefore differentiate between **client** machines, on which remote database users work, and **server** machines, on which the database system runs.

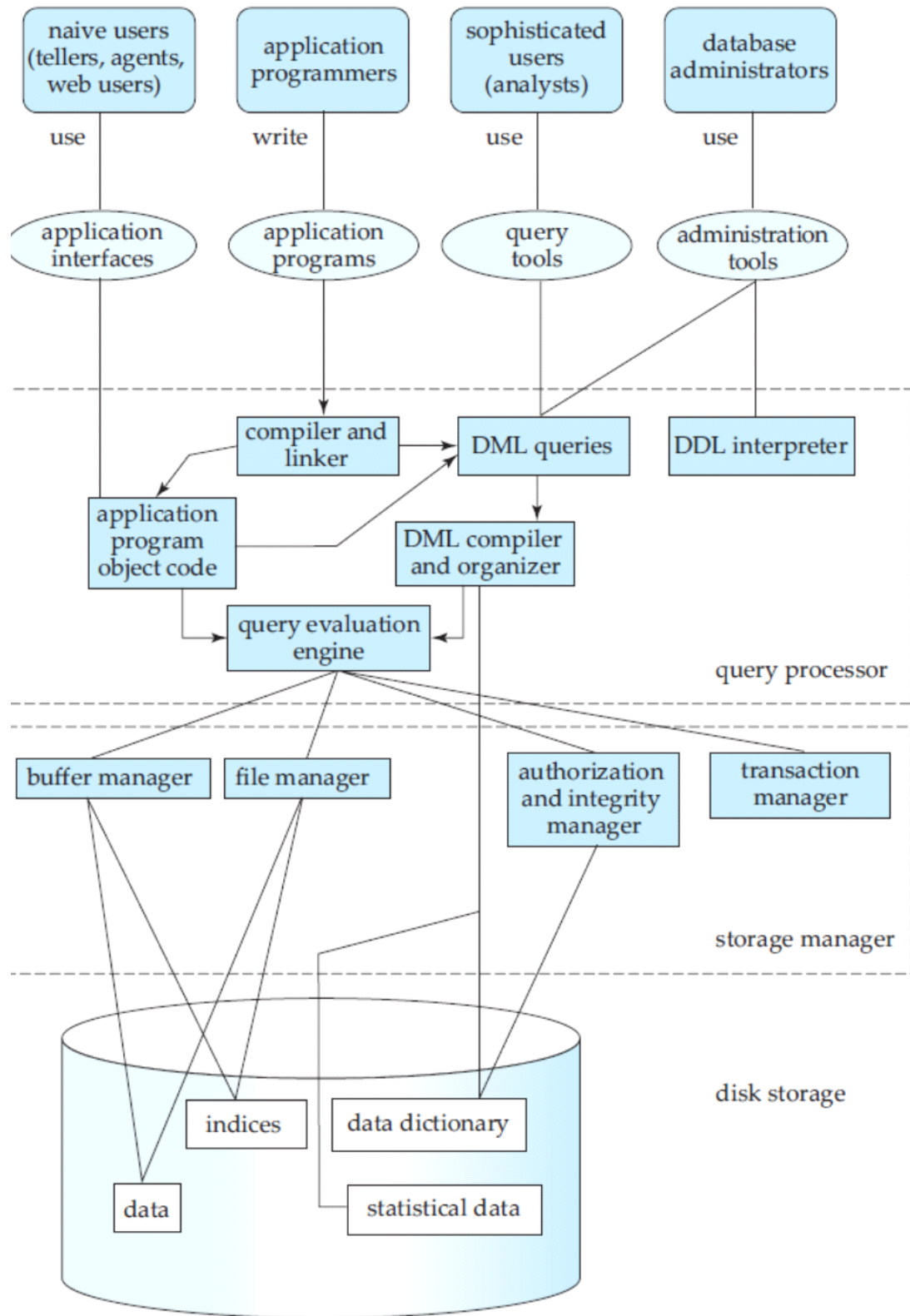


Figure 1.5 System Structure

Database applications are usually partitioned into two or three parts, as in Figure 1.6. In a **two-tier architecture**, the application resides at the client machine, where it invokes database system functionality at the server machine through query language statements. Application program interface standards like ODBC and JDBC are used for interaction between the client and the server.

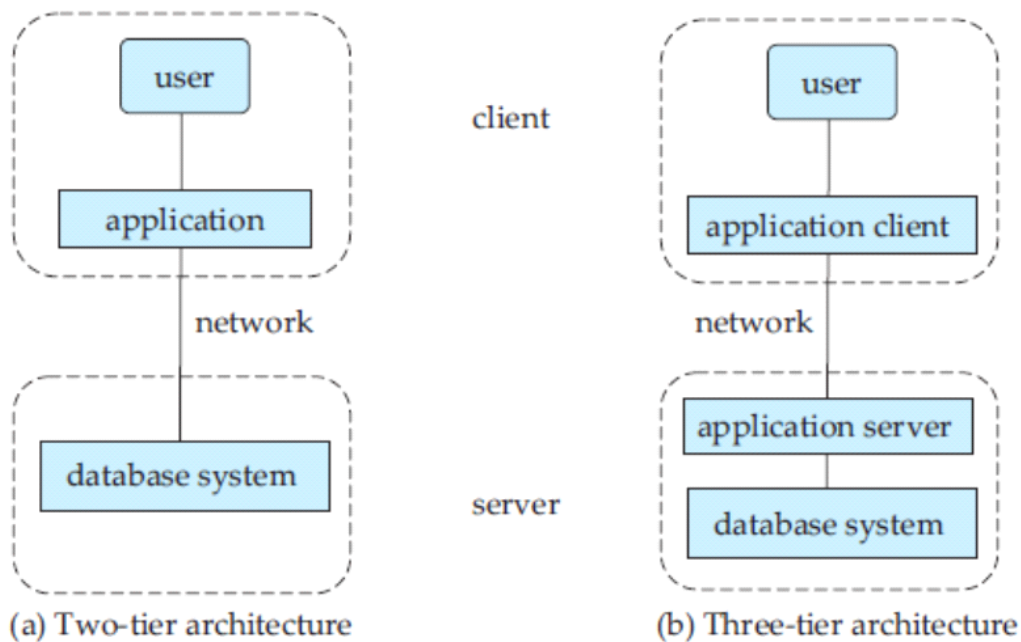


Figure 1.6 Two-tier and three-tier architectures

In contrast, in a **three-tier architecture**, the client machine acts as merely a front end and does not contain any direct database calls. Instead, the client end communicates with an **application server**, usually through a forms interface. The application server in turn communicates with a database system to access data. The **business logic** of the application, which says what actions to carry out under what conditions, is embedded in the application server, instead of being distributed across multiple clients. Three-tier applications are more appropriate for large applications, and for applications that run on the World Wide Web.

1.10 Database Users and Administrators

A primary goal of a database system is to retrieve information from and store new information into the database. People who work with a database can be categorized as database users or database administrators.

1.10.1 Database Users and User Interfaces

There are four different types of database-system users, differentiated by the way they expect to interact with the system. Different types of user interfaces have been designed for the different types of users.

- **Naïve users** are unsophisticated users who interact with the system by invoking one of the application programs that have been written previously. For example, a clerk in the university who needs to add a new instructor to department *A* invokes a program called *newhire*. This program asks the clerk for the name of the new instructor, her new *ID*, the name of the department (that is, *A*), and the salary.

The typical user interface for naïve users is a forms interface, where the user can fill in appropriate fields of the form. Naïve users may also simply read *reports* generated from the database.

As another example, consider a student, who during class registration period, wishes to register for a class by using a Web interface. Such a user connects to a Web application program that runs at a Web server. The application first verifies the identity of the user, and allows her to access a form where she enters the desired information. The form information is sent back to the Web application at the server, which then determines if there is room in the class (by retrieving information from the database) and if so adds the student information to the class roster in the database.

- **Application programmers** are computer professionals who write application programs. Application programmers can choose from many tools to develop user interfaces. **Rapid application development (RAD)** tools are tools that enable an application programmer to construct forms and reports with minimal programming effort.
- **Sophisticated users** interact with the system without writing programs. Instead, they form their requests either using a database query language or by using tools such as data analysis software. Analysts who submit queries to explore data in the database fall in this category.
- **Specialized users** are sophisticated users who write specialized database applications that do not fit into the traditional data-processing framework. Among these applications are computer-aided design systems, knowledge-base and expert systems, systems that store data with complex data types (for example,

graphics data and audio data), and environment-modeling systems. Chapter 22 covers several of these applications.

1.10.2 Database Administrator

One of the main reasons for using DBMSs is to have central control of both the data and the programs that access those data. A person who has such central control over the system is called a **database administrator (DBA)**. The functions of a DBA include:

- **Schema definition.** The DBA creates the original database schema by executing a set of data definition statements in the DDL.

Storage structure and access-method definition.

- **Schema and physical-organization modification.** The DBA carries out changes to the schema and physical organization to reflect the changing needs of the organization, or to alter the physical organization to improve performance.

Granting of authorization for data access. By granting different types of authorization, the database administrator can regulate which parts of the database various users can access. The authorization information is kept in a special system structure that the database system consults whenever someone attempts to access the data in the system.

Routine maintenance. Examples of the database administrator's routine maintenance activities are:

- Periodically backing up the database, either onto tapes or onto remote servers, to prevent loss of data in case of disasters such as flooding.
- Ensuring that enough free disk space is available for normal operations, and upgrading disk space as required.
- Monitoring jobs running on the database and ensuring that performance is not degraded by very expensive tasks submitted by some users.

1.11 Summary

- A **database-management system (DBMS)** consists of a collection of interrelated data and a collection of programs to access that data. The data describe one particular enterprise.
- The primary goal of a DBMS is to provide an environment that is both convenient and efficient for people to use in retrieving and storing information.

- Database systems are ubiquitous today, and most people interact, either directly or indirectly, with databases many times every day.
- Database systems are designed to store large bodies of information. The management of data involves both the definition of structures for the storage of information and the provision of mechanisms for the manipulation of information. In addition, the database system must provide for the safety of the information stored, in the face of system crashes or attempts at unauthorized access. If data are to be shared among several users, the system must avoid possible anomalous results.
- A major purpose of a database system is to provide users with an abstract view of the data. That is, the system hides certain details of how the data are stored and maintained.
- Underlying the structure of a database is the **data model**: a collection of conceptual tools for describing data, data relationships, data semantics, and data constraints.
- The relational data model is the most widely deployed model for storing data in databases. Other data models are the object-oriented model, the object-relational model, and semistructured data models.
- A **data-manipulation language (DML)** is a language that enables users to access or manipulate data. Nonprocedural DMLs, which require a user to specify only what data are needed, without specifying exactly how to get those data, are widely used today.
- A **data-definition language (DDL)** is a language for specifying the database schema and as well as other properties of the data.
- Database design mainly involves the design of the database schema. The entity-relationship (E-R) data model is a widely used data model for database design. It provides a convenient graphical representation to view data, relationships, and constraints.
- A database system has several subsystems.
 - * The **storage manager** subsystem provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.

- * The **query processor** subsystem compiles and executes DDL and DML statements.
- **Transaction management** ensures that the database remains in a consistent (correct) state despite system failures. The transaction manager ensures that concurrent transaction executions proceed without conflicting.
- The architecture of a database system is greatly influenced by the underlying computer system on which the database system runs. Database systems can be centralized, or client-server, where one server machine executes work on behalf of multiple client machines. Database systems can also be designed to exploit parallel computer architectures. Distributed databases span multiple geographically separated machines.

1.12 Questions for Discussing

- 1) Define Database
- 2) Explain the purpose of Database system
- 3) Describe the Database system Architecture
- 4) Explain the various Data models

1.13 Suggested Reading

Recommended Text

A.Silberchatz, H.F Korth and S.Sudharshan, 2006, Database System Concepts, 5th Edition, Tata McGraw Hill, New Delhi.

Reference Books

J.D Ullaman, 1988, Principles of Database Systems, Galgotia Publishers, New Delhi.

Lesson - 2

RELATIONAL DATABASES

CONTENTS

- 2.0 Aims and Objectives
- 2.1 Introduction
- 2.2 Structure of Relational Database
- 2.3 Database Schema
- 2.4 Keys
- 2.5 Relational Algebra
- 2.6 Null Values
- 2.7 Summary
- 2.8 Questions for Discussion
- 2.9 Suggested Reading

2.0 Aims and Objectives

After studying this lesson you should be able to:

- Define Relational Model
- Identify the different types of Relational Algebra operation
- Describe Null Values

2.1 Introduction

The relational model is today the primary data model for commercial data-processing applications. It attained its primary position because of its simplicity, which eases the job of the programmer, compared to earlier data models such as the network model or the hierarchical model.:

2.2 Structure of Relational Database

A relational database consists of a collection of **tables**, each of which is assigned a unique name. For example, consider the *instructor* table of Figure 2.1, which stores information about instructors. The table has four column headers: *ID*, *name*, *dept name*,

and *salary*. Each row of this table records information about an instructor, consisting of the instructor's *ID*, *name*, *deptname*, and *salary*. Similarly, the *course* table of Figure 2.2 stores information about courses, consisting of a *courseid*, *title*, *deptname*, and *credits*, for each course. Note that each instructor is identified by the value of the column *ID*, while each course is identified by the value of the column *courseid*.

Figure 2.3 shows a third table, *prereq*, which stores the prerequisite courses for each course. The table has two columns, *courseid* and *prereq id*. Each row consists of a pair of course identifiers such that the second course is a prerequisite for the first course.

Thus, a row in the *prereq* table indicates that two courses are *related* in the sense that one course is a prerequisite for the other. As another example, we consider the table *instructor*, a row in the table can be thought of as representing the relationship

<i>ID</i>	<i>name</i>	<i>deptname</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The instructor relation.

between a specified *ID* and the corresponding values for *name*, *deptname*, and *salary* values.

In general, a row in a table represents a *relationship* among a set of values. Since a table is a collection of such relationships, there is a close correspondence between the concept of *table* and the mathematical concept of *relation*, from which the relational data model takes its name. In mathematical terminology, a *tuple* is simply a sequence (or list) of

values. A relationship between n values is represented mathematically by an n -tuple of values, i.e., a tuple with n values, which corresponds to a row in a table.

<i>courseid</i>	<i>title</i>	<i>deptname</i>	<i>credits</i>
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Figure 2.2 The course relation.

<i>courseid</i>	<i>prereqid</i>
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

Figure 2.3 The prereq relation.

Thus, in the relational model the term **relation** is used to refer to a table, while the term **tuple** is used to refer to a row. Similarly, the term **attribute** refers to a column of a table.

Examining Figure 2.1, we can see that the relation *instructor* has four attributes: *ID*, *name*, *deptname*, and *salary*.

We use the term **relation instance** to refer to a specific instance of a relation, i.e., containing a specific set of rows. The instance of *instructor* shown in Figure 2.1 has 12 tuples, corresponding to 12 instructors.

In this chapter, we shall be using a number of different relations to illustrate the various concepts underlying the relational data model. These relations represent part of a university. They do not include all the data an actual university database would contain, in order to simplify our presentation. We shall discuss criteria for the appropriateness of relational structures in great detail in Chapters 6 and 7.

The order in which tuples appear in a relation is irrelevant, since a relation is a *set* of tuples. Thus, whether the tuples of a relation are listed in sorted order, as in Figure 2.1, or are unsorted, as in Figure 2.4, does not matter; the relations in

<i>ID</i>	<i>name</i>	<i>deptname</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Figure 2.4 *Unsorted display of the instructor relation.*

the two figures are the same, since both contain the same set of tuples. For ease of exposition, we will mostly show the relations sorted by their first attribute.

For each attribute of a relation, there is a set of permitted values, called the **domain** of that attribute. Thus, the domain of the *salary* attribute of the *instructor* relation is the set of all possible salary values, while the domain of the *name* attribute is the set of all possible instructor names.

We require that, for all relations r , the domains of all attributes of r be atomic. A domain is **atomic** if elements of the domain are considered to be indivisible units. For example, suppose the table *instructor* had an attribute *phone-number*, which can store a set of phone numbers corresponding to the instructor. Then the domain of *phone-number* would not be atomic, since an element of the domain is a set of phone numbers, and it has subparts, namely the individual phone numbers in the set.

The important issue is not what the domain itself is, but rather how we use domain elements in our database. Suppose now that the *phone-number* attribute stores a single phone number. Even then, if we split the value from the phone number attribute into a country code, an area code and a local number, we would be treating it as a nonatomic value. If we treat each phone number as a single indivisible unit, then the attribute *phone-number* would have an atomic domain.

In this chapter, as well as in Chapters 3 through 6, we assume that all attributes have atomic domains. We shall discuss extensions to the relational data model to permit nonatomic domains.

The **null** value is a special value that signifies that the value is unknown or does not exist. For example, suppose as before that we include the attribute *phone number* in the *instructor* relation. It may be that an instructor does not have a phone number at all, or that the telephone number is unlisted. We would then have to use the null value to signify that the value is unknown or does not exist. We shall see later that null values cause a number of difficulties when we access or update the database, and thus should be eliminated if at all possible.

2.3 Database Schema

When we talk about a database, we must differentiate between the **database schema**, which is the logical design of the database, and the **database instance**, which is a snapshot of the data in the database at a given instant in time.

The concept of a relation corresponds to the programming-language notion of a variable, while the concept of a **relation schema** corresponds to the programming-language notion of type definition.

In general, a relation schema consists of a list of attributes and their corresponding domains. We shall not be concerned about the precise definition of the domain of each attribute until we discuss the SQL language in Chapter 3.

The concept of a relation instance corresponds to the programming-language notion of a value of a variable. The value of a given variable may change with time;

<i>deptname</i>	<i>building</i>	<i>budget</i>
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

Figure 2.5 *The department relation.*

similarly the contents of a relation instance may change with time as the relation is updated. In contrast, the schema of a relation does not generally change.

Although it is important to know the difference between a relation schema and a relation instance, we often use the same name, such as *instructor*, to refer to both the schema and the instance. Where required, we explicitly refer to the schema or to the instance, for example “the *instructor* schema,” or “an instance of the *instructor* relation.” However, where it is clear whether we mean the schema or the instance, we simply use the relation name.

Consider the *department* relation of Figure 2.5. The schema for that relation is

department (*deptname*, *building*, *budget*)

Note that the attribute *deptname* appears in both the *instructor* schema and the *department* schema. This duplication is not a coincidence. Rather, using common attributes in relation schemas is one way of relating tuples of distinct relations. For example, suppose

we wish to find the information about all the instructors who work in the Watson building. We look first at the *department* relation to find the *deptname* of all the departments housed in Watson. Then, for each such department, we look in the *instructor* relation to find the information about the instructor associated with the corresponding *deptname*.

Let us continue with our university database example.

Each course in a university may be offered multiple times, across different semesters, or even within a semester. We need a relation to describe each individual offering, or section, of the class. The schema is

section (courseid, secid, semester, year, building, roomnumber, timeslotid)

Figure 2.6 shows a sample instance of the *section* relation.

We need a relation to describe the association between instructors and the class sections that they teach. The relation schema to describe this association is

teaches (ID, courseid, secid, semester, year)

<i>courseid</i>	<i>secid</i>	<i>semester</i>	<i>year</i>	<i>building</i>	<i>roomnumber</i>	<i>timeslot-id</i>
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

Figure 2.6 The section relation.

Figure 2.7 shows a sample instance of the *teaches* relation.

As you can imagine, there are many more relations maintained in a real uni-versity database. In addition to those relations we have listed already, *instructor*, *department*, *course*, *section*, *prereq*, and *teaches*, we use the following relations in this text:

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

Figure 2.7 The Teaches relation

- *student* (*ID*, *name*, *deptname*, *totcred*)
- *advisor* (*sid*, *i id*)
- *takes* (*ID*, *courseid*, *secid*, *semester*, *year*, *grade*)
- *classroom* (*building*, *room-number*, *capacity*)
- *timeslot* (*timeslotid*, *day*, *starttime*, *endtime*)

2.4 Keys

We must have a way to specify how tuples within a given relation are distinguished. This is expressed in terms of their attributes. That is, the values of the attribute values of a tuple must be such that they can *uniquely identify* the tuple. In other words, no two tuples in a relation are allowed to have exactly the same value for all attributes.

A **superkey** is a set of one or more attributes that, taken collectively, allow us to identify uniquely a tuple in the relation. For example, the *ID* attribute of the relation *instructor*

is sufficient to distinguish one instructor tuple from another. Thus, *ID* is a superkey. The *name* attribute of *instructor*, on the other hand, is not a superkey, because several instructors might have the same name.

Formally, let R denote the set of attributes in the schema of relation r . If we say that a subset K of R is a *superkey* for r , we are restricting consideration to instances of relations r in which no two distinct tuples have the same values on all attributes in K . That is, if t_1 and t_2 are in r and $t_1 \wedge t_2$, then $t_1.K \wedge t_2.K$.

A superkey may contain extraneous attributes. For example, the combination of *ID* and *name* is a superkey for the relation *instructor*. If K is a superkey, then so is any superset of K . We are often interested in superkeys for which no proper subset is a superkey. Such minimal superkeys are called **candidate keys**.

It is possible that several distinct sets of attributes could serve as a candidate key. Suppose that a combination of *name* and *deptname* is sufficient to distinguish among members of the *instructor* relation. Then, both $\{ID\}$ and $\{name, deptname\}$ are candidate keys. Although the attributes *ID* and *name* together can distinguish *instructor* tuples, their combination, $\{ID, name\}$, does not form a candidate key, since the attribute *ID* alone is a candidate key.

We shall use the term **primary key** to denote a candidate key that is chosen by the database designer as the principal means of identifying tuples within a relation. A key (whether primary, candidate, or super) is a property of the entire relation, rather than of the individual tuples. Any two individual tuples in the relation are prohibited from having the same value on the key attributes at the same time. The designation of a key represents a constraint in the real-world enterprise being modeled.

Primary keys must be chosen with care. As we noted, the name of a person is obviously not sufficient, because there may be many people with the same name. In the United States, the social-security number attribute of a person would be a candidate key. Since non-U.S. residents usually do not have social-security

numbers, international enterprises must generate their own unique identifiers. An alternative is to use some unique combination of other attributes as a key.

The primary key should be chosen such that its attribute values are never, or very rarely, changed. For instance, the address field of a person should not be part of the primary

key, since it is likely to change. Social-security numbers, on the other hand, are guaranteed never to change. Unique identifiers generated by enterprises generally do not change, except if two enterprises merge; in such a case the same identifier may have been issued by both enterprises, and a reallocation of identifiers may be required to make sure they are unique.

It is customary to list the primary key attributes of a relation schema before the other attributes; for example, the *deptname* attribute of *department* is listed first, since it is the primary key. Primary key attributes are also underlined.

A relation, say r_1 , may include among its attributes the primary key of an-other relation, say r_2 . This attribute is called a **foreign key** from r_1 , referencing r_2 . The relation r_1 is also called the **referencing relation** of the foreign key dependency, and r_2 is called the **referenced relation** of the foreign key. For example, the attribute *deptname* in *instructor* is a foreign key from *instructor*, referencing *department*, since *deptname* is the primary key of *department*. In any database instance, given any tuple, say t_a , from the *instructor* relation, there must be some tuple, say t_b , in the *department* relation such that the value of the *deptname* attribute of t_a is the same as the value of the primary key, *deptname*, of t_b .

Now consider the *section* and *teaches* relations. It would be reasonable to require that if a section exists for a course, it must be taught by at least one instructor; however, it could possibly be taught by more than one instructor. To enforce this constraint, we would require that if a particular (*courseid*, *secid*, *semester*, *year*) combination appears in *section*, then the same combination must appear in *teaches*. However, this set of values does not form a primary key for *teaches*, since more than one instructor may teach one such section. As a result, we cannot declare a foreign key constraint from *section* to *teaches* (although we can define a foreign key constraint in the other direction, from *teaches* to *section*).

The constraint from *section* to *teaches* is an example of a **referential integrity constraint**; a referential integrity constraint requires that the values appearing in specified attributes of any tuple in the referencing relation also appear in specified attributes of at least one tuple in the referenced relation.

2.4.1 Schema Diagrams

A database schema, along with primary key and foreign key dependencies, can be depicted by **schema diagrams**. Figure 2.8 shows the schema diagram for our university organization. Each relation appears as a box, with the relation name at the top in blue, and the attributes listed inside the box. Primary key attributes are shown underlined. Foreign

key dependencies appear as arrows from the foreign key attributes of the referencing relation to the primary key of the referenced relation.

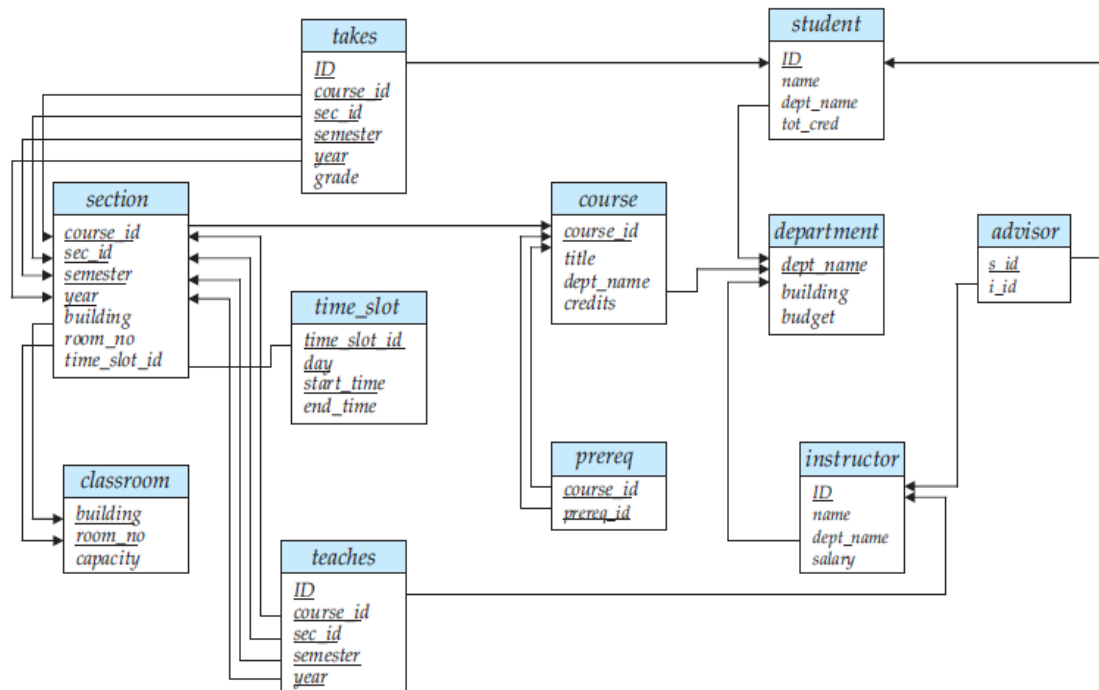


Figure 2.8 Schema diagram for the university database

Referential integrity constraints other than foreign key constraints are not shown explicitly in schema diagrams. We will study a different diagrammatic representation called the entity-relationship diagram later, in Chapter . Entity-relationship diagrams let us represent several kinds of constraints, including general referential integrity constraints.

2.5 Fundamental Relational-Algebra Operations

The select, project and rename operations are called unary operations, because they operate on one relation. The other three operations operate on pairs of relations and are therefore called binary operations

2.5.1 The Select Operation

The **select** operation selects tuples that satisfy a given predicate. We use the lowercase Greek letter sigma (σ) to denote selection. The predicate appears as a subscript to σ . The argument relation is in parentheses after the σ . Thus, to select

<i>ID</i>	<i>name</i>	<i>deptname</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.9 The instructor relation

those tuples of the *instructor* relation where the instructor is in the “Physics” department, we write:

$V_{dept.name} = \text{“Physics”}$ (*instructor*)

If the *instructor* relation is as shown in Figure 2.10, then the relation that results from the preceding query is as shown in Figure 2.11

We can find all instructors with salary greater than \$90,000 by writing:

$a_{salary} > 90000$ (*instructor*)

In general, we allow comparisons using =, ^, <, <, >, and > in the selection predicate. Furthermore, we can combine several predicates into a larger predicate by using the connectives *and* (∧), *or* (∨), and *not* (¬). Thus, to find the instructors in Physics with a salary greater than \$90,000, we write:

$U_{dept.name = \text{“Physics”} \wedge salary > 90000}$ (*instructor*)

The selection predicate may include comparisons between two attributes. To illustrate, consider the relation *department*. To find all departments whose name is the same as their building name, we can write:

$V_{dept-name} = building (department)$

<i>ID</i>	<i>name</i>	<i>deptname</i>	<i>salary</i>
22222 33456	Einstein Gold	Physics Physics	95000 87000

Figure 2.10 Result of $(T_{dept-name = \text{«physics- {instructor}}}$.

2.5.2 The Project Operation

Suppose we want to list all instructors' *ID*, *name*, and *salary*, but do not care about the *deptname*. The **project** operation allows us to produce this relation. The project operation is a unary operation that returns its argument relation, with certain attributes left out. Since a relation is a set, any duplicate rows are eliminated. Projection is denoted by the uppercase Greek letter pi (π). We list those attributes that we wish to appear in the result as a subscript to π . The argument relation follows in parentheses. We write the query to produce such a list as:

$\pi_{ID, name, salary}(instructor)$

Figure 6.3 shows the relation that results from this query.

2.5.3 Composition of Relational Operations

The fact that the result of a relational operation is itself a relation is important. Consider the more complicated query "Find the name of all instructors in the Physics department." We write:

<i>ID</i>	<i>name</i>	<i>salary</i>
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000

76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

Figure 2.11 Result of $\Pi_{ID, name, salary}(instructor)$.

$\sigma_{deptname = \text{"Physics"}}(instructor)$

Notice that, instead of giving the name of a relation as the argument of the projection operation, we give an expression that evaluates to a relation.

In general, since the result of a relational-algebra operation is of the same type (relation) as its inputs, relational-algebra operations can be composed together into a **relational-algebra expression**. Composing relational-algebra operations into relational-algebra expressions is just like composing arithmetic operations (such as +, —, *, and =) into arithmetic expressions.

2.5.4 The Union Operation

Consider a query to find the set of all courses taught in the Fall 2009 semester, the Spring 2010 semester, or both. The information is contained in the *section* relation (Figure 2.12). To find the set of all courses taught in the Fall 2009 semester, we write:

$\sigma_{semester = \text{«Fall»}}(section)$

To find the set of all courses taught in the Spring 2010 semester, we write:

$\sigma_{semester = \text{«Spring»}}(section)$

To answer the query, we need the **union** of these two sets; that is, we need all section IDs that appear in either or both of the two relations. We find these data

<i>Courseid</i>	<i>secid</i>	<i>semester</i>	<i>year</i>	<i>building</i>	<i>roomnumber</i>	<i>timeslotid</i>
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A

CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

Figure 2.12 The section relation.

<i>course_id</i>
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

Figure 2.13 Courses offered in either Fall 2009, Spring 2010 or both semesters.

by the binary operation union, denoted, as in set theory, by $\cup$. So the expression needed is:

$\pi_{courseid} (\sigma_{semester = 'Fall'} \bowtie \sigma_{year=2009} (section)) \cup$

$\pi_{courseid} (\sigma_{semester = 'Spring'} \bowtie \sigma_{year=2010} (section))$

The result relation for this query appears in Figure 6.5. Notice that there are 8 tuples in the result, even though there are 3 distinct courses offered in the Fall 2009 semester and 6 distinct courses offered in the Spring 2010 semester. Since relations are sets, duplicate values such as CS-101, which is offered in both semesters, are replaced by a single occurrence.

Observe that, in our example, we took the union of two sets, both of which consisted of *courseid* values. In general, we must ensure that unions are taken between *compatible* relations. For example, it would not make sense to take the union of the *instructor* relation and the *student* relation. Although both relations have four attributes, they differ on the *salary* and *totcred* domains. The union of these two attributes would not make sense in most situations. Therefore, for a union operation $r \cup s$ to be valid, we require that two conditions hold:

1. The relations r and s must be of the same arity. That is, they must have the same number of attributes.
2. The domains of the i th attribute of r and the i th attribute of s must be the same, for all i .

Note that r and s can be either database relations or temporary relations that are the result of relational-algebra expressions.

2.5.5 The Set-Difference Operation

The **set-difference** operation, denoted by $-$, allows us to find tuples that are in one relation but are not in another. The expression $r - s$ produces a relation containing those tuples in r but not in s .

<i>course_id</i>
CS-347
PHY-101

Figure 2.14 Courses offered in the Fall 2009 semester but not in Spring 2010 semester.

We can find all the courses taught in the Fall 2009 semester but not in Spring 2010 semester by writing:

$\pi_{courseid} (\sigma_{semester = \text{<Fall>}, year=2009} (section)) -$

$\pi_{courseid} (\sigma_{semester = \text{"Spring"}, year=2010} (section))$

The result relation for this query appears in Figure 2.14.

As with the union operation, we must ensure that set differences are taken between *compatible* relations. Therefore, for a set-difference operation $r - s$ to be valid, we require

that the relations r and s be of the same arity and that the domains of the i th attribute of r and the i th attribute of s be the same, for all i .

2.5.6 The Cartesian-Product Operation

The **Cartesian-product** operation, denoted by a cross ($\times$), allows us to combine information from any two relations. We write the Cartesian product of relations r_1 and r_2 as $r_1 \times r_2$.

Recall that a relation is by definition a subset of a Cartesian product of a set of domains. From that definition, we should already have an intuition about the definition of the Cartesian-product operation. However, since the same attribute name may appear in both r_1 and r_2 , we need to devise a naming schema to distinguish between these attributes. We do so here by attaching to an attribute the name of the relation from which the attribute originally came. For example, the relation schema for $r = \text{instructor} \times \text{teaches}$ is:

(instructor.ID, instructor.name, instructor.deptname, instructor.salary teaches.ID, teaches.courseid, teaches.secid, teaches.semester, teaches.year)

With this schema, we can distinguish *instructor.ID* from *teaches.ID*. For those attributes that appear in only one of the two schemas, we shall usually drop the relation-name prefix. This simplification does not lead to any ambiguity. We can then write the relation schema for r as:

(instructor.ID, name, deptname, salary teaches.ID, courseid, secid, semester, year)

This naming convention *requires* that the relations that are the arguments of the Cartesian-product operation have distinct names. This requirement causes problems in some cases, such as when the Cartesian product of a relation with itself is desired. A similar problem arises if we use the result of a relational-algebra expression in a Cartesian product, since we shall need a name for the relation so

<i>ID</i>	<i>courseid</i>	<i>secid</i>	<i>semester</i>	<i>year</i>
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010

22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

Figure 2.15 The teaches relation.

that we can refer to the relation's attributes.

Now that we know the relation schema for $r = \text{instructor} \times \text{teaches}$, what tuples appear in r ? As you may suspect, we construct a tuple of r out of each possible pair of tuples: one from the *instructor* relation (Figure 2.11) and one from the *teaches* relation (Figure 2.15). Thus, r is a large relation, as you can see from Figure 2.16, which includes only a portion of the tuples that make up r .¹

Assume that we have $n1$ tuples in *instructor* and $n2$ tuples in *teaches*. Then, there are $n1 * n2$ ways of choosing a pair of tuples—one tuple from each relation; so there are $n1 * n2$ tuples in r . In particular, note that for some tuples t in r , it may be that $t[\text{instructor.ID}] \wedge t[\text{teaches.ID}]$.

In general, if we have relations $r1(R1)$ and $r2(R2)$, then $r1 \times r2$ is a relation whose schema is the concatenation of $R1$ and $R2$. Relation R contains all tuples t for which there is a tuple $t1$ in $r1$ and a tuple $t2$ in $r2$ for which $t[R_1] = t_1[R1]$ and $t[R_2] = t_2[R2]$.

Suppose that we want to find the names of all instructors in the Physics department together with the *courseid* of all courses they taught. We need the information in both the *instructor* relation and the *teaches* relation to do so. If we write:

$(Tdeptname \wedge Phys_{ics} (\text{instructor} \times \text{teaches}))$ then the result is the relation in Figure 2.17.

<i>inst.ID</i>	<i>name</i>	<i>deptname</i>	<i>salary</i>	<i>teaches.ID</i>	<i>courseid</i>	<i>secid</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Physics	95000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Physics	95000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	FIN-201	1	Spring	2010
10101	Srinivasan	Physics	95000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Physics ...	95000	22222	PHY-101	1	Fall ...	2009
12121	W u	... Physics	95000	10101	CS-101	1	... Fall	2009
12121	W u	Physics	95000	10101	CS-315	1	Spring	2010
12121	W u	Physics	95000	10101	CS-347	1	Fall	2009
12121	W u	Physics	95000	10101	FIN-201	1	Spring	2010
12121	W u	Physics	95000	15151	MU-199	1	Spring	2010
12121	W u	Physics ...	95000	22222	PHY-101	1	Fall ...	2009
15151	... Mozart	... Physics	95000	10101	CS-101	1	... Fall	2009
15151	Mozart	Physics	95000	10101	CS-315	1	Spring	2010
15151	Mozart	Physics	95000	10101	CS-347	1	Fall	2009
15151	Mozart	Physics	95000	10101	FIN-201	1	Spring	2010
15151	Mozart	Physics	95000	15151	MU-199	1	Spring	2010
15151	Mozart ...	Physics ...	95000	22222	PHY-101	1	Fall ...	2009
22222	... Einstein	... Physics	95000	10101	CS-101	1	... Fall	2009
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2010
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2009
22222	Einstein	Physics	95000	10101	FIN-201	1	Spring	2010
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...

Figure 2.16 Result of instructor × teaches.

We have a relation that pertains only to instructors in the Physics department. However, the *courseid* column may contain information about courses that were not taught by the corresponding instructor. (If you do not see why that is true, recall that the Cartesian product takes all possible pairings of one tuple from *instructor* with one tuple of *teaches*.)

Since the Cartesian-product operation associates *every* tuple of *instructor* with every tuple of *teaches*, we know that if a an instructor is in the Physics department, and has taught a course (as recorded in the *teaches* relation), then there is some

<i>inst.ID</i>	<i>name</i>	<i>deptname</i>	<i>salary</i>	<i>teaches.ID</i>	<i>courseid</i>	<i>secid</i>	<i>semester</i>	<i>year</i>
22222	Einstein	Physics	95000	10101	CS-437	1	Fall	2009
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2010
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2010
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2009
22222	Einstein ...	Physics ...	95000	32343	HIS-351	1	Spring ...	2010
33456	... Gold	... Physics	87000	10101	CS-437	1	... Fall	2009
33456	Gold	Physics	87000	10101	CS-315	1	Spring	2010
33456	Gold	Physics	87000	12121	FIN-201	1	Spring	2010
33456	Gold	Physics	87000	15151	MU-199	1	Spring	2010
33456	Gold	Physics	87000	22222	PHY-101	1	Fall	2009
33456	Gold	Physics	87000	32343	HIS-351	1	Spring	2010
...	...	...	...	...	...	...	...	...

Figure 2.17 Result of $\sigma_{deptname = \text{'Physics'}}(\pi_{inst.ID, teaches.ID}(instructor \times teaches))$.

tuple in $\sigma_{dept-name=\text{'Physics'}}(instructor \times teaches)$ that contains his name, and which satisfies *instructor.ID* = *teaches.ID*. So, if we write:

instructor.ID = *teaches.ID* ($\wedge$ dept.

name =

$\pi_{name}(\sigma_{dept-name=\text{'Physics'}}(instructor \times teaches))$

we get only those tuples of *instructor* $\times$ *teaches* that pertain to instructors in Physics and the courses that they taught.

Finally, since we only want the names of all instructors in the Physics department together with the *courseid* of all courses they taught, we do a projection:

$$\text{Uname, courseid (instructor.ID = teaches.ID (vdept} \\ \text{"Physics"(instructor} \times \text{teaches)))}$$

The result of this expression, shown in Figure 2.18, is the correct answer to our query. Observe that although instructor Gold is in the Physics department, he does not teach any course (as recorded in the *teaches* relation), and therefore does not appear in the result.

<i>name</i>	<i>course_id</i>
Einstein	PHY-101

Figure 2.18 Result of

$$\text{Tname, courseid (uinstructor.ID = teachesIDvdeptname =} \\ \text{<V*ysicAinstructor} \times \text{teaches)))}$$

Note that there is often more than one way to write a query in relational algebra. Consider the following query:

$$\text{Uname, courseid (ainstructor . ID}^{\text{teaches . ID}} \\ \text{(Kt name}^{\text{Physics (instructor))}} \times \text{teaches))}$$

Note the subtle difference between the two queries: in the query above, the selection that restricts *deptname* to Physics is applied to *instructor*, and the Cartesian product is applied subsequently; in contrast, the Cartesian product was applied before the selection in the earlier query. However, the two queries are **equivalent**; that is, they give the same result on any database.

2.5.7 The Rename Operation

Unlike relations in the database, the results of relational-algebra expressions do not have a name that we can use to refer to them. It is useful to be able to give them names; the **rename** operator, denoted by the lowercase Greek letter rho (ρ), lets us do this. Given a relational-algebra expression *E*, the expression

$$Px(E)$$

returns the result of expression E under the name x .

A relation r by itself is considered a (trivial) relational-algebra expression. Thus, we can also apply the rename operation to a relation r to get the same relation under a new name.

A second form of the rename operation is as follows: Assume that a relational-algebra expression E has arity n . Then, the expression

$$Px(A1, A2, \dots, An)(E)$$

returns the result of expression E under the name x , and with the attributes renamed to $A1, A2, \dots, A_n$.

To illustrate renaming a relation, we consider the query “Find the highest salary in the university.” Our strategy is to (1) compute first a temporary relation consisting of those salaries that are *not* the largest and (2) take the set difference between the relation n_{salary} (*instructor*) and the temporary relation just computed, to obtain the result.

1. Step 1: To compute the temporary relation, we need to compare the values of all salaries. We do this comparison by computing the Cartesian product *instructor* $\times$ *instructor* and forming a selection to compare the value of any two salaries appearing in one tuple. First, we need to devise a mechanism to distinguish between the two *salary* attributes. We shall use the rename operation to rename one reference to the instructor relation; thus we can reference the relation twice without ambiguity.

<i>salary</i>
65000
90000
40000
60000
87000
75000
62000
72000
80000
92000

Figure 2.19 Result of the subexpression

$\pi_{instructor.salary} (< \pi_{instructor.salary} < d.salary (instructor \times \rho_d (instructor)))$.

We can now write the temporary relation that consists of the salaries that are not the largest:

$\pi_{instructor.salary} (< \pi_{instructor.salary} < d.salary (instructor \times \rho_d (instructor)))$

This expression gives those salaries in the *instructor* relation for which a larger salary appears somewhere in the *instructor* relation (renamed as *d*). The result contains all salaries *except* the largest one. Figure 6.11 shows this relation.

2. Step 2: The query to find the largest salary in the university can be written as:

$\wedge salary (instructor) -$

$\pi_{instructor.salary} (\pi_{instructor.salary} < d.salary (instructor \times \rho_d (instructor)))$

Figure 2.20 shows the result of this query.

The rename operation is not strictly required, since it is possible to use a positional notation for attributes. We can name attributes of a relation implicitly by using a positional notation, where \$1, \$2,... refer to the first attribute, the second attribute, and so on. The positional notation also applies to results of relational-algebra operations. The following relational-algebra expression illustrates the

<i>salary</i>
95000

Figure 2.20 Highest salary in the university.

use of positional notation to write the expression we saw earlier, which computes salaries that are not the largest:

$\pi_{\$4} (cT_{\$4} < \$8 (instructor \times instructor))$

Note that the Cartesian product concatenates the attributes of the two relations. Thus, for the result of the Cartesian product (*instructor* $\times$ *instructor*), \$4 refers to the *salary* attribute from the first occurrence of *instructor*, while \$8 refers to the *salary* attribute from the second occurrence of *instructor*. A positional notation can also be used to refer to relation names, if a binary operation needs to distinguish between its two operand relations. For example, \$R1 could refer to the first operand relation, and \$R2 could refer to the second operand relation of a Cartesian product. However, the positional notation is inconvenient for humans,

since the position of the attribute is a number, rather than an easy-to-remember attribute name. Hence, we do not use the positional notation in this textbook.

2.5.8 Additional Relational-Algebra Operations

The fundamental operations of the relational algebra are sufficient to express any relational-algebra query. However, if we restrict ourselves to just the fundamental operations, certain common queries are lengthy to express. Therefore, we define additional operations that do not add any power to the algebra, but simplify

Course-Id
CS-101

Figure 2.21 Courses offered in both the Fall 2009 and Spring 2010 semesters.

common queries. For each new operation, we give an equivalent expression that uses only the fundamental operations.

2.5.8.1 The Set-Intersection Operation

The first additional relational-algebra operation that we shall define is **set intersection** ($\cap$). Suppose that we wish to find the set of all courses taught in both the Fall 2009 and the Spring 2010 semesters. Using set intersection, we can write

$\pi_{course_id} ((\sigma_{semester = \text{«Fall»} \wedge year = 2009} (section)) \cap \sigma_{semester = \text{«Spring»} \wedge year = 2010} (section))$

The result relation for this query appears in Figure 2.21.

Note that we can rewrite any relational-algebra expression that uses set intersection by replacing the intersection operation with a pair of set-difference operations as:

$$r \cap s = r - (r - s)$$

Thus, set intersection is not a fundamental operation and does not add any power to the relational algebra. It is simply more convenient to write $r \cap s$ than to write

$$r - (r - s).$$

2.5.8.2 The Natural-Join Operation

It is often desirable to simplify certain queries that require a Cartesian product. Usually, a query that involves a Cartesian product includes a selection operation on the result of the Cartesian product. The selection operation most often requires that all attributes that are common to the relations that are involved in the Cartesian product be equated.

In our example query from Section 2.5.6 that combined information from the *instructor* and *teaches* tables, the matching condition required *instructor.ID* to be equal to *teaches.ID*. These are the only attributes in the two relations that have the same name.

The *natural join* is a binary operation that allows us to combine certain selections and a Cartesian product into one operation. It is denoted by the **join** symbol $\bowtie$. The natural-join operation forms a Cartesian product of its two arguments, performs a selection forcing equality on those attributes that appear in both relation schemas, and finally removes duplicate attributes. Returning to the example of the relations *instructor* and *teaches*, computing *instructor natural join teaches* considers only those pairs of tuples where both the tuple from *instructor* and the

<i>ID</i>	<i>name</i>	<i>deptname</i>	<i>salary</i>	<i>courseid</i>	<i>secid</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000	EE-181	1	Spring	2009

Figure 2.22 The natural join of the *instructor* relation with the *teaches* relation.

tuple from *teaches* have the same value on the common attribute *ID*. The result relation, shown in Figure 2.22, has only 13 tuples, the ones that give information about an instructor and a course that that instructor actually teaches. Notice that we do not repeat those attributes that appear in the schemas of both relations; rather they appear only once. Notice also the order in which the attributes are listed: first the attributes common to the schemas of both relations, second those attributes unique to the schema of the first relation, and finally, those attributes unique to the schema of the second relation.

Although the definition of natural join is complicated, the operation is easy to apply. As an illustration, consider again the example “Find the names of all instructors together with the *courseid* of all courses they taught.” We express this query by using the natural join as follows:

$$\pi_{name, courseid}(instructor \bowtie teaches)$$

Since the schemas for *instructor* and *teaches* have the attribute *ID* in common, the natural-join operation considers only pairs of tuples that have the same value on *ID*. It combines each such pair of tuples into a single tuple on the union of the two schemas; that is, $(ID, name, deptname, salary, courseid)$. After performing the projection, we obtain the relation in Figure 2.23.

Consider two relation schemas *R* and *S*—which are, of course, lists of attribute names. If we consider the schemas to be *sets*, rather than lists, we can denote those attribute names that appear in both *R* and *S* by $R \cap S$, and denote those attribute names that appear in *R*, in *S*, or in both by $R \cup S$. Similarly, those attribute names that appear in *R* but not *S* are denoted by $R - S$, whereas $S - R$ denotes those

<i>name</i>	<i>courseid</i>
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101

Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-319
Kim	EE-181

Figure 2.23 Result of $\pi_{name, courseid}(instructor \bowtie teaches)$.

attribute names that appear in S but not in R . Note that the union, intersection, and difference operations here are on sets of attributes, rather than on relations. We are now ready for a formal definition of the natural join. Consider two relations $r(R)$ and $s(S)$. The **natural join** of r and s , denoted by $r \bowtie s$, is a relation on schema $R \cup S$ formally defined as follows:

$$r \bowtie s = \pi_{R \cup S} (\sigma_{(A_1=r.A_1 \wedge A_2=s.A_2 \wedge \dots \wedge A_n=r.A_n \wedge A_{n+1}=s.A_{n+1} \wedge \dots \wedge A_m=s.A_m)} (r \times s))$$

where $R \cap S = \{A_1, A_2, \dots, A_n\}$.

Please note that if $r(R)$ and $s(S)$ are relations without any attributes in common, that is, $R \cap S = \emptyset$, then $r \bowtie s = r \times s$.

Let us consider one more example of the use of natural join, to write the query “Find the names of all instructors in the Comp. Sci. department together with the course titles of all the courses that the instructors teach.”

$$\pi_{name, title} (\sigma_{dept_name = 'Comp. Sci.'} (instructor \bowtie teaches \bowtie course))$$

The result relation for this query appears in Figure 2.24.

Notice that we wrote $instructor \bowtie teaches \bowtie course$ without inserting parentheses to specify the order in which the natural-join operations on the three relations should be executed. In the preceding case, there are two possibilities:

$$(instructor \bowtie teaches) \bowtie course \quad instructor \bowtie (teaches \bowtie course)$$

We did not specify which expression we intended, because the two are equivalent. That is, the natural join is **associative**.

<i>Name</i>	<i>title</i>
Brandt	Game Design
Brandt	Image Processing
Katz	Image Processing
Katz	Intro. to Computer Science
Srinivasan	Intro. to Computer Science
Srinivasan	Robotics
Srinivasan	Database System Concepts

Figure 2.24 Result of

$\sigma_{(name=title)}(\pi_{(name)}(\sigma_{(name=title)}(r \bowtie s)))$

The *theta join* operation is a variant of the natural-join operation that allows us to combine a selection and a Cartesian product into a single operation. Consider relations $r(R)$ and $s(S)$, and let θ be a predicate on attributes in the schema $R \cup S$. The **theta join** operation $r \bowtie_{\theta} s$ is defined as follows:

$$r \bowtie_{\theta} s = \sigma_{\theta}(r \times s)$$

2.5.8.3 The Assignment Operation

It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables. The **assignment** operation, denoted by $\leftarrow$, works like assignment in a programming language. To illustrate this operation, consider the definition of the natural-join operation. We could write $r \bowtie s$ as:

$$temp1 \leftarrow r \times s$$

$$temp2 \leftarrow \sigma_{(r.A_1=s.A_1 \wedge r.A_2=s.A_2 \wedge \dots \wedge r.A_n=s.A_n)}(temp1)$$

$$result = \pi_{R \cup S}(temp2)$$

The evaluation of an assignment does not result in any relation being displayed to the user. Rather, the result of the expression to the right of the $\leftarrow$ is assigned to the relation variable on the left of the $\leftarrow$. This relation variable may be used in subsequent expressions.

With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed

as the result of the query. For relational-algebra queries, assignment must always be made to a temporary relation variable. Assignments to permanent relations constitute a database modification. Note that the assignment operation does not provide any additional power to the algebra. It is, however, a convenient way to express complex queries.

2.5.8.4 Outer join Operations

The **outer-join** operation is an extension of the join operation to deal with missing information. Suppose that there is some instructor who teaches no courses. Then

<i>ID</i>	<i>name</i>	<i>deptname</i>	<i>salary</i>	<i>courseid</i>	<i>secid</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
33456	Gold	Physics	87000	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
58583	Califieri	History	62000	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>
76543	Singh	Finance	80000	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000	EE-181	1	Spring	2009

Figure 2.25 Result of instructor HX teaches.

the tuple in the *instructor* relation (2.11) for that particular instructor would not satisfy the condition of a natural join with the *teaches* relation and that instructor's data would not appear in the result of the natural join, shown in Figure 2.22. For example, instructors Califieri, Gold, and Singh do not appear in the result of the natural join, since they do not teach any course.

More generally, some tuples in either or both of the relations being joined may be “lost” in this way. The **outer join** operation works in a manner similar to the natural join operation we have already studied, but preserves those tuples that would be lost in an join by creating tuples in the result containing null values.

We can use the *outer-join* operation to avoid this loss of information. There are actually three forms of the operation: *left outer join*, denoted $\bowtie$; *right outer join*, denoted $\bowtie$; and *full outer join*, denoted $\bowtie$. All three forms of outer join compute the join, and add extra tuples to the result of the join. For example, the results of the expression *instructor* $\bowtie$ *teaches* and *teaches* $\bowtie$ *instructor* appear in Figures 2.25 and 2.26, respectively.

The **left outer join** ($\bowtie$) takes all tuples in the left relation that did not match with any tuple in the right relation, pads the tuples with null values for all other attributes from the right relation, and adds them to the result of the natural join. In Figure 6.17, tuple (58583, Califieri, History, 62000, *null*, *null*, *null*, *null*), is such a tuple. All information from the left relation is present in the result of the left outer join.

<i>ID</i>	<i>courseid</i>	<i>secid</i>	<i>semester</i>	<i>year</i>	<i>name</i>	<i>deptname</i>	<i>salary</i>
10101	CS-101	1	Fall	2009	Srinivasan	Comp. Sci.	65000
10101	CS-315	1	Spring	2010	Srinivasan	Comp. Sci.	65000
10101	CS-347	1	Fall	2009	Srinivasan	Comp. Sci.	65000
12121	FIN-201	1	Spring	2010	Wu	Finance	90000
15151	MU-199	1	Spring	2010	Mozart	Music	40000
22222	PHY-101	1	Fall	2009	Einstein	Physics	95000
32343	HIS-351	1	Spring	2010	El Said	History	60000
33456	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	Gold	Physics	87000
45565	CS-101	1	Spring	2010	Katz	Comp. Sci.	75000
45565	CS-319	1	Spring	2010	Katz	Comp. Sci.	75000
58583	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	Califieri	History	62000

76543	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	Singh	Finance	80000
76766	BIO-101	1	Summer	2009	Crick	Biology	72000
76766	BIO-301	1	Summer	2010	Crick	Biology	72000
83821	CS-190	1	Spring	2009	Brandt	Comp. Sci.	92000
83821	CS-190	2	Spring	2009	Brandt	Comp. Sci.	92000
83821	CS-319	2	Spring	2010	Brandt	Comp. Sci.	92000
98345	EE-181	1	Spring	2009	Kim	Elec. Eng.	80000

Figure 2.26 Result of teaches $\hat{\text{instructor}}$.

The **right outer join** (IXC) is symmetric with the left outer join: It pads tuples from the right relation that did not match any from the left relation with nulls and adds them to the result of the natural join. In Figure 2.26, tuple (58583, *null*, *null*, *null*, *null*, Califieri, History, 62000), is such a tuple. Thus, all information from the right relation is present in the result of the right outer join.

The **full outer join**(xE) does both the left and right outer join operations, padding tuples from the left relation that did not match any from the right relation, as well as tuples from the right relation that did not match any from the left relation, and adding them to the result of the join.

It is interesting to note that the outer-join operations can be expressed by the basic relational-algebra operations. For instance, the left outer join operation, $r \bowtie_{\text{L}} s$, can be written as:

$$(r \bowtie s) \cup (r - \pi_{R(r \bowtie s))} \times \{(null, \dots, null)\}$$

where the constant relation $\{(null, \dots, null)\}$ is on the schema $S - R$.

2.6 Null Values

Null values present special problems in relational operations, including arithmetic operations, comparison operations, and set operations.

The result of an arithmetic expression (involving, for example $+$, $-$, $*$, or $/$) is null if any of the input values is null. For example, if a query has an expression $r.A + 5$, and $r.A$ is null for a particular tuple, then the expression result must also be null for that tuple.

Comparisons involving nulls are more of a problem. For example, consider the comparison “ $1 < \text{null}$ ”. It would be wrong to say this is true since we do not know what the null value represents. But it would likewise be wrong to claim this expression is false; if we did, “**not** ($1 < \text{null}$)” would evaluate to true, which does not make sense. SQL therefore treats as **unknown** the result of any comparison involving a *null* value (other than predicates **is null** and **is not null**, which are described later in this section). This creates a third logical value in addition to *true* and *false*.

Since the predicate in a **where** clause can involve Boolean operations such as **and**, **or**, and **not** on the results of comparisons, the definitions of the Boolean operations are extended to deal with the value **unknown**.

- **and**: The result of *true and unknown* is *unknown*, *false and unknown* is *false*, while *unknown and unknown* is *unknown*.
- **or**: The result of *true or unknown* is *true*, *false or unknown* is *unknown*, while *unknown or unknown* is *unknown*.
- **not**: The result of **not** *unknown* is *unknown*.

You can verify that if $r.A$ is null, then “ $1 < r.A$ ” as well as “**not** ($1 < r.A$)” evaluate to unknown.

If the **where** clause predicate evaluates to either **false** or **unknown** for a tuple, that tuple is not added to the result.

SQL uses the special keyword **null** in a predicate to test for a null value. Thus, to find all instructors who appear in the *instructor* relation with null values for *salary*, we write:

select name from instructor where salary is null;

The predicate **is not null** succeeds if the value on which it is applied is not null.

Some implementations of SQL also allow us to test whether the result of a comparison is unknown, rather than true or false, by using the clauses **is unknown** and **is not unknown**.

When a query uses the **select distinct** clause, duplicate tuples must be eliminated. For this purpose, when comparing values of corresponding attributes from two tuples, the values are treated as identical if either both are non-null and equal in value, or both are null. Thus two copies of a tuple, such as $\{('A', \text{null}), ('A', \text{null})\}$, are treated as being identical, even if some of the attributes have a null value. Using the **distinct** clause then retains only one copy of such identical tuples. Note that the treatment of null above is different from the way nulls are treated in predicates, where a comparison “ $\text{null}=\text{null}$ ” would return unknown, rather than true.

The above approach of treating tuples as identical if they have the same values for all attributes, even if some of the values are null, is also used for the set operations union, intersection and except.

2.7 Summary

- The **relational data model** is based on a collection of tables. The user of the database system may query these tables, insert new tuples, delete tuples, and update (modify) tuples. There are several languages for expressing these operations.
- The **schema** of a relation refers to its logical design, while an **instance** of the relation refers to its contents at a point in time. The schema of a database and an instance of a database are similarly defined. The schema of a relation includes its attributes, and optionally the types of the attributes and constraints on the relation such as primary and foreign key constraints.
- A **superkey** of a relation is a set of one or more attributes whose values are guaranteed to identify tuples in the relation uniquely. A candidate key is a minimal superkey, that is, a set of attributes that forms a superkey, but none of whose subsets is a superkey. One of the candidate keys of a relation is chosen as its **primary key**.

The **relational algebra** provides a set of operations that take one or more relations as input and return a relation as an output. Practical query languages such as SQL are based on the relational algebra, but add a number of useful syntactic features

2.8 Question for Discussion

- 1) Describe the difference in meaning between the terms relation and relation schema.
- 2) List two reasons why null values might be introduced into the database
- 3) Explain about the Cartesian product operation

2.9 Suggested Reading

Recommended Text

A.Silberchatz, H.F Korth and S.Sudharshan, 2006, Database System Concepts, 5th Edition, Tata McGraw Hill, New Delhi.

Reference Books

J.D Ullaman, 1988, Principles of Database Systems, Galgotia Publishers, New Delhi.

Lesson - 3

STRUCTURE QUERY LANGUAGE

CONTENTS

- 3.0 Aims and Objectives**
- 3.1 Introduction**
- 3.2 Overview of SQL**
- 3.3 Data Definition**
- 3.4 Basic Structures of SQL Queries**
- 3.5 Set Operations**
- 3.6 Views**
- 3.7 Summary**
- 3.8 Questions for Discussion**
- 3.9 Suggested Reading**

3.0 Aims and Objectives

After studying this lesson you should be able to:

- Define SQL Language
- Identify the different types SQL Commands
- Describe about the views and set operation

3.1 Introduction

There are a number of database query languages in use, either commercially or experimentally. In this chapter, we study the most widely used query language, SQL.

Although we refer to the SQL language as a “query language,” it can do much more than just query a database. It can define the structure of the data, modify data in the database, and specify security constraints.

It is not our intention to provide a complete users' guide for SQL. Rather, we present SQL's fundamental constructs and concepts. Individual implementations of SQL may differ in details, or may support only a subset of the full language.

3.2 Overview of the SQL Query Language

IBM developed the original version of SQL, originally called Sequel, as part of the System R project in the early 1970s. The Sequel language has evolved since then, and its name has changed to SQL (Structured Query Language). Many products now support the SQL language. SQL has clearly established itself as *the* standard relational database language.

In 1986, the American National Standards Institute (ANSI) and the International Organization for Standardization (ISO) published an SQL standard, called SQL-86. ANSI published an extended standard for SQL, SQL-89, in 1989. The next version of the standard was SQL-92 standard, followed by SQL:1999, SQL:2003, SQL:2006, and most recently SQL:2008. The bibliographic notes provide references to these standards.

The SQL language has several parts:

- **Data-definition language** (DDL). The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schemas.
- **Data-manipulation language** (DML). The SQL DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.
- **Integrity**. The SQL DDL includes commands for specifying integrity constraints that the data stored in the database must satisfy. Updates that violate integrity constraints are disallowed.
- **View definition**. The SQL DDL includes commands for defining views.
- **Transaction control**. SQL includes commands for specifying the beginning and ending of transactions.
- **Embedded SQL** and **dynamic SQL**. Embedded and dynamic SQL define how SQL statements can be embedded within general-purpose programming languages, such as C, C++, and Java.
- **Authorization**. The SQL DDL includes commands for specifying access rights to relations and views.

In this chapter, we present a survey of basic DML and the DDL features of SQL. Features described here have been part of the SQL standard since SQL-92.

In Chapter 4, we provide a more detailed coverage of the SQL query language, including (a) various join expressions; (b) views; (c) transactions; (d) integrity constraints; (e) type system; and (f) authorization.

In Chapter 5, we cover more advanced features of the SQL language, including (a) mechanisms to allow accessing SQL from a programming language; (b) SQL functions and procedures; (c) triggers; (d) recursive queries; (e) advanced aggregation features; and (f) several features designed for data analysis, which were introduced in SQL:1999, and subsequent versions of SQL. Later, in Chapter 22, we outline object-oriented extensions to SQL, which were introduced in SQL:1999.

Although most SQL implementations support the standard features we describe here, you should be aware that there are differences between implementations. Most implementations support some nonstandard features, while omitting support for some of the more advanced features. In case you find that some language features described here do not work on the database system that you use, consult the user manuals for your database system to find exactly what features it supports.

3.3 SQL Data Definition

The set of relations in a database must be specified to the system by means of a data-definition language (DDL). The SQL DDL allows specification of not only a set of relations, but also information about each relation, including:

- The schema for each relation.
- The types of values associated with each attribute.
- The integrity constraints. The set of indices to be maintained for each relation.
- The security and authorization information for each relation.
- The physical storage structure of each relation on disk.

3.3.1 Basic Types

The SQL standard supports a variety of built-in types, including:

- **char(*n*)**: A fixed-length character string with user-specified length *n*. The full form, **character**, can be used instead.
- **varchar(*n*)**: A variable-length character string with user-specified maximum length *n*. The full form, **character varying**, is equivalent.

- **int**: An integer (a finite subset of the integers that is machine dependent). The full form, **integer**, is equivalent.
- **smallint**: A small integer (a machine-dependent subset of the integer type).
- **numeric(p,d)**: A fixed-point number with user-specified precision. The number consists of p digits (plus a sign), and d of the p digits are to the right of the decimal point. Thus, **numeric**(3,1) allows 44.5 to be stored exactly, but neither 444.5 or 0.32 can be stored exactly in a field of this type.
- **real, double precision**: Floating-point and double-precision floating-point numbers with machine-dependent precision.
- **float(n)**: A floating-point number, with precision of at least n digits.

Each type may include a special value called the **null** value. A null value indicates an absent value that may exist but be unknown or that may not exist at all. In certain cases, we may wish to prohibit null values from being entered, as we shall see shortly.

The **char** data type stores fixed length strings. Consider, for example, an attribute A of type **char**(10). If we store a string “Av i “ in this attribute, 7 spaces are appended to the string to make it 10 characters long. In contrast, if attribute B were of type **varchar**(10), and we store “Av i “ in attribute B , no spaces would be added. When comparing two values of type **char**, if they are of different lengths extra spaces are automatically added to the shorter one to make them the same size, before comparison.

When comparing a **char** type with a **varchar** type, one may expect extra spaces to be added to the **varchar** type to make the lengths equal, before comparison; however, this may or may not be done, depending on the database system. As a result, even if the same value “Av i “ is stored in the attributes A and B above, a comparison $A=B$ may return false. We recommend you always use the **varchar** type instead of the **char** type to avoid these problems.

SQL also provides the **nvarchar** type to store multilingual data using the Unicode representation. However, many databases allow Unicode (in the UTF-8 representation) to be stored even in **varchar** types.

3.3.2 Basic Schema Definition

We define an SQL relation by using the **create table** command. The following command creates a relation *department* in the database.

create table *department*

(*deptname* **varchar** (20), *building* **varchar** (15), *budget* **numeric** (12,2),
primary key (*deptname*));

The relation created above has three attributes, *deptname*, which is a character string of maximum length 20, *building*, which is a character string of maximum length 15, and *budget*, which is a number with 12 digits in total, 2 of which are after the decimal point. The **create table** command also specifies that the *dept name* attribute is the primary key of the *department* relation. The general form of the **create table** command is:

create table *r*(*A1* *D1*, *A2* *D2*, *An* *Dn*, (integrity-constraint₁}, (integrity-constraint_{*k*}));

where *r* is the name of the relation, each *A* is the name of an attribute in the schema of relation *r*, and *Di* is the domain of attribute *A_i*; that is, *Di* specifies the type of attribute *A_i* along with optional constraints that restrict the set of allowed values for *A_i*.

The semicolon shown at the end of the **create table** statements, as well as at the end of other SQL statements later in this chapter, is optional in many SQL implementations.

SQL supports a number of different integrity constraints. In this section, we discuss only a few of them:

- **primary key** (*A_{j1}*, *A_{j2}*, ..., *A_{jm}*): The **primary-key** specification says that attributes *A_{j1}*, *A_{j2}*, ..., *A_{jm}* form the primary key for the relation. The primary-key attributes are required to be *nonnull* and *unique*; that is, no tuple can have a null value for a primary-key attribute, and no two tuples in the relation can be equal on all the primary-key attributes. Although the primary-key specification is optional, it is generally a good idea to specify a primary key for each relation.
- **foreign key** (*A_{k1}*, *A_{k2}*, ..., *A_{kn}*) **references** *s*: The **foreign key** specification says that the values of attributes (*A₁*, *A_{k2}*, ..., *A_{kn}*) for any tuple in the relation must correspond to values of the primary key attributes of some tuple in relations.

Figure 3.1 presents a partial SQL DDL definition of the university database we use in the text. The definition of the *course* table has a declaration “**foreign key** (*deptname*) **references** *department*”. This foreign-key declaration specifies that for each course tuple, the department name specified in the tuple must exist in the primary key attribute (*deptname*) of the *department* relation. Without this constraint, it is possible for a course to specify a nonexistent department name. Figure 3.1 also shows foreign key constraints on tables *section*, *instructor* and *teaches*.

- **not null:** The **not null** constraint on an attribute specifies that the null value is not allowed for that attribute; in other words, the constraint excludes the null value from the domain of that attribute. For example, in Figure 3.1, the **not null** constraint on the *name* attribute of the *instructor* relation ensures that the name of an instructor cannot be null.

More details on the foreign-key constraint, as well as on other integrity constraints that the **create table** command may include, are provided later, Section.

SQL prevents any update to the database that violates an integrity constraint. For example, if a newly inserted or modified tuple in a relation has null values for any primary-key attribute, or if the tuple has the same value on the primary-key attributes as does another tuple in the relation, SQL flags an error and prevents the update. Similarly, an insertion of a *course* tuple with a *deptname* value that does not appear in the *department* relation would violate the foreign-key constraint on *course*, and SQL prevents such an insertion from taking place.

A newly created relation is empty initially. We can use the **insert** command to load data into the relation. For example, if we wish to insert the fact that there is an instructor named Smith in the Biology department with *instructorid* 10211 and a salary of \$66,000, we write:

```
insert into instructor
values (10211, 'Smith', 'Biology', 66000);
```

The values are specified in the *order* in which the corresponding attributes are listed in the relation schema. The insert command has a number of useful features.

We can use the **delete** command to delete tuples from a relation. The command

```
delete from student;
create table department
(deptname   varchar (20),
building   varchar (15),
budget     numeric (12,2),
primary key (deptname));
create table course
(courseid   varchar (7),
```

```

title  varchar (50),
deptname  varchar (20),
credits  numeric (2,0),
primary key (courseid), foreign key (deptname) references department);
create table instructor
(ID  varchar (5),
namevarchar (20) not null,
deptname  varchar (20),
salary  numeric (8,2),
primary key (ID),
foreign key (deptname) references department);
create table section
(courseid  varchar (8),
secid varchar (8),
semester  varchar (6),
year  numeric (4,0),
building  varchar (15),
room-number varchar (7),
timeslotid  varchar (4),
primary key (courseid, secid, semester, year),
foreign key (courseid) references course);
create table teaches
(ID  varchar (5),
courseid  varchar (8),
secid varchar (8),
semester  varchar (6),
year  numeric (4,0),
primary key (ID, courseid, secid, semester, year),
foreign key (courseid, secid, semester, year) references section,
foreign key (ID) references instructor);

```

Figure 3.1 SQL data definition for part of the university database.

would delete all tuples from the *student* relation. Other forms of the delete command allow specific tuples to be deleted.

To remove a relation from an SQL database, we use the **drop table** command. The **drop table** command deletes all information about the dropped relation from the database. The command

drop table r ;

is a more drastic action than

delete from r ;

The latter retains relation r , but deletes all tuples in r . The former deletes not only all tuples of r , but also the schema for r . After r is dropped, no tuples can be inserted into r unless it is re-created with the **create table** command.

We use the **alter table** command to add attributes to an existing relation. All tuples in the relation are assigned *null* as the value for the new attribute. The form of the **alter table** command is

alter table r add A D ;

where r is the name of an existing relation, A is the name of the attribute to be added, and D is the type of the added attribute. We can drop attributes from a relation by the command

alter table r drop A ;

where r is the name of an existing relation, and A is the name of an attribute of the relation. Many database systems do not support dropping of attributes, although they will allow an entire table to be dropped

3.3 Basic Structure of SQL Queries

The basic structure of an SQL query consists of three clauses: **select**, **from**, and **where**. The query takes as its input the relations listed in the **from** clause, operates on them as specified in the **where** and **select** clauses, and then produces a relation as the result.

SQL query can contain three types of clauses, the **select** clause, the **from** clause, and the **where** clause. The role of each clause is as follows:

- The **select** clause is used to list the attributes desired in the result of a query.
- The **from** clause is a list of the relations to be accessed in the evaluation of the query.
- The **where** clause is a predicate involving attributes of the relation in the **from** clause.

A typical SQL query has the form

select $A_1, A_2, \dots, A_n$ **from** $r_1, r_2, \dots, r_m$ **where** P ;

Each A_i represents an attribute, and each r_i a relation. P is a predicate. If the **where** clause is omitted, the predicate P is **true**.

3.3.1 The select Clause

Let us consider a simple query using our university example, “Find the names of all instructors.” Instructor names are found in the *instructor* relation, so we

<i>name</i>
Srinivasan
Wu
Mozart
Einstein
El Said
Gold
Katz
Califieri
Singh
Crick
Brandt
Kim

Figure 3.2 Result of “*select name from instructor*”.

put that relation in the from clause. The instructor’s name appears in the *name* attribute, so we put that in the **select** clause.

select *name*
from *instructor*;

The result is a relation consisting of a single attribute with the heading *name*. If the *instructor* relation is as shown in Figure 2.1, then the relation that results from the preceding query is shown in Figure 3.2.

Now consider another query, “Find the department names of all instructors,” which can be written as:

```
select deptname from instructor;
```

Since more than one instructor can belong to a department, a department name could appear more than once in the *instructor* relation. The result of the above query is a relation containing the department names, shown in Figure 3.3.

In the formal, mathematical definition of the relational model, a relation is a set. Thus, duplicate tuples would never appear in relations. In practice, duplicate elimination is time-consuming. Therefore, SQL allows duplicates in relations as well as in the results of SQL expressions. Thus, the preceding SQL query lists each department name once for every tuple in which it appears in the *instructor* relation.

In those cases where we want to force the elimination of duplicates, we insert the keyword **distinct** after **select**. We can rewrite the preceding query as:

```
select distinct deptname  
  
from instructor;
```

<i>dept_name</i>
Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.

Figure 3.3 Result of “*select deptname from instructor*”.

if we want duplicates removed. The result of the above query would contain each department name at most once.

SQL allows us to use the keyword **all** to specify explicitly that duplicates are not removed:

```
select all deptname  
from instructor;
```

Since duplicate retention is the default, we shall not use **all** in our examples. To ensure the elimination of duplicates in the results of our example queries, we shall use **distinct** whenever it is necessary.

The **select** clause may also contain arithmetic expressions involving the operators **+**, **—**, *****, and **/** operating on constants or attributes of tuples. For example, the query:

```
select ID, name, deptname, salary * 1.1 from instructor;
```

returns a relation that is the same as the *instructor* relation, except that the attribute *salary* is multiplied by 1.1. This shows what would result if we gave a 10% raise to each instructor; note, however, that it does not result in any change to the *instructor* relation.

SQL also provides special data types, such as various forms of the *date* type, and allows several arithmetic functions to operate on these types.

3.3.2 The Where Clause

The **where** clause allows us to select only those rows in the result relation of the **from** clause that satisfy a specified predicate. Consider the query “Find the names of all instructors in the Computer Science department who have salary greater than \$70,000.” This query can be written in SQL as:

name
Katz
Brandt

Figure 3.4 Result of “Find the names of all instructors in the Computer Science department who have salary greater than \$70,000.”

```
select name  
from instructor  
where deptname = 'Comp. Sci.' and salary > 70000;
```

If the *instructor* relation is as shown in Figure 2.1, then the relation that results from the preceding query is shown in Figure 3.4.

SQL allows the use of the logical connectives **and**, **or**, and **not** in the **where** clause. The operands of the logical connectives can be expressions involving the comparison operators **<**, **<=**, **>**, **>=**, **=**, and **<>**. SQL allows us to use the comparison operators to compare strings and arithmetic expressions, as well as special types, such as date types.

3.3.3 The from Clause

The **from** clause by itself defines a Cartesian product of the relations listed in the clause. It is defined formally in terms of set theory, but is perhaps best understood as an iterative process that generates tuples for the result relation of the **from** clause.

for each tuple t_1 **in** relation r_1

for each tuple t_2 **in** relation r_2

for each tuple t_m **in** relation r_m

Concatenate $t_1, t_2, \dots, t_m$ into a single tuple t Add t into the result relation

The result relation has all attributes from all the relations in the **from** clause. Since the same attribute name may appear in both r_i and r_j , as we saw earlier, we prefix the the name of the relation from which the attribute originally came, before the attribute name.

For example, the relation schema for the Cartesian product of relations *instructor* and *teaches* is:

(instructor.ID, instructor.name, instructor.deptname, instructor.salary teaches.ID, teaches.courseid, teaches.secid, teaches.semester, teaches.year)

With this schema, we can distinguish *instructor.ID* from *teaches.ID*. For those attributes that appear in only one of the two schemas, we shall usually drop the relation-name prefix. This simplification does not lead to any ambiguity. We can then write the relation schema as:

(instructor.ID, name, deptname, salary teaches.ID, courseid, secid, semester, year)

3.3.4 The Rename Operation

Consider again the query that we used earlier:

select *name, courseid*

from *instructor, teaches*

where *instructor.ID = teaches.ID;*

The result of this query is a relation with the following attributes:

name, courseid

The names of the attributes in the result are derived from the names of the attributes in the relations in the **from** clause.

We cannot, however, always derive names in this way, for several reasons: First, two relations in the **from** clause may have attributes with the same name, in which case an attribute name is duplicated in the result. Second, if we used an arithmetic expression in the **select** clause, the resultant attribute does not have a name. Third, even if an attribute name can be derived from the base relations as in the preceding example, we may want to change the attribute name in the result. Hence, SQL provides a way of renaming the attributes of a result relation. It uses the **as** clause, taking the form:

old-name as new-name

The **as** clause can appear in both the **select** and **from** clauses.⁴

For example, if we want the attribute name *name* to be replaced with the name *instructorname*, we can rewrite the preceding query as:

```
select name as instructor name, courseid
from instructor, teaches
where instructor.ID= teaches.ID;
```

3.4 Set Operations

The SQL operations **union**, **intersect**, and **except** operate on relations and correspond to the mathematical set-theory operations **U**, **n**, and **—**. We shall now construct queries involving the **union**, **intersect**, and **except** operations over two sets.

- The set of all courses taught in the Fall 2009 semester:

```
select courseid
from section
where semester = 'Fall' and year= 2009;
```

- The set of all courses taught in the Spring 2010 semester:

```
select courseid
from section
where semester = 'Spring' and year= 2010;
```

In our discussion that follows, we shall refer to the relations obtained as the result of the preceding queries as *c1* and *c2*, respectively, and show the results when these queries are run on the *section* relation of Figure 2.6 in Figures 3.5. Observe that *c2* contains two tuples corresponding to *courseid* CS-319, since two sections of the course have been offered in Spring 2010.

<i>course_id</i>
CS-101
CS-315
CS-319
CS-319
FIN-201
HIS-351
MU-199

Figure 3.5 *The c2 relation, listing courses taught in Spring 2010.*

3.5.1 The Union Operation

To find the set of all courses taught either in Fall 2009 or in Spring 2010, or both, we write:⁶

```
(select courseid from section
where semester = 'Fall' and year= 2009) union
(select courseid from section where semester = 'Spring' and year= 2010);
```

The **union** operation automatically eliminates duplicates, unlike the **select** clause. Thus, using the *section* relation of Figure 2.6, where two sections of CS-319 are offered in Spring 2010, and a section of CS-101 is offered in the Fall 2009 as well as in the Fall 2010 semester, CS-101 and CS-319 appear only once in the result, shown in Figure 3.6.

If we want to retain all duplicates, we must write **union all** in place of **union**:

```
(select courseid from section
where semester = 'Fall' and year= 2009) union all (select courseid from section
where semester = 'Spring' and year= 2010);
```

The number of duplicate tuples in the result is equal to the total number of duplicates that appear in both *c1* and *c2*. So, in the above query, each of CS-319 and CS-101 would be listed twice. As a further example, if it were the case that 4 sections of ECE-101 were taught in the Fall 2009 semester and 2 sections of ECE-101

<i>course_id</i>
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

Figure 3.6 *The result relation for c1 union c2.*

were taught in the Fall 2010 semester, then there would be 6 tuples with ECE-101 in the result.

3.5.2 The Intersect Operation

To find the set of all courses taught in the Fall 2009 as well as in Spring 2010 we write:

```
(select courseid
from section
where semester = 'Fall' and year= 2009)
intersect
(select courseid
from section
where semester = 'Spring' and year= 2010);
```

The result relation, shown in Figure 3.7, contains only one tuple with CS-101. The **intersect** operation automatically eliminates duplicates. For example, if it were the case that 4 sections of ECE-101 were taught in the Fall 2009 semester and 2 sections of ECE-101 were taught in the Spring 2010 semester, then there would be only 1 tuple with ECE-101 in the result.

If we want to retain all duplicates, we must write **intersect all** in place of **intersect**:

<i>courseid</i>
CS-101

Figure 3.7 *The result relation for c1 intersect c2.*

```

(select courseid
from section
where semester = 'Fall' and year= 2009)
intersect all
(select courseid
from section
where semester = 'Spring' and year= 2010);

```

The number of duplicate tuples that appear in the result is equal to the minimum number of duplicates in both *c1* and *c2*. For example, if 4 sections of ECE-101 were taught in the Fall 2009 semester and 2 sections of ECE-101 were taught in the Spring 2010 semester, then there would be 2 tuples with ECE-101 in the result.

3.5.3 The Except Operation

To find all courses taught in the Fall 2009 semester but not in the Spring 2010 semester, we write:

```

(select courseid
from section
where semester = 'Fall' and year= 2009)
except
(select courseid
from section
where semester = 'Spring' and year= 2010);

```

The result of this query is shown in Figure 3.8. Note that this is exactly relation *c1* of Figure 3.9 except that the tuple for CS-101 does not appear. The **except** operation⁷ outputs all tuples from its first input that do not occur in the second input; that is, it performs set difference. The operation automatically eliminates duplicates in the inputs before performing set difference. For example, if 4 sections of ECE-101 were taught in the Fall 2009 semester and 2 sections of ECE-101 were taught in the Spring 2010 semester, the result of the **except** operation would not have any copy of ECE-101.

If we want to retain duplicates, we must write **except all** in place of **except**:

```

(select courseid
from section

```

```

where semester = 'Fall' and year= 2009)
except all
(select courseid
from section
where semester = 'Spring' and year= 2010);
```

<i>course_id</i>
CS-347
PHY-101

Figure 3.8 *The result relation for c1 except c2.*

The number of duplicate copies of a tuple in the result is equal to the number of duplicate copies in *c1* minus the number of duplicate copies in *c2*, provided that the difference is positive. Thus, if 4 sections of ECE-101 were taught in the Fall

2009 semester and 2 sections of ECE-101 were taught in Spring 2010, then there are 2 tuples with ECE-101 in the result. If, however, there were two or fewer sections of ECE-101 in the the Fall 2009 semester, and two sections of ECE-101 in the Spring

2010 semester, there is no tuple with ECE-101 in the result.

3.6 Views

In our examples up to this point, we have operated at the logical-model level. That is, we have assumed that the relations in the collection we are given are the actual relations stored in the database.

It is not desirable for all users to see the entire logical model. Security con-siderations may require that certain data be hidden from users. Consider a clerk who needs to know an instructor's ID, name and department name, but does not have authorization to see the instructor's salary amount. This person should see a relation described in SQL, by:

```
select ID, name, deptname from instructor;
```

Aside from security concerns, we may wish to create a personalized collection of relations that is better matched to a certain user's intuition than is the logical model. We may want to have a list of all course sections offered by the Physics department in the Fall 2009 semester, with the building and room number of each section. The relation that we would create for obtaining such a list is:


```

select course.courseid, secid, building, roomnumber
from course, section
where course.courseid = section.courseid
       and course.deptname = 'Physics'
       and section.semester = 'Fall'
       and section.year = '2009';

```

It is possible to compute and store the results of the above queries and then make the stored relations available to users. However, if we did so, and the underlying data in the relations *instructor*, *course*, or *section* changes, the stored query results would then no longer match the result of reexecuting the query on the relations. In general, it is a bad idea to compute and store query results such as those in the above examples (although there are some exceptions, which we study later).

Instead, SQL allows a “virtual relation” to be defined by a query, and the relation conceptually contains the result of the query. The virtual relation is not precomputed and stored, but instead is computed by executing the query when-ever the virtual relation is used.

Any such relation that is not part of the logical model, but is made visible to a user as a virtual relation, is called a **view**. It is possible to support a large number of views on top of any given set of actual relations.

3.6.1 View Definition

We define a view in SQL by using the **create view** command. To define a view, we must give the view a name and must state the query that computes the view. The form of the **create view** command is:

```

create view v as <query expression>;

```

where < query expression > is any legal query expression. The view name is represented by *v*.

Consider again the clerk who needs to access all data in the *instructor* relation, except *salary*. The clerk should not be authorized to access the *instructor* relation (we see later, in Section, how authorizations can be specified). Instead, a view relation *faculty* can be made available to the clerk, with the view defined as follows:

```
create view
faculty as
select ID,
name,
deptname from
instructor;
```

As explained earlier, the view relation conceptually contains the tuples in the query result, but is not precomputed and stored. Instead, the database system stores the query expression associated with the view relation. Whenever the view relation is accessed, its tuples are created by computing the query result. Thus, the view relation is created whenever needed, on demand.

To create a view that lists all course sections offered by the Physics department in the Fall 2009 semester with the building and room number of each section, we write:

```
create view physics-fall2009 as
select course.courseid, secid, building, room number
from course, section
where course.courseid = section.courseid
and course.deptname = 'Physics'
and section.semester = 'Fall'
and section.year = '2009';
```

3.6.2 Using Views in SQL Queries

Once we have defined a view, we can use the view name to refer to the virtual relation that the view generates. Using the view *physics-fall2009*, we can find all Physics courses offered in the Fall 2009 semester in the Watson building by writing:

```
select courseid
from physics-
fall2009 where
building=
'Watson';
```

View names may appear in a query any place where a relation name may appear. The attribute names of a view can be specified explicitly as follows:

```
create view departmentstotaljsalary(deptname,  
    totalsalary) as select deptname, sum  
    (salary) from instructor group by deptname;
```

The preceding view gives for each department the sum of the salaries of all the instructors at that department. Since the expression **sum(salary)** does not have a name, the attribute name is specified explicitly in the view definition.

Intuitively, at any given time, the set of tuples in the view relation is the result of evaluation of the query expression that defines the view. Thus, if a view relation is computed and stored, it may become out of date if the relations used to define it are modified. To avoid this, views are usually implemented as follows. When we define a view, the database system stores the definition of the view itself, rather than the result of evaluation of the query expression that defines the view. Wherever a view relation appears in a query, it is replaced by the stored query expression. Thus, whenever we evaluate the query, the view relation is recomputed.

One view may be used in the expression defining another view. For example, we can define a view *physicsfallJ2009-watson* that lists the course ID and room number of all Physics courses offered in the Fall 2009 semester in the Watson building as follows:

```
create view physics-fall2009-watson as  
    select courseid, roomnumber  
    from physics fall2009  
    where building= 'Watson';
```

where *physicsfallJ2009-watson* is itself a view relation. This is equivalent to:

```
create view physics-fall-2009watson as  
    (select courseid, room number from (select  
        course.courseid, building, room number from  
        course, section  
  
        where course.courseid = section.courseid and  
        course.deptname = 'Physics' and section.semester = 'Fall' and  
        section.year = '2009') where building= 'Watson');
```

3.6.3 Materialized Views

Certain database systems allow view relations to be stored, but they make sure that, if the actual relations used in the view definition change, the view is kept up-to-date. Such views are called **materialized views**.

For example, consider the view *departmentstotalsalary*. If the above view is materialized, its results would be stored in the database. However, if an *instructor* tuple is added to or deleted from the *instructor* relation, the result of the query defining the view would change, and as a result the materialized view's contents must be updated. Similarly, if an instructor's salary is updated, the tuple in *departments totalsalary* corresponding to that instructor's department must be updated.

3.7 Summary

SQL is the most influential commercially marketed relational query language.

The SQL language has several parts:

- a Data-definition language (DDL)**, which provides commands for defining relation schemas, deleting relations, and modifying relation schemas.
- ^ Data-manipulation language (DML)**, which includes a query language and commands to insert tuples into, delete tuples from, and modify tuples in the database.
 - The SQL data-definition language is used to create relations with specified schemas. In addition to specifying the names and types of relation attributes, SQL also allows the specification of integrity constraints such as primary-key constraints and foreign-key constraints.
 - SQL includes a variety of language constructs for queries on the database. These include the **select**, **from**, and **where** clauses, and support for the natural join operation.
 - SQL also provides mechanisms to rename both attributes and relations, and to order query results by sorting on specified attributes.
 - SQL supports basic set operations on relations including **union**, **intersect**, and **except**, which correspond to the mathematical set-theory operations **U**, **n**, and **—**.

3.8 Questions for Discussion

1. Explain Basic structure of SQL Queries
2. Describe about the Set Operations
3. Discuss about the Views

3.9 Suggested Reading

Recommended Text

A.Silberchatz, H.F Korth and S.Sudharshan, 2006, Database System Concepts, 5th Edition, Tata McGraw Hill, New Delhi.

Reference Books

J.D Ullaman, 1988, Principles of Database Systems, Galgotia Publishers, New Delhi.

Lesson - 4

ADVANCED SQL AND RELATIONAL CALCULAS

CONTENTS

- 4.0 Aims and Objectives**
- 4.1 Introduction**
- 4.2 SQL Data types and Schemas**
- 4.3 Integrity Constraints**
- 4.4 Embedded SQL**
- 4.5 Recursive Queries**
- 4.6 The Tuple Relational Calculus**
- 4.7 The Domain Relational Calculus**
- 4.8 Summary**
- 4.9 Questions for Discussion**
- 4.10 Suggested Reading**

4.0 Aims and Objectives

After studying this lesson you should be able to:

- Describe the Data types in SQL
- Identify the different Integrity constraints
- Describe about the Embedded SQL
- Define the Tuple and Domain Relational Calculas

4.1 Introduction

In this chapter, we cover some of the more advanced features of SQL.¹ We address the issue of how to access SQL from a general-purpose programming language, which is very important for building applications that use a database to store and retrieve data. We describe how procedural code can be executed within the database, either by extending the SQL language to support procedural actions, or by allowing functions defined in procedural languages to be executed within the database.

4.2 SQL Data Types and Schemas

In Chapter 3, we covered a number of built-in data types supported in SQL, such as integer types, real types, and character types. There are additional built-in data types supported by SQL, which we describe below. We also describe how to create basic user-defined types in SQL.

4.2.1 Date and Time Types in SQL

In addition to the basic data types we introduced in Section 3.3, the SQL standard supports several data types relating to dates and times:

- **date**: A calendar date containing a (four-digit) year, month, and day of the month.
- **time**: The time of day, in hours, minutes, and seconds. A variant, **time(*p*)**, can be used to specify the number of fractional digits for seconds (the default being 0). It is also possible to store time-zone information along with the time by specifying **time with timezone**.
- **timestamp**: A combination of **date** and **time**. A variant, **timestamp(*p*)**, can be used to specify the number of fractional digits for seconds (the default here being 6). Time-zone information is also stored if **with timezone** is specified.

Date and time values can be specified like this:

date '2001-04-25'

time '09:30:00'

timestamp '2001-04-25 10:29:01.45'

Dates must be specified in the format year followed by month followed by day, as shown. The seconds field of **time** or **timestamp** can have a fractional part, as in the timestamp above.

We can use an expression of the form **cast e as t** to convert a character string (or string valued expression) *e* to the type *t*, where *t* is one of **date**, **time**, or **timestamp**. The string must be in the appropriate format as illustrated at the beginning of this paragraph. When required, time-zone information is inferred from the system settings.

To extract individual fields of a **date** or **time** value *d*, we can use **extract (field from d)**, where *field* can be one of **year**, **month**, **day**, **hour**, **minute**, or **second**. Time-zone information can be extracted using **timezonehour** and **timezone_minute**.

SQL defines several functions to get the current date and time. For example, **current-date** returns the current date, **current_time** returns the current time (with time zone), and **localtime** returns the current local time (without time zone). Timestamps (date plus time) are returned by **current_timestamp** (with time zone) and **localtimestamp** (local date and time without time zone).

SQL allows comparison operations on all the types listed here, and it allows both arithmetic and comparison operations on the various numeric types. SQL also provides a data type called **interval**, and it allows computations based on dates and times and on intervals. For example, if x and y are of type **date**, then $x - y$ is an interval whose value is the number of days from date x to date y . Similarly, adding or subtracting an interval to a date or time gives back a date or time, respectively.

4.2.2 User-Defined Types

SQL supports two forms of user-defined data types. The first form, which we cover here, is called **distinct types**. The other form, called **structured data types**, allows the creation of complex data types with nested record structures, arrays, and multisets. We do not cover structured data types in this chapter, but describe them later.

It is possible for several attributes to have the same data type. For example, the *name* attributes for student name and instructor name might have the same domain: the set of all person names. However, the domains of *budget* and *deptname* certainly ought to be distinct. It is perhaps less clear whether *name* and *deptname* should have the same domain. At the implementation level, both instructor names and department names are character strings. However, we would normally not consider the query “Find all instructors who have the same name as a department” to be a meaningful query. Thus, if we view the database at the conceptual, rather than the physical, level, *name* and *deptname* should have distinct domains.

More importantly, at a practical level, assigning an instructor’s name to a department name is probably a programming error; similarly, comparing a monetary value expressed in dollars directly with a monetary value expressed in pounds is also almost surely a programming error. A good type system should be able to detect such assignments or comparisons. To support such checks, SQL provides the notion of **distinct types**.

The **create type** clause can be used to define new types. For example, the statements:

```
create type Dollars as numeric(12,2) final;
create type Pounds as numeric(12,2) final;
```


define the user-defined types *Dollars* and *Pounds* to be decimal numbers with a total of 12 digits, two of which are placed after the decimal point. (The keyword **final** isn't really meaningful in this context but is required by the SQL:1999 standard for reasons we won't get into here; some implementations allow the **final** keyword to be omitted.) The newly created types can then be used, for example, as types of attributes of relations. For example, we can declare the *department* table as:

```
create table department
  (deptname  varchar (20),
   building  varchar (15),
   budget    Dollars);
```

An attempt to assign a value of type *Dollars* to a variable of type *Pounds* results in a compile-time error, although both are of the same numeric type. Such an assignment is likely to be due to a programmer error, where the programmer forgot about the differences in currency. Declaring different types for different currencies helps catch such errors.

As a result of strong type checking, the expression *(department.budget+20)* would not be accepted since the attribute and the integer constant 20 have different types. Values of one type can be *cast* (that is, converted) to another domain, as illustrated below:

```
cast (department.budget to numeric(12,2))
```

We could do addition on the numeric type, but to save the result back to an attribute of type *Dollars* we would have to use another cast expression to convert the type back to *Dollars*.

SQL provides **drop type** and **alter type** clauses to drop or modify types that have been created earlier.

Even before user-defined types were added to SQL (in SQL:1999), SQL had a similar but subtly different notion of **domain** (introduced in SQL-92), which can add integrity constraints to an underlying type. For example, we could define a domain *DDollars* as follows.

```
create domain DDollars as numeric(12,2) not null;
```

The domain *DDollars* can be used as an attribute type, just as we used the type *Dollars*. However, there are two significant differences between types and domains:

1. Domains can have constraints, such as **not null**, specified on them, and can have default values defined for variables of the domain type, whereas user-defined types

cannot have constraints or default values specified on them. User-defined types are designed to be used not just for specifying attribute types, but also in procedural extensions to SQL where it may not be possible to enforce constraints.

2. Domains are not strongly typed. As a result, values of one domain type can be assigned to values of another domain type as long as the underlying types are compatible.

4.2.3 Large-Object Types

Many current-generation database applications need to store attributes that can be large (of the order of many kilobytes), such as a photograph, or very large (of the order of many megabytes or even gigabytes), such as a high-resolution medical image or video clip. SQL therefore provides large-object data types for character data (**clob**) and binary data (**blob**). The letters “lob” in these data types stand for “Large Object.” For example, we may declare attributes

```
book-review
clob(10KB) image
blob(10MB) movie
blob(2GB)
```

For result tuples containing large objects (multiple megabytes to gigabytes), it is inefficient or impractical to retrieve an entire large object into memory. Instead, an application would usually use an SQL query to retrieve a “locator” for a large object and then use the locator to manipulate the object from the host language in which the application itself is written.

4.2.4 Schemas, Catalogs, and Environments

To understand the motivation for schemas and catalogs, consider how files are named in a file system. Early file systems were flat; that is, all files were stored in a single directory. Current file systems, of course, have a directory (or, syn-onymously, folder) structure, with files stored within subdirectories. To name a file uniquely, we must specify the full path name of the file, for example, `/users/avi/db-book/chapter3.tex`.

Like early file systems, early database systems also had a single name space for all relations. Users had to coordinate to make sure they did not try to use the same name for different relations. Contemporary database systems provide a three-level hierarchy for naming relations. The top level of the hierarchy consists of **catalogs**, each of which can

contain **schemas**. SQL objects such as relations and views are contained within a **schema**. (Some database implementations use the term “database” in place of the term catalog.)

In order to perform any actions on a database, a user (or a program) must first *connect* to the database. The user must provide the user name and usually, a password for verifying the identity of the user. Each user has a default catalog and schema, and the combination is unique to the user. When a user connects to a database system, the default catalog and schema are set up for the connection; this corresponds to the current directory being set to the user’s home directory when the user logs into an operating system.

To identify a relation uniquely, a three-part name may be used, for example,

catalogb.univschema.course

We may omit the catalog component, in which case the catalog part of the name is considered to be the default catalog for the connection. Thus if *catalogb* is the default catalog, we can use *univschema.course* to identify the same relation uniquely.

If a user wishes to access a relation that exists in a different schema than the default schema for that user, the name of the schema must be specified. However, if a relation is in the default schema for a particular user, then even the schema name may be omitted. Thus we can use just *course* if the default catalog is *catalogb* and the default schema is *univschema*.

With multiple catalogs and schemas available, different applications and different users can work independently without worrying about name clashes. Moreover, multiple versions of an application—one a production version, other test versions—can run on the same database system.

The default catalog and schema are part of an **SQL environment** that is set up for each connection. The environment additionally contains the user identifier (also referred to as the *authorization identifier*). All the usual SQL statements, including the DDL and DML statements, operate in the context of a schema.

We can create and drop schemas by means of **create schema** and **drop schema** statements. In most database systems, schemas are also created automatically when user accounts are created, with the schema name set to the user account name. The schema is created in either a default catalog, or a catalog specified in creating the user account. The newly created schema becomes the default schema for the user account.

Creation and dropping of catalogs is implementation dependent and not part of the SQL standard.

4.3 Integrity Constraints

Integrity constraints ensure that changes made to the database by authorized users do not result in a loss of data consistency. Thus, integrity constraints guard against accidental damage to the database. Examples of integrity constraints are:

- An instructor name cannot be *null*.
- No two instructors can have the same instructor ID.
- Every department name in the *course* relation must have a matching department name in the *department* relation.
- The budget of a department must be greater than \$0.00.

In general, an integrity constraint can be an arbitrary predicate pertaining to the database. However, arbitrary predicates may be costly to test. Thus, most database systems allow one to specify integrity constraints that can be tested with minimal overhead.

Integrity constraints are usually identified as part of the database schema design process, and declared as part of the **create table** command used to create relations. However, integrity constraints can also be added to an existing relation by using the command **alter table table-name add constraint**, where *constraint* can be any constraint on the relation. When such a command is executed, the system first ensures that the relation satisfies the specified constraint. If it does, the constraint is added to the relation; if not, the command is rejected.

4.3.1 Constraints on a Single Relation

We described in Section 3 how to define tables using the **create table** command. The **create table** command may also include integrity-constraint statements. In addition to the primary-key constraint, there are a number of other ones that can be included in the **create table** command. The allowed integrity constraints include

- **not null**
- **unique**
- **check(<predicate>)**

We cover each of these types of constraints in the following sections.

4.3.2 Not Null Constraint

As we discussed in Chapter 3, the null value is a member of all domains, and as a result is a legal value for every attribute in SQL by default. For certain attributes, however, null values may be inappropriate. Consider a tuple in the *student* relation where *name* is *null*. Such a tuple gives student information for an unknown student; thus, it does not contain useful information. Similarly, we would not want the department budget to be *null*. In cases such as this, we wish to forbid null values, and we can do so by restricting the domain of the attributes *name* and *budget* to exclude null values, by declaring it as follows:

```
name  varchar(20) not null budget
      numeric(12,2) not null
```

The **not null** specification prohibits the insertion of a null value for the attribute. Any database modification that would cause a null to be inserted in an attribute declared to be **not null** generates an error diagnostic.

There are many situations where we want to avoid null values. In particular, SQL prohibits null values in the primary key of a relation schema. Thus, in our university example, in the *department* relation, if the attribute *deptname* is declared

as the primary key for *department*, it cannot take a null value. As a result it would not need to be declared explicitly to be **not null**.

4.3.3 Unique Constraint

SQL also supports an integrity constraint:

```
unique (A, A, ..., A )
          j 1   j 2       j m
```

The **unique** specification says that attributes *A*, *A*, ..., *A* form a candidate

```
j 1   j 2       j m
```

key; that is, no two tuples in the relation can be equal on all the listed attributes. However, candidate key attributes are permitted to be *null* unless they have explicitly been declared to be **not null**. Recall that a null value does not equal any other value.

4.3.4 The check Clause

When applied to a relation declaration, the clause **check**(P) specifies a predicate P that must be satisfied by every tuple in a relation.

A common use of the **check** clause is to ensure that attribute values satisfy specified conditions, in effect creating a powerful type system. For instance, a clause **check** ($\text{budget} > 0$) in the **create table** command for relation *department* would ensure that the value of **budget** is nonnegative.

As another example, consider the following:

```
create table section
    (courseid      varchar (8),
     secid        varchar (8),
     semester     varchar (6),
     year         numeric (4,0),
     building     varchar (15),
     room number  varchar (7),
     timeslotid   varchar (4),
    primary key (courseid, secid, semester, year),
    check (semester in ('Fall', 'Winter', 'Spring', 'Summer')));
```

Here, we use the **check** clause to simulate an enumerated type, by specifying that *semester* must be one of 'Fall', 'Winter', 'Spring', or 'Summer'. Thus, the **check** clause permits attribute domains to be restricted in powerful ways that most programming-language type systems do not permit.

The predicate in the **check** clause can, according to the SQL standard, be an arbitrary predicate that can include a subquery. However, currently none of the widely used database products allows the predicate to contain a subquery.

4.3.5 Referential Integrity

Often, we wish to ensure that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation. This condition is called **referential integrity**.

Foreign keys can be specified as part of the SQL **create table** statement by using the **foreign key** clause, as we saw earlier in Section . We illustrate foreign-key declarations

by using the SQL DDL definition of part of our university database, shown in Figure 4.1. The definition of the *course* table has a declaration “**foreign key (deptlname) references department**”. This foreign-key declaration specifies that for each course tuple, the department name specified in the tuple must exist in the *department* relation. Without this constraint, it is possible for a course to specify a nonexistent department name.

More generally, let r_1 and r_2 be relations whose set of attributes are R_1 and R_2 , respectively, with primary keys K_1 and K_2 . We say that a subset a of R_2 is a **foreign key** referencing K_1 in relation r_1 if it is required that, for every tuple f_2 in r_2 , there must be a tuple t_1 in r_1 such that $t_1.K_1 = f_2.a$.

Requirements of this form are called **referential-integrity constraints**, or **subset dependencies**. The latter term arises because the preceding referential-integrity constraint can be stated as a requirement that the set of values on a in r_2 must be a subset of the values on K_1 in r_1 . Note that, for a referential-integrity constraint to make sense, a and K_1 must be compatible sets of attributes; that is, either a must be equal to K_1 , or they must contain the same number of attributes, and the types of corresponding attributes must be compatible (we assume here that a and K_1 are ordered). Unlike foreign-key constraints, in general a referential integrity constraint does not require K_1 to be a primary key of r_1 ; as a result, more than one tuple in r_1 can have the same value for attributes K_1 .

By default, in SQL a foreign key references the primary-key attributes of the referenced table. SQL also supports a version of the **references** clause where a list of attributes of the referenced relation can be specified explicitly. The specified list of attributes must, however, be declared as a candidate key of the referenced relation, using either a **primary key** constraint, or a **unique** constraint. A more general form of a referential-integrity constraint, where the referenced columns need not be a candidate key, cannot be directly specified in SQL. The SQL standard specifies other constructs that can be used to implement such constraints; they are described.

We can use the following short form as part of an attribute definition to declare that the attribute forms a foreign key:

deptlname **varchar(20) references department**

When a referential-integrity constraint is violated, the normal procedure is to reject the action that caused the violation (that is, the transaction performing the update action is rolled back). However, a **foreign key** clause can specify that if a delete or update action on the referenced relation violates the constraint, then,

create table *classroom*

(*building* **varchar** (15),
room-number **varchar** (7),
capacity **numeric** (4,0),

primary key (*building*, *room-number*))

create table *department*

(*deptname* **varchar** (20),
building **varchar** (15),
budget **numeric** (12,2) **check** (*budget* > 0),

primary key (*deptname*))

create table *course*

(*courseid* **varchar** (8),
title **varchar** (50),
deptname **varchar** (20),
credits **numeric** (2,0) **check** (*credits* > 0),

primary key (*courseid*), **foreign key** (*deptname*) **references** *department*)

create table *instructor*

(*ID* **varchar** (5),
name **varchar** (20), **not null**
deptname **varchar** (20),
salary **numeric** (8,2), **check** (*salary* > 29000),

primary key (*ID*),

foreign key (*deptname*) **references** *department*)

create table *section*

(*courseid* **varchar** (8),
secid **varchar** (8),
semester **varchar** (6), **check** (*semester* in
('Fall', 'Winter', 'Spring', 'Summer'),
year **numeric** (4,0), **check** (*year* > 1759 and *year* < 2100)
building **varchar** (15),

room-number **varchar** (7), *timeslotid* **varchar** (4),
primary key (*courseid*, *secid*, *semester*, *year*), **foreign**
key (*courseid*) **references** *course*, **foreign key** (*building*,
roomnumber) **references** *classroom*)

Figure 4.1 SQL data definition for part of the university database.

instead of rejecting the action, the system must take steps to change the tuple in the referencing relation to restore the constraint. Consider this definition of an integrity constraint on the relation *course*:

create table *course*
foreign key (*deptname*) **references** *department* **on**
delete cascade on update cascade,

Because of the clause **on delete cascade** associated with the foreign-key declaration, if a delete of a tuple in *department* results in this referential-integrity constraint being violated, the system does not reject the delete. Instead, the delete “cascades” to the *course* relation, deleting the tuple that refers to the department that was deleted. Similarly, the system does not reject an update to a field referenced by the constraint if it violates the constraint; instead, the system updates the field *deptname* in the referencing tuples in *course* to the new value as well. SQL also allows the **foreign key** clause to specify actions other than **cascade**, if the constraint is violated: The referencing field (here, *deptname*) can be set to *null* (by using **set null** in place of **cascade**), or to the default value for the domain (by using **set default**).

If there is a chain of foreign-key dependencies across multiple relations, a deletion or update at one end of the chain can propagate across the entire chain. An interesting case where the **foreign key** constraint on a relation references the same relation appears in Practice Exercises 4.9. If a cascading update or delete causes a constraint violation that cannot be handled by a further cascading operation, the system aborts the transaction. As a result, all the changes caused by the transaction and its cascading actions are undone.

Null values complicate the semantics of referential-integrity constraints in SQL. Attributes of foreign keys are allowed to be *null*, provided that they have not otherwise been declared to be **not null**. If all the columns of a foreign key are nonnull in a given tuple, the usual definition of foreign-key constraints is used for that tuple. If any of the foreign-key columns is *null*, the tuple is defined automatically to satisfy the constraint.

An **assertion** is a predicate expressing a condition that we wish the database always to satisfy. Domain constraints and referential-integrity constraints are special forms of assertions. We have paid substantial attention to these forms of assertions because they are easily tested and apply to a wide range of database applications. However, there are many constraints that we cannot express by using only these special forms. Two examples of such constraints are:

- For each tuple in the *student* relation, the value of the attribute *totcred* must equal the sum of credits of courses that the student has completed successfully.
- An instructor cannot teach in two different classrooms in a semester in the same time slot.¹

An assertion in SQL takes the form:

create assertion <assertion-name> **check** <predicate>;

4.4 Embedded SQL

The SQL standard defines embeddings of SQL in a variety of programming languages, such as C, C++, Cobol, Pascal, Java, PL/I, and Fortran. A language in which SQL queries are embedded is referred to as a *host* language, and the SQL structures permitted in the host language constitute *embedded* SQL.

Programs written in the host language can use the embedded SQL syntax to access and update data stored in a database. An embedded SQL program must be processed by a special preprocessor prior to compilation. The preprocessor replaces embedded SQL requests with host-language declarations and procedure calls that allow runtime execution of the database accesses. Then, the resulting program is compiled by the host-language compiler. This is the main distinction between embedded SQL and JDBC or ODBC.

In JDBC, SQL statements are interpreted at runtime (even if they are prepared first using the prepared statement feature). When embedded SQL is used, some SQL-related errors (including data-type errors) may be caught at compile time.

To identify embedded SQL requests to the preprocessor, we use the EXEC SQL statement; it has the form:

EXEC SQL <embedded SQL statement >;

The exact syntax for embedded SQL requests depends on the language in which SQL is embedded. In some languages, such as Cobol, the semicolon is replaced with END-EXEC.

We place the statement `SQL INCLUDE SQLCA` in the program to identify the place where the preprocessor should insert the special variables used for communication between the program and the database system.

Before executing any SQL statements, the program must first connect to the database. This is done using:

```
EXEC SQL connect to server user user-name using password;
```

Here, *server* identifies the server to which a connection is to be established.

Variables of the host language can be used within embedded SQL statements, but they must be preceded by a colon (:) to distinguish them from SQL variables. Variables used as above must be declared within a `DECLARE` section, as illustrated below. The syntax for declaring the variables, however, follows the usual host language syntax.

```
EXEC SQL BEGIN DECLARE SECTION;
int creditamount; EXEC SQL
END DECLARE SECTION;
```

Embedded SQL statements are similar in form to regular SQL statements. There are, however, several important differences, as we note here.

To write a relational query, we use the **declare cursor** statement. The result of the query is not yet computed. Rather, the program must use the **open** and **fetch** commands (discussed later in this section) to obtain the result tuples. As we shall see, use of a cursor is analogous to iterating through a result set in JDBC.

Consider the university schema. Assume that we have a host-language variable *credit-amount* in our program, declared as we saw earlier, and that we wish to find the names of all students who have taken more than *credit-amount* credit hours. We can write this query as follows:

```
EXEC SQL
declare c cursor for
select ID, name
from student
where totcred > :credit-amount;
```

The variable *c* in the preceding expression is called a *cursor* for the query. We use this variable to identify the query. We then use the **open** statement, which causes the query to be evaluated.

The **open** statement for our sample query is as follows:

```
EXEC SQL open c;
```

This statement causes the database system to execute the query and to save the results within a temporary relation. The query uses the value of the host-language variable (*credit-amount*) at the time the **open** statement is executed.

If the SQL query results in an error, the database system stores an error diagnostic in the SQL communication-area (SQLCA) variables.

We then use a series of **fetch** statements, each of which causes the values of one tuple to be placed in host-language variables. The **fetch** statement requires one host-language variable for each attribute of the result relation. For our example query, we need one variable to hold the *ID* value and another to hold the *name* value. Suppose that those variables are *si* and *sn*, respectively, and have been declared within a DECLARE section. Then the statement:

```
EXEC SQL fetch c into :si, :sn;
```

produces a tuple of the result relation. The program can then manipulate the variables *si* and *sn* by using the features of the host programming language.

A single **fetch** request returns only one tuple. To obtain all tuples of the result, the program must contain a loop to iterate over all tuples. Embedded SQL assists the programmer in managing this iteration. Although a relation is conceptually a set, the tuples of the result of a query are in some fixed physical order. When the program executes an **open** statement on a cursor, the cursor is set to point to the first tuple of the result. Each time it executes a **fetch** statement, the cursor is updated to point to the next tuple of the result. When no further tuples remain to be processed, the character array variable SQLSTATE in the SQLCA is set to '02000' (meaning "no more data"); the exact syntax for accessing this variable depends on the specific database system you use. Thus, we can use a **while** loop (or equivalent loop) to process each tuple of the result.

We must use the **close** statement to tell the database system to delete the temporary relation that held the result of the query. For our example, this statement takes the form

```
EXEC SQL close c;
```

Embedded SQL expressions for database modification (**update**, **insert**, and **delete**) do not return a result. Thus, they are somewhat simpler to express. A database-modification request takes the form

EXEC SQL < any valid **update**, **insert**, or **delete**>;

Host-language variables, preceded by a colon, may appear in the SQL database-modification expression. If an error condition arises in the execution of the statement, a diagnostic is set in the SQLCA.

Database relations can also be updated through cursors. For example, if we want to add 100 to the *salary* attribute of every instructor in the Music department, we could declare a cursor as follows.

SQLJ

The Java embedding of SQL, called SQLJ, provides the same features as other embedded SQL implementations, but using a different syntax that more closely matches features already present in Java, such as iterators. For example, SQLJ uses the syntax `#sql` instead of EXEC SQL, and instead of cursors, uses the Java iterator interface to fetch query results. Thus the result of executing a query is a Java iterator, and the `nextO` method of the Java iterator interface can be used to step through the result tuples, just as the preceding examples use **fetch** on the cursor. The iterator must have attributes declared, whose types match the types of the attributes in the SQL query result. The code snippet below illustrates the use of iterators.

```
#sql iterator deptInfolter (String dept_name, int
    avgSal); deptInfolter iter = null;

#sql iter = { select deptname, avg(salary)
from instructor group by depLname };
while (iter.nextO) {

    String deptName = iter.dept_nameO;

    int avgSal = iter.avgSalO;

    System.out.println(deptName + " " + avgSal);

}

iter.closeO;
```

SQLJ is supported by IBM DB2 and Oracle; both provide translators that convert SQLJ code into JDBC code. The translator can connect to the database in order to check the syntactic correctness of queries at compile time, and to ensure that the SQL types of

query results are compatible with the Java types of variables they are assigned to. As of early 2009, SQLJ is not supported by other database systems.

We do not describe SQLJ in detail here; see the bibliographic notes for more information.

EXEC SQL

```
declare c cursor for
select *
from instructor
where deptname = 'Music'
for update;
```

We then iterate through the tuples by performing **fetch** operations on the cursor (as illustrated earlier), and after fetching each tuple we execute the following code:

EXEC SQL

```
update instructor
set salary = salary + 100
where current of c;
```

Transactions can be committed using EXEC SQL COMMIT, or rolled back using EXEC SQL ROLLBACK.

Queries in embedded SQL are normally defined when the program is written. There are rare situations where a query needs to be defined at runtime. For example, an application interface may allow a user to specify selection conditions on one or more attributes of a relation, and may construct the **where** clause of an SQL query at runtime, with conditions on only those attributes for which the user specifies selections. In such cases, a query string can be constructed and prepared at runtime, using a statement of the form EXEC SQL PREPARE <query-name> FROM :<variable>, and a cursor can be opened on the query name.

4.5 Recursive Queries **

Consider the instance of the relation *prereq* shown in Figure 4.2 containing information about the various courses offered at the university and the prerequisite for each course.⁷

Suppose now that we want to find out which courses are a prerequisite whether directly or indirectly, for a specific course—say, CS-347. That is, we wish to find a course that is a direct prerequisite for CS-347, or is a prerequisite for a course that is a prerequisite for CS-347, and so on.

<i>courseid</i>	<i>prereqid</i>
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

Figure 4.2 *The prereq relation.*

Thus, if CS-301 is a prerequisite for CS-347, and CS-201 is a prerequisite for CS-301, and CS-101 is a prerequisite for CS-201, then CS-301, CS-201, and CS-101 are all prerequisites for CS-347.

The **transitive closure** of the relation *prereq* is a relation that contains all pairs (*cid*, *pre*) such that *pre* is a direct or indirect prerequisite of *cid*. There are numerous applications that require computation of similar transitive closures on **hierarchies**. For instance, organizations typically consist of several levels of organizational units. Machines consist of parts that in turn have subparts, and so on; for example, a bicycle may have subparts such as wheels and pedals, which in turn have subparts such as tires, rims, and spokes. Transitive closure can be used on such hierarchies to find, for example, all parts in a bicycle.

4.5.1 Transitive Closure Using Iteration

One way to write the above query is to use iteration: First find those courses that are a direct prerequisite of CS-347, then those courses that are a prerequisite of all the courses under the first set, and so on. This iterative process continues until we reach an iteration where no courses are added. Figure 4.3 shows a function *findAllPrereqs(cid)* to carry out this task; the function takes the *course id* of the course as a parameter (*cid*), computes the set of all direct and indirect prerequisites of that course, and returns the set. The procedure uses three temporary tables:

- *cjprreq*: stores the set of tuples to be returned.
- *newc-prreq*: stores the courses found in the previous iteration.
- *temp*: used as temporary storage while sets of courses are manipulated.

Note that SQL allows the creation of temporary tables using the command **create temporary table**; such tables are available only within the transaction executing the query, and are dropped when the transaction finishes. Moreover, if two instances of *findAllPrereqs* run concurrently, each gets its own copy of the temporary tables; if they shared a copy, their result could be incorrect.

The procedure inserts all direct prerequisites of course *cid* into *newc-prreq* before the **repeat** loop. The **repeat** loop first adds all courses in *newc-prreq* to *cjprreq*. Next, it computes prerequisites of all those courses in *newcjprreq*, except those that have already been found to be prerequisites of *cid*, and stores them in the temporary table *temp*. Finally, it replaces the contents of *newcjprreq* by the contents of *temp*. The **repeat** loop terminates when it finds no new (indirect) prerequisites.

Figure 4.4 shows the prerequisites that would be found in each iteration, if the procedure were called for the course named CS-347.

We note that the use of the **except** clause in the function ensures that the function works even in the (abnormal) case where there is a cycle of prerequisites. For example, if *a* is a prerequisite for *b*, *b* is a prerequisite for *c*, and *c* is a prerequisite for *a*, there is a cycle.

create function *findAllPrereqs*(*cid* **varchar**(8))

– – Finds all courses that are prerequisite (directly or indirectly) for *cid* **returns table** (*courseid* **varchar**(8))

– – The relation *prereq*(*courseid*, *prereqid*) specifies which course is

– – directly a prerequisite for another course.

begin

create temporary table *cjprreq* (*courseid* **varchar**(8));

– – table *cjprreq* stores the set of courses to be returned **create temporary table** *newcjprreq* (*courseid* **varchar**(8));


```

-- table newcjprrreq contains courses found in the previous iteration create
temporary table temp (courseid varchar(8));

-- table temp is used to store intermediate results insert into newcjprrreq

select prereqid
from prereq
where courseid = cid; repeat
insert into cjprreq select courseid from newc-prreq;
insert into temp
(select prereq.courseid

from newcjprrreq, prereq where newc-prreq.courseid = prereq.prereqid ) except(

select courseid from cjprreq ); delete from newcjprrreq; insert into newcjprrreq
select * from temp; delete from temp;

until not exists (select * from newcjprrreq) end repeat; return table cjprreq;
end

```

Figure 4.3 Finding all prerequisites of a course.

While cycles may be unrealistic in course prerequisites, cycles are possible in other applications. For instance, suppose we have a relation *flights(to, from)* that says which cities can be reached from which other cities by a direct flight. We can

Iteration Number	Tuples in c1
0	(CS-301)
1	(CS-301), (CS-201)
2	(CS-301), (CS-201)
3	(CS-301), (CS-201),
4	(CS-101) (CS-301),
5	(CS-201), (CS-101)

Figure 4.4 Prerequisites of CS-347 in iterations of function *findAHPrrreqs*.

write code similar to that in the *findAUPrereqs* function, to find all cities that are reachable by a sequence of one or more flights from a given city. All we have to do is to replace *prereq* by *flight* and replace attribute names correspondingly. In this situation, there can be cycles of reachability, but the function would work correctly since it would eliminate cities that have already been seen.

4.5.2 Recursion in SQL

It is rather inconvenient to specify transitive closure using iteration. There is an alternative approach, using recursive view definitions, that is easier to use.

We can use recursion to define the set of courses that are prerequisites of a particular course, say CS-347, as follows. The courses that are prerequisites (directly or indirectly) of CS-347 are:

1. Courses that are prerequisites for CS-347.
2. Courses that are prerequisites for those courses that are prerequisites (directly or indirectly) for CS-347.

Note that case 2 is recursive, since it defines the set of courses that are prerequisites of CS-347 in terms of the set of courses that are prerequisites of CS-347. Other examples of transitive closure, such as finding all subparts (direct or indirect) of a given part can also be defined in a similar manner, recursively.

Since the SQL:1999 version, the SQL standard supports a limited form of recursion, using the **with recursive** clause, where a view (or temporary view) is expressed in terms of itself. Recursive queries can be used, for example, to express transitive closure concisely. Recall that the **with** clause is used to define a temporary view whose definition is available only to the query in which it is defined. The additional keyword **recursive** specifies that the view is recursive.

For example, we can find every pair (*cid*,*pre*) such that *pre* is directly or indirectly a prerequisite for course *cid*, using the recursive SQL view shown in Figure 4.5.

Any recursive view must be defined as the union of two subqueries: a **base query** that is nonrecursive and a **recursive query** that uses the recursive view. In the example in Figure 4.5, the base query is the select on *prereq* while the recursive query computes the join of *prereq* and *rec-prereq*.

```

with recursive cjprereq(courseJd, prereqJd) as (
    select courseJd, prereqJd
    from prereq union
    select prereq.prereqJd, cjprereq.courseJd
    from prereq, cjprereq
    where prereq.courseJd = cjprereq.prereqJd ) select * from cjprereq;

```

Figure 4.5 Recursive query in SQL.

The meaning of a recursive view is best understood as follows. First compute the base query and add all the resultant tuples to the recursively defined view relation *recjprereq* (which is initially empty). Next compute the recursive query using the current contents of the view relation, and add all the resulting tuples back to the view relation. Keep repeating the above step until no new tuples are added to the view relation. The resultant view relation instance is called a **fixed point** of the recursive view definition. (The term “fixed” refers to the fact that there is no further change.) The view relation is thus defined to contain exactly the tuples in the fixed-point instance.

Applying the above logic to our example, we first find all direct prerequisites of each course by executing the base query. The recursive query adds one more level of courses in each iteration, until the maximum depth of the course-prereq relationship is reached. At this point no new tuples are added to the view, and a fixed point is reached.

To find the prerequisites of a specific course, such as CS-347, we can modify the outer level query by adding a where clause “**where** *recjprereq.courseJd* = ‘CS-347’”. One way to evaluate the query with the selection is to compute the full contents of *recjprereq* using the iterative technique, and then select from this result only those tuples whose *courseJd* is CS-347. However, this would result in computing (course, prerequisite) pairs for all courses, all of which are irrelevant except for those for the course CS-347. In fact the database system is not required to use the above iterative technique to compute the full result of the recursive query and then perform the selection. It may get the same result using other techniques that may be more efficient, such as that used in the function *findAUPrereqs* which we saw earlier. See the bibliographic notes for references to more information on this topic.

There are some restrictions on the recursive query in a recursive view; specifically, the query should be **monotonic**, that is, its result on a view relation instance *V1* should be a superset of its result on a view relation instance *V2* if *V1* is a super-set of *V2*. Intuitively,

if more tuples are added to the view relation, the recursive query should return at least the same set of tuples as before, and possibly return additional tuples.

In particular, recursive queries should not use any of the following constructs, since they would make the query nonmonotonic:

- Aggregation on the recursive view.
- **not exists** on a subquery that uses the recursive view.
- Set difference (**except**) whose right-hand side uses the recursive view.

For instance, if the recursive query was of the form $r \text{ --- } v$ where v is the recursive view, if we add a tuple to v the result of the query can become smaller; the query is therefore not monotonic.

The meaning of recursive views can be defined by the iterative procedure as long as the recursive query is monotonic; if the recursive query is nonmonotonic, the meaning of the view is hard to define. SQL therefore requires the queries to be monotonic. Recursive queries are discussed in more detail in the context of the Datalog query language.

SQL also allows creation of recursively defined permanent views by using **create recursive view** in place of **with recursive**. Some implementations support recursive queries using a different syntax; see the respective system manuals for further details.

4.6 The Tuple Relational Calculus

When we write a relational-algebra expression, we provide a sequence of procedures that generates the answer to our query. The tuple relational calculus, by contrast, is a **nonprocedural** query language. It describes the desired information without giving a specific procedure for obtaining that information. A query in the tuple relational calculus is expressed as:

$$\{t \mid P(t)\}$$

That is, it is the set of all tuples t such that predicate P is true for t . Following our earlier notation, we use $t[A]$ to denote the value of tuple t on attribute A , and we use $t \in r$ to denote that tuple t is in relation r .

4.6.1 Example Queries

Find the *ID*, *name*, *deptname*, *salary* for instructors whose salary is greater than \$80,000:

$$\{t \mid t \in \text{instructor} \wedge t[\text{salary}] > 80000\}$$

Suppose that we want only the *ID* attribute, rather than all attributes of the *instructor* relation. To write this query in the tuple relational calculus, we need to write an expression for a relation on the schema (*ID*). We need those tuples on (*ID*) such that there is a tuple in *instructor* with the *salary* attribute > 80000. To express this request, we need the construct “there exists” from mathematical logic. The notation:

$$\exists t \in r (Q(t))$$

means “there exists a tuple *t* in relation *r* such that predicate *Q(t)* is true.”

Using this notation, we can write the query “Find the instructor *ID* for each instructor with a salary greater than \$80,000” as:

$$\{t \mid \exists s \in \text{instructor} (t[\text{ID}] = s[\text{ID}] \wedge s[\text{salary}] > 80000)\}$$

In English, we read the preceding expression as “The set of all tuples *t* such that there exists a tuple *s* in relation *instructor* for which the values of *t* and *s* for the *ID* attribute are equal, and the value of *s* for the *salary* attribute is greater than \$80,000.”

Tuple variable *t* is defined on only the *ID* attribute, since that is the only attribute having a condition specified for *t*. Thus, the result is a relation on (*ID*).

Consider the query “Find the names of all instructors whose department is in the Watson building.” This query is slightly more complex than the previous queries, since it involves two relations: *instructor* and *department*. As we shall see, however, all it requires is that we have two “there exists” clauses in our tuple-relational-calculus expression, connected by *and* (*∧*). We write the query as follows:

$$\{t \mid \exists s \in \text{instructor} (t[\text{name}] = s[\text{name}] \wedge \exists u \in \text{department} (u[\text{dept_name}] = s[\text{dept_name}] \wedge u[\text{building}] = \text{“Watson”}))\}$$

name
Einstein
Crick
Gold

Figure 4.6 Names of all instructors whose department is in the Watson building.

Tuple variable u is restricted to departments that are located in the Watson building, while tuple variable s is restricted to instructors whose *deptname* matches that of tuple variable u . Figure 4.6 shows the result of this query.

To find the set of all courses taught in the Fall 2009 semester, the Spring 2010 semester, or both, we used the union operation in the relational algebra. In the tuple relational calculus, we shall need two “there exists” clauses, connected by *or* ($\vee$):

$$[\exists s \in \text{section } (t[\text{courseid}] = s[\text{course_id}]) \wedge s[\text{semester}] = \text{“Fall”} \wedge s[\text{year}] = 2009)\}$$

$$\vee \exists u \in \text{section } (u[\text{course.id}] = t[\text{courseid}]) \wedge u[\text{semester}] = \text{“Spring”} \wedge u[\text{year}] = 2010)\}$$

This expression gives us the set of all *courseid* tuples for which at least one of the following holds:

- The *courseid* appears in some tuple of the *section* relation with *semester* = Fall and *year* = 2009.
- The *courseid* appears in some tuple of the *section* relation with *semester* = Spring and *year* = 2010.

If the same course is offered in both the Fall 2009 and Spring 2010 semesters, its *courseid* appears only once in the result, because the mathematical definition of a set does not allow duplicate members.

If we now want *only* those *courseid* values for courses that are offered in *both* the Fall 2009 and Spring 2010 semesters, all we need to do is to change the *or* ($\vee$) to *and* ($\wedge$) in the preceding expression.

$$\begin{aligned} &[\exists s \in \text{section } (t[\text{courseid}] = s[\text{course_id}]) \wedge \\ &\quad s[\text{semester}] = \text{“Fall”} \wedge s[\text{year}] = 2009)\} \end{aligned}$$

$$\begin{aligned} &\wedge \exists u \in \text{section } (u[\text{course_id}] = t[\text{course_id}]) \wedge \\ &\quad u[\text{semester}] = \text{“Spring”} \wedge u[\text{year}] = 2010)\} \end{aligned}$$

Now consider the query “Find all the courses taught in the Fall 2009 semester but not in Spring 2010 semester.” The tuple-relational-calculus expression for this query is similar to the expressions that we have just seen, except for the use of the *not* ($\neg$) symbol:

$$\{t \mid \exists s \in \text{section} (t[\text{courseid}] = s[\text{courseid}] \wedge s[\text{semester}] = \text{"Fall"} \wedge s[\text{year}] = 2009) \wedge \\ \neg \exists u \in \text{section} (u[\text{courseid}] = t[\text{course_id}]) \wedge u[\text{semester}] = \text{"Spring"} \wedge u[\text{year}] = 2010)\}$$

This tuple-relational-calculus expression uses the $\exists s \in \text{section} (\dots)$ clause to require that a particular *courseid* is taught in the Fall 2009 semester, and it uses the $\neg \exists u \in \text{section} (\dots)$ clause to eliminate those *course id* values that appear in some tuple of the *section* relation as having been taught in the Spring 2010 semester.

The query that we shall consider next uses implication, denoted by $\Rightarrow$. The formula $P \Rightarrow Q$ means “*P* implies *Q*”; that is, “if *P* is true, then *Q* must be true.” Note that $P \Rightarrow Q$ is logically equivalent to $\neg P \vee Q$. The use of implication rather than *not* and *or* often suggests a more intuitive interpretation of a query in English.

Consider the query that “Find all students who have taken all courses offered in the Biology department.” To write this query in the tuple relational calculus, we introduce the “for all” construct, denoted by $\forall$. The notation:

$$\forall t \in r (Q(t))$$

means “*Q* is true for all tuples *t* in relation *r*.”

We write the expression for our query as follows:

$$\{t \mid \exists r \in \text{student} (r[\text{ID}] = t[\text{ID}]) \wedge \\ (\forall u \in \text{course} (u[\text{dept-name}] = \text{"Biology"} \Rightarrow \exists s \in \text{takes} (t[\text{ID}] = s[\text{ID}] \wedge s[\text{courseid}] = \\ u[\text{course_id}])))\}$$

In English, we interpret this expression as “The set of all students (that is, (*ID*) tuples *t*) such that, for *all* tuples *u* in the *course* relation, if the value of *u* on attribute *deptname* is ‘Biology’, then there exists a tuple in the *takes* relation that includes the student *ID* and the *courseid*.”

Note that there is a subtlety in the above query: If there is no course offered in the Biology department, all student *IDs* satisfy the condition. The first line of the query expression is critical in this case—without the condition

$$\exists r \in \text{student} (r[ID] = t[ID])$$

if there is no course offered in the Biology department, any value of *t* (including values that are not student *IDs* in the *student* relation) would qualify.

4.6.2 Formal Definition

We are now ready for a formal definition. A tuple-relational-calculus expression is of the form:

$$\{t \mid P(t)\}$$

where *P* is a *formula*. Several tuple variables may appear in a formula. A tuple variable is said to be a *free variable* unless it is quantified by a $\exists$ or $\forall$. Thus, in:

$$t \in \text{instructor} \wedge \exists s \in \text{department} \{t[\text{dept.name}] = s[\text{dept.name}]\}$$

t is a free variable. Tuple variable *s* is said to be a *bound* variable.

A tuple-relational-calculus formula is built up out of *atoms*. An atom has one of the following forms:

- $s \in r$, where *s* is a tuple variable and *r* is a relation (we do not allow use of the $\bowtie$ operator).
- $s[x] \theta u[y]$, where *s* and *u* are tuple variables, *x* is an attribute on which *s* is defined, *y* is an attribute on which *u* is defined, and θ is a comparison operator ($<$, $\leq$, $=$, $\neq$, $>$, $>$); we require that attributes *x* and *y* have domains whose members can be compared by θ .
- $s[x] \theta c$, where *s* is a tuple variable, *x* is an attribute on which *s* is defined, θ is a comparison operator, and *c* is a constant in the domain of attribute *x*.

We build up formulae from atoms by using the following rules:

- An atom is a formula.
- If *P* is a formula, then so are $\neg P$ and (P) .
- If *P* and *P*₂ are formulae, then so are $P \vee P_2$, $P \wedge P_2$, and $P \Rightarrow P_2$.
- If $P\{s\}$ is a formula containing a free tuple variable *s*, and *r* is a relation, then

$$\exists s \in r(P_i(s)) \text{ and } \forall s \in r(P_i(s))$$

are also formulae.

As we could for the relational algebra, we can write equivalent expressions that are not identical in appearance. In the tuple relational calculus, these equivalences include the following three rules:

1. $P_i \wedge P_2$ is equivalent to $\neg(\neg(P_i) \vee \neg(P_2))$.
2. $\forall t \in r(P_i(t))$ is equivalent to $\neg \exists t \in r(\neg(P_i(t)))$.
3. $P_i \Rightarrow P_2$ is equivalent to $\neg(P_i) \vee P_2$.

4.6.3 Safety of Expressions

There is one final issue to be addressed. A tuple-relational-calculus expression may generate an infinite relation. Suppose that we write the expression:

$$\{t \mid \neg(t \in instructor)\}$$

There are infinitely many tuples that are not in *instructor*. Most of these tuples contain values that do not even appear in the database! Clearly, we do not wish to allow such expressions.

To help us define a restriction of the tuple relational calculus, we introduce the concept of the **domain** of a tuple relational formula, P . Intuitively, the domain of P , denoted $dom(P)$, is the set of all values referenced by P . They include values mentioned in P itself, as well as values that appear in a tuple of a relation mentioned in P . Thus, the domain of P is the set of all values that appear explicitly in P or that appear in one or more relations whose names appear in P . For example, $dom(t \in instructor \wedge t[salary] > 80000)$ is the set containing 80000 as well as the set of all values appearing in any attribute of any tuple in the *instructor* relation. Similarly, $dom(\neg(t \in instructor))$ is also the set of all values appearing in *instructor*, since the relation *instructor* is mentioned in the expression.

We say that an expression $\{t \mid P(t)\}$ is *safe* if all values that appear in the result are values from $dom(P)$. The expression $\{t \mid \neg(t \in instructor)\}$ is not safe. Note that $dom(\neg(t \in instructor))$ is the set of all values appearing in *instructor*. However, it is possible to have a tuple t not in *instructor* that contains values that do not appear in *instructor*. The other examples of tuple-relational-calculus expressions that we have written in this section are safe.

The number of tuples that satisfy an unsafe expression, such as $\{t \mid (t \in instructor)\}$, could be infinite, whereas safe expressions are guaranteed to have finite results. The class of tuple-relational-calculus expressions that are allowed is therefore restricted to those that are safe.

4.6.4 Expressive Power of Languages

The tuple relational calculus restricted to safe expressions is equivalent in expressive power to the basic relational algebra (with the operators $\cup$, $-$, $\times$, σ , and π , but without the extended relational operations such as generalized projection and aggregation (ρ)). Thus, for every relational-algebra expression using only the basic operations, there is an equivalent expression in the tuple relational calculus, and for every tuple-relational-calculus expression, there is an equivalent relational-algebra expression. We shall not prove this assertion here; the bibliographic notes contain references to the proof. Some parts of the proof are included in the exercises. We note that the tuple relational calculus does not have any equivalent of the aggregate operation, but it can be extended to support aggregation. Extending the tuple relational calculus to handle arithmetic expressions is straightforward.

4.7 The Domain Relational Calculus

A second form of relational calculus, called **domain relational calculus**, uses *domain* variables that take on values from an attributes domain, rather than values for an entire tuple. The domain relational calculus, however, is closely related to the tuple relational calculus.

Domain relational calculus serves as the theoretical basis of the widely used QBE language (see Appendix B.I), just as relational algebra serves as the basis for the SQL language.

4.7.1 Formal Definition

An expression in the domain relational calculus is of the form

$$\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$$

where $x_1, x_2, \dots, x_n$ represent domain variables. P represents a formula composed of atoms, as was the case in the tuple relational calculus. An atom in the domain relational calculus has one of the following forms:

- $\langle x_1, x_2, \dots, x_n \rangle \in r$, where r is a relation on n attributes and $x_1, x_2, \dots, x_n$ are domain variables or domain constants.

- $x \theta y$, where x and y are domain variables and θ is a comparison operator ($<$, $<=$, $=$, $>$, $>=$). We require that attributes x and y have domains that can be compared by θ .
- $x \theta c$, where x is a domain variable, θ is a comparison operator, and c is a constant in the domain of the attribute for which x is a domain variable.

We build up formulae from atoms by using the following rules:

- An atom is a formula.
- If P is a formula, then so are $\neg P$ and (P) .
- If P_1 and P_2 are formulae, then so are $P_1 \vee P_2$, $P_1 \wedge P_2$ and $P_1 \Rightarrow P_2$.
- If $P(x)$ is a formula in x , where x is a free domain variable, then

$\exists x (P(x))$ and $\forall x (P(x))$ are also formulae.

As a notational shorthand, we write $\exists a, b, c (P(a, b, c))$ for $\exists a (\exists b (\exists c (P(a, b, c))))$.

4.7.2 Example Queries

We now give domain-relational-calculus queries for the examples that we considered earlier. Note the similarity of these expressions and the corresponding tuple-relational-calculus expressions.

- Find the instructor ID , $name$, $deptname$, and $salary$ for instructors whose salary is greater than \$80,000:

$$\{ \langle i, n, d, s \rangle \mid \langle i, n, d, s \rangle \in instructor \wedge s > 80000 \}$$

- Find all instructor ID for instructors whose salary is greater than \$80,000:

$$\{ \langle n \rangle \mid \exists i, d, s (\langle i, n, d, s \rangle \in instructor \wedge s > 80000) \}$$

Although the second query appears similar to the one that we wrote for the tuple relational calculus, there is an important difference. In the tuple calculus, when we write $\exists s$ for some tuple variable s , we bind it immediately to a relation by writing $\exists s \in r$. However, when we write $\exists n$ in the domain calculus, n refers not to a tuple, but rather to a domain value. Thus, the domain of variable n is unconstrained until the subformula $\langle i, n, d, s \rangle \in instructor$ constrains n to instructor names that appear in the *instructor* relation.

We now give several examples of queries in the domain relational calculus.

- Find the names of all instructors in the Physics department together with the *courseId* of all courses they teach:

$$\{ \langle n, c \rangle \mid \exists i, a (\langle i, c, a, s, y \rangle \in \text{teaches} \wedge \exists d, s (\langle i, n, d, s \rangle \in \text{instructor} \wedge d = \text{"Physics"})) \}$$

- Find the set of all courses taught in the Fall 2009 semester, the Spring 2010 semester, or both:

$$\{ \langle c \rangle \mid \exists s (\langle c, a, s, y, b, r, t \rangle \in \text{section} \wedge s = \text{"Fall"} \vee \exists u (\langle c, a, s, y, b, r, t \rangle \in \text{section} \wedge s = \text{"2009"} \vee \exists u (\langle c, a, s, y, b, r, t \rangle \in \text{section} \wedge s = \text{"Spring"} \vee \exists u (\langle c, a, s, y, b, r, t \rangle \in \text{section} \wedge s = \text{"2010"})) \}$$

- Find all students who have taken all courses offered in the Biology department:

$$\{ \langle i \rangle \mid \exists n, d, t (\langle i, n, d, t \rangle \in \text{student}) \wedge \forall x, y, z, w (\langle x, y, z, w \rangle \in \text{course} \wedge z = \text{"Biology"} \Rightarrow \exists a, b (\langle a, x, b, r, p, q \rangle \in \text{takes} \wedge A \langle c, a \rangle \in \text{depositor})) \}$$

Note that as was the case for tuple-relational-calculus, if no courses are offered in the Biology department, all students would be in the result.

4.7.3 Safety of Expressions

We noted that, in the tuple relational calculus (Section 6.2), it is possible to write expressions that may generate an infinite relation. That led us to define *safety* for tuple-relational-calculus expressions. A similar situation arises for the domain relational calculus. An expression such as

$$\{ \langle i, n, d, s \rangle \mid \neg (\langle i, n, d, s \rangle \in \text{instructor}) \}$$

is unsafe, because it allows values in the result that are not in the domain of the expression.

For the domain relational calculus, we must be concerned also about the form of formulae within “there exists” and “for all” clauses. Consider the expression

$$\{ \langle x \rangle \mid \exists y (\langle x, y \rangle \in r) \wedge \exists z (\neg (\langle x, z \rangle \in r) \wedge P(x, z)) \}$$

where P is some formula involving x and z . We can test the first part of the formula, $\exists y (\langle x, y \rangle \in r)$, by considering only the values in r . However, to test the second part of the

formula, $\exists z (\langle x, z \rangle \in r \wedge P(x, z))$, we must consider values for z that do not appear in r . Since all relations are finite, an infinite number of values do not appear in r . Thus, it is not possible, in general, to test the second part of the formula without considering an infinite number of potential values for z . Instead, we add restrictions to prohibit expressions such as the preceding one.

In the tuple relational calculus, we restricted any existentially quantified variable to range over a specific relation. Since we did not do so in the domain calculus, we add rules to the definition of safety to deal with cases like our example. We say that an expression

$$\{ \langle X_1, X_2, \dots, X_n \rangle \mid P(X_1, X_2, \dots, X_n) \}$$

is safe if all of the following hold:

1. All values that appear in tuples of the expression are values from $dom(P)$.
2. For every “there exists” subformula of the form $\exists x (P_1(x))$, the subformula is true if and only if there is a value x in $dom(P_1)$ such that $P_1(x)$ is true.
3. For every “for all” subformula of the form $\forall x (P_1(x))$, the subformula is true if and only if $P_1(x)$ is true for all values x from $dom(P_1)$.

The purpose of the additional rules is to ensure that we can test “for all” and “there exists” subformulae without having to test infinitely many possibilities. Consider the second rule in the definition of safety. For $\exists x (P_1(x))$ to be true, we need to find only one x for which $P_1(x)$ is true. In general, there would be infinitely many values to test. However, if the expression is safe, we know that we can restrict our attention to values from $dom(P_1)$. This restriction reduces to a finite number the tuples we must consider.

The situation for subformulae of the form $\forall x (P_1(x))$ is similar. To assert that $\forall x (P_1(x))$ is true, we must, in general, test all possible values, so we must examine infinitely many values. As before, if we know that the expression is safe, it is sufficient for us to test $P_1(x)$ for those values taken from $dom(P_1)$.

All the domain-relational-calculus expressions that we have written in the example queries of this section are safe, except for the example unsafe query we saw earlier.

4.7.4 Expressive Power of Languages

When the domain relational calculus is restricted to safe expressions, it is equivalent in expressive power to the tuple relational calculus restricted to safe expressions. Since we noted earlier that the restricted tuple relational calculus is equivalent to the relational algebra, all three of the following are equivalent:

- The basic relational algebra (without the extended relational-algebra operations)
- The tuple relational calculus restricted to safe expressions
- The domain relational calculus restricted to safe expressions

We note that the domain relational calculus also does not have any equivalent of the aggregate operation, but it can be extended to support aggregation, and extending it to handle arithmetic expressions is straightforward.

4.8 Summary

- SQL queries can be invoked from host languages, via embedded and dynamic SQL. The ODBC and JDBC standards define application program interfaces to access SQL databases from C and Java language programs. Increasingly, programmers use these APIs to access databases.
- The **tuple relational calculus** and the **domain relational calculus** are non-procedural languages that represent the basic power required in a relational query language. The basic relational algebra is a procedural language that is equivalent in power to both forms of the relational calculus when they are restricted to safe expressions.

4.9 Questions for Discussion

1. Explain Built-in Data types in SQL
2. List out the Integrity Constraints
3. Describe about the Embedded SQL

4.10 Suggested Reading

Recommended Text

A.Silberchatz, H.F Korth and S.Sudharshan, 2006, Database System Concepts, 5th Edition, Tata McGraw Hill, New Delhi.

Reference Books

J.D Ullaman, 1988, Principles of Database Systems, Galgotia Publishers, New Delhi.

Lesson - 5

THE ENTITY RELATIONSHIP MODEL

CONTENTS

- 5.0 Aims and Objectives**
- 5.1 Introduction**
- 5.2 The Entity-Relationship Model**
- 5.3 Constraints**
- 5.4 E-R-Diagrams**
- 5.5 Weak Entity Sets**
- 5.6 Reduction to Relational Schemas**
- 5.7 Summary**
- 5.8 Questions for Discussion**
- 5.9 Suggested Reading**

5.0 Aims and Objectives

After studying this lesson you should be able to:

- Define E-R model
- Discuss about the Constraints
- Describe about E-R Diagram
- Define Weak entity sets

5.1 Introduction

In this chapter, we focus on the entity-relationship data model (E-R), which provides a means of identifying entities to be represented in the database and how those entities are related. Ultimately, the database design will be expressed in terms of a relational database design and an associated set of constraints. We show in this chapter how an E-R design can be transformed into a set of relation schemas and how some of the constraints can be captured in that design. Then, in Chapter 8, we consider in detail whether a set of relation schemas is a good or bad database design and study the process of creating good designs

using a broader set of constraints. These two chapters cover the fundamental concepts of database design.

5.2 The Entity-Relationship Model

The **entity-relationship (E-R)** data model was developed to facilitate database design by allowing specification of an *enterprise schema* that represents the overall logical structure of a database.

The E-R model is very useful in mapping the meanings and interactions of real-world enterprises onto a conceptual schema. Because of this usefulness, many database-design tools draw on concepts from the E-R model. The E-R data model employs three basic concepts: entity sets, relationship sets, and attributes, which we study first. The E-R model also has an associated diagrammatic representation, the E-R diagram, which we study later in this chapter.

5.2.1 Entity Sets

An **entity** is a “thing” or “object” in the real world that is distinguishable from all other objects. For example, each person in a university is an entity. An entity has a set of properties, and the values for some set of properties may uniquely identify an entity. For instance, a person may have a *personid* property whose value uniquely identifies that person. Thus, the value 677-89-9011 for *personid* would uniquely identify one particular person in the university. Similarly courses can be thought of as entities, and *courseid* uniquely identifies a course entity in the university. An entity may be concrete, such as a person or a book, or it may be abstract, such as a course, a course offering, or a flight reservation.

An **entity set** is a set of entities of the same type that share the same properties, or attributes. The set of all people who are instructors at a given university, for example, can be defined as the entity set *instructor*. Similarly, the entity set *student* might represent the set of all students in the university.

In the process of modeling, we often use the term *entity set* in the abstract, without referring to a particular set of individual entities. We use the term **ex-tension** of the entity set to refer to the actual collection of entities belonging to the entity set. Thus, the set of actual instructors in the university forms the extension of the entity set *instructor*. The above distinction is similar to the difference between a relation and a relation instance, which we saw in Chapter 2.

Entity sets do not need to be disjoint. For example, it is possible to define the entity set of all people in a university (*person*). A *person* entity may be an *instructor* entity, a *student* entity, both, or neither.

An entity is represented by a set of **attributes**. Attributes are descriptive properties possessed by each member of an entity set. The designation of an attribute for an entity set expresses that the database stores similar information concerning each entity in the entity set; however, each entity may have its own value for each attribute. Possible attributes of the *instructor* entity set are *ID*, *name*, *deptname*, and *salary*. In real life, there would be further attributes, such as street number, apartment number, state, postal code, and country, but we omit them to keep our examples simple. Possible attributes of the *course* entity set are *courseid*, *title*, *deptname*, and *credits*.

Each entity has a **value** for each of its attributes. For instance, a particular *instructor* entity may have the value 12121 for *ID*, the value Wu for *name*, the value Finance for *deptname*, and the value 90000 for *salary*.

The *ID* attribute is used to identify instructors uniquely, since there may be more than one instructor with the same name. In the United States, many enterprises find it convenient to use the *social-security* number of a person² as an attribute whose value uniquely identifies the person. In general the enterprise would have to create and assign a unique identifier for each instructor.

A database thus includes a collection of entity sets, each of which contains any number of entities of the same type. Figure 5.1 shows part of a university database that consists of two entity sets: *instructor* and *student*. To keep the figure simple, only some of the attributes of the two entity sets are shown.

A database for a university may include a number of other entity sets. For example, in addition to keeping track of instructors and students, the university also has information about courses, which are represented by the entity set *course*

76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

instructor

98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student

Figure 5.1 Entity sets *instructor* and *student*.

with attributes *courseid*, *title*, *deptname* and *credits*. In a real setting, a university database may keep dozens of entity sets.

5.2.2 Relationship Sets

A **relationship** is an association among several entities. For example, we can define a relationship *advisor* that associates instructor Katz with student Shankar. This relationship specifies that Katz is an advisor to student Shankar.

A **relationship set** is a set of relationships of the same type. Formally, it is a mathematical relation on $n > 2$ (possibly nondistinct) entity sets. If $E_1, E_2, \dots, E_n$ are entity sets, then a relationship set R is a subset of

$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

where $(e_1, e_2, \dots, e_n)$ is a relationship.

Consider the two entity sets *instructor* and *student* in Figure 7.1. We define the relationship set *advisor* to denote the association between instructors and students. Figure 5.2 depicts this association.

As another example, consider the two entity sets *student* and *section*. We can define the relationship set *takes* to denote the association between a student and the course sections in which that student is enrolled.

The association between entity sets is referred to as participation; that is, the entity sets $E_1, E_2, \dots, E_n$ **participate** in relationship set R . A **relationship in-stance** in an E-R schema represents an association between the named entities in the real-world enterprise that is being modeled. As an illustration, the individual *instructor* entity Katz, who has instructor *ID* 45565, and the *student* entity Shankar, who has student *ID* 12345, participate in a relationship instance of *advisor*. This relationship instance represents that in the university, the instructor Katz is advising student Shankar.

The function that an entity plays in a relationship is called that entity's **role**. Since entity sets participating in a relationship set are generally distinct, roles

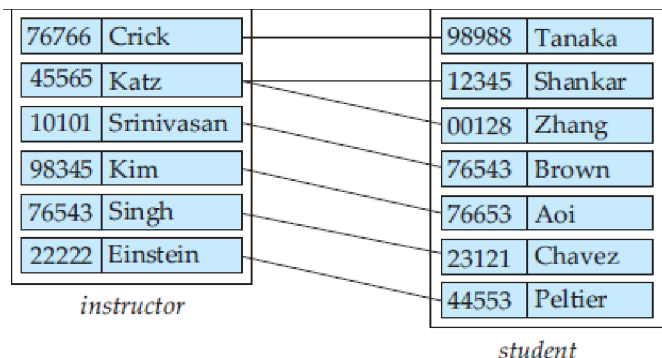


Figure 5.2 Relationship set advisor.

are implicit and are not usually specified. However, they are useful when the meaning of a relationship needs clarification. Such is the case when the entity sets of a relationship set are not distinct; that is, the same entity set participates in a relationship set more than once, in different roles. In this type of relationship set, sometimes called a **recursive** relationship set, explicit role names are necessary to specify how an entity participates in a relationship instance. For example, consider the entity set *course* that records information about all the courses offered in the university. To depict the situation where one course (C2) is a prerequisite for another course (C1) we have relationship set *prereq* that is modeled by ordered pairs of *course* entities. The first course of a pair takes the role of course C1, whereas the second takes the role of prerequisite course C2. In this way, all relationships of *prereq* are characterized by (C1, C2) pairs; (C2, C1) pairs are excluded.

A relationship may also have attributes called **descriptive attributes**. Consider a relationship set *advisor* with entity sets *instructor* and *student*. We could associate the attribute *date* with that relationship to specify the date when an instructor became the advisor of a student. The *advisor* relationship among the entities corresponding to instructor Katz and student Shankar has the value “10 June 2007” for attribute *date*, which means that Katz became Shankar’s advisor on 10 June 2007.

Figure 5.3 shows the relationship set *advisor* with a descriptive attribute *date*. Please note that Katz advises two students with two different advising dates.

As a more realistic example of descriptive attributes for relationships, consider the entity sets *student* and *section*, which participate in a relationship set *takes*. We may wish to store a descriptive attribute *grade* with the relationship to record the grade that a student got in the class. We may also store a descriptive attribute *for credit* to record whether a student has taken the course for credit, or is auditing (or sitting in on) the course.

A relationship instance in a given relationship set must be uniquely identifiable from its participating entities, without using the descriptive attributes. To understand this point, suppose we want to model all the dates when an instructor

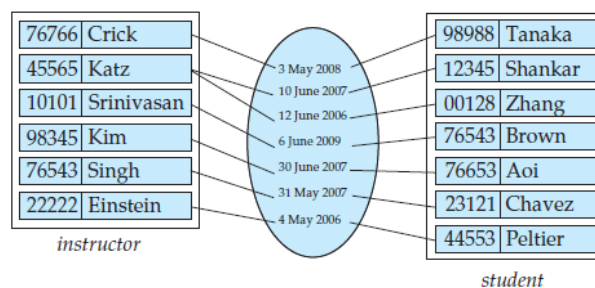


Figure 5.3 *date as attribute of the advisor relationship set.*

became an advisor of a particular student. The single-valued attribute *date* can store a single *date* only. We cannot represent multiple dates by multiple relation-ship instances between the same instructor and a student, since the relationship instances would not be uniquely identifiable using only the participating entities. The right way to handle this case is to create a multivalued attribute *date*, which can store all the dates.

It is possible to have more than one relationship set involving the same entity sets. In our example, the *instructor* and *student* entity sets participate in the relationship set *advisor*. Additionally, suppose each student must have another instructor who serves as a department advisor (undergraduate or graduate). Then the *instructor* and *student* entity sets may participate in another relationship set, *dept-advisor*.

The relationship sets *advisor* and *dept-advisor* provide examples of a **binary** relationship set—that is, one that involves two entity sets. Most of the relationship sets in a database system are binary. Occasionally, however, relationship sets involve more than two entity sets.

As an example, suppose that we have an entity set *project* that represents all the research projects carried out in the university. Consider the entity sets *instructor*, *student*, and *project*. Each project can have multiple associated students and multiple associated instructors. Furthermore, each student working on a project must have an associated instructor who guides the student on the project. For now, we ignore the first two relationships, between project and instructor, and between project and student. Instead, we focus on the information about which instructor is guiding which student on a particular project. To represent this information, we relate the three entity sets through the relationship set *proj-guide*, which indicates that a particular student is guided by a particular instructor on a particular project.

Note that a student could have different instructors as guides for different projects, which cannot be captured by a binary relationship between students and instructors.

The number of entity sets that participate in a relationship set is the **degree** of the relationship set. A binary relationship set is of degree 2; a ternary relationship set is of degree 3.

5.2.3 Attributes

For each attribute, there is a set of permitted values, called the **domain**, or **value set**, of that attribute. The domain of attribute *courseid* might be the set of all text strings of

a certain length. Similarly, the domain of attribute *semester* might be strings from the set {Fall, Winter, Spring, Summer}.

Formally, an attribute of an entity set is a function that maps from the entity set into a domain. Since an entity set may have several attributes, each entity can be described by a set of (attribute, data value) pairs, one pair for each attribute of the entity set. For example, a particular *instructor* entity may be described by the set $\{(ID, 76766), (name, Crick), (deptname, Biology), (salary, 72000)\}$, meaning that the entity describes a person named Crick whose instructor *ID* is 76766, who is a member of the Biology department with salary of \$72,000. We can see, at this point, an integration of the abstract schema with the actual enterprise being modeled. The attribute values describing an entity constitute a significant portion of the data stored in the database.

An attribute, as used in the E-R model, can be characterized by the following attribute types.

- **Simple** and **composite** attributes. In our examples thus far, the attributes have been simple; that is, they have not been divided into subparts. **Composite** attributes, on the other hand, can be divided into subparts (that is, other attributes). For example, an attribute *name* could be structured as a composite attribute consisting of *firstname*, *middleinitial*, and *lastname*. Using composite attributes in a design schema is a good choice if a user will wish to refer to an entire attribute on some occasions, and to only a component of the attribute on other occasions. Suppose we were to add an address to the *student* entity-set. The address can be defined as the composite attribute *address* with the attributes *street*, *city*, *state*, and *zipcode*.³ Composite attributes help us to group together related attributes, making the modeling cleaner.

Note also that a composite attribute may appear as a hierarchy. In the composite attribute *address*, its component attribute *street* can be further divided into *street-number*, *street-name*, and *apartment-number*. Figure 7.4 depicts these examples of composite attributes for the *instructor* entity set.

- **Single-valued** and **multivalued** attributes. The attributes in our examples all have a single value for a particular entity. For instance, the *studentID* attribute for a specific student entity refers to only one student *ID*. Such attributes are said to be **single valued**. There may be instances where an attribute has a set of values for a specific entity. Suppose we add to the *instructor* entity set

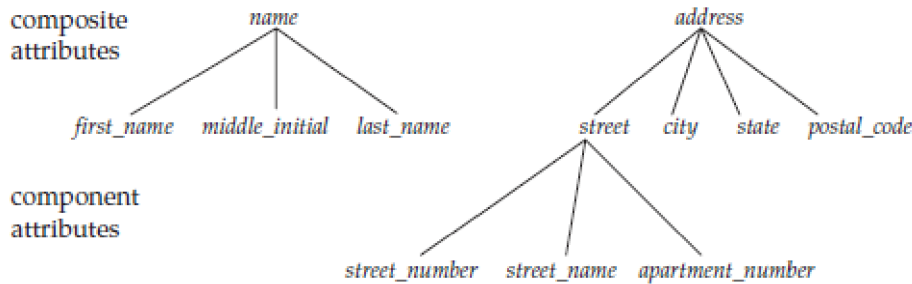


Figure 5.4 Composite attributes instructor name and address.

a *phone-number* attribute. An *instructor* may have zero, one, or several phone numbers, and different instructors may have different numbers of phones. This type of attribute is said to be **multivalued**. As another example, we could add to the *instructor* entity set an attribute *dependent-name* listing all the dependents. This attribute would be multivalued, since any particular instructor may have zero, one, or more dependents.

To denote that an attribute is multivalued, we enclose it in braces, for example *{phone-number}* or *{dependent-name}*.

Where appropriate, upper and lower bounds may be placed on the number of values in a multivalued attribute. For example, a university may limit the number of phone numbers recorded for a single instructor to two. Placing bounds in this case expresses that the *phone-number* attribute of the *instructor* entity set may have between zero and two values.

- **Derived** attribute. The value for this type of attribute can be derived from the values of other related attributes or entities. For instance, let us say that the *instructor* entity set has an attribute *students-advised*, which represents how many students an instructor advises. We can derive the value for this attribute by counting the number of *student* entities associated with that instructor.

As another example, suppose that the *instructor* entity set has an attribute *age* that indicates the instructor's age. If the *instructor* entity set also has an attribute *dateof-birth*, we can calculate *age* from *date-of-birth* and the current date. Thus, *age* is a derived attribute. In this case, *dateof-birth* may be referred to as a *base* attribute, or a *stored* attribute. The value of a derived attribute is not stored but is computed when required.

An attribute takes a **null** value when an entity does not have a value for it. The *null* value may indicate “not applicable”—that is, that the value does not exist for the entity. For example, one may have no middle name. *Null* can also designate that an attribute value is unknown. An unknown value may be either *missing* (the value does exist, but we do not have that information) or *not known* (we do not know whether or not the value actually exists).

For instance, if the *name* value for a particular instructor is *null*, we assume that the value is missing, since every instructor must have a name. A null value for the *apartmentnumber* attribute could mean that the address does not include

an apartment number (not applicable), that an apartment number exists but we do not know what it is (missing), or that we do not know whether or not an apartment number is part of the instructor's address (unknown).

5.3 Constraints

An E-R enterprise schema may define certain constraints to which the contents of a database must conform. In this section, we examine mapping cardinalities and participation constraints.

5.3.1 Mapping Cardinalities

Mapping cardinalities, or cardinality ratios, express the number of entities to which another entity can be associated via a relationship set.

Mapping cardinalities are most useful in describing binary relationship sets, although they can contribute to the description of relationship sets that involve more than two entity sets. In this section, we shall concentrate on only binary relationship sets.

For a binary relationship set R between entity sets A and B , the mapping cardinality must be one of the following:

- **One-to-one.** An entity in A is associated with *at most* one entity in B , and an entity in B is associated with *at most* one entity in A . (See Figure 5.5a.)
- **One-to-many.** An entity in A is associated with any number (zero or more) of entities in B . An entity in B , however, can be associated with *at most* one entity in A . (See Figure 5.5b.)

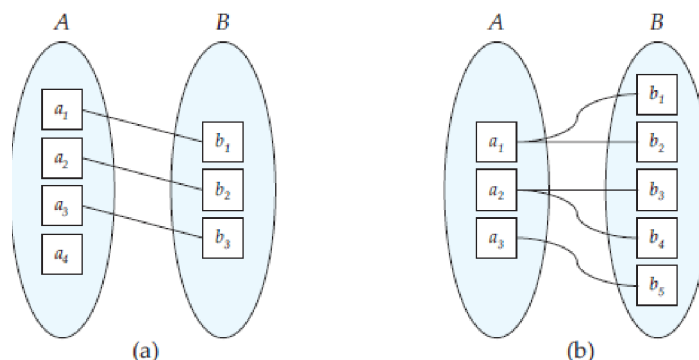


Figure 5.5 Mapping cardinalities. (a) One-to-one. (b) One-to-many.

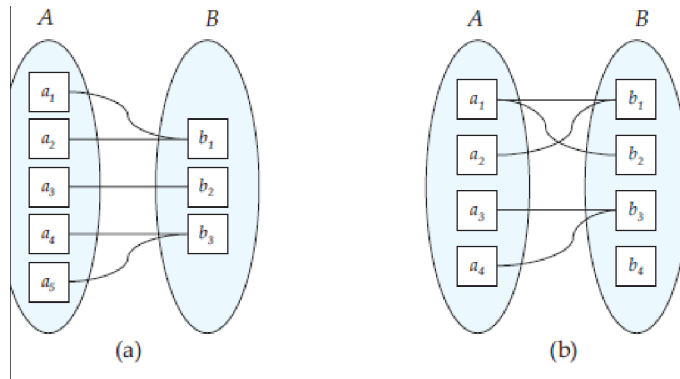


Figure 5.6 Mapping cardinalities. (a) Many-to-one. (b) Many-to-many.

- **Many-to-one.** An entity in *A* is associated with *at most* one entity in *B*. An entity in *B*, however, can be associated with any number (zero or more) of entities in *A*. (See Figure 5.6a.)
- **Many-to-many.** An entity in *A* is associated with any number (zero or more) of entities in *B*, and an entity in *B* is associated with any number (zero or more) of entities in *A*. (See Figure 5.6b.)

The appropriate mapping cardinality for a particular relationship set obviously depends on the real-world situation that the relationship set is modeling.

As an illustration, consider the *advisor* relationship set. If, in a particular university, a student can be advised by only one instructor, and an instructor can advise several students, then the relationship set from *instructor* to *student* is one-to-many. If a student can be advised by several instructors (as in the case of students advised jointly), the relationship set is many-to-many.

5.3.2 Participation Constraints

The participation of an entity set *E* in a relationship set *R* is said to be **total** if every entity in *E* participates in at least one relationship in *R*. If only some entities in *E* participate in relationships in *R*, the participation of entity set *E* in relationship *R* is said to be **partial**. In Figure 7.5a, the participation of *B* in the relationship set is total while the participation of *A* in the relationship set is partial. In Figure 5.5b, the participation of both *A* and *B* in the relationship set are total.

For example, we expect every *student* entity to be related to at least one instructor through the *advisor* relationship. Therefore the participation of *student* in the relationship set *advisor* is total. In contrast, an *instructor* need not advise any students. Hence, it is possible that only some of the *instructor* entities are related to the *student* entity set through

the *advisor* relationship, and the participation of *instructor* in the *advisor* relationship set is therefore partial.

5.3.3 Keys

We must have a way to specify how entities within a given entity set are distinguished. Conceptually, individual entities are distinct; from a database perspective, however, the differences among them must be expressed in terms of their attributes.

Therefore, the values of the attribute values of an entity must be such that they can *uniquely identify* the entity. In other words, no two entities in an entity set are allowed to have exactly the same value for all attributes.

The notion of a *key* for a relation schema, as defined in Section 2.3, applies directly to entity sets. That is, a key for an entity is a set of attributes that suffice to distinguish entities from each other. The concepts of superkey candidate key, and primary key are applicable to entity sets just as they are applicable to relation schemas.

Keys also help to identify relationships uniquely, and thus distinguish relationships from each other. Below, we define the corresponding notions of keys for relationships.

The primary key of an entity set allows us to distinguish among the various entities of the set. We need a similar mechanism to distinguish among the various relationships of a relationship set.

Let R be a relationship set involving entity sets $E_1, E_2, \dots, E_n$. Let $primary-key(E_i)$ denote the set of attributes that forms the primary key for entity set E_i . Assume for now that the attribute names of all primary keys are unique. The composition of the primary key for a relationship set depends on the set of attributes associated with the relationship set R .

If the relationship set R has no attributes associated with it, then the set of attributes

$$primary-key(E_1) \cup primary-key(E_2) \cup \dots \cup primary-key(E_n)$$

describes an individual relationship in set R .

If the relationship set R has attributes $a_1, a_2, \dots, a_m$ associated with it, then the set of attributes

$$primary-key(E_1) \cup primary-key(E_2) \cup \dots \cup primary-key(E_n) \cup \{a_1, a_2, \dots, a_m\}$$

describes an individual relationship in set R .

In both of the above cases, the set of attributes

$$\text{primary-key}(E_1) \cup \text{primary-key}(E_2) \cup \dots \cup \text{primary-key}(E_n)$$

forms a superkey for the relationship set.

If the attribute names of primary keys are not unique across entity sets, the attributes are renamed to distinguish them; the name of the entity set combined with the name of the attribute would form a unique name. If an entity set participates more than once in a relationship set (as in the *prereq* relationship in Section 5.2.2), the role name is used instead of the name of the entity set, to form a unique attribute name.

The structure of the primary key for the relationship set depends on the mapping cardinality of the relationship set. As an illustration, consider the entity sets *instructor* and *student*, and the relationship set *advisor*, with attribute *date*, in Section 7.2.2. Suppose that the relationship set is many-to-many. Then the primary key of *advisor* consists of the union of the primary keys of *instructor* and *student*. If the relationship is many-to-one from *student* to *instructor*—that is, each student can have at most one advisor—then the primary key of *advisor* is simply the primary key of *student*. However, if an instructor can advise only one student—that is, if the *advisor* relationship is many-to-one from *instructor* to *student*—then the primary key of *advisor* is simply the primary key of *instructor*. For one-to-one relationships either candidate key can be used as the primary key.

5.4 Entity-Relationship Diagrams

As we saw briefly in Section 1.3.3, an **E-R diagram** can express the overall logical structure of a database graphically. E-R diagrams are simple and clear—qualities that may well account in large part for the widespread use of the E-R model. Such a diagram consists of the following major components:

- Rectangles, which represent entity sets
- Ellipses, which represent attributes
- Diamonds, which represent relationship sets
- Lines, which link attributes to entity sets and entity sets to relationship sets
- Double ellipses, which represent multivalued attributes
- Dashed ellipses, which denote derived attributes
- Double lines, which indicate total participation of an entity in a relationship set
- Double rectangles, which represent weak entity set (described later)

Consider the entity - relationship diagram in Figure 5.7, which consists of two entity sets, customer and loan, related through a binary relationship set borrower. The attributes associated with customer are customer_id, customer_name, customer_street, and customer_city. The attributes associated with loan are loan_number and amount. In Figure 5.7, attributes of an entity set that are members of the primary key are underlined.

The relationship set borrower may be many-to-many, one-to-many, many-to-one, or one-to-one. To distinguish among these types, we draw either a directed line ($\rightarrow$) or an undirected line ($-$) between the relationship set and the entity set in question.

- A directed line from the relationship set borrower to the entity set loan specified that borrower is either a one-to-one or many-to-one relationship set, from customer to loan; borrower cannot be a many-to-many or a one-to-many relationship set from customer to loan.
- An undirected line from the relationship set borrower to the entity set loan specifies that borrower is either a many-to-many or one-to-many relationship set from customer to loan.

Returning to the E-R diagram of Figure 5.7, we see that the relationship set borrower is many-to-many. If the relationship set borrower were one-to-many, from customer to loan, then the line from borrower to customer would be directed, with an arrow pointing to the customer entity set (Figure 6.8a). Similarly, if the relationship set borrower were many-to-one from customer to loan, then the line from borrower to loan would have an arrow

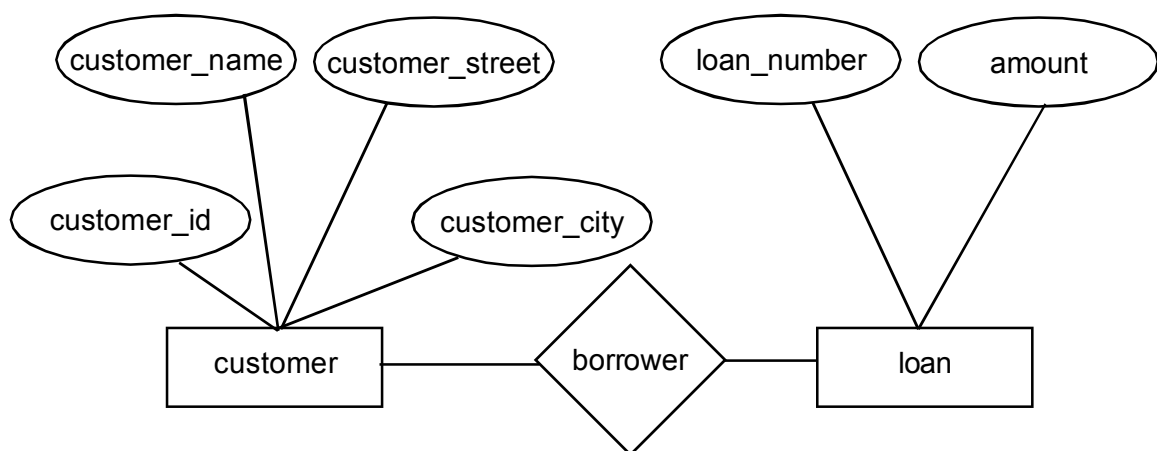
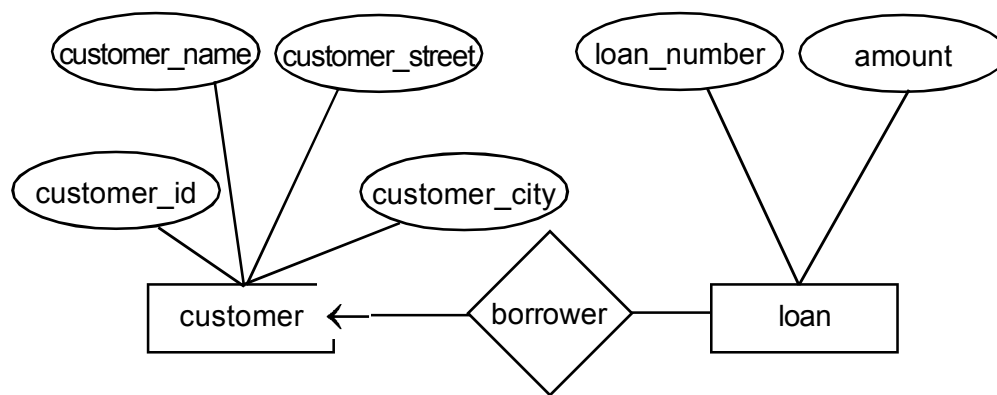


Figure 5.7 E-R diagram corresponding to customers and loans

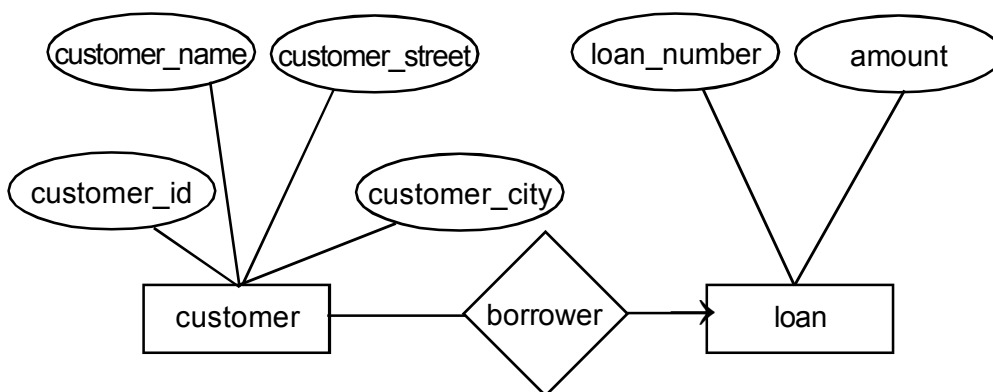
pointing to the loan entity set (Figure 5.8b). Finally, if the relationship set borrower were one-to-one, then both lines from borrower would have arrows: one pointing to the loan entity set and one pointing to the customer entity set (Figure 5.8c).

If a relationship set has also some attributes associated with it, then we like these attributes to the relationship set. For example, in Figure 5.98, we have the `access.date` descriptive attribute attached to the relationship set `depositor` to specify the most recent date on which a customer accessed that account.

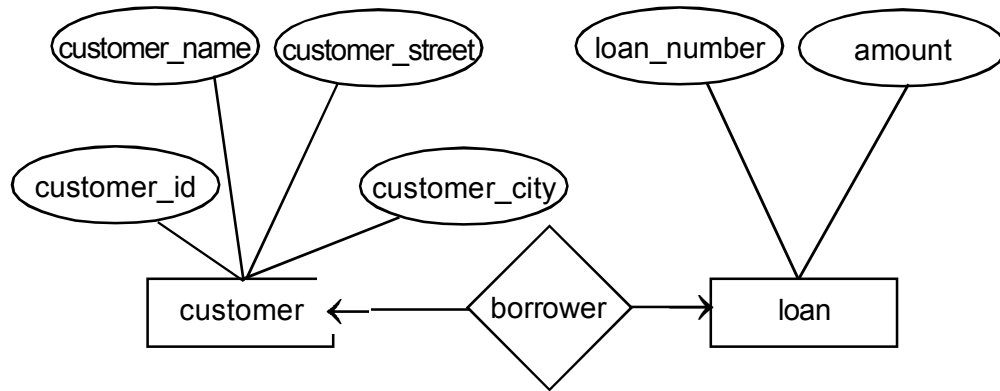
Figure 5.10 shows how composite attributes can be represented in the E-R notation. Here, a composite attribute name, with component attributes `first_name`, `middle_initial`, and `last_name` replaces the simple attribute `customer_name` of `customer`. Also, a composite



(a)



(b)



(c)

Figure 5.8 Relationships. (a) One-to-many (b) Many-to-one (c) One-to-one

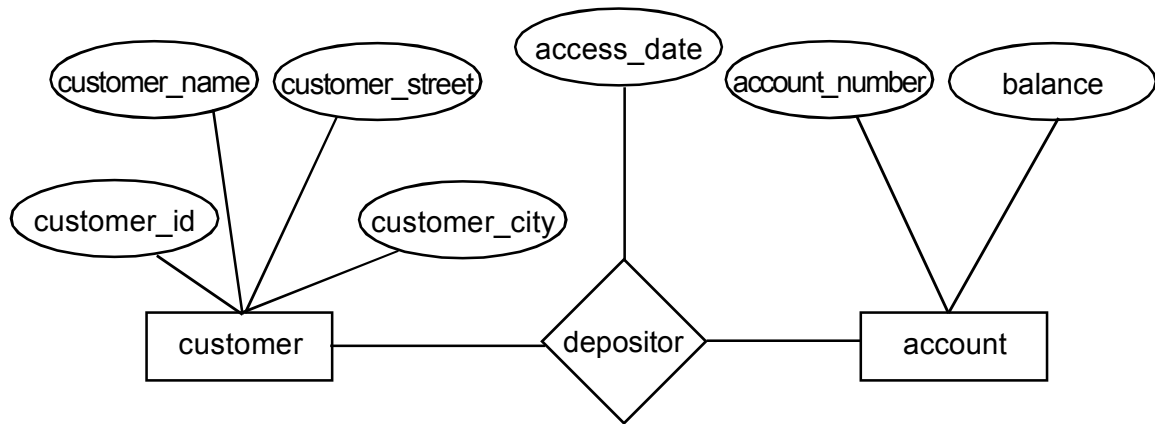


Figure 5.9 E-R diagram with an attribute attached to a relationship set

attributes address, whose component attributes are street, city, state and zip_code replaces the attributes customer_street and customer_city of customer. The attribute street is itself a composite attribute whose component attributes are street_number, street_name, and apartment_number.

Figure 5.10 also illustrates a multivalued attribute phone_number, depicted by a double ellipse, and a derived attribute age, depicted by a dashed ellipse.

We indicate roles in E-R diagrams by labeling the lines that connect diamonds to rectangles. Figure 5.11 shows the role indicators manager and worker between the employee entity set and the works for relationship set.

Nonbinary relationship sets can be specified easily in an E-R diagram. Figure 5.123 consists of the three entity sets employee, job and branch, related through the relationship set works.on.

We can specify some types of many-to-one relationships in the case of nonbinary relationship sets. Suppose an employee can have at most one job in each branch for

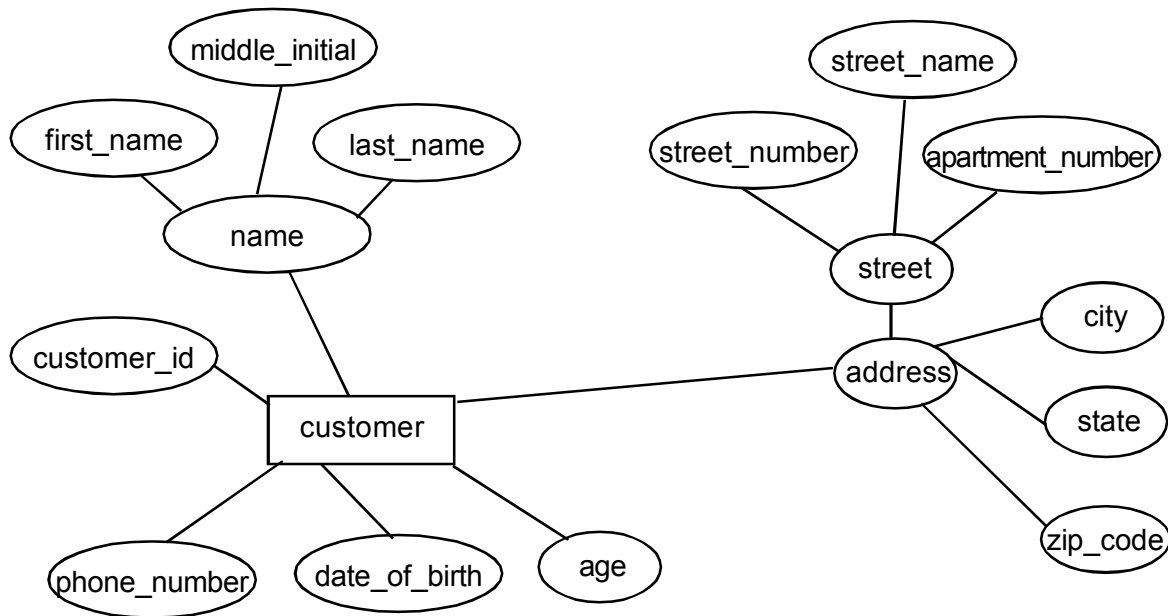


Figure 5.10 E-R diagram with composite, multivalued and derived attributes

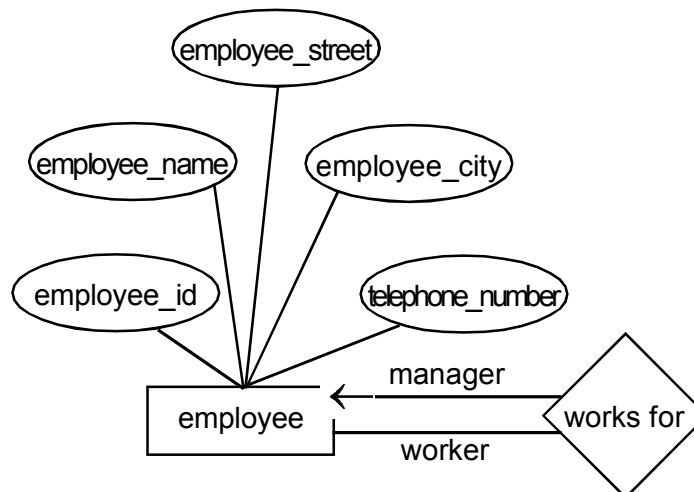


Figure 5.11 E-R diagram with role indicators

example, Jones cannot be a manager and an auditor at the same branch). This constraint can be specified by an arrow pointing to job on the edge from works.on.

We permit at most one arrow out of a relationship set, since an E-R diagram with two or more arrows out of a nonbinary relationship set can be interpreted in two ways. Suppose there is a relationship set R between entity sets $A_1, A_2, \dots, A_n$, and the only arrows are on the edges to entity sets $A_{i+1}, A_{i+2}, \dots, A_n$. Then, the two possible interpretations are:

1. A particular combination of entities from $A_1, A_2, \dots, A_i$ can be associated with at most one combination of entities from $A_{i+1}, A_{i+2}, \dots, A_n$. Thus, the primary key for the relationship R can be constructed by the union of the primary key of $A_1, A_2, \dots, A_i$.
2. For each entity set $A_k, i < k \leq n$, each combination of the entities from the other entity sets can be associated with at most one entity from A_k . Each set $\{A_1, A_2, \dots, A_{k-1}, A_{k+1}, \dots, A_n\}$, for $i < k \leq n$, then forms a candidate key.

Each of these interpretations has been used in different books and systems. To avoid confusion, we permit only one arrow out of a relationship set, in which case the two

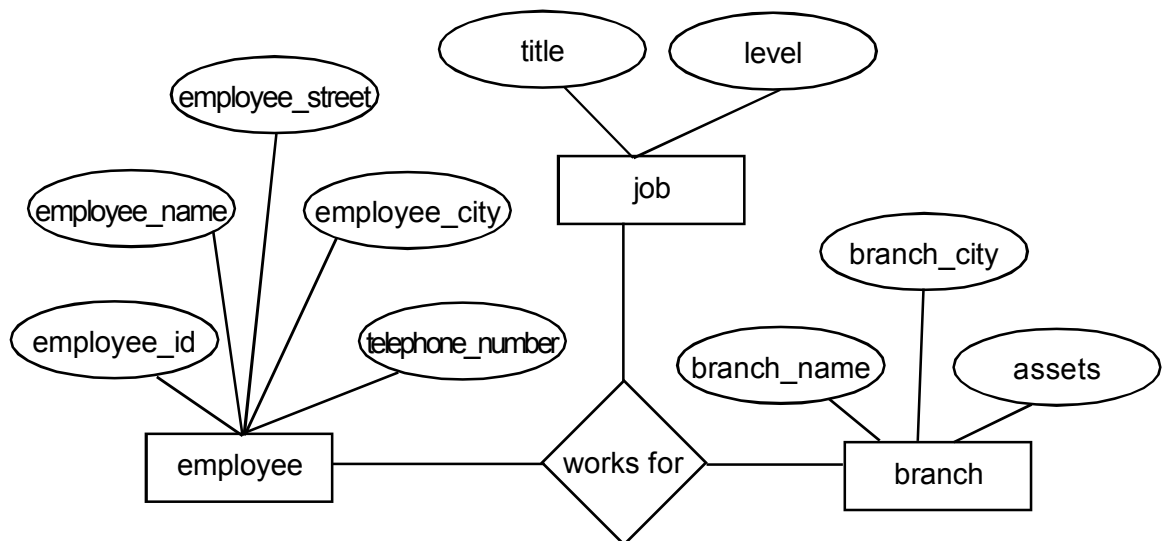


Figure 5.12 E-R diagram with a ternary relationship

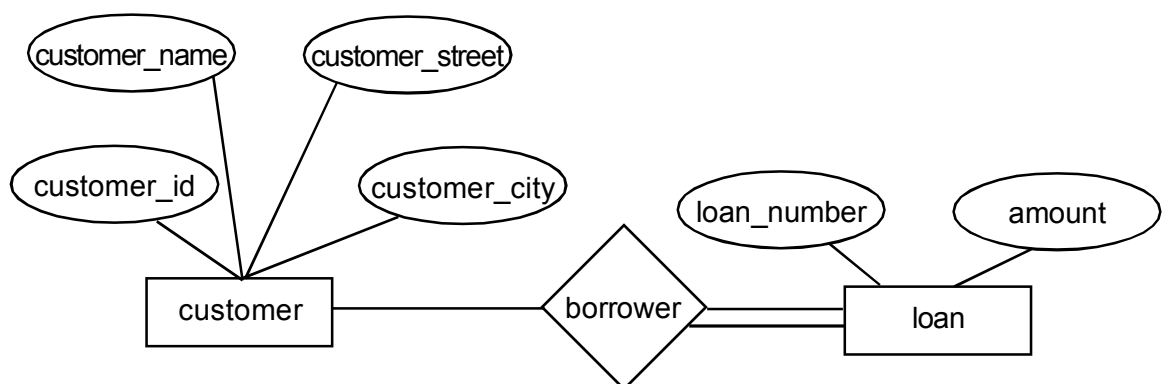


Figure 5.13 Total participation of an entity set in a relationship set

interpretations are equivalent. We study the notion of functional dependencies, which allow either of these interpretations to be specified in an unambiguous manner.

Double lines are used in an E-R diagram to indicate that the participation of an entity set in a relationship set is total. that is, each entity in the entity set occurs in at least one relationship in that relationship set. For instance, consider the relationship borrower between customers and loans. A double line from loan to borrower, as in Figure 5.13, indicates that each loan must have at least one associated customer.

E-R diagrams also provide a way to indicate more complex constraints on the number of times each entity participates in relationships in a relationship set. An edge between an entity set and a binary relationships set can have an associated minimum and maximum cardinality, shown in the form $l..h$, where l is the minimum and h the maximum cardinality. A minimum value of 1 indicates total participation of the entity set in the relationship set. A maximum value of 1 indicates that the entity participates in at most one relationship, while a maximum value $*$ indicates no limit. Note that a label $1..*$ on an edge is equivalent to a double line.

For example, consider Figure 5.14. The edge between loan and borrower has a cardinality constraint of $1..1$, meaning the minimum and the maximum cardinality are both 1. That is, each loan must have exactly one associated customer. The limit $0..*$ on the edge from customer to borrower indicates that a customer can have zero or more loans. Thus, the relationship borrower is one-to-many from customer to loan, and further the participation of loan in borrower is total.

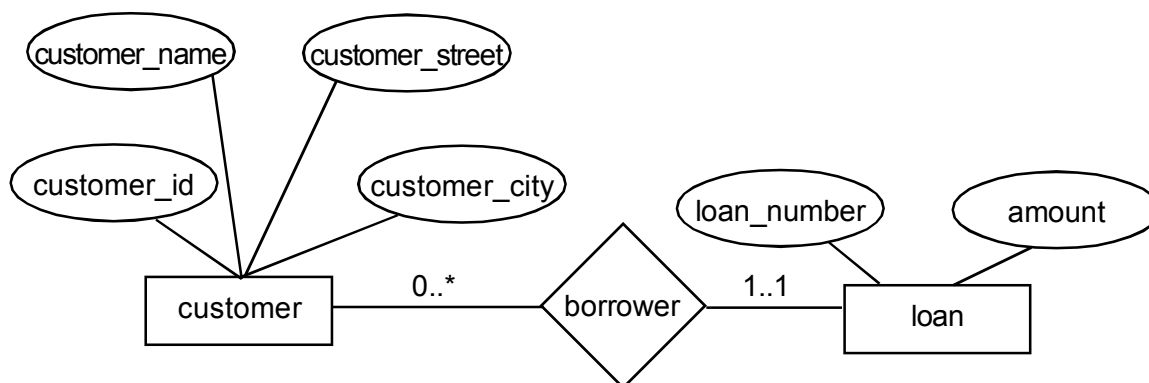


Figure 5.14 Cardinality limits on relationship sets

5.5 Weak Entity Sets

An entity set that does not have sufficient attributes to form a primary key is termed a **weak entity set**. An entity set that has a primary key is termed a **strong entity set**.

For a weak entity set to be meaningful, it must be associated with another entity set, called the **identifying** or **owner entity set**. Every weak entity must be associated with an identifying entity; that is, the weak entity set is said to be **existence dependent** on the identifying entity set. The identifying entity set is said to **own** the weak entity set that it identifies. The relationship associating the weak entity set with the identifying entity set is called the **identifying relationship**.

The identifying relationship is many-to-one from the weak entity set to the identifying entity set, and the participation of the weak entity set in the relationship is total. The identifying relationship set should not have any descriptive attributes, since any such attributes can instead be associated with the weak entity set.

In our example, the identifying entity set for *section* is *course*, and the relationship *seccourse*, which associates *section* entities with their corresponding *course* entities, is the identifying relationship.

Although a weak entity set does not have a primary key, we nevertheless need a means of distinguishing among all those entities in the weak entity set that depend on one particular strong entity. The **discriminator** of a weak entity set is a set of attributes that allows this distinction to be made. For example, the discriminator of the weak entity set *section* consists of the attributes *secid*, *year*, and *semester*, since, for each course, this set of attributes uniquely identifies one single section for that course. The discriminator of a weak entity set is also called the *partial key* of the entity set.

The primary key of a weak entity set is formed by the primary key of the identifying entity set, plus the weak entity set's discriminator. In the case of the entity set *section*, its primary key is $\{courseid, secid, year, semester\}$, where *course id* is the primary key of the identifying entity set, namely *course*, and $\{secid, year, semester\}$ distinguishes *section* entities for the same course.

Note that we could have chosen to make *secid* globally unique across all courses offered in the university, in which case the *section* entity set would have

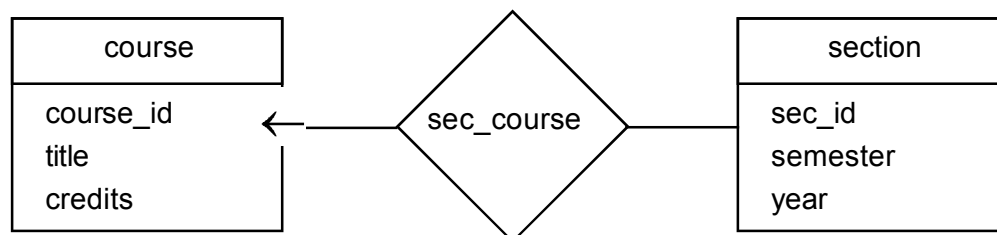


Figure 5.15 E-R diagram with a weak entity set.

had a primary key. However, conceptually, a *section* is still dependent on a *course* for its existence, which is made explicit by making it a weak entity set.

In E-R diagrams, a weak entity set is depicted via a rectangle, like a strong entity set, but there are two main differences:

- The discriminator of a weak entity is underlined with a dashed, rather than a solid, line.
- The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond.

In Figure 5.15, the weak entity set *section* depends on the strong entity set *course* via the relationship set *seccourse*.

The figure also illustrates the use of double lines to indicate *total participation*; the participation of the (weak) entity set *section* in the relationship *seccourse* is total, meaning that every section must be related via *sec-course* to some course. Finally, the arrow from *seccourse* to *course* indicates that each section is related to a single course.

A weak entity set can participate in relationships other than the identifying relationship. For instance, the *section* entity could participate in a relationship with the *timeslot* entity set, identifying the time when a particular class section meets. A weak entity set may participate as owner in an identifying relationship with another weak entity set. It is also possible to have a weak entity set with more than one identifying entity set. A particular weak entity would then be identified by a combination of entities, one from each identifying entity set. The primary key of the weak entity set would consist of the union of the primary keys of the identifying entity sets, plus the discriminator of the weak entity set.

In some cases, the database designer may choose to express a weak entity set as a multivalued composite attribute of the owner entity set. In our example, this alternative would require that the entity set *course* have a multivalued, composite attribute *section*. A weak entity set may be more appropriately modeled as an attribute if it participates in only the identifying relationship, and if it has few attributes. Conversely, a weak entity set representation more aptly models a situation where the set participates in relationships other than the identifying relationship, and where the weak entity set has several attributes. It is clear that *section* violates the requirements for being modeled as a multivalued composite attribute, and is modeled more aptly as a weak entity set.

5.6 Reduction to Relational Schemas

We can represent a database that conforms to an E-R database schema by a collection of relation schemas. For each entity set and for each relationship set in the database design, there is a unique relation schema to which we assign the name of the corresponding entity set or relationship set.

Both the E-R model and the relational database model are abstract, logical representations of real-world enterprises. Because the two models employ similar design principles, we can convert an E-R design into a relational design.

In this section, we describe how an E-R schema can be represented by relation schemas and how constraints arising from the E-R design can be mapped to constraints on relation schemas.

5.6.1 Representation of Strong Entity Sets with Simple Attributes

Let E be a strong entity set with only simple descriptive attributes $a_1, a_2, \dots, a_n$. We represent this entity by a schema called E with n distinct attributes. Each tuple in a relation on this schema corresponds to one entity of the entity set E .

For schemas derived from strong entity sets, the primary key of the entity set serves as the primary key of the resulting schema. This follows directly from the fact that each tuple corresponds to a specific entity in the entity set.

As an illustration, consider the entity set *student* of the E-R diagram in Figure 5.15. This entity set has three attributes: *ID*, *name*, *totcred*. We represent this entity set by a schema called *student* with three attributes:

student (*ID*, *name*, *totcred*)

Note that since *student ID* is the primary key of the entity set, it is also the primary key of the relation schema.

Continuing with our example, for the E-R diagram in Figure 5.15, all the strong entity sets, except *timeslot*, have only simple attributes. The schemas derived from these strong entity sets are:

classroom (*building*, *room-number*, *capacity*) *department* (*deptname*, *building*, *budget*) *course* (*courseid*, *title*, *credits*) *instructor* (*ID*, *name*, *salary*) *student* (*ID*, *name*, *totcred*)

As you can see, both the *instructor* and *student* schemas are different from the schemas we have used in the previous chapters (they do not contain the attribute *deptname*). We shall revisit this issue shortly.

5.6.2 Representation of Strong Entity Sets with Complex Attributes

When a strong entity set has nonsimple attributes, things are a bit more complex. We handle composite attributes by creating a separate attribute for each of the component attributes; we do not create a separate attribute for the composite attribute itself. To illustrate, consider the version of the *instructor* entity set depicted. For the composite attribute *name*, the schema generated for *instructor* contains the attributes *firstname*, *middlename*, and *last-name*; there is no separate attribute or schema for *name*. Similarly, for the composite attribute *address*, the schema generated contains the attributes *street*, *city*, *state*, and *zipcode*. Since *street* is a composite attribute it is replaced by *street-number*, *street-name*, and *apt-number*.

Multivalued attributes are treated differently from other attributes. We have seen that attributes in an E-R diagram generally map directly into attributes for the appropriate relation schemas. Multivalued attributes, however, are an exception; new relation schemas are created for these attributes, as we shall see shortly.

Derived attributes are not explicitly represented in the relational data model. However, they can be represented as “methods” in other data models such as the object-relational data model, which is described later.

The relational schema derived from the version of entity set *instructor* with complex attributes, without including the multivalued attribute, is thus:

instructor (*ID*, *first-name*, *middle-name*, *last-name*, *street-number*, *street name*, *apt-number*, *city*, *state*, *zipcode*, *dateof-birth*)

For a multivalued attribute *M*, we create a relation schema *R* with an attribute *A* that corresponds to *M* and attributes corresponding to the primary key of the entity set or relationship set of which *M* is an attribute.

As an illustration, consider the E-R diagram in Figure 7.11 that depicts the entity set *instructor*, which includes the multivalued attribute *phone-number*. The primary key of *instructor* is *ID*. For this multivalued attribute, we create a relation schema

instructorjphone (*ID*, *phone-number*)

Each phone number of an instructor is represented as a unique tuple in the relation on this schema. Thus, if we had an instructor with *ID* 22222, and phone numbers 555-1234 and 555-4321, the relation *instructor jphone* would have two tuples (22222, 555-1234) and (22222, 555-4321).

We create a primary key of the relation schema consisting of all attributes of the schema. In the above example, the primary key consists of both attributes of the relation *instructor jphone*.

In addition, we create a foreign-key constraint on the relation schema created from the multivalued attribute, with the attribute generated from the primary key of the entity set referencing the relation generated from the entity set. In the above example, the foreign-key constraint on the *instructor jphone* relation would be that attribute *ID* references the *instructor* relation.

In the case that an entity set consists of only two attributes — a single primary-key attribute *B* and a single multivalued attribute *M* — the relation schema for the entity set would contain only one attribute, namely the primary-key attribute *B*. We can drop this relation, while retaining the relation schema with the attribute *B* and attribute *A* that corresponds to *M*.

To illustrate, consider the entity set *timeslot* depicted in Figure 7.15. Here, *time slotid* is the primary key of the *timeslot* entity set and there is a single multivalued attribute that happens also to be composite. The entity set can be represented by just the following schema created from the multivalued composite attribute:

timeslot (timeslotid, day, starttime, endtime)

Although not represented as a constraint on the E-R diagram, we know that there cannot be two meetings of a class that start at the same time of the same day-of-the-week but end at different times; based on this constraint, *endtime* has been omitted from the primary key of the *timeslot* schema.

The relation created from the entity set would have only a single attribute *time slotid*; the optimization of dropping this relation has the benefit of simplifying the resultant database schema, although it has a drawback related to foreign keys, which we briefly discuss in Section 7.6.4.

5.6.3 Representation of Weak Entity Sets

Let *A* be a weak entity set with attributes $a_1, a_2, \dots, a_m$. Let *B* be the strong entity set on which *A* depends. Let the primary key of *B* consist of attributes $b_1, b_2, \dots, b_n$. We represent the entity set *A* by a relation schema called *A* with one attribute for each member of the set:

$$\{a_1, a_2, \dots, a_m\} \cup \{b_1, b_2, \dots, b_n\}$$

For schemas derived from a weak entity set, the combination of the primary key of the strong entity set and the discriminator of the weak entity set serves as the primary key of the schema. In addition to creating a primary key, we also create a foreign-key constraint on the relation *A*, specifying that the attributes *b1, b2, ..., bn* reference the primary key of the relation *B*. The foreign-key constraint ensures that for each tuple representing a weak entity, there is a corresponding tuple representing the corresponding strong entity.

As an illustration, consider the weak entity set *section* in the E-R diagram of Figure 7.15. This entity set has the attributes: *secid*, *semester*, and *year*. The primary key of the *course* entity set, on which *section* depends, is *course-id*. Thus, we represent *section* by a schema with the following attributes:

$$\text{section}(\text{courseid}, \text{secid}, \text{semester}, \text{year})$$

The primary key consists of the primary key of the entity set *course*, along with the discriminator of *section*, which is *secid*, *semester*, and *year*. We also create a foreign-key constraint on the *section* schema, with the attribute *courseid* referencing the primary key of the *course* schema, and the integrity constraint “on delete cascade”.⁷ Because of the “on delete cascade” specification on the foreign key constraint, if a *course* entity is deleted, then so are all the associated *section* entities.

5.6.4 Representation of Relationship Sets

Let *R* be a relationship set, let *a1, a2, ..., am* be the set of attributes formed by the union of the primary keys of each of the entity sets participating in *R*, and let the descriptive attributes (if any) of *R* be *b1, b2, ..., bn*. We represent this relationship set by a relation schema called *R* with one attribute for each member of the set:

$$\{a_1, a_2, \dots, a_m\} \cup \{b_1, b_2, \dots, b_n\}$$

We described earlier, in Section 7.3.3, how to choose a primary key for a binary relationship set. As we saw in that section, taking all the primary-key attributes from all the related entity sets serves to identify a particular tuple, but for one-to-one, many-to-one, and one-to-many relationship sets, this turns out to be a larger set of attributes than we need in the primary key. The primary key is instead chosen as follows:

- For a binary many-to-many relationship, the union of the primary-key attributes from the participating entity sets becomes the primary key.

- For a binary one-to-one relationship set, the primary key of either entity set can be chosen as the primary key. The choice can be made arbitrarily.
- For a binary many-to-one or one-to-many relationship set, the primary key of the entity set on the “many” side of the relationship set serves as the primary key.
- For an n -ary relationship set without any arrows on its edges, the union of the primary key-attributes from the participating entity sets becomes the primary key.
- For an n -ary relationship set with an arrow on one of its edges, the primary keys of the entity sets not on the “arrow” side of the relationship set serve as the primary key for the schema. Recall that we allowed only one arrow out of a relationship set.

We also create foreign-key constraints on the relation schema R as follows: For each entity set E_i related to relationship set R , we create a foreign-key constraint from relation schema R , with the attributes of R that were derived from primary-key attributes of E_i referencing the primary key of the relation schema representing E_i .

As an illustration, consider the relationship set *advisor* in the E-R diagram of Figure 5.15. This relationship set involves the following two entity sets:

- *instructor* with the primary key *ID*.
- *student* with the primary key *ID*.

Since the relationship set has no attributes, the *advisor* schema has two attributes, the primary keys of *instructor* and *student*. Since both attributes have the same name, we rename them *iID* and *sID*. Since the *advisor* relationship set is many-to-one from *student* to *instructor* the primary key for the *advisor* relation schema is *sID*.

We also create two foreign-key constraints on the *advisor* relation, with attribute *iID* referencing the primary key of *instructor* and attribute *sID* referencing the primary key of *student*.

Observe that for the case of the relationship set *prereq*, the role indicators associated with the relationship are used as attribute names, since both roles refer to the same relation *course*.

Similar to the case of *advisor*, the primary key for each of the relations *seccourse*, *sectimeslot*, *secclass*, *inst-dept*, *studdept* and *coursedept* consists of the primary key of only one of the two related entity sets, since each of the corresponding relationships is many-to-one.

Foreign keys are not shown in Figure 5.16, but for each of the relations in the figure there are two foreign-key constraints, referencing the two relations created from the two related entity sets. Thus, for example, *seccourse* has foreign keys referencing *section* and *classroom*, *teaches* has foreign keys referencing *instructor* and *section*, and *takes* has foreign keys referencing *student* and *section*.

The optimization that allowed us to create only a single relation schema from the entity set *timeslot*, which had a multivalued attribute, prevents the creation of a foreign key from the relation schema *sectimeslot* to the relation created from entity set *timeslot*, since we dropped the relation created from the entity set *time*

teaches (*ID*, *courseid*, *secid*, *semester*, *year*)
takes (*ID*, *course id*, *secid*, *semester*, *year*, *grade*)
prereq (*courseid*, *prereqid*)
advisor (*sID*, *iID*)
seccourse (*courseid*, *secid*, *semester*, *year*)
sectimeslot (*courseid*, *secid*, *semester*, *year*, *timeslotid*)
secclass (*courseid*, *secid*, *semester*, *year*, *building*, *room-number*)
instdept (*ID*, *deptname*)
studdept (*ID*, *deptname*)
coursedept (*courseid*, *deptname*)

Figure 5.16 Schemas derived from relationship sets in the E-R diagram in Figure 5.15.

slot. We retained the relation created from the multivalued attribute, and named it *timeslot*, but this relation may potentially have no tuples corresponding to a *timeslotid*, or may have multiple tuples corresponding to a *timeslotid*; thus, *time slotid* in *sectimeslot* cannot reference this relation.

The astute reader may wonder why we have not seen the schemas *seccourse*, *sectimeslot*, *secclass*, *instdept*, *studdept*, and *coursedept* in the previous chapters. The reason is that the algorithm we have presented thus far results in some schemas that can be either eliminated or combined with other schemas. We explore this issue next.

5.6.4.1 Redundancy of Schemas

A relationship set linking a weak entity set to the corresponding strong entity set is treated specially. As we noted in Section 7.5.6, these relationships are many-to-one and have no descriptive attributes. Furthermore, the primary key of a weak entity set includes the primary key of the strong entity set. In the E-R diagram of Figure 7.14, the weak entity set *section* is dependent on the strong entity set *course* via the relationship set *seccourse*. The primary key of *section* is $\{courseid, secid, semester, year\}$ and the primary key of *course* is *courseid*. Since *seccourse* has no descriptive attributes, the *seccourse* schema has attributes *courseid*, *secid*, *semester*, and *year*. The schema for the entity set *section* includes the attributes *courseid*, *secid*, *semester*, and *year* (among others). Every $(courseid, secid, semester, year)$ combination in a *seccourse* relation would also be present in the relation on schema *section*, and vice versa. Thus, the *seccourse* schema is redundant.

In general, the schema for the relationship set linking a weak entity set to its corresponding strong entity set is redundant and does not need to be present in a relational database design based upon an E-R diagram.

5.6.4.2 Combination of Schemas

Consider a many-to-one relationship set *AB* from entity set *A* to entity set *B*. Using our relational-schema construction algorithm outlined previously, we get three schemas: *A*, *B*, and *AB*. Suppose further that the participation of *A* in the relationship is total; that is, every entity *a* in the entity set *B* must participate in the relationship *AB*. Then we can combine the schemas *A* and *AB* to form a single schema consisting of the union of attributes of both schemas. The primary key of the combined schema is the primary key of the entity set into whose schema the relationship set schema was merged.

To illustrate, let's examine the various relations in the E-R diagram of Figure 5.15 that satisfy the above criteria:

- *instdept*. The schemas *instructor* and *department* correspond to the entity sets *A* and *B*, respectively. Thus, the schema *instdept* can be combined with the *instructor* schema. The resulting *instructor* schema consists of the attributes $\{ID, name, deptname, salary\}$.
- *studdept*. The schemas *student* and *department* correspond to the entity sets *A* and *B*, respectively. Thus, the schema *studdept* can be combined with the *student* schema. The resulting *student* schema consists of the attributes $\{ID, name, deptname, tot-cred\}$.

- *courseddept*. The schemas *course* and *department* correspond to the entity sets *A* and *B*, respectively. Thus, the schema *courseddept* can be combined with the *course* schema. The resulting *course* schema consists of the attributes {*course id*, *title*, *deptname*, *credits*}.
- *secclass*. The schemas *section* and *classroom* correspond to the entity sets *A* and *B*, respectively. Thus, the schema *secclass* can be combined with the *section* schema. The resulting *section* schema consists of the attributes {*courseid*, *sec id*, *semester*, *year*, *building*, *roomnumber*}.
- *sectimeslot*. The schemas *section* and *timeslot* correspond to the entity sets *A* and *B* respectively, Thus, the schema *sectimejslot* can be combined with the *section* schema obtained in the previous step. The resulting *section* schema consists of the attributes {*courseid*, *secid*, *semester*, *year*, *building*, *roomnumber*, *timejslotid*}.

In the case of one-to-one relationships, the relation schema for the relationship set can be combined with the schemas for either of the entity sets.

We can combine schemas even if the participation is partial by using null values. In the above example, if *instdept* were partial, then we would store null values for the *deptname* attribute for those instructors who have no associated department.

Finally, we consider the foreign-key constraints that would have appeared in the schema representing the relationship set. There would have been foreign-key constraints referencing each of the entity sets participating in the relationship set. We drop the constraint referencing the entity set into whose schema the relationship set schema is merged, and add the other foreign-key constraints to the combined schema. For example, *instdept* has a foreign key constraint of the attribute *deptname* referencing the *department* relation. This foreign constraint is added to the *instructor* relation when the schema for *instdept* is merged into *instructor*.

5.7 Summary

- Database design mainly involves the design of the database schema. The **entity-relationship (E-R)** data model is a widely used data model for database design. It provides a convenient graphical representation to view data, relationships, and constraints.
- The E-R model is intended primarily for the database-design process. It was developed to facilitate database design by allowing the specification of an **enterprise schema**.

Such a schema represents the overall logical structure of the database. This overall structure can be expressed graphically by an **E-R diagram**.

- An **entity** is an object that exists in the real world and is distinguishable from other objects. We express the distinction by associating with each entity a set of attributes that describes the object.
- A **relationship** is an association among several entities. A **relationship set** is a collection of relationships of the same type, and an **entity set** is a collection of entities of the same type.
- The terms **superkey**, **candidate key**, and **primary key** apply to entity and relationship sets as they do for relation schemas. Identifying the primary key of a relationship set requires some care, since it is composed of attributes from one or more of the related entity sets.
- **Mapping cardinalities** express the number of entities to which another entity can be associated via a relationship set.
- An entity set that does not have sufficient attributes to form a primary key is termed a **weak entity set**. An entity set that has a primary key is termed a **strong entity set**.

5.8 Questions for Discussion

1. Explain ER-model with diagram
2. Discuss about Mapping Cardinalities
3. What is Weak entity set? Explain

5.9 Suggested Reading

Recommended Text

A.Silberchatz, H.F Korth and S.Sudharshan, 2006, Database System Concepts, 5th Edition, Tata McGraw Hill, New Delhi.

Reference Books

J.D Ullaman, 1988, Principles of Database Systems, Galgotia Publishers, New Delhi.

Lesson - 6

RELATIONAL DATABASE DESIGN

CONTENTS

- 6.0 Aims and Objectives**
- 6.1 Introduction**
- 6.2 Features of Relational Design**
- 6.3 Atomic Domains and First Normal Form**
- 6.4 Decomposition Using Functional Dependencies**
- 6.5 Multivalued Dependencies**
- 6.6 More Normal Forms**
- 6.7 Web Interface**
- 6.8 Summary**
- 6.9 Questions for Discussion**
- 6.10 Suggested Reading**

6.0 Aims and Objectives

After studying this lesson you should be able to:

- Express the Features of Relational Design
- Discuss about the Multivalued Dependencies
- Describe about the Normal Form

6.1 Introduction

In this chapter, we introduce a formal approach to relational database design based on the notion of functional dependencies. We then define normal forms in terms of functional dependencies and other types of data dependencies. First, however, we view the problem of relational design from the standpoint of the schemas derived from a given entity-relationship design.

6.2 Features of Relational Design

Our study of entity-relationship design in Chapter 5 provides an excellent starting point for creating a relational database design. We saw in Later that it is possible to generate a set of relation schemas directly from the E-R design. Obviously, the goodness (or badness) of the resulting set of schemas depends on how good the E-R design was in the first place. Later in this chapter, we shall study precise ways of assessing the desirability of a collection of relation schemas. However, we can go a long way toward a good design using concepts we have already studied.

For ease of reference, we repeat the schemas for the university database in Figure 6.1.

```

classroom(building, roomnumber, capacity)
department(deptname, building, budget)
course(courseid, title, deptname, credits)
instructor(ID, name, deptname, salary)
section(courseid, secid, semester, year, building, room-number, timeslotid)
teaches(ID, courseid, secid, semester, year)
student(ID, name, deptname, totcred)
takes(ID, courseid, secid, semester, year, grade)
advisor(sID, iID)
timeslot(timeslotid, day, start_time, endtime)
prereq(courseid, prereqid)

```

Figure 6.1 Schema for the university database.

6.2.1 Design Alternative: Larger Schemas

Now, let us explore features of this relational database design as well as some alternatives. Suppose that instead of having the schemas *instructor* and *department*, we have the schema:

```
instjdept (ID, name, salary, deptname, building, budget)
```

This represents the result of a natural join on the relations corresponding to *instructor* and *department*. This seems like a good idea because some queries can be expressed using fewer joins, until we think carefully about the facts about the university that led to our E-R design.

Let us consider the instance of the *instdept* relation shown in Figure 6.2. Notice that we have to repeat the department information (“building” and “budget”) once for each instructor in the department. For example, the information about the Comp. Sci. department (Taylor, 100000) is included in the tuples of instructors Katz, Srinivasan, and Brandt.

It is important that all these tuples agree as to the budget amount since other-wise our database would be inconsistent. In our original design using *instructor* and *department*, we stored the amount of each budget exactly once. This suggests that using *instdept* is a bad idea since it stores the budget amounts redundantly and runs the risk that some user might update the budget amount in one tuple but not all, and thus create inconsistency.

Even if we decided to live with the redundancy problem, there is still another problem with the *instdept* schema. Suppose we are creating a new department in the university. In the alternative design above, we cannot represent directly the information concerning a department (*deptname*, *building*, *budget*) unless that department has at least one instructor at the university. This is because tuples in the *instdept* table require values for *ID*, *name*, and *salary*. This means that we cannot record information about the newly created department until the first instructor

<i>ID</i>	<i>name</i>	<i>salary</i>	<i>Deptname</i>	<i>building</i>	<i>budget</i>
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

Figure 6.2 The *inst-dept* table.

is hired for the new department. In the old design, the schema *department* can handle this, but under the revised design, we would have to create a tuple with a null value for *building* and *budget*. In some cases null values are troublesome, as we saw in our study of SQL. However, if we decide that this is not a problem to us in this case, then we can proceed to use the revised design.

6.2.2 Design Alternative: Smaller Schemas

Suppose again that, somehow, we had started out with the schema *inst-dept*. How would we recognize that it requires repetition of information and should be split into the two schemas *instructor* and *department*?

By observing the contents of actual relations on schema *inst-dept*, we could note the repetition of information resulting from having to list the building and budget once for each instructor associated with a department. However, this is an unreliable process. A real-world database has a large number of schemas and an even larger number of attributes. The number of tuples can be in the millions or higher. Discovering repetition would be costly. There is an even more fundamental problem with this approach. It does not allow us to determine whether the lack of repetition is just a “lucky” special case or whether it is a manifestation of a general rule. In our example, how would we *know* that in our university organization, each department (identified by its department name) *must* reside in a single building and must have a single budget amount? Is the fact that the budget amount for the Comp. Sci. department appears three times with the same budget amount just a coincidence? We cannot answer these questions without going back to the enterprise itself and understanding its rules. In particular, we would need to discover that the university requires that every department (identified by its department name) *must* have only one building and one budget value.

In the case of *inst-dept*, our process of creating an E-R design successfully avoided the creation of this schema. However, this fortuitous situation does not always occur. Therefore, we need to allow the database designer to specify rules such as “each specific value for *deptname* corresponds to at most one *budget*” even in cases where *deptname* is not the primary key for the schema in question. In other words, we need to write a rule that says “if there were a schema (*deptname*, *budget*), then *deptname* is able to serve as the primary key.” This rule is specified as a **functional dependency**

deptname $\longrightarrow$ *budget*

Given such a rule, we now have sufficient information to recognize the problem of the *instdept* schema. Because *deptname* cannot be the primary key for *instdept* (because a department may need several tuples in the relation on schema *instdept*), the amount of a budget may have to be repeated.

Observations such as these and the rules (functional dependencies in particular) that result from them allow the database designer to recognize situations where a schema ought to be split, or *decomposed*, into two or more schemas. It is not hard to see that the right way to decompose *instdept* is into schemas *instructor* and *department* as in the original design. Finding the right decomposition is much harder for schemas with a large number of attributes and several functional dependencies. To deal with this, we shall rely on a formal methodology that we develop later in this chapter.

Not all decompositions of schemas are helpful. Consider an extreme case where all we had were schemas consisting of one attribute. No interesting relationships of any kind could be expressed.

employee (ID, name, street, city, salary)

into the following two schemas:

employee1 (ID, name)

employee2 (name, street, city, salary)

The flaw in this decomposition arises from the possibility that the enterprise has two employees with the same name. This is not unlikely in practice, as many cultures have certain highly popular names. Of course each person would have a unique employee-id, which is why *ID* can serve as the primary key. As an example, let us assume two employees, both named Kim, work at the university and have the following tuples in the relation on schema *employee* in the original design:

(57766, Kim, Main, Perryridge, 75000) (98776, Kim, North, Hampton, 67000)

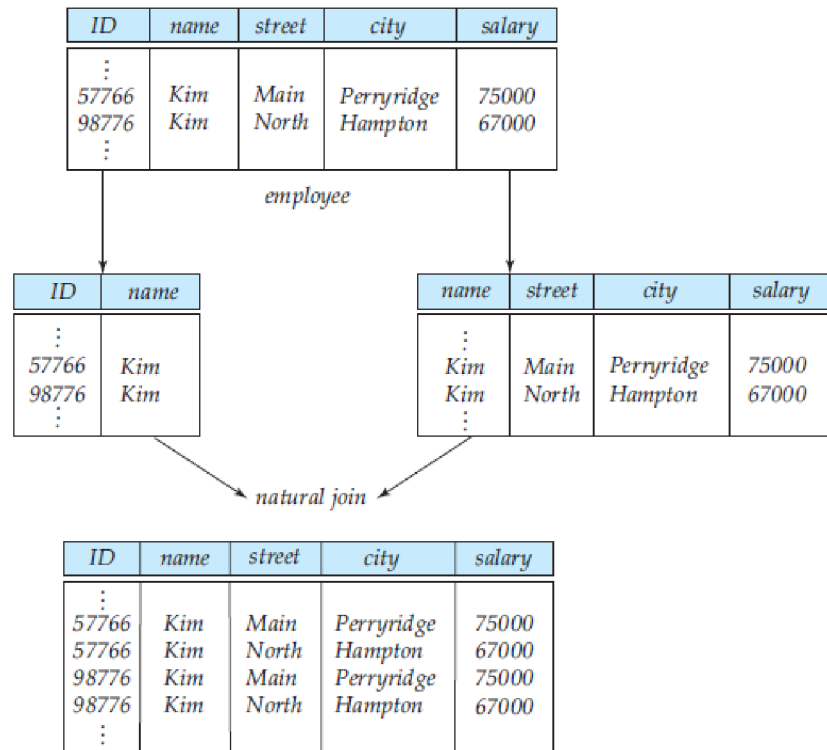


Figure 6.3 Loss of information via a bad decomposition.

Figure 6.3 shows these tuples, the resulting tuples using the schemas resulting from the decomposition, and the result if we attempted to regenerate the original tuples using a natural join. As we see in the figure, the two original tuples appear in the result along with two new tuples that incorrectly mix data values pertaining to the two employees named Kim. Although we have more tuples, we actually have less information in the following sense. We can indicate that a certain street, city, and salary pertain to someone named Kim, but we are unable to distinguish which of the Kims. Thus, our decomposition is unable to represent certain important facts about the university employees. Clearly, we would like to avoid such decompositions. We shall refer to such decompositions as being **lossy decompositions**, and, conversely, to those that are not as **lossless decompositions**.

6.3 Atomic Domains and First Normal Form

The E-R model allows entity sets and relationship sets to have attributes that have some degree of substructure. Specifically, it allows multivalued attributes such as *phone-number* in Figure 5.11 and composite attributes (such as an attribute *address* with component attributes *street*, *city*, *state*, and *zip*). When we create tables from E-R designs that contain these types of attributes, we eliminate this substructure. For composite attributes, we let each component be an attribute in its own right. For multivalued attributes, we create one tuple for each item in a multivalued set.

In the relational model, we formalize this idea that attributes do not have any substructure. A domain is **atomic** if elements of the domain are considered to be indivisible units. We say that a relation schema R is in **first normal form** (1NF) if the domains of all attributes of R are atomic.

A set of names is an example of a nonatomic value. For example, if the schema of a relation *employee* included an attribute *children* whose domain elements are sets of names, the schema would not be in first normal form.

Composite attributes, such as an attribute *address* with component attributes *street*, *city*, *state*, and *zip* also have nonatomic domains.

Integers are assumed to be atomic, so the set of integers is an atomic domain; however, the set of all sets of integers is a nonatomic domain. The distinction is that we do not normally consider integers to have subparts, but we consider sets of integers to have subparts—namely, the integers making up the set. But the important issue is not what the domain itself is, but rather how we use domain elements in our database. The domain of all integers would be nonatomic if we considered each integer to be an ordered list of digits.

As a practical illustration of the above point, consider an organization that assigns employees identification numbers of the following form: The first two letters specify the department and the remaining four digits are a unique number within the department for the employee. Examples of such numbers would be “CS001” and “EE1127”. Such identification numbers can be divided into smaller units, and are therefore nonatomic. If a relation schema had an attribute whose domain consists of identification numbers encoded as above, the schema would not be in first normal form.

When such identification numbers are used, the department of an employee can be found by writing code that breaks up the structure of an identification number. Doing so requires extra programming, and information gets encoded in the application program rather than in the database. Further problems arise if such identification numbers are used as primary keys: When an employee changes departments, the employee’s identification number must be changed everywhere it occurs, which can be a difficult task, or the code that interprets the number would give a wrong result.

From the above discussion, it may appear that our use of course identifiers such as “CS-101”, where “CS” indicates the Computer Science department, means that the domain of course identifiers is not atomic. Such a domain is not atomic as far as humans using the system are concerned. However, the database application still treats the domain as atomic, as long as it does not attempt to split the identifier and interpret parts of the identifier as a

department abbreviation. The *course* schema stores the department name as a separate attribute, and the database application can use this attribute value to find the department of a course, instead of interpreting particular characters of the course identifier. Thus, our university schema can be considered to be in first normal form.

The use of set-valued attributes can lead to designs with redundant storage of data, which in turn can result in inconsistencies. For instance, instead of having the relationship between instructors and sections being represented as a separate relation *teaches*, a database designer may be tempted to store a set of course section identifiers with each instructor and a set of instructor identifiers with each section. (The primary keys of *section* and *instructor* are used as identifiers.) Whenever data pertaining to which instructor teaches which section is changed, the update has to be performed at two places: in the set of instructors for the section, and the set of sections for the instructor. Failure to perform both updates can leave the database in an inconsistent state. Keeping only one of these sets, that either the set of instructors of a section, or the set of sections of an instructor, would avoid repeated information; however keeping only one of these would complicate some queries, and it is unclear which of the two to retain.

Some types of nonatomic values can be useful, although they should be used with care. For example, composite-valued attributes are often useful, and set-valued attributes are also useful in many cases, which is why both are supported in the E-R model. In many domains where entities have a complex structure, forcing a first normal form representation represents an unnecessary burden on the application programmer, who has to write code to convert data into atomic form. There is also the runtime overhead of converting data back and forth from the atomic form. Support for nonatomic values can thus be very useful in such domains. In fact, modern database systems do support many types of nonatomic values. However, in this chapter we restrict ourselves to relations in first normal form and, thus, all domains are atomic.

6.4 Decomposition Using Functional Dependencies

In discussing algorithms for relational database design, we shall need to talk about arbitrary relations and their schema, rather than talking only about examples. Recalling our introduction to the relational model in Chapter 2, we summarize our notation here.

- In general, we use Greek letters for sets of attributes (for example, α). We use a lowercase Roman letter followed by an uppercase Roman letter in parentheses to refer to a relation schema (for example, $r(R)$). We use the notation $r(R)$ to show that

the schema is for relation r , with R denoting the set of attributes, but at times simplify our notation to use just R when the relation name does not matter to us.

Of course, a relation schema is a set of attributes, but not all sets of attributes are schemas. When we use a lowercase Greek letter, we are referring to a set of attributes that may or may not be a schema. A Roman letter is used when we wish to indicate that the set of attributes is definitely a schema.

- When a set of attributes is a superkey, we denote it by K . A superkey pertains to a specific relation schema, so we use the terminology “ K is a superkey of $r(R)$.”
- We use a lowercase name for relations. In our examples, these names are intended to be realistic (for example, *instructor*), while in our definitions and algorithms, we use single letters, like r .
- A relation, of course, has a particular value at any given time; we refer to that as an instance and use the term “instance of r ”. When it is clear that we are talking about an instance, we may use simply the relation name (for example, r).

6.4.1 Keys and Functional Dependencies

A database models a set of entities and relationships in the real world. There are usually a variety of constraints (rules) on the data in the real world. For example, some of the constraints that are expected to hold in a university database are:

1. Students and instructors are uniquely identified by their ID.
2. Each student and instructor has only one name.
3. Each instructor and student is (primarily) associated with only one department.¹
4. Each department has only one value for its budget, and only one associated building.

An instance of a relation that satisfies all such real-world constraints is called a **legal instance** of the relation; a legal instance of a database is one where all the relation instances are legal instances.

Some of the most commonly used types of real-world constraints can be represented formally as keys (superkeys, candidate keys and primary keys), or as functional dependencies, which we define below.

In Section 2.3, we defined the notion of a *superkey* as a set of one or more attributes that, taken collectively, allows us to identify uniquely a tuple in the relation. We restate that definition here as follows: Let $r(R)$ be a relation schema. A subset K of R is a **superkey** of $r(R)$ if, in any legal instance of $r(R)$, for all pairs t_1 and t_2 of tuples in the instance of r if $t_1 = t_2$, then $t_1[K] = t_2[K]$. That is, no two tuples in any legal instance of relation $r(R)$ may have the same value on attribute set K . Clearly, if no two tuples in r have the same value on K , then a K -value uniquely identifies a tuple in r .

Whereas a superkey is a set of attributes that uniquely identifies an entire tuple, a functional dependency allows us to express constraints that uniquely identify the values of certain attributes. Consider a relation schema $r(R)$, and let $a \in R$ and $p \in R$.

- Given an instance of $r(R)$, we say that the instance **satisfies** the **functional dependency** $a \rightarrow p$ if for all pairs of tuples t_1 and t_2 in the instance such that $t_1[a] = t_2[a]$, it is also the case that $t_1[p] = t_2[p]$.
- We say that the functional dependency $a \rightarrow p$ **holds** on schema $r(R)$ if, in every legal instance $of r(R)$ it satisfies the functional dependency.

Using the functional-dependency notation, we say that K is a *superkey* of $r(R)$ if the functional dependency $K \rightarrow R$ holds on $r(R)$. In other words, K is a superkey if, for every legal instance of $r(R)$, for every pair of tuples t_1 and t_2 from the instance, whenever $t_1[K] = t_2[K]$, it is also the case that $t_1[R] = t_2[R]$ (that is, $t_1 = t_2$).²

Functional dependencies allow us to express constraints that we cannot express with superkeys.

inst-dept (ID, name, salary, deptname, building, budget)

in which the functional dependency $deptname \rightarrow budget$ holds because for each department (identified by *deptname*) there is a unique budget amount.

We denote the fact that the pair of attributes $(ID, deptname)$ forms a superkey for *inst-dept* by writing:

$ID, deptname \rightarrow - name, salary, building, budget$

We shall use functional dependencies in two ways:

1. To test instances of relations to see whether they satisfy a given set F of functional dependencies.

2. To specify constraints on the set of legal relations. We shall thus concern our-selves with *only* those relation instances that satisfy a given set of functional dependencies. If we wish to constrain ourselves to relations on schema $r(R)$ that satisfy a set F of functional dependencies, we say that F **holds** on $r(R)$.

Let us consider the instance of relation r of Figure 6.4, to see which functional dependencies are satisfied. Observe that $A \rightarrow C$ is satisfied. There are two tuples

A	B	C	D
a_1	$b1$	$c1$	d_1
a_1	$b2$	$c1$	d_2
a_2	$b2$	$c2$	d_2
a_2	$b3$	$c2$	d_3
a_3	$b3$	$c2$	d_4

Figure 6.4 Sample instance of relation r .

that have an A value of a_1 . These tuples have the same C value—namely, c_1 . Similarly, the two tuples with an A value of a_2 have the same C value, c_2 . There are no other pairs of distinct tuples that have the same A value. The functional dependency $C \rightarrow A$ is not satisfied, however. To see that it is not, consider the tuples $t_1 = (a_2, b_3, c_2, d_3)$ and $t_2 = (a_3, b_3, c_2, d_4)$. These two tuples have the same C values, c_2 , but they have different A values, a_2 and a_3 , respectively. Thus, we have found a pair of tuples t_1 and t_2 such that $t_1[C] = t_2[C]$, but $t_1[A] \neq t_2[A]$.

Some functional dependencies are said to be **trivial** because they are satisfied by all relations. For example, $A \rightarrow A$ is satisfied by all relations involving attribute A . Reading the definition of functional dependency literally, we see that, for all tuples t_1 and t_2 such that $t_1[A] = t_2[A]$, it is the case that $t_1[A] = t_2[A]$. Similarly, $AB \rightarrow A$ is satisfied by all relations involving attribute A . In general, a functional dependency of the form $\mathbf{a} \rightarrow \mathbf{b}$ is **trivial** if $\mathbf{b}$ is a subset of $\mathbf{a}$.

It is important to realize that an instance of a relation may satisfy some functional dependencies that are not required to hold on the relation's schema. In the instance of the *classroom* relation of Figure 6.5, we see that *room-number* $\rightarrow$ *capacity* is satisfied. However, we believe that, in the real world, two classrooms in different buildings can have

the same room number but with different room capacity. Thus, it is possible, at some time, to have an instance of the *classroom* relation in which *room-number* $\rightarrow$ *capacity* is not satisfied. So, we would not include *room-number* $\rightarrow$ *capacity* in the set of functional dependencies that hold on the schema for the *classroom* relation. However, we would expect the functional dependency *building, roomnumber* $\rightarrow$ *capacity* to hold on the *classroom* schema.

Given that a set of functional dependencies F holds on a relation $r(R)$, it may be possible to infer that certain other functional dependencies must also hold on

<i>building</i>	<i>roomnumber</i>	<i>capacity</i>
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

Figure 6.5 An instance of the *classroom* relation.

the relation. For example, given a schema $r(A, B, C)$, if functional dependencies $A \rightarrow B$ and $B \rightarrow C$ hold on r , we can infer the functional dependency $A \rightarrow C$ must also hold on r . This is because, given any value of A there can be only one corresponding value for B , and for that value of B , there can only be one corresponding value for C . We study later, in Section 8.4.1, how to make such inferences.

We will use the notation F^+ to denote the **closure** of the set F , that is, the set of all functional dependencies that can be inferred given the set F . Clearly F^+ contains all of the functional dependencies in F .

6.4.2 Boyce-Codd Normal Form

One of the more desirable normal forms that we can obtain is **Boyce-Codd normal form (BCNF)**. It eliminates all redundancy that can be discovered based on functional dependencies, though, as we shall see in Section 8.6, there may be other types of redundancy remaining. A relation schema R is in BCNF with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $\mathbf{a} \rightarrow \mathbf{b}$, where $\mathbf{a} \subset R$ and $\mathbf{b} \subset R$, at least one of the following holds:

- $\mathbf{a} \rightarrow \mathbf{3}$ is a trivial functional dependency (that is, $\mathbf{3} \subset \mathbf{a}$).
- $\mathbf{a}$ is a superkey for schema R .

A database design is in BCNF if each member of the set of relation schemas that constitutes the design is in BCNF.

We have already seen in Section 8.1 an example of a relational schema that is not in BCNF:

instdept (*ID*, *name*, *salary*, *deptname*, *building*, *budget*)

The functional dependency $\text{deptname} \rightarrow \text{budget}$ holds on *instdept*, but *deptname* is not a superkey (because, a department may have a number of different instructors). In Section 8.1.2, we saw that the decomposition of *instdept* into *instructor* and *department* is a better design. The *instructor* schema is in BCNF. All of the nontrivial functional dependencies that hold, such as:

$ID \rightarrow \text{name}, \text{deptname}, \text{salary}$

include *ID* on the left side of the arrow, and *ID* is a superkey (actually, in this case, the primary key) for *instructor*. (In other words, there is no nontrivial functional dependency with any combination of *name*, *deptname*, and *salary*, without *ID*, on the side.) Thus, *instructor* is in BCNF.

Similarly, the *department* schema is in BCNF because all of the nontrivial functional dependencies that hold, such as:

$\text{deptname} \rightarrow \text{building}, \text{budget}$

include *deptname* on the left side of the arrow, and *deptname* is a superkey (and the primary key) for *department*. Thus, *department* is in BCNF.

We now state a general rule for decomposing that are not in BCNF. Let R be a schema that is not in BCNF. Then there is at least one nontrivial functional dependency $\mathbf{a} \rightarrow \mathbf{3}$ such that $\mathbf{a}$ is not a superkey for R . We replace R in our design with two schemas:

- $(\mathbf{a} \cup \mathbf{p})$
- $(R - (\mathbf{p} - \mathbf{a}))$

In the case of *instdept* above, $\mathbf{a} = \text{deptname}$, $\mathbf{p} = \{\text{building}, \text{budget}\}$, and *instdept* is replaced by

- $(a \cup \{3\}) = (deptname, building, budget)$
- $(R - (P - a)) = (ID, name, deptname, salary)$

In this example, it turns out that $\{3\} - a = p$. We need to state the rule as we did so as to deal correctly with functional dependencies that have attributes that appear on both sides of the arrow.

When we decompose a schema that is not in BCNF, it may be that one or more of the resulting schemas are not in BCNF. In such cases, further decomposition is required, the eventual result of which is a set of BCNF schemas.

6.4.3 BCNF and Dependency Preservation

We have seen several ways in which to express database consistency constraints: primary-key constraints, functional dependencies, **check** constraints, assertions, and triggers. Testing these constraints each time the database is updated can be costly and, therefore, it is useful to design the database in a way that constraints can be tested efficiently. In particular, if testing a functional dependency can be done by considering just one relation, then the cost of testing this constraint is low. We shall see that, in some cases, decomposition into BCNF can prevent efficient testing of certain functional dependencies.

To illustrate this, suppose that we make a small change to our university organization. In the design of Figure 6.6, a student may have only one advisor. This follows from the relationship set *advisor* being many-to-one from *student* to *advisor*. The “small” change we shall make is that an instructor can be associated with only a single department and a student may have more than one advisor, but at most one from a given department.³

One way to implement this change using the E-R design is by replacing the *advisor* relationship set with a ternary relationship set, *dept-advisor*, involving entity sets *instructor*, *student*, and *department* that is many-to-one from the pair

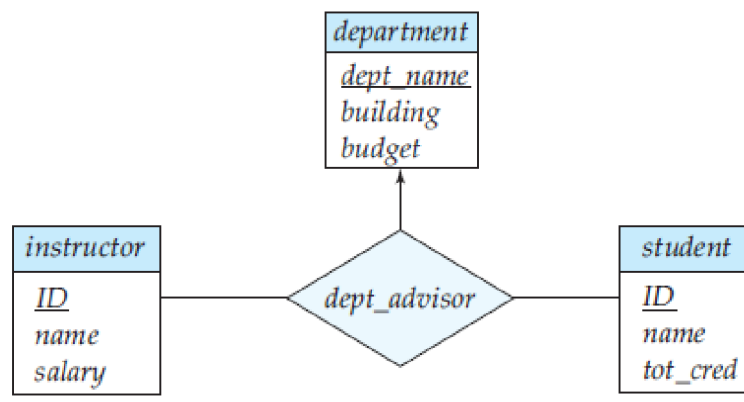


Figure 6.6 The deptadvisor relationship set.

$\{student, instructor\}$ to *department* as shown in Figure 8.6. The E-R diagram specifies the constraint that “a student may have more than one advisor, but at most one corresponding to a given department”.

With this new E-R diagram, the schemas for the *instructor*, *department*, and *student* are unchanged. However, the schema derived from *deptadvisor* is now:

deptadvisor (*sID*, *iID*, *deptname*)

Although not specified in the E-R diagram, suppose we have the additional constraint that “an instructor can act as advisor for only a single department.” Then, the following functional dependencies hold on *deptadvisor*:

$iID \twoheadrightarrow deptname$ $sID, deptname \twoheadrightarrow iID$

The first functional dependency follows from our requirement that “an instructor can act as an advisor for only one department.” The second functional dependency follows from our requirement that “a student may have at most one advisor for a given department.”

Notice that with this design, we are forced to repeat the department name once for each time an instructor participates in a *deptadvisor* relationship. We see that *deptadvisor* is not in BCNF because *iID* is not a superkey. Following our rule for BCNF decomposition, we get:

(*sID*, *iID*) (*iID*, *deptname*)

Both the above schemas are BCNF. (In fact, you can verify that any schema with only two attributes is in BCNF by definition.) Note however, that in our BCNF design, there is no schema that includes all the attributes appearing in the functional

Because our design makes it computationally hard to enforce this functional dependency, we say our design is not **dependency preserving**.⁴ Because dependency preservation is usually considered desirable, we consider another normal form, weaker than BCNF, that will allow us to preserve dependencies. That normal form is called third normal form.⁵

6.4.4 Third Normal Form

BCNF requires that all nontrivial dependencies be of the form $\mathbf{a} \twoheadrightarrow \mathbf{b}$, where $\mathbf{a}$ is a superkey. Third normal form (3NF) relaxes this constraint slightly by allowing certain nontrivial functional dependencies whose left side is not a superkey. Before we define 3NF, we recall that a candidate key is a minimal superkey—that is, a superkey no proper subset of which is also a superkey.

A relation schema R is in **third normal form** with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $\mathbf{a} \longrightarrow \mathbf{(3)}$, where $\mathbf{a} \subset R$ and $\mathbf{(3)} \subset R$, at least one of the following holds:

- $\mathbf{a} \longrightarrow \mathbf{(3)}$ is a trivial functional dependency.
- $\mathbf{a}$ is a superkey for R .
- Each attribute A in $\mathbf{(3)} - \mathbf{a}$ is contained in a candidate key for R .

Note that the third condition above does not say that a single candidate key must contain all the attributes in $\mathbf{(3)} - \mathbf{a}$; each attribute A in $\mathbf{(3)} - \mathbf{a}$ may be contained in a *different* candidate key.

The first two alternatives are the same as the two alternatives in the definition of BCNF. The third alternative of the 3NF definition seems rather unintuitive, and it is not obvious why it is useful. It represents, in some sense, a minimal relaxation of the BCNF conditions that helps ensure that every schema has a dependency-preserving decomposition into 3NF. Its purpose will become more clear later, when we study decomposition into 3NF.

Observe that any schema that satisfies BCNF also satisfies 3NF, since each of its functional dependencies would satisfy one of the first two alternatives. BCNF is therefore a more restrictive normal form than is 3NF.

The definition of 3NF allows certain functional dependencies that are not allowed in BCNF. A dependency $\mathbf{a} \longrightarrow \mathbf{(3)}$ that satisfies only the third alternative of the 3NF definition is not allowed in BCNF, but is allowed in 3NF.⁶

Now, let us again consider the *deptadvisor* relationship set, which has the following functional dependencies:

$$iID \longrightarrow deptname \ sID, deptname \longrightarrow iID$$

In Section 8.3.3 we argued that the functional dependency " $iID \longrightarrow deptname$ " caused the *deptadvisor* schema not to be in BCNF. Note that here $\mathbf{a} = iID$, $\mathbf{(3)} = deptname$, and $\mathbf{(3)} - \mathbf{a} = deptname$. Since the functional dependency $sID, deptname \longrightarrow iID$ holds on *deptadvisor*, the attribute *deptname* is contained in a candidate key and, therefore, *deptadvisor* is in 3NF.

We have seen the trade-off that must be made between BCNF and 3NF when there is no dependency-preserving BCNF design.

6.4.5 Higher Normal Forms

Using functional dependencies to decompose schemas may not be sufficient to avoid unnecessary repetition of information in certain cases. Consider a slight variation in the *instructor* entity-set definition in which we record with each instructor a set of children's names and a set of phone numbers. The phone numbers may be shared by multiple people. Thus, *phone-number* and *child-name* would be multivalued attributes and, following our rules for generating schemas from an E-R design, we would have two schemas, one for each of the multivalued attributes, *phonenum* and *child-name*:

(*ID*, *child-name*)

(*ID*, *phonenum*)

If we were to combine these schemas to get

(*ID*, *child-name*, *phonenum*)

we would find the result to be in BCNF because only nontrivial functional dependencies hold. As a result we might think that such a combination is a good idea. However, such a combination is a bad idea, as we can see by considering the example of an instructor with two children and two phone numbers. For example, let the instructor with *ID* 99999 have two children named "David" and "William" and two phone numbers, 512-555-1234 and 512-555-4321. In the combined schema, we must repeat the phone numbers once for each dependent:

(99999, David, 512-555-1234)

(99999, David, 512-555-4321)

(99999, William, 512-555-1234)

(99999, William, 512-555-4321)

If we did not repeat the phone numbers, and stored only the first and last tuple, we would have recorded the dependent names and the phone numbers, but the resultant tuples would imply that David corresponded to 512-555-1234, while William corresponded to 512-555-4321. As we know, this would be incorrect.

Because normal forms based on functional dependencies are not sufficient to deal with situations like this, other dependencies and normal forms have been defined.

6.5 Multivalued Dependencies

Functional dependencies rule out certain tuples from being in a relation. If $A \twoheadrightarrow B$, then we cannot have two tuples with the same A value but different B values. Multivalued dependencies, on the other hand, do not rule out the existence of certain tuples. Instead, they *require* that other tuples of a certain form be present in the relation. For this reason, functional dependencies sometimes are referred to as **equality-generating dependencies**, and multivalued dependencies are referred to as **tuple-generating dependencies**.

Let $r(R)$ be a relation schema and let $\alpha \subset R$ and $\beta \subset R$. The **multivalued dependency**

$$\alpha \twoheadrightarrow \beta$$

holds on R if, in any legal instance of relation $r(R)$, for all pairs of tuples t_1 and t_2 in r such that $t_1[\alpha] = t_2[\alpha]$, there exist tuples t_3 and t_4 in r such that

$$t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha]$$

$$t_3[\beta] = t_1[\beta]$$

$$t_3[R - \beta] = t_2[R - \beta]$$

$$t_4[\beta] = t_2[\beta]$$

$$t_4[R - \beta] = t_1[R - \beta]$$

	α	β	$R - \alpha - \beta$
t_1	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$a_{j+1} \dots a_n$
t_2	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$b_{j+1} \dots b_n$
t_3	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$b_{j+1} \dots b_n$
t_4	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$a_{j+1} \dots a_n$

Figure 6.7 Tabular Representation of $\alpha \twoheadrightarrow \beta$.

This definition is less complicated than it appears to be. Figure 6.7 gives a tabular picture of t_1 , t_2 , t_3 , and t_4 . Intuitively, the multivalued dependency $\alpha \twoheadrightarrow \beta$ says that the relationship between α and β is independent of the relationship between α and $R - \beta$. If the multivalued dependency $\alpha \twoheadrightarrow \beta$ is satisfied by all relations on schema R , then $\alpha \twoheadrightarrow \beta$ is a *trivial* multivalued dependency on schema R . Thus, $\alpha \twoheadrightarrow \beta$ is trivial if $\beta \subset \alpha$ or $\beta \cup \alpha = R$.

To illustrate the difference between functional and multivalued dependencies, we consider the schema r_2 again, and an example relation on that schema shown in Figure

6.8. We must repeat the department name once for each address that an instructor has, and we must repeat the address for each department with which an instructor is associated. This repetition is unnecessary, since the relationship between an instructor and his address is independent of the relationship between that instructor and a department. If an instructor with *ID* 22222 is associated with the Physics department, we want that department to be associated with all of that instructor's addresses. Thus, the relation of Figure 6.9 is illegal. To make this relation legal, we need to add the tuples (Physics, 22222, Main, Manchester) and (Math, 22222, North, Rye) to the relation of Figure 6.9.

Comparing the preceding example with our definition of multivalued dependency, we see that we want the multivalued dependency:

$$ID \twoheadrightarrow\!\!\rightarrow\! street, city$$

to hold. (The multivalued dependency $ID \twoheadrightarrow\!\!\rightarrow\! deptname$ will do as well. We shall soon see that they are equivalent.)

As with functional dependencies, we shall use multivalued dependencies in two ways:

1. To test relations to determine whether they are legal under a given set of functional and multivalued dependencies

<i>ID</i>	<i>deptname</i>	<i>street</i>	<i>city</i>
22222	Physics	North	Rye
22222	Physics	Main	Manchester
12121	Finance	Lake	Horseneck

Figure 6.8 An example of redundancy in a relation on a BCNF schema.

<i>ID</i>	<i>deptname</i>	<i>street</i>	<i>city</i>
22222	Physics	North	Rye
22222	Math	Main	Manchester

Figure 6.9 An illegal r_2 relation.

2. To specify constraints on the set of legal relations; we shall thus concern ourselves with *only* those relations that satisfy a given set of functional and multivalued dependencies

Note that, if a relation r fails to satisfy a given multivalued dependency we can construct a relation r' that *does* satisfy the multivalued dependency by adding tuples to r .

Let D denote a set of functional and multivalued dependencies. The **closure** D^+ of D is the set of all functional and multivalued dependencies logically implied by D . As we did for functional dependencies, we can compute D^+ from D , using the formal definitions of functional dependencies and multivalued dependencies. We can manage with such reasoning for very simple multivalued dependencies. Luckily, multivalued dependencies that occur in practice appear to be quite simple. For complex dependencies, it is better to reason about sets of dependencies by using a system of inference rules.

From the definition of multivalued dependency, we can derive the following rules for $\mathbf{a}, (\mathbf{3} \subset R$:

- If $\mathbf{a} \twoheadrightarrow (\mathbf{3}$, then $\mathbf{a} \twoheadrightarrow + (\mathbf{3}$. In other words, every functional dependency is also a multivalued dependency.
- If $\mathbf{a} \twoheadrightarrow - (\mathbf{3}$, then $\mathbf{a} \twoheadrightarrow - R - \mathbf{a} - (\mathbf{3}$

Appendix C.1.1 outlines a system of inference rules for multivalued dependencies.

6.5.1 Fourth Normal Form

Consider again our example of the BCNF schema:

$r_2 (ID, deptname, street, city)$

in which the multivalued dependency " $ID \twoheadrightarrow + street, city$ " holds. We saw in the opening paragraphs of Section 8.6 that, although this schema is in BCNF, the design is not ideal, since we must repeat an instructor's address information for each department. We shall see that we can use the given multivalued dependency to improve the database design, by decomposing this schema into a **fourth normal form** decomposition.

A relation schema $r(R)$ is in **fourth normal form** (4NF) with respect to a set D of functional and multivalued dependencies if, for all multivalued dependencies

in D^+ of the form $\mathbf{a} \twoheadrightarrow + (\mathbf{3}$, where $\mathbf{a} \subset R$ and $(\mathbf{3} \subset R$, at least one of the following holds:

- $\mathbf{a} \twoheadrightarrow + (\mathbf{3}$ is a trivial multivalued dependency.
- $\mathbf{a}$ is a superkey for R .

A database design is in 4NF if each member of the set of relation schemas that constitutes the design is in 4NF.

Note that the definition of 4NF differs from the definition of BCNF in only the use of multivalued dependencies. Every 4NF schema is in BCNF. To see this fact, we note that, if a schema $r(R)$ is not in BCNF, then there is a nontrivial functional dependency $a \twoheadrightarrow b$ holding on R , where a is not a superkey. Since $a \twoheadrightarrow b$ implies $a \twoheadrightarrow -+ b$, $r(R)$ cannot be in 4NF.

Let $r(R)$ be a relation schema, and let $r_1(R_1), \dots, r_n(R_n)$ be a decomposition of $r(R)$. To check if each relation schema r_i in the decomposition is in 4NF, we need to find what multivalued dependencies hold on each r_i . Recall that, for a set F of functional dependencies, the restriction F_j of F to R_j is all functional dependencies in F^+ that include *only* attributes of R_j . Now consider a set D of both functional and multivalued dependencies. The **restriction** of D to R_j is the set D_j consisting of:

1. All functional dependencies in D^+ that include only attributes of R_j .
2. All multivalued dependencies of the form:

$$a \twoheadrightarrow -+ b \text{ in } R_j \text{ where } a \subseteq R_j \text{ and } a \twoheadrightarrow -+ b \text{ is in } D^+.$$

6.5.2 4NF Decomposition

The analogy between 4NF and BCNF applies to the algorithm for decomposing a schema into 4NF. Figure 6.10 shows the 4NF decomposition algorithm. It is identical to the BCNF decomposition algorithm, except that it uses multivalued dependencies and uses the restriction of D^+ to R_j .

If we apply the algorithm of Figure 6.10 to $(ID, deptname, street, city)$, we find that $ID \twoheadrightarrow -+ deptname$ is a nontrivial multivalued dependency, and ID is not a superkey for the schema. Following the algorithm, we replace it by two schemas:

$$r_1(ID, deptname) \wedge r_2(ID, street, city)$$

This pair of schemas, which is in 4NF, eliminates the redundancy we encountered earlier.

As was the case when we were dealing solely with functional dependencies, we are interested in decompositions that are lossless and that preserve dependen-

$$result := \{R\};$$


```

done := false;
compute  $D^+$ ; Given schema  $R_i$ , let  $D_i$  denote the restriction of  $D^+$  to  $R_i$ 
while (not done) do
  if (there is a schema  $R_i$  in result that is not in 4NF w.r.t.  $D_i$ ) then begin
    let  $a \twoheadrightarrow^+ (3$  be a nontrivial multivalued dependency that holds on  $R_i$  such that  $a \twoheadrightarrow$ 
     $> R_i$  is not in  $D_i$ , and  $a \downarrow (3 = 0$ ; result := (result  $\multimap R$ )  $\cup (R \multimap (3) \cup (a, (3)$ ; end else done
    := true;

```

Figure 6.10 4NF decomposition algorithm.

cies. The following fact about multivalued dependencies and losslessness shows that the algorithm of Figure 8.10 generates only lossless decompositions:

- Let $r(R)$ be a relation schema, and let D be a set of functional and multivalued dependencies on R . Let $r_1(R_1)$ and $r_2(R_2)$ form a decomposition of R . This decomposition is lossless of R if and only if at least one of the following multivalued dependencies is in D^+ :

$$R_1 \cap R_2 \twoheadrightarrow^+ R_1$$

$$R_1 \cap R_2 \twoheadrightarrow^+ R_2$$

Recall that we stated in Section 6.4.4 that, if $R_1 \cap R_2 \twoheadrightarrow^+ R_1$ or $R_1 \cap R_2 \twoheadrightarrow^+ R_2$, then $r_1(R_1)$ and $r_2(R_2)$ are a lossless decomposition $r(R)$. The preceding fact about multivalued dependencies is a more general statement about losslessness. It says that, for every lossless decomposition of $r(R)$ into two schemas $r_1(R_1)$ and $r_2(R_2)$, one of the two dependencies $R_1 \cap R_2 \twoheadrightarrow^+ R_1$ or $R_1 \cap R_2 \twoheadrightarrow^+ R_2$ must hold.

The issue of dependency preservation when we decompose a relation schema becomes more complicated in the presence of multivalued dependencies. Appendix C.1.2 pursues this topic.

6.6 More Normal Forms

The fourth normal form is by no means the “ultimate” normal form. As we saw earlier, multivalued dependencies help us understand and eliminate some forms of repetition of information that cannot be understood in terms of functional dependencies. There are types of constraints called **join dependencies** that generalize multivalued dependencies,

and lead to another normal form called **project-join normal form (PJNF)** (PJNF is called **fifth normal form** in some books). There is a class of even more general constraints that leads to a normal form called **domain-key normal form (DKNF)**.

A practical problem with the use of these generalized constraints is that they are not only hard to reason with, but there is also no set of sound and complete inference rules for reasoning about the constraints. Hence PJNF and DKNF are used quite rarely. Appendix C provides more details about these normal forms.

Conspicuous by its absence from our discussion of normal forms is **second normal form (2NF)**. We have not discussed it, because it is of historical interest only. We simply define it, and let you experiment with it in Practice Exercise 8.17.

6.7 Web Interfaces to Databases

The World Wide Web (Web, for short), is a distributed information system based on hypertext. Web interfaces to databases have become very important. After outlining several reasons for interfacing databases with the Web, we provide an overview of Web technology (Section 8.3). We then outline techniques for building Web interfaces to databases, using servlets and server-side scripting languages (Section 8.4). We round out this topic by outlining techniques for building large scale Web applications and improving their performance in Section 8.5.

The Web has become important as a front end to databases for several reasons: Web browsers provide a universal front end to information supplied by back ends located anywhere in the world. The front end can run on any computer system, and there is no need for a user to download any special-purpose software to access information. Further, today, almost everyone who can afford it has access to the Web.

With the growth of information services and electronic commerce on the Web, databases used for information services, decision support, and transaction processing must be linked with the Web. The HTML forms interface is convenient for transaction processing. The user can fill in details in an order form, then click a submit button to send a message to the server. The server executes an application program corresponding to the order form and this action in turn executes transactions on a database at the server site. The server formats the results of the transaction and sends them back to the user.

Another reason for interfacing databases to the Web is that presenting only static (fixed) documents on a Web site has some limitations, even when the user is not doing any querying or transaction processing.

- Fixed Web documents do not allow the display to be tailored to the user. For instance, a newspaper may want to tailor its display on a per-user basis, to give prominence to news articles that are likely to be of interest to the user.
- When data about an organization is updated, Web documents based on the data become obsolete if they are not updated simultaneously. The problem becomes more acute if multiple Web documents replicate important data and all must be updated.

We can fix these problems by generating Web documents dynamically from a database. When a document is requested, a program gets executed at the server site, which in turn runs queries on a database and generates the requested document on the basis of the query results. Whenever relevant data in the database are updated, the generated documents will automatically become up-to-date. The generated document can also be tailored to the user on the basis of user information stored in the database.

Web interfaces provide attractive benefits even for database applications that are used only with a single organization. The HyperText Markup Language (HTML) standard allows text to be neatly formatted, with important information highlighted. Hyperlinks, which are links to other documents, can be associated with regions of the displayed data. Clicking on a hyperlink fetches and displays the linked document. Hyperlinks are very useful for browsing data, permitting users to get more details of parts of the data as desired.

Finally, browsers today can fetch programs along with HTML documents and run the programs on the browser, in safe mode - that is, without damaging data on the user's computer. Programs can be written in client-side scripting languages, such as Javascript, or can be "applets" written in the Java language, or animations written in languages such as Flash or Shockwave. These programs permit the construction of sophisticated user interfaces, beyond what is possible with just HTML, interfaces that can be used without downloading and installing any software. Thus, Web interfaces are powerful and visually attractive and have eclipsed special-purpose user interfaces for a large variety of database applications.

6.8 Summary

- We showed pitfalls in database design, and how to systematically design a database schema that avoids the pitfalls. The pitfalls included repeated information and inability to represent some information.

- We showed the development of a relational database design from an E-R design, when schemas may be combined safely, and when a schema should be decomposed. All valid decompositions must be lossless.
- We described the assumptions of atomic domains and first normal form.
- We introduced the concept of functional dependencies, and used it to present two normal forms, Boyce-Codd normal form (BCNF) and third normal form (3NF).
- If the decomposition is dependency preserving, given a database update, all functional dependencies can be verifiable from individual relations, without computing a join of relations in the decomposition.
- We showed how to reason with functional dependencies. We placed special emphasis on what dependencies are logically implied by a set of dependencies. We also defined the notion of a canonical cover, which is a minimal set of functional dependencies equivalent to a given set of functional dependencies.
- We outlined an algorithm for decomposing relations into BCNF. There are relations for which there is no dependency-preserving BCNF decomposition.
- We used the canonical covers to decompose a relation into 3NF, which is a small relaxation of the BCNF condition. Relations in 3NF may have some redundancy, but there is always a dependency-preserving decomposition into 3NF.

6.9 Questions for Discussion

1. Express the Features of Relational Design
2. Discuss about the Multivalued Dependencies
3. Describe about the Normal Form

6.10 Suggested Reading

Recommended Text

A.Silberchatz, H.F Korth and S.Sudharshan, 2006, Database System Concepts, 5th Edition, Tata McGraw Hill, New Delhi.

Reference Books

J.D Ullaman, 1988, Principles of Database Systems, Galgotia Publishers, New Delhi.

Lesson - 7

OBJECT BASED DATABASES

CONTENTS

- 7.0 Aims and Objectives**
- 7.1 Introduction**
- 7.2 Complex Data Types**
- 7.3 Structured Types and Inheritance in SQL**
- 7.4 Table Inheritance**
- 7.5 Persistent**
- 7.6 Summary**
- 7.7 Questions for Discussion**
- 7.8 Suggested Reading**

7.0 Aims and Objectives

- Define Complex Data types
- Discuss about the Inheritance in SQL
- Describe about the Persistent

7.1 Introduction

Traditional database applications consist of data-processing tasks, such as bank-ing and payroll management, with relatively simple data types that are well suited to the relational data model. As database systems were applied to a wider range of applications, such as computer-aided design and geographical informa-tion systems, limitations imposed by the relational model emerged as an obstacle. The solution was the introduction of object-based databases, which allow one to deal with complex data types.

7.2 Complex Data Types

Traditional database applications have conceptually simple data types. The basic data items are records that are fairly small and whose fields are atomic—that is, they are not further structured, and first normal form holds . Further, there are only a few record types.

In recent years, demand has grown for ways to deal with more complex data types. Consider, for example, addresses. While an entire address could be viewed as an atomic data item of type string, this view would hide details such as the street address, city, state, and postal code, which could be of interest to queries. On the other hand, if an address were represented by breaking it into the components (street address, city, state, and postal code), writing queries would be more complicated since they would have to mention each field. A better alternative is to allow structured data types that allow a type *address* with subparts *street-address*, *city*, *state*, and *postalcode*.

As another example, consider multivalued attributes from the E-R model. Such attributes are natural, for example, for representing phone numbers, since people

<i>title</i>	<i>author-array</i>	<i>Publisher</i> (<i>name</i> , <i>branch</i>)	<i>keywordset</i>
Compilers	[Smith, Jones]	(McGraw-Hill, NewYork)	{parsing, analysis}
Networks	[Jones, Frick]	(Oxford, London)	{Internet, Web}

Figure 7.1 Non-1NF books relation, books.

may have more than one phone. The alternative of normalization by creating a new relation is expensive and artificial for this example.

With complex type systems we can represent E-R model concepts, such as composite attributes, multivalued attributes, generalization, and specialization directly, without a complex translation to the relational model.

In Chapter 6, we defined *first normal form* (1NF), which requires that all attributes have *atomic domains*. Recall that a domain is *atomic* if elements of the domain are considered to be indivisible units.

The assumption of 1NF is a natural one in the database application examples we have considered. However, not all applications are best modeled by 1NF relations. For example, rather than view a database as a set of records, users of certain applications view it as a set of objects (or entities). These objects may require several records for their representation. A simple, easy-to-use interface requires a one-to-one correspondence between the user's intuitive notion of an object and the database system's notion of a data item.

Consider, for example, a library application, and suppose we wish to store the following information for each book:

- Book title.
- List of authors.
- Publisher.
- Set of keywords.

We can see that, if we define a relation for the preceding information, several domains will be nonatomic.

- **Authors.** A book may have a list of authors, which we can represent as an array. Nevertheless, we may want to find all books of which Jones was one of the authors. Thus, we are interested in a subpart of the domain element “authors.”
- **Keywords.** If we store a set of keywords for a book, we expect to be able to retrieve all books whose keywords include one or more specified keywords. Thus, we view the domain of the set of keywords as nonatomic.
- **Publisher.** Unlike *keywords* and *authors*, *publisher* does not have a set-valued domain. However, we may view *publisher* as consisting of the subfields *name* and *branch*. This view makes the domain of *publisher* nonatomic.

<i>title</i>	<i>author</i>	<i>position</i>
Compilers	Smith	1
Compilers	Jones	2
Networks	Jones	1
Networks	Frick	2

authors

<i>title</i>	<i>keyword</i>
Compilers	parsing
Compilers	analysis
Networks	Internet
Networks	Web

keywords

Figure 7.2 shows an example relation, *books*.

<i>title</i>	<i>pub_name</i>	<i>pub_branch</i>
Compilers	McGraw-Hill	New York
Networks	Oxford	London

books4

Figure 7.3 4NF version of the relation books.

For simplicity, we assume that the title of a book uniquely identifies the book.¹ We can then represent the same information using the following schema, where the primary key attributes are underlined:

- *authors*(title, author, position)
- *keywords*(title, keyword)
- *books4*(title, pub-name, pub-branch)

The above schema satisfies 4NF. Figure 7.2 shows the normalized representation of the data from Figure 7.1.

Although our example book database can be adequately expressed without using nested relations, the use of nested relations leads to an easier-to-understand model. The typical user or programmer of an information-retrieval system thinks of the database in terms of books having sets of authors, as the non-1NF design models. The 4NF design requires queries to join multiple relations, whereas the non-1NF design makes many types of queries easier.

On the other hand, it may be better to use a first normal form representation in other situations. For instance, consider the *takes* relationship in our university example. The relationship is many-to-many between *student* and *section*. We could conceivably store a set of sections with each student, or a set of students with each section, or both. If we store both, we would have data redundancy (the relationship of a particular student to a particular section would be stored twice). The ability to use complex data types such as sets and arrays can be useful in many applications but should be used with care.

7.3 Structured Types and Inheritance in SQL

Before SQL:1999, the SQL type system consisted of a fairly simple set of predefined types. SQL:1999 added an extensive type system to SQL, allowing structured types and type inheritance.

7.3.1 Structured Types

Structured types allow composite attributes of E-R designs to be represented directly. For instance, we can define the following structured type to represent a composite attribute *name* with component attribute *firstname* and *lastname*:

```
create type Name as
    (firstname varchar(20), lastname varchar(20)) final;
```

Similarly, the following structured type can be used to represent a composite attribute *address*:

```
create type Address as (street varchar(20),
    city varchar(20),
    zipcode varchar(9))
not final;
```

Such types are called **user-defined** types in SQL². The above definition corresponds to the E-R diagram in Figure 7.4. The **final** and **not final** specifications are related to subtyping, which we describe later.

We can now use these types to create composite attributes in a relation, by simply declaring an attribute to be of one of these types. For example, we could create a table *person* as follows:

```
create table person ( name Name, address Address, dateOfBirth date);
```

The components of a composite attribute can be accessed using a “dot” notation; for instance *name.firstname* returns the *firstname* component of the *name* attribute. An access to attribute *name* would return a value of the structured type *Name*.

We can also create a table whose rows are of a user-defined type. For example, we could define a type *PersonType* and create the table *person* as follows:⁴

```
create type PersonType as (
    name Name,
    address Address,
    dateOfBirth date)
not final create table person of PersonType;
```

An alternative way of defining composite attributes in SQL is to use unnamed **row types**. For instance, the relation representing person information could have been created using row types as follows:

```
create table persons (  
  name row (firstname varchar(20),  
    lastname varchar(20)), address row  
  (street varchar(20), city varchar(20),  
    zipcode varchar(9)), dateOfBirth date);
```

This definition is equivalent to the preceding table definition, except that the attributes *name* and *address* have unnamed types, and the rows of the table also have an unnamed type.

The following query illustrates how to access component attributes of a composite attribute. The query finds the last name and city of each person.

```
select name.lastname, address.city from person;
```

A structured type can have **methods** defined on it. We declare methods as part of the type definition of a structured type:

```
create type PersonType as (  
  name Name,  
  address Address,  
  dateOfBirth date)  
not final method ageOnDate(onDate date)  
returns interval year;
```

We create the method body separately:

```
create instance method ageOnDate (onDate date) returns interval year  
for PersonType begin  
  return onDate — self.dateOfBirth; end
```

Note that the **for** clause indicates which type this method is for, while the keyword **instance** indicates that this method executes on an instance of the *Person* type. The variable **self** refers to the *Person* instance on which the method is invoked. The body of the method can contain procedural statements, which we saw earlier in Section 5.2. Methods can update the attributes of the instance on which they are executed.

Methods can be invoked on instances of a type. If we had created a table *person* of type *PersonType*, we could invoke the method *ageOnDate()* as illustrated below, to find the age of each person.

```
select name.lastname,
       ageOnDate(current_date) from person;
```

In SQL:1999, **constructor functions** are used to create values of structured types. A function with the same name as a structured type is a constructor function for the structured type. For instance, we could declare a constructor for the type *Name* like this:

```
create function Name (firstname varchar(20), lastname varchar(20))
returns Name
begin
set self.firstname = firstname; set self.lastname = lastname; end
```

We can then use **new** *Name*('John', 'Smith') to create a value of the type *Name*. We can construct a row value by listing its attributes within parentheses. For instance, if we declare an attribute *name* as a row type with components *firstname* and *lastname* we can construct this value for it: ('Ted', 'Codd') without using a constructor.

By default every structured type has a constructor with no arguments, which sets the attributes to their default values. Any other constructors have to be created explicitly. There can be more than one constructor for the same structured type; although they have the same name, they must be distinguishable by the number of arguments and types of their arguments.

The following statement illustrates how we can create a new tuple in the *Person* relation. We assume that a constructor has been defined for *Address*, just like the constructor we defined for *Name*.

```
insert into Person values
(new Name('John', 'Smith'),
 new Address('20 Main St', 'New York', '11001'),
 date '1960-8-22');
```

7.3.2 Type Inheritance

Suppose that we have the following type definition for people:

```
create type Person
```

```
(name varchar(20), address varchar(20));
```

We may want to store extra information in the database about people who are students, and about people who are teachers. Since students and teachers are also people, we can use inheritance to define the student and teacher types in SQL:

```
create type Student under Person
```

```
(degree varchar(20), department varchar(20));
```

```
create type Teacher under Person
```

```
(salary integer, department varchar(20));
```

Both *Student* and *Teacher* inherit the attributes of *Person*—namely, *name* and *address*. *Student* and *Teacher* are said to be subtypes of *Person*, and *Person* is a supertype of *Student*, as well as of *Teacher*.

Methods of a structured type are inherited by its subtypes, just as attributes are. However, a subtype can redefine the effect of a method by declaring the method again, using **overriding method** in place of **method** in the method declaration.

The SQL standard requires an extra field at the end of the type definition, whose value is either **final** or **not final**. The keyword **final** says that subtypes may not be created from the given type, while **not final** says that subtypes may be created.

Now suppose that we want to store information about teaching assistants, who are simultaneously students and teachers, perhaps even in different departments. We can do this if the type system supports **multiple inheritance**, where a type is declared as a subtype of multiple types. Note that the SQL standard does not support multiple inheritance, although future versions of the SQL standard may support it, so we discuss the concept here.

For instance, if our type system supports multiple inheritance, we can define a type for teaching assistant as follows:

```
create type TeachingAssistant under Student, Teacher;
```

TeachingAssistant inherits all the attributes of *Student* and *Teacher*. There is a problem, however, since the attributes *name*, *address*, and *department* are present in *Student*, as well as in *Teacher*.

The attributes *name* and *address* are actually inherited from a common source, *Person*. So there is no conflict caused by inheriting them from *Student* as well as *Teacher*.

However, the attribute *department* is defined separately in *Student* and *Teacher*. In fact, a teaching assistant may be a student of one department and a teacher in another department. To avoid a conflict between the two occurrences of *department*, we can rename them by using an **as** clause, as in this definition of the type *TeachingAssistant*:

```
create type TeachingAssistant
under Student with
  (department as student dept),
  Teacher with (department as teacher dept);
```

In SQL, as in most other languages, a value of a structured type must have exactly one *most-specific type*. That is, each value must be associated with one specific type, called its **most-specific type**, when it is created. By means of inheritance, it is also associated with each of the supertypes of its most-specific type. For example, suppose that an entity has the type *Person*, as well as the type *Student*. Then, the most-specific type of the entity is *Student*, since *Student* is a subtype of *Person*. However, an entity cannot have the type *Student* as well as the type *Teacher* unless it has a type, such as *TeachingAssistant*, that is a subtype of *Teacher*, as well as of *Student* (which is not possible in SQL since multiple inheritance is not supported by SQL).

7.4 Table Inheritance

Subtables in SQL correspond to the E-R notion of specialization/generalization. For instance, suppose we define the *people* table as follows:

```
create table people of Person;
```

We can then define tables *students* and *teachers* as **subtables** of *people*, as follows:

```
create table students of Student under people;
create table teachers of Teacher under people;
```

The types of the subtables (*Student* and *Teacher* in the above example) are subtypes of the type of the parent table (*Person* in the above example). As a result, every attribute present in the table *people* is also present in the subtables *students* and *teachers*.

Further, when we declare *students* and *teachers* as subtables of *people*, every tuple present in *students* or *teachers* becomes implicitly present in *people*. Thus, if a query uses the table *people*, it will find not only tuples directly inserted into that table, but also tuples inserted into its subtables, namely *students* and *teachers*. However, only those attributes that are present in *people* can be accessed by that query.

SQL permits us to find tuples that are in *people* but not in its subtables by using “**only people**” in place of *people* in a query. The **only** keyword can also be used in delete and update statements. Without the **only** keyword, a delete statement on a supertable, such as *people*, also deletes tuples that were originally inserted in subtables (such as *students*); for example, a statement:

delete from people where *P*;

would delete all tuples from the table *people*, as well as its subtables *students* and *teachers*, that satisfy *P*. If the **only** keyword is added to the above statement, tuples that were inserted in subtables are not affected, even if they satisfy the **where** clause conditions. Subsequent queries on the supertable would continue to find these tuples.

Conceptually, multiple inheritance is possible with tables, just as it is possible with types. For example, we can create a table of type *TeachingAssistant*:

**create table teachingassistants of TeachingAssistant
under students, teachers;**

As a result of the declaration, every tuple present in the *teaching-assistants* table is also implicitly present in the *teachers* and in the *students* table, and in turn in the *people* table. We note, however, that multiple inheritance of tables is not supported by SQL.

There are some consistency requirements for subtables. Before we state the constraints, we need a definition: we say that tuples in a subtable and parent table **correspond** if they have the same values for all inherited attributes. Thus, corresponding tuples represent the same entity.

The consistency requirements for subtables are:

1. Each tuple of the supertable can correspond to at most one tuple in each of its immediate subtables.
2. SQL has an additional constraint that all the tuples corresponding to each other must be derived from one tuple (inserted into one table).

For example, without the first condition, we could have two tuples in *students* (or *teachers*) that correspond to the same person.

The second condition rules out a tuple in *people* corresponding to both a tuple in *students* and a tuple in *teachers*, unless all these tuples are implicitly present because a tuple was inserted in a table *teaching-assistants*, which is a subtable of both *teachers* and *students*.

Since SQL does not support multiple inheritance, the second condition actually prevents a person from being both a teacher and a student. Even if multiple inheritance were supported, the same problem would arise if the subtable *teaching .assistants* were absent. Obviously it would be useful to model a situation where a person can be a teacher and a student, even if a common subtable *teaching -assistants* is not present. Thus, it can be useful to remove the second consistency constraint. Doing so would allow an object to have multiple types, without requiring it to have a most-specific type.

For example, suppose we again have the type *Person*, with subtypes *Student* and *Teacher*, and the corresponding table *people*, with subtables *teachers* and *students*. We can then have a tuple in *teachers* and a tuple in *students* corresponding to the same tuple in *people*. There is no need to have a type *TeachingAssistant* that is a subtype of both *Student* and *Teacher*. We need not create a type *TeachingAssistant* unless we wish to store extra attributes or redefine methods in a manner specific to people who are both students and teachers.

We note, however, that SQL unfortunately prohibits such a situation, because of consistency requirement 2. Since SQL also does not support multiple inheritance, we cannot use inheritance to model a situation where a person can be both a student and a teacher. As a result, SQL subtables cannot be used to represent overlapping specializations from the E-R model.

We can of course create separate tables to represent the overlapping specializations/generalizations without using inheritance. The process was described earlier, Section we would create tables *people*, *students*, and *teachers*, with the *students* and *teachers* tables containing the primary-key attribute of *Person* and other attributes specific to *Student* and *Teacher*, respectively. The *people* table would contain information about all persons, including students and teachers. We would then have to add appropriate referential-integrity constraints to ensure that students and teachers are also represented in the *people* table.

In other words, we can create our own improved implementation of the subtable mechanism using existing features of SQL, with some extra effort in defining the table, as well as some extra effort at query time to specify joins to access required attributes.

We note that SQL defines a privilege called **under**, which is required in order to create a subtype or subtable under another type or table. The motivation for this privilege is similar to that for the **references** privilege.

7.5 Persistent Programming Languages

Database languages differ from traditional programming languages in that they directly manipulate data that are persistent—that is, data that continue to exist even after the program that created it has terminated. A relation in a database and tuples in a relation are examples of persistent data. In contrast, the only persistent data that traditional programming languages directly manipulate are files.

Access to a database is only one component of any real-world application. While a data-manipulation language like SQL is quite effective for accessing data, a programming language is required for implementing other components of the application such as user interfaces or communication with other computers. The traditional way of interfacing database languages to programming languages is by embedding SQL within the programming language.

A **persistent programming language** is a programming language extended with constructs to handle persistent data. Persistent programming languages can be distinguished from languages with embedded SQL in at least two ways:

1. With an embedded language, the type system of the host language usually differs from the type system of the data-manipulation language. The programmer is responsible for any type conversions between the host language and SQL. Having the programmer carry out this task has several drawbacks:
 - The code to convert between objects and tuples operates outside the object-oriented type system, and hence has a higher chance of having undetected errors.
 - Conversion between the object-oriented format and the relational format of tuples in the database takes a substantial amount of code. The format translation code, along with the code for loading and unloading data from a database, can form a significant percentage of the total code required for an application.

In contrast, in a persistent programming language, the query language is fully integrated with the host language, and both share the same type system. Objects can be created and stored in the database without any explicit type or format changes; any format changes required are carried out transparently.

2. The programmer using an embedded query language is responsible for writing explicit code to fetch data from the database into memory. If any updates are performed, the programmer must write code explicitly to store the updated data back in the database.

In contrast, in a persistent programming language, the programmer can manipulate persistent data without writing code explicitly to fetch it into memory or store it back to disk.

In this section, we describe how object-oriented programming languages, such as C++ and Java, can be extended to make them persistent programming languages. These language features allow programmers to manipulate data directly from the programming language, without having to go through a data-manipulation language such as SQL. They thereby provide tighter integration of the programming languages with the database than, for example, embedded SQL.

There are certain drawbacks to persistent programming languages, however, that we must keep in mind when deciding whether to use them. Since the programming language is usually a powerful one, it is relatively easy to make programming errors that damage the database. The complexity of the language makes automatic high-level optimization, such as to reduce disk I/O, harder. Support for declarative querying is important for many applications, but persistent programming languages currently do not support declarative querying well.

In this section, we describe a number of conceptual issues that must be addressed when adding persistence to an existing programming language. We first address language-independent issues, and in subsequent sections we discuss issues that are specific to the C++ language and to the Java language. However, we do not cover details of language extensions; although several standards have been proposed, none has met universal acceptance. See the references in the bibliographical notes to learn more about specific language extensions and further details of implementations.

7.5.1 Persistence of Objects

Object-oriented programming languages already have a concept of objects, a type system to define object types, and constructs to create objects. However, these objects are *transient*—they vanish when the program terminates, just as variables in a Java or C program vanish when the program terminates. If we wish to turn such a language into a database programming language, the first step is to provide a way to make objects persistent. Several approaches have been proposed.

- **Persistence by class.** The simplest, but least convenient, way is to declare that a class is persistent. All objects of the class are then persistent objects by default. Objects of nonpersistent classes are all transient.

This approach is not flexible, since it is often useful to have both transient and persistent objects in a single class. Many object-oriented database systems interpret declaring a class to be persistent as saying that objects in the class potentially can be made persistent, rather than that all objects in the class are persistent. Such classes might more appropriately be called “persistable” classes.

- **Persistence by creation.** In this approach, new syntax is introduced to create persistent objects, by extending the syntax for creating transient objects. Thus, an object is either persistent or transient, depending on how it was created. Several object-oriented database systems follow this approach.
- **Persistence by marking.** A variant of the preceding approach is to mark objects as persistent after they are created. All objects are created as transient objects, but, if an object is to persist beyond the execution of the program, it must be marked explicitly as persistent before the program terminates. This approach, unlike the previous one, postpones the decision on persistence or transience until after the object is created.
- **Persistence by reachability.** One or more objects are explicitly declared as (root) persistent objects. All other objects are persistent if (and only if) they are reachable from the root object through a sequence of one or more references.

Thus, all objects referenced by (that is, whose object identifiers are stored in) the root persistent objects are persistent. But also, all objects referenced from these objects are persistent, and objects to which they refer are in turn persistent, and so on.

A benefit of this scheme is that it is easy to make entire data structures persistent by merely declaring the root of such structures as persistent. However, the database system has the burden of following chains of references to detect which objects are persistent, and that can be expensive.

7.5.2 Object Identity and Pointers

In an object-oriented programming language that has not been extended to handle persistence, when an object is created, the system returns a transient object identifier. Transient object identifiers are valid only when the program that created them is executing; after that program terminates, the objects are deleted, and the identifier is meaningless. When a persistent object is created, it is assigned a persistent object identifier.

The notion of object identity has an interesting relationship to pointers in programming languages. A simple way to achieve built-in identity is through pointers to physical locations in storage. In particular, in many object-oriented languages such as C++, a transient object identifier is actually an in-memory pointer.

However, the association of an object with a physical location in storage may change over time. There are several degrees of permanence of identity:

- **Intraprocedure.** Identity persists only during the execution of a single procedure. Examples of intraprogram identity are local variables within procedures.
- **Intraprogram.** Identity persists only during the execution of a single program or query. Examples of intraprogram identity are global variables in programming languages. Main-memory or virtual-memory pointers offer only intraprogram identity.
- **Interprogram.** Identity persists from one program execution to another. Pointers to file-system data on disk offer interprogram identity, but they may change if the way data is stored in the file system is changed.
- **Persistent.** Identity persists not only among program executions, but also among structural reorganizations of the data. It is the persistent form of identity that is required for object-oriented systems.

In persistent extensions of languages such as C++, object identifiers for persistent objects are implemented as “persistent pointers.” A *persistent pointer* is a type of pointer that, unlike in-memory pointers, remains valid even after the end of a program execution, and across some forms of data reorganization. A programmer may use a persistent pointer in the same ways that she may use an in-memory pointer in a programming language. Conceptually, we may think of a persistent pointer as a pointer to an object in the database.

7.5.3 Storage and Access of Persistent Objects

What does it mean to store an object in a database? Clearly, the data part of an object has to be stored individually for each object. Logically, the code that implements methods of a class should be stored in the database as part of the database schema, along with the type definitions of the classes. However, many implementations simply store the code in files outside the database, to avoid having to integrate system software such as compilers with the database system.

There are several ways to find objects in the database. One way is to give names to objects, just as we give names to files. This approach works for a relatively small number of objects, but does not scale to millions of objects. A second way is to expose object identifiers or persistent pointers to the objects, which can be stored externally. Unlike names, these pointers do not have to be mnemonic, and they can even be physical pointers into a database.

A third way is to store collections of objects, and to allow programs to iterate over the collections to find required objects. Collections of objects can themselves be modeled as objects of a *collection type*. Collection types include sets, multisets (that is, sets with possibly many occurrences of a value), lists, and so on. A special case of a collection is a **class extent**, which is the collection of all objects belonging to the class. If a class extent is present for a class, then, whenever an object of the class is created, that object is inserted in the class extent automatically, and, whenever an object is deleted, that object is removed from the class extent. Class extents allow classes to be treated like relations in that we can examine all objects in the class, just as we can examine all tuples in a relation.

Most object-oriented database systems support all three ways of accessing persistent objects. They give identifiers to all objects. They usually give names only to class extents and other collection objects, and perhaps to other selected objects, but not to most objects. They usually maintain class extents for all classes that can have persistent objects, but, in many of the implementations, the class extents contain only persistent objects of the class.

7.5.4 Persistent C++ Systems

There are several object-oriented databases based on persistent extensions to C++ (see the bibliographical notes). There are differences among them in terms of the system architecture, yet they have many common features in terms of the programming language.

Several of the object-oriented features of the C++ language provide support for persistence without changing the language itself. For example, we can declare a class called `PersistentObject` with attributes and methods to support persistence; any other class that should be persistent can be made a subclass of this class, and thereby inherit the support for persistence. The C++ language (like some other modern programming languages) also lets us redefine standard function names and operators—such as `+`, `—`, the pointer dereference operator `—>`, and so on—according to the types of the operands on which they are applied. This ability is called *overloading*; it is used to redefine operators to behave in the required manner when they are operating on persistent objects.

Providing persistence support via class libraries has the benefit of making only minimal changes to C++ necessary; moreover, it is relatively easy to implement. However, it has the drawback that the programmer has to spend much more time to write a program that handles persistent objects, and it is not easy for the programmer to specify integrity constraints on the schema or to provide support for declarative querying. Some persistent C++ implementations support extensions to the C++ syntax to make these tasks easier.

The following aspects need to be addressed when adding persistence support to C++ (and other languages):

- **Persistent pointers:** A new data type has to be defined to represent persistent pointers. For example, the ODMG C++ standard defines a template class `d_Ref < T >` to represent persistent pointers to a class `T`. The dereference operator on this class is redefined to fetch the object from disk (if not already present in memory), and it returns an in-memory pointer to the buffer where the object has been fetched. Thus if `p` is a persistent pointer to a class `T`, one can use standard syntax such as `p—>A` or `p—>f(v)` to access attribute `A` of class `T` or invoke method `f` of class `T`.

The ObjectStore database system uses a different approach to persistent pointers. It uses normal pointer types to store persistent pointers. This poses two problems: (1) in-memory pointer sizes may be only 4 bytes, which is too small to use with databases larger than 4 gigabytes, and (2) when an object is moved on disk, in-memory pointers to its old physical location are meaningless. ObjectStore uses a technique called “hardware swizzling” to address both problems; it prefetches objects from the database into memory, and replaces persistent pointers with in-memory pointers, and when data are stored back on disk, in-memory pointers are replaced by persistent pointers. When on disk, the value stored in the in-memory pointer field is not the actual persistent pointer; instead, the value is looked up in a table that contains the full persistent pointer value.

- **Creation of persistent objects:** The C++ `new` operator is used to create persistent objects by defining an “overloaded” version of the operator that takes extra arguments specifying that it should be created in the database. Thus instead of `new T()`, one would call `new (db) T()` to create a persistent object, where `db` identifies the database.
- **Class extents:** Class extents are created and maintained automatically for each class. The ODMG C++ standard requires the name of the class to be

passed as an additional parameter to the new operation. This also allows multiple extents to be maintained for a class, by passing different names.

- **Relationships:** Relationships between classes are often represented by storing pointers from each object to the objects to which it is related. Objects related to multiple objects of a given class store a set of pointers. Thus if a pair of objects is in a relationship, each should store a pointer to the other. Persistent C++ systems provide a way to specify such integrity constraints and to enforce them by automatically creating and deleting pointers: For example, if a pointer is created from an object *a* to an object *b*, a pointer to *a* is added automatically to object *b*.
- **Iterator interface:** Since programs need to iterate over class members, an interface is required to iterate over members of a class extent. The iterator interface also allows selections to be specified, so that only objects satisfying the selection predicate need to be fetched.
- **Transactions:** Persistent C++ systems provide support for starting a transaction, and for committing it or rolling it back.
- **Updates:** One of the goals of providing persistence support in a programming language is to allow transparent persistence. That is, a function that operates on an object should not need to know that the object is persistent; the same functions can thus be used on objects regardless of whether they are persistent or not.

However, one resultant problem is that it is difficult to detect when an object has been updated. Some persistent extensions to C++ require the programmer to specify explicitly that an object has been modified by calling a function `mark_modified()`. In addition to increasing programmer effort, this approach increases the chance that programming errors can result in a corrupt database. If a programmer omits a call to `mark_modified()`, it is possible that one update made by a transaction may never be propagated to the database, while another update made by the same transaction is propagated, violating atomicity of transactions.

Other systems, such as ObjectStore, use memory-protection support provided by the operating system/hardware to detect writes to a block of memory and mark the block as a dirty block that should be written later to disk.

- **Query language:** Iterators provide support for simple selection queries. To support more complex queries, persistent C++ systems define a query language.

A large number of object-oriented database systems based on C++ were developed in the late 1980s and early 1990s. However, the market for such databases turned out to be much smaller than anticipated, since most application requirements are more than met by using SQL through interfaces such as ODBC or JDBC. As a result, most of the object-oriented database systems developed in that period do not exist any longer. In the 1990s, the Object Data Management Group (ODMG) defined standards for adding persistence to C++ and Java. However, the group wound up its activities around 2002. ObjectStore and Versant are among the original object-oriented database systems that are still in existence.

Although object-oriented database systems did not find the commercial success that they had hoped for, the motivation for adding persistence to programming language remains. There are several applications with high performance requirements that run on object-oriented database systems; using SQL would impose too high a performance overhead for many such systems. With object-relational database systems now providing support for complex data types, including references, it is easier to store programming language objects in an SQL database.

7.6 Summary

- The object-relational data model extends the relational data model by providing a richer type system including collection types and object orientation.
- Collection types include nested relations, sets, multisets, and arrays, and the object-relational model permits attributes of a table to be collections.
- Object orientation provides inheritance with subtypes and subtables, as well as object (tuple) references.
- The SQL standard includes extensions of the SQL data-definition and query language to deal with new data types and with object orientation. These include support for collection-valued attributes, inheritance, and tuple references. Such extensions attempt to preserve the relational foundations—in particular, the declarative access to data—while extending the modeling power.
- Object-relational database systems (that is, database systems based on the object-relation model) provide a convenient migration path for users of relational databases who wish to use object-oriented features.

- Persistent extensions to C++ and Java integrate persistence seamlessly and orthogonally with existing programming language constructs and so are easy to use.

7.7 Questions for Discussion

1. Define Complex Data types
2. Discuss about the Inheritance in SQL
3. Describe about the Persistent programming

7.8 Suggested Reading

Recommended Text

A.Silberchatz, H.F Korth and S.Sudharshan, 2006, Database System Concepts, 5th Edition, Tata McGraw Hill, New Delhi.

Reference Books

J.D Ullaman, 1988, Principles of Database Systems, Galgotia Publishers, New Delhi.

Lesson - 8

STORAGE AND FILE STRUTURE

CONTENTS

8.0 Aim and Objectives

8.1 Introduction

8.1.1 Overview of Physical Storage Media

8.2 Magnetic Disk and Flash Storage

8.2.1 Physical Characteristics of Disks

8.2.2 Performance Measures of Disks

8.2.3 Optimization of Disk-Block & Class

8.3 RAID

8.3.1 Improvement of Reliability via Redundancy

8.3.2 Improvement in Performance via Parallelism

8.3.3 RAID Levels

8.3.4 Choice of RAID Level

8.3.5 Hardware Issues

8.3.6 Other RAID Applications

8.4 Storage Access

8.4.1 Buffer Manager

8.4.2 Buffer-Replacement Policies

8.5 Tertiary Storage

8.5.1 Optical Disks

8.5.2 Magnetic Tapes

8.6 File Organization

8.6.1 Fixed-Length Records

8.6.2 Variable-Length Records

8.7 Summary

8.8 Lesson End Activities

8.9 Keywords

8.10 Questions for Discussion

8.11 Suggested Reading

8.0 AIM AND OBJECTIVES

After studying this lesson, you should be able to:

- overview of physical storage media, including mechanisms to minimize the chance of data loss due to device failures.
- How records are mapped to files, which in turn are mapped to bits on the disk.
- What is RAID techniques.

8.1 Introduction

Storage and File Structure

In preceding chapters, we have emphasized the higher-level models of a database. For example, at the *conceptual* or *logical* level, we viewed the database, in the relational model, as a collection of tables. Indeed, the logical model of the database is the correct level for database *users* to focus on. This is because the goal of a database system is to simplify and facilitate access to data; users of the system should not be burdened unnecessarily with the physical details of the implementation of the system.

8.1.1 Overview of Physical Storage Media

Several types of data storage exist in most computer systems. These storage media are classified by the speed with which data can be accessed, by the cost per unit of data to buy the medium, and by the medium's reliability. Among the media typically available are these:

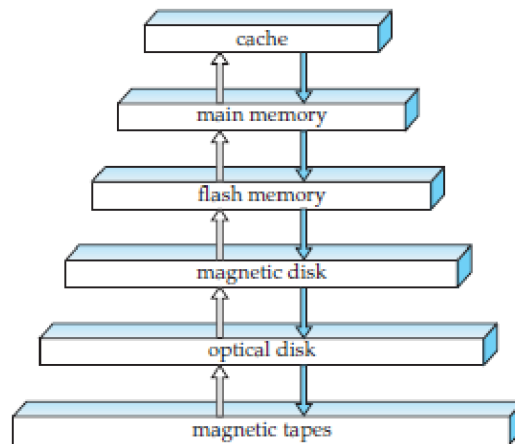


Figure 8.1 Storage device hierarchy.

- **Cache.** The cache is the fastest and most costly form of storage. Cachenemory is relatively small; its use is managed by the computer system hardware. We shall not be concerned about managing cache storage in the database system. It is, however, worth noting that database implementors do pay attention to cache effects when designing query processing data structures and algorithms.
- **Main memory.** The storage medium used for data that are available to be operated on is main memory. The general-purpose machine instructions operate on main memory. Although main memory may contain several gigabytes of data on a personal computer, or even hundreds of gigabytes of data in large server systems, it is generally too small (or too expensive) for storing the entire database. The contents of main memory are usually lost if a power failure or system crash occurs.
- **Flash memory.** Flash memory differs from main memory in that stored data are retained even if power is turned off (or fails). There are two types of flash memory, called *NAND* and *NOR* flash.

NAND flash has a much higher storage capacity for a given cost, and is widely used for data storage in devices such as cameras, music players, and cell phones, and increasingly, in laptop computers as well. Flash memory has a lower cost per byte than main memory, in addition to being nonvolatile; that is, it retains stored data even if power is switched off. Flash memory is also widely used for storing data in “USB keys,” which can be plugged into the **Universal Serial Bus** (USB) slots of computing devices.

- **Magnetic-disk storage.** The primary medium for the long-term online storage of data is the magnetic disk. Usually, the entire database is stored on magnetic disk. The system must move the data from disk to main memory so that they can be accessed. After the system has performed the designated operations, the data that have been modified must be written to disk.
- **Optical storage.** The most popular forms of optical storage are the *compact disk* (CD), which can hold about 700 megabytes of data and has a playtime of about 80 minutes, and the *digital video disk* (DVD), which can hold 4.7 or 8.5 gigabytes of data per side of the disk (or up to 17 gigabytes on a two-sided disk). The expression **digital versatile disk** is also used in place of **digital video disk**, since DVDs can hold any digital data, not just video data. Data are stored optically on a disk, and are read by a laser. A higher capacity format called *Blu-ray DVD* can store 27 gigabytes per layer, or 54 gigabytes in a double-layer disk.
- **Tape storage.** Tape storage is used primarily for backup and archival data. Although magnetic tape is cheaper than disks, access to data is much slower, because the tape must be accessed sequentially from the beginning. For this reason, tape storage is referred to as **sequential-access** storage. In contrast, disk storage is referred to as **direct-access** storage because it is possible to read data from any location on disk. Tapes have a high capacity (40- to 300-gigabyte tapes are currently available), and can be removed from the tape drive, so they are well suited to cheap archival storage.

8.2 Magnetic Disk and Flash Storage

Magnetic disks provide the bulk of secondary storage for modern computer systems. Although disk capacities have been growing year after year, the storage requirements of large applications have also been growing very fast, in some cases even faster than the growth rate of disk capacities. A very large database may require hundreds of disks. In recent years, flash-memory storage sizes have grown rapidly, and flash storage is increasingly becoming a competitor to magnetic disk storage for several applications.

8.2.1 Physical Characteristics of Disks

Physically, disks are relatively simple (Figure 8.2). Each disk **platter** has a flat, circular shape. Its two surfaces are covered with a magnetic material, and information is recorded on the surfaces. Platters are made from rigid metal or glass. When the disk is in use, a drive motor spins it at a constant high speed (usually 60, 90, or 120 revolutions per second, but disks running at 250 revolutions per second are available). There is a read–write head

positioned just above the surface of the platter. The disk surface is logically divided into **tracks**, which are subdivided into **sectors**. A **sector** is the smallest unit of information that can be read from or written to the disk.

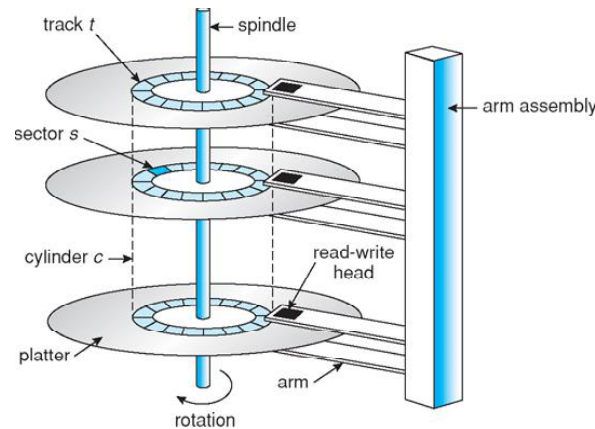


Figure 8.2 Moving head disk mechanism.

typically 512 bytes; there are about 50,000 to 100,000 tracks per platter, and 1 to 5 platters per disk. The inner tracks (closer to the spindle) are of smaller length, and in current-generation disks, the outer tracks contain more sectors than the inner tracks; typical numbers are around 500 to 1000 sectors per track in the inner tracks, and around 1000 to 2000 sectors per track in the outer tracks. The numbers vary among different models; higher-capacity models usually have more sectors per track and more tracks on each platter.

The **read–write head** stores information on a sector magnetically as reversals of the direction of magnetization of the magnetic material. Each side of a platter of a disk has a read–write head that moves across the platter to access different tracks. A disk typically contains many platters, and the read–write heads of all the tracks are mounted on a single assembly called a **disk arm**, and move together. The disk platters mounted on a spindle and the heads mounted on a disk arm are together known as **head–disk assemblies**. Since the heads on all the platters move together, when the head on one platter is on the i th track, the heads on all other platters are also on the i th track of their respective platters. Hence, the i th tracks of all the platters together are called the i th **cylinder**.

Today, disks with a platter diameter of 3.5 inches dominate the market. They have a lower cost and faster seek times (due to smaller seek distances) than do the larger-diameter disks (up to 14 inches) that were common earlier, yet they provide high storage capacity. Disks with even smaller diameters are used in portable devices such as laptop computers, and some handheld computers and portable music players.

The read–write heads are kept as close as possible to the disk surface to increase the recording density. The head typically floats or flies only microns from the disk surface; the spinning of the disk creates a small breeze, and the head assembly is shaped so that the breeze keeps the head floating just above the disk surface. Because the head floats so close to the surface, platters must be machined carefully to be flat. Head crashes can be a problem. If the head contacts the disk surface, the head can scrape the recording medium off the disk, destroying the data that had been there. In older-generation disks, the head touching the surface caused the removed medium to become airborne and to come between the other heads and their platters, causing more crashes; a head crash could thus result in failure of the entire disk. Current-generation disk drives use a thin film of magnetic metal as recording medium. They are much less susceptible to failure by head crashes than the older oxide-coated disks.

A **disk controller** interfaces between the computer system and the actual hardware of the disk drive; in modern disk systems, the disk controller is implemented within the disk drive unit. A disk controller accepts high-level commands to read or write a sector, and initiates actions, such as moving the disk arm to the right track and actually reading or writing the data. Disk controllers also attach **checksums** to each sector that is written; the checksum is computed from the data written to the sector. When the sector is read back, the controller computes the checksum again from the retrieved data and compares it with the stored checksum; if the data are corrupted, with a high probability the newly computed checksum will not match the stored checksum. If such an error occurs, the controller will retry the read several times; if the error continues to occur, the controller will signal a read failure.

Another interesting task that disk controllers perform is **remapping of bad sectors**. If the controller detects that a sector is damaged when the disk is initially formatted, or when an attempt is made to write the sector, it can logically map the sector to a different physical location (allocated from a pool of extra sectors set aside for this purpose). The remapping is noted on disk or in nonvolatile memory, and the write is carried out on the new location. Disks are connected to a computer system through a high-speed interconnection.

There are a number of common interfaces for connecting disks to computers of which the most commonly used today are

The ATA attachment (ATA) interface and a small-computer system interconnect (SCSI) are commonly used to connect disk to personal computer and workstations.

While disks are usually connected directly by cables to the disk interface of the computer system, they can be situated remotely and connected by a high-speed network to the disk controller. In the **storage area network (SAN)** architecture, large numbers of disks are connected by a high-speed network to a number 1ATA is a storage-device connection standard from the 1980s. of server computers. The disks are usually organized locally using a storage organization technique called **redundant arrays of independent disks (RAID)** to give the servers a logical view of a very large and very reliable disk. The computer and the disk subsystem continue to use the SCSI, SAS, or Fiber Channel interface protocols to talk with each other, although they may be separated by a network. Remote access to disks across a storage area network means that disks can be shared by multiple computers that could run different parts of an application in parallel. Remote access also means that disks containing important data can be kept in a central server room where they can be monitored and maintained by system administrators, instead of being scattered in different parts of an organization.

Network attached storage (NAS) is an alternative to SAN. NAS is much like SAN, except that instead of the networked storage appearing to be a large disk, it provides a file system interface using networked file system protocols such as NFS or CIFS.

8.2.2 Performance Measures of Disks

The main measures of the qualities of a disk are capacity, access time, data-transfer rate, and reliability.

Access time is the time from when a read or write request is issued to when data transfer begins. To access (that is, to read or write) data on a given sector of a disk, the arm first must move so that it is positioned over the correct track, and then must wait for the sector to appear under it as the disk rotates. The time for repositioning the arm is called the **seek time**, and it increases with the distance that the arm must move. Typical seek times range from 2 to 30 milliseconds, depending on how far the track is from the initial arm position. Smaller disks tend to have lower seek times since the head has to travel a smaller distance.

The **average seek time** is the average of the seek times, measured over a sequence of (uniformly distributed) random requests. If all tracks have the same number of sectors, and we disregard the time required for the head to start moving and to stop moving, we can show that the average seek time is one-third the worst-case seek time. Taking these factors into account, the average seek time is around one-half of the maximum seek time. Average seek times currently range between 4 and 10 milliseconds, depending on the disk model.

Once the head has reached the desired track, the time spent waiting for the sector to be accessed to appear under the head is called the **rotational latency time**. Rotational speeds of disks today range from 5400 rotations per minute (90 rotations per second) up to 15,000 rotations per minute (250 rotations per second), or, equivalently, 4 milliseconds to 11.1 milliseconds per rotation. On an average, one-half of a rotation of the disk is required for the beginning of the desired sector to appear under the head. Thus, the **average latency time** of the disk is one-half the time for a full rotation of the disk.

The access time is then the sum of the seek time and the latency, and ranges from 8 to 20 milliseconds. Once the first sector of the data to be accessed has come under the head, data transfer begins. The **data-transfer rate** is the rate at which data can be retrieved from or stored to the disk. Current disk systems support maximum transfer rates of 25 to 100 megabytes per second; transfer rates are significantly lower than the maximum transfer rates for inner tracks of the disk, since they have fewer sectors. For example, a disk with a maximum transfer rate of 100 megabytes per second may have a sustained transfer rate of around 30 megabytes per second on its inner tracks.

The final commonly used measure of a disk is the **mean time to failure (MTTF)**, which is a measure of the reliability of the disk. The mean time to failure of a disk (or of any other system) is the amount of time that, on average, we can expect the system to run continuously without any failure. According to vendors' claims, the mean time to failure of disks today ranges from 500,000 to 1,200,000 hours—about 57 to 136 years. In practice the claimed mean time to failure is computed on the probability of failure when the disk is new—the figure means that given 1000 relatively new disks, if the MTTF is 1,200,000 hours, on an average one of them will fail in 1200 hours. A mean time to failure of 1,200,000 hours does not imply that the disk can be expected to function for 136 years! Most disks have an expected life span of about 5 years, and have significantly higher rates of failure once they become more than a few years old. Disk drives for desktop machines typically support the Serial ATA (SATA) interface, which supports 150 megabytes per second, or the SATA-II 3Gb interface, which supports 300 megabytes per second. The PATA 5 interface supported transfer rates of 133 megabytes per second. Disk drives designed for server systems typically support the Ultra320 SCSI interface, which provides transfer rates of up to 320 megabytes per second, or the Serial Attached SCSI (SAS) interface, versions of which provide transfer rates of 3 or 6 gigabits per second. Storage area network (SAN) devices, which are connected to servers by a network, typically use Fiber Channel FC 2-Gb or 4-Gb interface, which provides transfer rates of up to 256 or 512 megabytes per second. The transfer rate of an interface is shared between all disks attached to the interface, except for the serial interfaces which allow only one disk to be connected to each interface.

8.2.3 Optimization of Disk-Block Access

Requests for disk I/O are generated both by the file system and by the virtual memory manager found in most operating systems. Each request specifies the address on the disk to be referenced; that address is in the form of a *block number*.

A **block** is a logical unit consisting of a fixed number of contiguous sectors. Block sizes range from 512 bytes to several kilobytes.

Data are transferred between disk and main memory in units of blocks. The term **page** is often used to refer to blocks, although in a few contexts (such as flash memory) they refer to different things. A sequence of requests for blocks from disk may be classified as a sequential access pattern or a random access pattern. In a **sequential access** pattern, successive requests are for successive block numbers, which are on the same track, or on adjacent tracks. To read blocks in sequential access, a disk seek may be required for the first block, but successive requests would either not require a seek, or require a seek to an adjacent track, which is faster than a seek to a track that is farther away.

In contrast, in a **random access** pattern, successive requests are for blocks that are randomly located on disk. Each such request would require a seek. The number of random block accesses that can be satisfied by a single disk in a second depends on the seek time, and is typically about 100 to 200 accesses per second.

Since only a small amount (one block) of data is read per seek, the transfer rate is significantly lower with a random access pattern than with a sequential access pattern.

A number of techniques have been developed for improving the speed of access to blocks.

- **Buffering.** Blocks that are read from disk are stored temporarily in an in-memory buffer, to satisfy future requests. Buffering is done by both the operating system and the database system.
- **Read-ahead.** When a disk block is accessed, consecutive blocks from the same track are read into an in-memory buffer even if there is no pending request for the blocks. In the case of sequential access, such read-ahead ensures that many blocks are already in memory when they are requested, and minimizes the time wasted in disk seeks and rotational latency per block read. Operating systems also routinely perform read-ahead for consecutive blocks of an operating system file. Read-ahead is, however, not very useful for random block accesses.

- **Scheduling.** If several blocks from a cylinder need to be transferred from disk to main memory, we may be able to save access time by requesting the blocks in the order in which they will pass under the heads. If the desired blocks are on different cylinders, it is advantageous to request the blocks in an order that minimizes disk-arm movement.

Disk-arm-scheduling algorithms attempt to order accesses to tracks in a fashion that increases the number of accesses that can be processed. A commonly used algorithm is the **elevator algorithm**, which works in the same way many elevators do. Suppose that, initially, the arm is moving from the innermost track toward the outside of the disk.

Under the elevator algorithm's control, for each track for which there is an access request, the arm stops at that track, services requests for the track, and then continues moving outward until there are no waiting requests for tracks farther out. At this point, the arm changes direction, and moves toward the inside, again stopping at each track for which there is a request, until it reaches a track where there is no request for tracks farther toward the center. Then, it reverses direction and starts a new cycle. Disk controllers usually perform the task of reordering read requests to improve performance, since they are intimately aware of the organization of blocks on disk, of the rotational position of the disk platters, and of the position of the disk arm.

- **File organization.** To reduce block-access time, we can organize blocks on disk in a way that corresponds closely to the way we expect data to be accessed. For example, if we expect a file to be accessed sequentially, then we should ideally keep all the blocks of the file sequentially on adjacent cylinders. Older operating systems, such as the IBM mainframe operating systems, provided programmers fine control on placement of files, allowing a programmer to reserve a set of cylinders for storing a file. However, this control places a burden on the programmer or system administrator to decide, for example, how many cylinders to allocate for a file, and may require costly reorganization if data are inserted to or deleted from the file. Subsequent operating systems, such as Unix and Microsoft Windows, hide the disk organization from users, and manage the allocation internally.

Although they do not guarantee that all blocks of a file are laid out sequentially, they allocate multiple consecutive blocks (an **extent**) at a time to a file.

Sequential access to the file then only needs one seek per extent, instead of one seek per block. Over time, a sequential file that has multiple small appends may become **fragmented**; that is, its blocks become scattered all over the disk. To reduce fragmentation,

the system can make a backup copy of the data on disk and restore the entire disk. The restore operation writes back the blocks of each file contiguously (or nearly so). Some systems (such as different versions of the Windows operating system) have utilities that scan the disk and then move blocks to decrease the fragmentation. The performance increases realized from these techniques can be large.

- **Nonvolatile write buffers.** Since the contents of main memory are lost in a power failure, information about database updates has to be recorded on disk to survive possible system crashes. For this reason, the performance of update-intensive database applications, such as transaction-processing systems, is heavily dependent on the speed of disk writes.

We can use **nonvolatile random-access memory** (NVRAM) to speed up disk writes drastically. The contents of NVRAM are not lost in power failure. A common way to implement NVRAM is to use battery-backed-up RAM, although flash memory is also increasingly being used for nonvolatile write buffering. The idea is that, when the database system (or the operating system) requests that a block be written to disk, the disk controller writes the block to an NVRAM buffer, and immediately notifies the operating system that the write completed successfully. The controller writes the data to their destination on disk whenever the disk does not have any other requests, or when the NVRAM buffer becomes full. When the database system requests a block write, it notices a delay only if the NVRAM buffer is full. On recovery from a system crash, any pending buffered writes in the NVRAM are written back to the disk. NVRAM buffers are found in certain high end disks, but are more frequently found in “RAID controllers”.

- **Log disk.** Another approach to reducing write latencies is to use a log disk—that is, a disk devoted to writing a sequential log—in much the same way as a nonvolatile RAM buffer. All access to the log disk is sequential, essentially eliminating seek time, and several consecutive blocks can be written at once, making writes to the log disk several times faster than random writes. As before, the data have to be written to their actual location on disk as well, but the log disk can do the write later, without the database system having to wait for the write to complete. Furthermore, the log disk can reorder the writes to minimize disk-arm movement. If the system crashes before some writes to the actual disk location have completed, when the system comes back up it reads the log disk to find those writes that had not been completed, and carries them out then. File systems that support log disks as above are called **journaling file systems**. Journaling file systems can be implemented even without

a separate log disk, keeping data and the log on the same disk. Doing so reduces the monetary cost, at the expense of lower performance.

Most modern file systems implement journaling, and use the log disk when writing internal file system information such as file allocation information. Earlier-generation file systems allowed write reordering without using a log disk, and ran the risk that the file system data structures on disk would be corrupted if the system crashed. Suppose, for example, that a file system used a linked list, and inserted a new node at the end by first writing the data for the new node, then updating the pointer from the previous node.

Suppose also that the writes were reordered, so the pointer was updated first, and the system crashes before the new node is written. The contents of the node would then be whatever junk was on disk earlier, resulting in a corrupted data structure. To deal with the possibility of such data structure corruption, earlier-generation file systems had to perform a file system consistency check on system restart, to ensure that the data structures were consistent. And if they were not, extra steps had to be taken to restore them to consistency.

These checks resulted in long delays in system restart after a crash, and the delays became worse as disk systems grew to higher capacities. Journaling file systems allow quick restart without the need for such file system consistency checks.

However, writes performed by applications are usually not written to the log disk. Database systems implement their own forms of logging.

8.3 RAID

The data-storage requirements of some applications (in particular Web, database, and multimedia applications) have been growing so fast that a large number of disks are needed to store their data, even though disk-drive capacities have been growing very fast.

Having a large number of disks in a system presents opportunities for improving the rate at which data can be read or written, if the disks are operated in parallel. Several independent reads or writes can also be performed in parallel.

Furthermore, this setup offers the potential for improving the reliability of data storage, because redundant information can be stored on multiple disks. Thus, failure of one disk does not lead to loss of data. A variety of disk-organization techniques, collectively called **redundant arrays of independent disks (RAID)**, have been proposed to achieve improved performance and reliability. In the past, system designers viewed storage systems composed of several small, cheap disks as a cost-effective alternative to using large, expensive disks;

the cost per megabyte of the smaller disks was less than that of larger disks. In fact, the I in RAID, which now stands for *independent*, originally stood for *inexpensive*.

Today, however, all disks are physically small, and larger-capacity disks actually have a lower cost per megabyte. RAID systems are used for their higher reliability and higher performance rate, rather than for economic reasons. Another key justification for RAID use is easier management and operations.

8.3.1 Improvement of Reliability via Redundancy

Let us first consider reliability. The chance that at least one disk out of a set of N disks will fail is much higher than the chance that a specific single disk will fail. Suppose that the mean time to failure of a disk is 100,000 hours, or slightly over 11 years. Then, the mean time to failure of some disk in an array of 100 disks will be $100,000/100 = 1000$ hours, or around 42 days, which is not long at all! If we store only one copy of the data, then each disk failure will result in loss of a significant amount of data. Such a high frequency of data loss is unacceptable.

The solution to the problem of reliability is to introduce **redundancy**; that is, we store extra information that is not needed normally, but that can be used in the event of failure of a disk to rebuild the lost information. Thus, even if a disk fails, data are not lost, so the effective mean time to failure is increased, provided that we count only failures that lead to loss of data or to nonavailability of data. The simplest (but most expensive) approach to introducing redundancy is to duplicate every disk. This technique is called **mirroring** (or, sometimes, *shadowing*). A logical disk then consists of two physical disks, and every write is carried

out on both disks. If one of the disks fails, the data can be read from the other. Data will be lost only if the second disk fails before the first failed disk is repaired. The mean time to failure (where failure is the loss of data) of a mirrored disk depends on the mean time to failure of the individual disks, as well as on the **mean time to repair**, which is the time it takes (on an average) to replace a failed disk and to restore the data on it. Suppose that the failures of the two disks are *independent*; that is, there is no connection between the failure of one disk and the failure of the other. Then, if the mean time to failure of a single disk is 100,000 hours, and the mean time to repair is 10 hours, the **mean time to data loss** of a mirrored disk system is $100,000^2/(2 \times 10) = 500 \times 10^6$ hours, or 57,000 years! (We do not go into the derivations here; references in the bibliographical notes provide the details.)

You should be aware that the assumption of independence of disk failures is not valid. Power failures, and natural disasters such as earthquakes, fires, and floods, may

result in damage to both disks at the same time. As disks age, the probability of failure increases, increasing the chance that a second disk will fail while the first is being repaired. In spite of all these considerations, however, mirrored-disk systems offer much higher reliability than do single-disk systems. Mirrored-disk systems with mean time to data loss of about 500,000 to 1,000,000 hours, or 55 to 110 years, are available today.

Power failures are a particular source of concern, since they occur far more frequently than do natural disasters. Power failures are not a concern if there is no data transfer to disk in progress when they occur. However, even with mirroring of disks, if writes are in progress to the same block in both disks, and power fails before both blocks are fully written, the two blocks can be in an inconsistent state. The solution to this problem is to write one copy first, then the next, so that one of the two copies is always consistent. Some extra actions are required when we restart after a power failure, to recover from incomplete writes.

8.3.2 Improvement in Performance via Parallelism

Now let us consider the benefit of parallel access to multiple disks. With disk mirroring, the rate at which read requests can be handled is doubled, since read requests can be sent to either disk (as long as both disks in a pair are functional, as is almost always the case). The transfer rate of each read is the same as in a single-disk system, but the number of reads per unit time has doubled. With multiple disks, we can improve the transfer rate as well (or instead) by **striping data** across multiple disks. In its simplest form, data striping consists of splitting the bits of each byte across multiple disks; such striping is called **bitlevel striping**. For example, if we have an array of eight disks, we write bit i of each byte to disk i . The array of eight disks can be treated as a single disk with sectors that are eight times the normal size, and, more important, that has eight times the transfer rate. In such an organization, every disk participates in every access (read or write), so the number of accesses that can be processed per second is about the same as on a single disk, but each access can read eight times as many data in the same time as on a single disk. Bit-level striping can be generalized to a number of disks that either is a multiple of 8 or a factor of 8. For example, if we use an array of four disks, bits i and $4 + i$ of each byte go to disk i .

Block-level striping stripes blocks across multiple disks. It treats the array of disks as a single large disk, and it gives blocks logical numbers; we assume the block numbers start from 0. With an array of n disks, block-level striping assigns logical block i of the disk

array to disk $(i \bmod n) + 1$; it uses the $[i/n]$ th physical block of the disk to store logical block i .

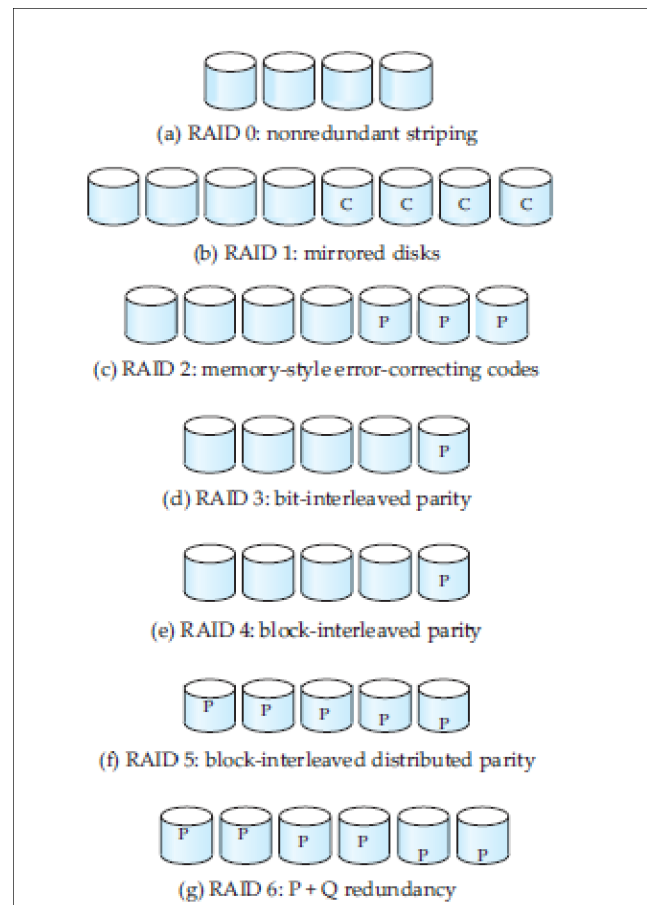


Figure 8.3 RAID levels.

For example, with 8 disks, logical block 0 is stored in physical block 0 of disk 1, while logical block 11 is stored in physical block 1 of disk 4. When reading a large file, block-level striping fetches n blocks at a time in parallel from the n disks, giving a high data-transfer rate for large reads. When a single block is read, the data-transfer rate is the same as on one disk, but the remaining $n - 1$ disks are free to perform other actions.

Block-level striping is the most commonly used form of data striping. Other levels of striping, such as bytes of a sector or sectors of a block, also are possible.

In summary, there are two main goals of parallelism in a disk system:

1. Load-balance multiple small accesses (block accesses), so that the throughput of such accesses increases.
2. Parallelize large accesses so that the response time of large accesses is reduced.

8.3.3 RAID Levels

Mirroring provides high reliability, but it is expensive. Striping provides high data-transfer rates, but does not improve reliability. Various alternative schemes aim to provide redundancy at lower cost by combining disk striping with “parity” bits. These schemes have different cost–performance trade-offs. The schemes are classified into **RAID levels**, as in Figure 8.3. (In the figure, P indicates error-correcting bits, and C indicates a second copy of the data.) For all levels, the figure depicts four disks’ worth of data, and the extra disks depicted are used to store redundant information for failure recovery.

- **RAID level 0** refers to disk arrays with striping at the level of blocks, but without any redundancy (such as mirroring or parity bits). Figure 8.3a shows an array of size 4.
- **RAID level 1** refers to disk mirroring with block striping. Figure 8.3b shows a mirrored organization that holds four disks’ worth of data.
- **RAID level 2**, known as memory-style error-correcting-code (ECC) organization, employs parity bits. Memory systems have long used parity bits for error detection and correction. Each byte in a memory system may have a parity bit associated with it that records whether the numbers of bits in the byte that are set to 1 is even (parity = 0) or odd (parity = 1). If one of the bits in the byte gets damaged (either a 1 becomes a 0, or a 0 becomes a 1), the parity of the byte changes and thus will not match the stored parity. Similarly, if the stored parity bit gets damaged, it will not match the computed parity. Thus, all 1-bit errors will be detected by the memory system. Error-correcting schemes store 2 or more extra bits, and can reconstruct the data if a single bit gets damaged.

The idea of error-correcting codes can be used directly in disk arrays by striping bytes across disks. For example, the first bit of each byte could be stored in disk 0, the second bit in disk 1, and so on until the eighth bit is stored in disk 7, and the error-correction bits are stored in further disks.

Figure 8.3c shows the level 2 scheme. The disks labeled *P* store the error-correction bits. If one of the disks fails, the remaining bits of the byte and the associated error-correction bits can be read from other disks, and can be used to reconstruct the damaged data. Figure 8.3c shows an array of size 4; note RAID level 2 requires only three disks’ overhead for four disks of data, unlike RAID level 1, which required four disks’ overhead.

- **RAID level 3**, bit-interleaved parity organization, improves on level 2 by exploiting the fact that disk controllers, unlike memory systems, can detect whether a sector has

been read correctly, so a single parity bit can be used for error correction, as well as for detection. The idea is as follows: If one of the sectors gets damaged, the system knows exactly which sector it is, and, for each bit in the sector, the system can figure out whether it is a 1 or a 0 by computing the parity of the corresponding bits from sectors in the other disks. If the parity of the remaining bits is equal to the stored parity, the missing bit is 0; otherwise, it is 1. RAID level 3 is as good as level 2, but is less expensive in the number of extra disks (it has only a one-disk overhead), so level 2 is not used in practice. Figure 8.3d shows the level 3 scheme.

RAID level 3 has two benefits over level 1. It needs only one parity disk for several regular disks, whereas level 1 needs one mirror disk for every disk, and thus level 3 reduces the storage overhead. Since reads and writes of a byte are spread out over multiple disks, with N -way striping of data, the transfer rate for reading or writing a single block is N times faster than a RAID level 1 organization using N -way striping. On the other hand, RAID level 3 supports a lower number of I/O operations per second, since every disk has to participate in every I/O request.

- **RAID level 4**, block-interleaved parity organization, uses block-level striping, like RAID 0, and in addition keeps a parity block on a separate disk for corresponding blocks from N other disks. This scheme is shown pictorially in Figure 10.3e. If one of the disks fails, the parity block can be used with the corresponding blocks from the other disks to restore the blocks of the failed disk.

A block read accesses only one disk, allowing other requests to be processed by the other disks. Thus, the data-transfer rate for each access is slower, but multiple read accesses can proceed in parallel, leading to a higher overall I/O rate. The transfer rates for large reads is high, since all the disks can be read in parallel; large writes also have high transfer rates, since the data and parity can be written in parallel. Small independent writes, on the other hand, cannot be performed in parallel. A write of a block has to access the disk on which the block is stored, as well as the parity disk, since the parity block has to be updated. Moreover, both the old value of the parity block and the old value of the block being written have to be read for the new parity to be computed. Thus, a single write requires four disk accesses: two to read the two old blocks, and two to write the two blocks.

- **RAID level 5**, block-interleaved distributed parity, improves on level 4 by partitioning data and parity among all $N+1$ disks, instead of storing data in N disks and parity in one disk. In level 5, all disks can participate in satisfying read requests, unlike RAID level 4, where the parity disk cannot participate, so level 5 increases the total number

of requests that can be met in a given amount of time. For each set of N logical blocks, one of the disks stores the parity, and the other N disks store the blocks. Figure 10.3f shows the setup. The P's are distributed across all the disks.

For example, with an array of 5 disks, the parity block, labeled P_k , for logical blocks $4k, 4k + 1, 4k + 2, 4k + 3$ is stored in disk $k \bmod 5$; the corresponding blocks of the other four disks store the 4 data blocks $4k$ to $4k+3$. The following table indicates how the first 20 blocks, numbered 0 to 19, and their parity blocks are laid out. The pattern shown gets repeated on further blocks.

P0	0	1	2	3
4	P1	5	6	7
8	9	P2	10	11
12	13	14	P3	15
16	17	18	19	P4

Note that a parity block cannot store parity for blocks in the same disk, since then a disk failure would result in loss of data as well as of parity, and hence would not be recoverable. Level 5 subsumes level 4, since it offers better read–write performance at the same cost, so level 4 is not used in practice.

- **RAID level 6**, the P + Q redundancy scheme, is much like RAID level 5, but stores extra redundant information to guard against multiple disk failures. Instead of using parity, level 6 uses error-correcting codes such as the Reed–Solomon codes (see the bibliographical notes). In the scheme in Figure 8.3g, 2 bits of redundant data are stored for every 4 bits of data—unlike 1 parity bit in level 5—and the system can tolerate two disk failures.

Finally, we note that several variations have been proposed to the basic RAID schemes described here, and different vendors use different terminologies for the variants.

8.3.4 Choice of RAID Level

The factors to be taken into account in choosing a RAID level are:

- Monetary cost of extra disk-storage requirements.
- Performance requirements in terms of number of I/O operations.

- Performance when a disk has failed.
- Performance during rebuild (that is, while the data in a failed disk are being rebuilt on a new disk).

The time to rebuild the data of a failed disk can be significant, and it varies with the RAID level that is used. Rebuilding is easiest for RAID level 1, since data can be copied from another disk; for the other levels, we need to access all the other disks in the array to rebuild data of a failed disk. The **rebuild performance** of a RAID system may be an important factor if continuous availability of data is required, as it is in high-performance database systems. Furthermore, since rebuild time can form a significant part of the repair time, rebuild performance also influences the mean time to data loss.

RAID level 0 is used in high-performance applications where data safety is not critical. Since RAID levels 2 and 4 are subsumed by RAID levels 3 and 5, the choice of RAID levels is restricted to the remaining levels. Bit striping (level 3) is inferior to block striping (level 5), since block striping gives as good data-transfer rates for large transfers, while using fewer disks for small transfers. For small transfers, the disk access time dominates anyway, so the benefit of parallel reads diminishes. In fact, level 3 may perform worse than level 5 for a small transfer, since the transfer completes only when corresponding sectors on all disks have been fetched; the average latency for the disk array thus becomes very close to the worst-case latency for a single disk, negating the benefits of higher transfer rates. Level 6 is not supported currently by many RAID implementations, but it offers better reliability than level 5 and can be used in applications where data safety is very important.

The choice between RAID level 1 and level 5 is harder to make. RAID level 1 is popular for applications such as storage of log files in a database system, since it offers the best write performance. RAID level 5 has a lower storage overhead than level 1, but has a higher time overhead for writes. For applications where data are read frequently, and written rarely, level 5 is the preferred choice. Disk-storage capacities have been growing at a rate of over 50 percent per year for many years, and the cost per byte has been falling at the same rate. As a result, for many existing database applications with moderate storage requirements, the monetary cost of the extra disk storage needed for mirroring has become relatively small (the extra monetary cost, however, remains a significant issue for storage intensive applications such as video data storage). Access speeds have improved at a much slower rate (around a factor of 3 over 10 years), while the number of I/O operations required per second has increased tremendously, particularly for Web application servers.

RAID level 5, which increases the number of I/O operations needed to write a single logical block, pays a significant time penalty in terms of write performance. RAID level 1 is therefore the RAID level of choice for many applications with moderate storage requirements and high I/O requirements. RAID system designers have to make several other decisions as well. For example, how many disks should there be in an array? How many bits should be protected by each parity bit? If there are more disks in an array, data-transfer rates are higher, but the system will be more expensive. If there are more bits protected by a parity bit, the space overhead due to parity bits is lower, but there is an increased chance that a second disk will fail before the first failed disk is repaired, and that will result in data loss.

8.3.5 Hardware Issues

Another issue in the choice of RAID implementations is at the level of hardware. RAID can be implemented with no change at the hardware level, using only software modification. Such RAID implementations are called **software RAID**.

However, there are significant benefits to be had by building special-purpose hardware to support RAID, which we outline below; systems with special hardware support are called **hardware RAID** systems.

Hardware RAID implementations can use nonvolatile RAM to record writes before they are performed. In case of power failure, when the system comes back up, it retrieves information about any incomplete writes from nonvolatile RAM and then completes the writes. Without such hardware support, extra work needs to be done to detect blocks that may have been partially written before power failure (see Practice Exercise 10.3). Even if all writes are completed properly, there is a small chance of a sector in a disk becoming unreadable at some point, even though it was successfully written earlier. Reasons for loss of data on individual sectors could range from manufacturing defects, to data corruption on a track when an adjacent track is written repeatedly. Such loss of data that were successfully written earlier is sometimes referred to as a *latent failure*, or as *bit rot*. When such a failure happens, if it is detected early the data can be recovered from the remaining disks in the RAID organization. However, if such a failure remains undetected, a single disk failure could lead to data loss if a sector in one of the other disks has a latent failure.

To minimize the chance of such data loss, good RAID controllers perform **scrubbing**; that is, during periods when disks are idle, every sector of every disk is read, and if any sector is found to be unreadable, the data are recovered from the remaining disks in the RAID organization, and the sector is written back. (If the physical sector is damaged, the

disk controller would remap the logical sector address to a different physical sector on disk.) Some hardware RAID implementations permit **hot swapping**; that is, faulty disks can be removed and replaced by new ones without turning power off. Hot swapping reduces the mean time to repair, since replacement of a disk does not have to wait until a time when the system can be shut down. In fact many critical systems today run on a 24×7 schedule; that is, they run 24 hours a day, 7 days a week, providing no time for shutting down and replacing a failed disk. Further, many RAID implementations assign a spare disk for each array (or for a set of disk arrays). If a disk fails, the spare disk is immediately used as a replacement. As a result, the mean time to repair is reduced greatly, minimizing the chance of any data loss. The failed disk can be replaced at leisure.

The power supply, or the disk controller, or even the system interconnection in a RAID system could become a single point of failure that could stop functioning of the RAID system. To avoid this possibility, good RAID implementations have multiple redundant power supplies (with battery backups so they continue to function even if power fails). Such RAID systems have multiple disk interfaces, and multiple interconnections to connect the RAID system to the computer system (or to a network of computer systems). Thus, failure of any single component will not stop the functioning of the RAID system.

8.3.6 Other RAID Applications

The concepts of RAID have been generalized to other storage devices, including arrays of tapes, and even to the broadcast of data over wireless systems. When applied to arrays of tapes, the RAID structures are able to recover data even if one of the tapes in an array of tapes is damaged. When applied to broadcast of data, a block of data is split into short units and is broadcast along with a parity unit; if one of the units is not received for any reason, it can be reconstructed from the other units.

8.4 Storage Access

A major goal of the database system is to minimize the number of block transfers between the disk and memory. One way to reduce the number of disk accesses is to keep as many blocks as possible in main memory. The goal is to maximize the chance that, when a block is accessed, it is already in main memory, and, thus, no disk access is required.

Since it is not possible to keep all blocks in main memory, we need to manage the allocation of the space available in main memory for the storage of blocks. The buffer is that part of main memory available for storage of copies of disk blocks. There is always a

copy kept on disk of every block, but the copy on disk may be a version of the block older than the version in the buffer. The subsystem responsible for the allocation of buffer space is called the buffer manager.

8.4.1 Buffer Manager

Programs in a database system make requests (that is, calls) on the buffer manager when they need a block from disk. If the block is already in the buffer, the buffer manager passes the address of the block in main memory to the requester. If the block is not in the buffer, the buffer manager first allocates space in the buffer for the block, throwing out some other block, if necessary, to make space for the new block. The thrown-out block is written back to disk only if it has been modified since the most recent time that it was written to the disk. Then, the buffer manager reads in the requested block from the disk to the buffer, and passes the address of the block in main memory to the requester. The internal actions of the buffer manager are transparent to the programs that issue disk-block requests. If you are familiar with operating-system concepts, you will note that the buffer manager appears to be nothing more than a virtual-memory manager, like those found in most operating systems. One difference is that the size of the database might be larger than the hardware address space of a machine, so memory addresses are not sufficient to address all disk blocks. Further, to serve the database system well, the buffer manager must use techniques more sophisticated than typical virtual-memory management schemes:

- Buffer replacement strategy. When there is no room left in the buffer, a block must be removed from the buffer before a new one can be read in. Most operating systems use a least recently used (LRU) scheme, in which the block that was referenced least recently is written back to disk and is removed from the buffer. This simple approach can be improved on for database applications.
- Pinned blocks. For the database system to be able to recover from crashes, it is necessary to restrict those times when a block may be written back to disk. For instance, most recovery systems require that a block should not be written to disk while an update on the block is in progress. A block that is not allowed to be written back to disk is said to be pinned. Although many operating systems do not support pinned blocks, such a feature is essential for a database system that is resilient to crashes.
- Forced output of blocks. There are situations in which it is necessary to write back the block to disk, even though the buffer space that it occupies is not needed. This

write is called the forced output of a block. We shall see the reason for forced output briefly, main-memory contents and thus buffer contents are lost in a crash, whereas data on disk usually survive a crash.

8.4.2 Buffer-Replacement Policies

The goal of a replacement strategy for blocks in the buffer is to minimize accesses to the disk. For general-purpose programs, it is not possible to predict accurately which blocks will be referenced. Therefore, operating systems use the past pattern of block references as a predictor of future references. The assumption generally made is that blocks that have been referenced recently are likely to be referenced again. Therefore, if a block must be replaced, the least recently referenced block is replaced. This approach is called the least recently used (LRU) block-replacement scheme.

```

for each tuple i of instructor do
    for each tuple d of department do
        if i[dept name] = d[dept name]
            then begin
                let x be a tuple defined as follows:
                x[ID] := i[ID]
                x[dept name] := i[dept name]
                x[name] := i[name]
                x[salary] := i[salary]
                x[building] := d[building]
                x[budget] := d[budget]
                include tuple x as part of result of instructor _ department
            end
        end
    end
end

```

Figure 8.4 Procedure for computing join.

LRU is an acceptable replacement scheme in operating systems. However, a database system is able to predict the pattern of future references more accurately than an operating system. A user request to the database system involves several steps. The database system is often able to determine in advance which blocks will be needed by looking at each of the steps required to perform the user requested operation. Thus, unlike operating systems, which must rely on the past to predict the future, database systems may have information regarding at least the short-term future.

To illustrate how information about future block access allows us to improve the LRU strategy, consider the processing of the SQL query:

```
select * from instructor natural join department;
```

Assume that the strategy chosen to process this request is given by the pseudocode program shown in Figure 8.4.

Assume that the two relations of this example are stored in separate files. In this example, we can see that, once a tuple of *instructor* has been processed, that tuple is not needed again. Therefore, once processing of an entire block of *instructor* tuples is completed, that block is no longer needed in main memory, even though it has been used recently. The buffer manager should be instructed to free the space occupied by an *instructor* block as soon as the final tuple has been processed. This buffer-management strategy is called the toss-immediate strategy.

Now consider blocks containing *department* tuples. We need to examine every block of *department* tuples once for each tuple of the *instructor* relation. When processing of a *department* block is completed, we know that that block will not be accessed again until all other *department* blocks have been processed. Thus, the most recently used *department* block will be the final block to be re-referenced, and the least recently used *department* block is the block that will be referenced next. This assumption set is the exact opposite of the one that forms the basis for the LRU strategy. Indeed, the optimal strategy for block replacement for the above procedure is the most recently used (MRU) strategy. If a *department* block must be removed from the buffer, the MRU strategy chooses the most recently used block (blocks are not eligible for replacement while they are being used).

For the MRU strategy to work correctly for our example, the system must pin the *department* block currently being processed. After the final *department* tuple has been processed, the block is unpinned, and it becomes the most recently used block.

In addition to using knowledge that the system may have about the request being processed, the buffer manager can use statistical information about the probability that a request will reference a particular relation. For example, the data dictionary that keeps track of the logical schema of the relations as well as their physical storage information is one of the most frequently accessed parts of the database. Thus, the buffer manager should try not to remove data-dictionary blocks from main memory, unless other factors dictate that it do so. Since an index for a file may be accessed more frequently than the file itself, the buffer manager should, in general, not remove index blocks from main memory if alternatives are available.

The ideal database block-replacement strategy needs knowledge of the database operations—both those being performed and those that will be performed in the future. No single strategy is known that handles all the possible scenarios well. Indeed, a surprisingly large number of database systems use LRU, despite that strategy's faults. The practice questions and exercises explore alternative strategies.

The strategy that the buffer manager uses for block replacement is influenced by factors other than the time at which the block will be referenced again. If the system is processing requests by several users concurrently, the concurrency control subsystem may need to delay certain requests, to ensure preservation of database consistency. If the buffer manager is given information from the concurrency-control subsystem indicating which requests are being delayed, it can use this information to alter its block-replacement strategy. Specifically, blocks needed by active (nondelayed) requests can be retained in the buffer at the expense of blocks needed by the delayed requests. The crash-recovery subsystem imposes stringent constraints on block replacement. If a block has been modified, the buffer manager is not allowed to write back the new version of the block in the buffer to disk, since that would destroy the old version. Instead, the block manager must seek permission from the crash-recovery subsystem before writing out a block. The crash-recovery subsystem may demand that certain other blocks be force-output before it grants permission to the buffer manager to output the block requested. We define precisely the interaction between the buffer manager and the crashrecovery subsystem.

8.5 Tertiary Storage

In a large database system, some of the data may have to reside on tertiary storage. The two most common tertiary storage media are optical disks and magnetic tapes.

8.5.1 Optical Disks

Compact disks have been a popular medium for distributing software, multimedia data such as audio and images, and other electronically published information. They have a storage capacity of 640 to 700 megabytes, and they are cheap to mass-produce. Digital video disks (DVDs) have now replaced compact disks in applications that require larger amounts of data. Disks in the DVD-5 format can store 4.7 gigabytes of data (in one recording layer), while disks in the DVD-9 format can store 8.5 gigabytes of data (in two recording layers). Recording on both sides of a disk yields even larger capacities; DVD-10 and DVD-18 formats, which are the two-sided versions of DVD-5 and DVD-9, can store 9.4 gigabytes and 17 gigabytes, respectively. The *Blu-ray DVD* format has a significantly higher capacity of 27 to 54 gigabytes per disk.

CD and DVD drives have much longer seek times (100 milliseconds is common) than do magnetic-disk drives, since the head assembly is heavier. Rotational speeds are typically lower than those of magnetic disks, although the faster CD and DVD drives have rotation speeds of about 3000 rotations per minute, which is comparable to speeds of lower-end magnetic-disk drives. Rotational speeds of CD drives originally corresponded to the audio CD standards, and the speeds of DVD drives originally corresponded to the DVD video standards, but current generation drives rotate at many times the standard rate.

Data-transfer rates are somewhat less than for magnetic disks. Current CD drives read at around 3 to 6 megabytes per second, and current DVD drives read at 8 to 20 megabytes per second. Like magnetic-disk drives, optical disks store more data in outside tracks and less data in inner tracks. The transfer rate of optical drives is characterized as $n\times$, which means the drive supports transfers at n times the standard rate; rates of around $50\times$ for CD and $16\times$ for DVD are now common.

The record-once versions of optical disks (CD-R, DVD-R, and DVD+R) are popular for distribution of data and particularly for archival storage of data because they have a high capacity, have a longer lifetime than magnetic disks, and can be removed and stored at a remote location. Since they cannot be overwritten, they can be used to store information that should not be modified, such as audit trails. The multiple-write versions (CD-RW, DVD-RW, DVD+RW, and DVD-RAM) are also used for archival purposes.

Jukeboxes are devices that store a large number of optical disks (up to several hundred) and load them automatically on demand to one of a small number of drives

(usually 1 to 10). The aggregate storage capacity of such a system can be many terabytes. When a disk is accessed, it is loaded by a mechanical arm from a rack onto a drive (any disk that was already in the drive must first be placed back on the rack). The disk load/unload time is usually of the order of a few seconds—very much longer than disk access times.

8.5.2 Magnetic Tapes

Although magnetic tapes are relatively permanent, and can hold large volumes of data, they are slow in comparison to magnetic and optical disks. Even more important, magnetic tapes are limited to sequential access. Thus, they cannot provide random access for secondary-storage requirements, although historically, prior to the use of magnetic disks, tapes were used as a secondary-storage medium. Tapes are used mainly for backup, for storage of infrequently used information, and as an off-line medium for transferring information from one system to another. Tapes are also used for storing large volumes of data, such as video or image data, that either do not need to be accessible quickly or are so voluminous that magnetic-disk storage would be too expensive. A tape is kept in a spool, and is wound or rewound past a read–write head.

Moving to the correct spot on a tape can take seconds or even minutes, rather than milliseconds; once positioned, however, tape drives can write data at densities and speeds approaching those of disk drives. Capacities vary, depending on the length and width of the tape and on the density at which the head can read and write. The market is currently fragmented among a wide variety of tape formats. Currently available tape capacities range from a few gigabytes with the **Digital Audio Tape** (DAT) format, 10 to 40 gigabytes with the **Digital Linear Tape** (DLT) format, 100 gigabytes and higher with the **Ultrium** format, to 330 gigabytes with **Ampex helical scan** tape formats. Data-transfer rates are of the order of a few to tens of megabytes per second. Tape devices are quite reliable, and good tape drive systems perform a read of the just-written data to ensure that it has been recorded correctly. Tapes, however, have limits on the number of times that they can be read or written reliably. **Tape jukeboxes**, like optical disk jukeboxes, hold large numbers of tapes, with a few drives onto which the tapes can be mounted; they are used for storing large volumes of data, ranging up to many petabytes (10^{15} bytes), with access times on the order of seconds to a few minutes. Applications that need such enormous data storage include imaging systems that gather data from remote-sensing satellites, and large video

libraries for television broadcasters. Some tape formats (such as the Accelis format) support faster seek times (of the order of tens of seconds), and are intended for applications that retrieve information from jukeboxes. Most other tape formats provide larger capacities, at the cost of slower access; such formats are ideal for data backup, where fast seeks are not important.

Tape drives have been unable to keep up with the enormous improvements in disk drive capacity and corresponding reduction in storage cost. While the cost of tapes is low, the cost of tape drives and tape libraries is significantly higher than the cost of a disk drive: a tape library capable of storing a few terabytes can cost tens of thousands of dollars. Backing up data to disk drives has become a cost-effective alternative to tape backup for a number of applications.

8.6 File Organization

A database is mapped into a number of different files that are maintained by the underlying operating system. These files reside permanently on disks. A **file** is organized logically as a sequence of records. These records are mapped onto disk blocks. Files are provided as a basic construct in operating systems, so we shall assume the existence of an underlying *file system*. We need to consider ways of representing logical data models in terms of files.

Each file is also logically partitioned into fixed-length storage units called **blocks**, which are the units of both storage allocation and data transfer. Most databases use block sizes of 4 to 8 kilobytes by default, but many databases allow the block size to be specified when a database instance is created. Larger block sizes can be useful in some database applications. A block may contain several records; the exact set of records that a block contains is determined by the form of physical data organization being used. We shall assume that *no record is larger than a block*. This assumption is realistic for most data-processing applications, such as our university example. There are certainly several kinds of large data items, such as images, that can be significantly larger than a block. We briefly discuss how to handle such large data items later, and storing a pointer to the data item in the record.

In addition, we shall require that *each record is entirely contained in a single block*; that is, no record is contained partly in one block, and partly in another. This restriction simplifies and speeds up access to data items.

In a relational database, tuples of distinct relations are generally of different sizes. One approach to mapping the database to files is to use several files, and to store records of only one fixed length in any given file. An alternative is to structure our files so that we can accommodate multiple lengths for records; however, files of fixed-length records are easier to implement than are files of variable-length records. Many of the techniques used for the former can be applied to the variable-length case. Thus, we begin by considering a file of fixed-length records, and consider storage of variable-length records later.

8.6.1 Fixed-Length Records

As an example, let us consider a file of *instructor* records for our university database. Each record of this file is defined (in pseudocode) as:

```
type instructor = record

    ID varchar (5);

    name varchar(20);

    dept name varchar (20);

    salary numeric (8,2);

end
```

Assume that each character occupies 1 byte and that numeric (8,2) occupies 8 bytes. Suppose that instead of allocating a variable amount of bytes for the attributes *ID*, *name*, and *dept name*, we allocate the maximum number of bytes that each attribute can hold. Then, the *instructor* record is 53 bytes long. A simple approach is to use the first 53 bytes for the first record, the next 53 bytes for the second record, and so on (Figure 8.5). However, there are two problems with this simple approach:

1. Unless the block size happens to be a multiple of 53 (which is unlikely), some records will cross block boundaries. That is, part of the record will be stored in one block and part in another. It would thus require two block accesses to read or write such a record.
2. It is difficult to delete a record from this structure. The space occupied by the record to be deleted must be filled with some other record of the file, or we must have a way of marking deleted records so that they can be ignored.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 8.4 File containing instructor records.

To avoid the first problem, we allocate only as many records to a block as would fit entirely in the block (this number can be computed easily by dividing the block size by the record size, and discarding the fractional part). Any remaining bytes of each block are left unused.

When a record is deleted, we could move the record that came after it into the space formerly occupied by the deleted record, and so on, until every record following the deleted record has been moved ahead (Figure 8.6). Such an approach requires moving a large number of records. It might be easier simply to move the final record of the file into the space occupied by the deleted record (Figure 8.7).

It is undesirable to move records to occupy the space freed by a deleted record, since doing so requires additional block accesses. Since insertions tend to be more frequent than deletions, it is acceptable to leave open the space occupied by the deleted record, and to wait for a subsequent insertion before reusing the space.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 8.6 File of Figure 8.5, with record 3 deleted and all records moved.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Figure 8.7 File of Figure 8.5, with record 3 deleted and final record moved.

A simple marker on a deleted record is not sufficient, since it is hard to find this available space when an insertion is being done. Thus, we need to introduce an additional structure.

At the beginning of the file, we allocate a certain number of bytes as a **file header**. The header will contain a variety of information about the file. For now, all we need to store there is the address of the first record whose contents are deleted. We use this first record to store the address of the second available record, and so on. Intuitively, we can think of these stored addresses as *pointers*, since they point to the location of a record. The deleted records thus form a linked list, which is often referred to as a **free list**. Figure 8.8 shows the file of Figure 8.5, with the free list, after records 1, 4, and 6 have been deleted. On insertion of a new record, we use the record pointed to by the header. We change the header pointer to point to the next available record. If no space is available, we add the new record to the end of the file. Insertion and deletion for files of fixed-length records are simple to implement, because the space made available by a deleted record is exactly the space needed to insert a record. If we allow records of variable length in a file, this match no longer holds. An inserted record may not fit in the space left free by a deleted record, or it may fill only part of that space.

8.6.2 Variable-Length Records

Variable-length records arise in database systems in several ways:

- Storage of multiple record types in a file.
- Record types that allow variable lengths for one or more fields.
- Record types that allow repeating fields, such as arrays or multisets.

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

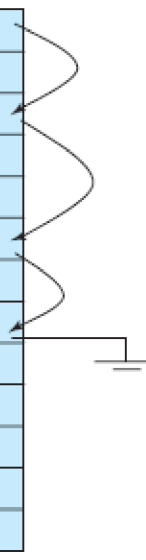


Figure 8.8 File of Figure 8.5, with free list after deletion of records 1, 4, and 6.

Different techniques for implementing variable-length records exist. Two different problems must be solved by any such technique:

- How to represent a single record in such a way that individual attributes can be extracted easily.
- How to store variable-length records within a block, such that records in a block can be extracted easily.

The representation of a record with variable-length attributes typically has two parts: an initial part with fixed length attributes, followed by data for variable length attributes. Fixed-length attributes, such as numeric values, dates, or fixedlength character strings are allocated as many bytes as required to store their value. Variable-length attributes, such as varchar types, are represented in the initial part of the record by a pair (*offset*, *length*), where *offset* denotes where the data for that attribute begins within the record, and *length* is the length in bytes of the variable-sized attribute. The values for these attributes are stored consecutively, after the initial fixed-length part of the record. Thus, the initial part of the record stores a fixed size of information about each attribute, whether it is fixed-length or variable-length.

An example of such a record representation is shown in Figure 8.9. The figure shows an *instructor* record, whose first three attributes *ID*, *name*, and *dept name* are variable-length strings, and whose fourth attribute *salary* is a fixed-sized number. We assume that

the offset and length values are stored in two bytes each, for a total of 4 bytes per attribute. The *salary* attribute is assumed to be stored in 8 bytes, and each string takes as many bytes as it has characters.

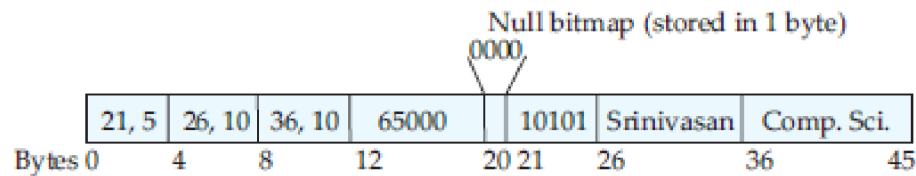


Figure 8.9 Representation of variable-length record.

The figure also illustrates the use of a **null bitmap**, which indicates which attributes of the record have a null value. In this particular record, if the salary were null, the fourth bit of the bitmap would be set to 1, and the *salary* value stored in bytes 12 through 19 would be ignored. Since the record has four attributes, the null bitmap for this record fits in 1 byte, although more bytes may be required with more attributes. In some representations, the null bitmap is stored at the beginning of the record, and for attributes that are null, no data (value, or offset/length) are stored at all. Such a representation would save some storage space, at the cost of extra work to extract attributes of the record. This representation is particularly useful for certain applications where records have a large number of fields, most of which are null.

We next address the problem of storing variable-length records in a block. The **slotted-page structure** is commonly used for organizing records within a block, and is shown in Figure 8.10. There is a header at the beginning of each block, containing the following information:

1. The number of record entries in the header.
2. The end of free space in the block.
3. An array whose entries contain the location and size of each record.

The actual records are allocated *contiguously* in the block, starting from the end of the block. The free space in the block is contiguous, between the final entry in the header array, and the first record. If a record is inserted, space is allocated for it at the end of free space, and an entry containing its size and location is added to the header. If a record is deleted, the space that it occupies is freed, and its entry is set to *deleted* (its size is set to "1, for example). Further, the records in the block before the deleted record are moved, so

that the free space created by the deletion gets occupied, and all free space is again between the final entry in the header array and the first record. The end-of-free-space pointer in the header is appropriately updated as well. Records can be grown or shrunk by similar techniques, as long as there is space in the block. The cost of moving the records is not too high, since the size of a block is limited: typical values are around 4 to 8 kilobytes. The slotted-page structure requires that there be no pointers that point directly to records. Instead, pointers must point to the entry in the header that contains the 3 Here, actual location of the record.

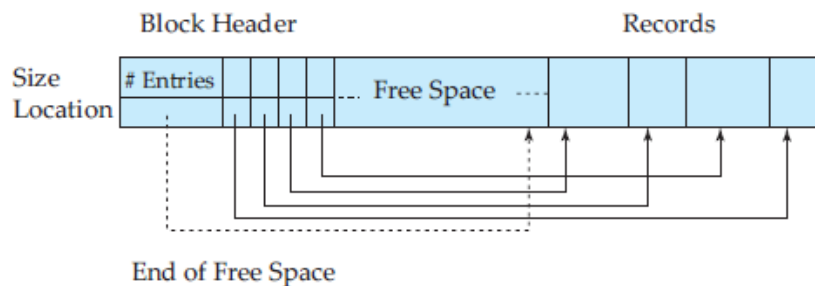


Figure 8.10 Slotted-page structure.

This level of indirection allows records to be removed to prevent fragmentation of space inside a block, while supporting indirect pointers to the record.

Databases often store data that can be much larger than a disk block. For instance, an image or an audio recording may be multiple megabytes in size, while a video object may be multiple gigabytes in size. Recall that SQL supports the types **blob** and **clob**, which store binary and character large objects. Most relational databases restrict the size of a record to be no larger than the size of a block, to simplify buffer management and free-space management. Large objects are often stored in a special file (or collection of files) instead of being stored with the other (short) attributes of records in which they occur. A (logical) pointer to the object is then stored in the record containing the large object. Large objects are often represented using B+tree file organizations. B+tree file organizations permit us to read an entire object, or specified byte ranges in the object, as well as to insert and delete parts of the object.

8.7 Summary

Several types of data storage exist in most computer systems. They are classified by the speed with which they can access data, by their cost per unit of data to buy the memory, and by their reliability. Among the media available are cache, main memory, flash memory, magnetic disks, optical disks, and magnetic tapes.

- Two factors determine the reliability of storage media: whether a power failure or system crash causes data to be lost, and what the likelihood is of physical failure of the storage device.
- We can reduce the likelihood of physical failure by retaining multiple copies of data. For disks, we can use mirroring. Or we can use more sophisticated methods based on redundant arrays of independent disks (RAID). By striping data across disks, these methods offer high throughput rates on large accesses; by introducing redundancy across disks, they improve reliability greatly. Several different RAID organizations are possible, each with different cost, performance, and reliability characteristics. RAID level 1 (mirroring) and RAID level 5 are the most commonly used.
- We can organize a file logically as a sequence of records mapped onto disk blocks. One approach to mapping the database to files is to use several files, and to store records of only one fixed length in any given file. An alternative is to structure files so that they can accommodate multiple lengths for records. The slotted-page method is widely used to handle varying length records within a disk block.
- Since data are transferred between disk storage and main memory in units of a block, it is worthwhile to assign file records to blocks in such a way that a single block contains related records. If we can access several of the records we want with only one block access, we save disk accesses. Since disk accesses are usually the bottleneck in the performance of a database system, careful assignment of records to blocks can pay significant performance dividends.

8.8 Lesson End Activities

1. A power failure that occurs while a disk block is being written could result in the block being only partially written. Assume that partially written blocks can be detected. An atomic block write is one where either the disk block is fully written or nothing is written (i.e., there are no partial writes). Suggest schemes for getting the effect of atomic block writes with the following RAID schemes. Your schemes should involve work on recovery from failure.
 - a. RAID level 1 (mirroring)
 - b. RAID level 5 (block interleaved, distributed parity)

8.9 Keywords

- Physical storage media
 - * Cache
 - * Main memory
 - * Flash memory
 - * Magnetic disk
 - * Optical storage
- Magnetic disk
 - * Platter
 - * Hard disks
 - * Floppy disks
 - * Tracks
 - * Sectors
 - * Read–write head
 - * Disk arm
 - * Cylinder
 - * Disk controller
 - * Checksums
 - * Remapping of bad sectors
- Performance measures of disks
 - * Access time
 - * Seek time
 - * Rotational latency
 - * Data-transfer rate
 - * Mean time to failure (MTTF)
- Disk block
- Optimization of disk-block access
 - * Disk-arm scheduling
 - * Elevator algorithm
 - * File organization

- * Defragmenting
- * Nonvolatile write buffers
- * Nonvolatile random-access memory (NVRAM)
- * Log disk
- Redundant arrays of independent disks (RAID)
 - * Mirroring
 - * Data striping
 - * Bit-level striping
 - * Block-level striping
- RAID levels
 - * Level 0 (block striping, no redundancy)
 - * Level 1 (block striping, mirroring)
 - * Level 3 (bit striping, parity)
 - * Level 5 (block striping, distributed parity)
 - * Level 6 (block striping, P + Q redundancy)
- Rebuild performance
- Software RAID
- Hardware RAID
- Hot swapping
- Tertiary storage
 - * Optical disks
 - * Magnetic tapes
 - * Jukeboxes
- File
- File organization
 - * File header
 - * Free list
- Variable-length records
 - * Slotted-page structure

- Large objects
- Heap file organization
- Sequential file organization
- Hashing file organization
- Multitable clustering file organization
- Search key
- Data dictionary
- System catalog
- Buffer
 - * Buffer manager
 - * Pinned blocks
 - * Forced output of blocks
- Buffer-replacement policies
 - * Least recently used (LRU)
 - * Toss-immediate
 - * Most recently used (MRU)

8.10 Questions for Discussion

1. Explain memory management.
2. What is RAID? Explain its importance.
3. Discuss the buffer manager and buffer replacement policy.
4. Explain file organization in DBMS.

8.11 Suggested Reading

“Database System Concepts” -5th Edition by Silberchatz. Korth. Sudarshan., TMH.

The storage structure of specific database systems, such as IBM DB2, Oracle,

Microsoft SQL Server, and PostgreSQL are documented in their respective system manuals.

Lesson - 9

INDEXING AND HASHING

CONTENTS

- 9.0 Aim and Objectives**
- 9.1 Introduction**
 - 9.1.1 Basic Concepts**
- 9.2 B+-Tree Index Files**
 - 9.2.1 Structure of a B⁺ Tree**
 - 9.2.2 Queries on B⁺ Trees**
 - 9.2.3 Updates on B⁺ Trees**
 - 9.2.3.1 Insertion**
 - 9.2.3.2 Deletion**
 - 9.2.4 Non Unique Search key**
 - 9.2.5 Complexity of B⁺ Trees Updates**
- 9.3 B-Tree Index Files**
- 9.4 Multiple-Key Access**
 - 9.4.1 Using multiple single key indices**
 - 9.4.2 Indices on Multiple Keys**
 - 9.4.3 Covering Indices**
- 9.5 Static Hashing**
 - 9.5.1 Hash Functions**
 - 9.5.2 Handling of Bucket Overflows**
 - 9.5.3 Hash Indices**
- 9.6 Dynamic Hashing**
 - 9.6.1 Data Structure**

9.6.2 Queries and Updates

9.6.3 Static Hashing versus Dynamic Hashing

9.7 Summary

9.8 Lesson End Activities

9.9 Keywords

9.10 Questions for Discussion

9.11 Suggested Reading

9.0 AIM AND OBJECTIVES

After studying this lesson, you should be able to:

- Understanding of an index for a file in a database system works.
- We can search for the topic in the index.
- Database-system indices play the same role as book indices in libraries.
- advantageous to use multiple indices if they exist, or to use an index built on a multiattribute search key.
- Study of dynamic and static hasing.

9.1 Introduction

Many queries reference only a small proportion of the records in a file. For example, a query like “Find all instructors in the Physics department” or “Find the total number of credits earned by the student with *ID* 22201” references only a fraction of the student records. It is inefficient for the system to read every tuple in the *instructor* relation to check if the *dept name* value is “Physics”. Likewise, it is inefficient to read the entire *student* relation just to find the one tuple for the *ID* “32556,”. Ideally, the system should be able to locate these records directly. To allow these forms of access, we design additional structures that we associate with files.

9.1.1 Basic Concepts

An index for a file in a database system works in much the same way as the index in this textbook. If we want to learn about a particular topic (specified by a word or a phrase) in this textbook, we can search for the topic in the index at the back of the book, find the

pages where it occurs, and then read the pages to find the information for which we are looking. The words in the index are in sorted order, making it easy to find the word we want. Moreover, the index is much smaller than the book, further reducing the effort needed.

Database-system indices play the same role as book indices in libraries. For example, to retrieve a *student* record given an *ID*, the database system would look up an index to find on which disk block the corresponding record resides, and then fetch the disk block, to get the appropriate *student* record. Keeping a sorted list of students' *ID* would not work well on very large databases with thousands of students, since the index would itself be very big; further, even though keeping the index sorted reduces the search time, finding a student can still be rather time-consuming. Instead, more sophisticated indexing techniques may be used. We shall discuss several of these techniques. There are two basic kinds of indices:

- **Ordered indices.** Based on a sorted ordering of the values
- **Hash indices.** Based on a uniform distribution of values across a range of buckets. The bucket to which a value is assigned is determined by a function, called a *hash function*. We shall consider several techniques for both ordered indexing and hashing. No one technique is the best. Rather, each technique is best suited to particular database applications. Each technique must be evaluated on the basis of these factors:
 - **Access types:** The types of access that are supported efficiently. Access types can include finding records with a specified attribute value and finding records whose attribute values fall in a specified range.
 - **Access time:** The time it takes to find a particular data item, or set of items, using the technique in question.
 - **Insertion time:** The time it takes to insert a newdata item. This value includes the time it takes to find the correct place to insert the new data item, as well as the time it takes to update the index structure.
 - **Deletion time:** The time it takes to delete a data item. This value includes the time it takes to find the item to be deleted, as well as the time it takes to update the index structure.
 - **Space overhead:** The additional space occupied by an index structure. Provided that the amount of additional space is moderate, it is usually worthwhile to sacrifice the space to achieve improved performance. We often want to have more than one

index for a file. For example, we may wish to search for a book by author, by subject, or by title. An attribute or set of attributes used to look up records in a file is called a **search key**. Note that this definition of *key* differs from that used in *primary key*, *candidate key*, and *superkey*. This duplicate meaning for *key* is (unfortunately) well established in practice. Using our notion of a search key, we see that if there are several indices on a file, there are several search keys.

9.2 B⁺-Tree Index Files

The main disadvantage of the index-sequential file organization is that performance degrades as the file grows, both for index lookups and for sequential scans through the data. Although this degradation can be remedied by reorganization of the file, frequent reorganizations are undesirable.

The **B⁺-tree** index structure is the most widely used of several index structures that maintain their efficiency despite insertion and deletion of data. A B⁺-tree index takes the form of a **balanced tree** in which every path from the root of the tree to a leaf of the tree is of the same length. Each nonleaf node in the tree has between $\lceil n/2 \rceil$ and n children, where n is fixed for a particular tree. We shall see that the B⁺-tree structure imposes performance overhead on insertion and deletion, and adds space overhead. The overhead is acceptable even for frequently modified files, since the cost of file reorganization is avoided. Furthermore, since nodes may be as much as half empty (if they have the minimum number of children), there is some wasted space. This space overhead, too, is acceptable given the performance benefits of the B⁺-tree structure.

9.2.1 Structure of a B⁺-Tree

A B⁺-tree index is a multilevel index, but it has a structure that differs from that of the multilevel index-sequential file. Figure 9.1 shows a typical node of a B⁺-tree. It contains up to $n - 1$ search-key values $K_1, K_2, \dots, K_{n-1}$, and n pointers $P_1, P_2, \dots, P_n$. The search-key values within a node are kept in sorted order; thus, if $i < j$, then $K_i < K_j$.

We consider first the structure of the **leaf nodes**. For $i = 1, 2, \dots, n-1$, pointer P_i points to a file record with search-key value K_i . Pointer P_n has a special purpose that we shall discuss shortly. Figure 9.2 shows one leaf node of a B⁺-tree for the *instructor* file, in which we have chosen n to be 4, and the search key is *name*. Now that we have seen the structure of a leaf node, let us consider how search-key values are assigned to particular nodes. Each leaf can hold up to $n - 1$ values. We allow leaf nodes to contain as few as $\lceil (n - 1)/2 \rceil$ values. With $n = 4$ in our example B⁺-tree, each leaf must contain at least 2 values, and at most 3 values.

The ranges of values in each leaf do not overlap, except if there are duplicate search-key values, in which case a value may be present in more than one leaf. Specifically, if L_i and L_j are leaf nodes and $i < j$, then every search-key value in L_i is less than or equal to every search-key value in L_j . If the B⁺-tree index is used as a dense index (as is usually the case) every search-key value must appear in some leaf node.

Now we can explain the use of the pointer P_n . Since there is a linear order on the leaves based on the search-key values that they contain, we use P_n to chain $P_1 K_1 P_2 \dots P_{n-1} K_{n-1} P_n$ together the leaf nodes in search-key order. This ordering allows for efficient sequential processing of the file.



Figure 9.1 Typical node of a B⁺-tree.

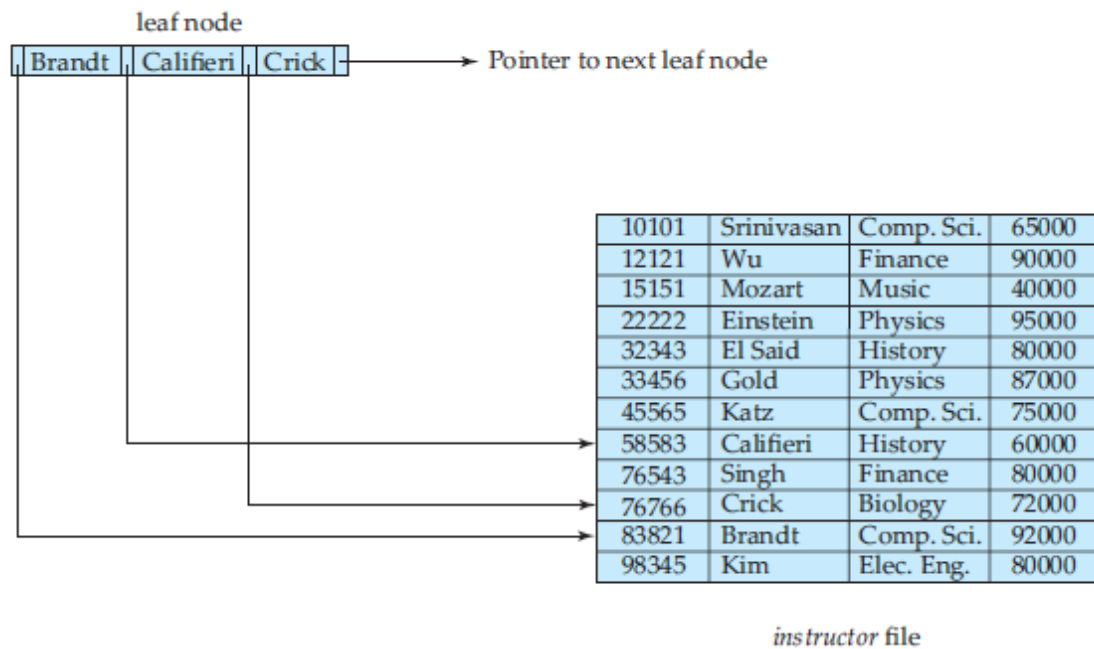


Figure 9.2 A leaf node for instructor B⁺-tree index ($n = 4$).

The **nonleaf nodes** of the B⁺-tree form a multilevel (sparse) index on the leaf nodes. The structure of nonleaf nodes is the same as that for leaf nodes, except that all pointers are pointers to tree nodes. A nonleaf node may hold up to n pointers, and *must* hold at least $\lceil n/2 \rceil$ pointers. The number of pointers in a node is called the *fanout* of the node. Nonleaf nodes are also referred to as **internal nodes**.

Let us consider a node containing m pointers ($m \leq n$). For $i = 2, 3, \dots, m-1$, pointer P_i points to the subtree that contains search-key values less than K_i and greater than or equal to K_{i-1} . Pointer P_m points to the part of the subtree that contains those key values greater than or equal to K_{m-1} , and pointer P_1 points to the part of the subtree that contains those search-key values less than K_1 . Unlike other nonleaf nodes, the root node can hold fewer than $\lceil n/2 \rceil$ pointers; however, it must hold at least two pointers, unless the tree consists of only one node. It is always possible to construct a B⁺-tree, for any n , that satisfies the preceding requirements. Figure 9.3 shows a complete B⁺-tree for the instructor file (with $n = 4$). We have shown instructor names abbreviated to 3 characters in order to depict the tree clearly; in reality, the tree nodes would contain the full names. We have also omitted null pointers for simplicity; any pointer field in the figure that does not have an arrow is understood to have a null value. Figure 9.4 shows another B⁺-tree for the instructor file, this time with $n = 6$.

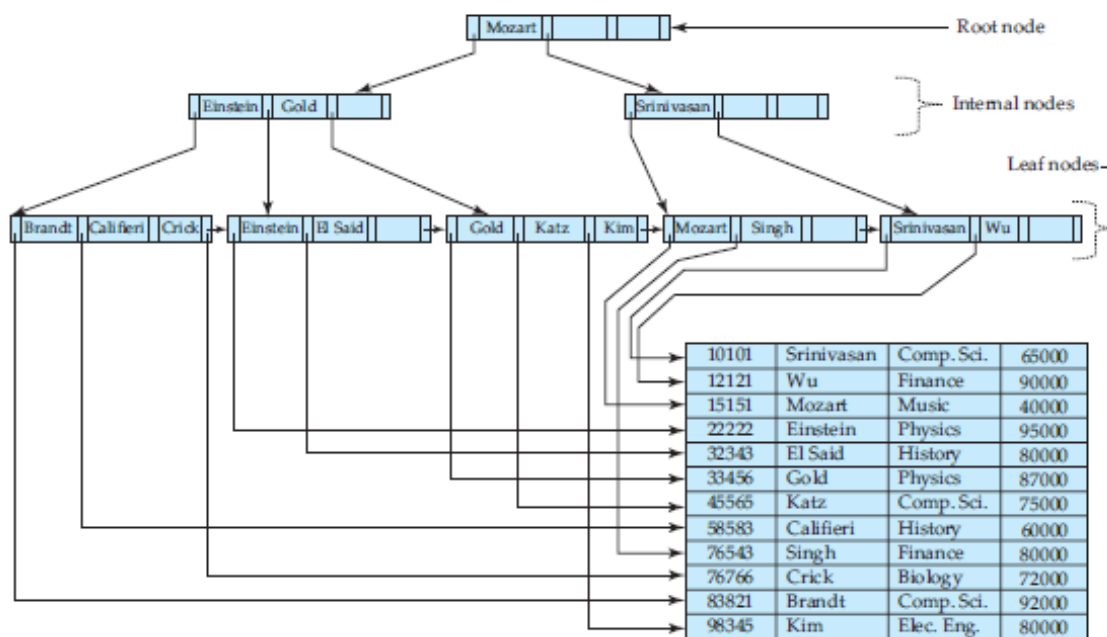


Figure 9.3 B⁺-tree for instructor file ($n = 4$).

Observe that the height of this tree is less than that of the previous tree, which had $n = 4$.

These examples of B⁺-trees are all balanced. That is, the length of every path from the root to a leaf node is the same. This property is a requirement for a B⁺-tree. Indeed, the “B” in B⁺-tree stands for “balanced.” It is the balance property of B⁺-trees that ensures good performance for lookup, insertion, and deletion.

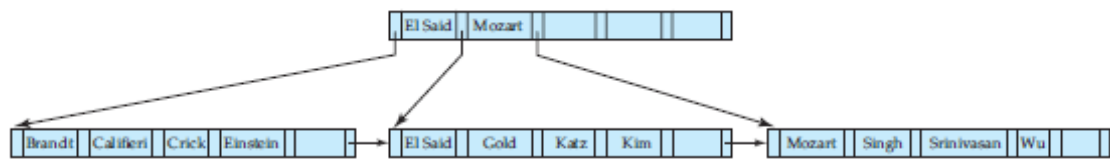


Figure 9.4 B+-tree for instructor file with $n = 6$.

9.2.2 Queries on B+-Trees

Let us consider how we process queries on a B+-tree. Suppose that we wish to find records with a search-key value of V . Figure 9.5 presents pseudocode for a function `find()` to carry out this task. Intuitively, the function starts at the root of the tree, and traverses the tree down until it reaches a leaf node that would contain the specified value if it exists in the tree. Specifically, starting with the root as the current node, the function

repeats the following steps until a leaf node is reached. First, the current node is examined, looking for the smallest i such that search-key value K_i is greater than or equal to V .

```

function find(value V)
/* Returns leaf node C and index i such that C.Pi points to first record
 * with search key value V */
  Set C = root node
  while (C is not a leaf node) begin
    Let i = smallest number such that  $V \leq C.K_i$ 
    if there is no such number i then begin
      Let  $P_m$  = last non-null pointer in the node
      Set  $C = C.P_m$ 
    end
    else if ( $V = C.K_i$ )
      then Set  $C = C.P_{i+1}$ 
    else  $C = C.P_i$  /*  $V < C.K_i$  */
  end
  /* C is a leaf node */
  Let i be the least value such that  $K_i = V$ 
  if there is such a value i
    then return (C, i)
    else return null ; /* No record with key value V exists */

procedure printAll(value V)
/* prints all records with search key value V */
  Set done = false;
  Set (L, i) = find(V);
  
```

```

if ((L, i) is null) return
repeat
  repeat
    Print record pointed to by  $L.P_i$ 
    Set  $i = i + 1$ 
  until ( $i > \text{number of keys in } L$  or  $L.K_i > V$ )
  if ( $i > \text{number of keys in } L$ )
    then  $L = L.P_n$ 
    else Set done = true;
until (done or L is null)

```

Figure 9.5 Querying a B+-tree.

Suppose such a value is found; then, if K_i is equal to V , the current node is set to the node pointed to by P_{i+1} , otherwise $K_i > V$, and the current node is set to the node pointed to by P_i . If no such value K_i is found, then clearly $V > K_{m-1}$, where P_m is the last nonnull pointer in the node. In this case the current node is set to that pointed to by P_m . The above procedure is repeated, traversing down the tree until a leaf node is reached.

At the leaf node, if there is a search-key value equal to V , let K_i be the first such value; pointer P_i directs us to a record with search-key value K_i . The function then returns the leaf node L and the index i . If no search-key with value V is found in the leaf node, no record with key value V exists in the relation, and function find returns null, to indicate failure. If there is at most one record with a search key value V (for example, if the index is on a primary key) the procedure that calls the find function simply uses the pointer $L.P_i$ to retrieve the record and is done. However, in case there may be more than one matching record, the remaining records also need to be fetched. Procedure print.

All shown in Figure 9.5 shows how to fetch all records with a specified search key V . The procedure first steps through the remaining keys in the node L , to find other records with search-key value V . If node L contains at least one search-key value greater than V , then there are no more records matching V . Otherwise, the next leaf, pointed to by P_n may contain further entries for V . The node pointed to by P_n must then be searched to find further records with search-key value V . If the highest search-key value in the node pointed to by P_n is also V , further leaves may have to be traversed, in order to find all matching records. The repeat loop in printAll carries out the task of traversing leaf nodes until all matching records have been found.

A real implementation would provide a version of find supporting an iterator interface similar to that provided by the JDBC ResultSet. Such an iterator interface would provide a

method `next()`, which can be called repeatedly to fetch successive records with the specified search-key. The `next()` method would step through the entries at the leaf level, in a manner similar to `printAll`, but each call takes only one step, and records where it left off, so that successive calls next step through successive records. We omit details for simplicity, and leave the pseudocode for the iterator interface as an exercise for the interested reader.

B+-trees can also be used to find all records with search key values in a specified range (L, U) . For example, with a B+-tree on attribute *salary* of *instructor*, we can find all *instructor* records with salary in a specified range such as (50000, 100000) (in other words, all salaries between 50000 and 100000). Such queries are called **range queries**. To execute such queries, we can create a procedure `printRange( $L, U$ )`, whose body is the same as `printAll` except for these differences: `printRange` calls `find( $L$ )`, instead of `find( $V$ )`, and then steps through records as in procedure `printAll`, but with the stopping condition being that $L.K_i > U$, instead of $L.K_i > V$.

In processing a query, we traverse a path in the tree from the root to some leaf node. If there are N records in the file, the path is no longer than $\lceil \log_{n/2}(N) \rceil$. In practice, only a few nodes need to be accessed. Typically, a node is made to be the same size as a disk block, which is typically 4 kilobytes. With a search-key size of 12 bytes, and a disk-pointer size of 8 bytes, n is around 200. Even with a more conservative estimate of 32 bytes for the search-key size, n is around 100. With $n = 100$, if we have 1 million search-key values in the file, a lookup requires only $\lceil \log_{50}(1,000,000) \rceil = 4$ nodes to be accessed. Thus, at most four blocks need to be read from disk for the lookup. The root node of the tree is usually heavily accessed and is likely to be in the buffer, so typically only three or fewer blocks need to be read from disk.

An important difference between B+-tree structures and in-memory tree structures, such as binary trees, is the size of a node, and as a result, the height of the tree. In a binary tree, each node is small, and has at most two pointers. In a B+-tree, each node is large—typically a disk block—and a node can have a large number of pointers. Thus, B+-trees tend to be fat and short, unlike thin and tall binary trees. In a balanced binary tree, the path for a lookup can be of length $\lceil \log_2(N) \rceil$, where N is the number of records in the file being indexed. With $N = 1,000,000$ as in the previous example, a balanced binary tree requires around 20 node accesses. If each node were on a different disk block, 20 block reads would be required to process a lookup, in contrast to the four block reads for the B+-tree. The difference is significant, since each block read could require a disk arm seek, and a

block read together with the disk arm seek takes about 10 milliseconds on a typical disk.

9.2.3 Updates on B+-Trees

When a record is inserted into, or deleted from a relation, indices on the relation must be updated correspondingly. Recall that updates to a record can be modeled as a deletion of the old record followed by insertion of the updated record. Hence we only consider the case of insertion and deletion. Insertion and deletion are more complicated than lookup, since it may be necessary to **split** a node that becomes too large as the result of an insertion, or to **coalesce** nodes (that is, combine nodes) if a node becomes too small (fewer than $\lceil n/2 \rceil$ pointers). Furthermore, when a node is split or a pair of nodes is combined, we must ensure that balance is preserved. To introduce the idea behind insertion and deletion in a B+-tree, we shall assume temporarily that nodes never become too large or too small. Under this assumption, insertion and deletion are performed as defined next.

- **Insertion.** Using the same technique as for lookup from the `find()` function (Figure 9.7), we first find the leaf node in which the search-key value would appear. We then insert an entry (that is, a search-key value and record pointer pair) in the leaf node, positioning it such that the search keys are still in order.
- **Deletion.** Using the same technique as for lookup, we find the leaf node containing the entry to be deleted, by performing a lookup on the search-key value of the deleted record; if there are multiple entries with the same search-key value, we search across all entries with the same search-key value until we find the entry that points to the record being deleted. We then remove the entry from the leaf node. All entries in the leaf node that are to the right of the deleted entry are shifted left by one position, so that there are no gaps in the entries after the entry is deleted. We now consider the general case of insertion and deletion, dealing with node splitting and node coalescing.

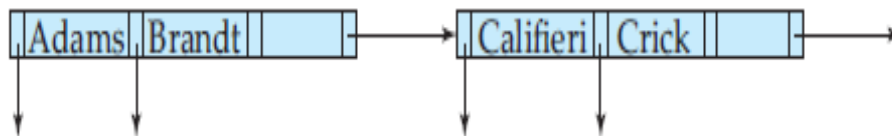


Figure 9.6 Split of leaf node on insertion of “Adams”

9.2.3.1 Insertion

We now consider an example of insertion in which a node must be split. Assume that a record is inserted on the *instructor* relation, with the *name* value being Adams. We then need to insert an entry for “Adams” into the B+-tree of Figure 9.6. Using the algorithm for lookup, we find that “Adams” should appear in the leaf node containing “Brandt”, “Califieri”, and “Crick.” There is no room in this leaf to insert the search-key value “Adams.” Therefore, the node is *split* into two nodes. Figure 9.7 shows the two leaf nodes that result from the split of the leaf node on inserting “Adams”. The search-key values “Adams” and “Brandt” are in one leaf, and “Califieri” and “Crick” are in the other. In general, we take the n search-key values (the $n-1$ values in the leaf node plus the value being inserted), and put the first $\lceil n/2 \rceil$ in the existing node and the remaining values in a newly created node. Having split a leaf node, we must insert the new leaf node into the B+-tree structure.

In our example, the new node has “Califieri” as its smallest search-key value. We need to insert an entry with this search-key value, and a pointer to the new node, into the parent of the leaf node that was split. The B+-tree of Figure 9.8 shows the result of the insertion. It was possible to perform this insertion with no further node split, because there was room in the parent node for the new entry. If there were no room, the parent would have had to be split, requiring an entry to be added to its parent. In the worst case, all nodes along the path to the root must be split. If the root itself is split, the entire tree becomes deeper.

Splitting of a nonleaf node is a little different from splitting of a leaf node. Figure 9.9 shows the result of inserting a record with search key “Lamport” into the tree shown in Figure 9.8. The leaf node in which “Lamport” is to be inserted already has entries “Gold”, “Katz”, and “Kim”, and as a result the leaf node has to be split. The new right-hand-side node resulting from the split contains the search-key values “Kim” and “Lamport”. An entry (Kim, n_1) must then be added to the parent node, where n_1 is a pointer to the new node. However, there is no space in the parent node to add a new entry, and the parent node has to be split.

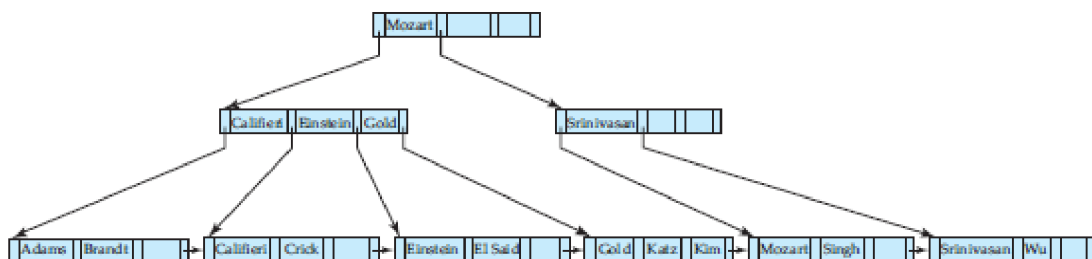


Figure 9.7 Insertion of “Adams” into the B+-tree of Figure 11.9.

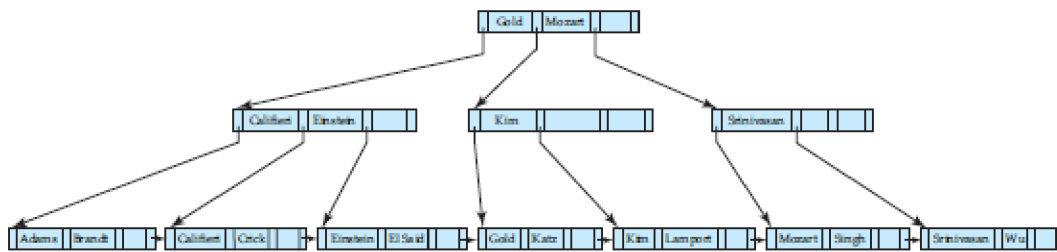


Figure 9.8 Insertion of “Lamport” into the B+-tree of Figure 11.13.

To do so, the parent node is conceptually expanded temporarily, the entry added, and the overfull node is then immediately split.

When an overfull nonleaf node is split, the child pointers are divided among the original and the newly created nodes; in our example, the original node is left with the first three pointers, and the newly created node to the right gets the remaining two pointers. The search key values are, however, handled a little differently. The search key values that lie between the pointers moved to the right node (in our example, the value “Kim”) are moved along with the pointers, while those that lie between the pointers that stay on the left (in our example, “Califiori” and “Einstein”) remain undisturbed.

However, the search key value that lies between the pointers that stay on the left, and the pointers that move to the right node is treated differently. In our example, the search key value “Gold” lies between the three pointers that went to the left node, and the two pointers that went to the right node. The value “Gold” is not added to either of the split nodes. Instead, an entry (Gold, n_2) is added to the parent node, where n_2 is a pointer to the newly created node that resulted from the split. In this case, the parent node is the root, and it has enough space for the new entry.

The general technique for insertion into a B+-tree is to determine the leaf node l into which insertion must occur. If a split results, insert the new node into the parent of node l . If this insertion causes a split, proceed recursively up the tree until either an insertion does not cause a split or a new root is created. Figure 9.10 outlines the insertion algorithm in pseudocode. The procedure insert inserts a key-value pointer pair into the index, using two subsidiary procedures insert in leaf and insert in parent. In the pseudocode, L , N , P and T denote pointers to nodes, with L being used to denote a leaf node. $L.K_i$ and $L.P_i$ denote the i th value and the i th pointer in node L , respectively; $T.K_i$ and $T.P_i$ are used similarly. The pseudocode also makes use of the function $\text{parent}(N)$ to find the parent of a node N . We can compute a list of nodes in the path from

the root to the leaf while initially finding the leaf node, and can use it later to find the parent of any node in the path efficiently.

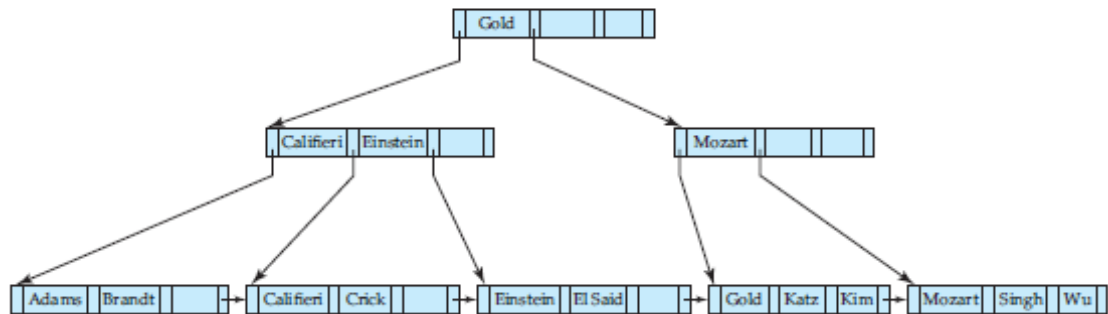


Figure 9.9 Insertion of entry in a B+-tree.

```

procedure insert(value  $K$ , pointer  $P$ )
  if (tree is empty) create an empty leaf node  $L$ , which is also the root
  else Find the leaf node  $L$  that should contain key value  $K$ 
  if ( $L$  has less than  $n - 1$  key values)
    then insert_in_leaf ( $L$ ,  $K$ ,  $P$ )
  else begin /*  $L$  has  $n - 1$  key values already, split it */
    Create node  $L'$ 
    Copy  $L.P_1 \dots L.K_{n-1}$  to a block of memory  $T$  that can
      hold  $n$  (pointer, key-value) pairs
    insert_in_leaf ( $T$ ,  $K$ ,  $P$ )
    Set  $L'.P_n = L.P_n$ ; Set  $L.P_n = L'$ 
    Erase  $L.P_1$  through  $L.K_{n-1}$  from  $L$ 
    Copy  $T.P_1$  through  $T.K_{[n/2]}$  from  $T$  into  $L$  starting at  $L.P_1$ 
    Copy  $T.P_{[n/2]+1}$  through  $T.K_n$  from  $T$  into  $L'$  starting at  $L'.P_1$ 
    Let  $K'$  be the smallest key-value in  $L'$ 
    insert_in_parent( $L$ ,  $K'$ ,  $L'$ )
  end

procedure insert_in_leaf (node  $L$ , value  $K$ , pointer  $P$ )
  if ( $K < L.K_1$ )
    then insert  $P$ ,  $K$  into  $L$  just before  $L.P_1$ 
  else begin
    Let  $K_i$  be the highest value in  $L$  that is less than  $K$ 
    Insert  $P$ ,  $K$  into  $L$  just after  $T.K_i$ 
  end

procedure insert_in_parent(node  $N$ , value  $K'$ , node  $N'$ )
  if ( $N$  is the root of the tree)
    then begin
      Create a new node  $R$  containing  $N$ ,  $K'$ ,  $N'$  /*  $N$  and  $N'$  are pointers */
      Make  $R$  the root of the tree
    return
  end

```

```

Let  $P = \text{parent}(N)$ 
if ( $P$  has less than  $n$  pointers)
  then insert  $(K', N')$  in  $P$  just after  $N$ 
  else begin /* Split  $P$  */
    Copy  $P$  to a block of memory  $T$  that can hold  $P$  and  $(K', N')$ 
    Insert  $(K', N')$  into  $T$  just after  $N$ 
    Erase all entries from  $P$ ; Create node  $P'$ 
    Copy  $T.P_1 \dots T.P_{\lceil n/2 \rceil}$  into  $P$ 
    Let  $K'' = T.K_{\lceil n/2 \rceil}$ 
    Copy  $T.P_{\lceil n/2 \rceil + 1} \dots T.P_{n+1}$  into  $P'$ 
    insert_in_parent( $P, K'', P'$ )
  end

```

Figure 9.10 Deletion of “Srinivasan” from the B+-tree of Figure 11.13.

modifies the parent of N to record the split. The procedures insert into index and insert in parent use a temporary area of memory T to store the contents of a node being split. The procedures can be modified to copy data from the node being split directly to the newly created node, reducing the time required for copying data. However, the use of the temporary space T simplifies the procedures.

9.2.3.2 Deletion

We now consider deletions that cause tree nodes to contain too few pointers. First, let us delete “Srinivasan” from the B+-tree of Figure 9.8. The resulting B+-tree appears in Figure 9.11. We now consider how the deletion is performed. We first locate the entry for “Srinivasan” by using our lookup algorithm. When we delete the entry for “Srinivasan” from its leaf node, the node is left with only one entry, “Wu”. Since, in our example, $n = 4$ and $1 < [(n - 1)/2]$, we must either merge the node with a sibling node, or redistribute the entries between the nodes, to ensure that each node is at least half-full. In our example, the underfull node with the entry for “Wu” can be merged with its left sibling node. We merge the nodes by moving the entries from both the nodes into the left sibling, and deleting the now empty right sibling. Once the node is deleted, we must also delete the entry in the parent node that pointed to the just deleted node.

In our example, the entry to be deleted is $(\text{Srinivasan}, n3)$, where $n3$ is a pointer to the leaf containing “Srinivasan”. (In this case the entry to be deleted in the nonleaf node happens to be the same value as that deleted from the leaf; that would not be the case for most deletions.) After deleting the above entry, the parent node, which had a search key value “Srinivasan” and two pointers, now has one pointer (the leftmost pointer in the node) and no search-key values. Since $1 < \lceil n/2 \rceil$ for $n = 4$, the parent node is underfull. (For larger n , a node that becomes underfull would still have some values as well as pointers.)

In this case, we look at a sibling node; in our example, the only sibling is the nonleaf node containing the search keys “Califieri”, “Einstein”, and “Gold”. If possible, we try to coalesce the node with its sibling. In this case, coalescing is not possible, since the node and its sibling together have five pointers, against a maximum of four. The solution in this case is to **redistribute** the pointers between the node and its sibling, such that each has at least $\lceil n/2 \rceil = 2$ child pointers. To do so, we move the rightmost pointer from the left sibling (the one pointing to the leaf node containing “Mozart”) to the underfull right sibling.

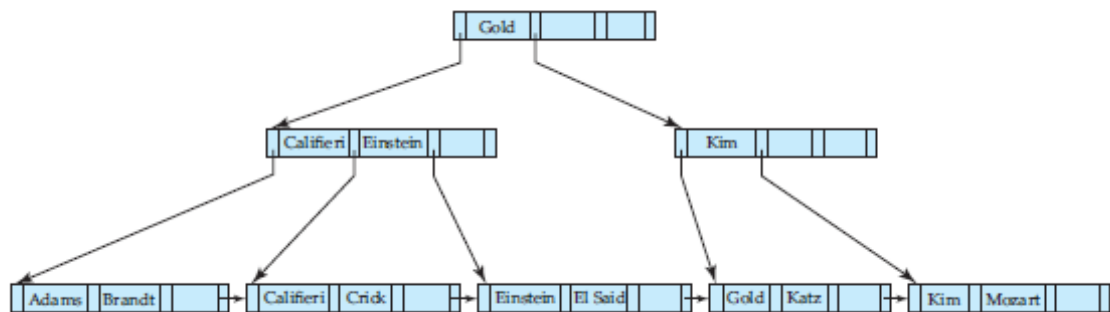


Figure 9.11 Deletion of “Singh” and “Wu” from the B+-tree of Figure 11.16.

However, the underfull right sibling would now have two pointers, namely its leftmost pointer, and the newly moved pointer, with no value separating them. In fact, the value separating them is not present in either of the nodes, but is present in the parent node, between the pointers from the parent to the node and its sibling. In our example, the value “Mozart” separates the two pointers, and is present in the right sibling after the redistribution. Redistribution of the pointers also means that the value “Mozart” in the parent no longer correctly separates search-key values in the two siblings. In fact, the value that now correctly separates search-key values in the two sibling nodes is the value “Gold”, which was in the left sibling before redistribution.

As a result, as can be seen in the B+-tree in Figure 9.11, after redistribution of pointers between siblings, the value “Gold” has moved up into the parent, while the value that was there earlier, “Mozart”, has moved down into the right sibling. We next delete the search-key values “Singh” and “Wu” from the B+-tree of Figure 9.11. The result is shown in Figure 9.12. The deletion of the first of these values does not make the leaf node underfull, but the deletion of the second value does. It is not possible to merge the underfull node with its sibling, so a redistribution of values is carried out, moving the search-key value “Kim” into the node containing “Mozart”, resulting in the tree shown in

Figure 9.12. The value separating the two siblings has been updated in the parent, from “Mozart” to “Kim”.

Now we delete “Gold” from the above tree;

This results in an underfull leaf, which can now be merged with its sibling. The resultant deletion of an entry from the parent node (the nonleaf node containing “Kim”) makes the parent underfull (it is left with just one pointer). This time around, the parent node can be merged with its sibling. This merge results in the search-key value “Gold” moving down from the parent into the merged node. As a result of this merge, an entry is deleted from its parent, which happens to be the root of the tree. And as a result of that deletion, the root is left with only one child pointer and no search-key value, violating the condition that the root have at least two children.

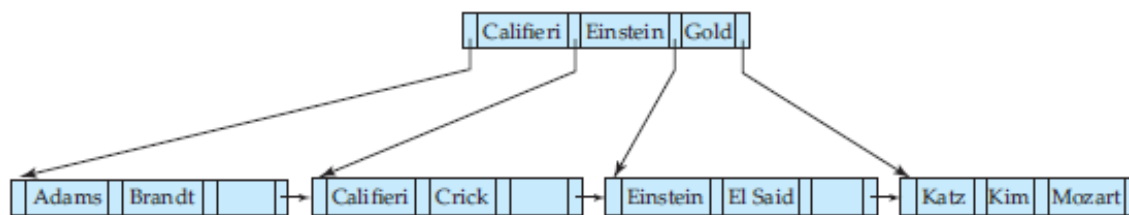


Figure 9.12 Deletion of “Gold” from the B+-tree of Figure 11.17.

As a result, the root node is deleted and its sole child becomes the root, and the depth of the B+-tree has been decreased by 1. It is worth noting that, as a result of deletion, a key value that is present in a nonleaf node of the B+-tree may not be present at any leaf of the tree. For example, the value “Gold” has been deleted from the leaf level, but is still present in a nonleaf node.

In general, to delete a value in a B+-tree, we perform a lookup on the value and delete it. If the node is too small, we delete it from its parent. This deletion results in recursive application of the deletion algorithm until the root is reached, a parent remains adequately full after deletion, or redistribution is applied.

9.2.4 Nonunique Search Keys

If a relation can have more than one record containing the same search key value (that is, two or more records can have the same values for the indexed attributes), the search key is said to be a **nonunique search key**.

One problem with nonunique search keys is in the efficiency of record deletion. Suppose a particular search-key value occurs a large number of times, and one of the records with that search key is to be deleted. The deletion may have to search through a number of entries, potentially across multiple leaf nodes, to find the entry corresponding to

the particular record being deleted. A simple solution to this problem, used by most database systems, is to make search keys unique by creating a composite search key containing the original search key and another attribute, which together are unique across all records. The extra attribute can be a record-id, which is a pointer to the record, or any other attribute whose value is unique among all records with the same search key value.

```

procedure delete(value K, pointer P)
    find the leaf node L that contains (K, P)
    delete_entry(L, K, P)

procedure delete_entry(node N, value K, pointer P)
    delete (K, P) from N
    if (N is the root and N has only one remaining child)
    then make the child of N the new root of the tree and delete N
    else if (N has too few values/pointers) then begin
        Let N' be the previous or next child of parent(N)
        Let K' be the value between pointers N and N' in parent(N)
        if (entries in N and N' can fit in a single node)
        then begin /* Coalesce nodes */
            if (N is a predecessor of N') then swap_variables(N, N')
            if (N is not a leaf)
                then append K' and all pointers and values in N to N'
                else append all (Ki, Pi) pairs in N to N'; set N'.Pn = N.Pn
            delete_entry(parent(N), K', N); delete node N
        end
    else begin /* Redistribution: borrow an entry from N' */
        if (N' is a predecessor of N) then begin
            if (N is a nonleaf node) then begin
                let m be such that N'.Pm is the last pointer in N'
                remove (N'.Km-1, N'.Pm) from N'
                insert (N'.Pm, K') as the first pointer and value in N,
                    by shifting other pointers and values right
                replace K' in parent(N) by N'.Km-1
            end
        else begin
            let m be such that (N'.Pm, N'.Km) is the last pointer/value
                pair in N'
            remove (N'.Pm, N'.Km) from N'
            insert (N'.Pm, N'.Km) as the first pointer and value in N,
                by shifting other pointers and values right
            replace K' in parent(N) by N'.Km
        end
    end
    else ... symmetric to the then case ...
    end
end

```

Figure 9.13 Deletion of entry from a B+-tree.

The extra attribute is called a **uniquifier** attribute. When a record is to be deleted, the composite search-key value is computed from the record, and then used to look up the index. Since the value is unique, the corresponding leaf 9.12 level entry can be found with a single traversal from root to leaf, with no further accesses at the leaf level. As a result, record deletion can be done efficiently.

A search with the original search-key attribute simply ignores the value of the uniquifier attribute when comparing search-key values. With nonunique search keys, our B+-tree structure stores each key value as many times as there are records containing that value. An alternative is to store each key value only once in the tree, and to keep a bucket (or list) of record pointers with a search-key value, to handle nonunique search keys. This approach is more space efficient since it stores the key value only once; however, it creates several complications when B+-trees are implemented. If the buckets are kept in the leaf node, extra code is needed to deal with variable-size buckets, and to deal with buckets that grow larger than the size of the leaf node. If the buckets are stored in separate blocks, an extra I/O operation may be required to fetch records. In addition to these problems, the bucket approach also has the problem of inefficiency for record deletion if a search-key value occurs a large number of times.

9.2.5 Complexity of B+-Tree Updates

Although insertion and deletion operations on B+-trees are complicated, they require relatively few I/O operations, which is an important benefit since I/O operations are expensive. It can be shown that the number of I/O operations needed in the worst case for an insertion is proportional to $\log_{[n/2]}(N)$, where n is the maximum number of pointers in a node, and N is the number of records in the file being indexed. The worst-case complexity of the deletion procedure is also proportional to $\log_{[n/2]}(N)$, provided there are no duplicate values for the search key. If there are duplicate values, deletion may have to search across multiple records with the same search-key value to find the correct entry to be deleted, which can be inefficient. However, making the search key unique by adding a uniquifier attribute, ensures the worst-case complexity of deletion is the same even if the original search key is nonunique.

In other words, the cost of insertion and deletion operations in terms of I/O operations is proportional to the height of the B+-tree, and is therefore low. It is the speed of operation on B+-trees that makes them a frequently used index structure in database implementations.

In practice, operations on B+-trees result in fewer I/O operations than the worst-case bounds. With fanout of 100, and assuming accesses to leaf nodes are uniformly distributed, the parent of a leaf node is 100 times more likely to get accessed than the leaf node. Conversely, with the same fanout, the total number of nonleaf nodes in a B+-tree would be just a little more than 1/100th of the number of leaf nodes. As a result, with memory sizes of several gigabytes being common today, for B+-trees that are used frequently, even if the relation is very large it is quite likely that most of the nonleaf nodes are already in the database buffer when they are accessed. Thus, typically only one or two I/O operations are required to perform a lookup. For updates, the probability of a node split occurring is correspondingly very small. Depending on the ordering of inserts, with a fanout of 100, only between 1 in 100 to 1 in 50 insertions will result in a node split, requiring more than one block to be written. As a result, on an average an insert will require just a little more than one I/O operation to write updated blocks.

Although B+-trees only guarantee that nodes will be at least half full, if entries are inserted in random order, nodes can be expected to be more than two-thirds full on average. If entries are inserted in sorted order, on the other hand, nodes will be only half full. (We leave it as an exercise to the reader to figure out why nodes would be only half full in the latter case.)

9.3 B-Tree Index Files

B-tree indices are similar to B+-tree indices. The primary distinction between the two approaches is that a B-tree eliminates the redundant storage of search-key values. In the B+-tree of Figure 9.8, the search keys “Califieri”, “Einstein”, “Gold”, “Mozart”, and “Srinivasan” appear in nonleaf nodes, in addition to appearing in the leaf nodes. Every search-key value appears in some leaf node; several are repeated in nonleaf nodes.

A B-tree allows search-key values to appear only once (if they are unique), unlike a B+-tree, where a value may appear in a nonleaf node, in addition to appearing in a leaf node. Figure 9.14 shows a B-tree that represents the same search keys as the B+-tree of Figure 9.7. Since search keys are not repeated in the B-tree, we may be able to store the index in fewer tree nodes than in the corresponding B+-tree index. However, since search keys that appear in nonleaf nodes appear nowhere else in the B-tree, we are forced to include an additional pointer field for each search key in a nonleaf node. These additional pointers point to either file records or buckets for the associated search key.

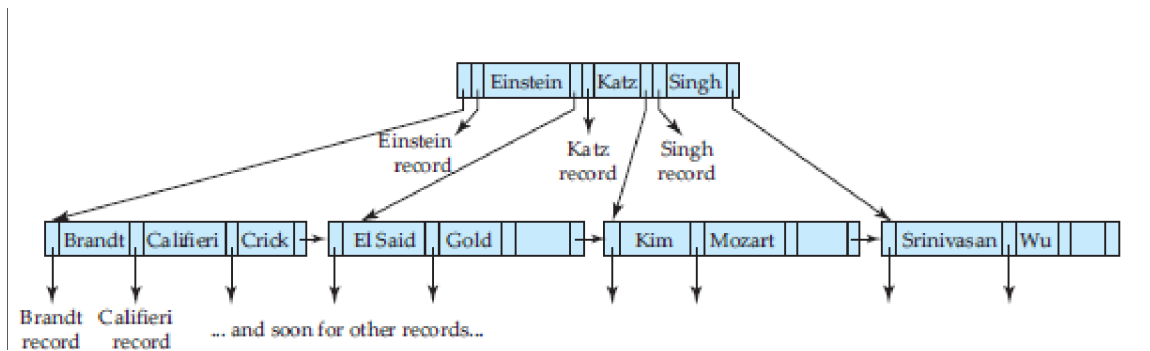


Figure 9.14 B-tree equivalent of B+-tree in Figure 11.13.

It is worth noting that many database system manuals, articles in industry literature, and industry professionals use the term B-tree to refer to the data structure that we call the B+-tree. In fact, it would be fair to say that in current usage, the term B-tree is assumed to be synonymous with B+-tree. However, in this book we use the terms B-tree and B+-tree as they were originally defined, to avoid confusion between the two data structures.

A generalized B-tree leaf node appears in Figure 9.15a; a nonleaf node appears in Figure 9.15b. Leaf nodes are the same as in B+-trees. In nonleaf nodes, the pointers P_i are the tree pointers that we used also for B+-trees, while the pointers B_i are bucket or file-record pointers. In the generalized B-tree in the figure, there are $n-1$ keys in the leaf node, but there are $m-1$ keys in the nonleaf node. This discrepancy occurs because nonleaf nodes must include pointers B_i , thus reducing the number of search keys that can be held in these nodes. Clearly, $m < n$, but the exact relationship between m and n depends on the relative size of search keys and pointers.

The number of nodes accessed in a lookup in a B-tree depends on where the search key is located. A lookup on a B+-tree requires traversal of a path from the root of the tree to some leaf node. In contrast, it is sometimes possible to find the desired value in a B-tree before reaching a leaf node. However, roughly n times as many keys are stored in the leaf level of a B-tree as in the nonleaf levels, and, since n is typically large, the benefit of finding certain values early is relatively small.

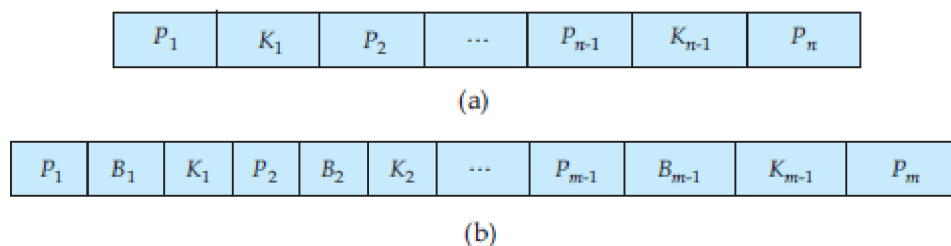


Figure 9.15 Typical nodes of a B-tree. (a) Leaf node. (b) Nonleaf node.

Moreover, the fact that fewer search keys appear in a nonleaf B-tree node, compared to B+-trees, implies that a B-tree has a smaller fanout and therefore may have depth greater than that of the corresponding B+-tree. Thus, lookup in a B-tree is faster for some search keys but slower for others, although, in general, lookup time is still proportional to the logarithm of the number of search keys. Deletion in a B-tree is more complicated. In a B+-tree, the deleted entry always appears in a leaf. In a B-tree, the deleted entry may appear in a nonleaf node. The proper value must be selected as a replacement from the subtree of the node containing the deleted entry. Specifically, if search key K_i is deleted, the smallest search key appearing in the subtree of pointer P_{i+1} must be moved to the field formerly occupied by K_i . Further actions need to be taken if the leaf node now has too few entries. In contrast, insertion in a B-tree is only slightly more complicated than is insertion in a B+-tree.

The space advantages of B-trees are marginal for large indices, and usually do not outweigh the disadvantages that we have noted. Thus, pretty much all database-system implementations use the B+-tree data structure, even if (as we discussed earlier) they refer to the data structure as a B-tree.

9.4 Multiple-Key Access

Until now, we have assumed implicitly that only one index on one attribute is used to process a query on a relation. However, for certain types of queries, it is advantageous to use multiple indices if they exist, or to use an index built on a multiattribute search key.

9.4.1 Using Multiple Single-Key Indices

Assume that the *instructor* file has two indices: one for *dept name* and one for *salary*. Consider the following query: “Find all instructors in the Finance department with salary equal to \$80,000.” We write **select ID from instructor where dept name = 'Finance' and salary= 80000;**

There are three strategies possible for processing this query:

1. Use the index on *dept name* to find all records pertaining to the Finance department. Examine each such record to see whether *salary*= 80000.
2. Use the index on *salary* to find all records pertaining to instructors with salary of \$80,000. Examine each such record to see whether the department name is “Finance”.
3. Use the index on *dept name* to find *pointers* to all records pertaining to the Finance department. Also, use the index on *salary* to find pointers to all records pertaining to

instructors with a salary of \$80,000. Take the intersection of these two sets of pointers. Those pointers that are in the intersection point to records pertaining to instructors of the Finance department and with salary of \$80,000.

The third strategy is the only one of the three that takes advantage of the existence of multiple indices. However, even this strategy may be a poor choice if all of the following hold:

- There are many records pertaining to the Finance department.
- There are many records pertaining to instructors with a salary of \$80,000.
- There are only a few records pertaining to *both* the Finance department and instructors with a salary of \$80,000.

If these conditions hold, we must scan a large number of pointers to produce a small result. An index structure called a “bitmap index” can in some cases greatly speed up the intersection operation used in the third strategy.

9.4.2 Indices on Multiple Keys

An alternative strategy for this case is to create and use an index on a composite search key (*dept name*, *salary*)—that is, the search key consisting of the department name concatenated with the instructor salary. We can use an ordered (B+-tree) index on the above composite search key to answer efficiently queries of the form

```
select ID from instructor where dept name = 'Finance' and salary= 80000;
```

Queries such as the following query, which specifies an equality condition on the first attribute of the search key (*dept name*) and a range on the second attribute of the search key (*salary*), can also be handled efficiently since they correspond to a range query on the search attribute.

```
select ID from instructor where dept name = 'Finance' and salary< 80000;
```

We can even use an ordered index on the search key (*dept name*, *salary*) to answer the following query on only one attribute efficiently:

```
select ID from instructor where dept name = 'Finance';
```

An equality condition *dept name* = “Finance” is equivalent to a range query on the range with lower end (Finance, “”) and upper end (Finance, “+”). Range queries on just the *dept name* attribute can be handled in a similar manner.

The use of an ordered-index structure on a composite search key, however, has a few shortcomings. As an illustration, consider the query

select *ID* from *instructor* where *dept name* < 'Finance' and *salary* < 80000;

We can answer this query by using an ordered index on the search key (*dept name*, *salary*): For each value of *dept name* that is less than “Finance” in alphabetic order, the system locates records with a *salary* value of 80000. However, each record is likely to be in a different disk block, because of the ordering of records in the file, leading to many I/O operations.

The difference between this query and the previous two queries is that the condition on the first attribute (*dept name*) is a comparison condition, rather than an equality condition. The condition does not correspond to a range query on the search key.

To speed the processing of general composite search-key queries (which can involve one or more comparison operations), we can use several special structures. We shall consider *bitmap indices*. There is another structure, called the *R-tree*, that can be used for this purpose. The R-tree is an extension of the B+-tree to handle indexing on multiple dimensions.

9.4.3 Covering Indices

Covering indices are indices that store the values of some attributes (other than the search-key attributes) along with the pointers to the record. Storing extra attribute values is useful with secondary indices, since they allow us to answer some queries using just the index, without even looking up the actual records.

For example, suppose that we have a nonclustering index on the *ID* attribute of the *instructor* relation. If we store the value of the *salary* attribute along with the record pointer, we can answer queries that require the salary (but not the other attribute, *dept name*) without accessing the *instructor* record. The same effect could be obtained by creating an index on the search key (*ID*, *salary*), but a covering index reduces the size of the search key, allowing a larger fanout in the nonleaf nodes, and potentially reducing the height of the index.

9.5 Static Hashing

One disadvantage of sequential file organization is that we must access an index structure to locate data, or must use binary search, and that results in more I/O operations. File organizations based on the technique of **hashing** allow us to avoid accessing an index structure. Hashing also provides a way of constructing indices. We study file organizations and indices based on hashing in the following sections.

In our description of hashing, we shall use the term **bucket** to denote a unit of storage that can store one or more records. A bucket is typically a disk block, but could be chosen to be smaller or larger than a disk block. Formally, let K denote the set of all search-key values, and let B denote the set of all bucket addresses. A **hash function** h is a function from K to B . Let h denote a hash function.

To insert a record with search key K_i , we compute $h(K_i)$, which gives the address of the bucket for that record. Assume for now that there is space in the bucket to store the record. Then, the record is stored in that bucket. To perform a lookup on a search-key value K_i , we simply compute $h(K_i)$, then search the bucket with that address. Suppose that two search keys, K_5 and K_7 , have the same hash value; that is, $h(K_5) = h(K_7)$. If we perform a lookup on K_5 , the bucket $h(K_5)$ contains records with search-key values K_5 and records with search-key values K_7 . Thus, we have to check the search-key value of every record in the bucket to verify that the record is one that we want. Deletion is equally straightforward. If the search-key value of the record to be deleted is K_i , we compute $h(K_i)$, then search the corresponding bucket for that record, and delete the record from the bucket.

Hashing can be used for two different purposes. In a **hash file organization**, we obtain the address of the disk block containing a desired record directly by computing a function on the search-key value of the record. In a **hash index organization** we organize the search keys, with their associated pointers, into a hash file structure.

9.5.1 Hash Functions

The worst possible hash function maps all search-key values to the same bucket. Such a function is undesirable because all the records have to be kept in the same bucket. A lookup has to examine every such record to find the one desired. An ideal hash function distributes the stored keys uniformly across all the buckets, so that every bucket has the same number of records.

Since we do not know at design time precisely which search-key values will be stored in the file, we want to choose a hash function that assigns search-key values to buckets in such a way that the distribution has these qualities:

- The distribution is *uniform*. That is, the hash function assigns each bucket the same number of search-key values from the set of *all* possible search-key values.
- The distribution is *random*. That is, in the average case, each bucket will have nearly the same number of values assigned to it, regardless of the actual distribution of search-key values. More precisely, the hash value will not be correlated to any externally visible ordering on the search-key values, such as alphabetic ordering or ordering by the length of the search keys; the hash function will appear to be random.

As an illustration of these principles, let us choose a hash function for the *instructor* file using the search key *dept name*. The hash function that we choose must have the desirable properties not only on the example *instructor* file that we have been using, but also on an *instructor* file of realistic size for a large university with many departments.

Assume that we decide to have 26 buckets, and we define a hash function that maps names beginning with the *i*th letter of the alphabet to the *i*th bucket. This hash function has the virtue of simplicity, but it fails to provide a uniform distribution, since we expect more names to begin with such letters as B and R than Q and X, for example.

Now suppose that we want a hash function on the search key *salary*. Suppose that the minimum salary is \$30,000 and the maximum salary is \$130,000, and we use a hash function that divides the values into 10 ranges, \$30,000–\$40,000, \$40,001–\$50,000 and so on. The distribution of search-key values is uniform (since each bucket has the same number of different salary values), but is not random.

bucket 0

bucket 1

15151	Mozart	Music	40000

bucket 4

12121	Wu	Finance	90000
76543	Singh	Finance	80000

bucket 5

76766	Crick	Biology	72000

bucket 2			
32343	El Said	History	80000
58583	Califieri	History	60000

bucket 6			
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

bucket 3			
22222	Einstein	Physics	95000
33456	Gold	Physics	87000
98345	Kim	Elec. Eng.	80000

bucket 7			

Figure 9.16 Hash organization of instructor file, with dept name as the key.

Records with salaries between \$60,001 and \$70,000 are far more common than are records with salaries between \$30,001 and \$40,000. As a result, the distribution of records is not uniform—some buckets receive more records than others do. If the function has a random distribution, even if there are such correlations in the search keys, the randomness of the distribution will make it very likely that all buckets will have roughly the same number of records, as long as each search key occurs in only a small fraction of the records. (If a single search key occurs in a large fraction of the records, the bucket containing it is likely to have more records than other buckets, regardless of the hash function used.)

Typical hash functions perform computation on the internal binary machine representation of characters in the search key. A simple hash function of this type first computes the sum of the binary representations of the characters of a key, then returns the sum modulo the number of buckets. Figure 9.16 shows the application of such a scheme, with eight buckets, to the *instructor* file, under the assumption that the *i*th letter in the alphabet is represented by the integer *i*.

The following hash function is a better alternative for hashing strings. Let *s* be a string of length *n*, and let *s*[*i*] denote the *i*th byte of the string. The hash function is defined as:

$$s[0] * 31^{(n-1)} + s[1] * 31^{(n-2)} + \dots + s[n-1]$$

The function can be implemented efficiently by setting the hash result initially to 0, and iterating from the first to the last character of the string, at each step multiplying the hash value by 31 and then adding the next character (treated as an integer). The above expression would appear to result in a very large number, but it is actually computed with fixed-size positive integers; the result of each multiplication and addition is thus automatically computed modulo the largest possible integer value plus 1. The result of the above function modulo the number of buckets can then be used for indexing.

Hash functions require careful design. A bad hash function may result in lookup taking time proportional to the number of search keys in the file. A well-designed function gives an average-case lookup time that is a (small) constant, independent of the number of search keys in the file.

9.5.2 Handling of Bucket Overflows

So far, we have assumed that, when a record is inserted, the bucket to which it is mapped has space to store the record. If the bucket does not have enough space, a **bucket overflow** is said to occur. Bucket overflow can occur for several reasons:

- **Insufficient buckets.** The number of buckets, which we denote n_B , must be chosen such that $n_B > n_r / f_r$, where n_r denotes the total number of records that will be stored and f_r denotes the number of records that will fit in a bucket. This designation, of course, assumes that the total number of records is known when the hash function is chosen.
- **Skew.** Some buckets are assigned more records than are others, so a bucket may overflow even when other buckets still have space. This situation is called bucket **skew**. Skew can occur for two reasons:
 1. Multiple records may have the same search key.
 2. The chosen hash function may result in nonuniform distribution of search keys.

So that the probability of bucket overflow is reduced, the number of buckets is chosen to be $(n_r / f_r) * (1 + d)$, where d is a fudge factor, typically around 0.2. Some space is wasted: About 20 percent of the space in the buckets will be empty. But the benefit is that the probability of overflow is reduced. Despite allocation of a few more buckets than required, bucket overflow can still occur. We handle bucket overflow by using **overflow buckets**. If a record must be inserted into a bucket b , and b is already full, the system provides an overflow bucket for b , and inserts the record into the overflow bucket. If the overflow bucket is also full, the system provides another overflow bucket, and so on. All the overflow buckets of a given bucket are chained together in a linked list, as in Figure 9.17. Overflow handling using such a linked list is called **overflow chaining**.

We must change the lookup algorithm slightly to handle overflow chaining. As before, the system uses the hash function on the search key to identify a bucket b .

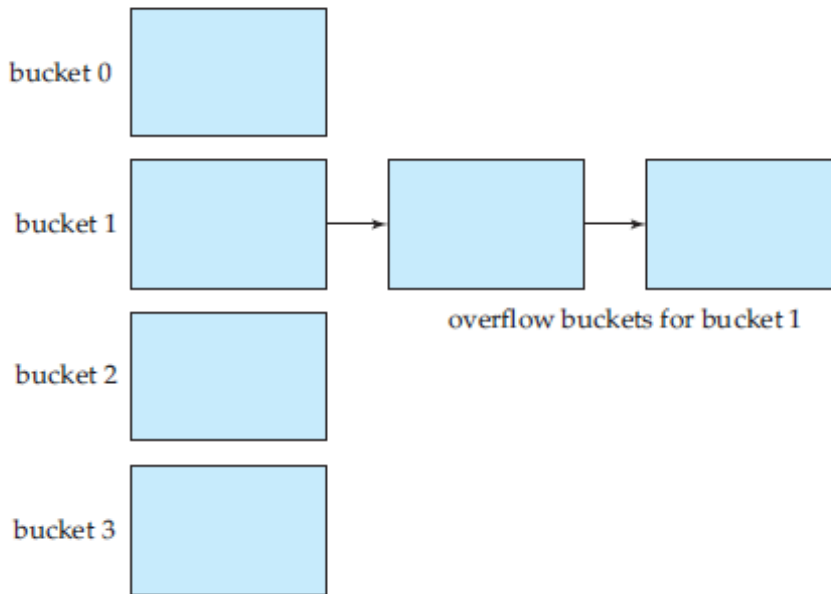


Figure 9.17 Overflow chaining in a hash structure.

The system must examine all the records in bucket b to see whether they match the search key, as before. In addition, if bucket b has overflow buckets, the system must examine the records in all the overflow buckets also. The form of hash structure that we have just described is sometimes referred to as **closed hashing**. Under an alternative approach, called **open hashing**, the set of buckets is fixed, and there are no overflow chains. Instead, if a bucket is full, the system inserts records in some other bucket in the initial set of buckets B . One policy is to use the next bucket (in cyclic order) that has space; this policy is called *linear probing*. Other policies, such as computing further hash functions, are also used. Open hashing has been used to construct symbol tables for compilers and assemblers, but closed hashing is preferable for database systems. The reason is that deletion under open hashing is troublesome. Usually, compilers and assemblers perform only lookup and insertion operations on their symbol tables.

However, in a database system, it is important to be able to handle deletion as well as insertion. Thus, open hashing is of only minor importance in database implementation.

An important drawback to the form of hashing that we have described is that we must choose the hash function when we implement the system, and it cannot be changed easily thereafter if the file being indexed grows or shrinks.

Since the function h maps search-key values to a fixed set B of bucket addresses, we waste space if B is made large to handle future growth of the file. If B is too small, the buckets contain records of many different search-key values, and bucket overflows can occur. As the file grows, performance suffers.

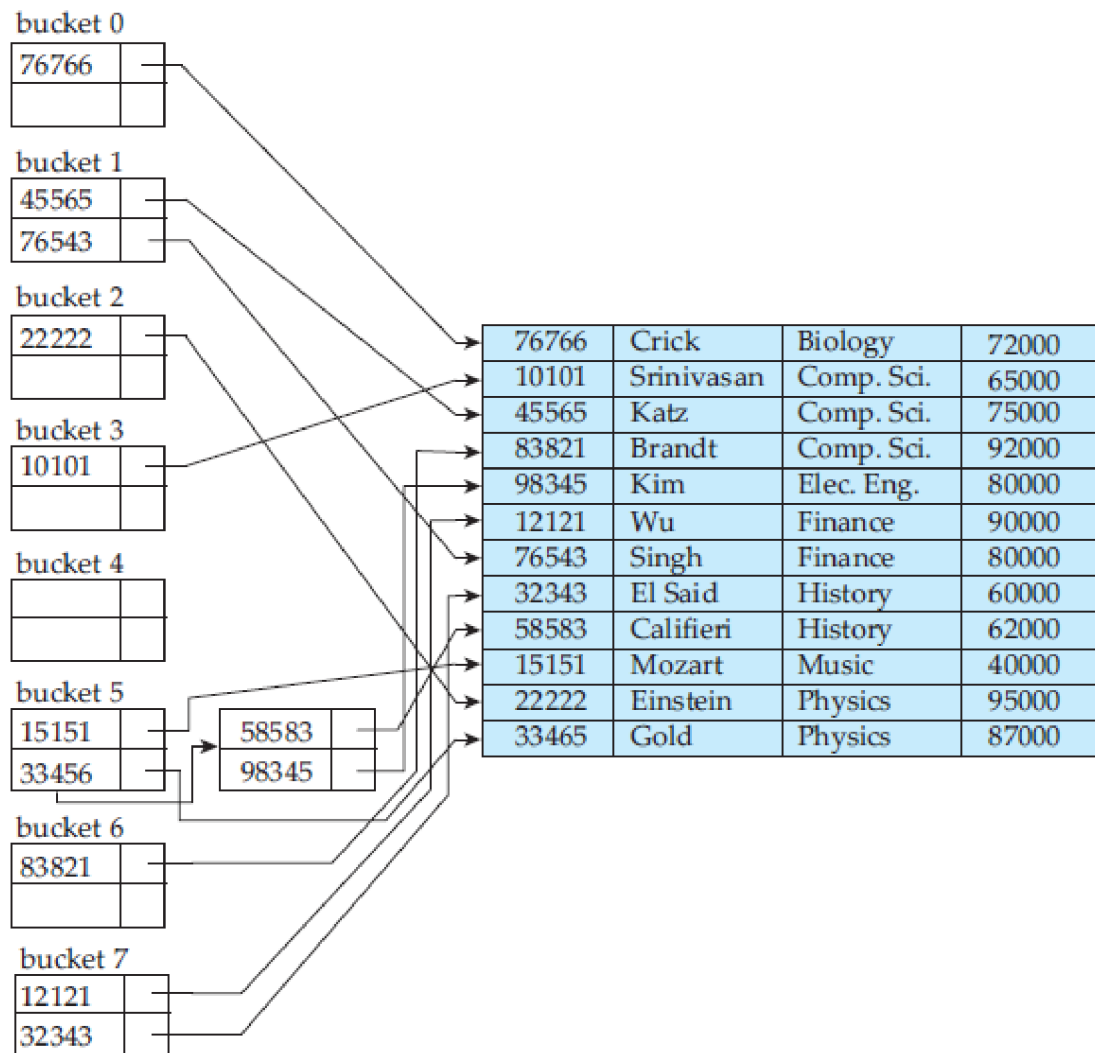


Figure 9.18 Hash index on search key *ID* of instructor file.

9.5.3 Hash Indices

Hashing can be used not only for file organization, but also for index-structure creation. A **hash index** organizes the search keys, with their associated pointers, into a hash file structure. We construct a hash index as follows. We apply a hash function on a search key to identify a bucket, and store the key and its associated pointers in the bucket (or in overflow buckets). Figure 9.5.3 shows a secondary hash index on the *instructor* file, for the search key *ID*. The hash function in the figure computes the sum of the digits of the *ID* modulo 8. The hash index has eight buckets, each of size 2 (realistic indices would, of course, have much larger bucket sizes). One of the buckets has three keys mapped to it, so it has an overflow bucket. In this example, *ID* is a primary key for *instructor*, so each

search key has only one associated pointer. In general, multiple pointers can be associated with each key.

We use the term *hash index* to denote hash file structures as well as secondary hash indices. Strictly speaking, hash indices are only secondary index structures. A hash index is never needed as a clustering index structure, since, if a file itself is organized by hashing, there is no need for a separate hash index structure on it.

However, since hash file organization provides the same direct access to records that indexing provides, we pretend that a file organized by hashing also has a clustering hash index on it.

9.6 Dynamic Hashing

As we have seen, the need to fix the set B of bucket addresses presents a serious problem with the static hashing technique of the previous section. Most databases grow larger over time. If we are to use static hashing for such a database, we have three classes of options:

1. Choose a hash function based on the current file size. This option will result in performance degradation as the database grows.
2. Choose a hash function based on the anticipated size of the file at some point in the future. Although performance degradation is avoided, a significant amount of space may be wasted initially.
3. Periodically reorganize the hash structure in response to file growth. Such a reorganization involves choosing a new hash function, recomputing the hash function on every record in the file, and generating new bucket assignments. This reorganization is a massive, time-consuming operation.

Furthermore, it is necessary to forbid access to the file during reorganization. Several **dynamic hashing** techniques allow the hash function to be modified dynamically to accommodate the growth or shrinkage of the database. In this section we describe one form of dynamic hashing, called **extendable hashing**.

The bibliographical notes provide references to other forms of dynamic hashing.

9.6.1 Data Structure

Extendable hashing copes with changes in database size by splitting and coalescing buckets as the database grows and shrinks. As a result, space efficiency is retained.

Moreover, since the reorganization is performed on only one bucket at a time, the resulting performance overhead is acceptably low. With extendable hashing, we choose a hash function h with the desirable properties of uniformity and randomness. However, this hash function generates values over a relatively large range—namely, b -bit binary integers. A typical value for b is 32.

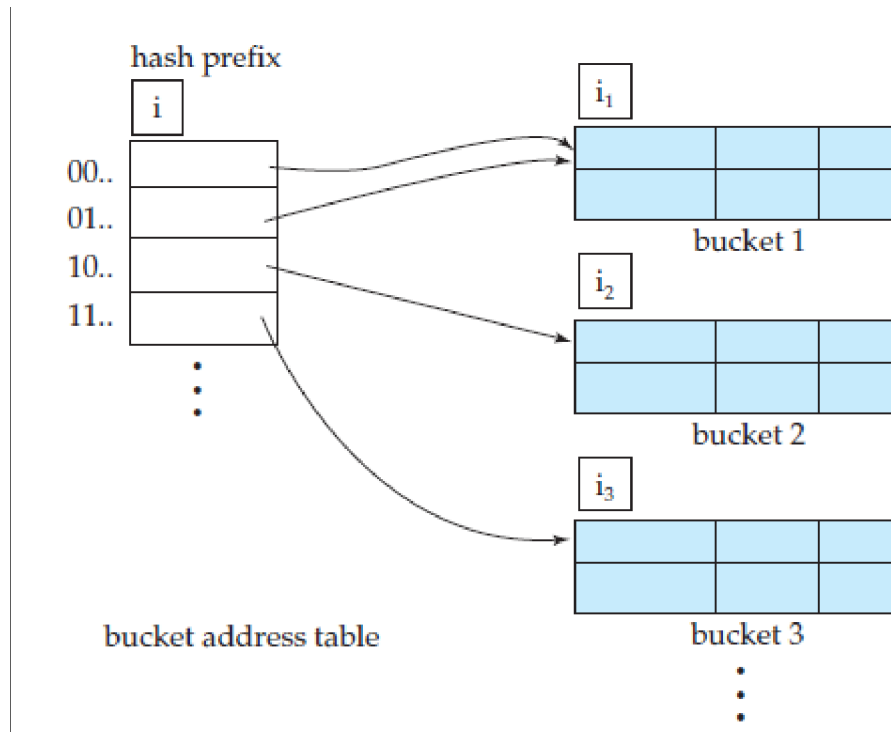


Figure 9.19 General extendable hash structure.

We do not create a bucket for each hash value. Indeed, 2^{32} is over 4 billion, and that many buckets is unreasonable for all but the largest databases. Instead, we create buckets on demand, as records are inserted into the file. We do not use the entire b bits of the hash value initially. At any point, we use i bits, where $0 \leq i \leq b$. These i bits are used as an offset into an additional table of bucket addresses. The value of i grows and shrinks with the size of the database.

Figure 9.19 shows a general extendable hash structure. The i appearing above the bucket address table in the figure indicates that i bits of the hash value $h(K)$ are required to determine the correct bucket for K . This number will, of course, change as the file grows. Although i bits are required to find the correct entry in the bucket address table, several consecutive table entries may point to the same bucket. All such entries will have a common hash prefix, but the length of this prefix may be less than i . Therefore, we associate with

each bucket an integer giving the length of the common hash prefix. In Figure 9.19 the integer associated with bucket j is shown as i_j . The number of bucket-address-table entries that point to bucket j is $2^{(i - i_j)}$.

9.6.2 Queries and Updates

We now see how to perform lookup, insertion, and deletion on an extendable hash structure. To locate the bucket containing search-key value K_i , the system takes the first i high-order bits of $h(K_i)$, looks at the corresponding table entry for this bit string, and follows the bucket pointer in the table entry.

To insert a record with search-key value K_i , the system follows the same procedure for lookup as before, ending up in some bucket—say, j . If there is room in the bucket, the system inserts the record in the bucket. If, on the other hand, the bucket is full, it must split the bucket and redistribute the current records, plus the new one. To split the bucket, the system must first determine from the hash value whether it needs to increase the number of bits that it uses.

- If $i = i_j$, only one entry in the bucket address table points to bucket j .

Therefore, the system needs to increase the size of the bucket address table so that it can include pointers to the two buckets that result from splitting bucket j . It does so by considering an additional bit of the hash value. It increments the value of i by 1, thus doubling the size of the bucket address table. It replaces each entry by two entries, both of which contain the same pointer as the original entry. Now two entries in the bucket address table point to bucket j . The system allocates a new bucket (bucket z), and sets the second entry to point to the new bucket. It sets i_j and i_z to i . Next, it rehashes each record in bucket j and, depending on the first i bits (remember the system has added 1 to i), either keeps it in bucket j or allocates it to the newly created bucket.

The system now reattempts the insertion of the new record. Usually, the attempt will succeed. However, if all the records in bucket j , as well as the new record, have the same hash-value prefix, it will be necessary to split a bucket again, since all the records in bucket j and the new record are assigned to the same bucket. If the hash function has been chosen carefully, it is unlikely that a single insertion will require that a bucket be split more than once, unless there are a large number of records with the same search key. If all the records in bucket j have the same search-key value, no amount of splitting will help. In such cases, overflow buckets are used to store the records, as in static hashing.

• If $i > j$, then more than one entry in the bucket address table points to bucket j . Thus, the system can split bucket j without increasing the size of the bucket address table. Observe that all the entries that point to bucket j correspond to hash prefixes that have the same value on the leftmost j bits. The system allocates a new bucket (bucket z), and sets i_j and iz to the value that results from adding 1 to the original i_j value. Next, the system needs to adjust the entries in the bucket address table that previously pointed to bucket j . (Note that with the new value for i_j , not all the entries correspond to hash prefixes that have the same value on the leftmost j bits.) The system leaves the first half of the entries as they were (pointing to bucket j), and sets all the remaining entries to point to the newly created bucket (bucket z).

Next, as in the previous case, the system rehashes each record in bucket j , and allocates it either to bucket j or to the newly created bucket z .

<i>dept_name</i>	<i>h(dept_name)</i>
Biology	0010 1101 1111 1011 0010 1100 0011 0000
Comp. Sci.	1111 0001 0010 0100 1001 0011 0110 1101
Elec. Eng.	0100 0011 1010 1100 1100 0110 1101 1111
Finance	1010 0011 1010 0000 1100 0110 1001 1111
History	1100 0111 1110 1101 1011 1111 0011 1010
Music	0011 0101 1010 0110 1100 1001 1110 1011
Physics	1001 1000 0011 1111 1001 1100 0000 0001

Figure 9.20 Hash function for dept name.

The system then reattempts the insert. In the unlikely case that it again fails, it applies one of the two cases, $i = i_j$ or $i > i_j$, as appropriate. Note that, in both cases, the system needs to recompute the hash function on only the records in bucket j .

To delete a record with search-key value K_p , the system follows the same procedure for lookup as before, ending up in some bucket—say, j . It removes both the search key from the bucket and the record from the file. The bucket, too, is removed if it becomes empty. Note that, at this point, several buckets can be coalesced, and the size of the bucket address table can be cut in half. The procedure for deciding on which buckets can be coalesced and how to coalesce buckets is left to you to do as an exercise. The conditions under which the bucket address table can be reduced in size are also left to you as an exercise. Unlike coalescing of buckets, changing the size of the bucket address table is a rather expensive operation if the table is large. Therefore it may be worthwhile to reduce the bucket-address-table size only if the number of buckets reduces greatly.

To illustrate the operation of insertion, we use the *instructor* file in Figure 9.20 and assume that the search key is *dept name* with the 32-bit hash values as appear in Figure 9.21. Assume that, initially, the file is empty, as in Figure 9.22. We insert the records one by one. To illustrate all the features of extendable hashing in a small structure, we shall make the unrealistic assumption that a bucket can hold only two records.

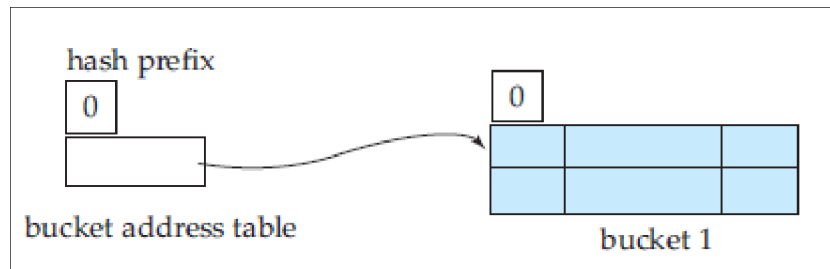


Figure 9.21 Initial extendable hash structure.

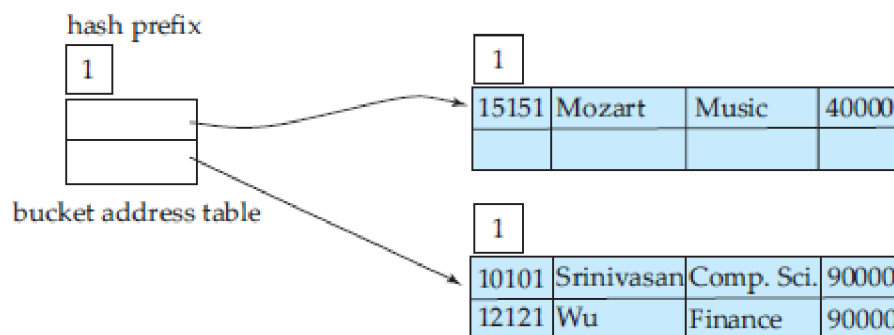


Figure 9.22 Hash structure after three insertions.

We insert the record (10101, Srinivasan, Comp. Sci., 65000). The bucket address table contains a pointer to the one bucket, and the system inserts the record. Next, we insert the record (12121, Wu, Finance, 90000). The system also places this record in the one bucket of our structure.

When we attempt to insert the next record (15151, Mozart, Music, 40000), we find that the bucket is full. Since $i = i_0$, we need to increase the number of bits that we use from the hash value. We now use 1 bit, allowing us $2^1 = 2$ buckets. This increase in the number of bits necessitates doubling the size of the bucket address table to two entries. The system splits the bucket, placing in the new bucket those records whose search key has a hash value beginning with 1, and leaving in the original bucket the other records. Figure 9.23 shows the state of our structure after the split.

Next, we insert (22222, Einstein, Physics, 95000). Since the first bit of $h(\text{Physics})$ is 1, we must insert this record into the bucket pointed to by the "1" entry in the bucket address table.

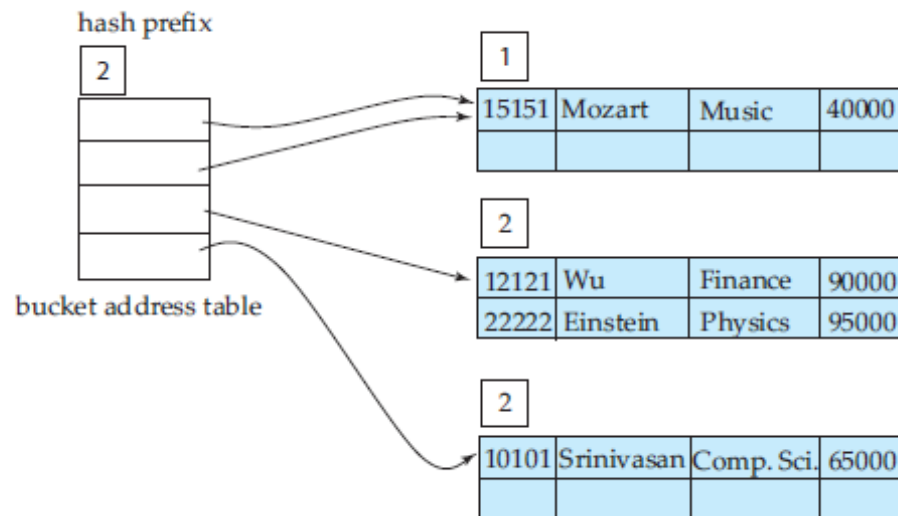


Figure 9.23 Hash structure after four insertions.

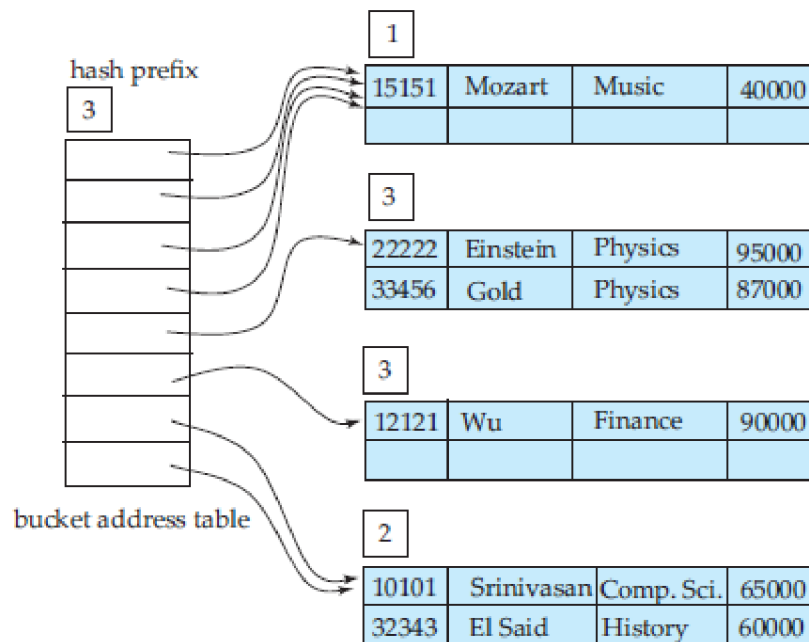


Figure 9.24 Hash structure after six insertions.

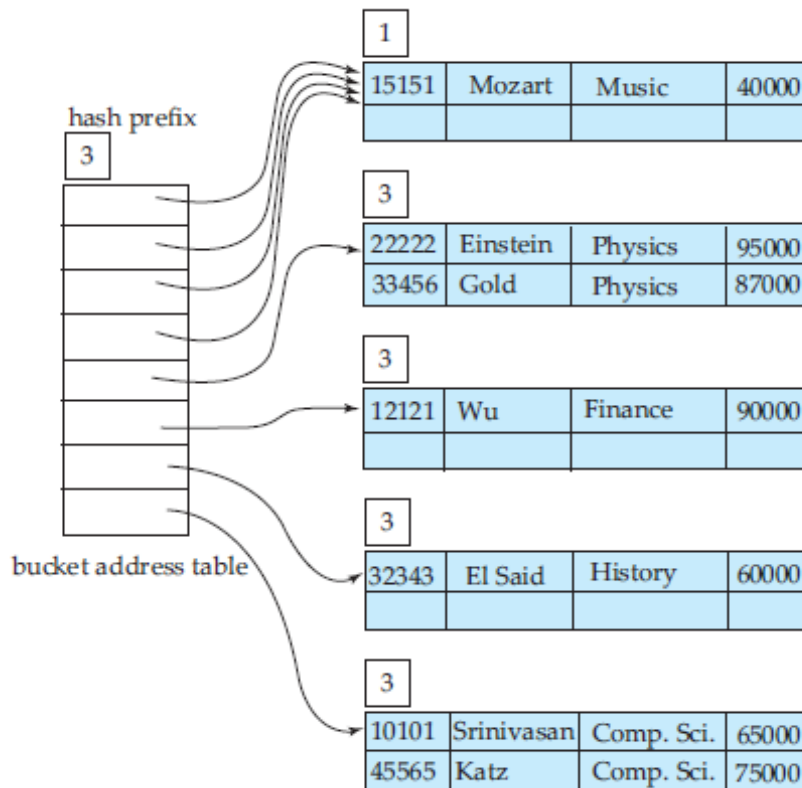


Figure 9.25 Hash structure after seven insertions.

Once again, we find the bucket full and $i = i/1$. We increase the number of bits that we use from the hash to 2. This increase in the number of bits necessitates doubling the size of the bucket address table to four entries, as in Figure 9.24. Since the bucket of Figure 9.23 for hash prefix 0 was not split, the two entries of the bucket address table of 00 and 01 both point to this bucket. For each record in the bucket of Figure 9.23 for hash prefix 1 (the bucket being split), the system examines the first 2 bits of the hash value to determine which bucket of the new structure should hold it.

Next, we insert (32343, El Said, History, 60000), which goes in the same bucket as Comp. Sci. The following insertion of (33456, Gold, Physics, 87000) results in a bucket overflow, leading to an increase in the number of bits, and a doubling of the size of the bucket address table (see Figure 9.25).

The insertion of (45565, Katz, Comp. Sci., 75000) leads to another bucket overflow; this overflow, however, can be handled without increasing the number of bits, since the bucket in question has two pointers pointing to it (see Figure 9.26).

Next, we insert the records of "Califieri", "Singh", and "Crick" without any bucket overflow. The insertion of the third Comp. Sci. record (83821, Brandt, Comp. Sci., 92000),

however, leads to another overflow. This overflow cannot be handled by increasing the number of bits, since there are three records with exactly the same hash value. Hence the system uses an overflow bucket, as in Figure 9.27.

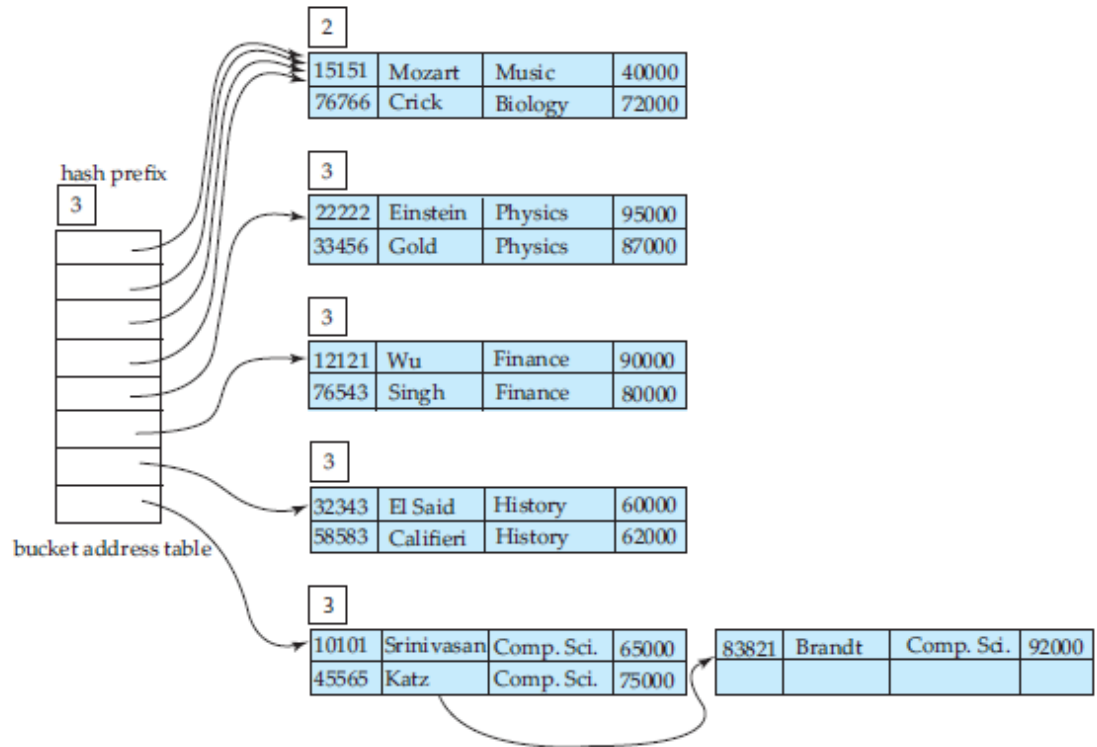


Figure 9.26 Hash structure after eleven insertions.

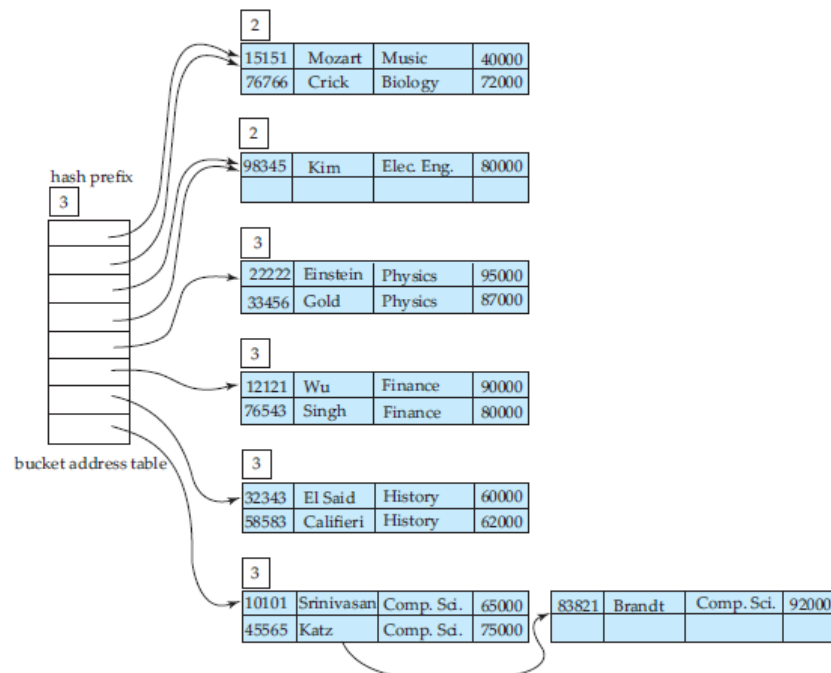


Figure 9.27 Extendable hash structure for the instructor file.

9.6.3 Static Hashing versus Dynamic Hashing

We now examine the advantages and disadvantages of extendable hashing, compared with static hashing. The main advantage of extendable hashing is that performance does not degrade as the file grows. Furthermore, there is minimal space overhead. Although the bucket address table incurs additional overhead, it contains one pointer for each hash value for the current prefix length. This table is thus small. The main space saving of extendable hashing over other forms of hashing is that no buckets need to be reserved for future growth; rather, buckets can be allocated dynamically.

A disadvantage of extendable hashing is that lookup involves an additional level of indirection, since the system must access the bucket address table before accessing the bucket itself. This extra reference has only a minor effect on performance. Although the hash structures do not have this extra level of indirection, they lose their minor performance advantage as they become full.

Thus, extendable hashing appears to be a highly attractive technique, provided that we are willing to accept the added complexity involved in its implementation. The bibliographical notes reference more detailed descriptions of extendable hashing implementation.

The bibliographical notes also provide references to another form of dynamic hashing called **linear hashing**, which avoids the extra level of indirection associated with extendable hashing, at the possible cost of more overflow buckets.

9.7 Summary

- Many queries reference only a small proportion of the records in a file. To reduce the overhead in searching for these records, we can construct *indices* for the files that store the database.
- Index-sequential files are one of the oldest index schemes used in database systems. To permit fast retrieval of records in search-key order, records are stored sequentially, and out-of-order records are chained together. To allow fast random access, we use an index structure.
- There are two types of indices that we can use: dense indices and sparse indices. Dense indices contain entries for every search-key value, whereas sparse indices contain entries only for some search-key values.
- The primary disadvantage of the index-sequential file organization is that performance degrades as the file grows. To overcome this deficiency, we can use a *B+-tree index*.

- Sequential file organizations require an index structure to locate data. File organizations based on hashing, by contrast, allow us to find the address of a data item directly by computing a function on the search-key value of the desired record. Since we do not know at design time precisely which search key values will be stored in the file, a good hash function to choose is one that assigns search-key values to buckets such that the distribution is both uniform and random.
- *Static hashing* uses hash functions in which the set of bucket addresses is fixed. Such hash functions cannot easily accommodate databases that grow significantly larger over time. There are several *dynamic hashing techniques* that allow the hash function to be modified. One example is *extendable hashing*, which copes with changes in database size by splitting and coalescing buckets as the database grows and shrinks.

9.8 Lesson and Activities

1. Indices speed query processing, but it is usually a bad idea to create indices on every attribute, and every combinations of attributes, that is a potential search keys. Explain why.
2. Records with the following search-key values:

2, 3, 5, 7, 11, 17, 19, 23, 29, 31

Show the extendable hash structure for this file if the hash function is $h(x) = x \bmod 8$ and buckets can hold three records.

9.9 Keywords

- Access types
- Access time
- Insertion time
- Deletion time
- Space overhead
- Ordered index
- Clustering index
- Primary index

- Nonclustering index
- Secondary index
- Index-sequential file
- Index entry/record
- Dense index
- Sparse index
- Multilevel index

9.10 Questions for Discussion

1. Explain B⁺ Tree Index file concepts in detail.
2. Discuss multiple key access method in detail.
3. Explain Static hasing.
4. Explain Dynamic hashing.

9.11 Suggested Reading

“Database System Concepts” -5th Edition by Silberchatz. Korth. Sudarshan., TMH.

Knuth [1973] analyzes a large number of different hashing techniques.

Vitter [2001] provides an extensive survey of external-memory data structures and algorithms.

Lesson - 10

QUERY PROCESSING

CONTENTS

10.0 Aim and Objectives

10.1 Introduction

10.2 Overview

10.3 Measures of Query Cost

10.4 Selection Operation

10.4.1 Selections Using File Scans and Indices

10.4.2 Selections Involving Comparisons

10.4.3 Implementation of Complex Selections

10.5 Sorting

10.5.1 External Sort-Merge Algorithm

10.5.2 Cost Analysis of External Sort-Merge

10.6 Join Operation

10.6.1 Nested-Loop Join

10.6.2 Block Nested-Loop Join

10.6.3 Indexed Nested-Loop Join

10.6.4 Merge Join

10.6.5 Hash Join

10.6.5.1 Basics

10.6.5.2 Recursive Partitioning

10.6.5.3 Handling Overflow

10.7 Query Optimization

10.7.1 Overview

10.7.2 Choice of Evaluation Plans

10.7.2.1 Cost-Based Join Order Selection

10.7.2.2 Cost-Based Optimization with Equivalence Rules

10.7.2.3 Heuristics in Optimization

10.7.2.4 Optimizing Nested Subqueries

10.8 Summary

10.9 Lesson End Activities

10.10 Keywords

10.11 Questions for Discussion

10.12 Suggested Reading

10.0 AIM AND OBJECTIVES

After studying this lesson, you should be able to:

- Analysis of measures of query cost.
- Understanding of selection operation.
- Analysis Sorting and Join operation.
- Understanding of query optimization.

10.1 Introduction

Query processing refers to the range of activities involved in extracting data from a database. The activities include translation of queries in high-level database languages into expressions that can be used at the physical level of the file system, a variety of query-optimizing transformations, and actual evaluation of queries.

10.2 Overview

The steps involved in processing a query appear in Figure 10.1. The basic steps are:

1. Parsing and translation.
2. Optimization.
3. Evaluation.

Before query processing can begin, the system must translate the query into a usable form. A language such as SQL is suitable for human use, but is ill suited to be the system's internal representation of a query. A more useful internal representation is one based on the extended relational algebra.

Thus, the first action the system must take in query processing is to translate a given query into its internal form. This translation process is similar to the work performed by the parser of a compiler. In generating the internal form of the query, the parser checks the syntax of the user's query, verifies that the relation names appearing in the query are names of the relations in the database, and so on. The system constructs a parse-tree representation of the query, which it then translates into a relational-algebra expression. If the query was expressed in terms of a view, the translation phase also replaces all uses of the view by the relational-algebra expression that defines the view. Most compiler texts cover parsing in detail.

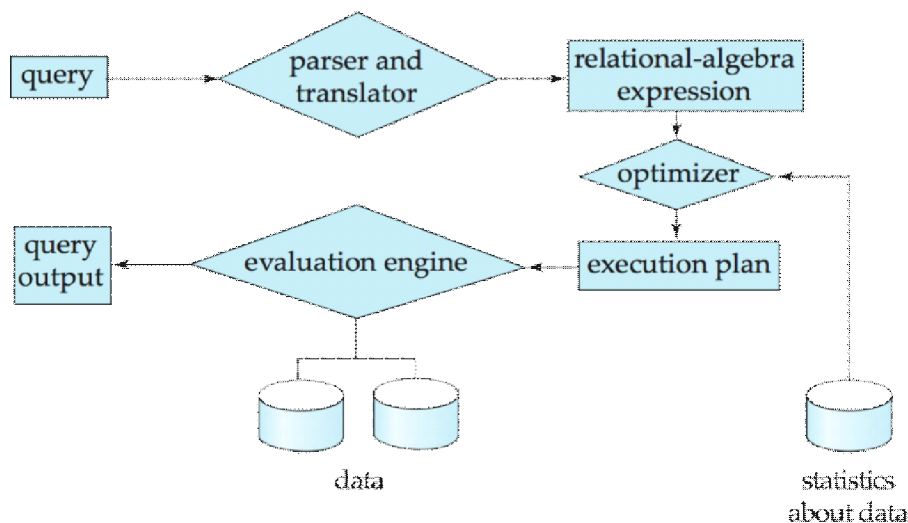


Figure 10.1 Steps in query processing.

Given a query, there are generally a variety of methods for computing the answer. For example, we have seen that, in SQL, a query could be expressed in several different ways. Each SQL query can itself be translated into a relationalalgebra expression in one of several ways. Furthermore, the relational-algebra representation of a query specifies only partially howto evaluate a query; there are usually several ways to evaluate relational-algebra expressions. As an illustration, consider the query:

select salary from instructor where salary < 75000;

This query can be translated into either of the following relational-algebra expressions:

- $\sigma_{salary < 75000} (\pi_{salary} (instructor))$
- $\pi_{salary} (\sigma_{salary < 75000} (instructor))$

Further, we can execute each relational-algebra operation by one of several different algorithms. For example, to implement the preceding selection, we can search every tuple in *instructor* to find tuples with salary less than 75000. If a B+-tree index is available on the attribute *salary*, we can use the index instead to locate the tuples.

To specify fully how to evaluate a query, we need not only to provide the relational-algebra expression, but also to annotate it with instructions specifying how to evaluate each operation. Annotations may state the algorithm to be used for a specific operation, or the particular index or indices to use.

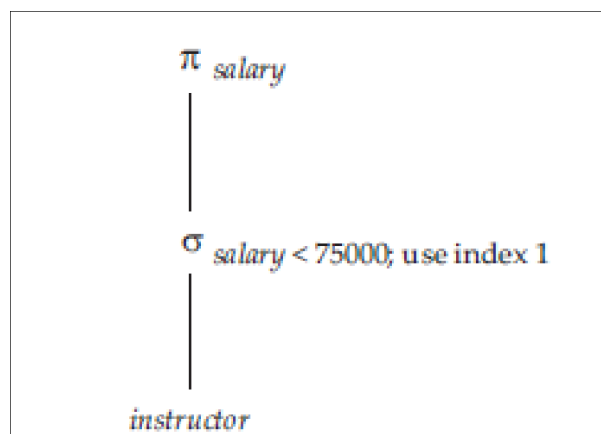


Figure 10.2 A query-evaluation plan.

A relational algebra operation annotated with instructions on how to evaluate it is called an **evaluation primitive**. A sequence of primitive operations that can be used to evaluate a query is a **query-execution plan** or **query-evaluation plan**. Figure 10.2 illustrates an evaluation plan for our example query, in which a particular index (denoted in the figure as “index 1”) is specified for the selection operation. The **query-execution engine** takes a query-evaluation plan, executes that plan, and returns the answers to the query.

The different evaluation plans for a given query can have different costs. We do not expect users to write their queries in a way that suggests the most efficient evaluation plan. Rather, it is the responsibility of the system to construct a queryevaluation plan that minimizes the cost of query evaluation; this task is called *query optimization*.

Once the query plan is chosen, the query is evaluated with that plan, and the result of the query is output. The sequence of steps already described for processing a query is representative; not all databases exactly follow those steps. For instance, instead of using

the relational-algebra representation, several databases use an annotated parsetree representation based on the structure of the given SQL query. However, the concepts that we describe here form the basis of query processing in databases.

In order to optimize a query, a query optimizer must know the cost of each operation. Although the exact cost is hard to compute, since it depends on many parameters such as actual memory available to the operation, it is possible to get a rough estimate of execution cost for each operation.

In this chapter we study how to evaluate individual operations in a query plan, and how to estimate their cost; we return to query optimization. How we measure the cost of a query. Several operations may be grouped together into a **pipeline**, in which each of the operations starts working on its input tuples even as they are being generated by another operation. We examine how to coordinate the execution of multiple operations in a query evaluation plan, in particular, how to use pipelined operations to avoid writing intermediate results to disk.

10.3 Measures of Query Cost

There are multiple possible evaluation plans for a query, and it is important to be able to compare the alternatives in terms of their (estimated) cost, and choose the best plan. To do so, we must estimate the cost of individual operations, and combine them to get the cost of a query evaluation plan.

The cost of query evaluation can be measured in terms of a number of different resources, including disk accesses, CPU time to execute a query, and, in a distributed or parallel database system, the cost of communication.

In large database systems, the cost to access data from disk is usually the most important cost, since disk accesses are slow compared to in-memory operations. Moreover, CPU speeds have been improving much faster than have disk speeds. Thus, it is likely that the time spent in disk activity will continue to dominate the total time to execute a query. The CPU time taken for a task is harder to estimate since it depends on low-level details of the execution code. Although real-life query optimizers do take CPU costs into account, for simplicity in this book we ignore CPU costs and use only disk-access costs to measure the cost of a query-evaluation plan.

We use the *number of block transfers* from disk as a measure of the actual cost. For

simplicity, we ignore this detail, and leave it to you to work out more precise cost estimates for various operations.

A More accurate measure would therefore estimate

- 1) The number of seek operations performed
- 2) The number of blocks read
- 3) The number of blocks written

The cost estimates we give do not include the cost of writing the final result of an operation back to disk. These are taken into account separately where required. The costs of all the algorithms that we consider depend on the size of the buffer in main memory. In the best case, all data can be read into the buffers, and the disk does not need to be accessed again. In the worst case, we assume that the buffer can hold only a few blocks of data—approximately one block per relation. When presenting cost estimates, we generally assume the worst case.

In addition, although we assume that data must be read from disk initially, it is possible that a block that is accessed is already present in the in-memory buffer. Again, for simplicity, we ignore this effect; as a result, the actual disk-access cost during the execution of a plan may be less than the estimated cost. The **response time** for a query-evaluation plan (that is, the wall-clock time required to execute the plan), assuming no other activity is going on in the computer, would account for all these costs, and could be used as a measure of the cost of the plan. Unfortunately, the response time of a plan is very hard to estimate without actually executing the plan, for the following reasons:

1. The response time depends on the contents of the buffer when the query begins execution; this information is not available when the query is optimized, and is hard to account for even if it were available.
2. In a system with multiple disks, the response time depends on how accesses are distributed among disks, which is hard to estimate without detailed knowledge of data layout on disk.

Interestingly, a plan may get a better response time at the cost of extra resource consumption. For example, if a system has multiple disks, a plan *A* that requires extra disk reads, but performs the reads in parallel across multiple disks may finish faster than another

plan *B* that has fewer disk reads, but from only one disk. However, if many instances of a query using plan *A* run concurrently, the overall response time may actually be more than if the same instances are executed using plan *B*, since plan *A* generates more load on the disks.

As a result, instead of trying to minimize the response time, optimizers generally try to minimize the total **resource consumption** of a query plan. Our model of estimating the total disk access time (including seek and data transfer) is an example of such a resource consumption–based model of query cost.

10.4 Selection Operation

In query processing, the **file scan** is the lowest-level operator to access data. File scans are search algorithms that locate and retrieve records that fulfill a selection condition. In relational systems, a file scan allows an entire relation to be read in those cases where the relation is stored in a single, dedicated file.

10.4.1 Selections Using File Scans and Indices

Consider a selection operation on a relation whose tuples are stored together in one file. The most straightforward way of performing a selection is as follows:

- **A1 (linear search).** In a linear search, the system scans each file block and tests all records to see whether they satisfy the selection condition. An initial seek is required to access the first block of the file. In case blocks of the file are not stored contiguously, extra seeks may be required, but we ignore this effect for simplicity.

Although it may be slower than other algorithms for implementing selection, the linear-search algorithm can be applied to any file, regardless of the ordering of the file, or the availability of indices, or the nature of the selection operation. The other algorithms that we shall study are not applicable in all cases, but when applicable they are generally faster than linear search.

Cost estimates for linear scan, as well as for other selection algorithms, are shown in Figure 10.3. In the figure, we use *hi* to represent the height of the B+- tree. Real-life optimizers usually assume that the root of the tree is present in the in-memory buffer since it is frequently accessed. Some optimizers even assume that all but the leaf level of the tree is present in memory, since they are accessed relatively frequently, and usually less than 1 percent of the nodes of a B+-tree are nonleaf nodes. The cost formulae can be modified appropriately.

Index structures are referred to as **access paths**, since they provide a path through which data can be located and accessed. Recall that a *primary index* (also referred to as a *clustering index*) is an index that allows the records of a file to be read in an order that corresponds to the physical order in the file. An index that is not a primary index is called a *secondary index*.

Search algorithms that use an index are referred to as **index scans**. We use the selection predicate to guide us in the choice of the index to use in processing the query. Search algorithms that use an index are:

- **A2 (primary index, equality on key).** For an equality comparison on a key attribute with a primary index, we can use the index to retrieve a single record that satisfies the corresponding equality condition. Cost estimates are shown in Figure 10.3.
- **A3 (primary index, equality on nonkey).** We can retrieve multiple records by using a primary index when the selection condition specifies an equality comparison on a nonkey attribute, A . The only difference from the previous case is that multiple records may need to be fetched. However, the records must be stored consecutively in the file since the file is sorted on the search key. Cost estimates are shown in Figure 10.3.
- **A4 (secondary index, equality).** Selections specifying an equality condition can use a secondary index. This strategy can retrieve a single record if the equality condition is on a key; multiple records may be retrieved if the indexing field is not a key.

In the first case, only one record is retrieved. The time cost in this case is the same as that for a primary index (case A2).

In the second case, each record may be resident on a different block, which may result in one I/O operation per retrieved record, with each I/O operation requiring a seek and a block transfer. The worst-case time cost in this case is $(h_i + n) * (t_s + t_r)$, where n is the number of records fetched, if each record is in a different disk block, and the block fetches are randomly ordered. The worst-case cost could become even worse than that of linear search if a large number of records are retrieved.

	Algorithm	Cost	Reason
A1	Linear Search	$t_s + b_r * t_T$	One initial seek plus b_r block transfers, where b_r denotes the number of blocks in the file.
A1	Linear Search, Equality on Key	Average case $t_s + (b_r/2) * t_T$	Since at most one record satisfies condition, scan can be terminated as soon as the required record is found. In the worst case, b_r blocks transfers are still required.
A2	Primary B ⁺ -tree Index, Equality on Key	$(h_i + 1) * (t_T + t_s)$	(Where h_i denotes the height of the index.) Index lookup traverses the height of the tree plus one I/O to fetch the record; each of these I/O operations requires a seek and a block transfer.
A3	Primary B ⁺ -tree Index, Equality on Nonkey	$h_i * (t_T + t_s) + b * t_T$	One seek for each level of the tree, one seek for the first block. Here b is the number of blocks containing records with the specified search key, all of which are read. These blocks are leaf blocks assumed to be stored sequentially (since it is a primary index) and don't require additional seeks.
A4	Secondary B ⁺ -tree Index, Equality on Key	$(h_i + 1) * (t_T + t_s)$	This case is similar to primary index.
A4	Secondary B ⁺ -tree Index, Equality on Nonkey	$(h_i + n) * (t_T + t_s)$	(Where n is the number of records fetched.) Here, cost of index traversal is the same as for A3, but each record may be on a different block, requiring a seek per record. Cost is potentially very high if n is large.
A5	Primary B ⁺ -tree Index, Comparison	$h_i * (t_T + t_s) + b * t_T$	Identical to the case of A3, equality on nonkey.
A6	Secondary B ⁺ -tree Index, Comparison	$(h_i + n) * (t_T + t_s)$	Identical to the case of A4, equality on nonkey.

Figure 10.3 Cost estimates for selection algorithms.

If the in-memory buffer is large, the block containing the record may already be in the buffer. It is possible to construct an estimate of the *average* or *expected* cost of the selection by taking into account the probability of the block containing the record already being in the buffer. For large buffers, that estimate will be much less than the worst-case estimate.

In certain algorithms, including A2, the use of a B+-tree file organization can save one access since records are stored at the leaf-level of the tree.

Accessing a record through such a secondary index is then more expensive: First the secondary index is searched to find the primary index search-key values, then the primary index is looked up to find the records. The cost formulae described for secondary indices have to be modified appropriately if such indices are used.

10.4.2 Selections Involving Comparisons

Consider a selection of the form $\sigma_{A \leq v}(r)$. We can implement the selection either by using linear search or by using indices in one of the following ways:

- **A5 (primary index, comparison).** A primary ordered index (for example, a primary B+-tree index) can be used when the selection condition is a comparison. For comparison conditions of the form $A > v$ or $A \geq v$, a primary index on A can be used to direct the retrieval of tuples, as follows: For $A \geq v$, we look up the value v in the index to find the first tuple in the file that has a value of $A = v$. A file scan starting from that tuple up to the end of the file returns all tuples that satisfy the condition. For $A > v$, the file scan starts with the first tuple such that $A > v$. The cost estimate for this case is identical to that for case A3.

For comparisons of the form $A < v$ or $A \leq v$, an index lookup is not required. For $A < v$, we use a simple file scan starting from the beginning of the file, and continuing up to (but not including) the first tuple with attribute $A = v$. The case of $A \leq v$ is similar, except that the scan continues up to (but not including) the first tuple with attribute $A > v$. In either case, the index is not useful.

- **A6 (secondary index, comparison).** We can use a secondary ordered index to guide retrieval for comparison conditions involving $<$, $\leq$, $\geq$, or $>$. The lowest-level index blocks are scanned, either from the smallest value up to v (for $<$ and $\leq$), or from v up to the maximum value (for $>$ and $\geq$).

The secondary index provides pointers to the records, but to get the actual records we have to fetch the records by using the pointers. This step may require an I/O operation for each record fetched, since consecutive records may be on different disk blocks; as before, each I/O operation requires a disk seek and a block transfer. If the number of retrieved records is large, using the secondary index may be even more expensive than using linear search. Therefore the secondary index should be used only if very few records are selected.

10.4.3 Implementation of Complex Selections

So far, we have considered only simple selection conditions of the form $A \text{ op } B$, where op is an equality or comparison operation. We now consider more complex selection predicates.

- **Conjunction:** A *conjunctive selection* is a selection of the form: $\sigma_{\theta_1 \wedge \theta_2 \dots \wedge \theta_n}(r)$
- **Disjunction:** A *disjunctive selection* is a selection of the form: $\sigma_{\theta_1 \vee \theta_2 \dots \vee \theta_n}(r)$
A disjunctive condition is satisfied by the union of all records satisfying the individual, simple conditions θ_i .
- **Negation:** The result of a selection $\sigma_{\neg\theta}(r)$ is the set of tuples of r for which the condition θ evaluates to false. In the absence of nulls, this set is simply the set of tuples in r that are not in $\sigma_{\theta}(r)$.

We can implement a selection operation involving either a conjunction or a disjunction of simple conditions by using one of the following algorithms:

- **A7 (conjunctive selection using one index).** We first determine whether an access path is available for an attribute in one of the simple conditions. If one is, one of the selection algorithms A2 through A6 can retrieve records satisfying that condition. We complete the operation by testing, in the memory buffer, whether or not each retrieved record satisfies the remaining simple conditions.

To reduce the cost, we choose a θ_i and one of algorithms A1 through A6 for which the combination results in the least cost for $\sigma_{\theta_i}(r)$. The cost of algorithm A7 is given by the cost of the chosen algorithm.

- **A8 (conjunctive selection using composite index).** An appropriate *composite index* (that is, an index on multiple attributes) may be available for some conjunctive selections. If the selection specifies an equality condition on two or more attributes, and a composite index exists on these combined attribute fields, then the index can be searched directly. The type of index determines which of algorithms A2, A3, or A4 will be used.
- **A9 (conjunctive selection by intersection of identifiers).** Another alternative for implementing conjunctive selection operations involves the use of record pointers or record identifiers. This algorithm requires indices with record pointers, on the fields involved in the individual conditions. The algorithm scans each index for pointers to

tuples that satisfy an individual condition. The intersection of all the retrieved pointers is the set of pointers to tuples that satisfy the conjunctive condition. The algorithm then uses the pointers to retrieve the actual records. If indices are not available on all the individual conditions, then the algorithm tests the retrieved records against the remaining conditions.

The cost of algorithm **A9** is the sum of the costs of the individual index scans, plus the cost of retrieving the records in the intersection of the retrieved lists of pointers. This cost can be reduced by sorting the list of pointers and retrieving records in the sorted order. Thereby, (1) all pointers to records in a block come together, hence all selected records in the block can be retrieved using a single I/O operation, and (2) blocks are read in sorted order, minimizing disk-arm movement.

- **A10 (disjunctive selection by union of identifiers).** If access paths are available on all the conditions of a disjunctive selection, each index is scanned for pointers to tuples that satisfy the individual condition. The union of all the retrieved pointers yields the set of pointers to all tuples that satisfy the disjunctive condition. We then use the pointers to retrieve the actual records.

However, if even one of the conditions does not have an access path, we have to perform a linear scan of the relation to find tuples that satisfy the condition. Therefore, if there is even one such condition in the disjunct, the most efficient access method is a linear scan, with the disjunctive condition tested on each tuple during the scan.

10.5 Sorting

Sorting of data plays an important role in database systems for two reasons. First, SQL queries can specify that the output be sorted. Second, and equally important for query processing, several of the relational operations, such as joins, can be implemented efficiently if the input relations are first sorted.

We can sort a relation by building an index on the sort key, and then using that index to read the relation in sorted order. However, such a process orders the relation only *logically*, through an index, rather than *physically*. Hence, the reading of tuples in the sorted order may lead to a disk access (disk seek plus block transfer) for each record, which can be very expensive, since the number of records can be much larger than the number of blocks.

For this reason, it may be desirable to order the records physically. The problem of sorting has been studied extensively, both for relations that fit entirely in main memory and for relations that are bigger than memory. In the first case, standard sorting techniques such as quick-sort can be used. Here, we discuss how to handle the second case.

10.5.1 External Sort-Merge Algorithm

Sorting of relations that do not fit in memory is called **external sorting**. The most commonly used technique for external sorting is the **external sort-merge** algorithm. We describe the external sort-merge algorithm next. Let M denote the number of blocks in the main-memory buffer available for sorting, that is, the number of disk blocks whose contents can be buffered in available main memory.

1. In the first stage, a number of sorted **runs** are created; each run is sorted, but contains only some of the records of the relation. $i = 0$;

repeat

read M blocks of the relation, or the rest of the relation,

whichever is smaller;

sort the in-memory part of the relation;

write the sorted data to run file R_i ;

$i = i + 1$;

until the end of the relation

2. In the second stage, the runs are *merged*. Suppose, for now, that the total number of runs, N , is less than M , so that we can allocate one block to each run and have space left to hold one block of output. The merge stage operates as follows: read one block of each of the N files R_i into a buffer block in memory;

repeat

choose the first tuple (in sort order) among all buffer blocks;

write the tuple to the output, and delete it from the buffer block;

if the buffer block of any run R_i is empty **and not** end-of-file(R_i)

then read the next block of R_i into the buffer block;

until all input buffer blocks are empty

The output of the merge stage is the sorted relation. The output file is buffered to reduce the number of disk write operations. The preceding merge operation is a generalization of the two-way merge used by the standard in-memory sort–merge algorithm; it merges N runs, so it is called an **N -way merge**.

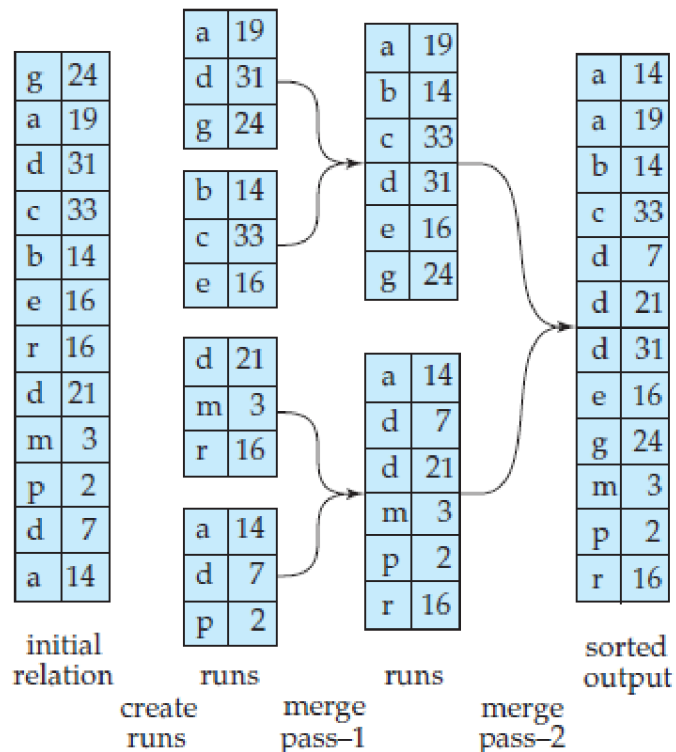


Figure 10.4 External sorting using sort–merge.

In general, if the relation is much larger than memory, there may be M or more runs generated in the first stage, and it is not possible to allocate a block for each run during the merge stage. In this case, the merge operation proceeds in multiple passes. Since there is enough memory for $M-1$ input buffer blocks, each merge can take $M-1$ runs as input. The initial *pass* functions in this way: It merges the first $M-1$ runs (as described in item 2 above) to get a single run for the next pass. Then, it merges the next $M-1$ runs similarly, and so on, until it has processed all the initial runs.

At this point, the number of runs has been reduced by a factor of $M-1$. If this reduced number of runs is still greater than or equal to M , another pass is made, with the runs created by the first pass as input. Each pass reduces the number of runs by a factor of $M-1$. The passes repeat as many times as required, until the number of runs is less than M ; a final pass then generates the sorted output. Figure 10.4 illustrates the steps of the external sort–merge for an example relation. For illustration purposes, we assume that only one

tuple fits in a block ($fr = 1$), and we assume that memory holds at most three blocks. During the merge stage, two blocks are used for input and one for output.

10.5.2 Cost Analysis of External Sort-Merge

We compute the disk-access cost for the external sort-merge in this way: Let br denote the number of blocks containing records of relation r . The first stage reads every block of the relation and writes them out again, giving a total of $2br$ block transfers. The initial number of runs is $\lceil br/M \rceil$. Since the number of runs decreases by a factor of $M-1$ in each merge pass, the total number of merge passes required is $\lceil \log_{M-1}(br/M) \rceil$. Each of these passes reads every block of the relation once and writes it out once, with two exceptions. First, the final pass can produce the sorted output without writing its result to disk. Second, there may be runs that are not read in or written out during a pass—for example, if there are M runs to be merged in a pass, $M-1$ are read in and merged, and one run is not accessed during the pass. Ignoring the (relatively small) savings due to the latter effect, the total number of block transfers for external sorting of the relation is:

$$\left\lceil b_r(2\lceil \log_{M-1}(b_r/M) \rceil + 1) \right\rceil$$

Applying this equation to the example in Figure 10.4, we get a total of $12 * (4+1) = 60$ block transfers, as you can verify from the figure. Note that the above numbers do not include the cost of writing out the final result.

We also need to add the disk-seek costs. Run generation requires seeks for reading data for each of the runs as well as for writing the runs. During the merge phase, if data are read bb blocks at a time from each run (that is, bb buffer blocks are allocated to each run), then each merge pass would require around $\lceil b_r/b_b \rceil$ seeks for reading data.⁴ Although the output is written sequentially, if it is on the same disk as the input runs the head may have moved away between writes of consecutive blocks. Thus we would have to add a total of $2\lceil br/bb \rceil$ seeks for each merge pass, except the final pass (since we assume the final result is not written back to disk). The total number of seeks is then:

$$\left\lceil 2\lceil b_r/M \rceil + \lceil b_r/b_b \rceil(2\lceil \log_{M-1}(b_r/M) \rceil - 1) \right\rceil$$

Applying this equation to the example in Figure 12.4, we get a total of $8 + 12 * (2 * 2 - 1) = 44$ disk seeks if we set the number of buffer blocks per run, bb to 1.

10.6 Join Operation

We study several algorithms for computing the join of relations, and we analyze their respective costs.

We use the term **equi-join** to refer to a join of the form $r \bowtie_{r.A=s.B} s$, where A and B are attributes or sets of attributes of relations r and s , respectively.

We use as a running example the expression:

$student \bowtie takes$

using the same relation schemas. We assume the following information about the two relations:

- Number of records of *student*: $nstudent = 5,000$.
- Number of blocks of *student*: $bstudent = 100$.
- Number of records of *takes*: $ntakes = 10,000$.
- Number of blocks of *takes*: $btakes = 400$.

10.6.1 Nested-Loop Join

Figure 10.5 shows a simple algorithm to compute the theta join, $r \bowtie_{\theta} s$, of two relations r and s . This algorithm is called the **nested-loop join** algorithm, since it basically consists of a pair of nested **for** loops. Relation r is called the **outer relation** and relation s the **inner relation** of the join, since the loop for r encloses the loop for s . The algorithm uses the notation $t_r \cdot t_s$, where t_r and t_s are tuples; $t_r \cdot t_s$ denotes the tuple constructed by concatenating the attribute values of tuples t_r and t_s .

Like the linear file-scan algorithm for selection, the nested-loop join algorithm requires no indices, and it can be used regardless of what the join condition is. Extending the algorithm to compute the natural join is straightforward, since the natural join can be expressed as a theta join followed by elimination of repeated attributes by a projection. The only change required is an extra step of deleting repeated attributes from the tuple $t_r \cdot t_s$, before adding it to the result. The nested-loop join algorithm is expensive, since it examines every pair of tuples in the two relations. Consider the cost of the nested-loop join algorithm. The number of pairs of tuples to be considered is $n_r * n_s$, where n_r denotes the number of tuples in r , and n_s denotes the number of tuples in s . For each record in r , we have to perform a complete scan on s . In the worst case, the buffer can hold only one block of each relation, and a total of $n_r * b_s + b_r$ block transfers would be required, where b_r and b_s denote the number of blocks containing tuples of r and s , respectively. We need only one seek for each scan on the inner relation s since it is read sequentially, and a total of b_r seeks to read r , leading to

a total of $n_r + b_r$ seeks. In the best case, there is enough space for both relations to fit simultaneously in memory, so each block would have to be read only once; hence, only $b_r + b_s$ block transfers would be required, along with 2 seeks.

```

for each tuple  $t_r$  in  $r$  do begin
  for each tuple  $t_s$  in  $s$  do begin
    test pair  $(t_r, t_s)$  to see if they satisfy the join condition  $\theta$ 
    if they do, add  $t_r \cdot t_s$  to the result;
  end
end

```

Figure 10.5 Nested-loop join.

```

for each block  $B_r$  of  $r$  do begin
  for each block  $B_s$  of  $s$  do begin
    for each tuple  $t_r$  in  $B_r$  do begin
      for each tuple  $t_s$  in  $B_s$  do begin
        test pair  $(t_r, t_s)$  to see if they satisfy the join condition
        if they do, add  $t_r \cdot t_s$  to the result;
      end
    end
  end
end

```

Figure 10.6 Block nested-loop join.

If one of the relations fits entirely in main memory, it is beneficial to use that relation as the inner relation, since the inner relation would then be read only once. Therefore, if s is small enough to fit in main memory, our strategy requires only a total $b_r + b_s$ block transfers and 2 seeks—the same cost as that for the case where both relations fit in memory.

Now consider the natural join of *student* and *takes*. Assume for now that we have no indices whatsoever on either relation, and that we are not willing to create any index. We can use the nested loops to compute the join; assume that *student* is the outer relation and *takes* is the inner relation in the join. We will have to examine $5000 * 10,000 = 50 * 10^6$ pairs of tuples. In the worst case, the number of block transfers is $5000 * 400 + 100 = 2,000,100$, plus $5000 + 100 = 5100$ seeks. In the bestcase scenario, however, we can read both relations only once, and perform the computation. This computation requires at most $100 + 400 = 500$ block transfers, plus 2 seeks—a significant improvement over the worst-case scenario. If we had used *takes* as the relation for the outer loop and *student* for the inner loop, the worst-case cost of our final strategy would have been $10,000 * 100 + 400 = 1,000,400$ block transfers, plus 10,400 disk seeks. The number of block transfers is

significantly less, and although the number of seeks is higher, the overall cost is reduced, assuming $tS = 4$ milliseconds and $tT = 0.1$ milliseconds.

10.6.2 Block Nested-Loop Join

If the buffer is too small to hold either relation entirely in memory, we can still obtain a major saving in block accesses if we process the relations on a per-block basis, rather than on a per-tuple basis. Figure 10.6 shows **block nested-loop join**, which is a variant of the nested-loop join where every block of the inner relation is paired with every block of the outer relation. Within each pair of blocks, every tuple in one block is paired with every tuple in the other block, to generate all pairs of tuples. As before, all pairs of tuples that satisfy the join condition are added to the result.

The primary difference in cost between the block nested-loop join and the basic nested-loop join is that, in the worst case, each block in the inner relation s is read only once for each *block* in the outer relation, instead of once for each *tuple* in the outer relation. Thus, in the worst case, there will be a total of $b_r * b_s + b_r$ block transfers, where b_r and b_s denote the number of blocks containing records of r and s , respectively. Each scan of the inner relation requires one seek, and the scan of the outer relation requires one seek per block, leading to a total of $2 * b_r$ seeks. Clearly, it is more efficient to use the smaller relation as the outer relation, in case neither of the relations fits in memory. In the best case, where the inner relation fits in memory, there will be $b_r + b_s$ block transfers and just 2 seeks.

Now return to our example of computing `student ⋈ takes` using the block nested-loop join algorithm. In the worst case, we have to read each block of `takes` once for each block of `student`. Thus, in the worst case, a total of $100 * 400 + 100 = 40,100$ block transfers plus $2 * 100 = 200$ seeks are required. This cost is a significant improvement over the $5000 * 400 + 100 = 2,000,100$ block transfers plus 5100 seeks needed in the worst case for the basic nested-loop join. The best-case cost remains the same—namely, $100 + 400 = 500$ block transfers and 2 seeks. The performance of the nested-loop and block nested-loop procedures can be further improved:

- If the join attributes in a natural join or an equi-join form a key on the inner relation, then for each outer relation tuple the inner loop can terminate as soon as the first match is found.

- In the block nested-loop algorithm, instead of using disk blocks as the blocking unit for the outer relation, we can use the biggest size that can fit in memory, while leaving enough space for the buffers of the inner relation and the output. In other words, if memory has m blocks, we read in $M-2$ blocks of the outer relation at a time, and when we read each block of the inner relation we join it with all the $M-2$ blocks of the outer relation. This change reduces the number of scans of the inner relation from b_r to $\lceil b_r / (M - 2) \rceil$, where b_r is the number of blocks of the outer relation. The total cost is then $\lceil b_r / (M - 2) \rceil * b_s + b_r$ block transfers and $2\lceil b_r / (M - 2) \rceil$ seeks.
- We can scan the inner loop alternately forward and backward. This scanning method orders the requests for disk blocks so that the data remaining in the buffer from the previous scan can be reused, thus reducing the number of disk accesses needed.
- If an index is available on the inner loop's join attribute, we can replace file scans with more efficient index lookups.

10.6.3 Indexed Nested-Loop Join

In a nested-loop join (Figure 10.5), if an index is available on the inner loop's join attribute, index lookups can replace file scans. For each tuple tr in the outer relation r , the index is used to look up tuples in s that will satisfy the join condition with tuple t_r .

This join method is called an **indexed nested-loop join**; it can be used with existing indices, as well as with temporary indices created for the sole purpose of evaluating the join.

Looking up tuples in s that will satisfy the join conditions with a given tuple tr is essentially a selection on s . For example, consider *student* ⋈ *takes*. Suppose that we have a *student* tuple with *ID* "00128". Then, the relevant tuples in *takes* are those that satisfy the selection "*ID* = 00128".

The cost of an indexed nested-loop join can be computed as follows: For each tuple in the outer relation r , a lookup is performed on the index for s , and the relevant tuples are retrieved. In the worst case, there is space in the buffer for only one block of r and one block of the index. Then, b_r I/O operations are needed to read relation r , where b_r denotes the number of blocks containing records of r ; each I/O requires a seek and a block transfer,

since the disk head may have moved in between each I/O. For each tuple in r , we perform an index lookup on s . Then, the time cost of the join can be computed as $br(tT + tS) + nr$ “ c ”, where nr is the number of records in relation r , and c is the cost of a single selection on s using the join condition. How to estimate the cost of a single selection algorithm (possibly using indices); that estimate gives us the value of c .

The cost formula indicates that, if indices are available on both relations r and s , it is generally most efficient to use the one with fewer tuples as the outer relation.

For example, consider an indexed nested-loop join of *student* ⋈ *takes*, with *student* as the outer relation. Suppose also that *takes* has a primary B+-tree index on the join attribute *ID*, which contains 20 entries on average in each index node. Since *takes* has 10,000 tuples, the height of the tree is 4, and one more access is needed to find the actual data. Since *nstudent* is 5000, the total cost is $100 + 5000 * 5 = 25,100$ disk accesses, each of which requires a seek and a block transfer. In contrast, as we saw before, 40,100 block transfers plus 200 seeks were needed for a block nested-loop join. Although the number of block transfers has been reduced, the seek cost has actually increased, increasing the total cost since a seek is considerably more expensive than a block transfer. However, if we had a selection on the *student* relation that reduces the number of rows significantly, indexed nested-loop join could be significantly faster than block nested-loop join.

10.6.4 Merge Join

The **merge-join** algorithm (also called the **sort-merge-join** algorithm) can be used to compute natural joins and equi-joins. Let $r(R)$ and $s(S)$ be the relations whose natural join is to be computed, and let $R \cap S$ denote their common attributes. Suppose that both relations are sorted on the attributes $R \cap S$. Then, their join can be computed by a process much like the merge stage in the merge-sort algorithm.

Merge-Join Algorithm

Figure 10.7 shows the merge-join algorithm. In the algorithm, *JoinAttrs* refers to the attributes in $R \cap S$, and $tr \bowtie ts$, where tr and ts are tuples that have the same values for *JoinAttrs*, denotes the concatenation of the attributes of the tuples, followed by projecting out repeated attributes.

```

pr := address of first tuple of r;
ps := address of first tuple of s;
while (ps ≠ null and pr ≠ null) do
  begin
    ts := tuple to which ps points;
    Ss := {ts};
    set ps to point to next tuple of s;
    done := false;
    while (not done and ps ≠ null) do
      begin
        ts' := tuple to which ps points;
        if (ts'[JoinAttrs] = ts[JoinAttrs])
          then begin
            Ss := Ss ∪ {ts'};
            set ps to point to next tuple of s;
          end
          else done := true;
        end
      tsr := tuple to which pr points;
      while (pr ≠ null and tsr[JoinAttrs] < ts[JoinAttrs]) do
        begin
          set pr to point to next tuple of r;
          tsr := tuple to which pr points;
        end
      while (pr ≠ null and tsr[JoinAttrs] = ts[JoinAttrs]) do
        begin
          for each ts in Ss do
            begin
              add ts ⋈ tsr to result;
            end
          set pr to point to next tuple of r;
          tsr := tuple to which pr points;
        end
      end
    end.

```

Figure 10.7 Merge join.

The merge-join algorithm associates one pointer with each relation. These pointers point initially to the first tuple of the respective relations. As the algorithm proceeds, the pointers move through the relation. A group of tuples of one relation with the same value on the join attributes is read into S_s . The algorithm in Figure 10.7 *requires* that every set of tuples S_s fit in main memory; we discuss extensions of the algorithm to avoid this requirement shortly. Then, the corresponding tuples (if any) of the other relation are read in, and are processed as they are read.

	<i>a1</i>	<i>a2</i>
<i>pr</i> →	a	3
	b	1
	d	8
	d	13
	f	7
	m	5
	q	6
	<i>r</i>	

	<i>a1</i>	<i>a3</i>
<i>ps</i> →	a	A
	b	G
	c	L
	d	N
	m	B
	<i>s</i>	

Figure 10.8 Sorted relations for merge join.

Figure 10.8 shows two relations that are sorted on their join attribute *a1*. It is instructive to go through the steps of the merge-join algorithm on the relations shown in the figure.

The merge-join algorithm of Figure 10.7 requires that each set *S_s* of all tuples with the same value for the join attributes must fit in main memory. This requirement can usually be met, even if the relation *s* is large. If there are some join attribute values for which *S_s* is larger than available memory, a block nested-loop join can be performed for such sets *S_s*, matching them with corresponding blocks of tuples in *r* with the same values for the join attributes. If either of the input relations *r* and *s* is not sorted on the join attributes, they can be sorted first, and then the merge-join algorithm can be used. The merge-join algorithm can also be easily extended from natural joins to the more general case of equi-joins.

10.6.5 Hash Join

Like the merge-join algorithm, the hash-join algorithm can be used to implement natural joins and equi-joins. In the hash-join algorithm, a hash function *h* is used to partition tuples of both relations. The basic idea is to partition the tuples of each of the relations into sets that have the same hash value on the join attributes.

We assume that:

- *h* is a hash function mapping *JoinAttrs* values to $\{0, 1, \dots, nh\}$, where *JoinAttrs* denotes the common attributes of *r* and *s* used in the natural join.
- $H_{r0}, H_{r1}, \dots, H_{rnh}$ denote partitions of *r* tuples, each initially empty. Each tuple $t_r \in r$ is put in partition H_{ri} , where $i = h(tr[JoinAttrs])$.

- $H_{s0}, H_{s1}, \dots, H_{snh}$ denote partitions of s tuples, each initially empty. Each tuple $t_s \in s$ is put in partition si , where $i = h(ts[JoinAttrs])$.

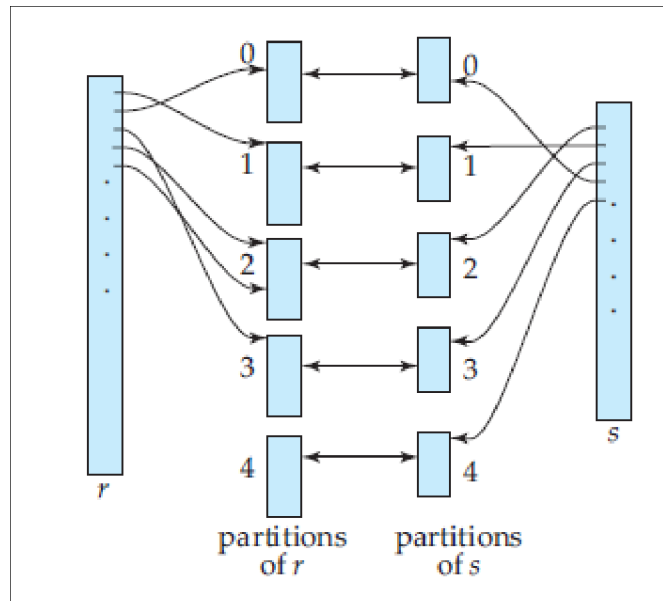


Figure 10.9 Hash partitioning of relations.

The hash function h should have the “goodness” properties of randomness and uniformity that we already discussed.

10.6.5.1 Basics

The idea behind the hash-join algorithm is this: Suppose that an r tuple and an s tuple satisfy the join condition; then, they have the same value for the join attributes. If that value is hashed to some value i , the r tuple has to be in ri and the s tuple in si . Therefore, r tuples in ri need only to be compared with s tuples in si ; they do not need to be compared with s tuples in any other partition. For example, if d is a tuple in *student*, c a tuple in *takes*, and h a hash function on the *ID* attributes of the tuples, then d and c must be tested only if $h(c) = h(d)$. If $h(c) = h(d)$, then c and d must have different values for *ID*. However, if $h(c) = h(d)$, we must test c and d to see whether the values in their join attributes are the same, since it is possible that c and d have different *iids* that have the same hash value.

Figure 10.10 shows the details of the **hash-join** algorithm to compute the natural join of relations r and s . As in the merge-join algorithm, $t_r \bowtie t_s$ denotes the concatenation of the attributes of tuples tr and ts , followed by projecting out repeated attributes. After the partitioning of the relations, the rest of the hash-join code performs a separate indexed nested-loop join on each of the partition pairs i , for $i = 0, \dots, nh$. To do so, it first **builds** a hash index on each si , and then **probes** (that is, looks up si) with tuples from ri . The relation s is the **build input**, and r is the **probe input**.

The hash index on s_i is built in memory, so there is no need to access the disk to retrieve the tuples. The hash function used to build this hash index must be different from the hash function h used earlier, but is still applied to only the join attributes.

```

/* Partition  $s$  */
for each tuple  $t_s$  in  $s$  do begin
     $i := h(t_s[JoinAttrs]);$ 
     $H_{s_i} := H_{s_i} \cup \{t_s\};$ 
end
/* Partition  $r$  */
for each tuple  $t_r$  in  $r$  do begin
     $i := h(t_r[JoinAttrs]);$ 
     $H_{r_i} := H_{r_i} \cup \{t_r\};$ 
end
/* Perform join on each partition */
for  $i := 0$  to  $n_h$  do begin
    read  $H_{s_i}$  and build an in-memory hash index on it;
    for each tuple  $t_r$  in  $H_{r_i}$  do begin
        probe the hash index on  $H_{s_i}$  to locate all tuples  $t_s$ 
        such that  $t_s[JoinAttrs] = t_r[JoinAttrs];$ 
        for each matching tuple  $t_s$  in  $H_{s_i}$  do begin
            add  $t_r \bowtie t_s$  to the result;
        end
    end
end
end

```

Figure 10.10 Hash join.

In the course of the indexed nested-loop join, the system uses this hash index to retrieve records that match records in the probe input. The build and probe phases require only a single pass through both the build and probe inputs. It is straightforward to extend the hash-join algorithm to compute general equi-joins.

The value nh must be chosen to be large enough such that, for each i , the tuples in the partition s_i of the build relation, along with the hash index on the partition, fit in memory. It is not necessary for the partitions of the probe relation to fit in memory. Clearly, it is best to use the smaller input relation as the build relation. If the size of the build relation is bs blocks, then, for each of the nh partitions to be of size less than or equal to M , nh must be at least $\lceil bs/M \rceil$. More precisely stated, we have to account for the extra space occupied by the hash index on the partition as well, so nh should be correspondingly larger. For simplicity, we sometimes ignore the space requirement of the hash index in our analysis.

10.6.5.2 Recursive Partitioning

If the value of nh is greater than or equal to the number of blocks of memory, the relations cannot be partitioned in one pass, since there will not be enough buffer blocks.

Instead, partitioning has to be done in repeated passes. In one pass, the input can be split into at most as many partitions as there are blocks available for use as output buffers. Each bucket generated by one pass is separately read in and partitioned again in the next pass, to create smaller partitions. The hash function used in a pass is, of course, different from the one used in the previous pass. The system repeats this splitting of the input until each partition of the build input fits in memory. Such partitioning is called **recursive partitioning**.

A relation does not need recursive partitioning if $M > n_h + 1$, or equivalently $M > (bs/M) + 1$, which simplifies (approximately) to $M > \sqrt{b_s}$. For example, consider a memory size of 12 megabytes, divided into 4-kilobyte blocks; it would contain a total of 3K (3072) blocks. We can use a memory of this size to partition relations of size up to 3K * 3K blocks, which is 36 gigabytes. Similarly, a relation of size 1 gigabyte requires just over 256K blocks, or 2 megabytes, to avoid recursive partitioning.

10.6.5.3 Handling of Overflows

Hash-table overflow occurs in partition i of the build relation s if the hash index on si is larger than main memory. Hash-table overflow can occur if there are many tuples in the build relation with the same values for the join attributes, or if the hash function does not have the properties of randomness and uniformity. In either case, some of the partitions will have more tuples than the average, whereas others will have fewer; partitioning is then said to be **skewed**.

We can handle a small amount of skew by increasing the number of partitions so that the expected size of each partition (including the hash index on the partition) is somewhat less than the size of memory. The number of partitions is therefore increased by a small value, called the **fudge factor**, that is usually about 20 percent of the number of hash partitions computed.

Even if, by using a fudge factor, we are conservative on the sizes of the partitions, overflows can still occur. Hash-table overflows can be handled by either *overflow resolution* or *overflow avoidance*. **Overflow resolution** is performed during the build phase, if a hash-index overflow is detected. Overflow resolution proceeds in this way: If si , for any i , is found to be too large, it is further partitioned into smaller partitions by using a different hash

function. Similarly, r_i is also partitioned using the new hash function, and only tuples in the matching partitions need to be joined.

In contrast, **overflow avoidance** performs the partitioning carefully, so that overflows never occur during the build phase. In overflow avoidance, the build relation s is initially partitioned into many small partitions, and then some partitions are combined in such a way that each combined partition fits in memory.

The probe relation r is partitioned in the same way as the combined partitions on s , but the sizes of r_i do not matter.

If a large number of tuples in s have the same value for the join attributes, the resolution and avoidance techniques may fail on some partitions. In that case, instead of creating an in-memory hash index and using a nested-loop join to join the partitions, we can use other join techniques, such as block nested-loop join, on those partitions.

10.7 Query Optimization

Query optimization is the process of selecting the most efficient query-evaluation plan from among the many strategies usually possible for processing a given query, especially if the query is complex. We do not expect users to write their queries so that they can be processed efficiently. Rather, we expect the system to construct a query-evaluation plan that minimizes the cost of query evaluation. This is where query optimization comes into play.

One aspect of optimization occurs at the relational-algebra level, where the system attempts to find an expression that is equivalent to the given expression, but more efficient to execute. Another aspect is selecting a detailed strategy for processing the query, such as choosing the algorithm to use for executing an operation, choosing the specific indices to use, and so on.

The difference in cost (in terms of evaluation time) between a good strategy and a bad strategy is often substantial, and may be several orders of magnitude. Hence, it is worthwhile for the system to spend a substantial amount of time on the selection of a good strategy for processing a query, even if the query is executed only once.

10.7.1 Overview

Consider the following relational-algebra expression, for the query “Find the names of all instructors in the Music department together with the course title of all the courses that the instructors teach.

$\Pi_{name, title} (\sigma_{dept_name = \text{"Music"}} (instructor \bowtie (teaches \bowtie \Pi_{course_id, title} (course))))$

Note that the projection of *course* on $(course_id, title)$ is required since *course* shares an attribute *dept name* with *instructor*; if we did not remove this attribute using the projection, the above expression using natural joins would return only courses from the Music department, even if some Music department instructors taught courses in other departments.

The above expression constructs a large intermediate relation,

$instructor \bowtie teaches \bowtie (course_id, title (course)).$

However, we are interested in only a few tuples

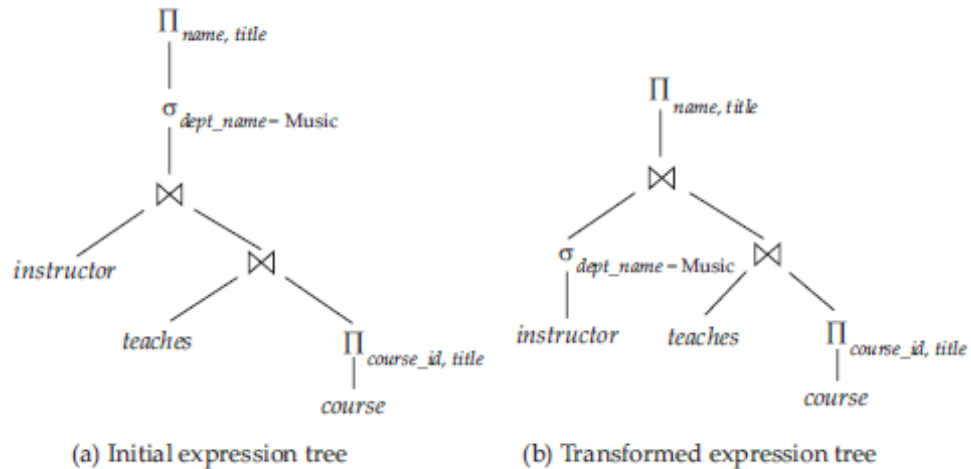


Figure 10.11 Equivalent expressions.

of this relation (those pertaining to instructors in the Music department), and in only two of the ten attributes of this relation. Since we are concerned with only those tuples in the *instructor* relation that pertain to the Music department, we do not need to consider those tuples that do not have *dept name* = "Music". By reducing the number of tuples of the *instructor* relation that we need to access, we reduce the size of the intermediate result. Our query is now represented by the relational-algebra expression:

$\Pi_{name, title} ((\sigma_{dept_name = \text{"Music"}} (instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title} (course)))$

which is equivalent to our original algebra expression, but which generates smaller intermediate relations. Figure 10.11 depicts the initial and transformed expressions.

An evaluation plan defines exactly what algorithm should be used for each operation, and how the execution of the operations should be coordinated. Figure 10.12 illustrates

one possible evaluation plan for the expression from Figure 10.11(b). As we have seen, several different algorithms can be used for each relational operation, giving rise to alternative evaluation plans. In the figure, hash join has been chosen for one of the join operations, while the other uses merge join, after sorting the relations on the join attribute, which is ID. Where edges are marked as pipelined, the output of the producer is pipelined directly to the consumer, without being written out to disk.

Given a relational-algebra expression, it is the job of the query optimizer to come up with a query-evaluation plan that computes the same result as the given expression, and is the least-costly way of generating the result (or, at least, is not much costlier than the least-costly way).

To find the least-costly query-evaluation plan, the optimizer needs to generate alternative plans that produce the same result as the given expression, and to choose the least-costly one. Generation of query-evaluation plans involves three steps: (1) generating expressions that are logically equivalent.

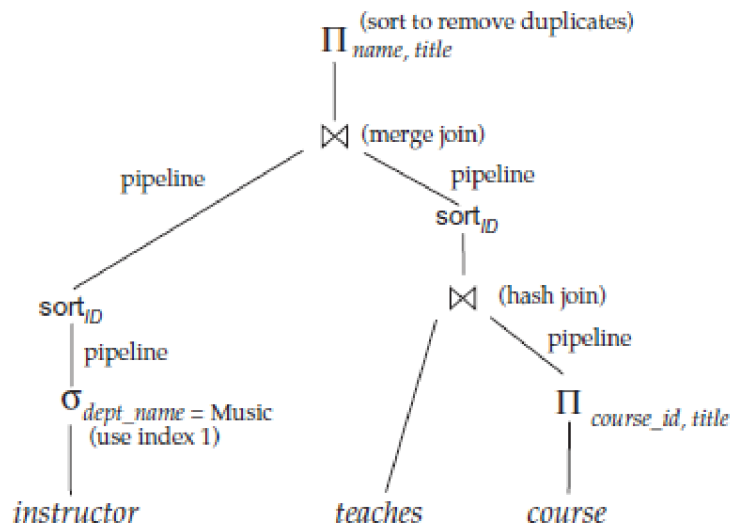


Figure 10.12 An evaluation plan.

pression, (2) annotating the resultant expressions in alternative ways to generate alternative query-evaluation plans, and (3) estimating the cost of each evaluation plan, and choosing the one whose estimated cost is the least.

Steps (1), (2), and (3) are interleaved in the query optimizer—some expressions are generated, and annotated to generate evaluation plans, then further expressions are generated and annotated, and so on. As evaluation plans are generated, their costs are

estimated by using statistical information about the relations, such as relation sizes and index depths.

To implement the first step, the query optimizer must generate expressions equivalent to a given expression. It does so by means of *equivalence rules* that specify how to transform an expression into a logically equivalent one.

We describe how to estimate statistics of the results of each operation in a query plan. The individual costs are combined to determine the estimated cost of evaluating a given relational-algebra expression.

We describe how to choose a query-evaluation plan. We can choose one based on the estimated cost of the plans. Since the cost is an estimate, the selected plan is not necessarily the least-costly plan; however, as long as the estimates are good, the plan is likely to be the least-costly one, or not much more costly than it.

Finally, materialized views help to speed up processing of certain queries. We study how to “maintain” materialized views—that is, to keep them up-to-date—and how to perform query optimization with materialized views.

10.7.2 Choice of Evaluation Plans

Generation of expressions is only part of the query-optimization process, since each operation in the expression can be implemented with different algorithms. An evaluation plan defines exactly what algorithm should be used for each operation and how the execution of the operations should be coordinated.

A cost-based optimizer explores the space of all query-evaluation plans that are equivalent to the given query, and chooses the one with the least estimated cost. We have seen how equivalence rules can be used to generate equivalent plans. However, cost-based optimization with arbitrary equivalence rules is fairly complicated. We first cover a simpler version of cost-based optimization, which involves only join-order and join algorithm selection, we briefly sketch how a general-purpose optimizer based on equivalence rules can be built, without going into details.

Exploring the space of all possible plans may be too expensive for complex queries. Most optimizers include heuristics to reduce the cost of query optimization, at the potential risk of not finding the optimal plan

10.7.2.1 Cost-Based Join Order Selection

The most common type of query in SQL consists of a join of a few relations, with join predicates and selections specified in the where clause. In this section we consider the problem of choosing the optimal join order for such a query. For a complex join query, the number of different query plans that are equivalent to the query can be large. As an illustration, consider the expression:

$$r_1 \bowtie r_2 \bowtie \dots \bowtie r_n$$

where the joins are expressed without any ordering. With $n = 3$, there are 12 different join orderings:

$$\begin{array}{cccc} r_1 \bowtie (r_2 \bowtie r_3) & r_1 \bowtie (r_3 \bowtie r_2) & (r_2 \bowtie r_3) \bowtie r_1 & (r_3 \bowtie r_2) \bowtie r_1 \\ r_2 \bowtie (r_1 \bowtie r_3) & r_2 \bowtie (r_3 \bowtie r_1) & (r_1 \bowtie r_3) \bowtie r_2 & (r_3 \bowtie r_1) \bowtie r_2 \\ r_3 \bowtie (r_1 \bowtie r_2) & r_3 \bowtie (r_2 \bowtie r_1) & (r_1 \bowtie r_2) \bowtie r_3 & (r_2 \bowtie r_1) \bowtie r_3 \end{array}$$

In general, with n relations, there are $(2(n-1))/(n-1)!$ different join orders. For joins involving small numbers of relations, this number is acceptable; for example, with $n = 5$, the number is 1680. However, as n increases, this number rises quickly. With $n = 7$, the number is 665,280; with $n = 10$, the number is greater than 17.6 billion!

Luckily, it is not necessary to generate all the expressions equivalent to a given expression. For example, suppose we want to find the best join order of the form:

$$(r_1 \bowtie r_2 \bowtie r_3) \bowtie r_4 \bowtie r_5$$

which represents all join orders where r_1 , r_2 , and r_3 are joined first (in some order), and the result is joined (in some order) with r_4 and r_5 . There are 12 different join orders for computing $r_1 \bowtie r_2 \bowtie r_3$, and 12 orders for computing the join of this result with r_4 and r_5 . Thus, there appear to be 144 join orders to examine. However, once we have found the best join order for the subset of relations $\{r_1, r_2, r_3\}$, we can use that order for further joins with r_4 and r_5 , and can ignore all costlier join orders of $r_1 \bowtie r_2 \bowtie r_3$. Thus, instead of 144 choices to examine, we need to examine only $12 + 12$ choices.

Using this idea, we can develop a *dynamic-programming* algorithm for finding optimal join orders. Dynamic-programming algorithms store results of computations and reuse them, a procedure that can reduce execution time greatly.

A recursive procedure implementing the dynamic-programming algorithm. The procedure applies selections on individual relations at the earliest possible point, that is, when the relations are accessed. It is easiest to understand the procedure assuming that all joins are natural joins, although the procedure works unchanged with any join condition. With arbitrary join conditions, the join of two subexpressions is understood to include all join conditions that relate attributes from the two subexpressions.

```

procedure FindBestPlan(S)
  if (bestplan[S].cost  $\neq$  ") /* bestplan[S] already computed */
    return bestplan[S]
  if (S contains only 1 relation)
    set bestplan[S]. plan and bestplan[S].cost based on best way of accessing S
  else for each non-empty subset S1 of S such that S1  $\cap$  S = S
    P1 = FindBestPlan(S1)
    P2 = FindBestPlan(S  $\setminus$  S1)
    A = best algorithm for joining results of P1 and P2
    cost = P1.cost + P2.cost + cost of A
    if cost < bestplan[S].cost
      bestplan[S].cost = cost
      bestplan[S]. plan = "execute P1. plan; execute P2. plan;"
      join results of P1 and P2 using A"
  return bestplan[S]

```

Figure 10.13 Dynamic-programming algorithm for join order optimization.

The procedure stores the evaluation plans it computes in an associative array *bestplan*, which is indexed by sets of relations. Each element of the associative array contains two components: the cost of the best plan of S, and the plan itself. The value of *bestplan*[S].*cost* is assumed to be initialized to "if *bestplan*[S] has not yet been computed.

The procedure first checks if the best plan for computing the join of the given set of relations S has been computed already (and stored in the associative array *bestplan*); if so, it returns the already computed plan. If S contains only one relation, the best way of

accessing S (taking selections on S , if any, into account) is recorded in *bestplan*. This may involve using an index to identify tuples, and then fetching the tuples (often referred to as an *index scan*), or scanning the entire relation (often referred to as a *relation scan*).¹ If there is any selection condition on S , other than those ensured by an index scan, a selection operation is added to the plan, to ensure all selections on S are satisfied.

Otherwise, if S contains more than one relation, the procedure tries every way of dividing S into two disjoint subsets. For each division, the procedure recursively finds the best plans for each of the two subsets, and then computes the cost of the overall plan by using that division.

Actually, the order in which tuples are generated by the join of a set of relations is also important for finding the best overall join order, since it can affect the cost of further joins (for instance, if merge join is used). A particular sort order of the tuples is said to be an interesting sort order if it could be useful for a later operation. For instance, generating the result of $r_1 \bowtie r_2 \bowtie r_3$ sorted on the attributes common with r_4 or r_5 may be useful, but generating it sorted on the attributes common to only r_1 and r_2 is not useful. Using merge join for computing $r_1 \bowtie r_2 \bowtie r_3$ may be costlier than using some other join technique, but it may provide an output sorted in an interesting sort order.

Hence, it is not sufficient to find the best join order for each subset of the set of n given relations. Instead, we have to find the best join order for each subset, for each interesting sort order of the join result for that subset. The number of subsets of n relations is 2^n . The number of interesting sort orders is generally not large.

Thus, about 2^n join expressions need to be stored. The dynamic-programming algorithm for finding the best join order can be easily extended to handle sort orders. The cost of the extended algorithm depends on the number of interesting orders for each subset of relations; since this number has been found to be small in practice, the cost remains at $O(3^n)$. With $n = 10$, this number is around 59,000, which is much better than the 17.6 billion different join orders. More important, the storage required is much less than before, since we need to store only one join order for each interesting sort order of each of 1024 subsets of $r_1, \dots, r_{10}$. Although both numbers still increase rapidly with n , commonly occurring joins usually have less than 10 relations, and can be handled easily.

10.7.2.2 Cost-Based Optimization with Equivalence Rules

The join order optimization technique we just saw handles the most common class of queries, which perform an inner join of a set of relations. However, clearly many queries use other features, such as aggregation, outer join, and nested queries, which are not addressed by join order selection.

Many optimizers follow an approach based on using heuristic transformations to handle constructs other than joins, and applying the cost-based join order selection algorithm to subexpressions involving only joins and selections. Details of such heuristics are for the most part specific to individual optimizers, and we do not cover them. However, heuristic transformations to handle nested queries are widely used.

In this section, however, we outline how to create a general-purpose cost-based optimizer based on equivalence rules, which can handle a wide variety of query constructs.

The benefit of using equivalence rules is that it is easy to extend the optimizer with new rules to handle different query constructs. For example, nested queries can be represented using extended relational-algebra constructs, and transformations of nested queries can be expressed as equivalence rules. We have already seen equivalence rules with aggregation operations, and equivalence rules can also be created for outer joins.

The procedure for generating equivalent expressions can be modified to generate all possible evaluation plans as follows: A new class of equivalence rules, called physical equivalence rules, is added that allows a logical operation, such as a join, to be transformed to a physical operation, such as a hash join, or a nested-loops join. By adding such rules to the original set of equivalence rules, the procedure can generate all possible evaluation plans. The cost estimation techniques we have seen earlier can then be used to choose the optimal (that is, the least-cost) plan.

1. A space-efficient representation of expressions that avoids making multiple copies of the same subexpressions when equivalence rules are applied.
2. Efficient techniques for detecting duplicate derivations of the same expression.
3. A form of dynamic programming based on memoization, which stores the optimal query evaluation plan for a subexpression when it is optimized for the first time; subsequent requests to optimize the same subexpression are handled by returning the already memoized plan.

4. Techniques that avoid generating all possible equivalent plans, by keeping track of the cheapest plan generated for any subexpression up to any point of time, and pruning away any plan that is more expensive than the cheapest plan found so far for that subexpression.

The details are more complex than we wish to deal with here. This approach was pioneered by the Volcano research project, and the query optimizer of SQL Server is based on this approach. See the bibliographical notes for references containing further information.

10.7.2.3 Heuristics in Optimization

A drawback of cost-based optimization is the cost of optimization itself. Although the cost of query optimization can be reduced by clever algorithms, the number of different evaluation plans for a query can be very large, and finding the optimal plan from this set requires a lot of computational effort. Hence, optimizers use heuristics to reduce the cost of optimization.

An example of a heuristic rule is the following rule for transforming relational algebra queries:

- Perform selection operations as early as possible.

A heuristic optimizer would use this rule without finding out whether the cost is reduced by this transformation.

We say that the preceding rule is a heuristic because it usually, but not always, helps to reduce the cost. For an example of where it can result in an increase in cost, consider an expression $\sigma_{\theta}(r \bowtie s)$, where the condition θ refers to only attributes in s . The selection can certainly be performed before the join. However, if r is extremely small compared to s , and if there is an index on the join attributes of s , but no index on the attributes used by θ , then it is probably a bad idea to perform the selection early. Performing the selection early that is, directly on s would require doing a scan of all tuples in s . It is probably cheaper, in this case, to compute the join by using the index, and then to reject tuples that fail the selection.

The projection operation, like the selection operation, reduces the size of relations. Thus, whenever we need to generate a temporary relation, it is advantageous to apply immediately any projections that are possible. This advantage suggests a companion to the “perform selections early” heuristic:

- Perform projections early.

It is usually better to perform selections earlier than projections, since selections have the potential to reduce the sizes of relations greatly, and selections enable the use of indices to access tuples. An example similar to the one used for the selection heuristic should convince you that this heuristic does not always reduce the cost.

Most practical query optimizers have further heuristics to reduce the cost of optimization. For example, many query optimizers, such as the System R optimizer, ³ do not consider all join orders, but rather restrict the search to particular kinds of join orders. The System R optimizer considers only those join orders where the right operand of each join is one of the initial relations $r_1, \dots, r_n$. Such join orders are called left-deep join orders. Left-deep join orders are particularly convenient for pipelined evaluation, since the right operand is a stored relation, and thus only one input to each join is pipelined.

Figure 10.14 illustrates the difference between left-deep join trees and non-left-deep join trees. The time it takes to consider all left-deep join orders is $O(n!)$, which is much less than the time to consider all join orders. With the use of dynamic programming optimizations, the System R optimizer can find the best join order in time $O(n2^n)$. Contrast this cost with the $O(3^n)$ time required to find the best overall join order. The System R optimizer uses heuristics to push selections and projections down the query tree.

A heuristic approach to reduce the cost of join-order selection, which was originally used in some versions of Oracle, works roughly this way: For an n -way join, it considers n evaluation plans. Each plan uses a left-deep join order, starting

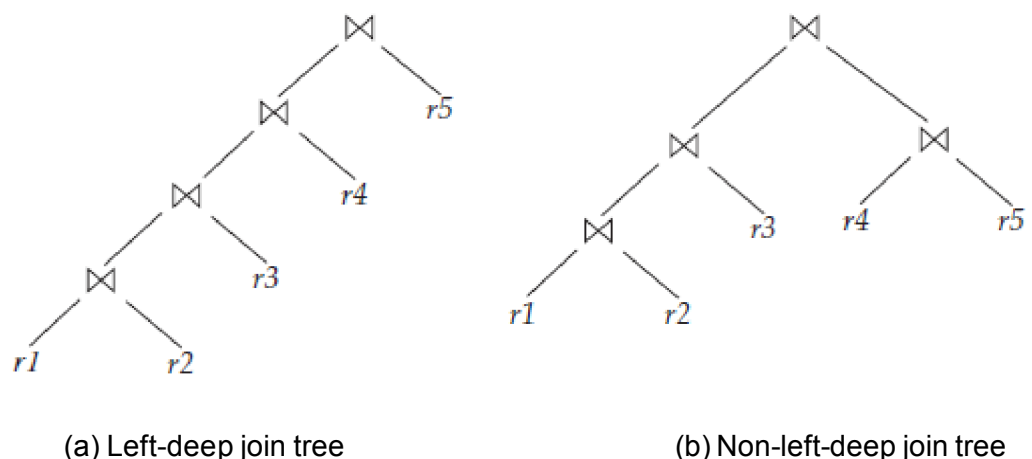


Figure 10.14 Left-deep join trees.

with a different one of the n relations. The heuristic constructs the join order for each of the n evaluation plans by repeatedly selecting the “best” relation to join next, on the basis

of a ranking of the available access paths. Either nested-loop or sort-merge join is chosen for each of the joins, depending on the available access paths. Finally, the heuristic chooses one of the n evaluation plans in a heuristic manner, on the basis of minimizing the number of nested-loop joins that do not have an index available on the inner relation and on the number of sort-merge joins.

Query-optimization approaches that apply heuristic plan choices for some parts of the query, with cost-based choice based on generation of alternative access plans on other parts of the query, have been adopted in several systems.

The approach used in System R and in its successor, the Starburst project, is a hierarchical procedure based on the nested-block concept of SQL. The costbased optimization techniques described here are used for each block of the query separately. The optimizers in several database products, such as IBM DB2 and Oracle, are based on the above approach, with extensions to handle other operations such as aggregation. For compound SQL queries (using the $\cup$, $\cap$, or $-$ operation), the optimizer processes each component separately, and combines the evaluation plans to form the overall evaluation plan.

Even with the use of heuristics, cost-based query optimization imposes a substantial overhead on query processing. However, the added cost of cost-based query optimization is usually more than offset by the saving at query-execution time, which is dominated by slow disk accesses. The difference in execution time between a good plan and a bad one may be huge, making query optimization essential. The achieved saving is magnified in those applications that run on a regular basis, where a query can be optimized once, and the selected query plan can be used each time the query is executed. Therefore, most commercial systems include relatively sophisticated optimizers. The bibliographical notes give references to descriptions of the query optimizers of actual database systems.

10.7.2.4 Optimizing Nested Subqueries

SQL conceptually treats nested subqueries in the where clause as functions that take parameters and return either a single value or a set of values (possibly an empty set). The parameters are the variables from an outer level query that are used in the nested subquery (these variables are called correlation variables). For instance, suppose we have the following query, to find the names of all instructors who taught a course in 2007:

```
select name from instructor where exists (select * from teaches where instructor.ID
= teaches.ID and teaches.year = 2007);
```

Conceptually, the subquery can be viewed as a function that takes a parameter (here, *instructor.ID*) and returns the set of all courses taught in 2007 by instructors (with the same *ID*).

SQL evaluates the overall query (conceptually) by computing the Cartesian product of the relations in the outer from clause and then testing the predicates in the where clause for each tuple in the product. In the preceding example, the predicate tests if the result of the subquery evaluation is empty.

This technique for evaluating a query with a nested subquery is called correlated evaluation. Correlated evaluation is not very efficient, since the subquery is separately evaluated for each tuple in the outer level query. A large number of random disk I/O operations may result.

SQL optimizers therefore attempt to transform nested subqueries into joins, where possible. Efficient join algorithms help avoid expensive random I/O. Where the transformation is not possible, the optimizer keeps the subqueries as separate expressions, optimizes them separately, and then evaluates them by correlated evaluation.

As an example of transforming a nested subquery into a join, the query in the preceding example can be rewritten as:

```
select name from instructor, teaches where instructor.ID = teaches.ID and
teaches.year = 2007;
```

(To properly reflect SQL semantics, the number of duplicate derivations should not change because of the rewriting; the rewritten query can be modified to ensure this property, as we shall see shortly.)

In the example, the nested subquery was very simple. In general, it may not be possible to directly move the nested subquery relations into the from clause of the outer query. Instead, we create a temporary relation that contains the results of the nested query *without* the selections using correlation variables from the outer query, and join the temporary table with the outer level query. For instance, a query of the form:

```
select . . .
from L1
where P1 and exists (select *
from L2
```

where $P2$);

where $P2$ is a conjunction of simpler predicates, can be rewritten as:

create table t_1 as

select distinct V

from $L2$

where $P1$

2 ;

select . . .

from $L1, t_1$

where $P1$ and $P2$

2 ;

where $P1$

2 contains predicates in $P2$ without selections involving correlation variables,

and $P2$

2 reintroduces the selections involving correlation variables (with

relations referenced in the predicate appropriately renamed). Here, V contains all attributes that are used in selections with correlation variables in the nested subquery.

In our example, the original query would have been transformed to:

```
create table  $t_1$  as
  select distinct  $ID$ 
  from  $teaches$ 
  where  $year = 2007$ ;
select  $name$ 
from  $instructor, t_1$ 
where  $t_1.ID = instructor.ID$ ;
```

The query we rewrote to illustrate creation of a temporary relation can be obtained by simplifying the above transformed query, assuming the number of duplicates of each tuple does not matter.

The process of replacing a nested query by a query with a join (possibly with a temporary relation) is called decorrelation.

Decorrelation is more complicated when the nested subquery uses aggregation, or when the result of the nested subquery is used to test for equality, or when the condition linking the nested subquery to the outer query is not exists, and so on. We do not attempt to give algorithms for the general case, and instead refer you to relevant items in the bibliographical notes.

Optimization of complex nested subqueries is a difficult task, as you can infer from the above discussion, and many optimizers do only a limited amount of decorrelation. It is best to avoid using complex nested subqueries, where possible, since we cannot be sure that the query optimizer will succeed in converting them to a form that can be evaluated efficiently.

10.8 Summary

- The first action that the system must perform on a query is to translate the query into its internal form, which (for relational database systems) is usually based on the relational algebra.
- We can process simple selection operations by performing a linear scan, or by making use of indices
- We can sort relations larger than memory by the external sort–merge algorithm.
- Queries involving a natural join may be processed in several ways, depending on the availability of indices and the form of physical storage for the relations.
 - * If the join result is almost as large as the Cartesian product of the two relations, a *block nested-loop* join strategy may be advantageous.
- Hashing and sorting are dual, in the sense that any operation such as duplicate elimination, projection, aggregation, join, and outer join that can be implemented by hashing can also be implemented by sorting, and vice versa; that is, any operation that can be implemented by sorting can also be implemented by hashing.
- Given a query, there are generally a variety of methods for computing the answer. It is the responsibility of the system to transform the query as entered by the user into an equivalent query that can be computed more efficiently. The process of finding a good strategy for processing a query is called query optimization.

- The evaluation of complex queries involves many accesses to disk. Since the transfer of data from disk is slow relative to the speed of main memory and the CPU of the computer system, it is worthwhile to allocate a considerable amount of processing to choose a method that minimizes disk accesses.
- There are a number of equivalence rules that we can use to transform an expression into an equivalent one. We use these rules to generate systematically all expressions equivalent to the given query.
- Each relational-algebra expression represents a particular sequence of operations. The first step in selecting a query-processing strategy is to find a relational-algebra expression that is equivalent to the given expression and is estimated to cost less to execute.
- The strategy that the database system chooses for evaluating an operation depends on the size of each relation and on the distribution of values within columns. So that they can base the strategy choice on reliable information, database systems may store statistics for each relation r . These statistics include:
 - * The number of tuples in the relation r .
 - * The size of a record (tuple) of relation r in bytes.
 - * The number of distinct values that appear in the relation r for a particular attribute.
- Most database systems use histograms to store the number of values for an attribute within each of several ranges of values. Histograms are often computed using sampling.
- These statistics allow us to estimate the sizes of the results of various operations, as well as the cost of executing the operations. Statistical information about relations is particularly useful when several indices are available to assist in the processing of a query. The presence of these structures has a significant influence on the choice of a query-processing strategy.
- Alternative evaluation plans for each expression can be generated by equivalence rules, and the cheapest plan across all expressions can be chosen. Several

optimization techniques are available to reduce the number of alternative expressions and plans that need to be generated.

- We use heuristics to reduce the number of plans considered, and thereby to reduce the cost of optimization. Heuristic rules for transforming relational algebra queries include “Perform selection operations as early as possible,”
“Perform projections early,” and “Avoid Cartesian products.”
- Materialized views can be used to speed up query processing. Incremental view maintenance is needed to efficiently update materialized views when the underlying relations are modified. The differential of an operation can be computed by means of algebraic expressions involving differentials of the inputs of the operation. Other issues related to materialized views include how to optimize queries by making use of available materialized views, and how to select views to be materialized.
- A number of advanced optimization techniques have been proposed such as top-K optimization, join minimization, optimization of updates, multiquery optimization, and parametric query optimization.

10.9 Lesson End Activities

Consider the issue of interesting orders in optimization. Suppose you are given a query that computes the natural join of a set of relations S . Given a subset S_1 of S , what are the interesting orders of S_1 ?

Describe how to incrementally maintain the results of the following operations, on both insertions and deletions:

- Union and set difference.
- Left outer join

10.10 Keywords

- Query optimization
- Transformation of expressions
- Equivalence of expressions

- Equivalence rules
 - * Join commutativity
 - * Join associativity
- Minimal set of equivalence rules
- Enumeration of equivalent expressions
- Statistics estimation
- Catalog information
- Size estimation
 - * Selection
 - * Selectivity
 - * Join
- Query processing
- Evaluation primitive
- Query-execution plan
- Query-evaluation plan
- Query-execution engine
- Measures of query cost
- Sequential I/O
- Random I/O
- File scan
- Linear search
- Selections using indices
- Access paths
- Index scans

- Conjunctive selection
- Disjunctive selection
- Composite index
- Intersection of identifiers
- External sorting
- External sort–merge
- Runs
- *N*-way merge
- Equi-join
- Nested-loop join
- Block nested-loop join
- Indexed nested-loop join
- Merge join
- Sort-merge join
- Hybrid merge join
- Hash join
 - * Build
 - * Probe
 - * Build input
 - * Probe input
 - * Recursive partitioning
 - * Hash-table overflow
 - * Skew
 - * Fudge factor

- * Overflow resolution
 - * Overflow avoidance
- Hybrid hash join
- Operator tree
- Materialized evaluation
- Double buffering
- Pipelined evaluation
 - * Demand-driven pipeline (lazy, pulling)
 - * Producer-driven pipeline (eager, pushing)
 - * Iterator
- Double-pipelined join

10.11 Questions for Discussion

1. Discuss the measure of query cost.
2. Explain different selection operation.
3. Describe the joining operation and its types.
4. Explain hash join with example.
5. Explain query optimization in detail with example.

10.12 Suggested Reading

“Database System Concepts” -5th Edition by Silberchatz. Korth. Sudarshan., TMH.

Lesson - 11

TRANSACTION MANAGEMENT

CONTENTS

11.0 Aim and Objectives

11.1 Introduction

11.2 Transaction Management

11.2.1 Transaction Concepts

11.2.2 Transaction Atomicity and Durability

11.2.3 Implementation of Atomicity and Durability

11.2.4 Serializability

11.2.4.1 Conflict serializability

11.2.4.2 View serializability

11.3 Recoverability & Schedule

11.4 Summary

11.5 Lesson End Activities

11.6 Keywords

11.7 Questions for Discussion

11.8 Suggested Reading

11.0 AIM AND OBJECTIVES

After studying this lesson, you should be able to know:

- Basic concepts of transaction processing.
- Details on concurrent transaction processing and recovery from failures

11.1 Introduction

Collection of operations that form a single logical unit of work are called **transactions**. A database system must ensure proper execution of transactions despite failures, either the entire transaction executes, or none of it does. Furthermore, it must manage concurrent

execution of transactions in a way that avoids the introduction of inconsistency. In our funds-transfer example, a transaction computing the customer's total balance might see the checking-account balance before it is debited by the funds-transfer transaction, but see the savings balance after it is credited. As a result, it would obtain an incorrect result.

11.2.1 Transaction Concepts

Transaction Concept

A **transaction** is a *unit* of program execution that accesses and possibly updates various data items.

E.g. transaction to transfer \$50 from account A to account B:

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

Two main issues to deal with: Failures of various kinds, such as hardware failures and system crashes. Concurrent execution of multiple transactions.

A transaction is initiated by a user program written in a high-level data-manipulation language (typically SQL), or programming language (for example, C++, or Java), with embedded database accesses in JDBC or ODBC. A transaction is delimited by statements (or function calls) of the form **begin transaction** and **end transaction**. The transaction consists of all operations executed between the **begin transaction** and **end transaction**.

This collection of steps must appear to the user as a single, indivisible unit. Since a transaction is indivisible, it either executes in its entirety or not at all. Thus, if a transaction begins to execute but fails for whatever reason, any changes to the database that the transaction may have made must be undone. This requirement holds regardless of whether the transaction itself failed (for example, if it divided by zero), the operating system crashed, or the computer itself stopped operating. As we shall see, ensuring that this requirement is met is difficult since some changes to the database may still be stored only in the main-

memory variables of the transaction, while others may have been written to the database and stored on disk. This “all-or-none” property is referred to as **atomicity**.

Furthermore, since a transaction is a single unit, its actions cannot appear to be separated by other database operations not part of the transaction. While we wish to present this user-level impression of transactions, we know that reality is quite different. Even a single SQL statement involves many separate accesses to the database, and a transaction may consist of several SQL statements. Therefore, the database system must take special actions to ensure that transactions operate properly without interference from concurrently executing database statements. This property is referred to as **isolation**.

Even if the system ensures correct execution of a transaction, this serves little purpose if the system subsequently crashes and, as a result, the system “forgets” about the transaction. Thus, a transaction’s actions must persist across crashes. This property is referred to as **durability**.

Because of the above three properties, transactions are an ideal way of structuring interaction with a database. This leads us to impose a requirement on transactions themselves. A transaction must reserve database consistency—if a transaction is run atomically in isolation starting from a consistent database, the database must again be consistent at the end of the transaction. This consistency requirement goes beyond the data integrity constraints we have seen earlier (such as primary-key constraints, referential integrity, **check** constraints, and the like). Rather, transactions are expected to go beyond that to ensure preservation of those application-dependent consistency constraints that are too complex to state using the SQL constructs for data integrity. How this is done is the responsibility of the programmer who codes a transaction. This property is referred to as **consistency**.

To restate the above more concisely, we require that the database system maintain the following properties of the transactions:

- **Atomicity.** Either all operations of the transaction are reflected properly in the database, or none are.
- **Consistency.** Execution of a transaction in isolation (that is, with no other transaction executing concurrently) preserves the consistency of the database.
- **Isolation.** Even though multiple transactions may execute concurrently, the system guarantees that, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started or T_j started execution after T_i finished. Thus,

each transaction is unaware of other transactions executing concurrently in the system.

- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

These properties are often called the **ACID properties**; the acronym is derived from the first letter of each of the four properties.

As we shall see later, ensuring the isolation property may have a significant adverse effect on system performance. For this reason, some applications compromise on the isolation property. We shall study these compromises after first studying the strict enforcement of the ACID properties.

11.2.2 Transaction Atomicity and Durability

A transaction may not always complete its execution successfully. Such a transaction is termed **aborted**. If we are to ensure the atomicity property, an aborted transaction must have no effect on the state of the database. Thus, any changes that the aborted transaction made to the database must be undone. Once the changes caused by an aborted transaction have been undone, we say that the transaction has been **rolled back**. It is part of the responsibility of the recovery scheme to manage transaction aborts. This is done typically by maintaining a **log**. Each database modification made by a transaction is first recorded in the log. We record the identifier of the transaction performing the modification, the identifier of the data item being modified, and both the old value (prior to modification) and the new value (after modification) of the data item. Only then is the database itself modified. Maintaining a log provides the possibility of redoing a modification to ensure atomicity and durability as well as the possibility of undoing a modification to ensure atomicity in case of a failure during transaction execution.

A transaction that completes its execution successfully is said to be **committed**. A committed transaction that has performed updates transforms the database into a new consistent state, which must persist even if there is a system failure.

Once a transaction has committed, we cannot undo its effects by aborting it. The only way to undo the effects of a committed transaction is to execute a **compensating transaction**. For instance, if a transaction added \$20 to an account, the compensating transaction would subtract \$20 from the account. However, it is not always possible to create such a compensating transaction. Therefore, the responsibility of writing and

executing a compensating transaction is left to the user, and is not handled by the database system.

We need to be more precise about what we mean by *successful completion* of a transaction. We therefore establish a simple abstract transaction model. A transaction must be in one of the following states:

- **Active**, the initial state; the transaction stays in this state while it is executing.
- **Partially committed**, after the final statement has been executed.
- **Failed**, after the discovery that normal execution can no longer proceed.
- **Aborted**, after the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction.
- **Committed**, after successful completion. The state diagram corresponding to a transaction appears in Figure 11.1. We say that a transaction has committed only if it has entered the committed state.

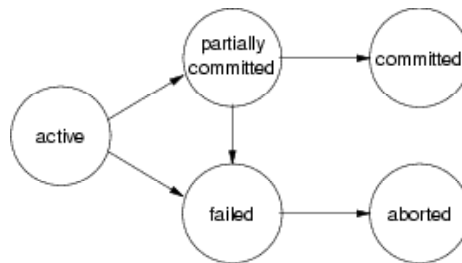


Figure 11.1 State diagram of a transaction.

Similarly, we say that a transaction has aborted only if it has entered the aborted state. A transaction is said to have **terminated** if it has either committed or aborted.

A transaction starts in the active state. When it finishes its final statement, it enters the partially committed state. At this point, the transaction has completed its execution, but it is still possible that it may have to be aborted, since the actual output may still be temporarily residing in main memory, and thus a hardware failure may preclude its successful completion.

The database system then writes out enough information to disk that, even in the event of a failure, the updates performed by the transaction can be re-created when the system restarts after the failure. When the last of this information is written out, the transaction enters the committed state.

A transaction enters the failed state after the system determines that the transaction can no longer proceed with its normal execution (for example, because of hardware or logical errors). Such a transaction must be rolled back. Then, it enters the aborted state. At this point, the system has two options:

- It can **restart** the transaction, but only if the transaction was aborted as a result of some hardware or software error that was not created through the internal logic of the transaction. A restarted transaction is considered to be a new transaction.
- It can **kill** the transaction. It usually does so because of some internal logical error that can be corrected only by rewriting the application program, or because the input was bad, or because the desired data were not found in the database.

We must be cautious when dealing with **observable external writes**, such as writes to a user's screen, or sending email. Once such a write has occurred, it cannot be erased, since it may have been seen external to the database system. Most systems allow such writes to take place only after the transaction has entered the committed state. One way to implement such a scheme is for the database system to store any value associated with such external writes temporarily in a special relation in the database, and to perform the actual writes only after the transaction enters the committed state. If the system should fail after the transaction has entered the committed state, but before it could complete the external writes, the database system will carry out the external writes (using the data in nonvolatile storage) when the system is restarted.

Handling external writes can be more complicated in some situations. For example, suppose the external action is that of dispensing cash at an automated teller machine, and the system fails just before the cash is actually dispensed (we assume that cash can be dispensed atomically). It makes no sense to dispense cash when the system is restarted, since the user may have left the machine. In such a case a compensating transaction, such as depositing the cash back in the user's account needs to be executed when the system is restarted.

As another example, consider a user making a booking over the Web. It is possible that the database system or the application server crashes just after the booking transaction commits. It is also possible that the network connection to the user is lost just after the booking transaction commits. In either case, even though the transaction has committed,

the external write has not taken place. To handle such situations, the application must be designed such that when the user connects to the Web application again, she will be able to see whether her transaction had succeeded or not.

For certain applications, it may be desirable to allow active transactions to display data to users, particularly for long-duration transactions that run for minutes or hours. Unfortunately, we cannot allow such output of observable data unless we are willing to compromise transaction atomicity. We discuss alternative transaction models that support long-duration, interactive transactions.

11.2.3 Implementation of Atomicity and Durability

System Malfunctions

- Transaction processing systems have to maintain correctness in spite of malfunctions
 - Crash
 - Abort
 - Media Failure

Failures: Crash
- Processor failure, software bug
 - Program behaves unpredictably, destroying contents of main (volatile) memory
 - Contents of mass store (non-volatile memory) generally unaffected
 - Active transactions interrupted, database left in inconsistent state
- Server supports atomicity by providing a recovery procedure to restore database to consistent state
 - Since roll forward is generally not feasible, recovery rolls active transactions back

Failures: Abort
- Causes:
 - User (e.g., cancel button)
 - Transaction (e.g., deferred constraint check)
 - System (e.g., deadlock, lack of resources)
- The technique used by the recovery procedure supports atomicity

– Roll transaction back

Failures: Media

- Durability requires that database state produced by committed transactions be Preserved
- Possibility of failure of mass store implies that database state must be stored redundantly (in some form) on independent non-volatile devices

Log

- Sequence of records (sequential file)
 - Modified by appending (no updating)
- Contains information from which database can be reconstructed
 - Read by routines that handle abort and crash recovery
- Log and database stored on different mass storage devices
- Often replicated to survive media failure
- Contains valuable historical data not in database
 - How did database reach current state?
- Each modification of the database causes an update record to be appended to log
- Update record contains:
 - Identity of data item modified
 - Identity of transaction (tid) that did the modification
 - Before image (undo record) – copy of data item before update occurred
- Referred to as physical logging
- Update records in a log

Log – Ex.

x	y	z	u	y	w	z
T ₁	T ₁	T ₂	T ₃	T ₁	T ₄	T ₂
17	A	2.4	18	ab	3	4.5

*most recent
database update*

Transaction Abort Using Log

- Scan log backwards:
 - using tid to identify transaction's update records
 - reverse each update using before image
 - reversal done in last-in-first-out order
- In a strict system new values are unavailable to concurrent transactions (result of long term x-locks);
 - hence rollback makes a transaction atomic
- Problem: terminating scan (log can be long)
- Solution: append a begin record for each transaction, containing tid, prior to its first update record

B	U	U	U	U	U	U	U
T ₁	x	y	z	u	y	w	z
	T ₁	T ₁	T ₂	T ₃	T ₁	T ₄	T ₂
	17	A	2.4	18	ab	3	4.5

↑
abort T₁

Key:

B – begin record

U – update record

- Abort Procedure: Scan back to begin record using update records to reverse changes

Logging Savepoints

- Savepoint record inserted in log:
 - when savepoint created
 - contains tid, savepoint identity
- Rollback Procedure:

- scan log backwards using tid to identify update records
- undo updates using before image terminate scan when appropriate savepoint record encountered

Crash Recovery Using Log

- Abort all transactions active at time of crash
- Problem: How do you identify them?
- Solution: abort record or commit record appended to log when transaction terminates
- Recovery Procedure:
 - Scan log backwards - if T's first record is an update record, T was active at time of crash. Roll it back A transaction is not committed until its commit record is in the log

Crash Recovery Using Log – Ex.

B	U	U	U	U	U	C	U	A	U
T ₁	<i>x</i> T ₁	<i>y</i> T ₁	<i>z</i> T ₂	<i>u</i> T ₃	<i>y</i> T ₁	T ₃	<i>w</i> T ₄	T ₁	<i>z</i> T ₂
	17	A	2.4	18	ab		3		4.5

↑
crash

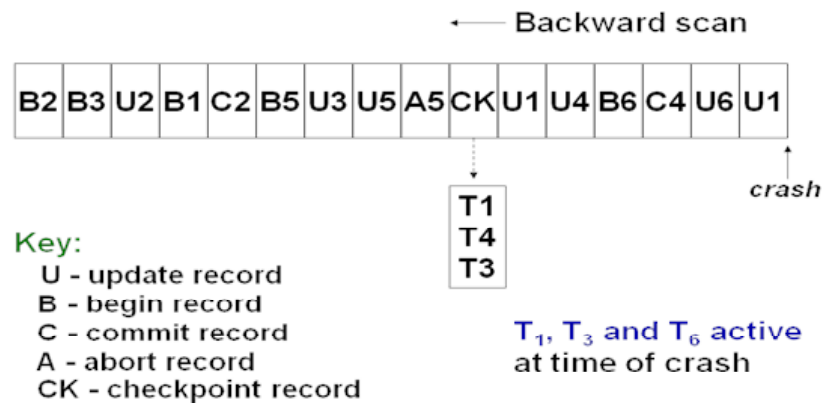
Key:

B – begin record
 U – update record
 C – commit record
 A – abort record

- T₁ and T₃ were not active at time of crash
- Problem: Scan must retrace entire log
- Solution: Periodically append checkpoint rec. to log.
 - Contains tid's of all active trans. at time of append
 - Backward scan goes at least as far as last checkpoint record appended Transactions active at time of crash determined from log suffix that includes last checkpoint record

- Scan continues until those transactions have been rolled back

Example



11.2.4 Serializability

We consider how to determine when a schedule is serializable. Certainly, serial schedules are serializable, but if steps of multiple transactions are interleaved, it is harder to determine whether a schedule is serializable. Since transactions are programs, it is difficult to determine exactly what operations a transaction performs and how operations of various transactions interact. For this reason, we shall not consider the various types of operations that a transaction can perform on a data item, but instead consider only two operations: read and write.

Basic Assumption – Each transaction preserves database consistency. Thus serial execution of a set of transactions preserves database consistency. A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule. Different forms of schedule equivalence give rise to the notions of:

1. conflict serializability
2. view serializability

Simplified view of transactions

We ignore operations other than **read** and **write** instructions.

We assume that transactions may perform arbitrary computations on data in local buffers in between reads and writes.

Our simplified schedules consist of only **read** and **write** instructions.

Conflicting Instructions

Instructions li and lj of transactions T_i and T_j respectively, **conflict** if and only if there exists some item Q accessed by both li and lj , and at least one of these instructions wrote Q .

1. $li = \text{read}(Q)$, $lj = \text{read}(Q)$. li and lj don't conflict.
2. $li = \text{read}(Q)$, $lj = \text{write}(Q)$. They conflict.
3. $li = \text{write}(Q)$, $lj = \text{read}(Q)$. They conflict.
4. $li = \text{write}(Q)$, $lj = \text{write}(Q)$. They conflict.

Intuitively, a conflict between li and lj forces a (logical) temporal order between them. If li and lj are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.

11.2.4.1 Conflict Serializability

If a schedule S can be transformed into a schedule S by a series of swaps of non-conflicting instructions, we say that S and S are **conflict equivalent**.

We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule.

Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1 , by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

T_1	T_2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 3

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

Schedule 6

Example of a schedule that is not conflict serializable:

Schedule 7

T_3	T_4
read(Q)	
write(Q)	write(Q)

We are unable to swap instructions in the above schedule to obtain either the serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$.

11.2.4.2 View Serializability

Let S and S' be two schedules with the same set of transactions. S and S' are **view equivalent** if the following three conditions are met, for each data item Q . If in schedule S , transaction T_i reads the initial value of Q , then in schedule S' also transaction T_i must read the initial value of Q .

If in schedule S transaction T_i executes **read**(Q), and that value was produced by transaction T_j (if any), then in schedule S' also transaction T_i must read the value of Q that was produced by the same **write**(Q) operation of transaction T_j .

The transaction (if any) that performs the final **write**(Q) operation in schedule S must also perform the final **write**(Q) operation in schedule S' .

As can be seen, view equivalence is also based purely on **reads** and **writes** alone.

A schedule S is **view serializable** if it is view equivalent to a serial schedule.

Every conflict serializable schedule is also view serializable.

Below is a schedule which is view-serializable but *not* conflict serializable.

T_{27}	T_{28}	T_{29}
read(Q)		
write(Q)	write(Q)	
		write(Q)

What serial schedule is above equivalent to?

Every view serializable schedule that is not conflict serializable has **blind writes**.

Other Notions of Serializability

The schedule below produces same outcome as the serial schedule $\langle T_1, T_5 \rangle$, yet is not conflict equivalent or view equivalent to it.

T_1	T_5
$\text{read}(A)$ $A := A - 50$ $\text{write}(A)$	
	$\text{read}(B)$ $B := B - 10$ $\text{write}(B)$
$\text{read}(B)$ $B := B + 50$ $\text{write}(B)$	
	$\text{read}(A)$ $A := A + 10$ $\text{write}(A)$

Determining such equivalence requires analysis of operations other than read and write.

Testing for Serializability

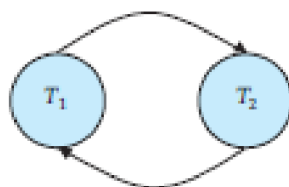
Consider some schedule of a set of transactions $T_1, T_2, \dots, T_n$.

Precedence graph — a directed graph where the vertices are the transactions (names).

We draw an arc from T_i to T_j if the two transaction conflict, and T_i accessed the data item on which the conflict arose earlier.

We may label the arc by the item that was accessed.

Example 1

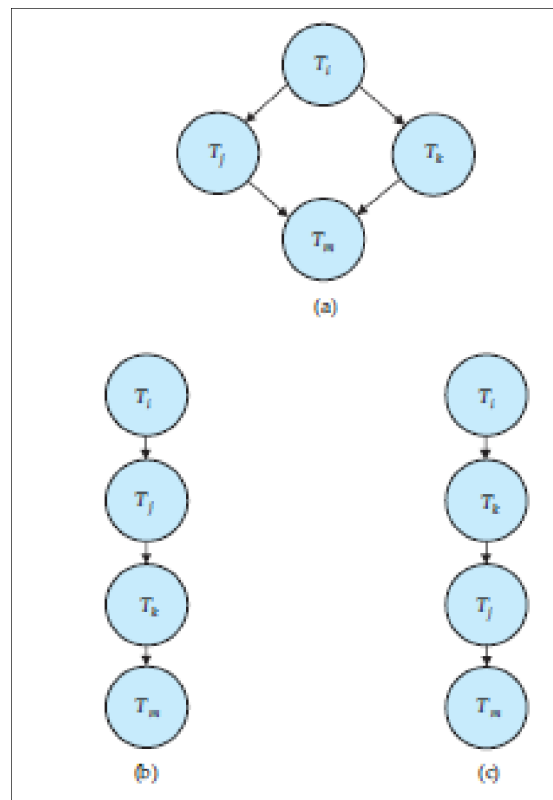


Test for Conflict Serializability

A schedule is conflict serializable if and only if its precedence graph is acyclic.

Cycle-detection algorithms exist which take order n^2 time, where n is the number of vertices in the graph. (Better algorithms take order $n + e$ where e is the number of edges.)

If precedence graph is acyclic, the serializability order can be obtained by a *topological sorting* of the graph. This is a linear order consistent with the partial order of the graph.



Test for View Serializability

The precedence graph test for conflict serializability cannot be used directly to test for view serializability. Extension to test for view serializability has cost exponential in the size of the precedence graph.

The problem of checking if a schedule is view serializable falls in the class of *NP*-complete problems. Thus existence of an efficient algorithm is *extremely* unlikely.

However practical algorithms that just check some **sufficient conditions** for view serializability can still be used.

11.3 Recoverability and Schedules

Need to address the effect of transaction failures on concurrently running transactions.

Recoverable schedule — if a transaction T_j reads a data item previously written by a transaction T_i , then the commit operation of T_i appears before the commit operation of T_j .

The following schedule (Schedule 11) is not recoverable if T_9 commits immediately after the read

T_8	T_9	T_{10}
read(A) read(B) write(A)		
	read(A) write(A)	
abort		read(A)

If T_8 should abort, T_9 would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable.

Cascading Rollbacks

Cascading rollback – a single transaction failure leads to a series of transaction rollbacks. Consider the following schedule where none of the transactions has yet committed (so the schedule is recoverable)

T_8	T_9	T_{10}
read(A) read(B) write(A)		
	read(A) write(A)	
abort		read(A)

If T_{10} fails, T_{11} and T_{12} must also be rolled back.

Can lead to the undoing of a significant amount of work

Cascadeless Schedules

Cascading rollbacks cannot occur; for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i appears before the read operation of T_j .

Every cascadeless schedule is also recoverable

It is desirable to restrict the schedules to those that are cascadeless.

Schedules

Schedule a sequence of instructions that specify the chronological order in which instructions of concurrent transactions are executed a schedule for a set of transactions must consist of all instructions of those transactions must preserve the order in which the instructions appear in each individual transaction.

A transaction that successfully completes its execution will have a commit instructions as the last statement by default transaction assumed to execute commit instruction as its last step

A transaction that fails to successfully complete its execution will have an abort instruction as the last statement

Schedule 1

Let T_1 transfer \$50 from A to B , and T_2 transfer 10% of the balance from A to B .

A serial schedule in which T_1 is followed by T_2 :

T_1	T_2
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code> <code>commit</code>	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code> <code>commit</code>

Schedule 2

A serial schedule where T_2 is followed by T_1

T_1	T_2
	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$ $\text{read}(B)$ $B := B + \text{temp}$ $\text{write}(B)$ commit
$\text{read}(A)$ $A := A - 50$ $\text{write}(A)$ $\text{read}(B)$ $B := B + 50$ $\text{write}(B)$ commit	

Schedule 3

Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule, but it is *equivalent* to Schedule 1.

T_1	T_2
$\text{read}(A)$ $A := A - 50$ $\text{write}(A)$	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$
$\text{read}(B)$ $B := B + 50$ $\text{write}(B)$ commit	$\text{read}(B)$ $B := B + \text{temp}$ $\text{write}(B)$ commit

In Schedules 1, 2 and 3, the sum $A + B$ is preserved.

Schedule 4

The following concurrent schedule does not preserve the value of $(A + B)$.

T_1	T_2
$\text{read}(A)$ $A := A - 50$	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$ $\text{read}(B)$
$\text{write}(A)$ $\text{read}(B)$ $B := B + 50$ $\text{write}(B)$ commit	$B := B + \text{temp}$ $\text{write}(B)$ commit

11.4 Summary

A *transaction* is a *unit* of program execution that accesses and possibly updates various data items. Understanding the concept of a transaction is critical for understanding and implementing updates of data in a database in such a way that concurrent executions and failures of various forms do not result in the database becoming inconsistent.

Transactions are required to have the ACID properties: atomicity, consistency, isolation, and durability.

- Atomicity ensures that either all the effects of a transaction are reflected in the database, or none are; a failure cannot leave the database in a state where a transaction is partially executed.
- Consistency ensures that, if the database is initially consistent, the execution of the transaction (by itself) leaves the database in a consistent state.
- Isolation ensures that concurrently executing transactions are isolated from one another, so that each has the impression that no other transaction is executing concurrently with it.
- Durability ensures that, once a transaction has been committed, that transaction's updates do not get lost, even if there is a system failure.

Serializability of schedules generated by concurrently executing transactions can be ensured through one of a variety of mechanisms called *concurrency control* policies.

11.5 Lesson End Activities

Consider the following two transactions:

```

 $T_{13}$ : read( $A$ );
read( $B$ );
if  $A = 0$  then  $B := B + 1$ ;
write( $B$ ).

 $T_{14}$ : read( $B$ );
read( $A$ );
if  $B = 0$  then  $A := A + 1$ ;
write( $A$ ).

```

Let the consistency requirement be $A = 0 \vee B = 0$, with $A = B = 0$ the initial values.

- Show that every serial execution involving these two transactions preserves the consistency of the database.
- Show a concurrent execution of T_{13} and T_{14} that produces a nonserializable schedule.
- Is there a concurrent execution of T_{13} and T_{14} that produces a serializable schedule?

11.6 Keywords

- Transaction
- ACID properties
 - * Atomicity
 - * Consistency
 - * Isolation
 - * Durability
- Inconsistent state
- Storage types
 - * Volatile storage
 - * Nonvolatile storage
 - * Stable storage

- Concurrency control system
- Recovery system
- Transaction state
 - * Active
 - * Partially committed
 - * Failed
 - * Aborted
 - * Committed
 - * Terminated

11.7 Questions for Discussion

1. List the ACID properties. Explain the usefulness of each.
2. Explain the distinction between the terms *serial schedule* and *serializable Schedule*.
3. Give an example of a serializable schedule with two transactions such that the order in which the transactions commit is different from the serialization order.
4. For each of the following isolation levels, give an example of a schedule that respects the specified level of isolation, but is not serializable:
 - a. Read uncommitted.
 - b. Read committed.
 - c. Repeatable read.

11.8 Suggested Reading

“Database System Concepts” -5th Edition by Silberchatz. Korth. Sudarshan., TMH.

Lesson - 12

CONCURRENCY CONTROL

CONTENTS

12.0 Aim and Objectives

12.1 Introduction

12.2 Lock based protocols

12.2.1 Locks

12.2.2 Granting of Locks

12.2.3 The Two-phase Locking Protocols

12.3 Deadlock Handling

12.3.1 Deadlock Prevention

12.3.2 Deadlock Detection and Recovery

12.3.2.1 Deadlock Detection

12.3.2.2 Recovery from Deadlock

12.4 Recovery System

12.4.1 Failure Classification

12.4.2 Storage Structure

12.4.2.1 Storage Types

12.4.2.2 Stable-Storage Implementation

12.4.2.3 Data Access

12.5 Recovery and Atomicity

12.5.1 Log Records

12.5.2 Database Modification

12.5.3 Concurrency Control and Recovery

12.5.4 Transaction Commit

12.5.5 Using the Log to Redo and Undo Transaction

12.5.6 Checkpoints**12.6 Recovery Algorithm****12.6.1 Transaction Rollback****12.6.2 Recovery after a system crash****12.7 Buffer Management****12.7.1 Log Record Buffering****12.7.2 Database Buffering****12.7.3 Operating System Role in Buffer Management****12.7.3.1 Fuzzy Check pointing****12.8 Summary****12.9 Lesson End Activities****12.10 Keywords****12.11 Questions for Discussion****12.12 Suggested Reading****12.0 AIM AND OBJECTIVES**

After studying this lesson, you should be able to:

- Basic concepts of Lock based protocol.
- Details on concurrency controls.
- Details of deadlocks and preventions.
- Details Recovery System
- Details of buffer management

12.1 Introduction

The system must control the interaction among the concurrent transactions, this control is achieved through one of a variety of mechanisms called *concurrency control* schemes. We discuss concurrency-control schemes that admit non serializable schedules. We consider the management of concurrently executing transactions, and we ignore failures. We shall see how the system can recover from failures. As we shall see, there are a variety of concurrency-control schemes. No one scheme is clearly the best; each one

has advantages. In practice, the most frequently used schemes are *two-phase locking* and *snapshot isolation*.

12.2 Lock-Based Protocols

Oneway to ensure isolation is to require that data items be accessed in a mutually exclusive manner; that is, while one transaction is accessing a data item, no other transaction can modify that data item. The most common method used to implement this requirement is to allow a transaction to access a data item only if it is currently holding a lock on that item.

12.2.1 Locks

There are various modes in which a data item may be locked. In this section, we restrict our attention to two modes:

1. **Shared.** If a transaction T_i has obtained a **shared-mode lock** (denoted by S) on item Q , then T_i can read, but cannot write, Q .
2. **Exclusive.** If a transaction T_i has obtained an **exclusive-mode lock** (denoted by X) on item Q , then T_i can both read and write Q .

	S	X
S	true	false
X	false	false

Figure 12.1 Lock-compatibility matrix comp.

We require that every transaction **request** a lock in an appropriate mode on data item Q , depending on the types of operations that it will perform on Q . The transaction makes the request to the concurrency-control manager. The transaction can proceed with the operation only after the concurrency-control manager **grants** the lock to the transaction. The use of these two lock modes allows multiple transactions to read a data item but limits write access to just one transaction at a time.

To state this more generally, given a set of lock modes, we can define a **compatibility function** on them as follows: Let A and B represent arbitrary lock modes. Suppose that a transaction T_i requests a lock of mode A on item Q on which transaction T_j ($T_i \neq T_j$) currently holds a lock of mode B . If transaction T_i can be granted a lock on Q immediately, in spite of the presence of the mode B lock, then we say mode A is **compatible** with mode

B. Such a function can be represented conveniently by a matrix. The compatibility relation between the two modes of locking discussed in this section appears in the matrix comp of Figure 12.1. An element $\text{comp}(A, B)$ of the matrix has the value *true* if and only if mode *A* is compatible with mode *B*.

Note that shared mode is compatible with shared mode, but not with exclusive mode. At any time, several shared-mode locks can be held simultaneously (by different transactions) on a particular data item. A subsequent exclusive-mode lock request has to wait until the currently held shared-mode locks are released.

A transaction requests a shared lock on data item *Q* by executing the lock-*S*(*Q*) instruction. Similarly, a transaction requests an exclusive lock through the lock-*X*(*Q*) instruction. A transaction can unlock a data item *Q* by the unlock(*Q*) instruction.

To access a data item, transaction *T_i* must first lock that item. If the data item is already locked by another transaction in an incompatible mode, the concurrency control manager will not grant the lock until all incompatible locks held by other transactions have been released. Thus, *T_i* is made to **wait** until all incompatible locks held by other transactions have been released.

Transaction *T_i* may unlock a data item that it had locked at some earlier point. Note that a transaction must hold a lock on a data item as long as it accesses that item. Moreover, it is not necessarily desirable for a transaction to unlock a data item immediately after its final access of that data item, since serializability may not be ensured.

As an illustration, consider again the banking example that we introduced in the previous lesson. Let *A* and *B* be two accounts that are accessed by transactions *T₁* and *T₂*.

```

T1: lock-X(B);
    read(B);
    B := B - 50;
    write(B);
    unlock(B);
    lock-X(A);
    read(A);
    A := A + 50;
    write(A);
    unlock(A).

```

Figure 12.2 Transaction *T₁*.

Transaction T_1 transfers \$50 from account B to account A (Figure. 12.2). Transaction T_2 displays the total amount of money in accounts A and B —that is, the sum $A + B$ (**Fig 12.3**).

Suppose that the values of accounts A and B are \$100 and \$200, respectively. If these two transactions are executed serially, either in the order T_1, T_2 or the order T_2, T_1 , then transaction T_2 will display the value \$300. If, however, these transactions are executed concurrently, then schedule 1, in Figure 12.4, is possible. In this case, transaction T_2 displays \$250, which is incorrect. The reason for this mistake is that the transaction T_1 unlocked data item B too early, as a result of which T_2 saw an inconsistent state.

The schedule shows the actions executed by the transactions, as well as the points at which the concurrency-control manager grants the locks. The transaction making a lock request cannot execute its next action until the concurrency control manager grants the lock. Hence, the lock must be granted in the interval of time between the lock-request operation and the following action of the transaction. Exactly when within this interval the lock is granted is not important; we can safely assume that the lock is granted just before the following action of the transaction. We shall therefore drop the column depicting the actions of the concurrency-control manager from all schedules depicted in the rest of the chapter. We let you infer when locks are granted.

T_2 : lock-S(A);

read(A);

unlock(A);

lock-S(B);

read(B);

unlock(B);

display($A + B$).

Figure 12.3 Transaction T_2 .

T_1	T_2	concurrency-control manager
lock-X(B)		grant-X(B, T_1)
read(B)		
$B := B - 50$		
write(B)		
unlock(B)		
	lock-S(A)	
	read(A)	grant-S(A, T_2)
	unlock(A)	
	lock-S(B)	
		grant-S(B, T_2)
	read(B)	
	unlock(B)	
	display($A + B$)	
lock-X(A)		grant-X(A, T_1)
read(A)		
$A := A - 50$		
write(A)		
unlock(A)		

Figure 12.4 Schedule 1.

Suppose now that unlocking is delayed to the end of the transaction. Transaction T_3 corresponds to T_1 with unlocking delayed (Figure 12.5). Transaction T_4 corresponds to T_2 with unlocking delayed (Figure 15.6).

You should verify that the sequence of reads and writes in schedule 1, which lead to an incorrect total of \$250 being displayed, is no longer possible with T_3 and T_4 .

T_3 : lock-X(B);

read(B);

$B := B - 50$;

write(B);

lock-X(A);

read(A);

$A := A + 50$;

write(A);

```

unlock(B);
unlock(A).

```

Figure 12.5 Transaction *T3* (transaction *T1* with unlocking delayed).

```

T4: lock-S(A);
read(A);
lock-S(B);
read(B);
display(A + B);
unlock(A);
unlock(B).

```

Figure 12.6 Transaction *T4* (transaction *T2* with unlocking delayed).

Other schedules are possible. *T4* will not print out an inconsistent result in any of them; we shall see why later.

Unfortunately, locking can lead to an undesirable situation. Consider the partial schedule of Figure 12.7 for *T3* and *T4*. Since *T3* is holding an exclusive mode lock on *B* and *T4* is requesting a shared-mode lock on *B*, *T4* is waiting for *T3* to unlock *B*. Similarly, since *T4* is holding a shared-mode lock on *A* and *T3* is requesting an exclusive-mode lock on *A*, *T3* is waiting for *T4* to unlock *A*. Thus, we have arrived at a state where neither of these transactions can ever proceed with its normal execution. This situation is called **deadlock**. When deadlock occurs, the system must roll back one of the two transactions. Once a transaction has been rolled back, the data items that were locked by that transaction are unlocked. These data items are then available to the other transaction, which can continue with its execution. We shall return to the issue of deadlock handling.

If we do not use locking, or if we unlock data items too soon after reading or writing them, we may get inconsistent states. On the other hand, if we do not unlock a data item before requesting a lock on another data item, deadlocks may occur. There are ways to avoid deadlock in some situations. However, in general, deadlocks are a necessary evil associated with locking, if we want to avoid inconsistent states. Deadlocks are definitely preferable to inconsistent states, since they can be handled by rolling back transactions, whereas inconsistent states may lead to real-world problems that cannot be handled by the database system.

T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

Figure 12.7 Schedule 2.

We shall require that each transaction in the system follow a set of rules, called a **locking protocol**, indicating when a transaction may lock and unlock each of the data items. Locking protocols restrict the number of possible schedules. The set of all such schedules is a proper subset of all possible serializable schedules. We shall present several locking protocols that allow only conflict-serializable schedules, and thereby ensure isolation. Before doing so, we introduce some terminology.

Let $\{T_0, T_1, \dots, T_n\}$ be a set of transactions participating in a schedule S . We say that T_i **precedes** T_j in S , written $T_i \rightarrow T_j$, if there exists a data item Q such that T_i has held lock mode A on Q , and T_j has held lock mode B on Q later, and $\text{comp}(A, B) = \text{false}$. If $T_i \rightarrow T_j$, then that precedence implies that in any equivalent serial schedule, T_i must appear before T_j . Observe that this graph is similar to the precedence graph to test for conflict serializability. Conflicts between instructions correspond to noncompatibility of lock modes.

We say that a schedule S is **legal** under a given locking protocol if S is a possible schedule for a set of transactions that follows the rules of the locking protocol. We say that a locking protocol **ensures** conflict serializability if and only if all legal schedules are conflict serializable in other words, for all legal schedules the associated $\rightarrow$ relation is acyclic.

12.2.2 Granting of Locks

When a transaction requests a lock on a data item in a particular mode, and no other transaction has a lock on the same data item in a conflicting mode, the lock can be granted. However, care must be taken to avoid the following scenario. Suppose a transaction T_2 has a shared-mode lock on a data item, and another transaction T_1 requests an exclusive-mode lock on the data item. Clearly, T_1 has to wait for T_2 to release the shared-mode lock. Meanwhile, a transaction T_3 may request a shared-mode lock on the same data item. The lock request is compatible with the lock granted to T_2 , so T_3 may be granted the shared-

mode lock. At this point T_2 may release the lock, but still T_1 has to wait for T_3 to finish. But again, there may be a new transaction T_4 that requests a shared-mode lock on the same data item, and is granted the lock before T_3 releases it. In fact, it is possible that there is a sequence of transactions that each requests a shared-mode lock on the data item, and each transaction releases the lock a short while after it is granted, but T_1 never gets the exclusive-mode lock on the data item. The transaction T_1 may never make progress, and is said to be **starved**.

We can avoid starvation of transactions by granting locks in the following manner: When a transaction T_i requests a lock on a data item Q in a particular mode M , the concurrency-control manager grants the lock provided that:

1. There is no other transaction holding a lock on Q in a mode that conflicts with M .
2. There is no other transaction that is waiting for a lock on Q and that made its lock request before T_i .

Thus, a lock request will never get blocked by a lock request that is made later.

12.2.3 The Two-Phase Locking Protocol

One protocol that ensures serializability is the **two-phase locking protocol**. This protocol requires that each transaction issue lock and unlock requests in two phases:

1. **Growing phase.** A transaction may obtain locks, but may not release any lock.
2. **Shrinking phase.** A transaction may release locks, but may not obtain any new locks.

Initially, a transaction is in the growing phase. The transaction acquires locks as needed. Once the transaction releases a lock, it enters the shrinking phase, and it can issue no more lock requests.

For example, transactions T_3 and T_4 are two phase. On the other hand, transactions T_1 and T_2 are not two phase. Note that the unlock instructions do not need to appear at the end of the transaction. For example, in the case of transaction T_3 , we could move the $\text{unlock}(B)$ instruction to just after the $\text{lock-X}(A)$ instruction, and still retain the two-phase locking property. We can show that the two-phase locking protocol ensures conflict serializability.

Consider any transaction. The point in the schedule where the transaction has obtained its final lock (the end of its growing phase) is called the **lock point** of the transaction.

Now, transactions can be ordered according to their lock points-this ordering is, in fact, a serializability ordering for the transactions. We leave the proof as an exercise for you to do (see Practice Exercise 7.1).

Two-phase locking does *not* ensure freedom from deadlock. Observe that transactions T_3 and T_4 are two phase, but, in schedule 2 (Figure 12.7), they are deadlocked.

12.3 Deadlock Handling

A system is in a deadlock state if there exists a set of transactions such that every transaction in the set is waiting for another transaction in the set. More precisely, there exists a set of waiting transactions $\{T_0, T_1, \dots, T_n\}$ such that T_0 is waiting for a data item that T_1 holds, and T_1 is waiting for a data item that T_2 holds, and $\dots$, and T_{n-1} is waiting for a data item that T_n holds, and T_n is waiting for a data item that T_0 holds. None of the transactions can make progress in such a situation.

The only remedy to this undesirable situation is for the system to invoke some drastic action, such as rolling back some of the transactions involved in the deadlock. Rollback of a transaction may be partial: That is, a transaction may be rolled back to the point where it obtained a lock whose release resolves the deadlock.

There are two principal methods for dealing with the deadlock problem. We can use a deadlock prevention protocol to ensure that the system will *never* enter a deadlock state. Alternatively, we can allow the system to enter a deadlock state, and then try to recover by using a deadlock detection and deadlock recovery scheme. As we shall see, both methods may result in transaction rollback. Prevention is commonly used if the probability that the system would enter a deadlock state is relatively high; otherwise, detection and recovery are more efficient.

Note that a detection and recovery scheme requires overhead that includes not only the run-time cost of maintaining the necessary information and of executing the detection algorithm, but also the potential losses inherent in recovery from a deadlock.

12.3.1 Deadlock Prevention

There are two approaches to deadlock prevention. One approach ensures that no cyclic waits can occur by ordering the requests for locks, or requiring all locks to be acquired together. The other approach is closer to deadlock recovery, and performs transaction rollback instead of waiting for a lock, whenever the wait could potentially result in a deadlock.

The simplest scheme under the first approach requires that each transaction locks all its data items before it begins execution. Moreover, either all are locked in one step or none are locked. There are two main disadvantages to this protocol: (1) it is often hard to predict, before the transaction begins, what data items need to be locked; (2) data-item utilization may be very low, since many of the data items may be locked but unused for a long time.

Another approach for preventing deadlocks is to impose an ordering of all data items, and to require that a transaction lock data items only in a sequence consistent with the ordering. We have seen one such scheme in the tree protocol, which uses a partial ordering of data items.

A variation of this approach is to use a total order of data items, in conjunction with two-phase locking. Once a transaction has locked a particular item, it cannot request locks on items that precede that item in the ordering. This scheme is easy to implement, as long as the set of data items accessed by a transaction is known when the transaction starts execution. There is no need to change the underlying concurrency-control system if two-phase locking is used: All that is needed is to ensure that locks are requested in the right order.

The second approach for preventing deadlocks is to use preemption and transaction rollbacks. In preemption, when a transaction T_j requests a lock that transaction T_i holds, the lock granted to T_i may be **preempted** by rolling back of T_i , and granting of the lock to T_j . To control the preemption, we assign a unique timestamp, based on a counter or on the system clock, to each transaction when it begins. The system uses these timestamps only to decide whether a transaction should wait or roll back. Locking is still used for concurrency control. If a transaction is rolled back, it retains its *old* timestamp when restarted. Two different deadlock-prevention schemes using timestamps have been proposed:

1. The **wait-die** scheme is a non preemptive technique. When transaction T_i requests a data item currently held by T_j , T_i is allowed to wait only if it has a timestamp smaller than that of T_j (that is, T_i is older than T_j). Otherwise, T_i is rolled back (dies).

For example, suppose that transactions T_{14} , T_{15} , and T_{16} have timestamps 5, 10, and 15, respectively. If T_{14} requests a data item held by T_{15} , then T_{14} will wait. If T_{16} requests a data item held by T_{15} , then T_{16} will be rolled back.

2. The **wound-wait** scheme is a preemptive technique. It is a counterpart to the wait-die scheme. When transaction T_i requests a data item currently held by T_j , T_i is

allowed to wait only if it has a timestamp larger than that of T_j (that is, T_i is younger than T_j). Otherwise, T_j is rolled back (T_j is *wounded* by T_i).

Returning to our example, with transactions T_{14} , T_{15} , and T_{16} , if T_{14} requests a data item held by T_{15} , then the data item will be preempted from T_{15} , and T_{15} will be rolled back. If T_{16} requests a data item held by T_{15} , then T_{16} will wait.

The major problem with both of these schemes is that unnecessary rollbacks may occur.

Another simple approach to deadlock prevention is based on **lock timeouts**. In this approach, a transaction that has requested a lock waits for at most a specified amount of time. If the lock has not been granted within that time, the transaction is said to time out, and it rolls itself back and restarts. If there was in fact a deadlock, one or more transactions involved in the deadlock will time out and roll back, allowing the others to proceed. This scheme falls somewhere between deadlock prevention, where a deadlock will never occur, and deadlock detection and recovery.

The timeout scheme is particularly easy to implement, and works well if transactions are short and if long waits are likely to be due to deadlocks. However, in general it is hard to decide how long a transaction must wait before timing out. Too long a wait results in unnecessary delays once a deadlock has occurred. Too short a wait results in transaction rollback even when there is no deadlock, leading to wasted resources. Starvation is also a possibility with this scheme. Hence, the timeout-based scheme has limited applicability.

12.3.2 Deadlock Detection and Recovery

If a system does not employ some protocol that ensures deadlock freedom, then a detection and recovery scheme must be used. An algorithm that examines the state of the system is invoked periodically to determine whether a deadlock has occurred. If one has, then the system must attempt to recover from the deadlock. To do so, the system must:

- Maintain information about the current allocation of data items to transactions, as well as any outstanding data item requests.
- Provide an algorithm that uses this information to determine whether the system has entered a deadlock state.
- Recover from the deadlock when the detection algorithm determines that a deadlock exists.

In this section, we elaborate on these issues.

12.3.2.1 Deadlock Detection

Deadlocks can be described precisely in terms of a directed graph called a **wait for graph**. This graph consists of a pair $G = (V, E)$, where V is a set of vertices and E is a set of edges. The set of vertices consists of all the transactions in the system. Each element in the set E of edges is an ordered pair $T_i \rightarrow T_j$. If $T_i \rightarrow T_j$ is in E ,

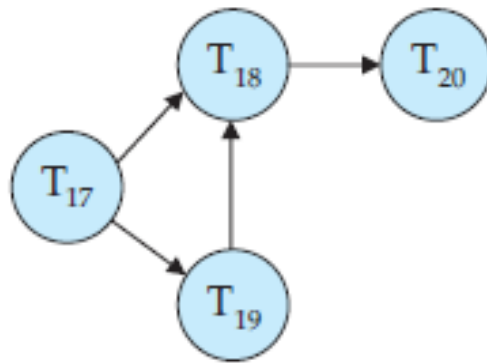


Figure 12.8 Wait-for graph with no cycle.

then there is a directed edge from transaction T_i to T_j , implying that transaction T_i is waiting for transaction T_j to release a data item that it needs.

When transaction T_i requests a data item currently being held by transaction T_j , then the edge $T_i \rightarrow T_j$ is inserted in the wait-for graph. This edge is removed only when transaction T_j is no longer holding a data item needed by transaction T_i .

A deadlock exists in the system if and only if the wait-for graph contains a cycle. Each transaction involved in the cycle is said to be deadlocked. To detect deadlocks, the system needs to maintain the wait-for graph, and periodically to invoke an algorithm that searches for a cycle in the graph.

To illustrate these concepts, consider the wait-for graph in Figure 12.13, which depicts the following situation:

- Transaction T_{17} is waiting for transactions T_{18} and T_{19} .
- Transaction T_{19} is waiting for transaction T_{18} .
- Transaction T_{18} is waiting for transaction T_{20} .

Since the graph has no cycle, the system is not in a deadlock state.

Suppose now that transaction T_{20} is requesting an item held by T_{19} . The edge $T_{20} \rightarrow T_{19}$ is added to the wait-for graph, resulting in the new system state in Figure 12.9. This time, the graph contains the cycle:

$$T_{18} \rightarrow T_{20} \rightarrow T_{19} \rightarrow T_{18}$$

implying that transactions T_{18} , T_{19} , and T_{20} are all deadlocked.

Consequently, the question arises: When should we invoke the detection algorithm? The answer depends on two factors:

1. How often does a deadlock occur?
2. How many transactions will be affected by the deadlock?

If deadlocks occur frequently, then the detection algorithm should be invoked more frequently. Data items allocated to deadlocked transactions will be unavailable to other transactions until the deadlock can be broken.

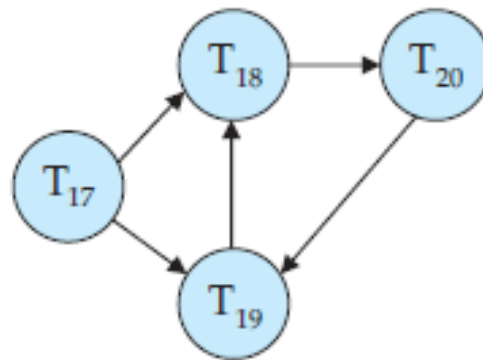


Figure 12.9 Wait-for graph with a cycle.

In addition, the number of cycles in the graph may also grow. In the worst case, we would invoke the detection algorithm every time a request for allocation could not be granted immediately.

12.3.2.2 Recovery from Deadlock

When a detection algorithm determines that a deadlock exists, the system must **recover** from the deadlock. The most common solution is to roll back one or more transactions to break the deadlock. Three actions need to be taken:

1. **Selection of a victim.** Given a set of deadlocked transactions, we must determine which transaction (or transactions) to roll back to break the deadlock. We should roll

back those transactions that will incur the minimum cost. Unfortunately, the term *minimum cost* is not a precise one. Many factors may determine the cost of a rollback, including:

- a. How long the transaction has computed, and how much longer the transaction will compute before it completes its designated task.
- b. How many data items the transaction has used.
- c. How many more data items the transaction needs for it to complete.
- d. How many transactions will be involved in the rollback.

2. **Rollback.** Once we have decided that a particular transaction must be rolled back, we must determine how far this transaction should be rolled back.

The simplest solution is a **total rollback**: Abort the transaction and then restart it. However, it is more effective to roll back the transaction only as far as necessary to break the deadlock. Such **partial rollback** requires the system to maintain additional information about the state of all the running transactions. Specifically, the sequence of lock requests/grants and updates performed by the transaction needs to be recorded. The deadlock detection mechanism should decide which locks the selected transaction needs to release in order to break the deadlock. The selected transaction must be rolled back to the point where it obtained the first of these locks, undoing all actions it took after that point. The recovery mechanism must be capable of performing such partial rollbacks. Furthermore, the transactions must be capable of resuming execution after a partial rollback. See the bibliographical notes for relevant references.

3. **Starvation.** In a system where the selection of victims is based primarily on cost factors, it may happen that the same transaction is always picked as a victim. As a result, this transaction never completes its designated task, thus there is **starvation**. We must ensure that a transaction can be picked as a victim only a (small) finite number of times. The most common solution is to include the number of rollbacks in the cost factor.

12.4 Recovery System

A computer system, like any other device, is subject to failure from a variety of causes: disk crash, power outage, software error, a fire in the machine room, even sabotage. In any failure, information may be lost. Therefore, the database system must take actions in

advance to ensure that the atomicity and durability properties of transactions are preserved. An integral part of a database system is a **recovery scheme** that can restore the database to the consistent state that existed before the failure. The recovery scheme must also provide **high availability**; that is, it must minimize the time for which the database is not usable after a failure.

12.4.1 Failure Classification

There are various types of failure that may occur in a system, each of which needs to be dealt with in a different manner. In this chapter, we shall consider only the following types of failure:

- Transaction failure. There are two types of errors that may cause a transaction to fail:
 - * Logical error. The transaction can no longer continue with its normal execution because of some internal condition, such as bad input, data not found, overflow, or resource limit exceeded.
 - * System error. The system has entered an undesirable state (for example, deadlock), as a result of which a transaction cannot continue with its normal execution. The transaction, however, can be reexecuted at a later time.
- System crash. There is a hardware malfunction, or a bug in the database software or the operating system, that causes the loss of the content of volatile storage, and brings transaction processing to a halt. The content of nonvolatile storage remains intact, and is not corrupted.

The assumption that hardware errors and bugs in the software bring the system to a halt, but do not corrupt the nonvolatile storage contents, is known as the **fail-stop assumption**. Well-designed systems have numerous internal checks, at the hardware and the software level, that bring the system to a halt when there is an error. Hence, the fail-stop assumption is a reasonable one.

- **Disk failure.** A disk block loses its content as a result of either a head crash or failure during a data-transfer operation. Copies of the data on other disks, or archival backups on tertiary media, such as DVD or tapes, are used to recover from the failure.

To determine how the system should recover from failures, we need to identify the failure modes of those devices used for storing data. Next, we must consider how these failure modes affect the contents of the database. We can then propose algorithms to

ensure database consistency and transaction atomicity despite failures. These algorithms, known as recovery algorithms, have two parts:

1. Actions taken during normal transaction processing to ensure that enough information exists to allow recovery from failures.
2. Actions taken after a failure to recover the database contents to a state that ensures database consistency, transaction atomicity, and durability.

12.4.2 Storage Structure

As we saw, the various data items in the database may be stored and accessed in a number of different storage media. We identified three categories of storage:

12.4.2.1 Storage Types

- Volatile storage
- Nonvolatile storage
- Stable storage

Stable storage or, more accurately, an approximation thereof, plays a critical role in recovery algorithms.

12.4.2.2 Stable-Storage Implementation

To implement stable storage, we need to replicate the needed information in several nonvolatile storage media (usually disk) with independent failure modes, and to update the information in a controlled manner to ensure that failure during data transfer does not damage the needed information.

Recall that RAID systems guarantee that the failure of a single disk (even during data transfer) will not result in loss of data. The simplest and fastest form of RAID is the mirrored disk, which keeps two copies of each block, on separate disks. Other forms of RAID offer lower costs, but at the expense of lower performance.

RAID systems, however, cannot guard against data loss due to disasters such as fires or flooding. Many systems store archival backups of tapes off site to guard against such disasters. However, since tapes cannot be carried off site continually, updates since the most recent time that tapes were carried off site could be lost in such a disaster. More secure systems keep a copy of each block of stable storage at a remote site, writing it out

over a computer network, in addition to storing the block on a local disk system. Since the blocks are output to a remote system as and when they are output to local storage, once an output operation is complete, the output is not lost, even in the event of a disaster such as a fire or flood. We study such *remote backup* systems.

In the remainder of this section, we discuss how storage media can be protected from failure during data transfer. Block transfer between memory and disk storage can result in:

- **Successful completion.** The transferred information arrived safely at its destination.
- **Partial failure.** A failure occurred in the midst of transfer, and the destination block has incorrect information.
- **Total failure.** The failure occurred sufficiently early during the transfer that the destination block remains intact.

We require that, if a **data-transfer failure** occurs, the system detects it and invokes a recovery procedure to restore the block to a consistent state. To do so, the system must maintain two physical blocks for each logical database block; in the case of mirrored disks, both blocks are at the same location; in the case of remote backup, one of the blocks is local, whereas the other is at a remote site. An output operation is executed as follows:

1. Write the information onto the first physical block.
2. When the first write completes successfully, write the same information onto the second physical block.
3. The output is completed only after the second write completes successfully.

If the system fails while blocks are being written, it is possible that the two copies of a block are inconsistent with each other. During recovery, for each block, the system would need to examine two copies of the blocks. If both are the same and no detectable error exists, then no further actions are necessary. (Recall that errors in a disk block, such as a partial write to the block, are detected by storing a checksum with each block.) If the system detects an error in one block, then it replaces its content with the content of the other block. If both blocks contain no detectable error, but they differ in content, then the system replaces the content of the first block with the value of the second. This recovery procedure ensures that a write to stable storage either succeeds completely (that is, updates all copies) or results in no change.

The requirement of comparing every corresponding pair of blocks during recovery is expensive to meet. We can reduce the cost greatly by keeping track of block writes that are in progress, using a small amount of nonvolatile RAM. On recovery, only blocks for which writes were in progress need to be compared.

The protocols for writing out a block to a remote site are similar to the protocols for writing blocks to a mirrored disk system.

We can extend this procedure easily to allow the use of an arbitrarily large number of copies of each block of stable storage. Although a large number of copies reduces the probability of a failure to even lower than two copies do, it is usually reasonable to simulate stable storage with only two copies.

12.4.2.3 Data Access

As we saw, the database system resides permanently on nonvolatile storage (usually disks) and only parts of the database are in memory at any time.¹ The database is partitioned into fixed-length storage units called **blocks**. Blocks are the units of data transfer to and from disk, and may contain several data items. We shall assume that no data item spans two or more blocks. This assumption is realistic for most data-processing applications, such as a bank or a university.

Transactions input information from the disk to main memory, and then output the information back onto the disk. The input and output operations are done in block units. The blocks residing on the disk are referred to as **physical blocks**; the blocks residing temporarily in main memory are referred to as **buffer blocks**. The area of memory where blocks reside temporarily is called the **disk buffer**. Block movements between disk and main memory are initiated through the following two operations:

1. $\text{input}(B)$ transfers the physical block B to main memory.
2. $\text{output}(B)$ transfers the buffer block B to the disk, and replaces the appropriate physical block there. Figure 12.10 illustrates this scheme.

Conceptually, each transaction T_i has a private work area in which copies of data items accessed and updated by T_i are kept. The system creates this work area when the transaction is initiated; the system removes it when the transaction either commits or aborts. Each data item X kept in the work area of transaction T_i is denoted by x_i .

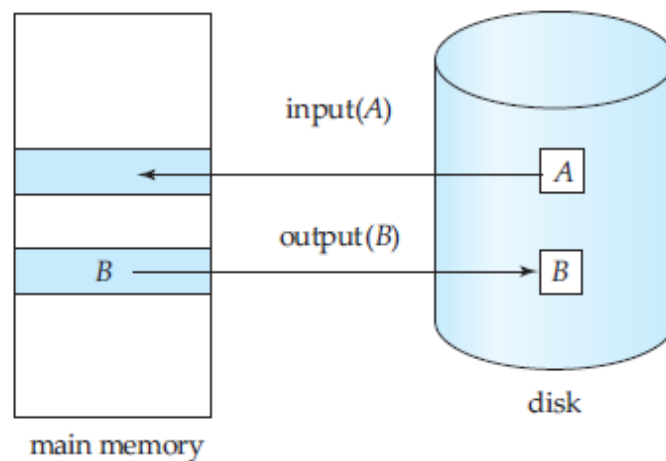


Figure 12.10 Block storage operations.

Transaction T_i interacts with the database system by transferring data to and from its work area to the system buffer. We transfer data by these two operations:

1. `read(X)` assigns the value of data item X to the local variable x_i . It executes this operation as follows:
 - a. If block BX on which X resides is not in main memory, it issues `input(BX)`.
 - b. It assigns to x_i the value of X from the buffer block.
2. `write(X)` assigns the value of local variable x_i to data item X in the buffer block. It executes this operation as follows:
 - a. If block BX on which X resides is not in main memory, it issues `input(BX)`.
 - b. It assigns the value of x_i to X in buffer BX .

Note that both operations may require the transfer of a block from disk to main memory. They do not, however, specifically require the transfer of a block from main memory to disk.

A buffer block is eventually written out to the disk either because the buffer manager needs the memory space for other purposes or because the database system wishes to reflect the change to B on the disk. We shall say that the database system performs a **force-output** of buffer B if it issues an `output(B)`.

When a transaction needs to access a data item X for the first time, it must execute `read(X)`. The system then performs all updates to X on x_i . At any point during its execution

a transaction may execute $\text{write}(X)$ to reflect the change to X in the database itself; $\text{write}(X)$ must certainly be done after the final write to X .

The $\text{output}(BX)$ operation for the buffer block BX on which X resides does not need to take effect immediately after $\text{write}(X)$ is executed, since the block BX may contain other data items that are still being accessed. Thus, the actual output may take place later. Notice that, if the system crashes after the $\text{write}(X)$ operation was executed but before $\text{output}(BX)$ was executed, the new value of X is never written to disk and, thus, is lost. As we shall see shortly, the database system executes extra actions to ensure that updates performed by committed transactions are not lost even if there is a system crash.

12.5 Recovery and Atomicity

12.5.1 Log Records

The most widely used structure for recording database modifications is the **log**. The log is a sequence of **log records**, recording all the update activities in the database.

There are several types of log records. An **update log record** describes a single database write. It has these fields:

- **Transaction identifier**, which is the unique identifier of the transaction that performed the write operation.
- **Data-item identifier**, which is the unique identifier of the data item written. Typically, it is the location on disk of the data item, consisting of the block identifier of the block on which the data item resides, and an offset within the block.
- **Old value**, which is the value of the data item prior to the write.
- **New value**, which is the value that the data item will have after the write.

We represent an update log record as $\langle T_i, X_j, V_1, V_2 \rangle$, indicating that transaction T_i has performed a write on data item X_j . X_j had value V_1 before the write, and has value V_2 after the write. Other special log records exist to record significant events during transaction processing, such as the start of a transaction and the commit or abort of a transaction. Among the types of log records are:

- $\langle T_i \text{ start} \rangle$. Transaction T_i has started.
- $\langle T_i \text{ commit} \rangle$. Transaction T_i has committed.
- $\langle T_i \text{ abort} \rangle$. Transaction T_i has aborted.

We shall introduce several other types of log records later.

Whenever a transaction performs a write, it is essential that the log record for that write be created and added to the log, before the database is modified. Once a log record exists, we can output the modification to the database if that is desirable. Also, we have the ability to *undo* a modification that has already been output to the database. We undo it by using the old-value field in log records.

For log records to be useful for recovery from system and disk failures, the log must reside in stable storage. For now, we assume that every log record is written to the end of the log on stable storage as soon as it is created. We shall see when it is safe to relax this requirement so as to reduce the overhead imposed by logging. Observe that the log contains a complete record of all database activity. As a result, the volume of data stored in the log may become unreasonably large.

12.5.2 Database Modification

As we noted earlier, a transaction creates a log record prior to modifying the database. The log records allow the system to undo changes made by a transaction in the event that the transaction must be aborted; they allow the system also to redo changes made by a transaction if the transaction has committed but the system crashed before those changes could be stored in the database on disk. In order for us to understand the role of these log records in recovery, we need to consider the steps a transaction takes in modifying a data item:

1. The transaction performs some computations in its own private part of main memory.
2. The transaction modifies the data block in the disk buffer in main memory holding the data item.
3. The database system executes the output operation that writes the data block to disk.

We say a transaction *modifies the database* if it performs an update on a disk buffer, or on the disk itself; updates to the private part of main memory do not count as database modifications. If a transaction does not modify the database until it has committed, it is said to use the **deferred-modification** technique. If database modifications occur while the

transaction is still active, the transaction is said to use the **immediate-modification** technique. Deferred modification has the overhead that transactions need to make local copies of all updated data items; further, if a transaction reads a data item that it has updated, it must read the value from its local copy.

The recovery algorithms we describe in this chapter support immediate modification. As described, they work correctly even with deferred modification, but can be optimized to reduce overhead when used with deferred modification; we leave details as an exercise.

A recovery algorithm must take into account a variety of factors, including:

- The possibility that a transaction may have committed although some of its database modifications exist only in the disk buffer in main memory and not in the database on disk.
- The possibility that a transaction may have modified the database while in the active state and, as a result of a subsequent failure, may need to abort.

Because all database modifications must be preceded by the creation of a log record, the system has available both the old value prior to the modification of the data item and the new value that is to be written for the data item. This allows the system to perform *undo* and *redo* operations as appropriate.

- **Undo** using a log record sets the data item specified in the log record to the old value.
- **Redo** using a log record sets the data item specified in the log record to the new value.

12.5.3 Concurrency Control and Recovery

If the concurrency control scheme allows a data item X that has been modified by a transaction T_1 to be further modified by another transaction T_2 before T_1 commits, then undoing the effects of T_1 by restoring the old value of X (before T_1 updated X) would also undo the effects of T_2 . To avoid such situations, recovery algorithms usually require that if a data item has been modified by a transaction, no other transaction can modify the data item until the first transaction commits or aborts.

This requirement can be ensured by acquiring an exclusive lock on any updated data item and holding the lock until the transaction commits; in other words, by using strict two-phase locking. Snapshot-isolation and validation based concurrency-control techniques also acquire exclusive locks on data items at the time of validation, before modifying the data items, and hold the locks until the transaction is committed; as a result the above requirement is satisfied even by these concurrency control protocols.

How the above requirement can be relaxed in certain cases. When either snapshot-isolation or validation is used for concurrency control, database updates of a transaction are (conceptually) deferred until the transaction is partially committed; the deferred-modification technique is a natural fit with these concurrency control schemes. However, it is worth noting that some implementations of snapshot isolation use immediate modification, but provide a logical snapshot on demand: when a transaction needs to read an item that a concurrent transaction has updated, a copy of the (already updated) item is made, and updates made by concurrent transactions are rolled back on the copy of the item. Similarly, immediate modification of the database is a natural fit with two-phase locking, but deferred modification can also be used with two-phase locking.

12.5.4 Transaction Commit

We say that a transaction has **committed** when its commit log record, which is the last log record of the transaction, has been output to stable storage; at that point all earlier log records have already been output to stable storage. Thus, there is enough information in the log to ensure that even if there is a system crash, the updates of the transaction can be redone. If a system crash occurs before a log record $\langle T_i \text{ commit} \rangle$ is output to stable storage, transaction T_i will be rolled back. Thus, the output of the block containing the commit log record is the single atomic action that results in a transaction getting committed.²

With most log-based recovery techniques, including the ones we describe in this chapter, blocks containing the data items modified by a transaction do not have to be output to stable storage when the transaction commits, but can be output some time later.

12.5.5 Using the Log to Redo and Undo Transactions

We now provide an overview of how the log can be used to recover from a system crash, and to roll back transactions during normal operation.

Consider our simplified banking system. Let T_0 be a transaction that transfers \$50 from account A to account B :

```

<T0 start>
<T0, A, 1000, 950>
<T0, B, 2000, 2050>
<T0 commit>
<T1 start>
<T1, C, 700, 600>
<T1 commit>

```

Figure 12.11 Portion of the system log corresponding to T₀ and T₁.

```

T0: read(A);
    A := A - 50;
    write(A);
    read(B);
    B := B + 50;
    write(B).

```

Let T₁ be a transaction that withdraws \$100 from account C:

```

T1: read(C);
    C := C - 100;
    write(C).

```

The portion of the log containing the relevant information concerning these two transactions appears in Figure 12.11.

Figure 12.12 shows one possible order in which the actual outputs took place in both the database system and the log as a result of the execution of T₀ and T₁.³

Using the log, the system can handle any failure that does not result in the loss of information in nonvolatile storage. The recovery scheme uses two recovery procedures. Both these procedures make use of the log to find the set of data items updated by each transaction T_i, and their respective old and new values.

- redo(T_i) sets the value of all data items updated by transaction T_i to the new values.

The order in which updates are carried out by redo is important; when recovering from a system crash, if updates to a particular data item are applied in an order different from the order in which they were applied originally, the final state of that data item will have a wrong value. Most recovery algorithms, including the one we describing, do not perform redo of each transaction separately; instead they perform a single scan of the log, during

which redo actions are performed for each log record as it is encountered. This approach ensures the order of updates is preserved, and is more efficient since the log needs to be read only once overall, instead of once per transaction.

Log	Database
$\langle T_0 \text{ start} \rangle$	
$\langle T_0, A, 1000, 950 \rangle$	
$\langle T_0, B, 2000, 2050 \rangle$	
	$A = 950$
	$B = 2050$
$\langle T_0 \text{ commit} \rangle$	
$\langle T_1 \text{ start} \rangle$	
$\langle T_1, C, 700, 600 \rangle$	
	$C = 600$
$\langle T_1 \text{ commit} \rangle$	

Figure 12.12 State of system log and database corresponding to T_0 and T_1 .

- $\text{undo}(T_i)$ restores the value of all data items updated by transaction T_i to the old values.
- * The undo operation not only restores the data items to their old value, but also writes log records to record the updates performed as part of the undo process. These log records are special **redo-only** log records, since they do not need to contain the old-value of the updated data item.

As with the redo procedure, the order in which undo operations are performed is important.

- * When the undo operation for transaction T_i completes, it writes a $\langle T_i \text{ abort} \rangle$ log record, indicating that the undo has completed.

The $\text{undo}(T_i)$ procedure is executed only once for a transaction, if the transaction is rolled back during normal processing or if on recovering from a system crash, neither a commit nor an abort record is found for transaction T_i . As a result, every transaction will eventually have either a commit or an abort record in the log.

After a system crash has occurred, the system consults the log to determine which transactions need to be redone, and which need to be undone so as to ensure atomicity.

- Transaction T_i needs to be undone if the log contains the record $\langle T_i \text{ start} \rangle$, but does not contain either the record $\langle T_i \text{ commit} \rangle$ or the record $\langle T_i \text{ abort} \rangle$.

Transaction T_i needs to be redone if the log contains the record $\langle T_i \text{ start} \rangle$ and either the record $\langle T_i \text{ commit} \rangle$ or the record $\langle T_i \text{ abort} \rangle$. It may seem strange to redo T_i if the

record $\langle T_i \text{ abort} \rangle$ is in the log. To see why this works, note that if $\langle T_i \text{ abort} \rangle$ is in the log, so are the redo-only records written by the undo operation.

$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$
$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$
$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$
	$\langle T_0 \text{ commit} \rangle$	$\langle T_0 \text{ commit} \rangle$
	$\langle T_1 \text{ start} \rangle$	$\langle T_1 \text{ start} \rangle$
	$\langle T_1, C, 700, 600 \rangle$	$\langle T_1, C, 700, 600 \rangle$
		$\langle T_1 \text{ commit} \rangle$
(a)	(b)	(c)

Figure 12.13 The same log, shown at three different times.

Thus, the end result will be to undo T_i 's modifications in this case. This slight redundancy simplifies the recovery algorithm and enables faster overall recovery time.

As an illustration, return to our banking example, with transaction T_0 and T_1 executed one after the other in the order T_0 followed by T_1 . Suppose that the system crashes before the completion of the transactions. We shall consider three cases. The state of the logs for each of these cases appears in Figure 12.13.

First, let us assume that the crash occurs just after the log record for the step: $\text{write}(B)$ of transaction T_0 has been written to stable storage (Figure 12.13a). When the system comes back up, it finds the record $\langle T_0 \text{ start} \rangle$ in the log, but no corresponding

$\langle T_0 \text{ commit} \rangle$ or $\langle T_0 \text{ abort} \rangle$ record. Thus, transaction T_0 must be undone, so an $\text{undo}(T_0)$ is performed. As a result, the values in accounts A and B (on the disk) are restored to \$1000 and \$2000, respectively.

Next, let us assume that the crash comes just after the log record for the step:

$\text{write}(C)$

of transaction T_1 has been written to stable storage (Figure 12.13b). When the system comes back up, two recovery actions need to be taken. The operation $\text{undo}(T_1)$ must be performed, since the record $\langle T_1 \text{ start} \rangle$ appears in the log, but there is no record $\langle T_1 \text{ commit} \rangle$ or $\langle T_1 \text{ abort} \rangle$. The operation $\text{redo}(T_0)$ must be performed, since the log contains both the record $\langle T_0 \text{ start} \rangle$ and the record $\langle T_0 \text{ commit} \rangle$. At the end of the entire recovery procedure, the values of accounts A , B , and C are \$950, \$2050, and \$700, respectively.

Finally, let us assume that the crash occurs just after the log record:

<T1 commit>

has been written to stable storage (Figure 12.13c). When the system comes back up, both T_0 and T_1 need to be redone, since the records <T0 start> and <T0 commit> appear in the log, as do the records <T1 start> and <T1 commit>. After the system performs the recovery procedures redo(T_0) and redo(T_1), the values in accounts A , B , and C are \$950, \$2050, and \$600, respectively.

12.5.6 Checkpoints

When a system crash occurs, we must consult the log to determine those transactions that need to be redone and those that need to be undone. In principle, we need to search the entire log to determine this information. There are two major difficulties with this approach:

1. The search process is time-consuming.
2. Most of the transactions that, according to our algorithm, need to be redone have already written their updates into the database. Although redoing them will cause no harm, it will nevertheless cause recovery to take longer.

To reduce these types of overhead, we introduce checkpoints.

We describe below a simple checkpoint scheme that (a) does not permit any updates to be performed while the checkpoint operation is in progress, and (b) outputs all modified buffer blocks to disk when the checkpoint is performed. We discuss later how to modify the check pointing and recovery procedures to provide more flexibility by relaxing both these requirements.

A checkpoint is performed as follows:

1. Output onto stable storage all log records currently residing in main memory.
2. Output to the disk all modified buffer blocks.
3. Output onto stable storage a log record of the form <checkpoint L >, where L is a list of transactions active at the time of the checkpoint.

Transactions are not allowed to perform any update actions, such as writing to a buffer block or writing a log record, while a checkpoint is in progress.

The presence of a <checkpoint L > record in the log allows the system to streamline its recovery procedure. Consider a transaction T_i that completed prior to the checkpoint. For such a transaction, the < T_i commit> record (or < T_i abort> record) appears in the log

before the <checkpoint> record. Any database modifications made by T_i must have been written to the database either prior to the checkpoint or as part of the checkpoint itself. Thus, at recovery time, there is no need to perform a redo operation on T_i .

After a system crash has occurred, the system examines the log to find the last <checkpoint L > record (this can be done by searching the log backward, from the end of the log, until the first <checkpoint L > record is found).

The redo or undo operations need to be applied only to transactions in L , and to all transactions that started execution after the <checkpoint L > record was written to the log. Let us denote this set of transactions as T .

- For all transactions T_k in T that have no < T_k commit> record or < T_k abort> record in the log, execute $\text{undo}(T_k)$.
- For all transactions T_k in T such that either the record < T_k commit> or the record < T_k abort> appears in the log, execute $\text{redo}(T_k)$.

Note that we need only examine the part of the log starting with the last checkpoint log record to find the set of transactions T , and to find out whether a commit or abort record occurs in the log for each transaction in T .

As an illustration, consider the set of transactions $\{T_0, T_1, \dots, T_{100}\}$. Suppose that the most recent checkpoint took place during the execution of transaction T_{67} and T_{69} , while T_{68} and all transactions with subscripts lower than 67 completed before the checkpoint. Thus, only transactions $T_{67}, T_{69}, \dots, T_{100}$ need to be considered during the recovery scheme. Each of them needs to be redone if it has completed (that is, either committed or aborted); otherwise, it was incomplete, and needs to be undone.

Consider the set of transactions L in a checkpoint log record. For each transaction T_i in L , log records of the transaction that occur prior to the checkpoint log record may be needed to undo the transaction, in case it does not commit. However, all log records prior to the earliest of the < T_i start> log records, among transactions T_i in L , are not needed once the checkpoint has completed. These log records can be erased whenever the database system needs to reclaim the space occupied by these records.

The requirement that transactions must not perform any updates to buffer blocks or to the log during checkpointing can be bothersome, since transaction processing has to halt while a checkpoint is in progress. A **fuzzy checkpoint** is a checkpoint where transactions are allowed to perform updates even while buffer blocks are being written out.

12.6 Recovery Algorithm

Until now, in discussing recovery, we have identified transactions that need to be redone and those that need to be undone, but we have not given a precise algorithm for performing these actions. We are now ready to present the full recovery algorithm using log records for recovery from transaction failure and a combination of the most recent checkpoint and log records to recover from a system crash.

The recovery algorithm described in this section requires that a data item that has been updated by an uncommitted transaction cannot be modified by any other transaction, until the first transaction has either committed or aborted.

12.6.1 Transaction Rollback

First consider transaction rollback during normal operation (that is, not during recovery from a system crash). Rollback of a transaction T_i is performed as follows:

1. The log is scanned backward, and for each log record of T_i of the form $\langle T_i, X_j, V1, V2 \rangle$ that is found:
 - a. The value $V1$ is written to data item X_j , and
 - b. A special redo-only log record $\langle T_i, X_j, V1 \rangle$ is written to the log, where $V1$ is the value being restored to data item X_j during the rollback. These log records are sometimes called **compensation log records**. Such records do not need undo information, since we never need to undo such an undo operation. We shall explain later how they are used.
2. Once the log record $\langle T_i \text{ start} \rangle$ is found the backward scan is stopped, and a log record $\langle T_i \text{ abort} \rangle$ is written to the log.

Observe that every update action performed by the transaction or on behalf of the transaction, including actions taken to restore data items to their old value, have now been recorded in the log.

12.6.2 Recovery After a System Crash

Recovery actions, when the database system is restarted after a crash, take place in two phases:

1. In the **redo phase**, the system replays updates of *all* transactions by scanning the log forward from the last checkpoint. The log records that are replayed include log records for transactions that were rolled back before system crash, and those that

had not committed when the system crash occurred. This phase also determines all transactions that were incomplete at the time of the crash, and must therefore be rolled back. Such incomplete transactions would either have been active at the time of the checkpoint, and thus would appear in the transaction list in the checkpoint record, or would have started later; further, such incomplete transactions would have neither a $\langle Ti \text{ abort} \rangle$ nor a $\langle Ti \text{ commit} \rangle$ record in the log.

The specific steps taken while scanning the log are as follows:

- a. The list of transactions to be rolled back, undo-list, is initially set to the list L in the $\langle \text{checkpoint } L \rangle$ log record.
- b. Whenever a normal log record of the form $\langle Ti, Xj, V1, V2 \rangle$, or a redo-only log record of the form $\langle Ti, Xj, V2 \rangle$ is encountered, the operation is redone; that is, the value $V2$ is written to data item Xj .
- c. Whenever a log record of the form $\langle Ti \text{ start} \rangle$ is found, Ti is added to undo-list.
- d. Whenever a log record of the form $\langle Ti \text{ abort} \rangle$ or $\langle Ti \text{ commit} \rangle$ is found, Ti is removed from undo-list.

At the end of the redo phase, undo-list contains the list of all transactions that are incomplete, that is, they neither committed nor completed rollback before the crash.

2. In the **undo phase**, the system rolls back all transactions in the undo-list. It performs rollback by scanning the log backward from the end.

- a. Whenever it finds a log record belonging to a transaction in the undo list, it performs undo actions just as if the log record had been found during the rollback of a failed transaction.
- b. When the system finds a $\langle Ti \text{ start} \rangle$ log record for a transaction Ti in undo-list, it writes a $\langle Ti \text{ abort} \rangle$ log record to the log, and removes Ti from undo-list.
- c. The undo phase terminates once undo-list becomes empty, that is, the system has found $\langle Ti \text{ start} \rangle$ log records for all transactions that were initially in undo-list.

After the undo phase of recovery terminates, normal transaction processing can resume.

Observe that the redo phase replays every log record since the most recent checkpoint record. In other words, this phase of restart recovery repeats all the update

actions that were executed after the checkpoint, and whose log records reached the stable log. The actions include actions of incomplete transactions and the actions carried out to rollback failed transactions. The actions are repeated in the same order in which they were originally carried out; hence, this process is called **repeating history**. Although it may appear wasteful, repeating history even for failed transactions simplifies recovery schemes.

Figure 12.14 shows an example of actions logged during normal operation, and actions performed during failure recovery. In the log shown in the figure, transaction T_1 had committed, and transaction T_0 had been completely rolled back, before the system crashed. Observe how the value of data item B is restored during the rollback of T_0 . Observe also the checkpoint record, with the list of active transactions containing T_0 and T_1 .

When recovering from a crash, in the redo phase, the system performs a redo of all operations after the last checkpoint record. In this phase, the list undo-list initially contains T_0 and T_1 ; T_1 is removed first when its commit log record is found, while T_2 is added when its start log record is found. Transaction T_0 is removed from undo-list when its abort log record is found, leaving only T_2 in undo-list.

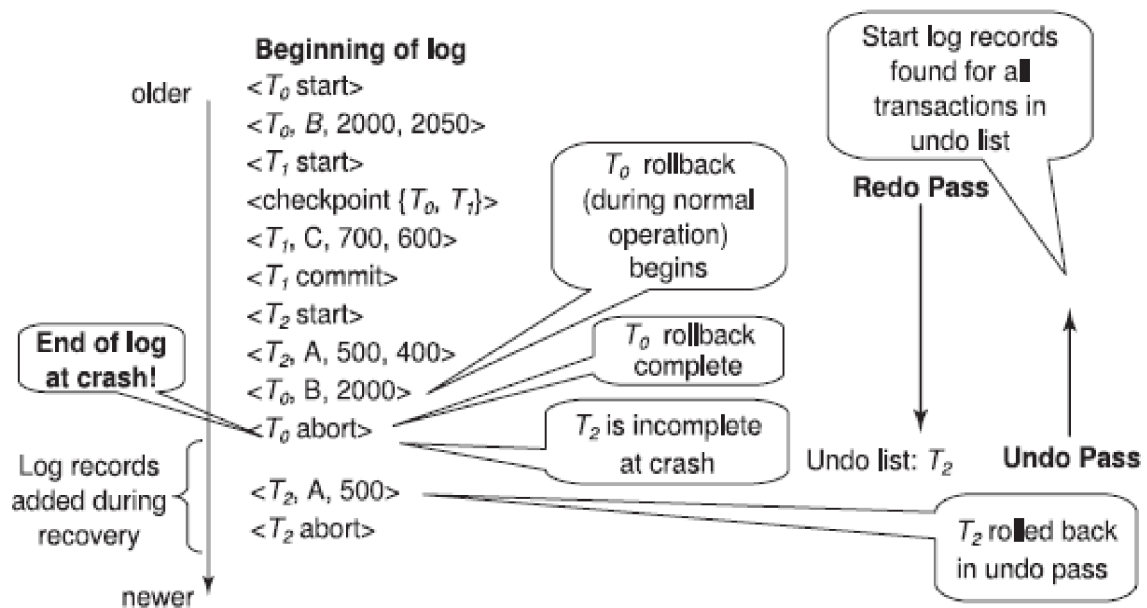


Figure 12.14 Example of logged actions, and actions during recovery.

The undo phase scans the log backwards from the end, and when it finds a log record of T_2 updating A , the old value of A is restored, and a redo-only log record written to the log. When the start record for T_2 is found, an abort record is added for T_2 . Since undo-list contains no more transactions, the undo phase terminates, completing recovery.

12.7 Buffer Management

12.7.1 Log Record Buffering

So far, we have assumed that every log record is output to stable storage at the time it is created. This assumption imposes a high overhead on system execution for several reasons: Typically, output to stable storage is in units of blocks. In most cases, a log record is much smaller than a block. Thus, the output of each log record translates to a much larger output at the physical level.

The cost of outputting a block to stable storage is sufficiently high that it is desirable to output multiple log records at once. To do so, we write log records to a log buffer in main memory, where they stay temporarily until they are output to stable storage. Multiple log records can be gathered in the log buffer and output to stable storage in a single output operation. The order of log records in the stable storage must be exactly the same as the order in which they were written to the log buffer.

As a result of log buffering, a log record may reside in only main memory (volatile storage) for a considerable time before it is output to stable storage. Since such log records are lost if the system crashes, we must impose additional requirements on the recovery techniques to ensure transaction atomicity:

- Transaction T_i enters the commit state after the $\langle T_i \text{ commit} \rangle$ log record has been output to stable storage.
- Before the $\langle T_i \text{ commit} \rangle$ log record can be output to stable storage, all log records pertaining to transaction T_i must have been output to stable storage.
- Before a block of data in main memory can be output to the database (in nonvolatile storage), all log records pertaining to data in that block must have been output to stable storage.

This rule is called the **write-ahead logging (WAL)** rule. (Strictly speaking, the WAL rule requires only that the undo information in the log has been output to stable storage, and it permits the redo information to be written later. The difference is relevant in systems where undo information and redo information are stored in separate log records.)

The three rules state situations in which certain log records *must* have been output to stable storage. There is no problem resulting from the output of log records *earlier* than necessary. Thus, when the system finds it necessary to output a log record to stable storage, it outputs an entire block of log records, if there are enough log records in main

memory to fill a block. If there are insufficient log records to fill the block, all log records in main memory are combined into a partially full block and are output to stable storage.

Writing the buffered log to disk is sometimes referred to as a **log force**.

12.7.2 Database Buffering

The systems stores the database in nonvolatile storage (disk), and brings blocks of data into main memory as needed. Since main memory is typically much smaller than the entire database, it may be necessary to overwrite a block $B1$ in main memory when another block $B2$ needs to be brought into memory. If $B1$ has been modified, $B1$ must be output prior to the input of $B2$. As discussed, this storage hierarchy is similar to the standard operating-system concept of *virtual memory*.

One might expect that transactions would force-output all modified blocks to disk when they commit. Such a policy is called the **force** policy. The alternative, the **no-force** policy, allows a transaction to commit even if it has modified some blocks that have not yet been written back to disk. All the recovery algorithms described in this chapter work correctly even with the no-force policy. The no force policy allows faster commit of transactions; moreover it allows multiple updates to accumulate on a block before it is output to stable storage, which can reduce the number of output operations greatly for frequently updated blocks. As a result, the standard approach taken by most systems is the no-force policy.

Similarly, one might expect that blocks modified by a transaction that is still active should not be written to disk. This policy is called the **no-steal** policy. The alternative, the **steal** policy, allows the system to write modified blocks to disk even if the transactions that made those modifications have not all committed. As long as the write-ahead logging rule is followed, all the recovery algorithms we study in the work correctly even with the steal policy. Further, the no-steal policy does not work with transactions that perform a large number of updates, since the buffer may get filled with updated pages that cannot be evicted to disk, and the transaction cannot then proceed. As a result, the standard approach taken by most systems is the steal policy.

To illustrate the need for the write-ahead logging requirement, consider our banking example with transactions $T0$ and $T1$. Suppose that the state of the log is:

< $T0$ start>

< $T0$, A , 1000, 950>

and that transaction $T0$ issues a read(B). Assume that the block on which B resides is not in main memory, and that main memory is full. Suppose that the block on which A

resides is chosen to be output to disk. If the system outputs this block to disk and then a crash occurs, the values in the database for accounts *A*, *B*, and *C* are \$950, \$2000, and \$700, respectively. This database state is inconsistent. However, because of the WAL requirements, the log record:

<*T0*, *A*, 1000, 950>

must be output to stable storage prior to output of the block on which *A* resides. The system can use the log record during recovery to bring the database back to a consistent state.

When a block *B1* is to be output to disk, all log records pertaining to data in *B1* must be output to stable storage before *B1* is output. It is important that no writes to the block *B1* be in progress while the block is being output, since such a write could violate the write-ahead logging rule. We can ensure that there are no writes in progress by using a special means of locking:

- Before a transaction performs a write on a data item, it acquires an exclusive lock on the block in which the data item resides. The lock is released immediately after the update has been performed.
- The following sequence of actions is taken when a block is to be output:
 - * Obtain an exclusive lock on the block, to ensure that no transaction is performing a write on the block.
 - * Output log records to stable storage until all log records pertaining to block *B1* have been output.
 - * Output block *B1* to disk.
 - * Release the lock once the block output has completed.

Locks on buffer blocks are unrelated to locks used for concurrency-control of transactions, and releasing them in a non-two-phase manner does not have any implications on transaction serializability. These locks, and other similar locks that are held for a short duration, are often referred to as **latches**.

Locks on buffer blocks can also be used to ensure that buffer blocks are not updated, and log records are not generated, while a checkpoint is in progress. This restriction may be enforced by acquiring exclusive locks on all buffer blocks, as well as an exclusive lock on the log, before the checkpoint operation is performed. These locks can be released as soon as the checkpoint operation has completed.

Database systems usually have a process that continually cycles through the buffer blocks, outputting modified buffer blocks back to disk. The above locking protocol must of course be followed when the blocks are output. As a result of continuous output of modified blocks, the number of **dirty blocks** in the buffer, that is, blocks that have been modified in the buffer but have not been subsequently output, is minimized. Thus, the number of blocks that have to be output during a checkpoint is minimized; further, when a block needs to be evicted from the buffer it is likely that there will be a non-dirty block available for eviction, allowing the input to proceed immediately instead of waiting for an output to complete.

12.7.3 Operating System Role in Buffer Management

We can manage the database buffer by using one of two approaches:

1. The database system reserves part of main memory to serve as a buffer that it, rather than the operating system, manages. The database system manages data-block transfer in accordance with the requirements.

This approach has the drawback of limiting flexibility in the use of main memory. The buffer must be kept small enough that other applications have sufficient main memory available for their needs. However, even when the other applications are not running, the database will not be able to make use of all the available memory. Likewise, non-database applications may not use that part of main memory reserved for the database buffer, even if some of the pages in the database buffer are not being used.

2. The database system implements its buffer within the virtual memory provided by the operating system. Since the operating system knows about the memory requirements of all processes in the system, ideally it should be in charge of deciding what buffer blocks must be force-output to disk, and when. But, to ensure the write-ahead logging requirements, the operating system should not write out the database buffer pages itself, but instead should request the database system to force-output the buffer blocks. The database system in turn would force-output the buffer blocks to the database, after writing relevant log records to stable storage.

Unfortunately, almost all current-generation operating systems retain complete control of virtual memory. The operating system reserves space on disk for storing virtual-memory pages that are not currently in main memory; this space is called **swap space**. If the operating system decides to output a block B_x , that block is output to the swap space on disk, and there is no way for the database system to get control of the output of buffer blocks.

Therefore, if the database buffer is in virtual memory, transfers between database files and the buffer in virtual memory must be managed by the database system, which enforces the write-ahead logging requirements that we discussed.

This approach may result in extra output of data to disk. If a block Bx is output by the operating system, that block is not output to the database. Instead, it is output to the swap space for the operating system's virtual memory. When the database system needs to output Bx , the operating system may need first to input Bx from its swap space. Thus, instead of a single output of Bx , there may be two outputs of Bx (one by the operating system, and one by the database system) and one extra input of Bx .

Although both approaches suffer from some drawbacks, one or the other must be chosen unless the operating system is designed to support the requirements of database logging.

12.7.3.1 Fuzzy Checkpointing

The checkpointing technique requires that all updates to the database be temporarily suspended while the checkpoint is in progress. If the number of pages in the buffer is large, a checkpoint may take a long time to finish, which can result in an unacceptable interruption in processing of transactions.

To avoid such interruptions, the checkpointing technique can be modified to permit updates to start once the checkpoint record has been written, but before the modified buffer blocks are written to disk. The checkpoint thus generated is a **fuzzy checkpoint**.

Since pages are output to disk only after the checkpoint record has been written, it is possible that the system could crash before all pages are written. Thus, a checkpoint on disk may be incomplete. One way to deal with incomplete checkpoints is this: The location in the log of the checkpoint record of the last completed checkpoint is stored in a fixed position, last-checkpoint, on disk. The system does not update this information when it writes the checkpoint record. Instead, before it writes the checkpoint record, it creates a list of all modified buffer blocks. The last-checkpoint information is updated only after all buffer blocks in the list of modified buffer blocks have been output to disk.

Even with fuzzy checkpointing, a buffer block must not be updated while it is being output to disk, although other buffer blocks may be updated concurrently. The write-ahead log protocol must be followed so that (undo) log records pertaining to a block are on stable storage before the block is output.

12.8 Summary

When several transactions execute concurrently in the database, the consistency of data may no longer be preserved. It is necessary for the system to control the interaction among the concurrent transactions, and this control is achieved through one of a variety of mechanisms called *concurrency-control* schemes.

- To ensure serializability, we can use various concurrency-control schemes. All these schemes either delay an operation or abort the transaction that issued the operation. The most common ones are locking protocols, time stamp ordering schemes, validation techniques, and multiversion schemes.
- A locking protocol is a set of rules that state when a transaction may lock and unlock each of the data items in the database.
- A computer system, like any other mechanical or electrical device, is subject to failure. There are a variety of causes of such failure, including disk crash, power failure, and software errors. In each of these cases, information concerning the database system is lost.
- In addition to system failures, transactions may also fail for various reasons, such as violation of integrity constraints or deadlocks.
- An integral part of a database system is a recovery scheme that is responsible for the detection of failures and for the restoration of the database to a state that existed before the occurrence of the failure.
- The various types of storage in a computer are volatile storage, nonvolatile storage, and stable storage. Data in volatile storage, such as in RAM, are lost when the computer crashes. Data in nonvolatile storage, such as disk, are not lost when the computer crashes, but may occasionally be lost because of failures such as disk crashes. Data in stable storage are never lost.
- Stable storage that must be accessible online is approximated with mirrored disks, or other forms of RAID, which provide redundant data storage. Offline, or archival, stable storage may consist of multiple tape copies of data stored **in a physically secure location**.

12.9 Lesson End Activities

1. Explain why log records for transactions on the undo-list must be processed in reverse order, whereas redo is performed in a forward direction.
2. Suppose a transaction deletes a record, and the free space generated thus is allocated to a record inserted by another transaction, even before the first transaction commits.
 - a. What problem can occur if the first transaction needs to be rolled back?
 - b. Would this problem be an issue if page-level locking is used instead of tuple-level locking?
 - c. Suggest how to solve this problem while supporting tuple-level locking, by logging post-commit actions in special log records, and executing them after commit. Make sure your scheme ensures that such actions are performed exactly once.
3. Consider the following two transactions:

T34: read(*A*);

read(*B*);

if *A* = 0 **then** *B* := *B* + 1;

write(*B*).

T35: read(*B*);

read(*A*);

if *B* = 0 **then** *A* := *A* + 1;

write(*A*).

Add lock and unlock instructions to transactions *T31* and *T32*, so that they observe the two-phase locking protocol. Can the execution of these transactions result in a deadlock?

12.10 Keywords

- Concurrency control
- Lock types
 - * Shared-mode (S) lock
 - * Exclusive-mode (X) lock

- Lock
 - * Compatibility
 - * Request
 - * Wait
 - * Grant
- Deadlock
- Starvation
- Locking protocol
- Legal schedule
- Two-phase locking protocol
- Recovery scheme
- Failure classification
 - * Transaction failure
 - * Logical error
 - * System error
 - * System crash
 - * Data-transfer failure
- Fail-stop assumption
- Disk failure
- Storage types
 - * Volatile storage
 - * Nonvolatile storage
 - * Stable storage
- Blocks
 - * Physical blocks
 - * Buffer blocks
- Disk buffer
- Force-output

- Log-based recovery
- Log
- Log records
- Update log record
- Deferred modification
- Immediate modification
- Uncommitted modifications
- Checkpoints
- Recovery algorithm

12.11 Questions for Discussion

1. Explain concurrency Control.
2. Discuss lock based protocols.
3. How to deadlock handling and prevention.
4. Explain deadlock detection and Recovery.
5. Explain recovery System.
6. Discuss the storage structure and types.
7. Explain buffer Management.

12.12 Suggested Reading

“Database System Concepts” -5th Edition by Silberchatz. Korth. Sudarshan., TMH.

Lesson - 13

DATABASE SYSTEM ARCHITECTURE

CONTENTS

13.0 Aim and Objectives

13.1 Introduction

13.2 Database System Architecture

13.2.1 Centralized and Client Server Architecture

13.2.1.1 Centralized System

13.2.1.2 Client Server System

13.2.2 Server System Architecture

13.2.2.1 Transaction Servers

13.2.2.2 Data Server

13.2.2.3 Cloud-Based Servers

13.2.3 Parallel Systems

13.2.3.1 Speedup and Scaleup

13.2.3.2 Interconnection Network

13.2.3.3 Parallel Database Architecture

13.2.4 Network Types

13.2.4.1 LAN

13.2.4.2 WAN

13.3 Summary

13.4 Lesson End Activities

13.5 Keywords

13.6 Questions for Discussion

13.7 Suggested Reading

13.0 AIM AND OBJECTIVES

After studying this lesson, you should be able to:

- The architectures of database systems running on server systems, which are used in centralized and client–server architectures.
- Describes how various actions of a database, in particular query processing, can be implemented to exploit parallel processing.
- How to store data, how to ensure atomicity of transactions that execute at multiple sites, how to perform concurrency control, and how to provide high availability in the presence of failures. a Cloud-based data storage systems, distributed query processing and directory systems are also described in this chapter

13.1 Introduction

The architecture of a database system is greatly influenced by the underlying computer system on which it runs, in particular by such aspects of computer architecture as networking, parallelism, and distribution:

- Networking of computers allows some tasks to be executed on a server system and some tasks to be executed on client systems. This division of work has led to *client–server database systems*.
- Parallel processing within a computer system allows database-system activities to be speeded up, allowing faster response to transactions, as well as more transactions per second. Queries can be processed in a way that exploits the parallelism offered by the underlying computer system. The need for parallel query processing has led to *parallel database systems*.
- Distributing data across sites in an organization allows those data to reside where they are generated or most needed, but still to be accessible from other sites and from other departments. Keeping multiple copies of the database across different sites also allows large organizations to continue their database operations even when one site is affected by a natural disaster, such as flood, fire, or earthquake. *Distributed database systems* handle geographically or administratively distributed data spread across multiple database systems.

We study the architecture of database systems in this chapter, starting with the traditional centralized systems, and covering client–server, parallel, and distributed database systems.

13.2 Database System Architecture

13.2.1 Centralized and Client Server Architecture

Centralized database systems are those that run on a single computer system and do not interact with other computer systems. Such database systems span a range from single -user database systems running on personal computers to high-performance database systems running on high-end server systems. Client –server systems, on the other hand, have functionality split between a server system and multiple client systems.

13.2.1.1 Centralized Systems

A modern, general-purpose computer system consists of one to a few processors and a number of device controllers that are connected through a common bus that provides access to shared memory (Figure 13.1). The processors have local cache memories that store local copies of parts of the memory, to speed up access to data. Each processor may have several independent **cores**, each of which can execute a separate instruction stream. Each device controller is in charge of a specific type of device (for example, a disk drive, an audio device, or a video display). The processors and the device controllers can execute concurrently, competing for memory access. Cache memory reduces the contention for memory access, since it reduces the number of times that the processor needs to access the shared memory.

We distinguish two ways in which computers are used: as single-user systems and as multiuser systems. Personal computers and workstations fall into the first category. A typical **single-user system** is a desktop unit used by a single person, usually with only one processor and one or two hard disks, and usually only one person using the machine at a time. A typical **multiuser system**, on the other hand, has more disks and more memory and may have multiple processors. It serves a large number of users who are connected to the system remotely.

Database systems designed for use by single users usually do not provide many of the facilities that a multiuser database provides. In particular, they may not support concurrency control, which is not required when only a single user can generate updates. Provisions for crash recovery in such systems are either absent or primitive—for example, they may consist of simply making a backup of the database before any update. Some such systems do not support SQL, and they provide a simpler query language, such as a variant of QBE. In contrast,

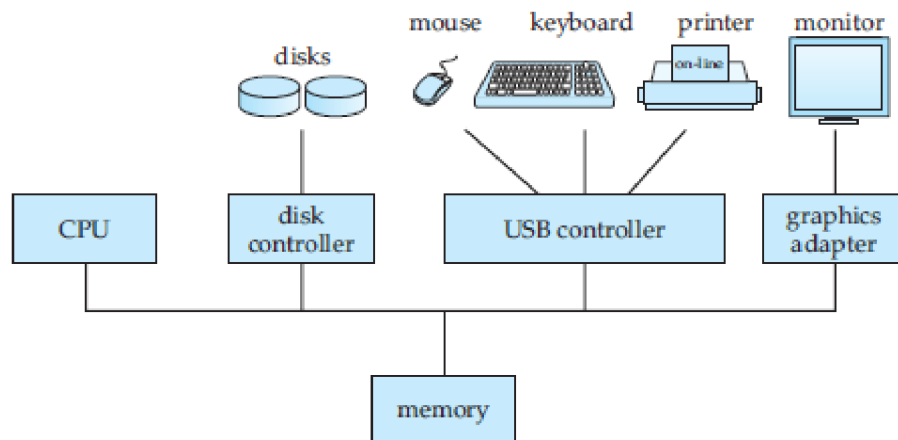


Figure 13.1 A centralized computer system.

database systems designed for multiuser systems support the full transactional features that we have studied earlier.

Although most general-purpose computer systems in use today have multiple processors, they have **coarse-granularity parallelism**, with only a few processors (about two to four, typically), all sharing the main memory. Databases running on such machines usually do not attempt to partition a single query among the processors; instead, they run each query on a single processor, allowing multiple queries to run concurrently. Thus, such systems support a higher through put, ie. they allow a greater number of transactions to run per second, although individual transactions do not run any faster.

Databases designed for single-processor machines already provide multitasking, allowing multiple processes to run on the same processor in a time-shared manner, giving a view to the user of multiple processes running in parallel. Thus, coarse-granularity parallel machines logically appear to be identical to single processor machines, and database systems designed for time-shared machines can be easily adapted to run on them.

In contrast, machines with **fine-granularity parallelism** have a large number of processors, and database systems running on such machines attempt to parallelize single tasks (queries, for example) submitted by users.

13.2.1.2 Client–Server Systems

As personal computers became faster, more powerful, and cheaper, there was a shift away from the centralized system architecture. Personal computers supplanted terminals connected to centralized systems. Correspondingly, personal computers assumed the user-interface functionality that used to be handled directly by the centralized

systems. As a result, centralized systems today act as **server systems** that satisfy requests generated by *client systems*. Figure 13.2 shows the general structure of a client–server system.

Functionality provided by database systems can be broadly divided into two parts—the front end and the back end. The back end manages access structures, query evaluation and optimization, concurrency control, and recovery. The front end of a database system consists of tools such as the SQL user interface, forms interfaces, report generation tools, and data mining and analysis tools (see Figure 13.3). The interface between the front end and the back end is through SQL, or through an application program.

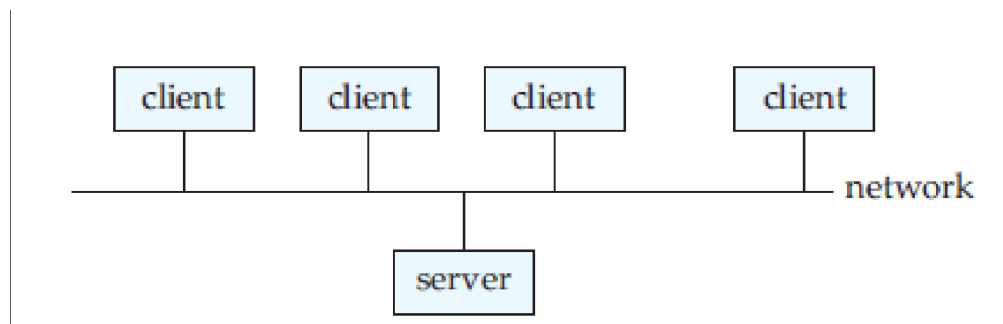


Figure 13.2 General structure of a client–server system.

Standards such as *ODBC* and *JDBC*, which were developed to interface clients with servers. Any client that uses the ODBC or JDBC interface can connect to any server that provides the interface.

Certain application programs, such as spreadsheets and statistical-analysis packages, use the client–server interface directly to access data from a back-end server. In effect, they provide front ends specialized for particular tasks.

Systems that deal with large numbers of users adopt a three-tier architecture, where the front end is a Web browser that talks to an application server. The application server, in effect, acts as a client to the database server.

Some transaction-processing systems provide a **transactional remote procedure call** interface to connect clients with a server. These calls appear like ordinary procedure calls to the programmer, but all the remote procedure calls from a client are enclosed in a single transaction at the server end. Thus, if the transaction aborts, the server can undo the effects of the individual remote procedure calls.

13.2.2 Server System Architectures

Server systems can be broadly categorized as transaction servers and data servers.

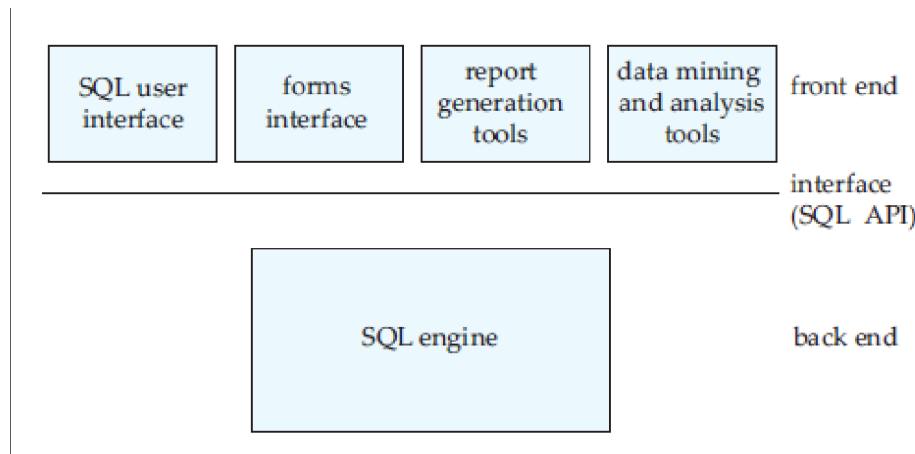


Figure 13.3 Front-end and back-end functionality.

- **Transaction-server** systems, also called **query-server** systems, provide an interface to which clients can send requests to perform an action, in response to which they execute the action and send back results to the client. Usually, client machines ship transactions to the server systems, where those transactions are executed, and results are shipped back to clients that are in charge of displaying the data. Requests may be specified by using SQL, or through a specialized application program interface.
- **Data-server systems** allow clients to interact with the servers by making requests to read or update data, in units such as files or pages. For example, file servers provide a file-system interface where clients can create, update, read, and delete files. Data servers for database systems offer much more functionality; they support units of data—such as pages, tuples, or objects—that are smaller than a file. They provide indexing facilities for data, and provide transaction facilities so that the data are never left in an inconsistent state if a client machine or process fails. Of these, the transaction-server architecture is by far the more widely used architecture.

13.2.2.1 Transaction Servers

Atypical transaction-server system today consists of multiple processes accessing data in shared memory, as in Figure 13.4. The processes that form part of the database system include:

- **Server processes:** These are processes that receive user queries (transactions), execute them, and send the results back. The queries may be submitted to the server processes from a user interface, or from a user process running embedded SQL, or via JDBC, ODBC, or similar protocols. Some database systems use a separate process for each user session, and a few use a single database process for all user sessions, but with multiple threads so that multiple queries can execute concurrently. (A **thread** is like a process, but multiple threads execute as part of the same process, and all threads within a process run in the same virtual-memory space. Multiple threads within a process can execute concurrently.) Many database systems use a hybrid architecture, with multiple processes, each one running multiple threads.
- **Lock manager process:** This process implements lock manager functionality, which includes lock grant, lock release, and deadlock detection.
- **Database writer process:** There are one or more processes that output modified buffer blocks back to disk on a continuous basis.
- **Log writer process:** This process outputs log records from the log record buffer to stable storage. Server processes simply add log records to the log

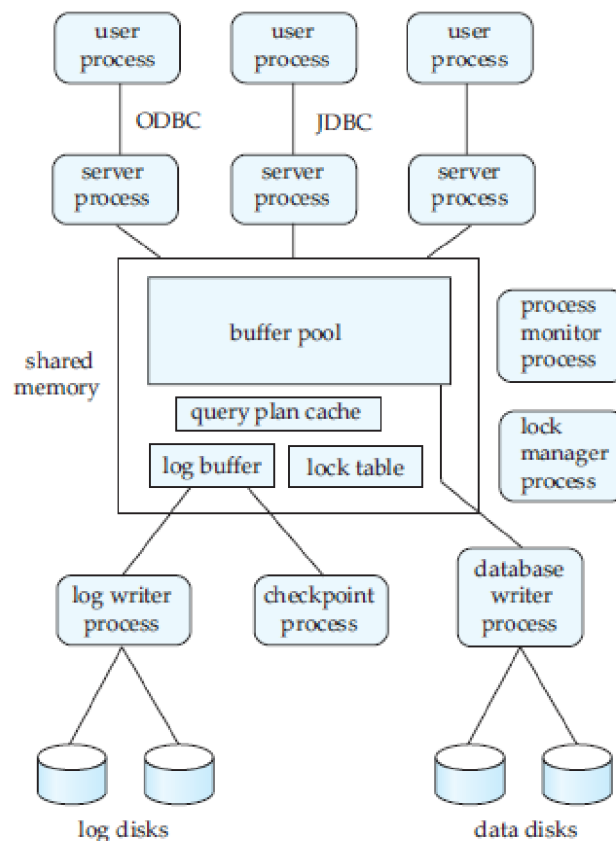


Figure 13.4 Shared memory and process structure.

record buffer in shared memory, and if a log force is required, they request the log writer process to output log records.

- **Checkpoint process:** This process performs periodic checkpoints.
- **Process monitor process:** This process monitors other processes, and if any of them fails, it takes recovery actions for the process, such as aborting any transaction being executed by the failed process, and then restarting the process.

The shared memory contains all shared data, such as:

- Buffer pool.
- Lock table.
- Log buffer, containing log records waiting to be output to the log on stable storage.
- Cached query plans, which can be reused if the same query is submitted again.

All database processes can access the data in shared memory. Since multiple processes may read or perform updates on data structures in shared memory, there must be a mechanism to ensure that a data structure is modified by at most one process at a time, and no process is reading a data structure while it is being written by others. Such **mutual exclusion** can be implemented by means of operating system functions called semaphores. Alternative implementations, with less overhead, use special **atomic instructions** supported by the computer hardware; one type of atomic instruction tests a memory location and sets it to 1 atomically. Further implementation details of mutual exclusion can be found in any standard operating system textbook. The mutual exclusion mechanisms are also used to implement latches.

To avoid the overhead of message passing, in many database systems, server processes implement locking by directly updating the lock table (which is in shared memory), instead of sending lock request messages to a lock manager process. The lock request procedure executes the actions that the lock manager process would take on getting a lock request.

- Since multiple server processes may access shared memory, mutual exclusion must be ensured on the lock table.
- If a lock cannot be obtained immediately because of a lock conflict, the lock request code may monitor the lock table to check when the lock has been granted. The lock release code updates the lock table to note which process has been granted the lock.

To avoid repeated checks on the lock table, operating system semaphores can be used by the lock request code to wait for a lock grant notification. The lock release code must then use the semaphore mechanism to notify waiting transactions that their locks have been granted.

Even if the system handles lock requests through shared memory, it still uses the lock manager process for deadlock detection.

13.2.2.2 Data Servers

Data-server systems are used in local-area networks, where there is a high-speed connection between the clients and the server, the client machines are comparable in processing power to the server machine, and the tasks to be executed are computation intensive. In such an environment, it makes sense to ship data to client machines, to perform all processing at the client machine (which may take a while), and then to ship the data back to the server machine. Note that this architecture requires full back-end functionality at the clients. Data-server architectures have been particularly popular in object-oriented database systems .

Interesting issues arise in such an architecture, since the time cost of communication between the client and the server is high compared to that of a local memory reference (milliseconds, versus less than 100 nanoseconds):

- **Page shipping versus item shipping.** The unit of communication for data can be of coarse granularity, such as a page, or fine granularity, such as a tuple (or an object, in the context of object-oriented database systems). We use the term **item** to refer to both tuples and objects.

If the unit of communication is a single item, the overhead of message passing is high compared to the amount of data transmitted. Instead, when an item is requested, it makes sense also to send back other items that are likely to be used in the near future. Fetching items even before they are requested is called **prefetching**. Page shipping can be considered a form of prefetching if multiple items reside on a page, since all the items in the page are shipped when a process desires to access a single item in the page.

- **Adaptive lock granularity.** Locks are usually granted by the server for the data items that it ships to the client machines. A disadvantage of page shipping is that client machines may be granted locks of too coarse a granularity—a lock on a page implicitly locks all items contained in the page. Even if the client is not accessing some items in the page, it has implicitly acquired locks on all prefetched items. Other

client machines that require locks on those items may be blocked unnecessarily. Techniques for lock **de-escalation** have been proposed where the server can request its clients to transfer back locks on prefetched items. If the client machine does not need a prefetched item, it can transfer locks on the item back to the server, and the locks can then be allocated to other clients.

- **Data caching.** Data that are shipped to a client on behalf of a transaction can be **cached** at the client, even after the transaction completes, if sufficient storage space is available. Successive transactions at the same client may be able to make use of the cached data. However, **cache coherency** is an issue: Even if a transaction finds cached data, it must make sure that those data are up to date, since they may have been updated by a different client after they were cached. Thus, a message must still be exchanged with the server to check validity of the data, and to acquire a lock on the data.
- **Lock caching.** If the use of data is mostly partitioned among the clients, with clients rarely requesting data that are also requested by other clients, locks can also be cached at the client machine. Suppose that a client finds a data item in the cache, and that it also finds the lock required for an access to the data item in the cache. Then, the access can proceed without any communication with the server. However, the server must keep track of cached locks; if a client requests a lock from the server, the server must **call back** all conflicting locks on the data item from any other client machines that have cached the locks. The task becomes more complicated when machine failures are taken into account. This technique differs from lock de-escalation in that lock caching takes place across transactions; otherwise, the two techniques are similar.

13.2.2.3 Cloud-Based Servers

Servers are usually owned by the enterprise providing the service, but there is an increasing trend for service providers to rely at least in part upon servers that are owned by a “third party” that is neither the client nor the service provider.

One model for using third-party servers is to outsource the entire service to another company that hosts the service on its own computers using its own software. This allows the service provider to ignore most details of technology and focus on the marketing of the service.

Another model for using third-party servers is **cloud computing**, in which the service provider runs its own software, but runs it on computers provided by another company.

Under this model, the third party does not provide any of the application software; it provides only a collection of machines. These machines are not “real” machines, but rather simulated by software that allows a single real computer to simulate several independent computers. Such simulated machines are called **virtual machines**. The service provider runs its software (possibly including a database system) on these virtual machines. A major advantage of cloud computing is that the service provider can add machines as needed to meet demand and release them at times of light load. This can prove to be highly cost-effective in terms of both money and energy.

A third model uses a cloud computing service as a data server; such *cloud-based data storage* systems. Database applications using cloud-based storage may run on the same cloud (that is, the same set of machines), or on another cloud. The bibliographical references provide more information about cloud-computing systems.

13.2.3 Parallel Systems

Parallel systems improve processing and I/O speeds by using multiple processors and disks in parallel. Parallel machines are becoming increasingly common, making the study of parallel database systems correspondingly more important. The driving force behind parallel database systems is the demands of applications that have to query extremely large databases (of the order of terabytes—that is, 10^{12} bytes) or that have to process an extremely large number of transactions per second (of the order of thousands of transactions per second). Centralized and client–server database systems are not powerful enough to handle such applications.

In parallel processing, many operations are performed simultaneously, as opposed to serial processing, in which the computational steps are performed sequentially. A **coarse-grain** parallel machine consists of a small number of powerful processors; a **massively parallel** or **fine-grain parallel** machine uses thousands of smaller processors. Virtually all high-end machines today offer some degree of coarse-grain parallelism: at least two or four processors. Massively parallel computers can be distinguished from the coarse-grain parallel machines by the much larger degree of parallelism that they support. Parallel computers with hundreds of processors and disks are available commercially.

There are two main measures of performance of a database system: (1) **throughput**, the number of tasks that can be completed in a given time interval, and (2) **response time**, the amount of time it takes to complete a single task from the time it is submitted. A system that processes a large number of small transactions can improve throughput by processing

many transactions in parallel. A system that processes large transactions can improve response time as well as throughput by performing subtasks of each transaction in parallel.

13.2.3.1 Speedup and Scaleup

Two important issues in studying parallelism are speedup and scaleup. Running a given task in less time by increasing the degree of parallelism is called **speedup**. Handling larger tasks by increasing the degree of parallelism is called **scaleup**.

Consider a database application running on a parallel system with a certain number of processors and disks. Now suppose that we increase the size of the system by increasing the number of processors, disks, and other components of the system. The goal is to process the task in time inversely proportional to the number of processors and disks allocated. Suppose that the execution time of a task on the larger machine is T_L , and that the execution time of the same task on the smaller machine is T_S . The speedup due to parallelism is defined as T_S/T_L . The parallel system is said to demonstrate **linear speedup** if the speedup is N when the larger system has N times the resources (processors, disk, and so on) of the smaller system. If the speedup is less than N , the system is said to demonstrate **sublinear speedup**. Figure 13.5 illustrates linear and sublinear speedup.

Scaleup relates to the ability to process larger tasks in the same amount of time by providing more resources. Let Q be a task, and let Q/N be a task that is N times bigger than Q . Suppose that the execution time of task Q on a given machine

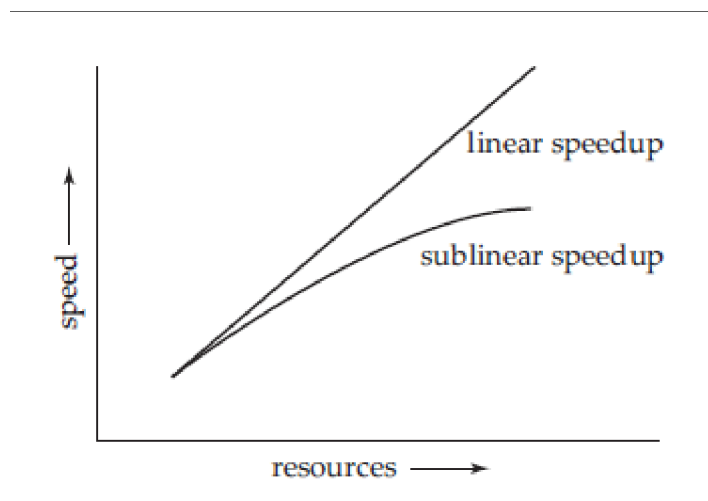


Figure 13.5 Speedup with increasing resources.

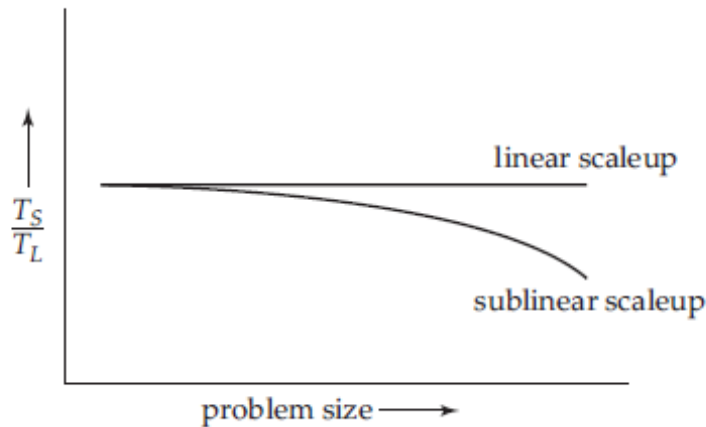


Figure 13.6 Scaleup with increasing problem size and resources.

M_S is T_S , and the execution time of task QN on a parallel machine M_L , which is N times larger than M_S , is T_L . The scaleup is then defined as T_S/T_L . The parallel system ML is said to demonstrate **linear scaleup** on task Q if $T_L = T_S$. If $T_L > T_S$, the system is said to demonstrate **sublinear scaleup**. Figure 13.6 illustrates linear and sublinear scaleups (where the resources increase in proportion to problem size). There are two kinds of scaleup that are relevant in parallel database systems, depending on how the size of the task is measured:

- In **batch scaleup**, the size of the database increases, and the tasks are large jobs whose runtime depends on the size of the database. An example of such a task is a scan of a relation whose size is proportional to the size of the database. Thus, the size of the database is the measure of the size of the problem. Batch scaleup also applies in scientific applications, such as executing a query at an N -times finer resolution or performing an N -times longer simulation.
- In **transaction scaleup**, the rate at which transactions are submitted to the database increases and the size of the database increases proportionally to the transaction rate. This kind of scaleup is what is relevant in transaction processing systems where the transactions are small updates—for example, a deposit or withdrawal from an account—and transaction rates grow as more accounts are created. Such transaction processing is especially well adapted for parallel execution, since transactions can run concurrently and independently on separate processors, and each transaction takes roughly the same amount of time, even if the database grows.

Scaleup is usually the more important metric for measuring efficiency of parallel database systems. The goal of parallelism in database systems is usually to make sure

that the database system can continue to perform at an acceptable speed, even as the size of the database and the number of transactions increases. Increasing the capacity of the system by increasing the parallelism provides a smoother path for growth for an enterprise than does replacing a centralized system with a faster machine (even assuming that such a machine exists). However, we must also look at absolute performance numbers when using scaleup measures; a machine that scales up linearly may perform worse than a machine that scales less than linearly, simply because the latter machine is much faster to start off with.

A number of factors work against efficient parallel operation and can diminish both speedup and scaleup.

- **Start-up costs.** There is a start-up cost associated with initiating a single process. In a parallel operation consisting of thousands of processes, the *start-up time* may overshadow the actual processing time, affecting speedup adversely.
- **Interference.** Since processes executing in a parallel system often access shared resources, a slowdown may result from the *interference* of each new process as it competes with existing processes for commonly held resources, such as a system bus, or shared disks, or even locks. Both speedup and scaleup are affected by this phenomenon.
- **Skew.** By breaking down a single task into a number of parallel steps, we reduce the size of the average step. Nonetheless, the service time for the single slowest step will determine the service time for the task as a whole. It is often difficult to divide a task into exactly equal-sized parts, and the way that the sizes are distributed is therefore *skewed*. For example, if a task of size 100 is divided into 10 parts, and the division is skewed, there may be some tasks of size less than 10 and some tasks of size more than 10; if even one task happens to be of size 20, the speedup obtained by running the tasks in parallel is only five, instead of ten as we would have hoped.

13.2.3.2 Interconnection Networks

Parallel systems consist of a set of components (processors, memory, and disks) that can communicate with each other via an **interconnection network**. Figure 8.7 shows three commonly used types of interconnection networks:

- **Bus.** All the system components can send data on and receive data from a single communication bus. This type of interconnection is shown in Figure 13.7a. The bus could be an Ethernet or a parallel interconnect. Bus architectures work well for small numbers of processors. However, they do not scale well with increasing parallelism, since the bus can handle communication from only one component at a time.
- **Mesh.** The components are nodes in a grid, and each component connects to all its adjacent components in the grid. In a two-dimensional mesh each node connects to four adjacent nodes, while in a three-dimensional mesh each node connects to six adjacent nodes. Figure 13.7b shows a two-dimensional mesh.

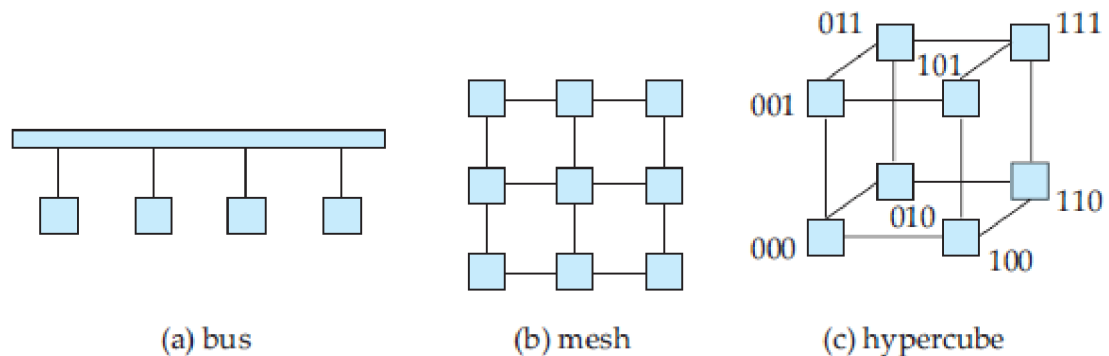


Figure 13.7 Interconnection networks.

Nodes that are not directly connected can communicate with one another by routing messages via a sequence of intermediate nodes that are directly connected to one another. The number of communication links grows as the number of components grows, and the communication capacity of a mesh therefore scales better with increasing parallelism.

- **Hypercube.** The components are numbered in binary, and a component is connected to another if the binary representations of their numbers differ in exactly one bit. Thus, each of the n components is connected to $\log(n)$ other components. Figure 13.7c shows a hypercube with eight nodes. In a hypercube interconnection, a message from a component can reach any other component by going through at most $\log(n)$ links. In contrast, in a mesh architecture a component may be $2(\sqrt{n} - 1)$ links away from some of the other components (or $\sqrt{n}$ links away, if the mesh interconnection wraps around at the edges of the grid). Thus communication delays in a hypercube are significantly lower than in a mesh.

13.2.3.3 Parallel Database Architectures

There are several architectural models for parallel machines. Among the most prominent ones are those in Figure 13.8 (in the figure, M denotes memory, P denotes a processor, and disks are shown as cylinders):

- **Shared memory.** All the processors share a common memory (Figure 13.8a).
- **Shared disk.** All the processors share a common set of disks (Figure 13.8b). Shared-disk systems are sometimes called **clusters**.
- **Shared nothing.** The processors share neither a common memory nor common disk (Figure 13.8c).
- **Hierarchical.** This model is a hybrid of the preceding three architectures (Figure 13.8d).

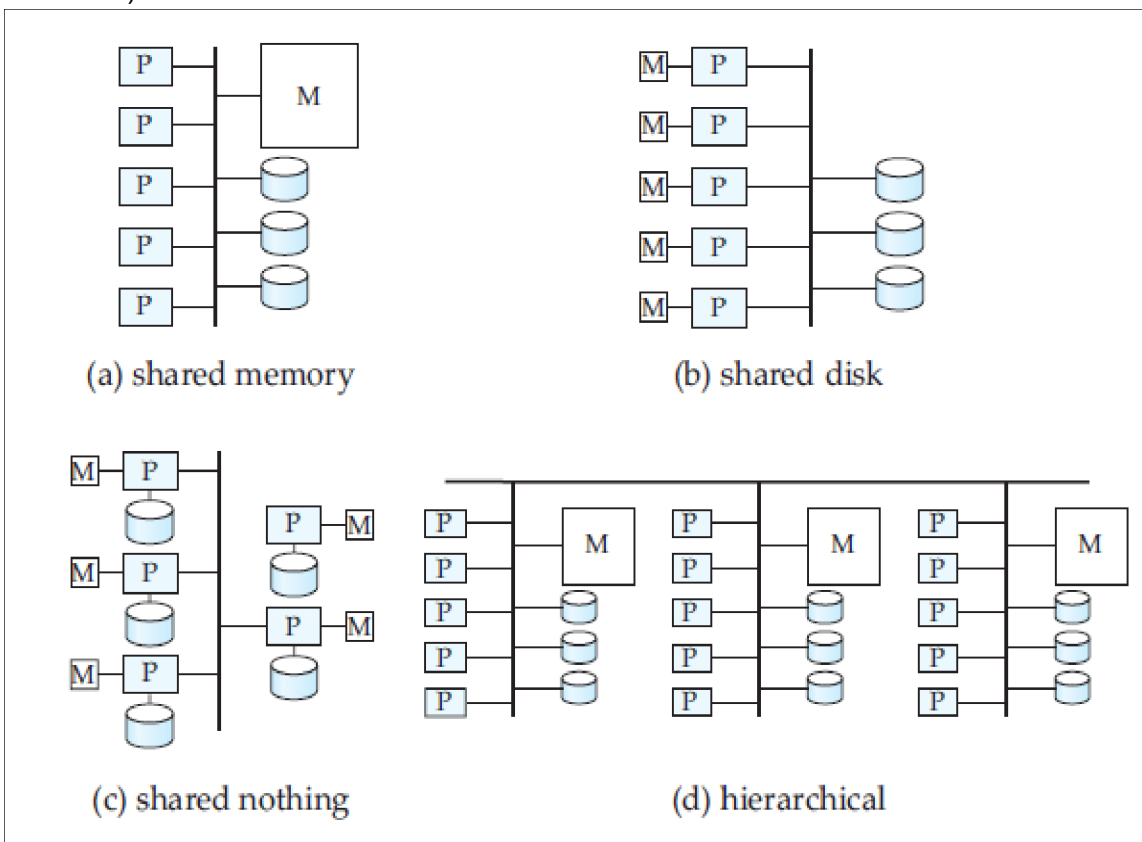


Figure 13.8 Parallel database architectures

Techniques used to speed up transaction processing on data-server systems, such as data and lock caching and lock de-escalation, can also be used in shared-disk parallel databases as well as in shared-nothing parallel databases. In fact, they are very important for efficient transaction processing in such systems.

Shared Memory

In a **shared-memory** architecture, the processors and disks have access to a common memory, typically via a bus or through an interconnection network. The benefit of shared memory is extremely efficient communication between processors—data in shared memory can be accessed by any processor without being moved with software. A processor can send messages to other processors much faster by using memory writes (which usually take less than a microsecond) than by sending a message through a communication mechanism. The downside of shared-memory machines is that the architecture is not scalable beyond 32 or 64 processors because the bus or the interconnection network becomes a bottleneck (since it is shared by all processors). Adding more processors does not help after a point, since the processors will spend most of their time waiting for their turn on the bus to access memory.

Shared-memory architectures usually have large memory caches at each processor, so that referencing of the shared memory is avoided whenever possible. However, at least some of the data will not be in the cache, and accesses will have to go to the shared memory. Moreover, the caches need to be kept coherent; that is, if a processor performs a write to a memory location, the data in that memory location should be either updated at or removed from any processor where the data are cached. Maintaining cache coherency becomes an increasing overhead with increasing numbers of processors. Consequently, shared-memory machines are not capable of scaling up beyond a point; current shared-memory machines cannot support more than 64 processors.

Shared Disk

In the **shared-disk** model, all processors can access all disks directly via an interconnection network, but the processors have private memories. There are two advantages of this architecture over a shared-memory architecture. First, since each processor has its own memory, the memory bus is not a bottleneck. Second, it offers a cheap way to provide a degree of **fault tolerance**: If a processor (or its memory) fails, the other processors can take over its tasks, since the database is resident on disks that are accessible from all processors. We can make the disk subsystem itself fault tolerant by using a RAID architecture. The shared-disk architecture has found acceptance in many applications.

The main problem with a shared-disk system is again scalability. Although the memory bus is no longer a bottleneck, the interconnection to the disk subsystem is now a bottleneck; it is particularly so in a situation where the database makes a large number of accesses to

disks. Compared to shared-memory systems, shared-disk systems can scale to a somewhat larger number of processors, but communication across processors is slower (up to a few milliseconds in the absence of special-purpose hardware for communication), since it has to go through a communication network.

Shared Nothing

In a **shared-nothing** system, each node of the machine consists of a processor, memory, and one or more disks. The processors at one node may communicate with another processor at another node by a high-speed interconnection network. A node functions as the server for the data on the disk or disks that the node owns. Since local disk references are serviced by local disks at each processor, the shared-nothing model overcomes the disadvantage of requiring all I/O to go through a single interconnection network; only queries, accesses to nonlocal disks, and result relations pass through the network. Moreover, the interconnection networks for shared-nothing systems are usually designed to be scalable, so that their transmission capacity increases as more nodes are added. Consequently, shared-nothing architectures are more scalable and can easily support a large number of processors. The main drawbacks of shared-nothing systems are the costs of communication and of nonlocal disk access, which are higher than in a shared-memory or shared-disk architecture since sending data involves software interaction at both ends.

Hierarchical

The **hierarchical architecture** combines the characteristics of shared-memory, shared-disk, and shared-nothing architectures. At the top level, the system consists of nodes that are connected by an interconnection network and do not share disks or memory with one another. Thus, the top level is a shared-nothing architecture. Each node of the system could actually be a shared-memory system with a few processors. Alternatively, each node could be a shared-disk system, and each of the systems sharing a set of disks could be a shared-memory system. Thus, a system could be built as a hierarchy, with shared-memory architecture with a few processors at the base, and a shared-nothing architecture at the top, with possibly a shared-disk architecture in the middle. Figure 13.8d illustrates a hierarchical architecture with shared-memory nodes connected together in a shared-nothing architecture. Commercial parallel database systems today run on several of these architectures.

Attempts to reduce the complexity of programming such systems have yielded **distributed virtual-memory** architectures, where logically there is a single shared memory, but physically there are multiple disjoint memory systems; the virtual memory- mapping

hardware, coupled with system software, allows each processor to view the disjoint memories as a single virtual memory. Since access speeds differ, depending on whether the page is available locally or not, such an architecture is also referred to as a **nonuniform memory architecture (NUMA)**.

13.2.4 Network Types

Distributed databases and client–server systems are built around communication networks. There are basically two types of networks: **local-area networks** and **wide-area networks**. The main difference between the two is the way in which they are distributed geographically. In local-area networks, processors are distributed over small geographical areas, such as a single building or a number of adjacent buildings. In wide-area networks, on the other hand, a number of autonomous processors are distributed over a large geographical area (such as the United States or the entire world). These differences imply major variations in the speed and reliability of the communication network, and are reflected in the distributed operating-system design.

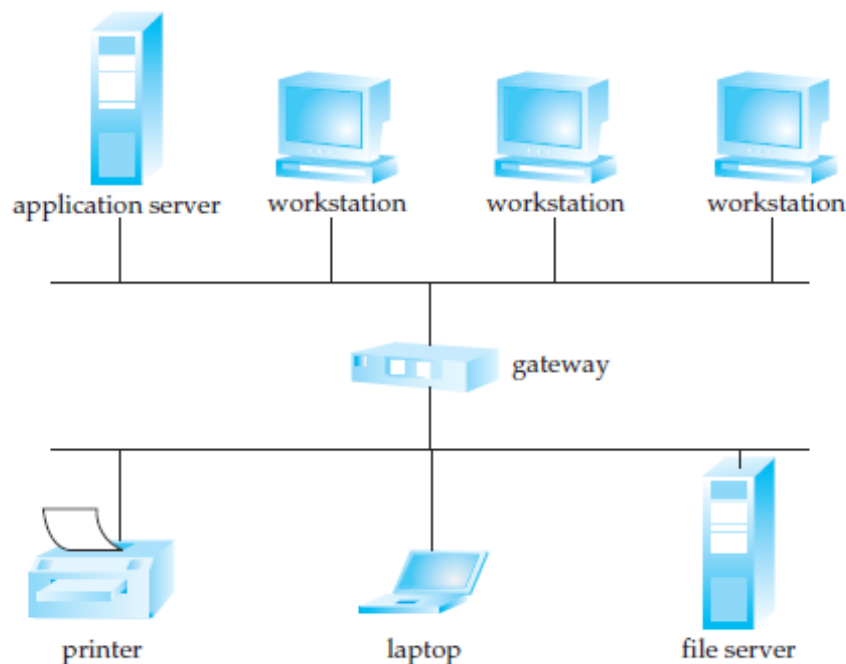


Figure 13.9 Local-area network.

13.2.4.1 Local-Area Networks

Local-area networks (LANs) (Figure 13.9) emerged in the early 1970s as a way for computers to communicate and to share data with one another. People recognized that, for many enterprises, numerous small computers, each with its own self-contained

applications, are more economical than a single large system. Because each small computer is likely to need access to a full complement of peripheral devices (such as disks and printers), and because some form of data sharing is likely to occur in a single enterprise, it was a natural step to connect these small systems into a network.

LANs are generally used in an office environment. All the sites in such systems are close to one another, so the communication links tend to have a higher speed and lower error rate than do their counterparts in wide-area networks. The most common links in a local-area network are twisted pair, coaxial cable, fiber optics, and wireless connections. Communication speeds range from tens of megabits per second (for wireless local-area networks), to 1 gigabit per second for Gigabit Ethernet. The most recent Ethernet standard is 10-gigabit Ethernet.

A **storage-area network (SAN)** is a special type of high-speed local-area network designed to connect large banks of storage devices (disks) to computers that use the data (see Figure 13.10).

Thus storage-area networks help build large-scale *shared-disk systems*. The motivation for using storage-area networks to connect multiple computers to large banks of storage devices is essentially the same as that for shared-disk databases, namely:

- Scalability by adding more computers.
- High availability, since data are still accessible even if a computer fails.

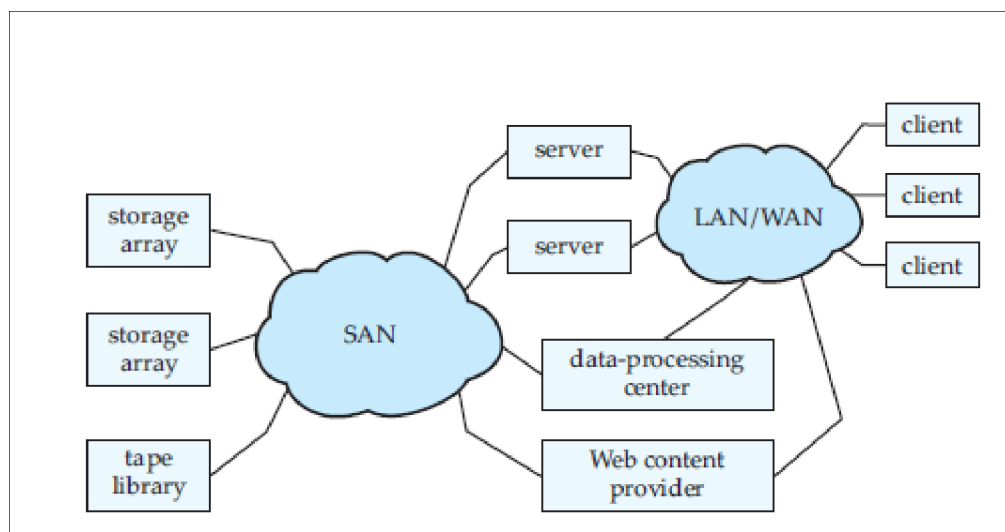


Figure 13.10 Storage-area network.

RAID organizations are used in the storage devices to ensure high availability of the data, permitting processing to continue even if individual disks fail. Storage area networks

are usually built with redundancy, such as multiple paths between nodes, so if a component such as a link or a connection to the network fails, the network continues to function.

13.2.4.2 Wide-Area Networks

Wide-area networks (WANs) emerged in the late 1960s, mainly as an academic research project to provide efficient communication among sites, allowing hardware and software to be shared conveniently and economically by a wide community of users. Systems that allowed remote terminals to be connected to a central computer via telephone lines were developed in the early 1960s, but they were not true WANs. The first WAN to be designed and developed was the *Arpanet*. Work on the Arpanet began in 1968. The Arpanet has grown from a four-site experimental network to a worldwide network of networks, the Internet, comprising hundreds of millions of computer systems. Typical links on the Internet are fiber-optic lines and, sometimes, satellite channels. Data rates for wide-area links typically range from a few megabits per second to hundreds of gigabits per second. The last link, to end user sites, has traditionally been the slowest link, using such technologies as *digital subscriber line* (DSL) technology (supporting a few megabits per second) or dial-up modem connections over land-based telephone lines (supporting up to 56 kilobits per second). Today, the last link is typically a cable modem or fiber optic connection (each supporting tens of megabits per second), or a wireless connection supporting several megabits per second.

In addition to limits on data rates, communication in a WAN must also contend with significant latency: a message may take up to a few hundred milliseconds to be delivered across the world, both due to speed of light delays, and due to queuing delays at a number of routers in the path of the message. Applications whose data and computing resources are distributed geographically have to be carefully designed to ensure latency does not affect system performance excessively.

WANs can be classified into two types:

- In discontinuous connection WANs, such as those based on mobile wireless connections, hosts are connected to the network only part of the time.
- In continuous connection WANs, such as the wired Internet, hosts are connected to the network at all times.

Networks that are not continuously connected typically do not allow transactions across sites, but may keep local copies of remote data, and refresh the copies periodically

(every night, for instance). For applications where consistency is not critical, such as sharing of documents, groupware systems such as Lotus Notes allow updates of remote data to be made locally, and the updates are then propagated back to the remote site periodically. There is a potential for conflicting updates at different sites, conflicts that have to be detected and resolved. A mechanism for detecting conflicting updates is described later, the resolution mechanism for conflicting updates is, however, application dependent.

13.3 Summary

- Centralized database systems run entirely on a single computer. With the growth of personal computers and local-area networking, the database frontend functionality has moved increasingly to clients, with server systems providing the back-end functionality. Client–server interface protocols have helped the growth of client–server database systems.
- Servers can be either transaction servers or data servers, although the use of transaction servers greatly exceeds the use of data servers for providing database services.
 - * Transaction servers have multiple processes, possibly running on multiple processors. So that these processes have access to common data, such as the database buffer, systems store such data in shared memory. In addition to processes that handle queries, there are system processes that carry out tasks such as lock and log management and checkpointing.
 - * Data-server systems supply raw data to clients. Such systems strive to minimize communication between clients and servers by caching data and locks at the clients. Parallel database systems use similar optimizations.
- Parallel database systems consist of multiple processors and multiple disks connected by a fast interconnection network. Speedup measures how much we can increase processing speed by increasing parallelism for a single transaction. Scaleup measures how well we can handle an increased number of transactions by increasing parallelism. Interference, skew, and start-up costs act as barriers to getting ideal speedup and scaleup.
- Parallel database architectures include the shared-memory, shared-disk, shared-nothing, and hierarchical architectures. These architectures have different trade-offs of scalability versus communication speed.

- A distributed database system is a collection of partially independent database systems that (ideally) share a common schema, and coordinate processing of transactions that access nonlocal data. The systems communicate with one another through a communication network.
- Local-area networks connect nodes that are distributed over small geographical areas, such as a single building or a few adjacent buildings. Wide-area networks connect nodes spread over a large geographical area. The Internet is the most extensively used wide-area network today.
- Storage-area networks are a special type of local-area network designed to provide fast interconnection between large banks of storage devices and multiple computers.

13.4 Lesson End Activities

- Instead of storing shared structures in shared memory, an alternative architecture would be to store them in the local memory of a special process, and access the shared data by interprocess communication with the process. What would be the drawback of such an architecture?
- In typical client–server systems the server machine is much more powerful than the clients; that is, its processor is faster, it may have multiple processors, and it has more memory and disk capacity. Consider instead a scenario where client and server machines have exactly the same power. Would it make sense to build a client–server system in such a scenario? Why? Which scenario would be better suited to a data-server architecture?
- Consider a database system based on a client–server architecture, with the server acting as a data server.
 - a. What is the effect of the speed of the interconnection between the client and the server on the choice between tuple and page shipping?
 - b. If page shipping is used, the cache of data at the client can be organized either as a tuple cache or a page cache. The page cache stores data in units of a page, while the tuple cache stores data in units of tuples. Assume tuples are smaller than pages. Describe one benefit of a tuple cache over a page cache.

13.5 Keywords

- Centralized systems
- Server systems

- Coarse-granularity parallelism
- Fine-granularity parallelism
- Database process structure
- Mutual exclusion
- Thread
- Server processes
 - * Lock manager process
 - * Database writer process
 - * Log writer process
 - * Checkpoint process
 - * Process monitor process
- Client–server systems
- Transaction server
- Query server
- Data server
 - * Prefetching
 - * De-escalation
 - * Data caching
 - * Cache coherency
 - * Lock caching
 - * Call back
- Parallel systems
- Throughput
- Response time
- Speedup
 - * Linear speedup
 - * Sublinear speedup
- Scaleup
 - * Linear scaleup

- * Sublinear scaleup
 - * Batch scaleup
 - * Transaction scaleup
- Start-up costs
- Interference
- Skew
- Interconnection networks
 - * Bus
 - * Mesh
 - * Hypercube
- Parallel database architectures
 - * Shared memory
 - * Shared disk (clusters)
 - * Shared nothing
 - * Hierarchical
- Fault tolerance
- Distributed virtual memory
- Non uniform memory architecture (NUMA)
- Distributed systems
- Distributed database
 - * Sites (nodes)
 - * Local transaction
 - * Global transaction
 - * Local autonomy
- Multi database systems
- Network types
 - * Local-area networks (LAN)
 - * Wide-area networks (WAN)
 - * Storage-area network (SAN)

13.6 Questions for Discussion

1. Discuss database System Architecture.
2. What is centralized and client Server Architecture?
3. What is parallel Systems?
4. Discuss the different types of network.

13.7 Suggested Reading

“Database System Concepts” -5th Edition by Silberchatz. Korth. Sudarshan., TMH.

Lesson - 14

DISTRIBUTED DATABASES

CONTENTS

14.0 Aim and Objectives

14.1 Introduction

14.2 Distributed Databases

14.2.1 Homogeneous and Heterogeneous Databases

14.2.2 Directory Systems

14.2.2.1 Directory Access Protocol

14.2.2.2 LDAP Data model

14.2.2.3 Data Manipulation

14.2.2.4 Distributed Directory Trees

14.3 Oracle

14.4 MSSQL Server

14.5 Summary

14.6 Lesson End Activities

14.7 Keywords

14.8 Questions for Discussion

14.9 Suggested Reading

14.0 AIM AND OBJECTIVES

After studying this lesson, you should be able to:

- Classifying distributed databases as homogeneous or heterogeneous, how to store data in a distributed database, outline a model for transaction processing in a distributed database.

- We describe how to implement atomic transactions in a distributed database by using special commit protocols. And we describe concurrency control in distributed databases.

14.1 Introduction

Unlike parallel systems, in which the processors are tightly coupled and constitute a single database system, a distributed database system consists of loosely coupled sites that share no physical components. Furthermore, the database systems that run on each site may have a substantial degree of mutual independence.

Each site may participate in the execution of transactions that access data at one site, or several sites. The main difference between centralized and distributed database systems is that, in the former, the data reside in one single location, whereas in the latter, the data reside in several locations. This distribution of data is the cause of many difficulties in transaction processing and query processing. In this chapter, we address these difficulties.

14.2 Distributed Databases

14.2.1 Homogeneous and Heterogeneous Databases

In a **homogeneous distributed database** system, all sites have identical database management system software, are aware of one another, and agree to cooperate in processing users' requests. In such a system, local sites surrender a portion of their autonomy in terms of their right to change schemas or database-management system software.

branch(branch name, branch city, assets)
account(account number, branch name, balance)
depositor(customer name, account number)

Figure 14.1 Banking database.

That software must also cooperate with other sites in exchanging information about transactions, to make transaction processing possible across multiple sites.

In contrast, in a heterogeneous distributed database, different sites may use different schemas, and different database-management system software. The sites may not be aware of one another, and they may provide only limited facilities for cooperation in transaction processing. The differences in schemas are often a major problem for query

processing, while the divergence in software becomes a hindrance for processing transactions that access multiple sites.

14.2.2 Directory Systems

Consider an organization that wishes to make data about its employees available to a variety of people in the organization; examples of the kinds of data include name, designation, employee-id, address, email address, phone number, fax number, and so on. In the precomputerization days, organizations would create physical directories of employees and distribute them across the organization. Even today, telephone companies create physical directories of customers.

In general, a directory is a listing of information about some class of objects such as persons. Directories can be used to find information about a specific object, or in the reverse direction to find objects that meet a certain requirement. In the world of physical telephone directories, directories that satisfy lookups in the forward direction are called **white pages**, while directories that satisfy lookups in the reverse direction are called **yellow pages**.

In today's networked world, the need for directories is still present and, if anything, even more important. However, directories today need to be available over a computer network, rather than in a physical (paper) form.

14.2.2.1 Directory Access Protocols

Directory information can be made available through Web interfaces, as many organizations, and phone companies in particular, do. Such interfaces are good for humans. However, programs too need to access directory information. Directories can be used for storing other types of information, much like file system directories. For instance, Web browsers can store personal bookmarks and other browser settings in a directory system. A user can thus access the same settings from multiple locations, such as at home and at work, without having to share a file system.

Several **directory access protocols** have been developed to provide a standardized way of accessing data in a directory. The most widely used among them today is the **Lightweight Directory Access Protocol (LDAP)**.

Obviously all the types of data in our examples can be stored without much trouble in a database system, and accessed through protocols such as JDBC or ODBC. The question then is, why come up with a specialized protocol for accessing directory information? There are at least two answers to the question.

- First, directory access protocols are simplified protocols that cater to a limited type of access to data. They evolved in parallel with the database access protocols.
- Second, and more important, directory systems provide a simple mechanism to name objects in a hierarchical fashion, similar to file system directory names, which can be used in a distributed directory system to specify what information is stored in each of the directory servers. For example, a particular directory server may store information for Bell Laboratories employees in Murray Hill, while another may store information for Bell Laboratories employees in Bangalore, giving both sites autonomy in controlling their local data. The directory access protocol can be used to obtain data from both directories across a network. More important, the directory system can be set up to automatically forward queries made at one site to the other site, without user intervention.

For these reasons, several organizations have directory systems to make organizational information available online through a directory access protocol. Information in an organizational directory can be used for a variety of purposes, such as to find addresses, phone numbers, or email addresses of people, to find which departments people are in, and to track department hierarchies. Directories are also used to authenticate users: applications can collect authentication information such as passwords from users and authenticate them using the directory.

As may be expected, several directory implementations find it beneficial to use relational databases to store data, instead of creating special-purpose storage systems.

LDAP: Lightweight Directory Access Protocol

In general a directory system is implemented as one or more servers, which service multiple clients. Clients use the application programmer interface defined by the directory system to communicate with the directory servers. Directory access protocols also define a data model and access control.

The **X.500 directory access protocol**, defined by the International Organization for Standardization (ISO), is a standard for accessing directory information. However, the protocol is rather complex, and is not widely used. The **Lightweight Directory Access Protocol (LDAP)** provides many of the X.500 features, but with less complexity, and is widely used. In the rest of this section, we shall outline the data model and access protocol details of LDAP.

14.2.2.2 LDAP Data Model

In LDAP, directories store **entries**, which are similar to objects. Each entry must have a **distinguished name (DN)**, which uniquely identifies the entry. A DN is in turn made up of a sequence of **relative distinguished names (RDNs)**. For example, an entry may have the following distinguished name:

```
cn=Silberschatz, ou=Computer Science, o=Yale University, c=USA
```

As you can see, the distinguished name in this example is a combination of a name and (organizational) address, starting with a person's name, then giving the organizational unit (ou), the organization (o), and country (c). The order of the components of a distinguished name reflects the normal postal address order, rather than the reverse order used in specifying path names for files. The set of RDNs for a DN is defined by the schema of the directory system.

Entries can also have attributes. LDAP provides binary, string, and time types, and additionally the types tel for telephone numbers, and Postal Address for addresses (lines separated by a "\$" character). Unlike those in the relational model, attributes are multivalued by default, so it is possible to store multiple telephone numbers or addresses for an entry.

LDAP allows the definition of **object classes** with attribute names and types. Inheritance can be used in defining object classes. Moreover, entries can be specified to be of one or more object classes. It is not necessary that there be a single most-specific object class to which an entry belongs.

Entries are organized into a **directory information tree (DIT)**, according to their distinguished names. Entries at the leaf level of the tree usually represent specific objects. Entries that are internal nodes represent objects such as organizational units, organizations, or countries. The children of a node have a DN containing all the RDNs of the parent, and one or more additional RDNs. For instance, an internal node may have a DN c=USA, and all entries below it have the value USA for the RDN c.

The entire distinguished name need not be stored in an entry. The system can generate the distinguished name of an entry by traversing up the DIT from the entry, collecting the RDN=value components to create the full distinguished name.

Entries may have more than one distinguished name—for example, an entry for a person in more than one organization. To deal with such cases, the leaf level of a DIT can be an **alias**, which points to an entry in another branch of the tree.

14.2.2.3 Data Manipulation

Unlike SQL, LDAP does not define either a data-definition language or a data manipulation language. However, LDAP defines a network protocol for carrying out data definition and manipulation. Users of LDAP can either use an application programming interface or use tools provided by various vendors to perform data definition and manipulation. LDAP also defines a file format called **LDAP**.

Data Interchange Format (LDIF) that can be used for storing and exchanging information.

The querying mechanism in LDAP is very simple, consisting of just selections and projections, without any join. A query must specify the following:

- A base—that is, a node within a DIT—by giving its distinguished name (the path from the root to the node).
- A search condition, which can be a Boolean combination of conditions on individual attributes. Equality, matching by wild-card characters, and approximate equality (the exact definition of approximate equality is system dependent) are supported.
- A scope, which can be just the base, the base and its children, or the entire subtree beneath the base.
- Attributes to return.
- Limits on number of results and resource consumption.

The query can also specify whether to automatically dereference aliases; if alias dereferences are turned off, alias entries can be returned as answers.

One way of querying an LDAP data source is by using LDAP URLs. Examples of LDAP URLs are:

```
ldap://codex.cs.yale.edu/o=Yale University,c=USA
```

```
ldap://codex.cs.yale.edu/o=Yale University,c=USA??sub?cn=Silberschatz
```

The first URL returns all attributes of all entries at the server with organization being Yale University, and country being USA. The second URL executes a search query (selection) `cn=Silberschatz` on the subtree of the node with distinguished name `o=Yale University, c=USA`. The question marks in the URL separate different fields. The first field is the distinguished name, here `o=Yale University,c=USA`. The second field, the list of attributes

to return, is left empty, meaning return all attributes. The third attribute, sub, indicates that the entire subtree is to be searched. The last parameter is the search condition.

A second way of querying an LDAP directory is by using an application programming interface. Figure 14.2 shows a piece of C code used to connect to an LDAP server and run a query against the server. The code first opens a connection to an LDAP server by `ldap_open` and `ldap_bind`. It then executes a query by `ldap_search_s`. The arguments to `ldap_search_s` are the LDAP connection handle, the DN of the base from which the search should be done, the scope of the search, the search condition, the list of attributes to be returned, and an attribute called `attrs only`, which, if set to 1, would result in only the schema of the result being returned, without any actual tuples. The last argument is an output argument that returns the result of the search as an LDAP Message structure.

The first **for** loop iterates over and prints each entry in the result. Note that an entry may have multiple attributes, and the second **for** loop prints each attribute. Since attributes in LDAP may be multivalued, the third **for** loop prints each value of an attribute. The calls `ldap_msgfree` and `ldap_value_free` free memory that is allocated by the LDAP libraries.

```
#include <stdio.h>
#include <ldap.h>
main() {
    LDAP *ld;
    LDAPMessage *res, *entry;
    char *dn, *attr, *attrList[] = {"telephoneNumber", NULL};
    BerElement *ptr;
    int vals, i;
    ld = ldap_open("codex.cs.yale.edu", LDAP_PORT);
    ldap_simple_bind(ld, "avi", "avi-passwd");
    ldap_search_s(ld, "o=Yale University, c=USA", LDAP_SCOPE_SUBTREE,
        "cn=Silberschatz", attrList, /*attronly*/ 0, &res);
    printf("found %d entries", ldap_count_entries(ld, res));
    for (entry=ldap_first_entry(ld, res); entry != NULL;
        entry = ldap_next_entry(ld, entry))
    {
        dn = ldap_get_dn(ld, entry);
        printf("dn: %s", dn);
        ldap_memfree(dn);
        for (attr = ldap_first_attribute(ld, entry, &ptr);
            attr != NULL;
            attr = ldap_next_attribute(ld, entry, ptr))
        {
            printf("%s: ", attr);
            vals = ldap_get_values(ld, entry, attr);
            for (i=0; vals[i] != NULL; i++)
                printf("%s, ", vals[i]);
            ldap_value_free(vals);
        }
    }
    ldap_msgfree(res);
    ldap_unbind(ld);
}
```

Figure 14.2 Example of LDAP code in C.

Figure 9.2 does not show code for handling error conditions.

The LDAP API also contains functions to create, update, and delete entries, as well as other operations on the DIT. Each function call behaves like a separate transaction; LDAP does not support atomicity of multiple updates.

14.2.2.4 Distributed Directory Trees

Information about an organization may be split into multiple DITs, each of which stores information about some entries. The **suffix** of a DIT is a sequence of RDN=value pairs that identify what information the DIT stores; the pairs are concatenated to the rest of the distinguished name generated by traversing from the entry to the root. For instance, the suffix of a DIT may be o=Lucent, c=USA, while another may have the suffix o=Lucent, c=India. The DITs may be organizationally and geographically separated.

A node in a DIT may contain a **referral** to another node in another DIT; for instance, the organizational unit Bell Labs under o=Lucent, c=USA may have its own DIT, in which case the DIT for o=Lucent, c=USA would have a node ou=Bell Labs representing a referral to the DIT for Bell Labs.

Referrals are the key component that help organize a distributed collection of directories into an integrated system. When a server gets a query on a DIT, it may return a referral to the client, which then issues a query on the referenced DIT. Access to the referenced DIT is transparent, proceeding without the user's knowledge. Alternatively, the server itself may issue the query to the referred DIT and return the results along with locally computed results.

The hierarchical naming mechanism used by LDAP helps break up control of information across parts of an organization. The referral facility then helps integrate all the directories in an organization into a single virtual directory.

Although it is not an LDAP requirement, organizations often choose to break up information either by geography (for instance, an organization may maintain a directory for each site where the organization has a large presence) or by organizational structure (for instance, each organizational unit, such as department, maintains its own directory).

Many LDAP implementations support master–slave and multi master replication of DITs, although replication is not part of the current LDAP version 3 standard. Work on standardizing replication in LDAP is in progress.

14.3 Oracle

When Oracle was founded in 1977 as Software Development Laboratories by Larry

Ellison, Bob Miner, and Ed Oates, there were no commercial relational database products. The company, which was later renamed Oracle, set out to build a relational database management system as a commercial product, and became a pioneer of the RDBMS market and has held a leading position in this market ever since. Over the years, its product and service offerings have grown beyond the relational database server to include middleware and applications. In addition to products developed inside the company, Oracle's offerings include software that was originally developed in companies that Oracle acquired. Oracle's acquisitions have ranged from small companies to large, publicly traded ones, including Peoplesoft, Siebel, Hyperion, and BEA. As a result of these acquisitions, Oracle has a very broad portfolio of enterprise software products.

This chapter is focused on Oracle's main relational database server and closely related products. New versions of the products are being developed continually, so all product descriptions are subject to change. The feature set described here is based on the first release of Oracle11g, which is Oracle's flagship database product.

Database Design and Querying Tools

Oracle provides a variety of tools for database design, querying, report generation and data analysis, including OLAP. These tools, along with various other application development tools, are part of a portfolio of software products called Oracle Fusion Middleware. Products include both traditional tools using Oracle's PL/SQL programming language and newer ones based on Java/J2EE technologies. The software supports open standards such as SOAP, XML, BPEL, and UML.

Database and Application Design Tools

The Oracle Application Development Framework (ADF) is an end-to-end J2EEbased development framework for a Model-View-Control design pattern. In this framework, an application consists of multiple layers. The Model and Business Services layers handle the interaction with the data sources and contains the business logic. The Viewlayer handles the user interface, and the Controller layer handles the flow of the application and the interaction between the other layers.

The primary development tool for Oracle ADF is Oracle JDeveloper, which provides an integrated development environment with support for Java, XML, PHP, HTML, Javascript, BPEL, SQL, and PL/SQL development. It has built-in support for UML modeling.

Oracle Designer is a database design tool, which translates business logic and data flows into schema definitions and procedural scripts for application logic. It supports such modeling techniques as E-R diagrams, information engineering and object analysis and

design.

Oracle also has an application development tool for data warehousing, Oracle Warehouse Builder. Warehouse Builder is a tool for design and deployment of all aspects of a data warehouse, including schema design, data mapping and transformations, data load processing, and metadata management. Oracle Warehouse Builder supports both 3NF and star schemas and can also import designs from Oracle Designer. This tool, in conjunction with database features, such as external tables and table functions, typically eliminates the need for third-party extraction, transformation, and loading (ETL) tools.

Querying Tools

Oracle provides tools for ad hoc querying, report generation, and data analysis, including OLAP.

Oracle Business Intelligence Suite (OBI) is a comprehensive suite of tools sharing a common service-oriented architecture. Components include a Business Intelligence server and tools for ad hoc querying, dashboard generation, reporting, and alerting. The components share infrastructure and services for data access and metadata management and have a common security model and administration tool.

The component for ad hoc querying, Oracle BI Answers, is an interactive tool that presents the user with a logical view of the data hiding the details of the physical implementation. Objects available to the user are displayed graphically and the user can build a query with a point-and-click interface. This logical query is sent to the Oracle BI Server component, which then generates the physical query or queries. Multiple physical data sources are supported, and a query could combine data stored in relational databases, OLAP sources, and Excel spreadsheets. Results can be presented as charts, reports, pivot tables, or dashboards that are drillable and can be saved and later modified.

14.3.1 SQL Variations and Extensions

Oracle supports all core SQL:2003 features fully or partially, with the exception of features-and-conformance views. In addition, Oracle supports a large number of other language constructs, some of which conform with Optional Features of SQL Foundation:2003, while others are Oracle-specific in syntax or functionality.

Object-Relational Features

Oracle has extensive support for object-relational constructs, including:

- **Object types.** A single-inheritance model is supported for type hierarchies.
- **Collection types.** Oracle supports **varrays**, which are variable length arrays, and nested tables.
- **Object tables.** These are used to store objects while providing a relational view of the attributes of the objects.
- **Table functions.** These are functions that produce sets of rows as output, and can be used in the **from** clause of a query. Table functions in Oracle can be nested. If a table function is used to express some form of data transformation, nesting multiple functions allows multiple transformations to be expressed in a single statement.
- **Object views.** These provide a virtual object table view of data stored in a regular relational table. They allow data to be accessed or viewed in an objectoriented style even if the data are really stored in a traditional relational format.
- **Methods.** These can be written in PL/SQL, Java, or C.
- **User-defined aggregate functions.** These can be used in SQL statements in the same way as built-in functions such as **sum** and **count**.

Oracle XML DB

Oracle XML DB provides in-database storage for XML data and support for a broad set of XML functionality including XML Schema and XQuery. It is built on the **XML Type** abstract data type, which is treated as a native Oracle data type. XML DB provides several options for how data of this data type are stored in the database, including:

- Structured in object-relational format. This format is usually space efficient and allows the use of a variety of standard relational features, such as B-tree indices, but incurs some overhead when mapping XML documents to the storage format and back. It is mainly suitable for XML data that are highly structured and the mapping includes a manageable number of relational tables and joins.
- Unstructured as a text string. This representation does not require any mapping and provides high throughput when inserting or retrieving an entire XML document. However, it is usually not very space efficient and provides for less intelligent processing when operating on parts of an XML document.

- Binary XML storage. This representation is a post-parse, XML Schema-aware, binary format. It is more space efficient than unstructured storage and can handle operations against parts of an XML document. It is also better than the structured format at handling data that are highly unstructured, but may not always be as space efficient. This format may make the processing of XQuery statements less efficient than when the structured format is used.

Both the binary and unstructured representation can be indexed with a special type of index called **XMLIndex**. This type of index allows document fragments to be indexed based on their corresponding XPath expression. Storing XML data inside the database means that they get the benefit of Oracle's functionality in areas such as backup, recovery, security, and query processing. It allows for accessing relational data as part of doing XML processing as well as accessing XML data as part of doing SQL processing. Some very high-level features of XML DB include:

- Support for the XQuery language (W3C XQuery 1.0 Recommendation).
- An XSLT process that lets XSL transformations be performed inside the database.
- An XPath rewrite optimization that can speed up queries against data stored in object-relational representation. By translating an expression used in an XQuery into conditions directly on the object-relational columns, regular indices on these columns can be used to speed up query processing.

Procedural Languages

Oracle has two main procedural languages, PL/SQL and Java. PL/SQL was Oracle's original language for stored procedures and it has syntax similar to that used in the Ada language. Java is supported through a Java virtual machine inside the database engine. Oracle provides a package to encapsulate related procedures, functions, and variables into single units. Oracle supports SQLJ (SQL embedded in Java) and JDBC, and provides a tool to generate Java class definitions corresponding to user-defined database types.

Dimensions

Dimensional modeling is a commonly used design technique for relational star schemas as well as multidimensional databases. Oracle supports the creation of **dimensions** as metadata objects in order to support query processing against databases designed based on this technique. The metadata objects can be used to store information

about various kinds of attributes of a dimension, but perhaps more importantly, about hierarchical relationships.

OLAP

Oracle provides support for analytical database processing in several different ways. In addition to support for a rich set of analytical SQL constructs (cube, rollup, grouping sets, window functions, etc.), Oracle provides native multidimensional storage inside the relational database server. The multidimensional data structures allow for array-based access to the data, and, in the right circumstances, this type of access can be vastly more efficient than traditional relational access methods. Using these data structures as an integrated part of a relational database provides a choice of storing data in a relational or multidimensional format while still taking advantage of Oracle features in areas such as backup and recovery, security, and administration tools.

Oracle provides storage containers for multidimensional data known as analytic workspaces. An analytic workspace contains both the dimensional data and measures (or facts) of an OLAP cube and is stored inside an Oracle table. From a traditional relational perspective, a cube stored inside a table would be an opaque object where the data could not normally be interpreted directly in terms of the table's rows and columns. However, Oracle's OLAP server inside the database has the knowledge to interpret and access the data and makes it possible to give SQL access to it as if it had been stored in a regular table format. Hence, it is possible to store data in either a multidimensional format or a traditional relational format, depending on what is optimal, and still be able to join data stored in both types of representations in a single SQL query. A materialized view can use either representation.

In addition to Oracle's OLAP support inside its relational database, Oracle's product suite includes Essbase. Essbase is a widely used multidimensional database that came to be part of Oracle with the acquisition of Hyperion.

Triggers

Oracle provides several types of triggers and several options for when and how they are invoked. Triggers can be written in PL/SQL or Java or as C callouts. For triggers that execute on DML statements such as insert, update, and delete, Oracle supports row triggers and statement triggers. Row triggers execute once for every row that is affected (updated or deleted, for example) by the DML operation. A statement trigger is executed just once per statement. In each case, the trigger can be defined as either a before or after trigger,

depending on whether it is to be invoked before or after the DML operation is carried out. Oracle allows the creation of instead of triggers for views that cannot be subject to DML operations. Depending on the view definition, it may not be possible for Oracle to translate a DML statement on a view to modifications of the underlying base tables unambiguously. Hence, DML operations on views are subject to numerous restrictions. A user can create an instead of trigger on a view to specify manually what operations on the base tables are to be carried out in response to a DML operation on the view. Oracle executes the trigger instead of the DML operation and therefore provides a mechanism to circumvent the restrictions on DML operations against views.

Oracle also has triggers that execute on a variety of other events, such as database start-up or shutdown, server error messages, user logon or logoff, and DDL statements such as create, alter and drop statements.

14.3.2 Storage and Indexing

In Oracle parlance, a database consists of information stored in files and is accessed through an instance, which is a shared memory area and a set of processes that interact with the data in the files. The control file is a small file that contains some very high-level metadata required to start or operate an instance. The storage structure of the regular data and metadata is described in the next section.

Table Spaces

A database consists of one or more logical storage units called table spaces. Each table space, in turn, consists of one or more physical structures called data files. These may be either part of a file system or raw devices. Usually, an Oracle database has the following table spaces:

- The system and the auxiliary sysaux table spaces, which are always created. They contain the data dictionary tables and storage for triggers and stored procedures.
- Table spaces created to store user data. While user data can be stored in the system table space, it is often desirable to separate the user data from the system data. Usually, the decision about what other table spaces should be created is based on performance, availability, maintainability, and ease of administration. For example, having multiple table spaces can be useful for partial backup and recovery operations.
- The undo table space, which is used solely for storing undo information for transaction management and recovery.
- Temporary table spaces. Many database operations require sorting the data, and the

sort routine may have to store data temporarily on disk if the sort cannot be done in memory. Temporary table spaces are allocated for sorting and hashing to make the space management operations involved in spilling to disk more efficient.

Table spaces can also be used as a means of moving data between databases. For example, it is common to move data from a transactional system to a data warehouse at regular intervals. Oracle allows moving all the data in a table space from one system to the other by simply copying the data files and exporting and importing a small amount of data-dictionary metadata. These operations can be much faster than unloading the data from one database and then using a loader to insert it into the other. This Oracle feature is known as transportable table spaces.

Segments

The space in a table space is divided into units, called segments, each of which contains data for a specific data structure. There are four types of segments:

- Data segments. Each table in a table space has its own data segment where the table data are stored unless the table is partitioned; if so, there is one data segment per partition.
- Index segments. Each index in a table space has its own index segment, except for partitioned indices, which have one index segment per partition.
- Temporary segments. These are segments used when a sort operation needs to write data to disk or when data are inserted into a temporary table.
- Undo segments. These segments contain undo information so that an uncommitted transaction can be rolled back. These segments are automatically allocated in a special undo table space. They also play an important role in Oracle's concurrency control model and for database recovery. In older implementations of Oracle's undo management, the term "rollback segment" was used.

Below the level of segment, space is allocated at a level of granularity called an extent. Each extent consists of a set of contiguous database blocks. A database block is the lowest level of granularity at which Oracle performs disk I/O. A database block does not have to be the same as an operating system block in size, but should be a multiple thereof.

Oracle provides storage parameters that allow for detailed control of how space is allocated and managed, parameters such as:

- The size of a new extent that is to be allocated to provide room for rows that are

inserted into a table.

- The percentage of space utilization at which a database block is considered full and at which no more rows will be inserted into that block. (Leaving some free space in a block can allow the existing rows to grow in size through updates, without running out of space in the block.)

Tables

A standard table in Oracle is heap organized; that is, the storage location of a row in a table is not based on the values contained in the row, and is fixed when the row is inserted. However, if the table is partitioned, the content of the row affects the partition in which it is stored. There are several features and variations. Heap tables can optionally be compressed.

Oracle supports nested tables; that is, a table can have a column whose data type is another table. The nested table is not stored in line in the parent table, but is stored in a separate table.

Oracle supports temporary tables where the duration of the data is either the transaction in which the data are inserted, or the user session. The data are private to the session and are automatically removed at the end of its duration.

A cluster is another form of file organization for table data, where it is called multi table clustering. The use of the term “cluster ” in this context, should not be confused with other meanings of the word cluster, such as those relating to hardware architecture. In a cluster file organization, rows from different tables are stored together in the same block on the basis of some common columns. For example, a department table and an employee table could be clustered so that each row in the department table is stored together with all the employee rows for those employees who work in that department. The primary key/foreign key values are used to determine the storage location.

The cluster organization implies that a row belongs in a specific place; for example, a new employee row must be inserted with the other rows for the same department. Therefore, an index on the clustering column is mandatory. An alternative organization is a hash cluster. Here, Oracle computes the location of a row by applying a hash function to the value for the cluster column. The hash function maps the row to a specific block in the hash cluster. Since no index traversal is needed to access a row according to its cluster column value, this organization can save significant amounts of disk I/O.

Index-Organized Tables

In an index-organized table (IOT), records are stored in an Oracle B-tree index instead of in a heap, where it is called B+-tree file organization. An IOT requires that a unique key be identified for use as the index key. While an entry in a regular index contains the key value and row-id of the indexed row, an IOT replaces the row-id with the column values for the remaining columns of the row. Compared to storing the data in a regular heap table and creating an index on the key columns, using an IOT can improve both performance and space utilization. Consider looking up all the column values of a row, given its primary key value. For a heap table, that would require an index probe followed by a table access by row-id. For an IOT, only the index probe is necessary.

Secondary indices on nonkey columns of an index-organized table are different from indices on a regular heap table. In a heap table, each row has a fixed row-id that does not change. However, a B-tree is reorganized as it grows or shrinks when entries are inserted or deleted, and there is no guarantee that a row will stay in a fixed place inside an IOT. Hence, a secondary index on an IOT contains not normal row-ids, but logical row-ids instead. A logical row-id consists of two parts: a physical row-id corresponding to where the row was when the index was created or last rebuilt and a value for the unique key. The physical row-id is referred to as a “guess” since it could be incorrect if the row has been moved. If so, the other part of a logical row-id, the key value for the row, is used to access the row; however, this access is slower than if the guess had been correct, since it involves a traversal of the B-tree for the IOT from the root all the way to the leaf nodes, potentially incurring several disk I/Os. However, if a table is highly volatile and a large percentage of the guesses are likely to be wrong, it can be better to create the secondary index with only key values, since using an incorrect guess may result in a wasted disk I/O.

Compression

Oracle's compression feature allows data to be stored in a compressed format, something that can drastically reduce the amount of space needed to store the data and the number of I/O operations needed to retrieve it. Oracle's compression method is a lossless dictionary-based algorithm that compresses each block individually. All the information needed to uncompress a block is contained in that block itself. The algorithm works by replacing repeated occurrences of a value in that block with pointers to an entry for that value in a symbol table (or dictionary) in the block. Entries can be based on repeated values for individual columns or a combination of columns.

Oracle's original table compression generated the compressed block format as the data were bulk-loaded into a table and was mainly intended for data warehousing

environments. A newer OLTP compression feature supports compression in conjunction with regular DML operations as well. In the latter case, Oracle compresses blocks only after certain thresholds have been reached for how much data have been written into the block. As a result, only transactions that cause a threshold to be passed will occur any overhead for compressing a block.

Data Security

In addition to regular access control features such as passwords, user privileges, and user roles, Oracle supports several features to protect the data from unauthorized access, including:

- **Encryption.** Oracle can automatically store table data in an encrypted format and transparently encrypt and decrypt data using the AES or 3DES algorithms. Encryption can be enabled for an entire database or just for individual table columns. The main motivation for this feature is to protect sensitive data outside the normally protected environment, such as when backup media is sent to a remote location.
- **Database Vault.** This feature is aimed at providing a separation of duties for users with access to the database. A database administrator is a highly privileged user that typically can do almost anything with the database. However, it may be inappropriate or illegal to let that person access sensitive corporate financial data or personal information about other employees. Database vault includes a variety of mechanisms that can be used to restrict or monitor access to sensitive data by highly privileged database users.
- **Virtual Private Database,** allows additional predicates to be automatically added to the where clause of a query that accesses a given table or view. Typically, the feature would be used so that the additional predicate filters out all the rows that the user does not have the right to see. For example, two users could submit identical queries to find all the employee information in the entire employee table. However, if a policy exists that limits each user to seeing only the information for the employee number that matches the user ID, the automatically added Predicates will ensure that each query only returns the employee information for the user who submitted the query. Hence, each user will be left with the impression of accessing a virtual database that contains only a subset of the data of the physical database.

Indices

Oracle supports several different types of indices. The most commonly used type is

a B- tree index, created on one or multiple columns. Note that in the terminology of Oracle (as also in several other database systems) a B-tree index is what is referred to as a B+-tree index in previous chapter. Index entries have the following format: for an index on columns col1, col2, and col3, each row in the table where at least one of the columns has a non null value would result in the index entry:

`<col1><col2><col3><row-id>`

where `<coli>` denotes the value for column *i* and `<row-id>` is the row-id for the row. Oracle can optionally compress the prefix of the entry to save space. For example, if there are many repeated combinations of `<col1><col2>` values, the representation of each distinct `<col1><col2>` prefix can be shared between the entries that have that combination of values, rather than stored explicitly for each such entry. Prefix compression can lead to substantial space savings.

Bitmap Indices

Bitmap indices use a bitmap representation for index entries, which can lead to substantial space saving (and therefore disk I/O savings), when the indexed column has a moderate number of distinct values. Bitmap indices in Oracle use the same kind of B-tree structure to store the entries as a regular index. However, where a regular index on a column would have entries of the form `<col1><row-id>`, a bitmap index entry has the form:

`<col1><start row-id><end row-id><compressed bitmap>`

The bitmap conceptually represents the space of all possible rows in the table between the start and end row-id. The number of such possible rows in a block depends on how many rows can fit into a block, which is a function of the number of columns in the table and their data types. Each bit in the bitmap represents one such possible row in a block. If the column value of that row is that of the index entry, the bit is set to 1. If the row has some other value, or the row does not actually exist in the table, the bit is set to 0. (It is possible that the row does not actually exist because a table block may well have a smaller number of rows than the number that was calculated as the maximum possible.) If the difference is large, the result may be long strings of consecutive zeros in the bitmap, but the compression algorithm deals with such strings of zeros, so the negative effect is limited.

The compression algorithm is a variation of a compression technique called Byte-Aligned Bitmap Compression (BBC). Essentially, a section of the bitmap where the distance between two consecutive 1s is small enough is stored as verbatim bitmaps. If the distance between two 1s is sufficiently large—that is, there is a sufficient number of adjacent 0s

between them—a run length of 0s, that is, the number of 0s, is stored.

Bitmap indices allow multiple indices on the same table to be combined in the same access path if there are multiple conditions on indexed columns in the where clause of a query. Bitmaps from the different indices are retrieved and combined using Boolean operations corresponding to the conditions in the where clause. All Boolean operations are performed directly on the compressed representation of the bitmaps—no decompression is necessary—and the resulting (compressed) bitmap represents those rows that match all the logical conditions.

The ability to use the Boolean operations to combine multiple indices is not limited to bitmap indices. Oracle can convert row-ids to the compressed bitmap representation, so it can use a regular B-tree index anywhere in a Boolean tree of bitmap operation simply by putting a row-id-to-bitmap operator on top of the index access in the execution plan.

As a rule of thumb, bitmap indices tend to be more space efficient than regular B-tree indices if the number of distinct key values is less than half the number of rows in the table. For example, in a table with 1 million rows, an index on a column with less than 500,000 distinct values would probably be smaller if it were created as a bitmap index. For columns with a very small number of distinct values—for example, columns referring to properties such as country, state, gender, marital status, and various status flags—a bitmap index might require only a small fraction of the space of a regular B-tree index. Any such space advantage can also give rise to corresponding performance advantages in the form of fewer disk I/Os when the index is scanned.

Function-Based Indices

In addition to creating indices on one or multiple columns of a table, Oracle allows indices to be created on expressions that involve one or more columns, such as `col1 + col2` “ 5. For example, by creating an index on the expression `upper(name)`, where `upper` is a function that returns the uppercase version of a string, and `name` is a column, it is possible to do case-insensitive searches on the name column. In order to find all rows with name “van Gogh” efficiently, the condition:

`upper(name) = 'VAN GOGH'`

would be used in the where clause of the query. Oracle then matches the condition with the index definition and concludes that the index can be used to retrieve all the rows matching “van Gogh” regardless of how the name was capitalized when it was stored in the database. A function-based index can be created as either a bitmap or a B-tree index.

Join Indices

A join index is an index where the key columns are not in the table that is referenced by the row-ids in the index. Oracle supports bitmap join indices primarily for use with star schemas. For example, if there is a column for product names in a product dimension table, a bitmap join index on the fact table with this key column could be used to retrieve the fact table rows that correspond to a product with a specific name, although the name is not stored in the fact table. How the rows in the fact and dimension tables correspond is based on a join condition that is specified when the index is created, and becomes part of the index metadata. When a query is processed, the optimizer will look for the same join condition in the where clause of the query in order to determine if the join index is applicable.

Oracle can combine a bitmap join index on a fact table with other indices on the same table—whether join indices or not—by using the operators for Boolean bitmap operations.

Domain Indices

Oracle allows tables to be indexed by index structures that are not native to Oracle. This extensibility feature of the Oracle server allows software vendors to develop so-called cartridges with functionality for specific application domains, such as text, spatial data, and images, with indexing functionality beyond that provided by the standard Oracle index types. In implementing the logic for creating, maintaining, and searching the index, the index designer must ensure that it adheres to a specific protocol in its interaction with the Oracle server.

A domain index must be registered in the data dictionary, together with the operators it supports. Oracle's optimizer considers domain indices as one of the possible access paths for a table. Oracle allows cost functions to be registered with the operators so that the optimizer can compare the cost of using the domain index to those of other access paths.

For example, a domain index for advanced text searches may support an operator contains. Once this operator has been registered, the domain index will be considered as an access path for a query like:

```
select *  
  
from employees
```

where contains(resume, 'LINUX');

where resume is a text column in the employee table. The domain index can be stored in either an external data file or inside an Oracle index-organized table.

A domain index can be combined with other (bitmap or B-tree) indices in the same access path by converting between the row-id and bitmap representation and using Boolean bitmap operations.

Partitioning

Oracle supports various kinds of horizontal partitioning of tables and indices, and this feature plays a major role in Oracle's ability to support very large databases. The ability to partition a table or index has advantages in many areas.

- Backup and recovery are easier and faster, since they can be done on individual partitions rather than on the table as a whole.
- Loading operations in a data warehousing environment are less intrusive: data can be added to a newly created partition, and then the partition added to a table, which is an instantaneous operation. Likewise, dropping a partition with obsolete data from a table is very easy in a data warehouse that maintains a rolling window of historical data.
- Query performance benefits substantially, since the optimizer can recognize that only a subset of the partitions of a table need to be accessed in order to resolve a query (partition pruning). Also, the optimizer can recognize that in a join, it is not necessary to try to match all rows in one table with all rows in the other, but that the joins need to be done only between matching pairs of partitions (partition wise join).

An index on a partitioned table can be either a global index or a local index. Entries in a global index can refer to rows in any partition. A locally indexed table has one physical index for each partition that only contains entries for that partition. Unless partition pruning restricts a query to a single partition, a table accessed through a local index will require multiple individual physical index probes. However, a local index has advantages in data warehousing environments where new data can be loaded into a new partition and indexed without the need to maintain any existing index. (Loading followed by index creation is much more efficient than maintaining an existing index while the data are being loaded. Similarly, dropping an old partition and the physical part of its local index can be done without causing any index maintenance.

Each row in a partitioned table is associated with a specific partition. This association is based on the partitioning column or columns that are part of the definition of a partitioned table. There are several ways to map column values to partitions, giving rise to several types of partitioning, each with different characteristics: range, hash, list, and composite partitioning.

Range Partitioning

In range partitioning, the partitioning criteria are ranges of values. This type of partitioning is especially well suited to date columns, in which case all rows in the same date range, say a day or a month, belong in the same partition. In a data warehouse where data are loaded from the transactional systems at regular intervals, range partitioning can be used to implement a rolling window of historical data efficiently. Each data load gets its own new partition, making the loading process faster and more efficient. The system actually loads the data into a separate table with the same column definition as the partitioned table. It can then check the data for consistency, cleanse them, and index them. After that, the system can make the separate table a new partition of the partitioned table, by a simple change to the metadata in the data dictionary—a nearly instantaneous operation.

Up until the metadata change, the loading process does not affect the existing data in the partitioned table in any way. There is no need to do any maintenance of existing indices as part of the loading. Old data can be removed from a table by simply dropping its partition; this operation does not affect the other partitions.

In addition, queries in a data warehousing environment often contain conditions that restrict them to a certain time period, such as a quarter or month. If date-range partitioning is used, the query optimizer can restrict the data access to those partitions that are relevant to the query, and avoid a scan of the entire table.

Partitions can either be created with explicitly set end points or be defined based on a fixed range, such as a day or a month. In the latter case, called interval partitioning, the creation of the partition happens automatically under the covers when trying to insert a row with a value in a previously nonexistent interval.

Hash Partitioning

In hash partitioning, a hash function maps rows to partitions according to the values in the partitioning columns. This type of partitioning is primarily useful when it is important to distribute the rows evenly among partitions or when partition wise joins are important for query performance.

List Partitioning

In list partitioning, the values associated with a particular partition are stated in a list. This type of partitioning is useful if the data in the partitioning column have a relatively small set of discrete values. For instance, a table with a state column can be implicitly partitioned by geographical region if each partition list has the states that belong in the same region.

Composite Partitioning

In composite partitioning, tables that are range, interval, or list partitioned can be subpartitioned by range, list, or hash. For example, a table may be range partitioned on a date column and hash subpartitioned on a column that is frequently used as a join column. The subpartitioning allows partition-wise joins to be used when the table is joined.

Reference Partitioning

In reference partitioning, the partitioning key is resolved based on a foreignkey constraint with another table. The dependency between the tables allows maintenance operations to be automatically cascaded.

14.3.4 Query Processing and Optimization

Oracle supports a large variety of processing techniques in its query processing engine. Some of the more important ones are described here briefly.

Execution Methods

Data can be accessed through a variety of access methods:

- Full table scan. The query processor scans the entire table by getting information about the blocks that make up the table from the extent map, and scanning those blocks.
- Index scan. The processor creates a start and/or stop key from conditions in the query and uses it to scan to a relevant part of the index. If there are columns that need to be retrieved, that are not part of the index, the index scan would be followed by a table access by index row-id. If no start or stop key is available, the scan would be a full index scan.
- Index fast full scan. The processor scans the extents the same way as the table extent in a full table scan. If the index contains all the table columns that are needed

for that table, and there are no good start/stop keys that would significantly reduce that portion of the index that would be scanned in a regular index scan, this method may be the fastest way to access the data. This is because the fast full scan can take full advantage of multiblock disk I/O. However, unlike a regular full scan, which traverses the index leaf blocks in order, a fast full scan does not guarantee that the output preserves the sort order of the index.

- **Index join.** If a query needs only a small subset of the columns of a wide table, but no single index contains all those columns, the processor can use an index join to generate the relevant information without accessing the table, by joining several indices that together contain the needed columns. It performs the joins as hash joins on the row-ids from the different indices.
- **Cluster and hash cluster access.** The processor accesses the data by using the cluster key.

Oracle has several ways to combine information from multiple indices in a single access path. This ability allows multiple where-clause conditions to be used together to compute the result set as efficiently as possible. The functionality includes the ability to perform Boolean operations and, or, and minus on bitmaps representing row-ids. There are also operators that map a list of row-ids into bitmaps and vice versa, which allows regular B-tree indices and bitmap indices to be used together in the same access path. In addition, for many queries involving count(*) on selections on a table, the result can be computed by just counting the bits that are set in the bitmap generated by applying the where clause conditions, without accessing the table.

Oracle supports several types of joins in the execution engine: inner joins, outer joins, semijoins, and antijoins. (An antijoin in Oracle returns rows from the left-hand side input that do not match any row in the right-hand side input; this operation is called anti-semijoin in other literature.) It evaluates each type of join by one of three methods: hash join, sort–merge join, or nested-loop join.

Optimization

The general topic of query optimization. Here, we discuss optimization in the context of Oracle.

Query Transformations

Oracle does query optimization in several steps. One such step is to perform various

query transformations and rewrites that fundamentally change the structure of the query. Another step is to perform access path selection to determine access paths, join methods, and join order. Since some transformations are not always beneficial, Oracle uses cost-based query transformations where the transformations and access path selection are interleaved. For each transformation that is tried, access path selection is performed in order to generate a cost estimate, and the transformation is accepted or rejected based on the cost for the resulting execution plan.

Some of the major types of transformations and rewrites supported by Oracle are as follows:

- View merging. A view reference in a query is replaced by the view definition. This transformation is not applicable to all views.
- Complex viewmerging. Oracle offers this feature for certain classes of views that are not subject to regular view merging because they have a group by or select distinct in the view definition. If such a view is joined to other tables, Oracle can commute the joins and the sort or hash operation used for the group by or distinct.
- Subquery flattening. Oracle has a variety of transformations that convert various classes of subqueries into joins, semijoins, or anti joins. Such conversion is also called decorrelation.
- Materialized view rewrite. Oracle has the ability to rewrite a query automatically to take advantage of materialized views. If some part of the query can be matched up with an existing materialized view, Oracle can replace that part of the query with a reference to the table in which the view is materialized. If need be, Oracle adds join conditions or group by operations to preserve the semantics of the query. If multiple materialized views are applicable, Oracle picks the one that gives the greatest advantage in reducing the amount of data that have to be processed. In addition, Oracle subjects both the rewritten query and the original version to the full optimization process producing an execution plan and an associated cost estimate for each. Oracle then decides whether to execute the rewritten or the original version of the query on the basis of the cost estimates.

Access Path Selection

Oracle has a cost-based optimizer that determines join order, join methods, and access paths. Each operation that the optimizer considers has an associated cost function, and the optimizer tries to generate the combination of operations that has the lowest overall

cost.

In estimating the cost of an operation, the optimizer relies on statistics that have been computed for schema objects such as tables and indices. The statistics contain information about the size of the object, the cardinality, the data distribution of table columns, and so forth. Oracle supports height-balanced and frequency histograms for data distributions. Height-balanced histograms are also referred to as equi-depth histograms. To facilitate the collection of optimizer statistics, Oracle can monitor modification activity on tables and keep track of those tables that have been subject to enough changes that recalculating the statistics may be appropriate. Oracle also tracks what columns are used in where clauses of queries, which makes them potential candidates for histogram creation. With a single command, a user can tell Oracle to refresh the statistics for those tables that were marked as sufficiently changed. Oracle uses sampling to speed up the process of gathering the new statistics and automatically chooses the smallest adequate sample percentage. It also determines whether the distribution of the marked columns merits the creation of histograms; if the distribution is close to uniform, Oracle uses a simpler representation of the column statistics.

In some cases, it may be impossible for the optimizer to accurately estimate the selectivity of a condition in the where clause of a query just based on simple column statistics. For example, the condition may be an expression involving a column, such as $f(col + 3) > 5$. Another class of problematic queries is those that have multiple predicates on columns that have some form of correlation. Assessing the combined selectivity of those predicates may be hard. Oracle therefore allows statistics to be created for expressions as well as for groups of columns. In addition, Oracle can address these issues through dynamic sampling. The optimizer can randomly sample a small portion of a table and apply all the relevant predicates to the sample to see the percentage of the rows that match. This feature can also handle temporary tables where the lifespan and visibility of the data may prevent regular statistics collection.

Oracle uses both CPU cost and disk I/Os in the optimizer cost model. To balance the two components, it stores measures about CPU speed and disk I/O performance as part of the optimizer statistics. Oracle's package for gathering optimizer statistics computes these measures. For queries involving a nontrivial number of joins, the search space is an issue for a query optimizer. Oracle addresses this issue in several ways. The optimizer generates an initial join order and then decides on the best join methods and access paths for that join order. It then changes the order of the tables and determines the best join methods and access paths for the new join order and so forth, while keeping the best plan

that has been found so far. Oracle cuts the optimization short if the number of different join orders that have been considered becomes so large that the time spent in the optimizer may be noticeable compared to the time it would take to execute the best plan found so far. Since this cutoff depends on the cost estimate for the best plan found so far, finding a good plan early is important so that the optimization can be stopped after a smaller number of join orders, resulting in better response time. Oracle uses several initial ordering heuristics to increase the likelihood that the first join order considered is a good one.

For each join order that is considered, the optimizer may make additional passes over the tables to decide join methods and access paths. Such additional passes would target specific global side effects of the access path selection. For instance, a specific combination of join methods and access paths may eliminate the need to perform an order by sort. Since such a global side effect may not be obvious when the costs of the different join methods and access paths are considered locally, a separate pass targeting a specific side effect is used to find a possible execution plan with a better overall cost.

Partition Pruning

For partitioned tables, the optimizer tries to match conditions in the where clause of a query with the partitioning criteria for the table, in order to avoid accessing partitions that are not needed for the result. For example, if a table is partitioned by date range and the query is constrained to data between two specific dates, the optimizer determines which partitions contain data between the specified dates and ensures that only those partitions are accessed. This scenario is very common, and the speedup can be dramatic if only a small subset of the partitions are needed.

Parallel Execution

Oracle allows the execution of a single SQL statement to be parallelized by dividing the work between multiple processes on a multiprocessor computer. This feature is especially useful for computationally intensive operations that would otherwise take an unacceptably long time to perform. Representative examples are decision support queries that need to process large amounts of data, data loads in a data warehouse, and index creation or rebuild.

In order to achieve good speedup through parallelism, it is important that the work involved in executing the statement be divided into granules that can be processed independently by the different parallel processors. Depending on the type of operation, Oracle has several ways to split up the work. For operations that access base objects

(tables and indices), Oracle can divide the work by horizontal slices of the data. For some operations, such as a full table scan, each such slice can be a range of blocks—each parallel query process scans the table from the block at the start of the range to the block at the end. For some operations on a partitioned table, such as an index range scan, the slice would be a partition. Parallelism based on block ranges is more flexible since these can be determined dynamically based on a variety of criteria and are not tied to the table definition. Joins can be parallelized in several different ways. One way is to divide one of the inputs to the join between parallel processes and let each process join its slice with the other input to the join; this is the asymmetric fragment-and-replicate method. For example, if a large table is joined to a small one by a hash join, Oracle divides the large table among the processes and broadcasts a copy of the small table to each process, which then joins its slice with the smaller table. If both tables are large, it would be prohibitively expensive to broadcast one of them to all processes. In that case, Oracle achieves parallelism by partitioning the data among processes by hashing on the values of the join columns. Each table is scanned in parallel by a set of processes and each row in the output is passed on to one of a set of processes that are to perform the join. Which one of these processes gets the row is determined by a hash function on the values of the join column. Hence, each join process gets only rows that could potentially match, and no rows that could match could end up in different processes. Oracle parallelizes sort operations by value ranges of the column on which the sort is performed. Each process participating in the sort is sent rows with values in its range, and it sorts the rows in its range. To maximize the benefits of parallelism, the rows need to be divided as evenly as possible among the parallel processes, and the problem of determining range boundaries that generates a good distribution then arises. Oracle solves the problem by dynamically sampling a subset of the rows in the input to the sort before deciding on the range boundaries.

Process Structure

The processes involved in the parallel execution of an SQL statement consist of a coordinator process and a number of parallel server processes. The coordinator is responsible for assigning work to the parallel servers and for collecting and returning data to the user process that issued the statement. The degree of parallelism is the number of parallel server processes that are assigned to execute a primitive operation as part of the statement. The degree of parallelism is determined by the optimizer, but can be throttled back dynamically if the load on the system increases.

The parallel servers operate on a producer/consumer model. When a sequence of operations is needed to process a statement, the producer set of servers performs the

first operation and passes the resulting data to the consumer set. For example, if a full table scan is followed by a sort and the degree of parallelism is 32, there would be 32 producer servers performing the table scan and passing the result to 32 consumer servers that perform the sort. If a subsequent operation is needed, such as another sort, the roles of the two sets of servers switch. The servers that originally performed the table scan take on the role of consumers of the output produced by the first sort and use it to perform the second sort. Hence, a sequence of operations proceeds by passing data back and forth between two sets of servers that alternate in their roles as producers and consumers. The servers communicate with each other through memory buffers on shared-memory hardware and through high-speed network connections on MPP (shared nothing) configurations and clustered (shared disk) systems.

For shared-nothing systems, the cost of accessing data on disk is not uniform among processes. A process running on a node that has direct access to a device is able to process data on that device faster than a process that has to retrieve the data over a network. Oracle uses knowledge about device-to-node and device-to-process affinity that is, the ability to access devices directly—when distributing work among parallel execution servers.

Result Caching

Oracle's result caching feature allows the result of a query or query block (e.g., a view referenced in a query) to be cached in memory and reused if the same query is executed again. Updates of the data in the underlying tables invalidate the cached results, so this feature works best for queries against tables that are relatively static and where the result sets are relatively small. Consider, as a usage example, some part of a Web page that is stored in the database and does not change very frequently compared to how often it accessed. For such an application, result caching would be a much more lightweight alternative to using materialized views, which would require explicitly creating and administering new persistent database objects.

14.3.5 Concurrency Control and Recovery

Oracle supports concurrency control and recovery techniques that provide a number of useful features.

Concurrency Control

Read-only queries are given a read consistent snapshot, which is a view of the database as it existed at a specific point in time, containing all updates that were committed

by that point in time, and not containing any updates that were not committed at that point in time. Thus, read locks are not used and read-only queries do not interfere with other database activity in terms of locking.

Oracle supports both statement- and transaction-level read consistency: at the beginning of the execution of either a statement or a transaction (depending on what level of consistency is used), Oracle determines the current system change number (SCN). The SCN essentially acts as a timestamp, where the time is measured in terms of transaction commits instead of wall-clock time. If in the course of a query a data block is found that has a higher SCN than the one being associated with the query, it is evident that the data block has been modified after the time of the original query's SCN by some other transaction that may or may not have committed. Hence, the data in the block cannot be included in a consistent view of the database as it existed at the time of the query's SCN.

Instead, an older version of the data in the block must be used; specifically, the one that has the highest SCN that does not exceed the SCN of the query. Oracle retrieves that version of the data from the undo segment. Hence, provided that the undo space is sufficiently large, Oracle can return a consistent result of the query even if the data items have been modified several times since the query started execution. Should the block with the desired SCN no longer exist in the undo, the query will return an error. It would be an indication that the undo table space has not been properly sized, given the activity on the system.

In the Oracle concurrency model, read operations do not block write operations and write operations do not block read operations, a property that allows a high degree of concurrency. In particular, the scheme allows for long-running queries (for example, reporting queries) to run on a system with a large amount of transactional activity. This kind of scenario is often problematic for database systems where queries use read locks, since the query may either fail to acquire them or lock large amounts of data for a long time, thereby preventing transactional activity against that data and reducing concurrency. (An alternative that is used in some systems is to use a lower degree of consistency, such as degree-two consistency, but that could result in inconsistent query results.)

Oracle's concurrency model is used as a basis for the flashback feature. This feature allows a user to set a certain SCN number or wall-clock time in his session and perform operations on the data that existed at that point in time (provided that the data still exist in the undo). Normally in a database system, once a change has been committed, there is no way to get back to the previous state of the data other than performing point-in-time recovery

from backups. However, recovery of a very large database can be very costly, especially if the goal is just to retrieve some data item that had been inadvertently deleted by a user. The flashback feature provides a much simpler mechanism to deal with user errors. The flashback feature includes the ability to restore a table or an entire database to an earlier point in time without recovering from backups, the ability to perform queries on the data as they existed at an earlier point in time, the ability to track how one or more rows have changed over time, and the ability to examine changes to the database at the transaction level. It may be desirable to be able to track changes to a table beyond what would be possible through normal undo retention. (For instance, corporate governance regulations may require that such changes be trackable for a certain number of years.) For this purpose, a table can be tracked by the flashback archive feature, which creates an internal, history version of the table. A background process converts the undo information into entries in the history table, which can be used to provide flashback functionality for arbitrarily long periods of time.

Oracle supports two ANSI/ISO isolation levels, read committed and serializable. There is no support for dirty reads since it is not needed. Statement-level read consistency corresponds to the read committed isolation level, while transaction level read consistency corresponds to the serializable isolation level. The isolation level can be set for a session or an individual transaction. Statement-level read consistency (that is, read committed) is the default. Oracle uses row-level locking. Updates to different rows do not conflict. If two writers attempt to modify the same row, one waits until the other either commits or is rolled back, and then it can either return a write-conflict error or go ahead and modify the row; write-conflict errors are detected based on the first-updaterwins version of snapshot isolation. Locks are held for the duration of a transaction. In addition to row-level locks that prevent inconsistencies due to DML activity, Oracle uses table locks that prevent inconsistencies due to DDL activity. These locks prevent one user from, say, dropping a table while another user has an uncommitted transaction that is accessing that table. Oracle does not use lock escalation to convert row locks to table locks for the purpose of its regular concurrency control. Oracle detects deadlocks automatically and resolves them by rolling back one of the transactions involved in the deadlock.

Oracle supports autonomous transactions, which are independent transactions generated within other transactions. When Oracle invokes an autonomous transaction, it generates a new transaction in a separate context. The new transaction can be either committed or rolled back before control returns to the calling transaction. Oracle supports multiple levels of nesting of autonomous transactions.

Oracle Data Guard

To ensure high availability, Oracle provides a standby database feature, data guard. A standby database is a copy of the regular database that is installed on a separate system. If a catastrophic failure occurs on the primary system, the standby system is activated and takes over, thereby minimizing the effect of the failure on availability. Oracle keeps the standby database up-to-date by constantly applying archived redo logs that are shipped from the primary database. The backup database can be brought online in read-only mode and used for reporting and decision support queries.

14.3.6 System Architecture

Whenever a database application executes an SQL statement, there is an operating system process that executes code in the database server. Oracle can be configured so that the operating system process is dedicated exclusively to the statement it is processing or so that the process can be shared among multiple statements. The latter configuration, known as the shared server, has somewhat different properties with regard to the process and memory architecture. We shall discuss the dedicated server architecture first and the multithreaded server architecture later.

Dedicated Server: Memory Structures

The memory used by Oracle falls mainly into three categories: software code areas, which are the parts of the memory where the Oracle server code resides, the system global area (SGA), and the program global area (PGA). A PGA is allocated for each process to hold its local data and control information. This area contains stack space for various session data and the private memory for the SQL statement that it is executing. It also contains memory for sorting and hashing operations that may occur during the evaluation of the statement.

The performance of such operations is sensitive to the amount of memory that is available. For example, a hash join that can be performed in memory will be faster than if it is necessary to spill to disk. Since there can be a large number of sorting and hashing operations active simultaneously (because of multiple queries as well as multiple operations within each query), deciding how much memory should be allocated to each operation is nontrivial, especially as the load on the system may fluctuate. Under allocation of memory can lead to extra disk I/Os if an operation needlessly spills to disk and over allocation of memory can lead to thrashing. Oracle lets the database administrator specify a target parameter for the total amount of memory that should be considered available for these

operations. The size of this target would typically be based on the total amount of memory available on the system and some calculation as to how it should be divided between various Oracle and non-Oracle activities. Oracle will dynamically decide the best way to divide the memory available within the target between the active operations in order to maximize throughput. The memory allocation algorithm knows the relationship between memory and performance for the different operations and seeks to ensure that the available memory is used as efficiently as possible.

The SGA is a memory area for structures that are shared among users. It is made up of several major structures, including the following.

- Buffer cache. This cache keeps frequently accessed data blocks (from tables as well as indices) in memory to reduce the need to perform physical disk I/O. A least recently used replacement policy is used except for blocks accessed during a full table scan. However, Oracle allows multiple buffer pools to be created that have different criteria for aging out data. Some Oracle operations bypass the buffer cache and read data directly from disk.
- Redo log buffer. This buffer contains the part of the redo log that has not yet been written to disk.
- Shared pool. Oracle seeks to maximize the number of users that can use the database concurrently by minimizing the amount of memory that is needed for each user. One important concept in this context is the ability to share the internal representation of SQL statements and procedural code written in PL/SQL. When multiple users execute the same SQL statement, they can share most data structures that represent the execution plan for the statement. Only data that are local to each specific invocation of the statement need to be kept in private memory.

The sharable parts of the data structures representing the SQL statement are stored in the shared pool, including the text of the statement. The caching of SQL statements in the shared pool also saves compilation time, since a new invocation of a statement that is already cached does not have to go through the complete compilation process. The determination of whether an SQL statement is the same as one existing in the shared pool is based on exact text matching and the setting of certain session parameters. Oracle can automatically replace constants in an SQL statement with bind variables; future queries that are the same except for the values of constants will then match the earlier query in the shared pool.

The shared pool also contains caches for dictionary information and various control structures. Caching dictionary metadata is important for speeding up the compilation time for SQL statements. In addition, the shared pool is used for Oracle's result cache feature.

14.3.7 Replication, Distribution, and External Data

Oracle provides support for replication and distributed transactions with two phase commit.

Replication

Oracle supports several types of replication. In one form, data in a master site are replicated to other sites in the form of materialized views. A materialized view does not have to contain all the master data—it can, for example, exclude certain columns from a table for security reasons. Oracle supports two types of materialized views for replication: read-only and updatable. An updatable materialized view can be modified and the modifications propagated to the master table. However, read-only materialized views allow for a wider range of view definitions. For instance, a read-only materialized view can be defined in terms of set operations on tables at the master site. Changes to the master data are propagated to the replicas through the materialized view refresh mechanism.

Oracle also supports multiple master sites for the same data, where all master sites act as peers. A replicated table can be updated at any of the master sites and the update is propagated to the other sites. The updates can be propagated either asynchronously or synchronously.

For asynchronous replication, the update information is sent in batches to the other master sites and applied. Since the same data could be subject to conflicting modifications at different sites, conflict resolution based on some business rules might be needed. Oracle provides a number of built-in conflict resolution methods and allows users to write their own if need be. With synchronous replication, an update to one master site is propagated immediately to all other sites.

Distributed Databases

Oracle supports queries and transactions spanning multiple databases on different systems. With the use of gateways, the remote systems can include non-Oracle databases. Oracle has built-in capability to optimize a query that includes tables at different sites, retrieve the relevant data, and return the result as if it had been a normal, local query. Oracle also transparently supports transactions spanning multiple sites by a built-in two-phase-commit protocol.

External Data Sources

Oracle has several mechanisms for supporting external data sources. The most common usage is in data warehousing when large amounts of data are regularly loaded from a transactional system.

SQL*Loader

Oracle has a direct-load utility, SQL*Loader, that supports fast parallel loads of large amounts of data from external files. It supports a variety of data formats and it can perform various filtering operations on the data being loaded.

External Tables

Oracle allows external data sources, such as flat files, to be referenced in the from clause of a query as if they were regular tables. An external table is defined by metadata that describe the Oracle column types and the mapping of the external data into those columns. An access driver is also needed to access the external data. Oracle provides a default driver for flat files. The external table feature is primarily intended for extraction, transformation, and loading (ETL) operations in a data warehousing environment. Data can be loaded into the data warehouse from a flat file using create table table as

```
select ... from < external table >
```

```
where ...
```

By adding operations on the data in either the select list or where clause, transformations and filtering can be done as part of the same SQL statement. Since these operations can be expressed either in native SQL or in functions written in PL/SQL or Java, the external table feature provides a very powerful mechanism for expressing all kinds of data transformation and filtering operations. For scalability, the access to the external table can be parallelized by Oracle's parallel execution feature.

Data Pump Export and Import

Oracle provides an export utility for unloading data and metadata into dump files. These files are regular files using a proprietary format that can be moved to another system and loaded into another Oracle database using the corresponding import utility.

14.3.8 Database Administration Tools

Oracle provides users a number of tools and features for system management and application development. In recent releases of Oracle, a lot of emphasis was put on the concept of manageability, that is, reducing the complexity of all aspects of creating and administering an Oracle database. This effort covered a wide variety of areas, including database creation, tuning, space management, storage management, backup and recovery, memory management, performance diagnostics, and workload management.

Oracle Enterprise Manager

Oracle Enterprise Manager (OEM) is Oracle's main tool for database systems management. It provides an easy-to-use graphical user interface for most tasks associated with administering an Oracle database including configuration, performance monitoring, resource management, security management, and access to the various advisors. In addition to database management, OEM provides integrated management of Oracle's applications and middleware software stack.

Automatic Workload Repository

The Automatic Workload Repository (AWR) is one of the central pieces of infrastructure for Oracle's manageability effort. Oracle monitors the activity on the database system and records a variety of information relating to workloads and resource consumption and records them in AWR at regular intervals. By tracking the characteristics of a workload over time, Oracle can detect and help diagnose deviations from normal behavior such as a significant performance degradation of a query, lock contention, and CPU bottlenecks.

The information recorded in AWR provides a basis for a variety of advisors that provide analysis of various aspects of the performance of the system and advice for how it can be improved. Oracle has advisors for SQL tuning, creating access structures, such as indices and materialized views, and memory sizing. Oracle also provides advisors for segment defragmentation and undo sizing.

Database Resource Management

A database administrator needs to be able to control how the processing power of the hardware is divided among users or groups of users. Some groups may execute interactive queries where response time is critical; others may execute long-running reports that can be run as batch jobs in the background when the system load is low. It is also

important to be able to prevent a user from inadvertently submitting an extremely expensive ad hoc query that will unduly delay other users.

Oracle's Database Resource Management feature allows the database administrator to divide users into resource consumer groups with different priorities and properties. For example, a group of high-priority, interactive users may be guaranteed at least 60 percent of the CPU. The remainder, plus any part of the 60 percent not used up by the high-priority group, would be allocated among resource consumer groups with lower priority. A really low-priority group could get assigned 0 percent, which would mean that queries issued by this group would run only when there are spare CPU cycles available. Limits for the degree of parallelism for parallel execution can be set for each group. The database administrator can also set time limits for how long an SQL statement is allowed to run for each group. When a user submits a statement, the resource manager estimates how long it would take to execute it and returns an error if the statement violates the limit. The resource manager can also limit the number of user sessions that can be active concurrently for each resource consumer group. Other resources that can be controlled by the resource manager include undo space.

14.4 Microsoft SQL Server

Microsoft SQL Server is a relational database-management system that scales from laptops and desktops to enterprise servers, with a compatible version, based on the Windows Mobile operating system, available for handheld devices such as Pocket PCs, Smart Phones, and Portable Media Centers. SQL Server was originally developed in the 1980s at Sybase for UNIX systems and later ported to Windows NT systems by Microsoft. Since 1994, Microsoft has shipped SQL Server releases developed independently of Sybase, which stopped using the SQL Server name in the late 1990s. The latest release, SQL Server 2008, is available in express, standard, and enterprise editions and localized for many languages around the world. In this chapter, the term SQL Server refers to all of these editions of SQL Server 2008.

SQL Server provides replication services among multiple copies of SQL Server and with other database systems. Its Analysis Services, an integral part of the system, includes online analytical processing (OLAP) and data-mining facilities. SQL Server provides a large collection of graphical tools and "wizards" that guide database administrators through tasks such as setting up regular backups, replicating data among servers, and tuning a database

for performance. Many development environments support SQL Server, including Microsoft's Visual Studio and related products, in particular the .NET products and services.

14.4.1 Management, Design, and Querying Tools

SQL Server provides a suite of tools for managing all aspects of SQL Server development, querying, tuning, testing, and administration. Most of these tools center around the SQL Server Management Studio. SQL Server Management Studio provides a common shell for administering all services associated with SQL Server, which includes Database Engine, Analysis Services, Reporting Services, SQL Server Mobile, and Integration Services.

Database Development and Visual Database Tools

While designing a database, the database administrator creates database objects such as tables, columns, keys, indices, relationships, constraints, and views. To help create these objects, the SQL Server Management Studio provides access to visual database tools. These tools provide three mechanisms to aid in database design: the Database Designer, the Table Designer, and the View Designer.

The Database Designer is a visual tool that allows the database owner or the owner's delegates to create tables, columns, keys, indices, relationships, and constraints. Within this tool, a user can interact with database objects through database diagrams, which graphically show the structure of the database. The View Designer provides a visual query tool that allows the user to create or modify SQL views through the use of Windows drag-and-drop capabilities. Figure 14.3 shows a view opened from the Management Studio.

Database Query and Tuning Tools

SQL Server Management Studio provides several tools to aid the application development process. Queries and stored procedures can be developed and tested using the integrated Query Editor. The Query Editor supports creating and editing scripts for a variety of environments, including Transact-SQL, the SQL Server scripting language SQLCMD, the multidimensional expression language MDX which is used for data analysis, the SQL Server data-mining language DMX, the XML-analysis language XMLA, and SQL Server Mobile. Further analysis can be done using the SQL ServerProfiler. Database tuning recommendations are provided by the Database Tuning Advisor.

Query Editor

The integrated Query Editor provides a simple graphical user interface for running SQL queries and viewing the results. The Query Editor also provides a graphical

representation of show plan, the steps chosen by the optimizer for query execution. The Query Editor is integrated with Management Studio's Object Explorer, which lets a user drag and drop object or table names into a query window and helps build select, insert, update, or delete statements for any table.

A database administrator or developer can use Query Editor to:

- Analyze queries: Query Editor can show a graphical or textual execution plan for any query, as well as displaying statistics regarding the time and resources required to execute any query.
- Format SQL queries: Including indenting and color syntax coding.

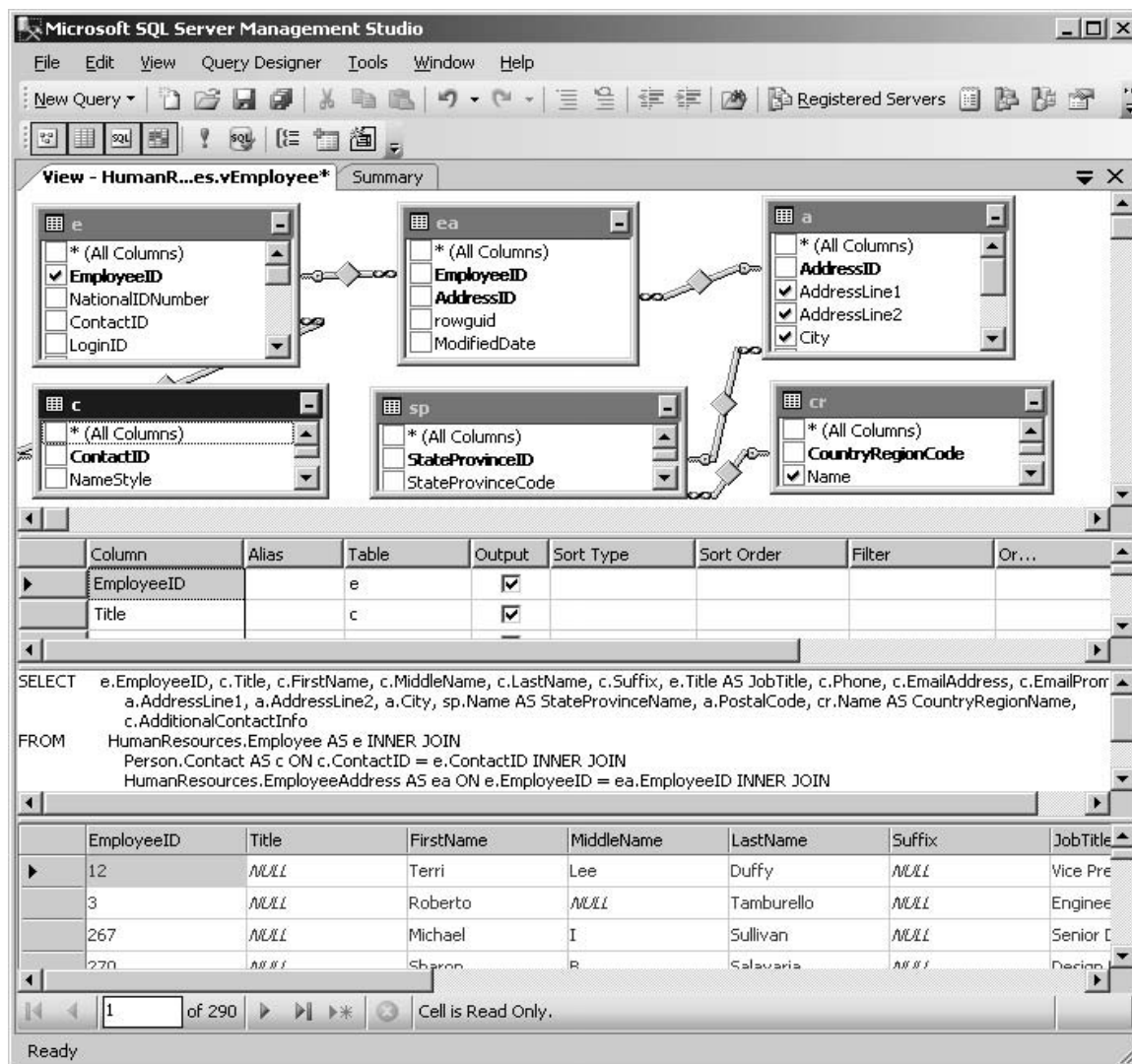


Figure 14.3 The View Designer opened for the Human Resources.vEmployee view.

- Use templates for stored procedures, functions, and basic SQL statements: The Management Studio comes with dozens of predefined templates for building DDL commands, or users can define their own.

Figure 14.4 shows the Management Studio with the Query Editor displaying the graphical execution plan for a query involving a four-table join and an aggregation.

SQL Profiler

SQL Profiler is a graphical utility that allows database administrators to monitor and record database activity of the SQL Server Database Engine and Analysis Services. SQL Profiler can display all server activity in real time, or it can create filters that focus on the actions of particular users, applications, or types of commands. SQL Profiler can display any SQL statement or stored procedure sent to any instance of SQL Server (if the security privileges allow it) in addition to performance data indicating how long the query took to run, how much CPU and I/O was needed, and the execution plan that the query used.

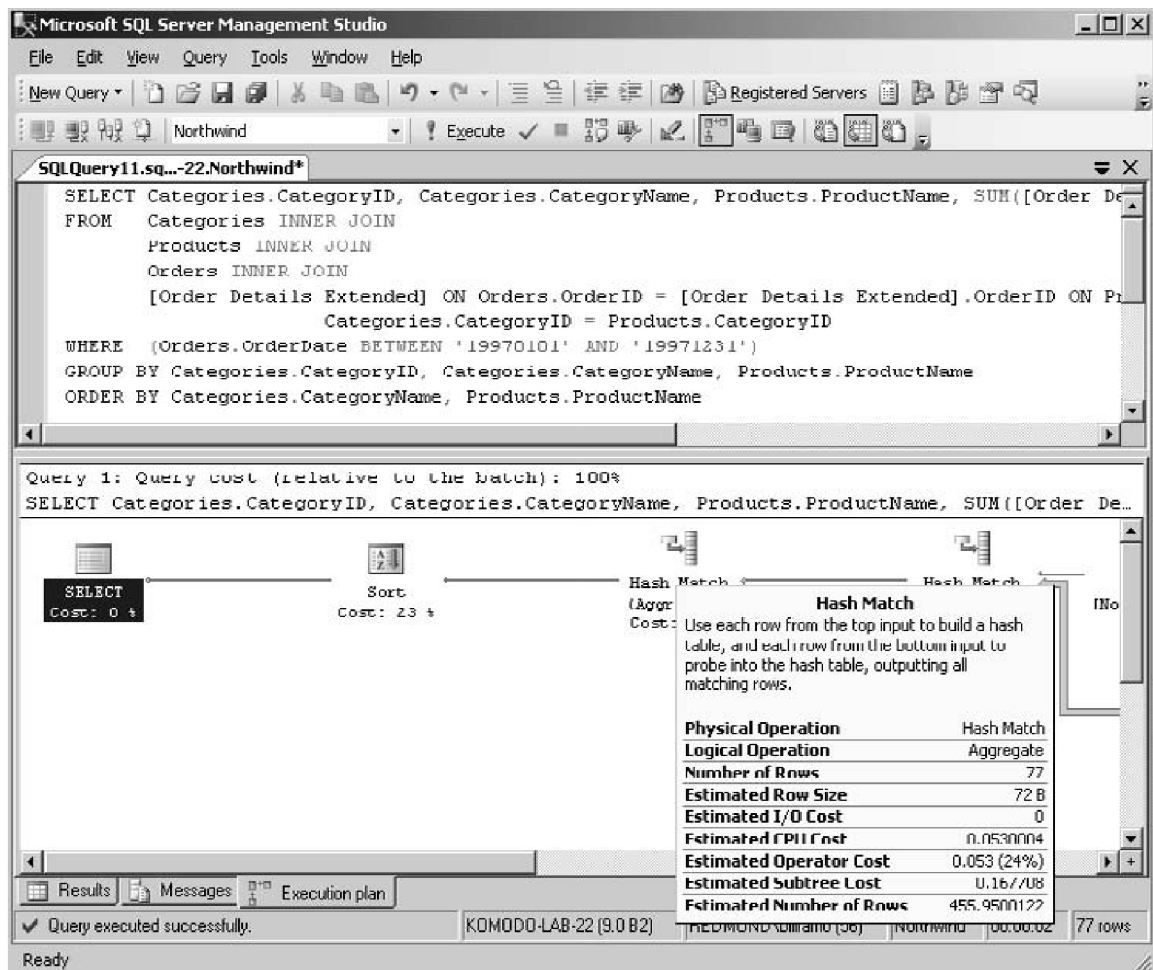


Figure 14.4 A show plan for a four-table join with group by aggregation.

SQL Profiler allows drilling down even deeper into SQL Server to monitor every statement executed as part of a stored procedure, every data modification operation, every lock acquired or released, or every occurrence of a database file growing automatically. Dozens of different events can be captured, and dozens of data items can be captured for each event. SQL Server actually divides the tracing functionality into two separate but connected components. The SQL Profiler is the client-side trace facility. Using SQL Profiler, a user can choose to save the captured data to a file or a table, in addition to displaying it in the Profiler User Interface (UI). The Profiler tool displays every event that meets the filter criteria as it occurs. Once trace data are saved, SQL Profiler can read the saved data for display or analysis purposes.

On the server side is the SQL trace facility, which manages queues of events generated by event producers. A consumer thread reads events from the queues and filters them before sending them to the process that requested them. Events are the main unit of activity as far as tracing is concerned, and an event can be anything that happens inside SQL Server, or between SQL Server and a client. For example, creating or dropping an object is an event, executing a stored procedure is an event, acquiring or releasing a lock is an event, and sending a Transact-SQL batch from a client to the SQL Server is an event. There is a set of stored system procedures to define which events should be traced, what data for each event are interesting, and where to save the information collected from the events. Filters applied to the events can reduce the amount of information collected and stored.

SQL Server guarantees that certain critical information will always be gathered, and it can be used as a useful auditing mechanism. SQL Server is certified for U.S. government C2-level security, and many of the traceable events are available solely to support C2-certification requirements.

The Database Tuning Advisor

Queries and updates can often execute much faster if an appropriate set of indices is available. Designing the best possible indices for the tables in a large database is a complex task; it not only requires a thorough knowledge of how SQL Server uses indices and how the query optimizer makes its decisions, but how the data will actually be used by applications and interactive queries. The SQL Server Database Tuning Advisor (DTA) is a powerful tool for designing the best possible indices and indexed (materialized) views based on observed query and update workloads.

DTA can tune across multiple databases and it bases its recommendations on a workload that can be a file of captured trace events, a file of SQL statements, or an XML

input file. SQL Profiler can be used to capture all SQL statements submitted by all users over a period of time. DTA can then look at the data access patterns for all users, for all applications, for all tables, and make balanced recommendations.

SQL Server Management Studio

In addition to providing access to the database design and visual database tools, the easy-to-use SQL Server Management Studio supports centralized management of all aspects of multiple installations of the SQL Server Database Engine, Analysis Services, Reporting Services, Integration Services, and SQL Server Mobile, including security, events, alerts, scheduling, backup, server configuration, tuning, full-text search, and replication. SQL Server Management Studio allows a database administrator to create, modify, and copy SQL Server database schemas and objects such as tables, views, and triggers. Because multiple installations of SQL Server can be organized into groups and treated as a unit, SQL Server Management Studio can manage hundreds of servers simultaneously.

Although it can run on the same computer as the SQL Server engine, SQL Server Management Studio offers the same management capabilities while running on any Windows 2000 (or later) machine. In addition, the efficient client–server architecture of SQL Server makes it practical to use the remote-access (dial-up networking) capabilities of Windows for administration and management.

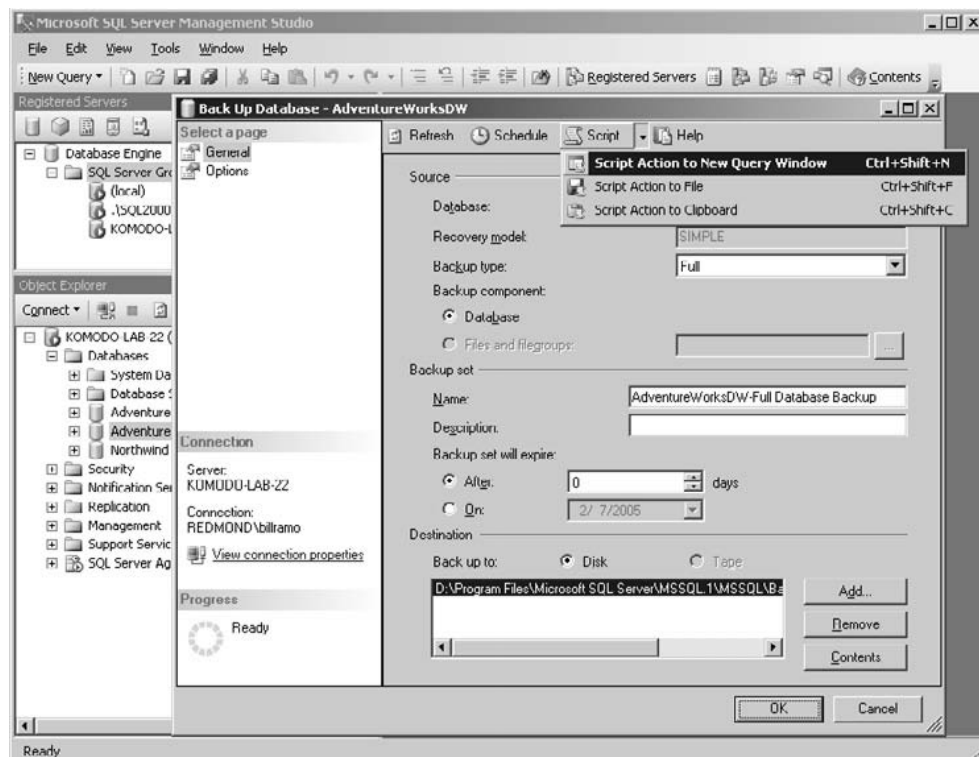


Figure 14.5 The SQL Server Management Studio interface.

SQL Server Management Studio relieves the database administrator from having to know the specific steps and syntax to complete a job. It provides wizards to guide the database administrator through the process of setting up and maintaining an installation of SQL Server. Management Studio's interface is shown in Figure 14.5 and illustrates how a script for database backup can be created directly from its dialogs.

14.4.2 SQL Variations and Extensions

SQL Server allows application developers to write server-side business logic using Transact-SQL or a .NET programming language such as C#, Visual Basic, COBOL, or J++. Transact-SQL is a complete database programming language that includes data definition and data-manipulation statements, iterative and conditional statements, variables, procedures, and functions. Transact-SQL supports most of the **mandatory** DDL query and data modification statements and constructs in the SQL:2003 standard. In addition to the mandatory features, Transact-SQL also supports many **optional** features in the SQL:2003 standard such as recursive queries, common table expressions, user-defined functions, and relational operators such as **intersect** and **except** among others.

Data Types

SQL Server 2008 supports all the mandatory scalar data types in the SQL:2003 standard. SQL Server also supports the ability to alias system types using user supplied names; the aliasing is similar in functionality to the SQL:2003 distinct types, but not fully compliant with them. Some primitive types unique to SQL Server include:

- Large character and binary string types of variable size up to $2^{31} - 1$ bytes, using the **varchar/nvarchar/varbinary(max)** data type, which has a programming model that is similar to the small-character and byte-string types. Additionally, they support a storage attribute called FILESTREAM to specify that data for each individual column value is stored as a separate file in the file system. FILESTREAM storage allows higher performance streaming access to applications using the native file system API.
- An XML type, which is used to store XML data inside a table column. The XML type can optionally have an associated XML *schema collection* specifying a constraint that the instances of the type should adhere to one of the XML types defined in the schema collection.
- **sql variant** is a scalar data type that can contain values of any SQL scalar type (except large character and binary types and **sql variant**). This type is used by

applications that need to store data whose type cannot be anticipated at data-definition time. **sql variant** is also the type of a column formed from the execution of an **unpivot** relational operator. Internally, the system keeps track of the original type of the data. It is possible to filter, join, and sort on **sql variant** columns. The system function **sql variant property** returns details on the actual data stored in a column of type **sql variant**, including the base type and size information.

- The **hierarchyid** data type makes it easier to store and query hierarchical data. Hierarchical data are defined as a set of data items related to one another by hierarchical relationships where one item of data is the parent of another item. Common examples include: an organizational structure, a hierarchical file system, a set of tasks in a project, a taxonomy of language terms, a single inheritance type hierarchy, part-subpart relationships, and a graph of links among Web pages.
- SQL Server supports storing and querying of geospatial data, that is, location data referenced to the earth. Common models of these data are the planar and geodetic coordinate systems. The main distinction between these two systems is that the latter takes into account the curvature of the earth. SQL Server supports geometry and geography, which correspond to the planar and geodetic models.

In addition, SQL Server supports a table type and a cursor type that cannot be used as columns in a table, but can be used in the Transact-SQL language as variables:

- A **table** type enables a variable to hold a set of rows. An instance of this type is used primarily to hold temporary results in a stored procedure or as the return value of a table-valued function. A table variable behaves like a local variable. It has a well-defined scope, which is the function, stored procedure, or batch in which it is declared. Within its scope, a table variable may be used like a regular table. It may be applied anywhere a table or table expression is used in **select**, **insert**, **update**, and **delete** statements.
- A **cursor** type that enables references to a cursor object. The cursor type can be used to declare variables, or routine input/output arguments to reference cursors across routine calls.

Query Language Enhancements

In addition to the SQL relational operators such as **inner join** and **outer join**, SQL Server supports the relational operators **pivot**, **unpivot**, and **apply**.

- **pivot** is an operator that transforms the shape of its input result set from two columns that represent name-value pairs into multiple columns, one for each name from the input. The name column from the input is called the pivot column. The user needs to indicate which names to transpose from the input into individual columns in the output. Consider the table *MonthlySales(ProductID, Month, SalesQty)*. The following query, using the **pivot** operator, returns the *SalesQty* for each of the months Jan, Feb, and Mar as separate columns. Note that the **pivot** operator also performs an implicit aggregation on all the other columns in the table and an explicit aggregation on the pivot column.

```
select * from MonthlySales pivot(sum(SalesQty) for Month in ('Jan', 'Feb', 'Mar')) T;
```

The inverse operation of **pivot** is **unpivot**.

- The **apply** operator is similar to join, except its right input is an expression that may contain references to columns in the left input, for example a table valued function invocation that takes as arguments one or more columns from the left input. The set of columns produced by the operator is the union of the columns from its two inputs. The **apply** operator can be used to evaluate its right input for each row of its left input and perform a **union all** of the rows across all these evaluations. There are two flavors of the **apply** operator similar to **join**, namely, **cross** and **outer**. The two flavors differ in terms of how they handle the case of the right input producing an empty result-set. In the case of **cross apply**, this causes the corresponding row from the left input to not appear in the result. In the case of **outer apply**, the row appears from the left input with NULL values for the columns in the right input. Consider a table-valued function called *FindReports* that takes as input the ID of a given employee and returns the set of employees reporting directly or indirectly to that employee in an organization. The following query calls this function for the manager of each department from the *Departments* table:

```
select * from Departments D cross apply FindReports(D.ManagerID)
```

Routines

Users can write routines that run inside the server process as scalar or table functions, stored procedures, and triggers using Transact-SQL or a .NET language. All these routines

are defined to the database by using the corresponding **create [function, procedure, trigger]** DDL statement. Scalar functions can be used in any scalar expression inside an SQL DML or DDL statement. Table-valued functions can be used anywhere a table is allowed in a **select** statement. Transact-SQL table valued functions whose body contains a single SQL **select** statement are treated as a view (expanded inline) in the query that references the function. Since table valued functions allow input arguments, inline table-valued functions can be considered parameterized views.

Indexed Views

In addition to traditional views as defined in ANSI SQL, SQL Server supports indexed (materialized) views. Indexed views can substantially enhance the performance of complex decision support queries that retrieve large numbers of base table rows and aggregate large amounts of information into concise sums, counts, and averages. SQL Server supports creating a clustered index on a view and subsequently any number of nonclustered indices. Once a view is indexed, the optimizer can use its indices in queries that reference the view or its base tables. There is no need for queries to refer to the view explicitly for the indexed view to be used in the query plan, as the matching is done automatically from the view definition. This way, existing queries can benefit from the improved efficiency of retrieving data directly from the indexed view without having to be rewritten. The indexed view is maintained consistent with the base tables by automatically propagating all updates.

Filtered Indexes

A filtered index is an optimized nonclustered index, especially suited to cover queries that select from a well-defined subset of data. It uses a filter predicate to index a portion of rows in the table. A well-designed filtered index can improve query performance, reduce index-maintenance costs, and reduce index-storage costs compared with full-table indices. Filtered indices can provide the following advantages over full-table indices:

- **Improved query performance and plan quality.** A well-designed filtered index improves query performance and execution plan quality because it is smaller than a full-table nonclustered index and has filtered statistics. The filtered statistics are more accurate than full-table statistics because they cover only the rows in the filtered index.

- **Reduced index maintenance costs.** An index is maintained only when data manipulation language (DML) statements affect the data in the index. A filtered index reduces index maintenance costs compared to a full-table nonclustered index because it is smaller and is only maintained when the data in the index are affected. It is possible to have a large number of filtered indices, especially when they contain data that are affected infrequently. Similarly, if a filtered index contains only the frequently affected data, the smaller size of the index reduces the cost of updating the statistics.
- **Reduced index storage costs.** Creating a filtered index can reduce disk storage for nonclustered indices when a full-table index is not necessary. You can replace a full-table nonclustered index with multiple filtered indices without significantly increasing the storage requirements. Filtered statistics can also be created explicitly, independently from filtered indices.

Updatable Views and Triggers

Generally, views can be the target of **update**, **delete**, or **insert** statements if the data modification applies to only one of the view's base tables. Updates to partitioned views can be propagated to multiple base tables. For example, the following **update** will increase the prices for publisher "0736" by 10 percent:

```
update titleview

set price = price * 1.10

where pub id = '0736';
```

For data modifications that affect more than one base table, the view can be updated if there is an **instead** trigger defined for the operation; **instead** triggers for **insert**, **update**, or **delete** operations can be defined on a view, to specify the updates that must be performed on the base tables to reflect the corresponding modifications on the view.

Triggers are Transact-SQL or .NET procedures that are automatically executed when either a DML (**update**, **insert**, or **delete**) or DDL statement is issued against a base table or view. Triggers are mechanisms that enable enforcement of business logic automatically when data are modified or when DDL statements are executed. Triggers can extend the integrity checking logic of declarative constraints, defaults, and rules, although declarative constraints should be used preferably whenever they suffice, as they can be used by the query optimizer to reason about the data contents.

Triggers can be classified into DML and DDL triggers depending on the kind of event that fires the trigger. DML triggers are defined against a table or view that is being modified. DDL triggers are defined against an entire database for one or more DDL statements such as **create table**, **drop procedure**, etc. Triggers can be classified into **after** and **instead** triggers according to when the trigger gets invoked relative to the action that fires the trigger. **After** triggers execute after the triggering statement and subsequent declarative constraints are enforced. **Instead** triggers execute instead of the triggering action. **Instead** triggers can be thought of as similar to **before** triggers, but they actually replace the triggering action. In SQL Server, DML **after** triggers can be defined only on base tables, while DML **instead** triggers can be defined on base tables or views. **Instead** triggers allow practically any view to be made updatable via user-provided logic.

DDL **instead** triggers can be defined on any DDL statement.

14.4.3 Query Processing and Optimization

The query processor of SQL Server is based on an extensible framework that allows rapid incorporation of new execution and optimization techniques. Any SQL query can be expressed as a tree of operators in an extended relational algebra.

Abstracting operators of this algebra into **iterators**, query execution encapsulates data-processing algorithms as logical units that communicate with each other by using a `GetNextRow()` interface. Starting out with an initial query tree, the query optimizer generates alternatives by using tree transformations and estimates their execution cost by taking into account iterator behavior and statistical models to estimate the number of rows to process.

Overview of Compilation Process

Complex queries present significant optimization opportunities that require reordering operators across query block boundaries and selecting plans solely on the basis of estimated costs. To go after these opportunities, the query optimizer deviates from traditional query-optimization approaches used in other commercial systems in favor of a more general, purely algebraic framework that is based on the Cascades optimizer prototype. Query optimization is part of the query compilation process, which consists of four steps:

- **Parsing/binding.** After parsing, the binder resolves table and column names by using the catalogs. SQL Server utilizes a plan cache to avoid repeated optimization of

identical or structurally similar queries. If no cached plan is available, an initial operator tree is generated. The operator tree is simply a combination of relational operators and is not constrained by concepts such as query blocks or derived tables, which typically obstruct optimization.

- **Simplification/normalization.** The optimizer applies simplification rules on the operator tree to obtain a normal, simplified form. During simplification, the optimizer determines and loads statistics required for cardinality estimation.
- **Cost-based optimization.** The optimizer applies exploration and implementation rules to generate alternatives, estimates execution cost, and chooses the plan with the cheapest anticipated cost. Exploration rules implement reordering for an extensive set of operators, including join and aggregation reordering. Implementation rules introduce execution alternatives such as merge join and hash join.
- **Plan preparation.** The optimizer creates query-execution structures for the selected plan. To achieve best results, cost-based optimization is not divided into phases that optimize different aspects of the query independently; also, it is not restricted to a single dimension such as join enumeration. Instead, a collection of transformation rules defines the space of interest, and cost estimation is used uniformly to select an efficient plan.

Query Simplification

During simplification, only transformations that are guaranteed to generate less costly substitutes are applied. The optimizer pushes selects down the operator tree as far as possible; it checks predicates for contradictions, taking into account declared constraints. It uses contradictions to identify sub expressions that can be removed from the tree. A common scenario is the elimination of **union** branches that retrieve data from tables with different constraints. A number of simplification rules are *context dependent*; that is, the substitution is valid only in the context of utilization of the subexpression. For example, an outer join can be simplified into an inner join if a later filter operation will discard nonmatching rows that were padded with **null**. Another example is the elimination of joins on foreign keys, when there are no later uses of columns from the referenced table. A third example is the context of duplicate insensitivity, which specifies that delivering one or multiple copies of a row does not affect the query result. Subexpressions under semijoins and under **distinct** are duplicate insensitive, which allows turning **union** into **union all**, for example. For grouping and aggregation, the *GbAgg* operator is used, which creates groups and optionally applies an aggregate function on each group. Duplicate removal, expressed in SQL by the **distinct**

keyword, is simply a *GbAgg* with no aggregate functions to compute. During simplification, information about keys and functional dependencies is used to reduce grouping columns. Subqueries are normalized by removing correlated query specifications and using some join variant instead. Removing correlations is not a “subquery action strategy,” but simply a normalization step. A variety of execution strategies is then considered during cost-based optimization.

14.4.4 Concurrency and Recovery

SQL Server’s transaction, logging, locking, and recovery subsystems realize the ACID properties expected of a database system.

Transactions

In SQL Server all statements are atomic and applications can specify various levels of isolation for each statement. A single transaction can include statements that not only select, insert, delete, or update records, but also create or drop tables, build indices, and bulk-import data. Transactions can span databases on remote servers. When transactions are spread across servers, SQL Server uses a Windows operating-system service called the Microsoft Distributed Transaction Coordinator (MS DTC) to perform two-phase commit processing. MS DTC supports the XA transaction protocol and, along with OLE-DB, provides the foundation for ACID transactions among heterogeneous systems. Concurrency control based on locking is the default for SQL Server. SQL Server also offers optimistic concurrency control for cursors. Optimistic concurrency control is based on the assumption that resource conflicts between multiple users are unlikely (but not impossible), and allows transactions to execute without locking any resources. Only when attempting to change data does SQL Server check resources to determine if any conflicts have occurred. If a conflict occurs, the application must read the data and attempt the change again. Applications can choose to detect changes either by comparing values or by checking a special row version column on a row.

SQL Server supports the SQL isolation levels of read uncommitted, read committed, repeatable read, and serializable. Read committed is the default level. In addition, SQL Server supports two snapshot-based isolation levels.

- **Snapshot:** Specifies that data read by any statement in a transaction will be the transactionally consistent version of the data that existed at the start of the transaction. The effect is as if the statements in a transaction see a snapshot of the committed data as it existed at the start of the transaction.

- **Read committed snapshot:** Specifies that each statement executed within a transaction sees a transactionally consistent snapshot of the data as it existed at the start of the statement. This contrasts with read committed isolation where the statement may see committed updates of transactions that commit while the statement is executing.

Recovery and Availability

SQL Server is designed to recover from system and media failures, and the recovery system can scale to machines with very large buffer pools (100 gigabytes) and thousands of disk drives.

Crash Recovery

Logically, the log is a potentially infinite stream of log records identified by log sequence numbers (LSNs). Physically, a portion of the stream is stored in log files. Log records are saved in the log files until they have been backed up and are no longer needed by the system for rollback or replication. Log files grow and shrink in size to accommodate the records that need to be stored. Additional log files can be added to a database (on new disks, for example) while the system is running and without blocking any current operations, and all logs are treated as if they were one continuous file. SQL Server's recovery system has many aspects in common with the ARIES recovery algorithm and some of the key differences are highlighted in this section. SQL Server has a configuration option called **recovery interval**, which allows an administrator to limit the length of time SQL Server should take to recover after a crash. The server dynamically adjusts the checkpoint frequency to reduce recovery time to within the recovery interval. Checkpoints flush all dirty pages from the buffer pool and adjust to the capabilities of the I/O system and its current workload to effectively eliminate any impact on running transactions. Upon start-up after a crash, the system starts multiple threads (automatically scaled to the number of CPUs) to start recovering multiple databases in parallel.

The first phase of recovery is an analysis pass on the log, which builds a dirty page table and active transaction list. The next phase is a redo pass starting from the last checkpoint and redoing all operations. During the redo phase, the dirty page table is used to drive read-ahead of data pages. The final phase is an undo phase where incomplete transactions are rolled back. The undo phase is actually divided into two parts as SQL Server uses a two-level recovery scheme. Transactions at the first level (those involving

internal operations such as space allocation and page splits) are rolled back first, followed by user transactions. Once the transactions at the first level are rolled back, the database is brought online and is available for new user transactions to start while the final rollback operations are performed. This is achieved by having the redo pass reacquire locks for all incomplete user transactions that will be rolled back in the undo phase.

Media Recovery

SQL Server's backup and restore capabilities allow recovery from many failures, including loss or corruption of disk media, user errors, and permanent loss of a server. Additionally, backing up and restoring databases is useful for other purposes, such as copying a database from one server to another and maintaining standby systems. SQL Server has three different recovery models that users can choose from for each database. By specifying a recovery model, an administrator declares the type of recovery capabilities required (such as point-in-time restore and log shipping) and the required backups to achieve them. Backups can be taken on databases, files, file-groups, and the transaction log. All backups are fuzzy and completely online; that is, they do not block any DML or DDL operations while they execute. Restores can also be done online such that only the portion of the database being restored (e.g., a corrupt disk block) is taken offline. Backup and restore operations are highly optimized and limited only by the speed of the media onto which the backup is targeted. SQL Server can back up to both disk and tape devices (up to 64 in parallel) and has high-performance backup APIs for use by third-party backup products.

Database Mirroring

Database mirroring involves immediately reproducing every update to a database (the principal database) onto a separate, complete copy of the database (the mirror database) generally located on another machine. In the event of a disaster on the primary server or even just maintenance, the system can automatically failover to the mirror in a matter of seconds. The communication library used by applications is aware of the mirroring and will automatically reconnect to the mirror machine in the event of a failover. A tight coupling between the primary database and the mirror is achieved by sending blocks of transaction log to the mirror as it is generated on the primary and redoing the log records on the mirror. In full-safety mode, a transaction cannot commit until the log records for the transaction have made it to disk on the mirror. Besides supporting failover, a mirror can also be used to automatically restore a page by copying it from the mirror in the event that the page is found to be corrupt during an attempt to read it.

14.4.5 System Architecture

An SQL Server instance is a single operating-system process that is also a named endpoint for requests for SQL execution. Applications interact with SQL Server via various client-side libraries (like ODBC, OLE-DB, and ADO.NET) in order to execute SQL.

Thread Pooling on the Server

In order to minimize the context switching on the server and to control the degree of multiprogramming, the SQL Server process maintains a pool of threads that execute client requests. As requests arrive from the client, they are assigned a thread on which to execute. The thread executes the SQL statements issued by the client and sends the results back to it. Once the user request completes, the thread is returned back to the thread pool. In addition to user requests, the thread pool is used to assign threads for internal background tasks such as:

- **Lazywriter:** This thread is dedicated to making sure a certain amount of the buffer pool is free and available at all times for allocation by the system. The thread also interacts with the operating system to determine the optimal amount of memory that should be consumed by the SQL Server process.
- **Checkpoint:** This thread periodically checkpoints all databases in order to maintain a fast recovery interval for the databases on server restart.
- **Deadlock monitor:** This thread monitors the other threads, looking for a deadlock in the system. It is responsible for the detection of deadlocks and also picking a victim in order to allow the system to make progress. When the query processor chooses a parallel plan to execute a particular query, it can allocate multiple threads that work on behalf of the main thread to execute the query. Since the Windows NT family of operating systems provides native thread support, SQL Server uses NT threads for its execution. However, SQL Server can be configured to run with user-mode threads in addition to kernel threads in very high-end systems to avoid the cost of a kernel context switch on a thread switch.

Memory Management

There are many different uses of memory within the SQL Server process:

- **Buffer pool.** The biggest consumer of memory in the system is the buffer pool. The buffer pool maintains a cache of the most recently used database pages. It uses a clock replacement algorithm with a steal, no-force policy; that is, buffer pages with

uncommitted updates may be replaced (“stolen”), and buffer pages are not forced to disk on transaction commit. The buffers also obey the write-ahead logging protocol to ensure correctness of crash and media recovery.

- **Dynamic memory allocation.** This is the memory that is allocated dynamically to execute requests submitted by the user.
- **Plan and execution cache.** This cache stores the compiled plans for various queries that have been previously executed by users in the system. This allows various users to share the same plan (saving memory) and also saves on query compilation time for similar queries.
- **Large memory grants.** These are for query operators that consume large amounts of memory, such as hash join and sort. SQL Server uses an elaborate scheme of memory management to divide its memory among the various uses described above. A single memory manager centrally manages all the memory used by SQL Server. The memory manager is responsible for dynamically partitioning and redistributing the memory between the various consumers of memory in the system. It distributes this memory in accordance with an analysis of the relative cost benefit of memory for any particular use. A generalized LRU infrastructure mechanism is available to all components.

This caching infrastructure tracks not only the lifetime of cached data but also the relative CPU and I/O costs incurred to create and cache it. This information is used to determine the relative costs of various cached data. The memory manager focuses on throwing out the cached data that have not been touched recently and were cheap to cache. As an example, complex query plans that require seconds of CPU time to compile are more likely to stay in memory than trivial plans, given equivalent access frequencies.

The memory manager interacts with the operating system to decide dynamically how much memory it should consume out of the total amount of memory in the system. This allows SQL Server to be quite aggressive in using the memory on the system but still return memory back to the system when other programs need it without causing excessive page faults.

In addition the memory manager is aware of the CPU and memory topology of the system. Specifically, it leverages the NUMA (non uniform memory access) that many machines employ and attempts to maintain locality between the processor that a thread is executing on and the memory it accesses.

Security

SQL Server provides comprehensive security mechanisms and policies for authentication, authorization, audit, and encryption. Authentication can be either through a username–password pair managed by SQL Server, or through a Windows OS account. Authorization is managed by permission grants to schema objects or covering permissions on container objects such as the database or server instance. At authorization-check time, permissions are rolled up and calculated, accounting for covering permissions and role memberships of the principal. Audits are defined in the same way as permissions — they are defined on schema objects for a given principal or containing objects and at the time of the operation they are dynamically calculated based on audit definitions on the object and accounting for any covering audits or role memberships of the principal. Multiple audits may be defined so that audits for different purposes, such as for Sarbanes- Oxley and HIPAA,¹ may be managed independently without risk of breaking each other. Audits records are written either in a file or to the Windows Security Log.

SQL Server provides both manual encryption of data and Transparent Data Encryption. Transparent Data Encryption encrypts all data pages and log pages when written to disk and decrypts when read from the disk so that the data are encrypted at rest on the disk but is plaintext to SQL Server users without application modification. Transparent Data Encryption can be more CPU efficient than manual encryption as data is only encrypted when written to disk and it is done in larger units, pages, rather than individual cells of data.

Two things are even more critical to users' security: (1) the quality of the entire code base itself and (2) the ability for users to determine if they have secured the system properly. The quality of the code base is enhanced by using the Security Development Lifecycle. All developers and testers of the product go through security training. All features are threat modeled to assure assets are appropriately protected. Wherever possible, SQL Server utilizes the underlying security features of the operating system rather than implementing its own, such as Windows OS Authorization and the Windows Security Log for an audit record. Furthermore, numerous internal tools are utilized to analyze the code base looking for potential security flaws. Security is verified using fuzz testing² and testing of the threat model. Before release, there is a final security review of the product and a response plan is in place for dealing with security issues found after release which is then executed as issues are discovered.

A number of features are provided to help users secure the system properly. One such feature is a fundamental policy called off-by-default, where many less commonly

used components or those requiring extra care for security, are completely disabled by default. Another feature is a best-practices analyzer that warns users about configurations of system settings that could lead to a security vulnerability. Policy-based management further allows users to define what the settings should be and either warns of or prevents changes that would conflict with the approved settings.

14.4.6 Data Access

SQL Server supports the following application programming interfaces (APIs) for building data-intensive applications:

The Sarbanes-Oxley Act is a U.S. government financial regulation law. HIPAA is a U.S. government health-care law that includes regulation of health-care-related information. 2Fuzz testing A randomization-based technique for testing for unexpected, possibly invalid, input.

- **ODBC.** This is Microsoft's implementation of the standard SQL:1999 call level interface (CLI). It includes object models—Remote Data Objects (RDOs) and Data Access Objects (DAOs)—that make it easier to program multitier database applications from programming languages like Visual Basic.
- **OLE-DB.** This is a low-level, systems-oriented API designed for programmers building database components. The interface is architected according to the Microsoft Component Object Model (COM), and it enables the encapsulation of low-level database services such as rowset providers, ISAM providers, and query engines. OLE-DB is used inside SQL Server to integrate the relational query processor and the storage engine and to enable replication and distributed access to SQL and other external data sources. Like ODBC, OLE-DB includes a higher-level object model called ActiveX Data Objects (ADO) to make it easier to program database applications from Visual Basic.
- **ADO.NET.** This is an API designed for applications written in .NET languages such as C# and Visual Basic.NET. This interface simplifies some common data access patterns supported by ODBC and OLE-DB. In addition, it provides a new *data set* model to enable stateless, disconnected data access applications.

ADO.NET includes the ADO.NET **Entity Framework**, which is a platform for programming against data that raises the level of abstraction from the logical (relational) level to the conceptual (entity) level, and thereby significantly reduces the impedance mismatch for applications and data services such as reporting, analysis, and replication.

The conceptual data model is implemented using an extended relational model, the **Entity Data Model (EDM)** that embraces entities and relationships as first-class concepts. It includes a query language for the EDM called **Entity SQL**, a comprehensive mapping engine that translates from the conceptual to the logical (relational) level, and a set of model-driven tools that help developers define mappings between objects and entities to tables.

- **LINQ. Language-integrated query**, or **LINQ** for short, allows declarative, set oriented constructs to be used directly in programming languages such as C# and Visual Basic. The query expressions are not processed by an external tool or language preprocessor but instead are first-class expressions of the languages themselves. LINQ allows query expressions to benefit from the rich metadata, compile-time syntax checking, static typing and auto-completion that was previously available only to imperative code. LINQ defines a set of general-purpose standard query operators that allow traversal, filter, join, projection, sorting, and grouping operations to be expressed in a direct yet declarative way in any .NET-based programming language. C# and Visual Basic also support query comprehensions, i.e., language syntax extensions that leverage the standard query operators.
- **DB-Lib.** The DB-Library for C API that was developed specifically to be used with earlier versions of SQL Server that predate the SQL-92 standard.
- **HTTP/SOAP.** Applications can use HTTP/SOAP requests to invoke SQL Server queries and procedures. Applications can use URLs that specify Internet Information Server (IIS) virtual roots that reference an instance of SQL Server. The URL can contain an XPath query, a Transact-SQL statement, or an XML template.

14.4.7 Distributed Heterogeneous Query Processing

SQLServer distributed heterogeneous query capability allows transactional queries and updates against a variety of relational and non relational sources via OLEDB data providers running in one or more computers. SQL Server supports two methods for referencing heterogeneous OLE-DB data sources in Transact-SQL statements. The linked-server-names method uses system-stored procedures to associate a server name with an OLE-DB data source. Objects in these linked servers can be referenced in Transact-SQL statements using the four-part name convention described below. For example, if a linked server name of *DeptSQLSrvr* is defined against another copy of SQL Server, the following statement references a table on that server:

```
select * from DeptSQLSvr.Northwind.dbo.Employees;
```

An OLE-DB data source is registered in SQL Server as a linked server. Once a linked server is defined, its data can be accessed using the four-part name:

```
<linked- server>.<catalog>.<schema>.<object>
```

The following example establishes a linked server to an Oracle server via an OLE-DB provider for Oracle:

```
exec sp addlinkedserver OraSvr, 'Oracle 7.3', 'MSDAORA', 'OracleServer'
```

A query against this linked server is expressed as:

```
select * from OraSvr.CORP.ADMIN.SALES;
```

In addition, SQL Server supports built-in, parameterized table-valued functions called **openrowset** and **openquery**, which allow sending uninterpreted queries to a provider or linked server, respectively, in the dialect supported by the provider. The following query combines information stored in an Oracle server and a Microsoft Index Server. It lists all documents and their author containing the words Data and Access ordered by the author's department and name.

```
select e.dept, f.DocAuthor, f.FileName  
from OraSvr.Corp.Admin.Employee e,  
openquery(EmpFiles,  
'select DocAuthor, FileName  
from scope("c:\EmpDocs")  
where contains(' ' "Data" near() "Access" ' ')>0') as f  
where e.name = f.DocAuthor  
order by e.dept, f.DocAuthor,
```

The relational engine uses the OLE-DB interfaces to open the rowsets on linked servers, to fetch the rows, and to manage transactions. For each OLE-DB data source accessed as a linked server, an OLE-DB provider must be present on the server running SQL Server. The set of Transact-SQL operations that can be used against a specific OLE-DB data source depends on the capabilities of the OLE-DB provider.

Whenever it is cost-effective, SQL Server pushes relational operations such as joins, restrictions, projections, sorts, and group by operations to the OLE-DB data source. SQL Server uses Microsoft Distributed Transaction Coordinator and the OLE-DB transaction interfaces of the provider to ensure atomicity of transactions spanning multiple data sources.

14.4.8 Replication

SQL Server replication is a set of technologies for copying and distributing data and database objects from one database to another, tracking changes, and synchronizing between databases to maintain consistency. SQL Server replication also provides inline replication of most database schema changes without requiring any interruptions or reconfiguration. Data are typically replicated to increase availability of data. Replication can roll up corporate data from geographically dispersed sites for reporting purposes and disseminate data to remote users on a local-area network or mobile users on dial-up connections or the Internet. Microsoft SQL Server replication also enhances application performance by scaling out for improved total read performance among replicas, as is common in providing mid tier data-caching services for Web sites.

Replication Model

SQL Server introduced the **Publish–Subscribe** metaphor to database replication and extends this publishing-industry metaphor throughout its replication administration and monitoring tools.

The **publisher** is a server that makes data available for replication to other servers. The publisher can have one or more publications, each representing a logically related set of data and database objects. The discrete objects within a publication, including tables, stored procedures, user-defined functions, views, materialized views, and more, are called **articles**. The addition of an article to a publication allows for extensive customizing of the way the object is replicated, e.g., restrictions on which users can subscribe to receive its data and how the data set should be filtered on the basis of a projection or selection of a table, by a “horizontal” or a “vertical” filter, respectively.

Subscribers are servers that receive replicated data from a publisher. Subscribers can conveniently subscribe to only the publications they require from one or more publishers regardless of the number or type of replication options each implements. Depending on the type of replication options selected, the subscriber either can be used as a read-only replica or can make data changes that are automatically propagated back to the publisher

and subsequently to all other replicas. Subscribers can also republish the data they subscribe to, supporting as flexible a replication topology as the enterprise requires.

The **distributor** is a server that plays different roles, depending on the replication options chosen. At a minimum it is used as a repository for history and error state information. In other cases, it is used additionally as an intermediate store-and-forward queue to scale up the delivery of the replicated payload to all the subscribers.

Replication Options

Microsoft SQL Server replication offers a wide spectrum of replication options. To decide on the appropriate replication options to use, a database designer must determine the application's needs with respect to autonomous operation of the sites involved and the degree of transactional consistency required.

Snapshot replication copies and distributes data and database objects exactly as they appear at a moment in time. Snapshot replication does not require continuous change tracking because changes are not propagated incrementally to subscribers. Subscribers are updated with a complete refresh of the data set defined by the publication on a periodic basis. Options available with snapshot replication can filter published data and can enable subscribers to modify replicated data and propagate those changes back to the publisher. This type of replication is best suited for smaller sizes of data and when updates typically affect enough of the data that replicating a complete refresh of the data is efficient. With **transactional replication**, the publisher propagates an initial snapshot of data to subscribers, then forwards incremental data modifications to subscribers as discrete transactions and commands. Incremental change tracking occurs inside the core engine of SQL Server, which marks transactions affecting replicated objects in the publishing database's transaction log. A replication process called the **log reader agent** reads these transactions from the database transaction log, applies an optional filter, and stores them in the distribution database, which acts as the reliable queue supporting the store-and-forward mechanism of transactional replication. Another replication process, called the **distribution agent**, then forwards the changes to each subscriber. Like snapshot replication, transactional replication offers subscribers the option to make updates that either use two-phase commit to reflect those changes consistently at the publisher and subscriber or queue the changes at the subscriber for asynchronous retrieval by a replication process that later propagates the change to the publisher. This type of replication is suitable when intermediate states between multiple updates need to be preserved. **Merge replication** allows each replica in the enterprise to work with total autonomy whether online

or offline. The system tracks metadata on the changes to published objects at publishers and subscribers in each replicated database, and the replication agent merges those data modifications together during synchronization between replicated pairs and ensures data convergence through automatic conflict detection and resolution. Numerous conflict resolution policy options are built into the replication agent used in the synchronization process, and custom conflict resolution can be written by using stored procedures or by using an extensible **component object model (COM)** interface. This type of replication does not replicate all intermediate states but only the current state of the data at the time of synchronization. It is suitable when replicas require the ability to make autonomous updates while not connected to any network.

14.4.9 Server Programming in .NET

SQL Server supports the hosting of the .NET Common Language Runtime (CLR) inside the SQL Server process to enable database programmers to write business logic as functions, stored procedures, triggers, data types, and aggregates. The ability to run application code inside the database adds flexibility to the design of application architectures that require business logic to execute close to the data and cannot afford the cost of shipping data to a middle-tier process to perform computation outside the database. The .NET Common Language Runtime (CLR) is a runtime environment with a strongly typed intermediate language that executes multiple modern programming languages such as C#, Visual Basic, C++, COBOL, and J++, among others, and has garbage-collected memory, preemptive threading, metadata services (type reflection), code verifiability, and code access security. The runtime uses metadata to locate and load classes, lay out instances in memory, resolve method invocations, generate native code, enforce security, and set runtime context boundaries.

Application code is deployed inside the database by using **assemblies**, which are the units of packaging, deployment, and versioning of application code in .NET. Deployment of application code inside the database provides a uniform way to administer, back up, and restore complete database applications (code and data). Once an assembly is registered inside the database, users can expose entry points within the assembly via SQL DDL statements, which can act as scalar or table functions, procedures, triggers, types, and aggregates, by using well-defined extensibility contracts enforced during the execution of these DDL statements. Stored procedures, triggers, and functions usually need to execute SQL queries and updates. This is achieved through a component that implements the ADO.NET data-access API for use inside the database process.

Basic .NET Concepts

In the .NET framework, a programmer writes program code in a high-level programming language that implements a class defining its structure (e.g., the fields or properties of the class) and methods. Some of these methods can be static functions. The compilation of the program produces a file, called an *assembly*, containing the compiled code in the **Microsoft Intermediate Language (MSIL)**, and a *manifest* containing all references to dependent assemblies. The manifest is an integral part of every assembly that renders the assembly self-describing. The assembly manifest contains the assembly's metadata, which describes all structures, fields, properties, classes, inheritance relationships, functions, and methods defined in the program. The manifest establishes the assembly identity, specifies the files that make up the assembly implementation, specifies the types and resources that make up the assembly, itemizes the compile-time dependencies on other assemblies, and specifies the set of permissions required for the assembly to run properly. This information is used at runtime to resolve references, enforce version-binding policy, and validate the integrity of loaded assemblies.

The .NET framework supports an out-of-band mechanism called *custom attributes* for annotating classes, properties, functions and methods with additional information or facets the application may want to capture in metadata. All .NET compilers consume these annotations without interpretation and store them in the assembly's metadata. All these annotations can be examined in the same way as any other metadata by using a common set of reflection APIs. **Managed code** refers to MSIL executed in the CLR rather than directly by the operating system. Managed-code applications gain common-language runtime services such as automatic garbage collection, runtime type checking, and security support. These services help provide uniform platform- and language-independent behavior of managed-code applications. At execution time, a **just-in-time (JIT)** compiler translates the MSIL into native code (e.g., Intel X86 code). During this translation, code must pass a verification process that examines the MSIL and metadata to find out whether the code can be determined to be type safe. of each of these server components.

SQL Server Integration Services

Microsoft SQL Server Integration Services (SSIS) is an enterprise data transformation and data integration solution that you can use to extract, transform, aggregate, and

consolidate data from disparate sources and move it to single or multiple destinations. You can use SSIS to perform the following tasks:

- Merge data from heterogeneous data stores.
- Refresh data in data warehouses and data marts.
- Cleanse data before loading it into destinations.
- Bulk-load data into online transaction processing (OLTP) and online analytical processing (OLAP) databases.
- Send notifications.
- Build business intelligence into a data transformation process.
- Automate administrative functions.

SSIS provides a complete set of services, graphical tools, programmable objects, and APIs for the above tasks. These provide the ability to build large, robust, and complex data transformation solutions without any custom programming.

However, an API and programmable objects are available when they are needed to create custom elements or integrate data transformation capabilities into custom applications.

The SSIS data-flow engine provides the in-memory buffers that move data from source to destination and calls the source adapters that extract data from files and relational databases. The engine also provides the transformations that modify data and the destination adapters that load data into data stores. Duplicate elimination based on fuzzy (approximate) match is an example of a transformation provided by SSIS. Users can program their own transformations if required. Figure 14.6 shows an example of how various transformations can be combined to cleanse and load book sales information; the book titles from the sales data are matched against a publications database, and in case there is no match, fuzzy lookup is performed to handle titles with minor errors (such as spelling errors). Information about confidence and data lineage is stored with the cleansed data.

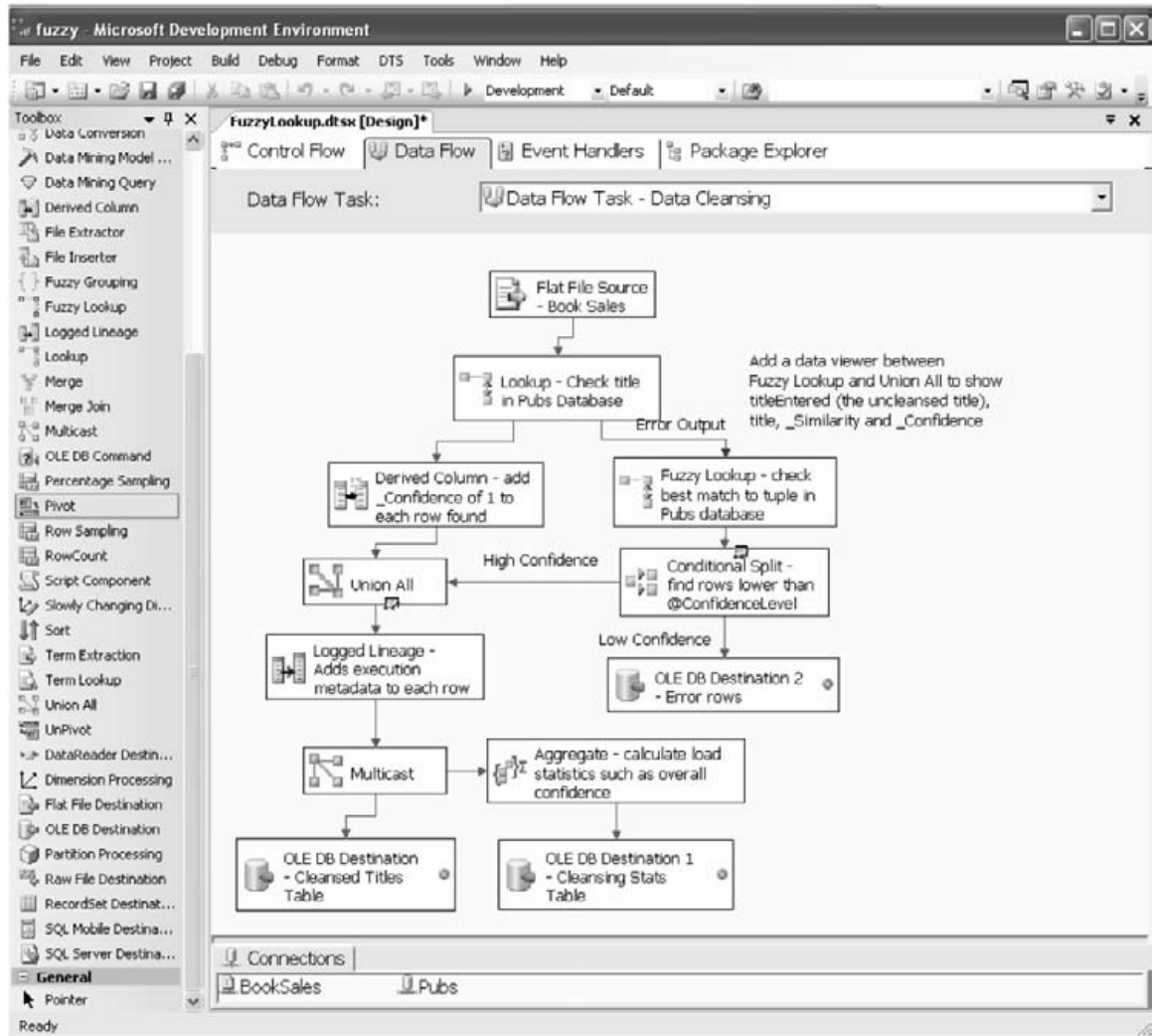


Figure 14.6 Loading of data by using fuzzy lookup.

SQL Server Analysis Services

The Analysis Services component delivers online analytical processing (OLAP) and data-mining functionality for business intelligence applications. Analysis Services supports a thin client architecture. The calculation engine is on the server, so queries are resolved on the server, avoiding the need to transfer large amounts of data between the client and the server.

SQL Server Analysis Services: OLAP

Analysis Services utilizes a Unified Dimensional Model (UDM), which bridges the gap between traditional relational reporting and OLAP ad hoc analysis. The role of a Unified Dimensional Model (UDM) is to provide a bridge between the user and the data sources. A UDM is constructed over one or more physical data sources, and then the end user issues

queries against the UDM, using one of a variety of client tools, such as Microsoft Excel. More than simply a dimension modeling layer of the Data Source schemas, the UDM provides a rich environment for defining powerful yet exhaustive business logic, rules, and semantic definition. Users can browse and generate reports on the UDM data in their native language (for example, French or Hindi) by defining local language translation of the metadata catalog as well as the dimensional data. Analysis Server defines complex time dimensions (fiscal, reporting, manufacturing, etc.), and enables the definition of powerful multidimensional business logic (year-to-year growth, year-to-date) using the multidimensional expression (MDX) language. The UDM allows users to define business-oriented perspectives, each one presenting only a specific subset of the model (measures, dimensions, attributes, business rules, and so forth) that is relevant to a particular group of users. Businesses often define key performance indicators (KPIs) that are important metrics used to measure the health of the business. Examples of such KPIs include sales, revenue per employee, and customer retention rate. The UDM allows such KPIs to be defined, enabling a much more understandable grouping and presentation of data.

SQL Server Analysis Services: Data Mining

SQL Server provides a variety of mining techniques, with a rich graphical interface to view mining results. Mining algorithms supported include:

- Association rules (useful for cross-sales applications).
- Classification and prediction techniques such as decision trees, regression trees, neural networks, and naive Bayes.
- Time series forecasting techniques including ARIMA and ARTXP.
- Clustering techniques such as expectation maximization and K-means (coupled with techniques for sequence clustering).

In addition, SQL Server provides an extensible architecture for plugging in third-party data mining algorithms and visualizers.

SQL Server also supports the *Data-Mining Extensions* (DMX) extensions for SQL. DMX is the language used to interact with data-mining models just as SQL is used to interact with tables and views. With DMX, models can be created and trained and then stored in an Analysis Services database. The model can then be browsed to look at patterns or, by using a special **prediction join** syntax, applied against new data to perform

predictions. The DMX language supports functions and constructs to easily determine a predicted class along with its confidence, predict a list of associated items as in a recommendation engine, or even return information and supporting facts about a prediction. Data mining in SQL Server can be used against data stored in relational or multidimensional data sources. Other data sources are supported as well through specialized tasks and transforms, allowing data mining directly in the operational data pipeline of Integration Services. Data-mining results can be exposed in graphical controls, special data-mining dimensions for OLAP cubes, or simply in Reporting Services reports.

SQL Server Reporting Services

Reporting Services is a server-based reporting platform that can be used to create and manage tabular, matrix, graphical, and free-form reports that contain data from relational and multidimensional data sources. The reports that you create can be viewed and managed over a Web-based connection. Matrix reports can summarize data for high-level reviews, while providing supporting detail in drilldown reports. Parameterized reports can be used to filter data on the basis of values that are provided at runtime. Users can choose from a variety of viewing formats to render reports on the fly in preferred formats for data manipulation or printing. An API is also available to extend or integrate report capabilities into custom solutions. Server-based reporting provides a way to centralize report storage and management, set policies and secure access to reports and folders, control how reports are processed and distributed, and standardize how reports are used in your business.

14.5 Summary

A distributed database system consists of a collection of sites, each of which maintains a local database system. Each site is able to process local transactions: those transactions that access data in only that single site. In addition, a site may participate in the execution of global transactions: those transactions that access data in several sites. The execution of global transactions requires communication among the sites.

- Distributed databases may be homogeneous, where all sites have a common schema and database system code, or heterogeneous, where the schemas and system codes may differ.
- There are several issues involved in storing a relation in the distributed database, including replication and fragmentation. It is essential that the system minimize the degree to which a user needs to be aware of how a relation is stored.

- A distributed system may suffer from the same types of failure that can afflict a centralized system. There are, however, additional failures with which we need to deal in a distributed environment, including the failure of a site, the failure of a link, loss of a message, and network partition. Each of these problems needs to be considered in the design of a distributed recovery scheme.

14.6 Lesson and Activities

How might a distributed database designed for a local-area network differ from one designed for a wide-area network?

Consider a distributed system with two sites, *A* and *B*. Can site *A* distinguish among the following?

- *B* goes down.
- The link between *A* and *B* goes down.
- *B* is extremely overloaded and response time is 100 times longer than normal.

What implications does your answer have for recovery in distributed systems?

14.7 Keywords

- Homogeneous distributed database
- Heterogeneous distributed database
- Data replication
- Primary copy
- Data fragmentation
 - * Horizontal fragmentation
 - * Vertical fragmentation
- Data transparency
 - * Fragmentation transparency
 - * Replication transparency
 - * Location transparency
- Name server
- Aliases

- Distributed transactions
 - * Local transactions
 - * Global transactions
- Transaction manager
- Transaction coordinator
- System failure modes
- Network partition
- Commit protocols
- Two-phase commit protocol (2PC)
 - * Ready state
 - * In-doubt transactions
 - * Blocking problem
- Three-phase commit protocol (3PC)
- Persistent messaging
- Concurrency control
- Single lock manager
- Distributed lock manager
- Protocols for replicas
 - * Primary copy
 - * Majority protocol
 - * Biased protocol
 - * Quorum consensus protocol
- Timestamping
- Master–slave replication
- Multimaster (update-anywhere) replication
- Transaction-consistent snapshot
- Lazy propagation
- Deadlock handling

- * Local wait-for graph
- * Global wait-for graph
- * False cycles
- Availability
- Robustness
 - * Majority-based approach
 - * Read one, write all
 - * Read one, write all available
 - * Site reintegration
- Coordinator selection

14.8 Questions for Discussion

1. Discuss the relative advantages of centralized and distributed databases.
2. Describe how LDAP can be used to provide multiple hierarchical views of data, without replicating the base-level data.
3. Explain homogeneous and heterogeneous databases.
4. Explain the directory systems.
5. case study for Oracle and Ms-SQL server.

14.9 Suggested Reading

“Database System Concepts” -5th Edition by Silberchatz. Korth. Sudarshan., TMH.

MODEL QUESTION PAPER
MCA
CORE PAPER - III
DATABASE MANAGEMENT SYSTEMS

Time: Three hours

Maximum: 80 marks

Section – A

(10 x 2 = 20Marks)

Answer any TEN Questions.

Each question carries 2 Marks.

1. Differentiate between Static and Dynamic SQL
2. Write the relational algebra statement to fetch data from a relation.
3. Define Integrity of Data
4. State BCNF with an example
5. What is Object-Relational Database Systems?
6. What are the Physical Storage Media?
7. Write the properties of types of resource locks
8. Name any four types of joins. Give an example for any one.
9. What are the advantages and disadvantages of Distributed Database Management System?
10. What is Heterogeneous Database?
11. Define: Entity Set.
12. What is XML Schema?

Section – B**(5 x 6 = 30 Marks)****Answer any FIVE Questions.****Each question carries 6 Marks.**

13. Describe Data Base Systems Structure and Discuss its components
14. Explain about DDL and DML
15. Explain the following
 - i) Views in SQL
 - ii) Triggers in SQL
16. Write short notes about Storage and File Structure
17. Describe: i) Hashing Techniques ii) Serializability of Transactions
18. Give an account on Query Processing Stages and explain Query Optimization
19. Discuss about Client – Server Architecture

Section – C**(3 x 10 = 30 Marks)****Answer any THREE Questions****Each question carries 10 Marks.**

20. Define DBMS. Discuss the advantages of Database management system and the types of Database end users.
 21. What is Normalization? Briefly discuss Normal Forms with example
 22. Discuss in detail about DBMS file Structure and how File Organization is done?
 23. Give in detail about Concurrency Transactions Management
 24. Explain about Database System Architecture.
-